



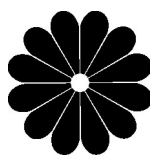
مجموعه مهندسی کامپیوتر — ☒ دکتری — ☒ ارشد

# طراحی الگوریتم

هادی یوسفی



به نام خداوند جان و خرد



پوران پژوهش

کتاب ارشد

مجموعه مهندسی کامپیوتر

## طراحی الگوریتم

چاپ چهاردهم

مؤلف:

هادی یوسفی

پاییز ۱۳۹۹

|                     |                             |
|---------------------|-----------------------------|
| سرشناسه             | : یوسفی، هادی، ۱۳۵۴-        |
| عنوان و نام پدیدآور | : طراحی الگوریتم            |
| مشخصات نشر          | : تهران: پوران پژوهش، ۱۳۹۴. |
| مشخصات ظاهری        | : ۵۳۲ ص.                    |
| فروست               | : کتاب ارشد.                |
| شابک                | : ۹۷۸-۹۶۴-۱۸۴-۴۶۱-۷         |
| وضعیت فهرست نویسی   | : فنیای مختصری              |
| یادداشت             | : چاپ دهم.                  |
| شماره کتابشناسی ملی | : ۳۷۹۵۱۷۹                   |

| انتشارات پوران پژوهش                                                                                 |                      |
|------------------------------------------------------------------------------------------------------|----------------------|
| نام کتاب:                                                                                            | طراحی الگوریتم       |
| تألیف:                                                                                               | هادی یوسفی           |
| ناشر:                                                                                                | پوران پژوهش          |
| حروفچینی:                                                                                            | پوران پژوهش          |
| چاپ و صحافی:                                                                                         | هدف - ولیعصر         |
| شمارگان:                                                                                             | ۱۰۰۰ نسخه            |
| نوبت چاپ:                                                                                            | چهاردهم - پاییز ۱۳۹۹ |
| شابک:                                                                                                | ۹۷۸-۹۶۴-۱۸۴-۴۶۱-۷    |
| ISBN: 978-964-184-461-7                                                                              |                      |
| دفتر مرکزی: میدان انقلاب - ابتدای کارگر جنوبی - کوچه مهدیزاده - پلاک ۹ - واحد ۵۴      تلفن: ۶۶۹۲۷۰۴۰ |                      |

این اثر، مشمول قانون حمایت مؤلفان و مصنفان و هنرمندان مصوب ۱۳۴۸ است، هر کس تمام یا قسمتی از این اثر را بدون اجازه مؤلف و ناشر، نشر یا پخش یا عرضه کند مورد پیگرد قانونی قرار خواهد گرفت.

# برنام تنها پرستیدنی که قلم را آفرید

## پیش گفتار انتشارات

نگاهی به شمار داوطلبان آزمون کارشناسی ارشد نشان می‌دهد که در این سال‌ها درخواست تحصیل در دوره‌های تحصیلات تکمیلی دانشگاه‌ها افزایش چشمگیری داشته است. دشواری پیش روی بیشتر داوطلبان، گوناگونی منابع درسی و نبود دسترسی به آنها و همچنین نمونه آزمون‌های مناسب برای تمرین و فهم بیش تر مفاهیم درسی است.

مدیریت بنیاد انتشاراتی پوران پژوهش، آقای دکتر احمد هژبر و همسرشان بانو افسانه عبدی با بیش از ۱۵ سال کوشش در راستای برآورده‌سازی نیاز داوطلبان، با سیاست کلی چاپ کتاب‌های ارزنده با بهایی درخور، آماده‌سازی و گردآوری چهار مجموعه‌ی گوناگون با آماجهای روشن را در دستور کار قرار داده است.

مجموعه‌ی نخست با نام کتاب ارشد (با جلد آبی‌رنگ) که تاکنون به دست داوطلبان رسیده، با پذیرش چشمگیری همراه بوده است. در هر عنوان کتاب ارشد، پس از شرح کامل درس در هر فصل، پرسش‌های چهارگزینه‌ای آزمون‌های سراسری و آزاد چند سال گذشته با پاسخ‌های تشریحی آورده شده است. شرح درس در هر کتاب از این مجموعه به گونه‌ای است که برای دانشجویان سال‌های پایین‌تر سودمند است و نیز یک منبع درسی مناسب برای دانشجویان و استادان دانشگاه‌ها می‌باشد. کتاب ارشد نخستین بار در مهرماه سال ۱۳۸۰ در قالب پانزده عنوان به داوطلبان شناسانده شد و هم‌اینک بیش از ۲۲۰ عنوان را دربرمی‌گیرد.

مجموعه‌ی دوم با نام چند آزمون ارشد (با جلد سیاه رنگ) به گونه‌ای گردآوری شده است که دانشجوی دفترچه‌های آزمون‌های سراسری سال‌های گذشته را با پاسخ‌های تشریحی در یک کتاب خواهد داشت.

مجموعه‌ی سوم با نام بانک تست ارشد (با جلد نارنجی رنگ) در دروس پایه و تخصصی هر رشته، یک کتاب کار به شمار می‌رود که در آن پرسش‌های طبقه‌بندی‌شده به همراه پاسخ‌های تشریحی آورده شده است تا دانشجو با حل و بررسی آن‌ها توانایی بایسته برای پاسخ‌گویی به آزمون‌ها را به دست آورد.

در آخر مجموعه‌ی چهارم که با جلد قهوه‌ای رنگ عرضه شده و به عنوان کتاب‌های مرجع در دانشگاه‌ها آموخته می‌شود.



## به نام خدا

### مقدمه مؤلف

کتاب حاضر پس از چندین بار ویرایش جامع‌ترین کتاب طراحی الگوریتم برای آمادگی کنکور کارشناسی ارشد و دکتری رشته‌های علوم کامپیوتر، مهندسی فناوری اطلاعات و مهندسی کامپیوتر است. این کتاب براساس جدیدترین تغییرات منابع و تست‌های کنکور کارشناسی ارشد و دکتری، تألیف شده است. این کتاب در یازده فصل تنظیم شده است و برخی فصول آن مثل فصل‌های ۱ تا ۷ مطالب بسیار مفید و ارزنده‌ای را شامل می‌باشند و توصیه می‌شود با دقت و وسواس زیاد مطالعه شوند. در پایان برخی از فصل‌ها تمریناتی گنجانده شده است که پاسخ برخی از آنها آمده است، توصیه می‌شود تمرینات بی‌پاسخ را حل کنید و یا حداقل صورت این سؤالات را به خاطر بسپارید. البته پاسخ تمامی این تمرینات به همراه تست‌های بسیار مفید در کتاب نازنجی طراحی الگوریتم آمده است که توصیه می‌کنم برای تسلط به مطالب حتماً کتاب‌های نازنجی را مطالعه بفرمایید. و همچنین توصیه می‌شود که کتاب «ساختمان داده‌ها» انتشارات پوران پژوهش را مطالعه بفرمایید. این دو کتاب تمام سرفصل‌های دروس ساختمان داده‌ها و طراحی الگوریتم را پوشش می‌دهد و کامل‌ترین منابع برای داوطلبان کنکور کارشناسی ارشد و همچنین برای دانشجویان رشته کامپیوتر هستند.

سعی شده است مطالب بدون اشکال باشند ولی ممکن است اشکالاتی در کتاب مشاهده بفرمایید و یا احساس کنید مطلبی که متوجه نمی‌شوید، نادرست است. اینجانب از شما خواننده گرامی تقاضا می‌کنم اشکالات احتمالی را مطرح بفرمایید تا با یکدیگر تبادل نظر کنیم.

دوستان عزیز در گردآوری این کتاب زحمت فراوان کشیده‌اند. همچنین، زحمات مدیر محترم تولید، آقای حسین رحیمی، و نیز خانم خسروی که با دقتی وصف‌ناپذیر آماده‌سازی کتاب را بر عهده داشتند، جای تقدیر ویژه دارد.

هادی یوسفی – پاییز ۱۳۹۹



@yousefi\_hadi

۰۹۱۲۱۷۸۸۵۳۴

# فهرست مطالب

|                                                                          |     |
|--------------------------------------------------------------------------|-----|
| فصل اول. مقدمات ریاضی، رشد توابع، نمادهای مجانبی .....                   | ۱   |
| خواص سیگما .....                                                         | ۱   |
| یافتن کران برای حاصل جمع .....                                           | ۳   |
| لگاریتم .....                                                            | ۶   |
| رشد توابع (Growth of functions) .....                                    | ۷   |
| تابع $lg^*$ (لگ استار) .....                                             | ۱۱  |
| نمادهای مجانبی asymptotic notations .....                                | ۱۲  |
| تمرین .....                                                              | ۲۰  |
| سؤالهای چهارگزینه‌ای فصل اول .....                                       | ۲۵  |
| فصل دوم. تحلیل الگوریتم‌های غیربازگشتی – آنالیز استهلاکی .....           | ۳۵  |
| تحلیل الگوریتم‌های غیربازگشتی .....                                      | ۳۵  |
| آنالیز استهلاکی (Amortized Analysis) .....                               | ۴۲  |
| تمرین .....                                                              | ۴۸  |
| سؤالهای چهارگزینه‌ای فصل دوم .....                                       | ۴۹  |
| فصل سوم. روابط بازگشتی – تحلیل الگوریتم‌های بازگشتی – تقسیم و غلبه ..... | ۶۵  |
| حل روابط بازگشتی خطی همگن / ناهمگن ضریب ثابت .....                       | ۶۷  |
| قضیه اساسی .....                                                         | ۷۱  |
| درخت بازگشت .....                                                        | ۷۳  |
| قضیه بمب اتم .....                                                       | ۷۵  |
| الگوریتم‌های بازگشتی .....                                               | ۷۶  |
| تقسیم و غلبه (divide & conquer) .....                                    | ۸۰  |
| ضرب ماتریس‌ها .....                                                      | ۸۰  |
| ضرب دو چند جمله‌ای .....                                                 | ۸۲  |
| ضرب اعداد بزرگ .....                                                     | ۸۶  |
| مسائل معروف بازگشتی .....                                                | ۸۷  |
| تمرین .....                                                              | ۹۳  |
| سؤالهای چهارگزینه‌ای فصل سوم .....                                       | ۹۶  |
| فصل چهارم. جستجو و درهم‌سازی .....                                       | ۱۳۲ |
| جستجوی دودویی (binary search) .....                                      | ۱۳۲ |

|                                        |     |
|----------------------------------------|-----|
| جستجوی دودویی در یک دنباله چرخشی       | ۱۳۳ |
| جستجوی دودویی برای یک اندیس خاص        | ۱۳۴ |
| جستجوی دودویی در دنباله با طول نامشخص  | ۱۳۴ |
| جستجوی درونیابی (interpolation search) | ۱۳۴ |
| درهم‌سازی                              | ۱۳۵ |
| سؤال‌های چهارگزینه‌ای فصل چهارم        | ۱۳۹ |

## فصل پنجم. مرتبه‌های آماری و مرتب‌سازی

|                                            |     |
|--------------------------------------------|-----|
| یافتن max و min در $A[1..n]$               | ۱۵۵ |
| یافتن دومین مینیمم (یا دومین ماکزیمم)      | ۱۵۸ |
| یافتن عنصر کمینه $k$ ام ( $k$ امین مینیمم) | ۱۵۹ |
| مرتب‌سازی                                  | ۱۶۴ |
| روش‌های مرتب‌سازی غیرمقایسه‌ای             | ۱۷۱ |
| تمرین                                      | ۱۷۷ |
| سؤال‌های چهارگزینه‌ای فصل پنجم             | ۱۸۶ |

## فصل ششم. مباحثی از درخت‌ها

|                                             |     |
|---------------------------------------------|-----|
| پیمایش درخت دودویی                          | ۲۱۹ |
| Heap                                        | ۲۲۱ |
| صف اولویت priority queue                    | ۲۲۵ |
| هیپ دو جمله‌ای (Binomial heap)              | ۲۲۶ |
| هیپ فیبوناچی                                | ۲۲۸ |
| درخت جستجوی دودویی Binary Search Tree (BST) | ۲۲۹ |
| AVL                                         | ۲۳۴ |
| مجموعه‌های مجزا                             | ۲۳۷ |
| تمرین                                       | ۲۴۰ |
| سؤال‌های چهارگزینه‌ای فصل ششم               | ۲۴۱ |

## فصل هفتم. گراف

|                   |     |
|-------------------|-----|
| گراف              | ۲۷۶ |
| پیمایش گراف       | ۲۷۷ |
| پیمایش عمقی (DFS) | ۲۷۷ |
| پیمایش سطحی (BFS) | ۲۸۶ |

|     |       |                                                           |
|-----|-------|-----------------------------------------------------------|
| ۲۸۹ | ..... | درخت پوشای مینیمم (Minimum Spanning Tree:MST)             |
| ۲۸۹ | ..... | الگوریتم کراسکال (kruskal)                                |
| ۲۹۱ | ..... | الگوریتم پریم (Prim)                                      |
| ۲۹۲ | ..... | کوتاهترین مسیرهای هم مبدا (Single-Source Shortest Paths)  |
| ۲۹۴ | ..... | الگوریتم بلمن فورد                                        |
| ۲۹۵ | ..... | یافتن کوتاهترین مسیرهای هم مبدأ در گراف جهت‌دار بدون سیکل |
| ۲۹۶ | ..... | الگوریتم دایجسترا (Dijkstra)                              |
| ۲۹۸ | ..... | شار (جریان) بیشینه (max flow)                             |
| ۳۰۶ | ..... | تمرین                                                     |
| ۳۱۰ | ..... | سؤال‌های چهارگزینه‌ای فصل هفتم                            |

## فصل هشتم. روش‌های حریصانه (greedy).....

|     |       |                                                                 |
|-----|-------|-----------------------------------------------------------------|
| ۳۵۵ | ..... | مقدمه                                                           |
| ۳۵۶ | ..... | ۱- مساله کوله‌پشتی غیرصفر و یک (کوله پشتی کسری)                 |
| ۳۵۹ | ..... | ۲- کدهافمن                                                      |
| ۳۶۲ | ..... | ۳- زمان‌بندی بر مبنای کمینه کردن زمان کل                        |
| ۳۶۲ | ..... | ۴- انتخاب (زمان‌بندی) فعالیتها                                  |
| ۳۶۳ | ..... | ۵- زمان‌بندی فعالیتها با مهلت معین (Scheduling with Dead Lines) |
| ۳۶۷ | ..... | سؤال‌های چهارگزینه‌ای فصل هشتم                                  |

## فصل نهم. برنامه‌نویسی پویا.....

|     |       |                                                     |
|-----|-------|-----------------------------------------------------|
| ۳۷۹ | ..... | مقدمه                                               |
| ۳۸۲ | ..... | ۱- ضرب زنجیره‌ای ماتریس‌ها                          |
| ۳۸۷ | ..... | ۲- الگوریتم فلویید برای یافتن تمام کوتاهترین مسیرها |
| ۳۹۳ | ..... | ۳- کوله پشتی ۱-۰ با ارزش ماکزیمم                    |
| ۳۹۴ | ..... | ۴- فروشنده دوره‌گرد                                 |
| ۳۵۱ | ..... | ۵- درخت جستجوی دودویی بهینه (Optimal BST)           |
| ۳۹۹ | ..... | ۶- بزرگترین زیردنباله مشترک (LCS)                   |
| ۴۰۱ | ..... | ۷- ضریب دو جمله‌ای                                  |
| ۴۰۲ | ..... | ۸- زمان‌بندی خط تولید (assembly-line scheduling)    |
| ۴۰۴ | ..... | ۹- خرد کردن سکه                                     |
| ۴۰۶ | ..... | ۱۰- برش میله                                        |
| ۴۰۸ | ..... | ۱۱- مسابقات جهانی                                   |
| ۴۱۰ | ..... | سؤال‌های چهارگزینه‌ای فصل نهم                       |

|                                                            |     |
|------------------------------------------------------------|-----|
| فصل دهم. بازگشت به عقب و انشعاب و تحدید (مطالعه آزاد)..... | ۴۲۹ |
| سؤال‌های چهارگزینه‌ای فصل دهم.....                         | ۴۴۱ |
| فصل یازدهم. آشنایی با نظریه NP.....                        | ۴۴۵ |
| سؤال‌های چهارگزینه‌ای فصل یازدهم.....                      | ۴۵۴ |
| کنکورهای سراسری ارشد و دکتری ۹۶-۹۹.....                    | ۴۶۱ |

## مقدمات ریاضی، رشد توابع، نمادهای جانبی

### فصل ۱

#### مقدمات ریاضی

برای تحلیل و بررسی الگوریتم‌ها نیاز به دانستن برخی مفاهیم ریاضی است. مثلاً برای محاسبه تعداد اجرای برخی از حلقه‌ها از سیگما استفاده می‌کنیم.

#### خواص سیگما

حاصل جمع  $a_1 + a_2 + \dots + a_n$  را به صورت  $\sum_{k=1}^n a_k$  می‌نویسیم. و اگر تعداد جملات

نامتناهی باشد یعنی  $a_1 + a_2 + \dots$  را به صورت  $\sum_{k=1}^{\infty} a_k$  یا  $\lim_{n \rightarrow \infty} \sum_{k=1}^n a_k$  می‌نویسیم. به خاطر

سپاری روابط زیر در تحلیل الگوریتم‌ها مفید است:

$$۱) \sum_{k=1}^n k = 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2} \quad (\text{تصاعد حسابی با قدر نسبت ۱})$$

$$۲) \sum_{k=1}^n k^2 = 1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

$$۳) \sum_{k=1}^n k^3 = 1^3 + 2^3 + 3^3 + \dots + n^3 = \left( \frac{n(n+1)}{2} \right)^2 = \frac{n^2(n+1)^2}{4}$$

$$۴) \sum_{k=0}^n ax^k = a + ax + ax^2 + \dots + ax^n = a(1 + x + x^2 + \dots + x^n)$$

$$= \frac{a(x^{n+1} - 1)}{x - 1} = \frac{a(1 - x^{n+1})}{1 - x} \quad (\text{تصاعد هندسی با قدر نسبت } x)$$

در تصاعد هندسی اگر  $|x| < 1$  و  $n \rightarrow \infty$  آنگاه  $x^{n+1} \rightarrow 0$  و داریم:

$$\sum_{k=0}^{\infty} ax^k = a + ax + ax^2 + \dots = \frac{a}{1-x}$$

**توجه:** برای اثبات تساوی‌های فوق می‌توان از استقرای ریاضی کمک گرفت. یک مدل از استقرای ریاضی چنین است که فرض کنید  $P(n)$  حکمی است راجع به عدد صحیح  $n$ . حال اگر  $P(1)$  درست باشد و از درستی  $P(k)$  درستی  $P(k+1)$  نتیجه شود آنگاه  $P(n)$  به ازای همه اعداد صحیح  $n \geq 1$  درست است. پس برای اثبات حکم  $P(n)$  با استقراء باید سه مرحله را طی کنیم:

۱- نشان دهیم  $P(1)$  درست است.

۲- فرض کنیم  $P(k)$  درست است.

۳- ثابت کنیم  $P(k+1)$  درست است.

**مثال ۱:** درستی تساوی ۳ یعنی  $\sum_{k=1}^n k^2 = \frac{n^2(n+1)^2}{4}$  را ثابت کنید.

**پاسخ:** فرض کنید  $P(n)$  عبارت است از  $1^2 + 2^2 + \dots + n^2 = \frac{n^2(n+1)^2}{4}$ . حال با استقراء اثبات می‌کنیم.

**مرحله ۱:**  $P(1)$  یعنی  $1^2 = \frac{1^2(1+1)^2}{4}$  درست است.

**مرحله ۲:** فرض می‌کنیم  $P(k)$  یعنی  $1^2 + 2^2 + \dots + k^2 = \frac{k^2(k+1)^2}{4}$  درست است.

**مرحله ۳:** ثابت می‌کنیم  $P(k+1)$  یعنی  $1^2 + 2^2 + \dots + k^2 + (k+1)^2 = \frac{(k+1)^2(k+2)^2}{4}$

درست است. که برای اثبات از مرحله ۲ کمک می‌گیریم و بجای  $1^2 + 2^2 + \dots + k^2$  مقدار

$$\frac{k^2(k+1)^2}{4}$$

قرار می‌دهیم پس داریم:

$$\begin{aligned} 1^2 + 2^2 + \dots + k^2 + (k+1)^2 &= \frac{k^2(k+1)^2}{4} + (k+1)^2 = \frac{(k+1)^2(k^2 + 4k + 4)}{4} \\ &= \frac{(k+1)^2(k+2)^2}{4} \quad \blacksquare \end{aligned}$$

**توجه:** می‌توان از سری هندسی (روابط ۴ و ۵) مشتق یا انتگرال گرفت و فرمول‌های جدیدی یافت.

**مثال ۲:** حاصل  $\sum_{k=1}^{\infty} \frac{k}{2^k}$  را بیابید.

**پاسخ:** با توجه به فرمول  $1 + x + x^2 + x^3 + \dots = \frac{1}{1-x}$  داریم  $1 + x + x^2 + x^3 + \dots = \frac{1}{1-x}$  مشتق می‌گیریم و داریم:  $1 + 2x + 3x^2 + \dots = \frac{1}{(1-x)^2}$  حال طرفین را در  $x$  ضرب می‌کنیم و داریم:

$$x + 2x^2 + 3x^3 + \dots = \frac{x}{(1-x)^2}$$

$$\frac{1}{2} + 2 \times \frac{1}{2^2} + 3 \times \frac{1}{2^3} + \dots = \sum_{k=1}^{\infty} k \times \frac{1}{2^k} = \frac{\frac{1}{2}}{\left(1 - \frac{1}{2}\right)^2} = 2 \quad \blacksquare$$

**توجه:** در این درس بعضی مواقع نیاز به محاسبه حد توابع هست. می‌دانیم چند جمله‌ای  $a_0 + a_1x + a_2x^2 + \dots + a_nx^n$  وقتی  $n \rightarrow \infty$  با  $a_nx^n$  هم ارز است.

**مثال ۳:** حد  $\frac{1+2+3+\dots+n}{3n^2+2n-5}$  وقتی  $n \rightarrow \infty$  را بیابید.

**پاسخ:**  $1+2+3+\dots+n = \frac{n(n+1)}{2}$  است که با  $\frac{1}{2}n^2$  هم ارز است و  $3n^2+2n-5$  با  $3n^2$  هم ارز است پس این حد برابر حد  $\frac{\frac{1}{2}n^2}{3n^2}$  است که برابر  $\frac{1}{6}$  می‌باشد.  $\blacksquare$

### یافتن کران برای حاصل جمع

تکنیک‌های مختلفی وجود دارد که برای یک حاصل جمع می‌تواند کران بالا و پایین مشخص کند، یک تکنیک مهم انتگرال است. ابتدا تعریف تابع صعودی و نزولی را یادآوری می‌کنیم. تابع  $y = f(x)$  را صعودی گویند هرگاه اگر  $i < j$  نتیجه شود  $f(i) \leq f(j)$ . مثلاً تابع  $y = f(x) = 2x + 1$  صعودی است. تابع  $y = f(x) = \frac{1}{x}$  نزولی است. مثلاً تابع  $f(i) \geq f(j)$ .

برای حاصل جمع  $\sum_{k=m}^n f(k)$  اگر  $f(k)$  صعودی باشد می‌توان با انتگرال برای این حاصل جمع کران مشخص کرد:

$$\int_{m-1}^n f(x) dx \leq \sum_{k=m}^n f(k) \leq \int_m^{n+1} f(x) dx$$

و اگر  $f(k)$  نزولی باشد داریم:

$$\int_m^{n+1} f(x) dx \leq \sum_{k=m}^n f(k) \leq \int_{m-1}^n f(x) dx$$



**مثال ۴:** سری همسازه (هارمونیک)  $\sum_{k=1}^n \frac{1}{k} = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$  است. کران بالا و پایین این سری را بیابید.

**پاسخ:** تابع  $f(k) = \frac{1}{k}$  نزولی است پس کران پایین به صورت زیر بدست می آید:

$$\sum_{k=1}^n \frac{1}{k} \geq \int_1^{n+1} \frac{dx}{x} = \ln(n+1)$$

برای کران بالا داریم:  $\sum_{k=2}^n \frac{1}{k}$  را از  $k=2$  شروع می کنیم که برای انتگرال مشکل ایجاد نشود

$$\sum_{k=2}^n \frac{1}{k} \leq \int_1^n \frac{dx}{x} = \ln n \Rightarrow \sum_{k=1}^n \frac{1}{k} \leq \ln n + 1$$

بنابراین سری همسازه بین  $\ln(n+1)$  و  $\ln n + 1$  است. پس می توان نتیجه گرفت

$$\sum_{k=1}^n \frac{1}{k} = \ln n + c \text{ که } c \text{ ثابت است.}$$

**توجه:** اگر در دنباله  $\sum_{k=0}^n a_k$  و برای هر  $k \geq 0$  داشته باشیم  $\frac{a_{k+1}}{a_k} \leq r$  که  $r$  ثابت بین ۰ و ۱ است آنگاه می توان برای این دنباله با کمک سری هندسی کران بالا یافت:

$$\frac{a_{k+1}}{a_k} \leq r \rightarrow a_{k+1} \leq a_k r \rightarrow a_k \leq a_{k-1} r \leq a_{k-2} r^2 \leq \dots \leq a_0 r^k$$

$$\sum_{k=0}^n a_k \leq \sum_{k=0}^{\infty} a_0 r^k = \frac{a_0}{1-r} \quad \text{در نتیجه:}$$

**مثال ۵:** برای  $\sum_{k=1}^{\infty} \frac{k}{3^k}$  کران بالا بیابید.

**پاسخ:** ابتدا سیگما را طوری می نویسیم که از  $k=0$  شروع شود:

$$\left( \sum_{k=m}^n f(k) = \sum_{k=m-P}^{n-P} f(k+P) \right)$$

$$\sum_{k=1}^{\infty} \frac{k}{3^k} = \sum_{k=0}^{\infty} \frac{k+1}{3^{k+1}}$$

جمله اول  $a_0 = \frac{1}{3}$  است و می توان ثابت  $r$  را برابر  $\frac{2}{3}$  انتخاب کرد زیرا:

$$\frac{a_{k+1}}{a_k} = \frac{(k+2)/3^{k+2}}{(k+1)/3^{k+1}} = \frac{1}{3} \frac{k+2}{k+1} \leq \frac{2}{3} \rightarrow r = \frac{2}{3}$$

بنابراین با توجه به  $\sum_{k=0}^{\infty} a_k \leq \frac{a_0}{1-r}$  داریم:

$$\sum_{k=1}^{\infty} \frac{k}{3^k} = \sum_{k=0}^{\infty} \frac{k+1}{3^{k+1}} \leq \frac{\frac{1}{3}}{1-\frac{1}{3}} = 1$$

**توجه:** مراقب استفاده نادرست از این تکنیک باشید! مثلاً برای سری همسازه  $\sum_{k=1}^{\infty} \frac{1}{k}$  نسبت

از یک کوچکتر است ولی نمی‌توان مقدار  $0 < r < 1$  را یافت. زیرا  $\frac{k}{k+1}$  به ۱ میل می‌کند و هر مقدار برای  $r$  در نظر بگیرید به ازای بعضی مقادیر  $k$  نادرست است. دقت کنید  $r$  باید ثابت باشد. در واقع  $\frac{k}{k+1} < r < 1$  به ازای هیچ  $0 < r < 1$  درست نیست. در مورد سری همسازه اگر این اشتباه را مرتکب شوید و از این تکنیک استفاده کنید، سری همسازه را به یک عدد ثابت محدود می‌کنید ولی می‌دانیم سری همسازه با  $Lnn$  معادل است و به  $\infty$  میل می‌کند.

**تست ۱:** در یک زمستان سرد، یک خرس قطبی،  $n$  قطعه گوشت دقیقاً به اندازه‌های ۱ و ۲ تا  $n$  را در غاری ذخیره کرده است. او هر روز یکی از این قطعه گوشت‌ها را به صورت تصادفی انتخاب می‌کند. اگر اندازه گوشت عدد فرد باشد، آن را کاملاً می‌خورد. اگر زوج باشد، نصف گوشت را خورده و نصف دیگر را مجدداً در غار قرار می‌دهد. اگر گوشتی موجود نباشد، خرس می‌میرد. برای  $n$ ‌های بزرگ، خرس حدوداً چند روز زنده است؟

$$(1) \quad n \cdot \lg n \quad (2) \quad 2^n \quad (3) \quad 2n \quad (4) \quad n^2$$

**پاسخ:**  $n$  قطعه گوشت یعنی حداقل  $n$  روز خرس زنده است. این  $n$  قطعه، نصف‌شان زوج و نصف فرد است. فردها خورده می‌شوند ولی زوج‌ها نصف می‌شوند. پس خرس  $\frac{n}{2}$  روز دیگر زنده می‌ماند. این  $\frac{n}{2}$  قطعه گوشت نصف‌شان زوج است پس خرس  $\frac{n}{4}$  روز دیگر زنده می‌ماند و به همین ترتیب تا اینکه تعداد قطعات گوشت  $\frac{n}{2^k} = 1$  و سپس صفر شود و خرس بمیرد. پس تعداد روزهای زنده ماندن خرس برابر است با:

$$n + \frac{n}{2} + \frac{n}{4} + \dots + \frac{n}{2^k} \leq n + \frac{n}{2} + \frac{n}{4} + \dots = n(1 + \frac{1}{2} + \frac{1}{4} + \dots) = n \times \frac{1}{1-\frac{1}{2}} = 2n$$

پس گزینه (۳) صحیح است.

## لگاریتم

لگاریتم معکوس تابع توان است. مثلاً  $f(x) = 2^x$  آنگاه  $f^{-1}(x) = \log_2 x$ . به عبارت ساده‌تر، اگر  $a^b = c$  آنگاه  $\log_a c = b$  و بر عکس. مثلاً  $\log_2^8 = 3$  زیرا  $2^3 = 8$ . یا  $\log_{10}^{1000} = 3$  زیرا  $10^3 = 1000$ . در این درس معمولاً لگاریتم‌ها در پایه ۲ هستند و بنابراین پایه ۲ را نمی‌نویسیم و به طور خاص برای لگاریتم پایه ۲ از علامت  $\lg$  استفاده می‌کنیم پس قرار داد می‌کنیم  $\log_2^n = \lg n = \lg n$ . فرض کنید می‌خواهیم  $\lceil \lg 1000 \rceil$  را بیابیم چون  $2^9 < 1000 < 2^{10}$  پس  $\lg 1000$  بین ۹ و ۱۰ است و سقف آن برابر ۱۰ می‌شود.

علامت  $\lceil \cdot \rceil$  سقف (ceiling) و علامت  $\lfloor \cdot \rfloor$  کف (floor) است.  $\lceil x \rceil$  برابر کوچکترین عدد صحیح بزرگتر یا مساوی  $x$  است و  $\lfloor x \rfloor$  برابر بزرگترین عدد صحیح کوچکتر یا مساوی  $x$  است. مثلاً  $\lceil 2.5 \rceil = 3$  و  $\lfloor 2.5 \rfloor = 2$  یا  $\lceil 3 \rceil = \lfloor 3 \rfloor = 3$ .

برخی خواص لگاریتم عبارتند از:

$$\begin{array}{ll} ۱) \log_c^{ab} = \log_c^a + \log_c^b & ۲) \log_c^{\frac{a}{b}} = \log_c^a - \log_c^b \\ ۳) \log_{b^n}^a = \frac{1}{n} \log_b^a & ۴) \log_b^a = \frac{\log_c^a}{\log_c^b} \\ ۵) \log_b^a = \frac{1}{\log_a^b} & ۶) b^{\log_b^a} = a \\ ۷) a^{\log_c^b} = b^{\log_c^a} & ۸) \log_b^a \times \log_c^b = \log_c^a \end{array}$$

**مثال ۶:** تساوی‌های زیر را نشان دهید:

$$\begin{array}{lll} a) n^{\frac{1}{\lg n}} = 2 & b) (\lg n)^{\lg n} = n^{\lg \lg n} & c) 4^{\lg n} = n^2 \\ d) 2^{\lg n} = n & e) (\sqrt{2})^{\lg n} = \sqrt{n} & f) 2^{\sqrt{2} \lg n} = n^{\sqrt{\frac{2}{\lg n}}} \end{array}$$

**پاسخ:** شماره خواصی که استفاده می‌کنیم را ذکر می‌کنیم:

**a)** طبق خاصیت ۵،  $\frac{1}{\lg n} = \log_n^2$  و طبق خاصیت ۶،  $n^{\log_n^2} = 2$ .

**b)** طبق خاصیت ۷، در  $n^{\lg \lg n}$  می‌توان جای  $n$  و  $\lg n$  را عوض کرد پس  $n^{\lg \lg n} = (\lg n)^{\lg n}$ .

**c)** طبق خاصیت ۷، می‌توان ۴ را با  $n$  تعویض کرد پس  $4^{\lg n} = n^{\lg 4} = n^2$ .

**d)** طبق خاصیت ۷، می‌توان ۲ را با  $n$  تعویض کرد پس  $2^{\lg n} = n^{\lg 2} = n$ .

$$\sqrt{2}^{\lg n} = (2^{\frac{1}{2}})^{\lg n} = 2^{\frac{1}{2} \lg n} = 2^{\lg n^{\frac{1}{2}}} = n^{\frac{1}{2}} = \sqrt{n} \quad (e)$$

(f) طبق خاصیت ۶ می‌توان گفت  $x = n^{\log_n x}$  پس:

$$\begin{aligned} 2^{\sqrt{2}^{\lg n}} &= n^{\log_n 2^{\sqrt{2}^{\lg n}}} = n^{\sqrt{2}^{\lg n} \times \log_n 2} = n^{\sqrt{2}^{\lg n} \times \frac{1}{\lg 2}} \\ &= n^{\sqrt{2}^{\lg n} \times \frac{1}{\lg 2}} = n^{\sqrt{\frac{2}{\lg 2}}} \end{aligned}$$

### رشد توابع (Growth of functions)

می‌خواهیم بررسی کنیم وقتی  $n$  را خیلی بزرگ کنیم، زمان اجرای الگوریتم‌ها بر حسب  $n$  (اندازه ورودی الگوریتم است) چگونه زیاد می‌شود. می‌خواهیم ابزاری ارائه دهیم که بتوان رشد توابع را با هم مقایسه کرد و بر اساس آن تشخیص دهیم که مثلاً زمان اجرای الگوریتمی کم یا زیاد است.

**مثال ۷:** توابع  $\lg n$  و  $2^n$  و  $n \cdot \lg n$  و  $n^2$  و  $n^3$  را به ترتیب افزایش رشد مرتب کنید.

**پاسخ:** در جدول مقابل مقدار این توابع به ازای برخی  $n$ ها آمده است.

ترتیب رشد این توابع به صورت  $2^n < n^3 < n^2 < n \lg n < n < \lg n$  است. توجه کنید تابع  $n^3$  تا  $n = 8$  از تابع  $2^n$  مقدار بیشتری دارد ولی رشد  $2^n$  بیشتر است. برای تشخیص رشد خیلی مواقع نمی‌توان عددگذاری انجام داد. چون از یک نقطه به بعد ممکن است رفتار یک تابع مشخص شود.

| $n$<br>تابع | ۱ | ۲ | ۴  | ۸   | ۱۶    | ۳۲         |
|-------------|---|---|----|-----|-------|------------|
| $\lg n$     | ۰ | ۱ | ۲  | ۳   | ۴     | ۵          |
| $n$         | ۱ | ۲ | ۴  | ۸   | ۱۶    | ۳۲         |
| $n \lg n$   | ۰ | ۲ | ۸  | ۲۴  | ۶۴    | ۱۶۰        |
| $n^2$       | ۱ | ۴ | ۱۶ | ۶۴  | ۲۵۶   | ۱۰۲۴       |
| $n^3$       | ۱ | ۸ | ۶۴ | ۵۱۲ | ۴۰۹۶  | ۳۲۷۶۸      |
| $2^n$       | ۲ | ۴ | ۱۶ | ۲۵۶ | ۶۵۵۳۶ | ۴۲۹۴۹۶۷۲۹۶ |

**مثال ۸:** رشد دو تابع  $n^2$  و  $2n^2$  را با هم مقایسه کنید.

**پاسخ:** به ازای هر مقدار  $n$ ، مقدار تابع  $2n^2$  از  $n^2$  بیشتر است (دو برابر است) ولی به این معنی نیست که رشد  $2n^2$  از  $n^2$  بیشتر است. این دو تابع هم رشد هستند یعنی ضریب ثابت در رشد تاثیری ندارد. ■

یک روش برای مقایسه رشد دو تابع  $f(n)$  و  $g(n)$ ، استفاده از حدگیری است. حد نسبت وقتی  $\frac{f(n)}{g(n)} \rightarrow +\infty$  اگر برابر صفر شد یعنی رشد  $g(n)$  بیشتر است. اگر  $\infty$  شد یعنی رشد  $f(n)$  بیشتر است و اگر عدد ثابت  $k \neq 0$  شد یعنی هم رشد هستند:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \begin{cases} 0 \leftrightarrow & \text{رشد } f(n) \text{ از } g(n) \text{ کمتر است.} \\ \infty \leftrightarrow & \text{رشد } f(n) \text{ از } g(n) \text{ بیشتر است.} \\ k \neq 0 \leftrightarrow & \text{رشد } f(n) \text{ با } g(n) \text{ هم رشد است.} \end{cases}$$

**مثال ۹:** رشد  $f(n) = \lg n$  و  $g(n) = \sqrt{n}$  را با هم مقایسه کنید.

**پاسخ:** از حد استفاده می‌کنیم و می‌دانیم اگر حد مبهم  $\frac{0}{0}$  یا  $\frac{\infty}{\infty}$  شود می‌توان از هوپیتال کمک گرفت یعنی  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{f'(n)}{g'(n)}$ . در ضمن مشتق  $\sqrt{n}$  برابر  $\frac{1}{2\sqrt{n}}$  است. و مشتق  $\log_a^n$  برابر  $\frac{1}{n \cdot \ln a}$  است پس:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{\lg n}{\sqrt{n}} = \frac{\infty}{\infty} \text{ مبهم}$$

$$\rightarrow \lim_{n \rightarrow \infty} \frac{f'(n)}{g'(n)} = \lim_{n \rightarrow \infty} \frac{\frac{1}{n \cdot \ln 2}}{\frac{1}{2\sqrt{n}}} = \lim_{n \rightarrow \infty} \frac{2\sqrt{n}}{\ln 2 \times n} = \lim_{n \rightarrow \infty} \frac{2}{\ln 2 \times \sqrt{n}} = 0.$$

پس رشد  $\lg n$  از  $\sqrt{n}$  کمتر است.

**مثال ۱۰:** رشد  $f(n) = \log_3^n$  را با  $g(n) = \log_4^n$  مقایسه کنید.

**پاسخ:** از حد کمک می‌گیریم و می‌دانیم  $\log_b^a = \frac{\ln a}{\ln b}$  پس:

$$\lim_{n \rightarrow \infty} \frac{\log_3^n}{\log_4^n} = \lim_{n \rightarrow \infty} \frac{\frac{\ln n}{\ln 3}}{\frac{\ln n}{\ln 4}} = \frac{\ln 4}{\ln 3} = \log_3^4 \neq 0.$$

بنابراین  $\log_3^n$  با  $\log_4^n$  هم رشد است. ■

لازم به ذکر است که ما فرض می‌کنیم دامنه توابع اعداد طبیعی  $\mathbb{N} = \{0, 1, 2, 3, \dots\}$  است. (توجه کنید در برخی منابع  $\mathbb{N} = \{1, 2, 3, \dots\}$  تعریف می‌شود) و همچنین توابع در این درس، مثبت هستند یعنی  $f(n) \geq 0$ . البته توابع می‌توانند صعودی یا نزولی یا حتی متناوب باشند.

**مثال ۱۱:** رشد  $f(n) = \frac{1}{n^2}$  و  $g(n) = \frac{1}{n}$  را با هم مقایسه کنید.

**پاسخ:** رشد  $\frac{1}{n^2}$  از  $\frac{1}{n}$  کمتر است:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{\frac{1}{n^2}}{\frac{1}{n}} = \lim_{n \rightarrow \infty} \frac{1}{n} = 0.$$

**مثال ۱۲:** رشد  $f(n) = 2^{n^2}$  را با  $g(n) = 2^n$  مقایسه کنید.

**پاسخ:** هم رشد هستند چون حاصل حد عدد ثابت مخالف صفر است:

$$\lim_{n \rightarrow \infty} \frac{2^{n^2}}{2^n} = \frac{2^0}{2^0} = 1 \quad \blacksquare$$

در ادامه نکاتی را مطرح می‌کنیم که در بررسی رشد توابع بسیار مفید هستند:

۱- ضریب ثابت تاثیری در رشد ندارد مثلاً  $2n$  با  $4n$  هم رشد است. دقت کنید  $2^{2n}$  رشد کمتری

از  $2^{4n}$  دارد. یعنی تابع  $f(n)$  با  $c \cdot f(n)$  که  $c$  ثابت، هم‌رشد است.

۲- رشد تابع  $a^n$  از  $n^b$  بیشتر است که  $a, b$  عدد ثابت هستند و  $a > b$ . مثلاً  $n^3$  از  $n^2$  رشد

بیشتری دارد. یا رشد  $n^{1/2}$  از رشد  $n^{0.5}$  بیشتر است. یا رشد  $n^{-1}$  از  $n^{-2}$  بیشتر است.

۳- رشد  $a^n$  از  $b^n$  بیشتر است که  $a, b$  عدد ثابت هستند و  $a > b$ . مثلاً رشد  $2^n$  از  $(1/5)^n$

بیشتر است. یا رشد  $(9/10)^n$  از  $(8/10)^n$  بیشتر است.

۴- رشد تابع  $a^n$  از  $n^b$  به شرطی که  $a, b$  ثابت و  $a > 1$ ، بیشتر است. مثلاً رشد تابع نمایی  $2^n$

از رشد تابع چند جمله‌ای  $n^{200}$  بیشتر است. یا رشد  $1/1^n$  از رشد  $n^4$  بیشتر است. دقت کنید

رشد  $9/10^n$  از  $n^2$  کمتر است. چون  $9/10^n$  تابع نزولی است و  $n^2$  صعودی است.

۵- رشد  $a^n$  از  $(\lg n)^b = \lg^b n$  که  $a, b$  ثابت و  $a > 0$ ، بیشتر است. مثلاً  $n^{0.5} = \sqrt{n}$  رشد

بیشتری از  $\lg^{20} n$  دارد.

۶- رشد  $(\lg n)^a = \lg^a n$  از  $(\lg \lg n)^b = \lg^b \lg n$  که  $a, b$  ثابت و  $a > 0$ ، بیشتر است. مثلاً

$\sqrt{\lg n}$  از  $\lg^5 \lg n$  رشد بیشتری دارد.

۷- رشد  $\log_a^n$  با  $\log_b^n$  که  $a, b > 1$  ثابت، یکسان است. مثلاً  $\log_3^n$  با  $\log_4^n$  هم‌رشد هستند.

دقت کنید تابعی مثل  $\log_{n/2}^n$  مقادیر منفی دارد و ما توابع مثبت را بررسی می‌کنیم پس چنین

تابعی اصلاً مورد بحث نیست.

۸- رشد  $n^n$  از  $n!$  بیشتر است.

۹- تابع  $\lg(n!)$  با  $n \cdot \lg n$  هم رشد است.

۱۰- اگر  $f(n)$  و  $g(n)$  صعودی باشند و  $\lg(f(n))$  از  $\lg(g(n))$  رشد بیشتری داشته باشد آنگاه

$f(n)$  از  $g(n)$  رشد بیشتری دارد. مثلاً رشد  $f(n) = \lg^n n$  از  $g(n) = n^{\lg n}$  بیشتر است

زیرا  $\lg(f(n)) = n \cdot \lg \lg n$  از  $\lg(g(n)) = \lg^n n$  رشد بیشتری دارد.

۱۱- اگر رشد  $f(n)$  از  $g(n)$  بیشتر باشد آنگاه رشد  $\lg(f(n))$  از رشد  $\lg(g(n))$  بیشتر یا

مساوی است. ( $f$  و  $g$  توابع صعودی)

**توجه:** برای اثبات ۱ تا ۸ می‌توانید از حد استفاده کنید. اثبات ۹ را خواهیم دید.

**مثال ۱۳:** توابع هر قسمت را به ترتیب افزایش رشد مرتب کنید.

$$f_1 = n^3, f_2 = \lg^3 n, f_3 = \left(\frac{3}{2}\right)^n, f_4 = 2^{2^n}, f_5 = \lg(n!), f_6 = n^{\frac{1}{\lg n}} \quad (a)$$

$$f_1 = 4^{\lg n}, f_2 = e^n, f_3 = (n+2)!, f_4 = \sqrt{\lg n}, f_5 = 2^{\lg n} \quad (b)$$

$$f_1 = 4^{(\lg n + \lg \lg n)}, f_2 = n^2 \lg^3 \sqrt{n} \quad (c)$$

$$f_1 = n!, f_2 = (1/0.5)^n, f_3 = n^{1.00} \quad (d)$$

$$f_1 = 2^n, f_2 = e^{2\sqrt{n}}, f_3 = e^{\sqrt{n}}, f_4 = e^{\frac{n}{2}} \quad (e)$$

$$f_1 = n(\log_2^n)^4, f_2 = n^{1/2}, f_3 = \lg n, f_4 = \frac{n}{\lg n} \quad (f)$$

**پاسخ:**

(a) می‌دانیم  $2 = n^{\frac{1}{\lg n}}$  و  $\lg n! \sim n \cdot \lg n$  ("~" یعنی هم رشد هستند) پس:

$$f_6 < f_2 < f_5 < f_1 < f_3 < f_4$$

(b)  $4^{\lg n} = n^2$  و  $2^{\lg n} = n$  پس  $f_4 < f_5 < f_1 < f_2 < f_3$

(c)  $f_1$  و  $f_2$  را ساده می‌کنیم:

$$f_1 = 4^{\lg n + \lg \lg n} = 4^{\lg n} \times 4^{\lg \lg n} = n^{\lg 4} \times (\lg n)^{\lg 4} = n^2 \cdot \lg^2 n$$

$$f_2 = n^2 \lg^3 \sqrt{n} = n^2 (\lg \sqrt{n})^3 = n^2 \left(\frac{1}{2} \lg n\right)^3 \sim n^2 \lg^3 n$$

در نتیجه  $f_1$  از  $f_2$  رشد کمتری دارد.

(d) با توجه به اینکه چند جمله‌ای از نمایی و نمایی از فاکتوریل رشد کمتری دارد پس

$$f_3 < f_2 < f_1$$

(e)  $e^{\sqrt{n}}$  از  $e^{\sqrt[2]{n}}$  رشد کمتری دارد و هر دو از  $e^{\frac{n}{2}}$  و  $2^n$  رشد کمتری دارند و  $e^{\frac{n}{2}} = \sqrt{e}^n$  و چون  $\sqrt{e} < 2$  پس  $e^{\frac{n}{2}} < 2^n$ ، در نتیجه  $f_3 < f_2 < f_4 < f_1$ .

(f) برای مقایسه  $\frac{n}{\lg n}$  و  $\lg n$  با توجه به اینکه  $\lg^2 n < n$  پس طرفین را به  $\lg n$  تقسیم کنیم

داریم  $\lg n < \frac{n}{\lg n}$ . سایر توابع به راحتی قابل بررسی هستند:  $f_3 < f_4 < f_1 < f_2$ .

### تابع $\lg^* n$ (لگ استار)

$\lg^* n$  برابر است با تعداد باری که باید از  $n$  لگاریتم پایه ۲ بگیریم که حاصل برابر عدد ۱ یا کمتر از ۱ شود. مثلاً  $\lg^* 16 = 3$  زیرا اگر از ۱۶ سه بار لگاریتم پایه ۲ بگیریم، حاصل برابر ۱ می‌شود. ( $\lg \lg \lg 16 = 1$ ). به چند نمونه دقت کنید:

$$\lg^* 2 = 1, \lg^* 2^2 = \lg^* 4 = 2, \lg^* 2^4 = \lg^* 16 = 3, \lg^* 2^{16} = \lg^* 65536 = 4, \lg^* 2^{65536} = 5$$

$\lg^* n$  تابعی است با رشد بسیار ناچیز. رشد این تابع از تمام توابع صعودی که تاکنون معرفی کردیم کمتر است.

**مثال ۱۴:** توابع هر قسمت را به ترتیب افزایش رشد مرتب کنید.

$$f_1 = n^2, f_2 = 2^{\lg^* n}, f_3 = n!, f_4 = 2^{2^{n+1}}, f_5 = (\sqrt{2})^{\lg n} \quad (a)$$

$$f_1 = 2^n, f_2 = \lg n, f_3 = \lg \lg n, f_4 = n.2^n, f_5 = \sqrt{\lg n}, f_6 = \lg^* n, f_7 = n^{\lg \lg n} \quad (b)$$

$$f_1 = n, f_2 = \left(\frac{3}{2}\right)^n, f_3 = 2\sqrt{2\lg n}, f_4 = \lg^* n^n, f_5 = n.\lg n, f_6 = n^{2+\lg n} \quad (c)$$

پاسخ:

$$\sqrt{2}^{\lg n} = 2^{\frac{1}{2}\lg n} = 2^{\lg \sqrt{n}} = \sqrt{n}. \quad (a)$$

برای مقایسه  $2^{2^{n+1}}$  با  $n!$ ، از این دو تابع لگاریتم بگیریم.  $\lg 2^{2^{n+1}} = 2^{n+1}$  و  $\lg n! \sim n.\lg n$  و چون  $2^{n+1}$  رشد بیشتری از  $\lg n!$  دارد پس  $2^{2^{n+1}}$  رشد بیشتری از  $n!$  دارد:

$$f_2 < f_5 < f_1 < f_3 < f_4$$

$$f_6 < f_3 < f_5 < f_2 < f_7 < f_1 < f_4 \quad (b)$$

$$f_4 < f_3 < f_1 < f_5 < f_6 < f_2 \quad (c)$$



## نمادهای مجانبی asymptotic notations

برای بیان زمان اجرا یا حافظه مصرفی الگوریتم‌ها از نمادهای مجانبی استفاده می‌شود که عبارتند از  $\omega, o, \theta, \Omega, O$ . این نمادها را به زبان ساده تعریف می‌کنیم و سپس تعریف دقیق‌تر آن را بیان می‌کنیم.

**نماد  $O$  (big-oh):** گوییم تابع  $f(n)$  از مرتبه  $O$  تابع  $g(n)$  است و می‌نویسیم  $f(n) = O(g(n))$  یا  $f(n) \in O(g(n))$  هرگاه رشد تابع  $f(n)$  از تابع  $g(n)$  کمتر یا مساوی باشد. مثلاً  $3n^2 + 2n + 4 \in O(n^2)$  زیرا  $3n^2 + 2n + 4 \in O(n^2)$  با  $n^2$  هم رشد است. یا  $2n + \lg n \in O(2^n)$  زیرا  $2n + \lg n$  رشد کمتری از  $2^n$  دارد.

**نماد  $\Omega$  (big omega):** گوییم تابع  $f(n)$  از مرتبه  $\Omega$  تابع  $g(n)$  است و می‌نویسیم  $f(n) = \Omega(g(n))$  یا  $f(n) \in \Omega(g(n))$  هرگاه رشد تابع  $f(n)$  از تابع  $g(n)$  بیشتر یا مساوی باشد. مثلاً  $3n^2 + 2n + 4 \in \Omega(n^2)$  زیرا  $3n^2 + 2n + 4 \in \Omega(n^2)$  با  $n^2$  هم رشد است. یا  $3n^2 + 2n \in \Omega(n^3)$  ولی  $3n^2 + 2n \notin \Omega(n^3)$  زیرا  $3n^2 + 2n$  رشد کمتری از  $n^3$  دارد.

**نماد  $\theta$  (تتا):** گوییم تابع  $f(n)$  از مرتبه  $\theta$  تابع  $g(n)$  است و می‌نویسیم  $f(n) = \theta(g(n))$  یا  $f(n) \in \theta(g(n))$  هرگاه  $f(n)$  و  $g(n)$  هم رشد باشند. مثلاً  $3n^2 + 2n + 4 \in \theta(n^2)$  چون  $3n^2 + 2n + 4 \in \theta(n^2)$  با  $n^2$  هم رشد است. یا  $3n^2 + 2n + 4 \in \theta(3^n)$  چون  $3n^2 + 2n + 4 \in \theta(3^n)$  با  $3^n$  هم رشد است. دقت کنید در جمع تعدادی تابع، تابعی که رشد بیشتری دارد برنده است. و چون  $3^n$  از  $n \cdot 2^n$  و از  $n$  رشد بیشتری دارد پس  $3^n$  برنده است.

**نماد  $o$  (small oh یا little oh):** گوییم تابع  $f(n)$  از مرتبه  $o$  تابع  $g(n)$  است و می‌نویسیم  $f(n) = o(g(n))$  یا  $f(n) \in o(g(n))$  هرگاه رشد  $f(n)$  از  $g(n)$  کمتر باشد. مثلاً  $3n^2 + 2n + 4 \in o(n^3)$  چون رشد  $3n^2 + 2n + 4$  از  $o(n^3)$  کمتر است. یا  $3n^2 + 2n + 4 \notin o(n^2)$  چون رشد  $3n^2 + 2n + 4$  با  $n^2$  یکسان است و کمتر نیست.

**نماد  $\omega$  (small omega):** گوییم تابع  $f(n)$  از مرتبه  $\omega$  تابع  $g(n)$  است و می‌نویسیم  $f(n) = \omega(g(n))$  یا  $f(n) \in \omega(g(n))$  هرگاه رشد  $f(n)$  از  $g(n)$  بیشتر باشد. مثلاً  $3n^2 + 2n \in \omega(\sqrt{n})$  زیرا رشد  $3n^2 + 2n$  از  $\sqrt{n}$  بیشتر است. یا  $\sqrt{n} + \lg n \in \omega(\lg n)$  ولی  $\sqrt{n} + \lg n \notin \omega(\sqrt{n})$ .

**تست ۲:** توابع  $f(n) = 4^{\lg n}$  و  $g(n) = (\log n)^{\log n}$  و  $h(n) = \log^2 n$  را در نظر بگیرید. کدام

(تخصصی هوش مصنوعی ۸۵)

یک از گزاره‌های زیر صحیح است؟

$$f(n) \in O(g(n)), f(n) \in \Omega(h(n)) \quad (۱)$$

$$g(n) \in \Omega(h(n)), h(n) \in \Omega(f(n)) \quad (۲)$$

$$f(n) \in \theta(h(n)), g(n) \in \Omega(f(n)) \quad (۳)$$

$$h(n) \in O(g(n)), f(n) \in \theta(g(n)) \quad (۴)$$

**پاسخ:** با توجه به اینکه  $4^{\lg n} = n^{\lg 4} = n^2$  و  $(\lg n)^{\lg n} = n^{\lg \lg n}$  و چون  $n^2$  از  $n^{\lg \lg n}$  رشد کمتری دارد (چون  $2$  از  $\lg \lg n$  رشد کمتری دارد) پس  $f(n) \in O(g(n))$  یا  $g(n) \in \Omega(f(n))$ . از طرفی  $g(n)$  از  $h(n)$  رشد بیشتری دارد (چون  $\lg n$  از  $2$  رشد بیشتری دارد) و همچنین  $n^2$  از  $\lg^2 n$  رشد بیشتری دارد پس ترتیب رشد ها  $h < f < g$  است که گزینه ۱ صحیح است.

(علوم کامپیوتر ۸۴)

**تست ۳:** کدام گزینه صحیح است؟

$$\sqrt{n} = O(\lg n) \quad (۲) \quad n^3 \lg n = O(n^{3+\epsilon}), \epsilon > 0 \quad (۱)$$

$$n^2 = O\left(\frac{n^2}{\lg n}\right) \quad (۴) \quad n^{1+\epsilon} = O(n \cdot \lg n), \epsilon > 0 \quad (۳)$$

**پاسخ:** می‌دانیم  $n^\epsilon$  از  $\lg n$  رشد بیشتری دارد (به ازای هر  $\epsilon > 0$ ). پس اگر در  $n^3$  ضرب کنید  $n^{3+\epsilon}$  از  $n^3 \lg n$  رشد بیشتری دارد بنابراین گزینه ۱ صحیح است. رشد  $\sqrt{n}$  از  $\lg n$  بیشتر است. رشد  $n^{1+\epsilon}$  از  $n \cdot \lg n$  بیشتر است ( $\epsilon > 0$ ). و رشد  $n^2$  از  $\frac{n^2}{\lg n}$  بیشتر است.

(علوم کامپیوتر ۸۱)

**تست ۴:** کدام عبارت صحیح است؟

$$3^n = O(2^n) \quad (۲) \quad \sum_{i=0}^n i^2 = O(n^3) \quad (۱)$$

$$\frac{n^2}{\lg n} = \Omega(n^2) \quad (۴) \quad n^2 \lg n = O(n^2) \quad (۳)$$

**پاسخ:** می‌دانیم  $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6} = \theta(n^3)$  پس گزینه ۱ صحیح است. رشد  $3^n$  از  $2^n$

بیشتر است. رشد  $n^2 \lg n$  از  $n^2$  بیشتر است. رشد  $\frac{n^2}{\lg n}$  از  $n^2$  کمتر است.

## نکات و خواص نمادها

۱- حدگیری: می‌توان با حدگرفتن رابطه بین توابع را مشخص کرد:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \begin{cases} 0 & \leftrightarrow f(n) < \text{رشد } g(n) \leftrightarrow f(n) \in o(g(n)) \rightarrow f(n) \in O(g(n)) \\ \infty & \leftrightarrow f(n) > \text{رشد } g(n) \leftrightarrow f(n) \in \omega(g(n)) \rightarrow f(n) \in \Omega(g(n)) \\ k \neq 0 & \leftrightarrow f(n) = \text{رشد } g(n) \leftrightarrow f(n) \in \theta(g(n)) \end{cases}$$

توجه کنید اگر  $f(n) \in o(g(n))$  آنگاه  $f(n) \in O(g(n))$  ولی لزوماً عکس این مطلب صحیح نیست. زیرا اگر رشد  $f(n)$  از  $g(n)$  کمتر باشد می‌توان گفت رشد  $f(n)$  از  $g(n)$  یا کمتر یا مساوی است. ولی اگر رشد  $f(n)$  کمتر یا مساوی  $g(n)$  باشد نمی‌توان گفت رشد  $f(n)$  کمتر از  $g(n)$  است. به همین ترتیب اگر  $f(n) \in \omega(g(n))$  آنگاه  $f(n) \in \Omega(g(n))$  ولی نه لزوماً بر عکس.

۲- ماکزیمم‌گیری:  $\theta(f+g)$  با  $\theta(\max(f,g))$  برابر است. مثلاً  $\theta(n^2 + 2n)$  با  $\theta(n^2)$  مساوی است. به همین ترتیب  $O(f+g)$  با  $O(\max(f,g))$  و  $\Omega(f+g)$  با  $\Omega(\max(f,g))$  برابر است.

۳- بازتابی (انعکاسی): نمادهای  $O$  و  $\Omega$  و  $\theta$  بازتاب هستند یعنی  $f(n) \in \theta(f(n))$  و  $f(n) \in O(f(n))$  و  $f(n) \in \Omega(f(n))$  ولی  $f(n) \notin o(f(n))$  و  $f(n) \notin \omega(f(n))$ .

۴- تقارنی: فقط نماد  $\theta$  متقارن است یعنی اگر  $f(n) \in \theta(g(n))$  آنگاه  $g(n) \in \theta(f(n))$  و برعکس.

۵- تقارنی ترانهاده (transpose symmetric): اگر  $f(n) \in O(g(n))$  آنگاه  $g(n) \in \Omega(f(n))$  و برعکس. و اگر  $f(n) \in o(g(n))$  آنگاه  $g(n) \in \omega(f(n))$  و برعکس.

۶- تعدی (transitive): همه نمادها خاصیت تعدی دارند. اگر  $f(n) \in \theta(g(n))$  و  $g(n) \in \theta(h(n))$  آنگاه  $f(n) \in \theta(h(n))$ . بجای نماد  $\theta$  می‌توان نمادهای  $\omega, o, \Omega, O$  قرار داد.

تست ۵: فرض کنید  $f$  و  $g$  دو تابع دلخواه به شکل  $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$ . بعلاوه فرض کنید:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty \quad \text{کدام گزینه صحیح است؟}$$

(ساختمان داده ۸۲)

$$(۱) \quad f(n) \in O(g(n)), g(n) \notin \Omega(f(n))$$

$$(۲) \quad g(n) \in O(f(n)), f(n) \in O(g(n))$$

$$(۳) \quad f(n) \in \theta(g(n)), g(n) \notin \Omega(f(n))$$

$$(۴) \quad f(n) \in \Omega(g(n)), f(n) \notin \theta(g(n))$$

**پاسخ:** چون حاصل حد برابر بی‌نهایت است پس رشد  $f(n)$  بیشتر است پس  $f \in \Omega(g)$  ولی  $f \notin \theta(g)$  می‌توان گفت  $f \in \omega(g)$ . گزینه ۴ صحیح است.

(علوم کامپیوتر ۸۰)

**تست ۶:** گزینه درست کدام است؟

$$(۱) f(n) \in O(g(n)), h(n) \in \Omega(g(n)) \Rightarrow f(n) \in \theta(h(n))$$

$$(۲) f(n) \in \theta(g(n)), g(n) \in O(h(n)) \Rightarrow h(n) \in O(f(n))$$

$$(۳) f(n) \in \Omega(g(n)), h(n) \in O(g(n)) \Rightarrow f(n) \in \theta(g(n))$$

$$(۴) f(n) \in \Omega(g(n)), g(n) \in \theta(h(n)) \Rightarrow f(n) \in \Omega(h(n))$$

**پاسخ:** در گزینه ۱،  $f \leq g$  (منظور رشد توابع است) و  $h \geq g$  از این دو نمی‌توان نتیجه گرفت  $f$  و  $h$  هم رشد هستند.

در گزینه ۲،  $f \approx g$  و  $g \leq h$  از این دو نمی‌توان نتیجه گرفت  $h \leq f$  بلکه می‌توان گفت  $f \leq h$  یعنی  $f \in O(h)$ . (منظور از  $\approx$  یعنی هم رشد هستند)

در گزینه ۳،  $f \geq g$  و  $h \leq g$  از این دو نمی‌توان نتیجه گرفت  $h \leq f$  و نمی‌توان نتیجه گرفت  $f \approx g$ .

در گزینه ۴،  $f \geq g$  و  $g \approx h$  که می‌توان نتیجه گرفت  $f \geq h$  پس گزینه ۴ صحیح است. ■

**۷- اشتراک:** اگر  $f(n) \in O(g(n))$  و  $f(n) \in \Omega(g(n))$  آنگاه  $f(n) \in \theta(g(n))$ . در واقع اشتراک  $\Omega, O$  برابر  $\theta$  است یعنی  $O(f(n)) \cap \Omega(f(n)) = \theta(f(n))$ .

**۸- اشتراک:** اشتراک  $o(f(n))$  با  $\omega(f(n))$  برابر تهی است:  $o(f(n)) \cap \omega(f(n)) = \emptyset$ .

**مثال ۱۵:**  $O(n^2)$  و  $\Omega(n^2)$  شامل چه توابعی هستند؟

**پاسخ:**  $O(n^2)$  یعنی توابعی با رشد کمتر یا مساوی  $n^2$ . در واقع  $f \in O(n^2)$  اگر رشد  $f$  از رشد  $n^2$  کمتر یا مساوی باشد.  $\Omega(n^2)$  یعنی توابع با رشد بیشتر یا مساوی  $n^2$ . بنابراین  $O(n^2) \cap \Omega(n^2)$  یعنی  $\theta(n^2)$ . دقت کنید  $O(n^2)$  یک مجموعه است که شامل بی‌شمار تابع است که رشد این توابع از  $n^2$  کمتر یا مساوی است. پس بهتر است از علامت عضویت استفاده شود. مثلاً بگوییم  $2n^2 \in O(n^2)$  نه  $2n^2 = O(n^2)$  هر چند استفاده علامت تساوی درست نیست ولی مراجع آن را پذیرفته‌اند پس هم می‌توان نوشت  $2n^2 \in O(n^2)$  و هم  $2n^2 = O(n^2)$ .

### تعریف ریاضی نمادهای مجانبی

مفهوم نمادها را اکنون می‌دانیم. می‌خواهیم نمادها را به طور دقیق‌تری تعریف کنیم:

**نماد O:** تابع  $f(n)$  از مرتبه  $O(g(n))$  است هرگاه بتوان اعداد بزرگتر از صفر  $n_0$  و  $c$  را طوری یافت که به ازای همه اعداد صحیح  $n \geq n_0$ ، داشته باشیم  $f(n) \leq c \cdot g(n)$ :

$$\exists c, n_0 > 0, \forall n \geq n_0 : f(n) \leq c \cdot g(n)$$

مثلاً  $4n^2 + 2n + 3 \in O(n^2)$  زیرا  $4n^2 + 2n + 3 \leq cn^2$  به ازای مثلاً  $c = 5$  می‌تواند هر عدد بزرگتر از ۴ باشد) نامساوی برای  $n$  های بیشتر یا مساوی  $n_0 = 3$  برقرار است. و یا  $4n^2 + 2n + 3 \notin O(n)$  زیرا نامساوی  $4n^2 + 2n + 3 \leq cn$  برقرار نیست. زیرا  $c$  را هر مقداری انتخاب کنیم، از یک نقطه  $n_0$  به بعد  $4n^2 + 2n + 3$  از  $cn$  بزرگتر می‌شود چون تابع درجه ۲ از درجه ۱ رشد بیشتری دارد. در واقع با این تعریف ریاضی مشخص می‌شود که  $f(n) \in O(g(n))$  یعنی رشد  $f(n)$  از رشد  $g(n)$  کمتر یا مساوی است.

**نماد  $\Omega$ :** گوییم  $f(n) \in \Omega(g(n))$  هرگاه:

$$\exists c, n_0 > 0, \forall n \geq n_0 : f(n) \geq c.g(n)$$

یعنی بتوان اعداد مثبت  $c$  و  $n_0$  را طوری یافت که به ازای همه  $n$  های بزرگتر مساوی  $n_0$ ، نامساوی  $f(n) \geq c.g(n)$  برقرار باشد. مثلاً  $4n^2 + 2n + 3 \in \Omega(n^2)$  زیرا نامساوی  $4n^2 + 2n + 3 \geq cn^2$  به ازای مثلاً  $c = 4$  و هر  $n_0$  مثبتی برای همه  $n$  های از  $n_0$  بیشتر برقرار است.

**نماد  $\theta$ :** گوییم  $f(n) \in \theta(g(n))$  هرگاه:

$$\exists c_1, c_2, n_0 > 0, \forall n \geq n_0 : c_1 g(n) \leq f(n) \leq c_2 g(n)$$

مثلاً  $4n^2 + 2n + 3 \in \theta(n^2)$  زیرا  $c_1 n^2 \leq 4n^2 + 2n + 3 \leq c_2 n^2$  به ازای مثلاً  $c_1 = 4$  و  $c_2 = 5$  و  $n_0 = 3$  به ازای همه  $n$  های از  $n_0$  بیشتر برقرار است.

**نماد  $o$ :** گوییم  $f(n) \in o(g(n))$  هرگاه نامساوی  $f(n) < c.g(n)$  به ازای همه  $c$  های مثبت ( $c > 0$ ) از یک مقدار  $n_0$  به بعد برقرار باشد:

$$\forall c > 0, \exists n_0 > 0, \forall n \geq n_0 : f(n) < c.g(n)$$

توجه کنید تفاوت تعریف ریاضی  $o$  با  $O$  این است که در  $O$  گفته می‌شود به ازای برخی  $c$  ها و در  $o$  گفته می‌شود به ازای همه  $c$  ها. مثلاً  $n^2 + 2n \notin o(n^2)$  چون نامساوی  $n^2 + 2n < cn^2$  به ازای  $c = \frac{1}{4}$  برقرار نیست. لازم به ذکر است که  $f(n) < c.g(n)$  را می‌توان به شکل  $f(n) \leq c.g(n)$  نیز نوشت. مثلاً  $n^2 + 2n \leq cn^2$  نیز به ازای  $c = \frac{1}{4}$  برقرار نیست. در واقع در تعریف ریاضی  $O$  و  $o$  می‌توان  $f(n) < c.g(n)$  یا  $f(n) \leq c.g(n)$  نوشت و مهم این است که  $O$  می‌گوید به ازای برخی  $c$  ها و  $o$  می‌گوید به ازای همه  $c$  ها.

**مثال ۱۶:** آیا  $n^2 + 2n \in o(n^3)$  درست است؟

**پاسخ:** بله. زیرا  $n^2 + 2n < c.n^3$  یا  $n^2 + 2n \leq c.n^3$  به ازای همه مقادیر  $c > 0$  از یک  $n_0$  به بعد برقرار است.

**نماد  $\omega$ :** گوییم  $f(n) \in \omega(g(n))$  هرگاه:

$$\forall c > 0, \exists n_0 > 0, \forall n \geq n_0 : f(n) > c.g(n) \text{ or } f(n) \geq c.g(n)$$

این تعریف نشان می‌دهد که رشد  $f(n)$  از  $g(n)$  بیشتر است.

**مثال ۱۷:** آیا می‌توان ادعا کرد به ازای هر دو تابع  $f(n)$  و  $g(n)$  همواره  $f(n) \in O(g(n))$  یا

$$f(n) \in \Omega(g(n)) \text{ (یا هر دو)}?$$

**پاسخ:** هر چند برای اکثر توابع این جمله درست است ولی برای برخی توابع مثلاً متناوب لزوماً

درست نیست. مثلاً فرض کنید  $f(n) = n$  و  $g(n) = n^{1+\sin n}$ . حال بررسی می‌کنیم

$f(n) \in O(g(n))$  نادرست است زیرا نامساوی  $n \leq c.n^{1+\sin n}$  به ازای مواقعی که  $\sin n$  منفی

می‌شود ( $-1 < \sin n < 0$ ) نادرست است.  $f(n) \in \Omega(g(n))$  نادرست است زیرا نامساوی

$n \geq c.n^{1+\sin n}$  به ازای مواقعی که  $\sin n$  مثبت می‌شود ( $0 < \sin n < 1$ ) نادرست است.

## خلاصه فصل

۱- خواص سیگما را بررسی کردیم و مهم‌ترین نتیجه‌ای که می‌توان گرفت:

$$\sum_{k=1}^n k^p = 1^p + 2^p + \dots + n^p \sim \frac{1}{p+1} n^{p+1} = \theta(n^{p+1}) \quad (p \geq 0)$$

که برای درک درستی این رابطه می‌توانید بجای سیگما، انتگرال قرار دهید.

۲- سری همسازه (هارمونیک)  $\sum_{k=1}^n \frac{1}{k}$  است که با  $\ln n$  معادل است، پس  $\sum_{k=1}^n \frac{1}{k} = \theta(\ln n)$ . زیرا لگاریتم‌ها با هر پایه ثابتی هم رشد هستند.

۳- سری هندسی  $1 + x + x^2 + \dots + x^n = \frac{1-x^{n+1}}{1-x}$  که اگر  $n \rightarrow \infty$  و  $|x| < 1$  می‌توان نتیجه

گرفت  $1 + x + x^2 + \dots = \frac{1}{1-x}$  و اگر از طرفین مشتق بگیریم نتیجه می‌شود

$1 + 2x + 3x^2 + \dots = \frac{1}{(1-x)^2}$  می‌توان با مشتق‌گیری یا انتگرال‌گیری فرمول‌های دیگری نیز یافت.

۴- خواص لگاریتم بررسی شد.  $\log_b^a = c$  یعنی  $a = b^c$  و قرارداد شد که  $\log_2^n$  را  $\lg n$  یا  $\lg n$  بنویسیم.

۵- رشد توابع را بررسی کردیم و می‌توان ترتیب رشد توابع معروف را به شکل زیر نوشت: (a ثابت مثبت)

$$\dots < \frac{1}{n^2} < \frac{1}{n} < 1 < \lg^* n < \lg^a n < \lg n < \lg^a n < n < n \cdot \lg n < n^2 < n^3 < \dots$$

$$\dots < 2^n < 3^n < \dots < n! < n^n$$

(تابع  $\lg^* n$  برابر است با تعداد باری که باید از  $n$  لگاریتم پایه ۲ بگیریم که حاصل برابر یک یا کمتر شود).

۶- برای مقایسه رشد  $f(n)$  با  $g(n)$  می‌توان از حد استفاده کرد. اگر حد  $\frac{f(n)}{g(n)}$  وقتی  $n \rightarrow \infty$

برابر صفر شود، رشد  $g(n)$  بیشتر است. اگر  $\infty$  شود رشد  $f(n)$  بیشتر است در غیر این صورت هم رشد هستند.

۷- نمادهای مجانبی بررسی شدند و می‌توان گفت مفهوم نمادها به شکل زیر است:

$$f(n) \in O(g(n)) \leftrightarrow \text{رشد } f \leq \text{رشد } g$$

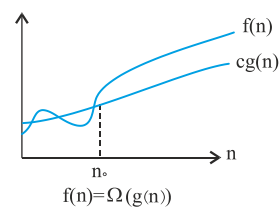
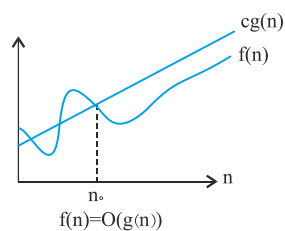
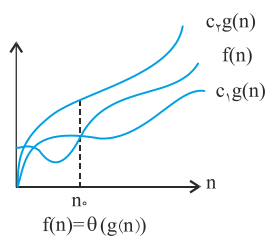
$$f(n) \in \Omega(g(n)) \leftrightarrow \text{رشد } f \geq \text{رشد } g$$

$$f(n) \in \theta(g(n)) \leftrightarrow \text{رشد } f = \text{رشد } g$$

$$f(n) \in o(g(n)) \leftrightarrow \text{رشد } f < \text{رشد } g$$

$$f(n) \in \omega(g(n)) \leftrightarrow \text{رشد } f > \text{رشد } g$$

۸- برای نمادهای  $\theta, \Omega, O$  می‌توان شکل زیر را کشید:





## تمرین

۱- حاصل عبارات زیر را بیابید:

a)  $\sum_{k=1}^{n-1} \frac{1}{k(k+1)}$

b)  $\sum_{k=1}^n (2k-1)$

c)  $\sum_{k=1}^{\infty} (2k+1)x^{2k}$

d)  $\sum_{k=0}^{\infty} \frac{(k-1)}{2^k}$

e)  $\prod_{k=1}^n 2(2^k)$

f)  $\prod_{k=2}^n \left(1 - \frac{1}{k^2}\right)$

۲- نشان دهید  $\sum_{k=0}^{\infty} k^2 x^k = \frac{x(1+x)}{(1-x)^3}$  به ازای  $0 < |x| < 1$ .۳- نشان دهید  $\sum_{k=1}^n \frac{1}{(2k-1)} = \ln(\sqrt{n}) + c$  که c ثابت است.۴- نشان دهید  $\sum_{k=1}^n \frac{1}{k^2}$  به یک ثابت محدود می‌شود.۵- برای مجموع‌های زیر، بهترین کران را بیابید.  $r \geq 0$  و s ثابت هستند.

a)  $\sum_{k=1}^n k^r$

b)  $\sum_{k=1}^n \lg^s k$

c)  $\sum_{k=1}^n k^r \lg^s k$

۶- ثابت کنید  $\lg n! = \theta(n \lg n)$  یعنی نشان دهید  $\lg n!$  با  $n \lg n$  هم رشد است.

۷- درستی یا نادرستی گزاره‌های زیر را بررسی کنید و یا شرایط درستی را بیان کنید.

a) اگر  $f(n) \in O(g(n))$  آنگاه  $2^{f(n)} \in O(2^{g(n)})$ b) اگر  $f(n) \in o(g(n))$  آنگاه  $2^{f(n)} \in o(2^{g(n)})$ c) اگر  $f(n) \in O(g(n))$  آنگاه  $\lg f(n) \in O(\lg g(n))$  برای n های به اندازه کافی بزرگ $Lg(g(n)) \geq 1$  و  $f(n) \geq 1$ d)  $f(n) \in O(f^2(n))$ e)  $f(n) \in \theta\left(f\left(\frac{n}{2}\right)\right)$ f)  $f(n) + O(f(n)) \in \theta(f(n))$ g)  $O(f(n)) - \Omega(f(n))$  با  $o(f(n))$  مساوی است.h)  $f(n) + g(n) = \theta(\min\{f(n), g(n)\})$ i)  $\max\{f(n), g(n)\} = \theta(f(n) + g(n))$ ۸- آیا تابع  $f(n) = \lceil \lg n \rceil!$  محدود به چند جمله‌ای است؟ یعنی آیا وجود دارد تابع  $n^a$ ثابت که رشد  $f(n)$  از  $n^a$  کمتر یا مساوی باشد؟

۹- آیا تابع  $f(n) = \lceil \lg \lg n \rceil$  محدود به چند جمله‌ای است؟

۱۰- گویند  $f(n) \in \tilde{O}(g(n))$  را بخوانید اُمگا بی‌نهایت) اگر به ازای حداقل یک  $c > 0$ ، نامساوی  $f(n) \geq c \cdot g(n)$  به ازای بی‌شمار  $n$  صحیح، درست باشد. آیا می‌توان ادعا کرد به ازای هر دو تابع  $f(n)$  و  $g(n)$  حداقل یکی از دو عبارت  $f(n) \in O(g(n))$  یا  $f(n) \in \tilde{O}(g(n))$  صحیح است؟

۱۱- آیا  $\tilde{O}$  متعدی است؟

۱۲- نشان دهید اگر  $k \cdot \ln k = \theta(n)$  آنگاه  $k = \theta\left(\frac{n}{\ln n}\right)$ .

۱۳- ترتیب رشد زیر را بررسی کنید.

$$\begin{aligned} \frac{1}{n^{\lg n}} &< \lg(\lg^* n) < \lg^* n \sim \lg^*(\lg n) < 2^{\lg^* n} < \ln \ln n < \sqrt{\lg n} < \ln n < \lg^2 n < 2^{\sqrt{\lg n}} \\ &< \sqrt{2}^{\lg n} = \sqrt{n} < 2^{\lg n} = n < n \lg n \sim \lg n! < 2^{\lg n} = n^2 < n^3 < (\lg n)! < (\lg n)^{\lg n} = n^{\lg \lg n} \\ &< \left(\frac{3}{2}\right)^n < 2^n < n \times 2^n < e^n < n! < (n+1)! < 2^{2^n} < 2^{2^{n+1}} \end{aligned}$$

منظور از علامت  $\sim$  در این مثال، یعنی هم رشد هستند.

۱۴- نشان دهید  $f(n) = \begin{cases} 2^{2^{n+2}}, & n \text{ زوج} \\ 0, & n \text{ فرد} \end{cases}$  از مرتبه  $\Omega, O$  هیچ یک توابع تمرین قبل نیست.

## پاسخ تمرین

$$a) \sum_{k=1}^{n-1} \frac{1}{k(k+1)} = \sum_{k=1}^{n-1} \frac{(k+1) - k}{k(k+1)} = \sum_{k=1}^{n-1} \left( \frac{1}{k} - \frac{1}{k+1} \right) \quad -1$$

$$= \left( \frac{1}{1} - \frac{1}{2} \right) + \left( \frac{1}{2} - \frac{1}{3} \right) + \left( \frac{1}{3} - \frac{1}{4} \right) + \dots + \left( \frac{1}{n-1} - \frac{1}{n} \right) = 1 - \frac{1}{n}$$

توجه:  $\sum_{k=0}^{n-1} (a_k - a_{k+1}) = a_0 - a_n$  به خاصیت تلسکوپی معروف است.

$$b) \sum_{k=1}^n (2k-1) = 2 \sum_{k=1}^n k - \sum_{k=1}^n 1 = 2 \frac{n(n+1)}{2} - n = n^2$$

**توجه:**  $\sum_{k=1}^n (ca_k + d.b_k) = c \sum_{k=1}^n a_k + d \sum_{k=1}^n b_k$  که  $c$  و  $d$  نسبت به  $k$  ثابت هستند. همچنین

$$\sum_{k=m}^n c = (n - m + 1)c \quad \text{که } c \text{ نسبت به } k \text{ ثابت است.}$$

$$\begin{aligned} \text{c) } x^r + x^{\delta} + x^{\gamma} + \dots &= \frac{x^r}{1-x^r} \xrightarrow{\text{مشتق}} rx^{r-1} + \delta x^{\delta-1} + \gamma x^{\gamma-1} + \dots = \frac{rx^{r-1}(1-x^r) - (-rx)x^{r-1}}{(1-x^r)^2} \\ \rightarrow \sum_{k=1}^{\infty} (rk+1)x^{rk} &= \frac{rx^{r-1} - x^{\delta}}{(1-x^r)^2} \end{aligned}$$

$$\text{d) } \sum_{k=0}^{\infty} \frac{k-1}{r^k} = \sum_{k=0}^{\infty} \frac{k}{r^k} - \sum_{k=0}^{\infty} \frac{1}{r^k} = r - \frac{1}{1-\frac{1}{r}} = r - r = 0.$$

**توجه:** در مثال ۲ متن درس دیدیم که  $\sum_{k=1}^{\infty} \frac{k}{r^k} = r$  که اگر از  $k=0$  شروع شود نیز همین می‌شود.

**(e)** علامت  $\Pi$  به معنی ضرب است پس:

$$\begin{aligned} \prod_{k=1}^n r \times r^k &= (r \times r^1)(r \times r^2)(r \times r^3) \dots (r \times r^n) = r^n r^{1+2+\dots+n} \\ &= r^n \times r^{\frac{n(n+1)}{2}} = r^n \times r^{n(n+1)} = r^{n^2+n} \end{aligned}$$

$$\begin{aligned} \text{f) } \prod_{k=2}^n \left(1 - \frac{1}{k^r}\right) &= \prod_{k=2}^n \left(\frac{k-1}{k}\right) \left(\frac{k+1}{k}\right) = \left(\frac{1}{2} \cdot \frac{3}{2}\right) \left(\frac{2}{3} \cdot \frac{4}{3}\right) \left(\frac{3}{4} \cdot \frac{5}{4}\right) \dots \left(\frac{n-1}{n} \cdot \frac{n+1}{n}\right) \\ &= \frac{1}{2} \cdot \frac{n+1}{n} = \frac{n+1}{2n} \end{aligned}$$

$$1 + x + x^2 + x^3 + \dots = \frac{1}{1-x} \xrightarrow{\text{مشتق}} 1 + 2x + 3x^2 + 4x^3 + \dots = \frac{1}{(1-x)^2} \quad -2$$

$$\xrightarrow{\times x} x + 2x^2 + 3x^3 + 4x^4 + \dots = \frac{x}{(1-x)^2}$$

$$\xrightarrow{\text{مشتق}} 1 + 2^2x + 3^2x^2 + 4^2x^3 + \dots = \frac{1+x}{(1-x)^3}$$

$$\xrightarrow{\times x} x + 2^2x^2 + 3^2x^3 + 4^2x^4 + \dots = \frac{x(1+x)}{(1-x)^3}$$

$$\sum_{k=1}^n \frac{1}{(2k-1)} \simeq \int_1^n \frac{1}{2k-1} dk = \frac{1}{2} \ln(2n-1) = \ln \sqrt{2n-1} \simeq \ln(\sqrt{n}) + C \quad -۳$$

۶- از تقریب استرلینگ برای  $n!$  استفاده می‌کنیم و از آن لگاریتم می‌گیریم:

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \theta\left(\frac{1}{n}\right)\right)$$

$$\rightarrow \lg n! = \lg \sqrt{2\pi n} + n \cdot \lg\left(\frac{n}{e}\right) + \lg\left(1 + \theta\left(\frac{1}{n}\right)\right) \simeq n \cdot \lg\left(\frac{n}{e}\right) = n \cdot \lg n - n \cdot \lg e \simeq n \cdot \lg n$$

پس  $\lg n!$  با  $n \cdot \lg n$  هم رشد می‌باشد.

(a - ۷) نادرست. مثلاً  $f(n) = 2n$  و  $g(n) = n$  را در نظر بگیرید.

(b) نادرست. مثلاً  $f(n) = \frac{1}{n^2}$  و  $g(n) = \frac{1}{n}$  را در نظر بگیرید.

(c) درست.

$$f(n) = O(g(n)) \Rightarrow f(n) \leq c g(n) \Rightarrow \lg(f(n)) \leq \lg c + \lg(g(n))$$

برای آنکه نشان دهیم این عبارت کمتر از  $b \cdot \lg(g(n))$  است (برای مقدار ثابتی مثل  $b$ )، فرض می‌کنیم  $\lg c + \lg(g(n)) = b \cdot \lg(g(n))$  حال طرفین را به  $\lg(g(n))$  تقسیم می‌کنیم:

$$b = \frac{\lg(c) + \lg(g(n))}{\lg(g(n))} = \frac{\lg(c)}{\lg(g(n))} + 1 \leq \lg c + 1$$

(d) اگر برای  $n$ های بزرگ  $f(n) < 1$  آنگاه نادرست و اگر  $f(n) > 1$  آنگاه درست. (مثلاً به

ازای  $f(n) = \frac{1}{n}$ ، نادرست است)

(e) نادرست. مثلاً  $f(n) = 4^n$  را در نظر بگیرید.

(f) درست. ( $O(f(n))$  یعنی توابعی که از  $f(n)$  رشد کمتر یا یکسان دارند)

(g) نادرست. به عنوان مثال  $g(n) = \begin{cases} n, & \text{زوج } n \\ 1, & \text{فرد } n \end{cases}$  می‌توان نشان داد  $g(n) \in O(n) - \Omega(n)$

ولی  $g(n) \notin o(n)$ . ولی می‌توان ثابت کرد اگر  $g(n) \in o(f(n))$  آنگاه  $g(n) \in O(f(n)) - \Omega(f(n))$  یعنی  $o(f(n)) \subseteq O(f(n)) - \Omega(f(n))$

(h) نادرست.

(i) درست. باید نشان دهیم اعداد مثبت  $n_0$  و  $C_1$  و  $C_2$  وجود دارند که

$$0 \leq C_1(f(n) + g(n)) \leq \max(f(n), g(n)) \leq C_2(f(n) + g(n))$$

برای همه  $n \geq n_0$ . کافیست  $C_2 = 1$  و  $C_1 = \frac{1}{2}$  انتخاب شوند.

۸- برای آنکه نشان دهیم  $f(n)$  محدود به چندجمله‌ای است باید نشان دهیم  $\lg(f(n))$  از  $\lg(n)$  رشد بیشتری ندارد:

$$f(n) = \lceil \lg n \rceil! \rightarrow \lg(f(n)) = \lg \lceil \lg n \rceil! \simeq \lceil \lg n \rceil \cdot \lg \lceil \lg n \rceil > \lg n$$

پس  $\lceil \lg n \rceil!$  محدود به چندجمله‌ای نیست یعنی رشدش از همه چندجمله‌ای‌ها بیشتر است. (نشان دهید رشد این تابع از نمایی‌ها کمتر است)

$$f(n) = \lceil \lg \lg n \rceil! \rightarrow \lg(f(n)) = \lg \lceil \lg \lg n \rceil! \simeq \lceil \lg \lg n \rceil \cdot \lg \lceil \lg \lg n \rceil < \lg n \quad 9-$$

پس  $\lceil \lg \lg n \rceil!$  محدود به چندجمله‌ای است.

۱۰- بله. مثلاً  $f(n) = n^{1+\sin n}$  و  $g(n) = n$  را در نظر بگیرید آنگاه  $f(n) \in \Omega(g(n))$  زیرا نامساوی  $n^{1+\sin n} \geq c \cdot n$  به ازای بی‌شمار  $n$  (نه همه  $n$  ها) صحیح است.

۱۱- خیر.  $\Omega$  متعددی نیست. مثلاً  $n \in \Omega(n^{1+\sin n})$  زیرا نامساوی  $n \geq c \cdot n^{1+\sin n}$  به ازای بی‌شمار  $n$  (در واقع به ازای همه  $n$  هایی که  $\sin n$  منفی است) برقرار است. و همچنین

$n^{1+\sin n} \in \Omega(n^{1/5})$  زیرا نامساوی  $n^{1+\sin n} \geq c \cdot n^{1/5}$  به ازای بی‌شمار  $n$  صحیح است. ولی  $n \notin \Omega(n^{1/5})$ .

۱۲- در عبارت  $k \cdot \ln k$  اگر بجای  $k$  قرار دهیم  $\frac{n}{\ln n}$  داریم:

$$k \cdot \ln k \simeq \frac{n}{\ln n} \cdot \ln \left( \frac{n}{\ln n} \right) = \frac{n}{\ln n} (\ln n - \ln \ln n) = n - \frac{n \cdot \ln \ln n}{\ln n} \simeq n = \theta(n)$$

## [سؤالهای چهارگزینه‌ای فصل اول]

(علوم کامپیوتر ۸۹)

-۱ کدام گزینه صحیح است؟ ( $0 < x < 1$ )

$$\frac{n}{\lg n} = O(n^{1-x}), 3^n = \Omega(n \cdot 2^n) \quad (۱)$$

$$n(\log_3 n)^\Delta = \Omega(n^{1/2}), \sqrt{n} = O((\lg n)^\Delta) \quad (۲)$$

$$\frac{n}{\lg n} = \Omega(n^{1-x}), 3^n = O(n^{2^n}) \quad (۳)$$

$$n(\log_3 n)^\Delta = O(n^{1/2}), \sqrt{n} = \Omega((\lg n)^\Delta) \quad (۴)$$

-۲ با فرض مثبت بودن توابع  $f$  و  $g$ ، تعداد گزاره‌های درست از میان گزاره‌های زیر چند است؟

(۸۸ IT)

$$f(n) = O(f(n)^2) \quad \text{گزاره ۱:} \quad \blacksquare$$

$$f(n) + o(f(n)) = \theta(f(n)) \quad \text{گزاره ۲:} \quad \blacksquare$$

$$f(n) = O(g(n)) \Rightarrow 2^{f(n)} = O(2^{g(n)}) \quad \text{گزاره ۳:} \quad \blacksquare$$

$$f(n) = \theta\left(f\left(\frac{n}{2}\right)\right) \quad \text{گزاره ۴:} \quad \blacksquare$$

(۱) یک گزاره صحیح است. (۲) دو گزاره صحیح است.

(۳) سه گزاره صحیح است. (۴) گزاره‌ای صحیح نیست.

(علوم کامپیوتر ۸۸)

-۳ برای  $k > 1$  و  $0 < \varepsilon < \frac{1}{2}$  کدام گزینه صحیح است؟

$$n^\varepsilon = O(\sqrt{n}) = O(\lg n!) = O((\lg n)^k) \quad (۱)$$

$$n^\varepsilon = O(\sqrt{n}) = O((\lg n)^k) = O(\lg n!) \quad (۲)$$

$$(\lg n)^k = O(n^\varepsilon) = O(\lg n!) = O(\sqrt{n}) \quad (۳)$$

$$(\lg n)^k = O(n^\varepsilon) = O(\sqrt{n}) = O(\lg n!) \quad (۴)$$

(علوم کامپیوتر - ۸۵)

-۴ در رشد توابع زیر کدام ترتیب صحیح است؟ (چپ به راست)

$$O(1 + \varepsilon)^n, O(n \log n), O\left(\frac{n^2}{\log n}\right) \quad (۲) \quad O(n \log n), O(1 + \varepsilon)^n, O\left(\frac{n^2}{\log n}\right) \quad (۱)$$

$$O(n \log n), O\left(\frac{n^2}{\log n}\right), O(1 + \varepsilon)^n \quad (۴) \quad O\left(\frac{n^2}{\log n}\right), O(n \log n), O(1 + \varepsilon)^n \quad (۳)$$

۵- کدام یک از گزینه‌های زیر صحیح است؟ (۸۴ IT)

$$\begin{array}{ll} n^{\sqrt{n}} \sin n \in O(n) & (۲) \\ n^{\sqrt{n}} \sin n \in \Omega(n) & (۱) \\ n \in \Omega(n^{\sqrt{n}} \sin n) & (۴) \\ n \in \theta(n^{\sqrt{n}} \sin n) & (۳) \end{array}$$

۶- کدام عبارت صحیح است؟ (علوم کامپیوتر - ۸۲)

$$\begin{array}{ll} (n+1)(n^{\sqrt{n}} - 2n+1) \in \theta(n) & (۲) \\ (n+1)(n^{\sqrt{n}} - 2n+1) \in O(2^n) & (۱) \\ (n+1)(n^{\sqrt{n}} - 2n+1) \in O(n^{\sqrt{n}} \log n) & (۴) \\ (n+1)(n^{\sqrt{n}} - 2n+1) \in \Omega(n^4) & (۳) \end{array}$$

۷- کدام گزینه صحیح است؟ (علوم کامپیوتر ۸۲)

$$\begin{array}{ll} n^a \neq O((\log n)^b) \text{ } a > b \text{ iff } & (۲) \\ n^a = O((\log n)^b) \text{ } a > b \text{ iff } & (۱) \\ n^a \neq O((\log n)^b), \forall b, a > 0 & (۴) \\ n^a = O((\log n)^b), \forall b, a > 0 & (۳) \end{array}$$

۸- گزینه صحیح انتخاب کنید؟ (علوم کامپیوتر - ۸۰)

$$\begin{array}{ll} \log n! \in O(\log n) & (۲) \\ n^{\frac{1}{10}} \in \Omega(\log n) & (۱) \\ 1^n + 2^n + \dots + n^n \in O(n^n) & (۴) \\ \lambda n^{\sqrt{n}} + 3n - 4 \in O(n \log n) & (۳) \end{array}$$

۹- کدام یک از موارد زیر نادرست است؟ (آزاد ۸۶)

$$\begin{array}{ll} 6 \times 2^n + n^{\sqrt{n}} = O(2^n) & (۲) \\ 6 \times 2^n + n^{\sqrt{n}} = \theta(n^{\sqrt{n}}) & (۱) \\ 6 \times 2^n + n^{\sqrt{n}} = \Omega(2^n) & (۴) \\ 6 \times 2^n + n^{\sqrt{n}} = \Omega(2^{\sqrt{n}}) & (۳) \end{array}$$

۱۰- کدام تساوی درست است؟ (تخصصی نرم افزار مهندسی کامپیوتر - آزاد ۸۵)

$$\begin{array}{ll} \frac{6n^{\sqrt{n}}}{(\log n + 1)} = O(n^{\sqrt{n}}) & (۲) \\ n^{\sqrt{n}} \log n = \theta(n^{\sqrt{n}}) & (۱) \\ n^{\sqrt{n}} 2^n + 6n^{\sqrt{n}} 3^n = O(n^{\sqrt{n}} 2^n) & (۴) \\ \frac{n^{\sqrt{n}}}{\log n} = \theta(n^{\sqrt{n}}) & (۳) \end{array}$$

۱۱- کدام یک از موارد زیر صحیح است؟ (آزاد ۸۳)

$$\begin{array}{ll} \frac{n^{\sqrt{n}}}{\log n} = \theta(n^{\sqrt{n}}) & (ب) \\ n! = O(n^n) & (الف) \\ n^{\sqrt{n}} + 6(2^n) = \theta(n^{2^n}) & (د) \\ n^{\sqrt{n}} \log n = \theta(n^{\sqrt{n}}) & (ج) \end{array}$$

(۱) الف و ج (۲) ب و ج (۳) الف و د (۴) همه موارد

۱۲- کدام یک از عبارت زیر درست‌اند:

(ساختمان داده‌ها - دولتی ۸۵)

$$I. e^{c\sqrt{n}} = O(e^{\sqrt{n}}) \quad c \geq 1$$

$$II. n^2 = O(n \log n)$$

$$III. n = O(n \log n)$$

(۱) فقط I (۲) فقط III (۳) I و II (۴) I و III

۱۳- کدام یک ترتیب صحیح از کوچکترین نرخ رشد تا بزرگترین نرخ رشد است؟

(گسسته - دولتی ۸۴)

$$(1) x \log x, 16x^2, (3x)^{3/2}, x^{3/2} \log \sqrt{x}$$

$$(2) x \log x, 16x^2, x^{3/2} \log \sqrt{x}, (3x)^{3/2}$$

$$(3) 16x^2, x \log x, (3x)^{3/2}, x^{3/2} \log \sqrt{x}$$

$$(4) 16x^2, x \log x, x^{3/2} \log \sqrt{x}, (3x)^{3/2}$$

۱۴- کدام گزینه غلط است؟

(مبانی نرم افزار - آزاد ۷۹)

$$(1) \text{ if } f(n) = O(g(n)) \text{ and } f(n) = \theta(g(n)) \text{ then } f(n) = \Omega(g(n))$$

$$(2) \text{ if } f(n) = \Omega(g(n)) \text{ and } f(n) = \theta(g(n)) \text{ then } f(n) = O(g(n))$$

$$(3) \text{ if } f(n) = \Omega(g(n)) \text{ and } f(n) = O(g(n)) \text{ then } f(n) = \theta(g(n))$$

$$(4) \text{ if } f(n) = O(g(n)) \text{ and } g(n) = \Omega(f(n)) \text{ then } f(n) = \theta(g(n))$$

۱۵- فرض کنید  $f(n) = O(g(n))$ ، کدام گزینه برای هر تابع با مقادیر مثبت مانند  $f(n)$  و

(تخصص هوش مصنوعی دولتی ۸۴)

$g(n)$  برقرار است؟

$$(1) g(n) = \theta(f(n)) \quad (2) f^{f(n)} = O(f^{g(n)})$$

$$(3) f^{f(n)} = \Omega(f^{g(n)}) \quad (4) \log(g(n)) = O(\log(f(n))) \text{ با فرض } \log(g(n)) \geq 1$$

۱۶- با توجه به تعاریف علامت  $O$  و  $\Omega$  و  $\theta$  کدام گزینه غلط است؟

(مبانی نرم افزار آزاد ۷۸ و ۸۰)

$$(1) O\left(\sum_{i=1}^m f_i(n)\right) = O(\max_{i=1}^m \{f_i\})$$

$$(2) \text{ if } f(n) = O(g(n)) \text{ and } g(n) = \Omega(f(n)) \text{ then } f(n) = \theta(g(n))$$

$$(3) \text{ I. } g(n) f(n) = \theta(f(n)) \text{ به شرط اینکه } g(n) \leq c \text{ برای } n \geq n_0$$

$$\text{II. } g(n) f(n) = \Omega(f(n)) \text{ به شرط اینکه } g(n) \geq c \text{ برای } n \geq n_0$$

$$(4) \text{ I. } O(O(f(n))) = O(f(n))$$

$$\text{II. } O(f(n) + g(n)) = O(f(n)) + O(g(n))$$



۱۷- آرایه‌ی  $A$  حاوی  $n$  بیت است. فرض کنید که  $A$  یکی از دو گونه‌ی زیر است: گونه‌ی ۱: نیمی از درایه‌های  $A$  حاوی بیت صفر و نیم دیگر حاوی بیت ۱ (بدون ترتیب خاصی) است. گونه‌ی ۲:  $A$  در مجموع  $\frac{2n}{3}$  صفر و  $\frac{n}{3}$  یک دارد. (تخصصی هوش مصنوعی ۹۲)

به شما یک آرایه‌ی  $A$  داده شده است که با احتمال یکسان یا از گونه‌ی ۱ است یا ۲. در بدترین حالت دست کم چند تا از درایه‌های  $A$  را باید بررسی کنید تا گونه‌ی آن را با اطمینان تشخیص دهید؟

$$(۱) \frac{2n}{3} \quad (۲) \frac{2n}{3} + ۱ \quad (۳) \frac{5n}{6} \quad (۴) \frac{5n}{6} + ۱$$

۱۸- کدام عبارت صحیح است؟ (علوم کامپیوتر - ۹۴)

$$(۱) O((\log n)!) < O(n) < O(\log n!) \quad (۲) O(n) < O(\log n!) < O((\log n)!) \\ (۳) O(\log n!) < O((\log n)!) < O(n) \quad (۴) O(\log n!) < O(n) < O((\log n)!)$$

۱۹- چند تا از گزاره‌های زیر درست‌اند؟ (IT - ۹۵)

$$\bullet f(n) = \Theta(n) \wedge g(n) = \Omega(n) \Rightarrow f(n)g(n) = \Omega(n^2) \\ \bullet f(n) = \Theta(1) \Rightarrow n^{f(n)} = O(n) \\ \bullet f(n) = \Omega(n) \wedge g(n) = O(n^2) \Rightarrow g(n) / f(n) = O(n) \\ \bullet f(n) = O(n^2) \wedge g(n) = O(n) \Rightarrow f(g(n)) = O(n^3) \\ \bullet f(n) = O(\log n) \Rightarrow 2^{f(n)} = O(n) \\ (۱) ۱ \quad (۲) ۲ \quad (۳) ۳ \quad (۴) ۴$$

۲۰- اگر زمان اجرای یک الگوریتم با رابطه‌ی  $T(n) = T(\log n) + O(n)$  تعریف شود، کدام

گزینه مرتبه‌ی زمانی این الگوریتم را نشان می‌دهد؟ (IT - ۹۵)

$$(۱) O(n \log n) \quad (۲) O(n \log^* n) \quad (۳) O(n) \quad (۴) O(\log n) \\ \text{(تعریف: } \log^* n = \min\{i > 0 : \log^{(i)} n \leq 1\} \text{ و } (\log^{(i)} n = \log(\log^{(i-1)} n)) \text{)}$$

۲۱- کدام یک از موارد زیر درست است؟ (علوم کامپیوتر - ۹۵)

$$(۱) n \in \Omega((\log n)^{\log n}) \quad (۲) n \in O((\log n)^{\log n}) \\ (۳) \log n! \in O(n) \quad (۴) n \in \Omega((\log n)!)$$

## پاسخنامه سؤال‌های چهارگزینه‌ای فصل اول

- ۱- گزینه (۴). برای مقایسه  $n^{1-x}$  و  $\frac{n}{\lg n}$ ، می‌دانیم  $n^x > \lg n$  حال طرفین را به  $n^x$  تقسیم کنید پس  $n^{-x} \lg n > 1$  حال در  $n$  ضرب کنید پس  $n > n^{1-x} \lg n$  حال به  $\lg n$  تقسیم کنید، پس  $\frac{n}{\lg n} > n^{1-x}$ . بنابراین  $\frac{n}{\lg n} = \Omega(n^{1-x})$
- برای مقایسه  $n^{2^n}, 3^n, n^{2^n}$ ، می‌دانیم  $\left(\frac{3}{2}\right)^n > n$  حال طرفین را در  $2^n$  ضرب کنید پس  $3^n > n \cdot 2^n$  بنابراین  $3^n = \Omega(n \cdot 2^n)$
- برای مقایسه  $n^{1/2}$  و  $n \cdot (\log_3 n)^\Delta$  می‌دانیم  $n^{1/2} > (\log_3 n)^\Delta$  حال طرفین را در  $n$  ضرب کنید پس  $n^{3/2} > n \cdot (\log_3 n)^\Delta$  پس  $n \cdot (\log_3 n)^\Delta = O(n^{3/2})$
- برای مقایسه  $\sqrt{n}$  و  $(\lg n)^\Delta$ ، می‌دانیم  $n^a$  رشد بیشتری از  $(\lg n)^b$  و  $a$  و  $b$  ثابت و  $a > 0$  دارد، پس  $\sqrt{n} = \Omega((\lg n)^\Delta)$
- ۲- گزینه (۱). گزاره ۱ برای تابعی که  $0 < f(n) < 1$  نادرست است. مثلاً اگر  $f(n) = \frac{1}{n}$  آنگاه  $\frac{1}{n} = O\left(\frac{1}{n^2}\right)$  غلط است زیرا  $\frac{1}{n}$  رشد بیشتری از  $\frac{1}{n^2}$  دارد. (با کمک حد می‌توان اثبات کرد.)
- گزاره ۲ صحیح است.  $o(f(n))$  یعنی توابع با رشد کمتر از  $f(n)$ . پس اگر  $f(n)$  را با تابعی که رشد کمتری از  $f(n)$  دارد جمع بزنیم، طبق خاصیت ماکزیمم‌گیری همان  $\theta(f(n))$  می‌شود. مثلاً  $n^2 + n = \theta(n^2)$
- گزاره ۳ نادرست است. مثلاً  $f(n) = 2n$  و  $g(n) = n$  فرض کنید.  $2n = O(n)$  صحیح است ولی  $2^{2n} = O(2^n)$  غلط است زیرا  $2^{2n} = 4^n$  رشد بیشتری از  $2^n$  دارد.
- گزاره ۴ نادرست است. مثلاً  $f(n) = 4^n$  فرض کنید،  $4^n = \theta(4^{n/2})$  غلط است. زیرا  $4^n$  رشد بیشتری از  $4^{n/2} = 2^n$  دارد.
- ۳- گزینه (۴). می‌دانیم  $n^a < n^b$  اگر  $a < b$  پس  $n^\epsilon$  که  $\frac{1}{p} < \epsilon$  رشد کمتری از  $\sqrt{n} = n^{1/2}$  دارد، بنابراین  $n^\epsilon = O(\sqrt{n})$  می‌دانیم  $\lg n!$  با  $n \cdot \lg n$  هم رشد است. و می‌دانیم  $n^\epsilon$  از  $(\lg n)^k$ ، که  $k$  ثابت است، رشد بیشتری دارد. پس ترتیب رشد توابع در این تست به شکل  $(\lg n)^k < n^\epsilon < \sqrt{n} < \lg n!$  است. لازم به ذکر است که طریقه نوشتن گزینه‌ها نادرست

است. یعنی مثلاً  $O(n^\epsilon) = O(\sqrt{n})$  نادرست است. زیرا به این معنی است که مجموعه  $O(n^\epsilon)$  با مجموعه  $O(\sqrt{n})$  برابر است که چنین نیست! ولی از بی‌دقتی طراح، چشم پوشی می‌کنیم.

۴- گزینه (۴). باید فرض کنیم  $\epsilon > 0$ .  $(1+\epsilon)^n$  تابع نمایی است که رشد بیشتری از سایر توابع در این تست دارد. می‌دانیم  $n > \lg^2 n$  حال طرفین را در  $n$  ضرب کنید و به  $\lg n$  تقسیم کنید پس  $\frac{n^2}{\lg n} > n \cdot \lg n$  بنابراین ترتیب رشد توابع در این تست  $n \lg n < \frac{n^2}{\lg n} < (1+\epsilon)^n$  است.

۵- گزینه (۱). با توجه به اینکه توابع در این درس باید مثبت باشند پس در گزینه‌ها  $\sin n$  باید داخل قدر مطلق باشد. که از این بی‌دقتی طراح چشم‌پوشی می‌کنیم. یادمان هست که دامنه  $n$  در این درس اعداد طبیعی  $\mathbb{N} = \{0, 1, 2, 3, \dots\}$  فرض می‌شود. با این دامنه،  $\sin n$  هیچگاه صفر نمی‌شود و  $|\sin n|$  همانند یک عدد ثابت با مقدار کمتر از ۱ می‌باشد. پس  $n^2 |\sin n|$  رشد بیشتری از  $n$  دارد. اگر فرض کنیم  $n$  عدد حقیقی است (در برخی منابع چنین است) آنگاه  $\sin n$  در نقاط  $n = k\pi$  صفر می‌شود پس  $n^2 |\sin n| \in \Omega(n)$  درست نخواهد بود چون نامساوی  $n^2 |\sin n| \geq c \cdot n$  بیشمار بار نقض می‌شود. (در نقاط  $n = k\pi$  که  $\sin n$  صفر می‌شود).

۶- گزینه (۱). تابع  $(n+1)(n^2 - 2n + 1)$  با  $n^3$  هم رشد است که رشد کمتری از  $2^n$  و  $n^4$  دارد و رشد بیشتری از  $n$  و  $n^2 \lg n$  دارد. پس گزینه ۱ صحیح است.

۷- گزینه (۴). به ازای هر  $a$  و  $b$  ثابت که  $a > 0$  تابع  $n^a$  رشد بیشتری از  $(\lg n)^b$  دارد. پس  $(\lg n)^b = O(n^a)$  یا  $n^a = \Omega((\lg n)^b)$  به ازای هر  $a, b$  ثابت که  $a > 0$  صحیح است. در گزینه ۲ گفته شده که  $n^a \neq O((\lg n)^b)$  اگر و فقط اگر  $a > b$  (iff)، که غلط است چون  $n^a \neq O((\lg n)^b)$  به ازای هر  $a, b > 0$ . بنابراین گزینه ۴ صحیح است.

۸- گزینه (۱). رشد  $n^{\frac{1}{e}}$  از  $\lg n$  بیشتر است پس گزینه ۱ صحیح است. رشد  $\lg n!$  با  $n \cdot \lg n$  یکسان و بیشتر از  $\lg n$  است پس گزینه ۲ غلط است. رشد  $4n - 3n^2 + 8n^2$  با  $n^2$  یکسان و بیشتر از  $n \cdot \lg n$  است که گزینه ۳ غلط است. رشد  $n^n + 2^n + \dots + 1^n$  با  $n^{n+1}$  یکسان و بیشتر از  $n^n$  است که گزینه ۴ غلط است.

۹- گزینه (۱). تابع  $n^2 + 2^n * 6$  هم رشد با  $2^n$  است که از  $n^2$  بیشتر است. پس گزینه‌های ۲ و ۳ و ۴ صحیح هستند.

- ۱۰- گزینه (۲).  $n^2 \lg n$  رشد بیشتری از  $n^2$  دارد پس گزینه ۱ غلط است. با توجه به اینکه  $n^2 \lg n > n^2$  اگر طرفین را به  $\lg n$  تقسیم کنید  $n^2 > \frac{n^2}{\lg n}$  پس گزینه ۳ غلط است. تابع  $n^2 \lg n + 6n^2 \cdot 3^n$  با  $n^2 \cdot 3^n$  هم رشد است و رشد  $n^2 \cdot 3^n$  از  $n^3 \cdot 2^n$  بیشتر است، پس گزینه ۴ غلط است. رشد  $\frac{6n^3}{\lg n + 1}$  از  $n^3$  کمتر است پس گزینه ۲ صحیح است.
- ۱۱- گزینه (۳). الف) رشد  $n!$  از  $n^n$  کمتر است پس  $n! = O(n^n)$  درست است.  
 ب) رشد  $\frac{n^2}{\lg n}$  از رشد  $n^2$  کمتر است پس  $\frac{n^2}{\lg n} = \theta(n^2)$  غلط است.  
 ج) رشد  $n^2 \lg n$  از رشد  $n^2$  بیشتر است پس  $n^2 \lg n = \theta(n^2)$  غلط است.  
 د) رشد  $n^{2^n} + 6(2^n) = \theta(n^{2^n})$  با  $n^{2^n}$  یکسان است پس  $n^{2^n} + 6(2^n) = \theta(n^{2^n})$  درست است.
- ۱۲- گزینه (۲). I. رشد  $e^{c\sqrt{n}}$  از  $e^{\sqrt{n}}$  بیشتر است چون  $e^{c\sqrt{n}} = (e^{\sqrt{n}})^c$  که از  $e^{\sqrt{n}}$  بیشتر است. پس  $e^{c\sqrt{n}} = O(e^{\sqrt{n}})$  غلط است.  
 II. رشد  $n^2$  از  $n \lg n$  بیشتر است پس  $n^2 = O(n \lg n)$  غلط است.  
 III. رشد  $n$  از  $n \lg n$  کمتر است پس  $n = O(n \lg n)$  درست است.
- ۱۳- گزینه (۱). توجه کنید  $\lg \sqrt{x} = \frac{1}{2} \lg x$  که با  $\lg x$  هم رشد است. با توجه به اینکه  $\lg x < x$  طرفین را در  $x$  ضرب کنید پس  $x \lg x < x^2$ . واضح است که  $x^2 < x^{3/2}$  و  $x^{3/2} \lg x < x^{3/2} \lg \sqrt{x} < x^{3/2} \lg (3x) < 16x^2 < x \lg x$  پس ترتیب رشد توابع  $x^{3/2} \lg \sqrt{x}$  است.
- ۱۴- گزینه (۴). در گزینه ۱، از  $f(n) \leq g(n)$  و  $f(n) \approx g(n)$  می‌توان نتیجه گرفت  $f(n) \geq g(n)$  پس درست است. (از  $f(n) \approx g(n)$  به تنهایی می‌توان نتیجه گرفت  $f(n) \geq g(n)$ )  
 در گزینه ۲، از  $f(n) \geq g(n)$  و  $f(n) \approx g(n)$  می‌توان نتیجه گرفت  $f(n) \leq g(n)$  پس درست است.  
 در گزینه ۳، از  $f(n) \geq g(n)$  و  $f(n) \leq g(n)$  نتیجه می‌شود  $f(n) \approx g(n)$  پس درست است.  
 در گزینه ۴، از  $f(n) \leq g(n)$  و  $g(n) \geq f(n)$  نتیجه نمی‌شود  $f(n) \approx g(n)$  پس غلط است.  
 توجه کنید منظور از  $f(n) \leq g(n)$  یعنی رشد  $f(n)$  کمتر یا مساوی رشد  $g(n)$  است.

۱۵- گزینه (۴). هر چهار گزینه اشتباه هستند و تست غلط است. متأسفانه هر سال تست‌های غلط در کنکور طرح می‌شوند که برخی حذف می‌شوند. با توجه به اینکه  $f(n) = O(g(n))$  پس  $f(n) \leq g(n)$  (منظور رشد است) که نتیجه نمی‌شود  $g(n) \approx f(n)$  پس گزینه ۱ غلط است. برای نقض گزینه ۲ فرض کنید  $f(n) = 2n$  و  $g(n) = n$ . برای نقض گزینه ۳ فرض کنید  $f(n) = n$  و  $g(n) = 2^n$ . برای نقض گزینه ۴ فرض کنید  $f(n) = n$  و  $g(n) = \log(f(n)) = \lg n$  و  $\log(g(n)) = n$  ولی  $f(n) = O(g(n))$  درست است ولی  $n \neq O(\lg n)$ .

گزینه ۴ اگر به صورت  $\log(f(n)) = O(\log(g(n)))$  می‌بود، درست بود.  
 ۱۶- گزینه (۴). گزینه ۱ در صورتی درست است که  $m$  ثابت باشد ولی اگر مثلاً فرض کنیم  $m = n$  و تابع  $f_i(n) = i$  آنگاه:

$$O\left(\sum_{i=1}^m f_i(n)\right) = O\left(\sum_{i=1}^n i\right) = O\left(\frac{n(n+1)}{2}\right) = O(n^2)$$

$$O(\max_{i=1}^m \{f_i\}) = O(\max_{i=1}^n \{i\}) = O(\max\{1, 2, \dots, n\}) = O(n)$$

پس گزینه ۱ ممکن است غلط باشد.

در گزینه ۲، از  $f(n) \leq g(n)$  و  $g(n) \geq f(n)$  نتیجه نمی‌شود  $f(n) \approx g(n)$  پس گزینه ۲ غلط است.

در گزینه ۳، قسمت I، گفته شده  $g(n) \leq c$ ، اگر  $g(n)$  را تابع ثابت فرض کنیم آنگاه درست است ولی می‌توان مثلاً  $g(n) = \frac{1}{n}$  فرض کرد آنگاه  $f(n) = \theta(g(n))$  غلط است مثلاً اگر  $f(n) = n^2$  باشد  $f(n) = \frac{1}{n} \cdot n^2 = n$  که از مرتبه  $\theta(n^2)$  نیست. در قسمت II،  $g(n) \geq c$  پس  $g(n) \cdot f(n) \geq f(n)$  درست است.

در گزینه ۴ هر دو جمله I و II صحیح هستند.

کلید مورد نظر طراح در این تست گزینه ۲ می‌باشد.

۱۷- گزینه (۴). بدترین حالت وقتی است که  $\frac{n}{3}$  درایه اول که خوانده می‌شود صفر باشد،  $\frac{n}{3}$  درایه بعدی ۱ باشد، تاکنون نوع آرایه مشخص نشده است، ولی اگر یک بیت دیگر خوانده شود نوع آرایه معلوم می‌شود. پس حداکثر باید  $1 + \frac{\Delta n}{6} + \frac{n}{3} + \frac{n}{3}$  درایه خوانده شود.

۱۸- گزینه (۲). می‌دانیم  $\lg n! \sim n \cdot \lg n$  و همچنین رشد  $(\lg n)!$  از رشد چند جمله‌ای‌ها بیشتر و از رشد نمایی‌ها کمتر است.

۱۹- گزینه (۲). فقط گزاره‌های اول و سوم صحیح هستند.

۲۰- گزینه (۳).

$$T(n) = T(\lg n) + O(n) = \Omega(n)$$

$$T(n) \leq T\left(\frac{n}{2}\right) + O(n) = \theta(n) \Rightarrow T(n) = O(n)$$

پس  $T(n) = \theta(n)$ .

۲۱- گزینه (۲). رشد  $(\lg n)^{\lg n}$  از  $n$  بیشتر است. رشد  $\lg(n!)$  با  $n \cdot \lg n$  یکی است که از  $n$  بیشتر است. رشد  $(\lg n)!$  از همه چندجمله‌ای‌ها بیشتر است.



## فصل ۲

### تحلیل الگوریتم‌های غیربازگشتی – آنالیز استهلاکی

#### تحلیل الگوریتم‌های غیربازگشتی

در این بخش می‌خواهیم نحوه محاسبه زمان اجرای الگوریتم‌های غیر بازگشتی را بررسی کنیم. زمان اجرا به عوامل مختلفی بستگی دارد از جمله: سخت افزاری که استفاده می‌کنیم، زبان برنامه‌نویسی که الگوریتم را با آن پیاده‌سازی می‌کنیم، اندازه ورودی، ترکیب داده‌های ورودی و .... هدف ما این است که زمان اجرا را بر حسب تعداد باری که دستورات اجرا می‌شوند محاسبه کنیم. به طور پیش فرض همه دستورات را از نظر زمان اجرا مثل هم فرض می‌کنیم (مگر اینکه خلاف آن گفته شود) و زمان هر بار اجرای یک دستور را ۱ فرض می‌کنیم بنابراین زمان اجرا را برابر با تعداد اجرای همه دستورات تعریف می‌کنیم. مثلاً اگر دستورات یک الگوریتم جمعاً ۱۰۰۰ بار اجرا شوند، زمان آن الگوریتم برابر ۱۰۰۰ است.

لازم به ذکر است که ما برای نوشتن کد الگوریتم‌ها از زبان خاصی استفاده نمی‌کنیم و شبه کد می‌نویسیم. گاهی ممکن است از زبان C استفاده کنیم. از علامت = یا ← یا := برای انتساب استفاده می‌کنیم از علامت == یا = برای مقایسه تساوی کمک می‌گیریم. برای نوشتن توضیحات (comment) همانند زبان C از علامت // استفاده می‌کنیم. برای مشخص کردن دستورات یک بلاک مثلاً دستورات یک حلقه از {} استفاده می‌کنیم و یا ممکن است در برخی تست‌ها فقط از فرورفتگی استفاده شود و علامت خاصی بکار نرود.



**مثال ۱:** در تکه برنامه‌های زیر تعداد اجرای همه خطوط را (یعنی زمان اجرا را) بیابید.

- |                     |                     |
|---------------------|---------------------|
| a) ۱. for(i=۱ to n) | b) ۱. for(i=m to n) |
| ۲. write(*)         | ۲. write(*)         |
| c) ۱. for(i=۱ to n) | d) ۱. for(i=۱ to n) |
| ۲. for(j=۱ to n)    | ۲. for(j=۱ to n)    |
| ۳. write(*)         | ۳. for(k=۱ to n)    |
|                     | ۴. write(*)         |

**پاسخ: a)** دستور ۲،  $n$  بار اجرا می‌شود و دستور ۱ یعنی حلقه for خودش  $n+1$  بار اجرا می‌شود. چون حلقه for، متغیر  $i$  را از ۱ تا  $n$  مقدار می‌دهد و write اجرا می‌شود. (تاکنون هر یک  $n$  بار اجرا شده‌اند) و در نهایت مقدار  $i$  را یک واحد دیگر افزایش می‌دهد و خارج می‌شود. پس خود حلقه از جمله داخلش، یک بار بیشتر اجرا شده است. پس زمان این کد  $2n+1$  است.

**b)** دستور ۲،  $n-m+1$  بار و خود دستور ۱، یک بار بیشتر یعنی  $n-m+2$  بار اجرا می‌شود پس این دو دستور جمعاً  $2(n-m)+3$  بار اجرا می‌شوند.

**c)** دستور ۱،  $n+1$  بار، دستور ۲،  $n(n+1)$  بار و دستور ۳،  $n^2$  بار اجرا می‌شوند پس جمعاً  $2n^2+2n+1$  بار اجرا می‌شوند.

**d)** دستور ۱ و ۲ و ۳ و ۴ به ترتیب  $n+1$ ،  $n(n+1)$ ،  $n^2(n+1)$  و  $n^3$  بار اجرا می‌شوند. پس جمع تعداد اجراها برابر  $2n^3+2n^2+2n+1$  است.

**توجه:** در بسیاری از تست‌ها، زمان اجرا به طور کامل مورد نظر نیست و فقط مرتبه زمان اجرا مطلوب است. مثلاً در قسمت (a) مثال ۱، زمان اجرا  $2n+1$  است که از مرتبه  $\theta(n)$  می‌باشد. یا قسمت (d) از مرتبه  $\theta(n^3)$  است. برای یافتن مرتبه زمان اجرا، لازم نیست تعداد اجرای همه دستورات محاسبه شود بلکه فقط تعداد اجرای جمله اصلی که در این مثال write است، کافی است.

**مثال ۲:** در تکه برنامه‌های زیر، دستور write چند بار اجرا می‌شود؟

- |                     |                     |
|---------------------|---------------------|
| a) ۱. for(i=۱ to n) | b) ۱. for(i=۱ to n) |
| ۲. for(j=۱ to i)    | ۲. for(j=۱ to i)    |
| ۳. write(*)         | ۳. for(k=۱ to j)    |
|                     | ۴. write(*)         |

**پاسخ: a)** حلقه دوم به حلقه اول وابسته است. بار اول، وقتی  $i=1$ ، در حلقه دوم  $j$  از ۱ تا ۱ تغییر می‌کند و write، یک بار اجرا می‌شود. در دور بعد مقدار  $i=2$  و در حلقه دوم،  $j$  از ۱ تا ۲ تغییر

می‌کند و write، دو بار اجرا می‌شود. به همین ترتیب در نهایت  $i = n$  و مقدار  $j$  از ۱ تا  $n$  تغییر می‌کند و write،  $n$  بار اجرا می‌شود پس تعداد بار اجرای دستور write برابر  $1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$  است. البته می‌توان با استفاده از سیگما نیز تعداد اجرای دستور write را یافت. کافی است بجای هر حلقه یک سیگما قرار دهید و بجای write، عدد ۱ قرار دهید:

$$\sum_{i=1}^n \sum_{j=1}^i 1 = \sum_{i=1}^n i = \frac{n(n+1)}{2}$$

(b) با کمک سیگما تعداد اجرای write را محاسبه می‌کنیم:

$$\begin{aligned} \sum_{i=1}^n \sum_{j=1}^i \sum_{k=1}^j 1 &= \sum_{i=1}^n \sum_{j=1}^i j = \sum_{i=1}^n \frac{i(i+1)}{2} = \frac{1}{2} \left[ \sum_{i=1}^n i^2 + \sum_{i=1}^n i \right] \\ &= \frac{1}{2} \left[ \frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2} \right] = \frac{n(n+1)(n+2)}{6} = \binom{n+2}{3} = \theta(n^3) \end{aligned}$$

جالب است بدانید که تعداد اجرای دستور write در قسمت (b) را با روش‌های شمارشی که در درس ساختمان گسسته می‌خوانید نیز می‌توان یافت. هر بار اجرای دستور write متناظر است با یک دسته  $(i, j, k)$  صحیح که  $1 \leq k \leq j \leq i \leq n$  و در واقع می‌خواهیم سه شی متمایز از ۱ تا  $n+2$  (یعنی  $n+2$  شی) انتخاب کنیم که  $\binom{n+2}{3}$  حالت دارد.

**نتیجه:** تعداد اجرای دستور write در  $k$  حلقه وابسته مقابل برابر است با  $\binom{n+k-1}{k}$  که از مرتبه  $\theta(n^k)$  است.

```
for( $i_1 = 1$  to  $n$ )
  for( $i_2 = 1$  to  $i_1$ )
    for( $i_3 = 1$  to  $i_2$ )
      :
      for( $i_k = 1$  to  $i_{k-1}$ )
        write(*)
```

**تست ۱:** در تکه برنامه زیر دستور write چند بار اجرا می‌شود؟

```
i = ۱
while(i < n)
{
    write(*)
    i = i * ۲
}
```

(۱)  $\lceil \lg n \rceil$  (۲)  $\lceil \lg n \rceil + ۱$  (۳)  $\lfloor \lg n \rfloor + ۱$  (۴)  $\lfloor \lg n \rfloor + ۱$

**پاسخ:** دستور write به ازای مقادیر  $i$  برابر  $۱, ۲, ۲^۲, ۲^۳, \dots, ۲^k$  که  $۲^k \leq n$  و  $۲^{k+1} > n$  اجرا می‌شود پس تعداد اجرای دستور write، حدوداً برابر  $\lg n$  است. برای یافتن تعداد بار دقیق اجرا، می‌توان به ازای  $n$ های مختلف بررسی کرد. مثلاً به ازای  $n = ۸$  تعداد اجرا ۳ بار است که یا گزینه ۱ یا ۲ صحیح هستند. به ازای  $n = ۹$  تعداد اجرا ۴ بار است که گزینه ۲ صحیح است.

**تست ۲:** در تکه برنامه زیر دستور write چند بار اجرا می‌شود؟

```
i = n
while(i > ۱)
{
    write(*)
    i = ⌊ i / ۲ ⌋
}
```

(۱)  $\lfloor \lg n \rfloor$  (۲)  $\lceil \lg n \rceil$  (۳)  $\lfloor \lg n \rfloor + ۱$  (۴)  $\lceil \lg n \rceil + ۱$

**پاسخ:** همانند تست قبل تعداد اجرا لگاریتمی است چون  $i$  هر بار نصف می‌شود. به ازای  $n = ۸$  دستور write ۳ بار و به ازای  $n = ۹$  نیز ۳ بار اجرا می‌شود. پس گزینه ۱ صحیح است.

**توجه:** در دو تست قبل اگر بجای  $i * ۲$  یا  $\frac{i}{۲}$ ، قرار دهیم  $i * k$  یا  $\frac{i}{k}$  که  $k$  ثابت است، آنگاه تعداد اجرا  $\log_k n$  می‌باشد که برای یافتن مقدار دقیق می‌توان برای  $n$  مقادیر مختلفی قرار داد.

**تست ۳:** زمان اجرای قطعه کد مقابل، از چه مرتبه‌ای است؟

```
i = ۲
while(i ≤ n)
{
    write(*)
    i = i * i
}
```

(۱)  $\theta(\lg n)$  (۲)  $\theta(\lg^۲ n)$  (۳)  $\theta(\sqrt{\lg n})$  (۴)  $\theta(\lg \lg n)$

**پاسخ:** دستور write به ازای مقادیر  $i = ۲, ۲^۲, ۲^۴, ۲^۸, \dots$  اجرا می‌شود در واقع مقادیر  $i$  که باعث اجرای دستور write می‌شوند به شکل  $۲^{۲^k} (۲^۲, ۲^{۲^۲}, ۲^{۲^۳}, \dots)$  هستند و تا وقتی write اجرا شود

که  $2^k \leq n$ . پس  $2^k \leq \lg n$  در نتیجه  $k \leq \lg \lg n$ . دقت کنید تعداد اجرای دستور write،  $k+1$  بار است که  $k$  حداکثر  $\lg \lg n$  است. پس گزینه ۴ صحیح است.

**مثال ۳:** زمان اجرای تکه برنامه‌های زیر، از چه مرتبه‌ای است؟

a) ۱. for ( $i=1; i \leq n; i++$ )

۲. for ( $j=1; j \leq n; j=j*2$ )

۳. write(\*)

b) ۱. for ( $i=1; i \leq n; i++$ )

۲. for ( $j=1; j \leq i; j=j*2$ )

۳. write(\*)

c) ۱. for ( $i=1; i \leq n; i=i*2$ )

۲. for ( $j=1; j \leq n; j++$ )

۳. write(\*)

d) ۱. for ( $i=1; i \leq n; i=i*2$ )

۲. for ( $j=1; j \leq i; j++$ )

۳. write(\*)

**پاسخ: a)** حلقه اول خطی و حلقه دوم لگاریتمی است و حلقه‌ها به هم وابسته نیستند. پس تعداد اجرای دستور write،  $n \lg n$  است. پس مرتبه  $\theta(n \lg n)$  است.

**b)** حلقه دوم به حلقه اول وابسته است ( $j \leq i$ ). حلقه دوم به ازای مقادیر  $j$  از ۱ تا  $i$  که  $j$  هر بار در ۲ ضرب می‌شود، اجرا می‌شود. پس تعداد اجرای این حلقه  $\lg i$  است. و چون  $i$  از ۱ تا  $n$  و به صورت خطی تغییر می‌کند، پس تعداد اجرای دستور write برابر  $\sum_{i=1}^n \lg i$  است که برابر است با

$$\lg 1 + \lg 2 + \dots + \lg n = \lg n!$$

**c)** همانند قسمت a، چون حلقه‌ها به هم وابسته نیستند، مرتبه  $\theta(n \lg n)$  است.

**d)** حلقه دوم به حلقه اول وابسته است پس نیاز به بررسی دارد. به ازای هر  $i$ ، حلقه دوم دقیقاً  $i$  بار اجرا می‌شود. مقادیر  $i$  عبارتند از  $1, 2, 2^2, 2^3, \dots, 2^k$  که  $2^k \simeq n$ . پس تعداد اجرای دستور write برابر است با:

$$1 + 2 + 2^2 + \dots + 2^k = 2^{k+1} - 1 = \theta(2^k) = \theta(n)$$

یعنی مرتبه اجرا بر خلاف ظاهر حلقه‌ها،  $\theta(n)$  یعنی خطی است!

**توجه:** در برخی الگوریتم‌ها زمان اجرا در حالات مختلف، متفاوت است. مثلاً در مرتب‌سازی  $n$  عنصر اگر آرایه از قبل مرتب باشد، ممکن است زمان کمتری نسبت به حالتی که آرایه مثلاً برعکس مرتب است، صرف شود. در تحلیل الگوریتم‌ها، سه حالت مطرح می‌شود که عبارتند از: بهترین حالت (Best case)، حالت متوسط (Average case) و بدترین حالت (Worst case). مثلاً فرض کنید می‌خواهیم  $x$  را در آرایه  $n$  عنصری  $c[1..n]$  جستجوی خطی کنیم، ابتدا  $x$  را با  $c[1]$  مقایسه می‌کنیم اگر مساوی نبود با  $c[2]$  و اگر مساوی نبودند با  $c[3]$  و به همین ترتیب. واضح

است که حداقل تعداد مقایسه ۱ است. پس بهترین حالت زمان اجرا ۱ است ( $B(n) = 1$ ). حداکثر تعداد مقایسه برابر  $n$  است. پس بدترین حالت زمان اجرا  $n$  است ( $W(n) = n$ ). تحلیل حالت متوسط معمولاً دشوار است و باید بین همه حالات، میانگین گرفته شود (امید ریاضی). در جستجوی خطی اگر فرض کنیم عدد  $x$  حتماً در آرایه وجود دارد آنگاه با احتمال  $\frac{1}{n}$ ، عدد  $x$  در  $c[i]$  ( $1 \leq i \leq n$ ) است بنابراین زمان اجرای حالت متوسط جستجوی خطی عبارت است از:

$$A(n) = 1 \times \frac{1}{n} + 2 \times \frac{1}{n} + \dots + n \times \frac{1}{n} = (1 + 2 + \dots + n) \times \frac{1}{n} = \frac{n+1}{2}$$

دقت کنید برای محاسبه  $A(n)$ ، همه حالات  $x$  را در نظر گرفته‌ایم: اگر  $x$  در  $c[1]$  باشد، ۱ مقایسه نیاز است و احتمال آن  $\frac{1}{n}$  است. اگر  $x$  در  $c[2]$  باشد، ۲ مقایسه نیاز است و احتمال آن  $\frac{1}{n}$  است، و به همین ترتیب اگر  $x$  در  $c[n]$  باشد،  $n$  مقایسه نیاز است و احتمال آن  $\frac{1}{n}$  است که با محاسبه  $A(n) = \frac{n+1}{2}$  بدست آمد. ما  $A(n)$  را با این فرض که  $x$  حتماً در آرایه هست یافتیم. حال فرض کنید  $x$  با احتمال  $P$  در آرایه  $c[1..n]$  است پس با احتمال  $\frac{P}{n}$  در مکان  $c[i]$  ( $1 \leq i \leq n$ ) است و با احتمال  $1-P$  در آرایه نیست، بنابراین زمان اجرای حالت متوسط برابر است با:

$$\begin{aligned} A(n) &= 1 \times \frac{P}{n} + 2 \times \frac{P}{n} + \dots + n \times \frac{P}{n} + n \times (1-P) \\ &= (1 + 2 + \dots + n) \times \frac{P}{n} + n(1-P) = \frac{(n+1) \cdot P}{2} + n(1-P) \end{aligned}$$

توجه کنید برای محاسبه  $A(n)$  همه حالت‌ها را در نظر گرفتیم، احتمال وجود  $x$  در  $A[i]$  برابر  $\frac{P}{n}$  است و تعداد  $i$  تا مقایسه ( $1 \leq i \leq n$ ) برای یافتن  $x$  نیاز است و همچنین احتمال نبود  $x$  در  $c[1..n]$  برابر  $1-P$  است و با  $n$  مقایسه مشخص می‌شود که  $x$  در آرایه نیست.

**تست ۴:** فرض کنید با احتمال ۵۰٪ عنصر  $x$  در  $c[1..n]$  باشد، به طور متوسط چه تعداد عنصر آرایه  $c$  برای یافتن  $x$  بررسی می‌شوند؟

$$\frac{1}{2} \quad (۱) \quad \frac{2}{3} \quad (۲) \quad \frac{3}{4} \quad (۳) \quad \frac{1}{4} \quad (۴)$$

**پاسخ:**  $P = 0.5$  است پس متوسط تعداد مقایسه برای یافتن  $x$  برابر است.

$$\frac{(n+1)P}{2} + n(1-P) = \frac{(n+1)}{4} + \frac{n}{2} = \frac{3n+1}{4}$$

یعنی به طور متوسط  $\frac{3}{4}$  عناصر باید بررسی شوند که گزینه ۳ صحیح است.

**مثال ۴:** الگوریتم زیر مرتب‌سازی درجی (insertion sort) است که در فصل مرتب‌سازی کتاب ساختمان داده‌ها با آن آشنا شدید. فرض کنید هر بار که خط  $\text{iam}$  اجرا می‌شود زمان  $c_i$  صرف می‌شود (برخلاف فرضیاتی که تاکنون داشتیم و زمان هر بار اجرای هر دستوری را ۱ واحد فرض می‌کردیم، در این مثال زمان اجراها را متفاوت فرض کرده‌ایم) و  $c_i$  ثابت است. همچنین فرض کنید به ازای هر  $j = 2, 3, \dots, n$  تعداد اجرای دستور ۴ (while) برابر  $t_j$  باشد. در ستون هزینه (cost) زمان هر بار اجرای دستورات نوشته شده است. در ستون تعداد بار (times) تعداد اجرای هر خط نوشته شده است حاصل ضرب  $\text{cost}$  در  $\text{times}$  زمان کل هر خط را نشان می‌دهد. می‌خواهیم زمان اجرا را بیابیم:

| تعداد بار (times)        | هزینه (cost) | INSERTIONSORT ( $A[1..n]$ )                      |
|--------------------------|--------------|--------------------------------------------------|
| $n$                      | $C_1$        | ۱. for ( $j = 2$ to $n$ )                        |
| $n - 1$                  | $C_2$        | ۲. {key = $A[j]$ }                               |
| $n - 1$                  | $C_3$        | ۳. $i = j - 1$                                   |
| $\sum_{j=2}^n t_j$       | $C_4$        | ۴. while ( $i > 0$ ) and ( $A[i] > \text{key}$ ) |
| $\sum_{j=2}^n (t_j - 1)$ | $C_5$        | ۵. { $A[i + 1] = A[i]$ }                         |
| $\sum_{j=2}^n (t_j - 1)$ | $C_6$        | ۶. $i = i - 1$ }                                 |
| $n - 1$                  | $C_7$        | ۷. $A[i + 1] = \text{key}$ }                     |

توجه کنید دستورات داخل حلقه while (دستورات ۵ و ۶) همیشه ۱ بار کمتر از خود حلقه while (دستور ۴) اجرا می‌شوند. برای محاسبه زمان اجرا یعنی  $T(n)$  باید هزینه‌ها را در تعداد بار ضرب کرده و مقادیر بدست آمده را جمع کنیم:

$$T(n) = c_1 n + c_2 (n - 1) + c_3 (n - 1) + c_4 \sum_{j=2}^n t_j + c_5 \sum_{j=2}^n (t_j - 1) + c_6 \sum_{j=2}^n (t_j - 1) + c_7 (n - 1)$$

حال می‌خواهیم زمان اجرای  $T(n)$  را در حالات مختلف بررسی کنیم. بهترین حالت وقتی است که آرایه  $A$  از قبل مرتب صعودی باشد آنگاه شرط  $A[i] > \text{key}$  در حلقه while همواره غلط است. و دستور while هر دور فقط یک بار اجرا می‌شود و دستورات داخل حلقه while یعنی دستورات ۵ و ۶ اصلاً اجرا نمی‌شوند، یعنی  $t_j = 1$  پس زمان اجرای بهترین حالت برابر است با:

$$B(n) = c_1n + c_2(n-1) + c_3(n-1) + c_4 \sum_{j=2}^n 1 + \dots + c_5(n-1)$$

$$= c_1n + c_2(n-1) + c_3(n-1) + c_4(n-1) + c_5(n-1)$$

$$= (c_1 + c_2 + c_3 + c_4 + c_5)n - (c_2 + c_3 + c_4 + c_5) = \theta(n)$$

بدترین حالت وقتی است که آرایه نزولی مرتب باشد، در این صورت هر بار  $A[j]$  با همه عناصر قبل خودش یعنی با عناصر  $A[1 \dots j-1]$  مقایسه می‌شود پس  $T_j = j$  بنابراین زمان اجرای بدترین حالت برابر است با:

$$W(n) = c_1n + c_2(n-1) + c_3(n-1) + c_4 \left( \frac{n(n+1)}{2} - 1 \right) + c_5 \frac{n(n-1)}{2} + c_6 \frac{n(n-1)}{2} + c_7(n-1) = \theta(n^2)$$

دقت کنید  $W(n)$  تابعی درجه ۲ بر حسب  $n$  است پس  $\theta(n^2)$  است. در حالت متوسط

می‌توان فرض کرد  $A[j]$  از نصف عناصر قبل خودش کوچکتر است که در این صورت  $T_j = \frac{j}{2}$  می‌شود و زمان اجرای حالت متوسط برابر است با:

$$A(n) = c_1n + c_2(n-1) + c_3(n-1) + \frac{1}{2}c_4 \left( \frac{n(n+1)}{2} - 1 \right) + \frac{1}{2}c_5 \left( \frac{n(n-1)}{2} \right) + \frac{1}{2}c_6 \left( \frac{n(n-1)}{2} \right) + c_7(n-1) = \theta(n^2)$$

### آنالیز استهلاکی (Amortized Analysis)

در این نوع آنالیز،  $n$  عمل روی یک ساختمان داده انجام می‌شود و هزینه زمانی این عملیات روی  $n$  عمل سرشکن می‌شود. در واقع ممکن است عملیات انجام شده هزینه‌های مختلفی داشته باشند و برخی هزینه زیادی داشته باشند ولی هزینه متوسط هر عمل کوچک باشد. لازم به ذکر است که آنالیز استهلاکی با آنالیز در حالت میانگین متفاوت است و روش‌های آماری را شامل نمی‌شود. در واقع آنالیز استهلاکی زمان اجرای متوسط هر عمل در بدترین حالت را نشان می‌دهد.

برای این نوع آنالیز ۳ روش مطرح است:

- آنالیز تجمعی (Aggregate Analysis)

- روش حسابرسی (Accounting method)

- روش پتانسیل (Potential Method)

## آنالیز تجمعی

در این روش مجموع هزینه  $n$  عمل را به دست می‌آوریم سپس به  $n$  تقسیم می‌کنیم. مثلاً  $n$  عمل  $push$ ,  $pop$  و  $Multipop(k)$  روی پشته  $S$  که در ابتدا خالی است انجام داده‌ایم ( $Multipop(k)$ ,  $k$  عنصر از پشته  $pop$  می‌کند، البته اگر کمتر از  $k$  عنصر در پشته باشند، همه عناصر  $pop$  می‌شوند) درست است که هزینه  $Multipop(k)$  ممکن است  $O(k)$  باشد ولی  $n$  عمل از عملیات فوق روی هم هزینه  $O(n)$  دارند پس هزینه هر عمل به صورت سرشکن شده برابر  $O(1)$  است.

**مثال ۵:** فرض کنید آرایه  $A[0..k-1]$  شامل یک عدد باینری است که  $A[0]$  بیت کم ارزش است

$$\text{بنابراین اگر } x \text{ مقدار عدد باشد: } x = \sum_{i=0}^{k-1} A[i] \cdot 2^i.$$

ابتدا  $x = 0$  یعنی عناصر  $A$  همگی صفر هستند، می‌خواهیم  $x$  را با عدد یک جمع کنیم

الگوریتم به صورت مقابل است:

```

INC(A)
{
  i = 0;
  while (i < k and A[i] == 1)
    {A[i] = 0; i = i + 1;}
  if (i < k) A[i] = 1;
}

```

می‌خواهیم هزینه زمانی  $n$  عمل  $INC$  را به دست آوریم.

هزینه هر تغییر بیت را یک فرض کنید.

با اولین عمل  $INC$ ، یک تغییر بیت داریم ( $A[0]$ ) با

دومین عمل، دو تغییر بیت ( $A[1]$ ,  $A[0]$ ) با سومین عمل،

یک تغییر بیت ( $A[0]$ ) و ...

ولی می‌توان بهتر به مسئله نگاه کرد. بیت  $A[0]$  با هر بار اجرای عمل  $INC$ ، تغییر می‌کند

$A[1]$ ،  $\left\lfloor \frac{n}{2} \right\rfloor$  بار تغییر می‌کند.  $A[2]$ ،  $\left\lfloor \frac{n}{4} \right\rfloor$  بار و به طور کلی  $A[i]$ ،  $\left\lfloor \frac{n}{2^i} \right\rfloor$  بار تغییر می‌کند که

$\left\lfloor \lg n \right\rfloor, 1, 0, \dots, i$ . دقت کنید برای  $i > \lg n$ ، بیت  $A[i]$  اصلاً تغییر نمی‌کند. بنابراین تعداد کل

تغییر بیت‌ها:

$$\sum_{i=0}^{\left\lfloor \lg n \right\rfloor} \left\lfloor \frac{n}{2^i} \right\rfloor < n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n$$

پس در بدترین حالت هزینه  $n$  عمل  $INC$ ،  $2n$  است. پس متوسط هزینه هر عمل یعنی هزینه

مستهلك شده به ازای هر عمل  $\frac{2n}{n} = 2 = O(1)$  است.

**تمرین:** نشان دهید اگر عمل  $DEC$  نیز وجود داشته باشد، آنگاه  $n$  عمل می‌توانند هزینه‌ای معادل

$\theta(nk)$  داشته باشند.



**مثال ۶:** دنباله‌ای از  $n$  عمل روی یک ساختمان داده اجرا شده است. هزینه عمل  $i$  ام برابر  $i$  است اگر  $i$  توانی از ۲ باشد و در غیر این صورت هزینه عمل برابر ۱ است. هزینه استهلاکی به ازای هر عمل را بیابید.

پاسخ:

|           |                        |   |   |   |   |   |   |   |   |    |     |
|-----------|------------------------|---|---|---|---|---|---|---|---|----|-----|
|           | هزینه عمل $i$ ام $C_i$ |   |   |   |   |   |   |   |   |    |     |
| operation | ۱                      | ۲ | ۳ | ۴ | ۵ | ۶ | ۷ | ۸ | ۹ | ۱۰ | ... |
| cost      | ۱                      | ۲ | ۱ | ۴ | ۱ | ۱ | ۱ | ۸ | ۱ | ۱  | ... |

$$\text{هزینه } n \text{ عمل} = \sum_{i=1}^n C_i \leq n + \sum_{j=0}^{\lg n} 2^j = n + (2n - 1) < 3n$$

$$\text{هزینه میانگین هر عمل} = \frac{\text{هزینه کل}}{\text{تعداد عملیات}} = \frac{3n}{n} = 3$$

## روش حسابرسی

در این روش به عملیات مختلف، شارژهای مختلفی نسبت می‌دهیم که ممکن است شارژی که نسبت می‌دهیم، از هزینه واقعی عمل کمتر یا بیشتر باشد. مقداری که یک عمل شارژ می‌شود را «هزینه استهلاک» آن عمل گویند.

اگر «هزینه استهلاک» یک عمل از «هزینه واقعی» آن عمل بیشتر باشد، تفاوت این دو به عنوان «اعتبار» به عناصر خاصی از ساختمان داده تخصیص می‌یابد. این اعتبار را می‌توان بعداً برای عملیاتی که هزینه استهلاکشان کمتر از هزینه واقعی‌شان است، صرف کرد.

به عنوان مثال عملیات پشته را در نظر بگیرید. می‌دانیم هزینه واقعی عملیات عبارت است از:

$$\text{push} = 1, \text{pop} = 1, \text{Multipop}(k) = \min(k, s)$$

که  $s$  تعداد عناصر موجود در پشته هنگام فراخوانی Multipop است.

ولی ما هزینه استهلاکی زیر را به عملیات نسبت می‌دهیم:

$$\text{push} = 2, \text{pop} = 0, \text{Multipop} = 0$$

با این مقادیر، می‌توان هر دنباله از عملیات را پشتیبانی کرد. هر  $\text{push}$  که انجام می‌شود، هزینه واقعی‌اش ۱ است، پس از شارژ ۲ مقداری آن، ۱ مقدار برای هزینه‌اش صرف می‌شود و ۱ مقدار در عنصر  $\text{push}$  شده به عنوان اعتبار ذخیره می‌شود، پس تمام عناصری که در پشته هستند، اعتبار ۱ مقداری دارند. حال اگر  $\text{pop}$  انجام شود، چون هزینه واقعی  $\text{pop}$ ، ۱ است، این ۱ را از اعتبار عنصری که  $\text{pop}$  می‌شود، دریافت می‌کند. پس از آنجایی که هزینه استهلاکی  $\text{push}$ ،  $\text{pop}$  و  $\text{Multipop}$  ثابت  $O(1)$  است پس هزینه  $n$  عمل،  $O(n)$  است.

به عنوان مثالی دیگر، عمل INC (جمع عدد  $k$  بیتی با ۱) را در نظر بگیرید. هزینه واقعی INC برابر تعداد بیت‌هایی است که عوض می‌شود.

حال فرض کنید هزینه استهلاکی تغییر صفر به یک را ۲ واحد در نظر بگیریم. و برای تغییر یک به صفر، هزینه استهلاکی صفر. وقتی بیتی ۱ می‌شود، ۱ واحد از شارژش را صرف هزینه واقعی‌اش می‌کنیم و ۱ واحد را در بیت ذخیره می‌کنیم، بنابراین هر ۱ در عدد دارای یک اعتبار ۱ واحدی است که می‌توان آن را وقتی صرف کرد که بیت مربوطه صفر شود. از آنجایی که هر عمل INC حداکثر یک بیت را ۱ می‌کند پس هزینه استهلاکی هر عمل INC برابر ۲ واحد است، پس هر عمل INC دارای هزینه استهلاکی  $O(1)$  است پس  $n$  عمل INC دارای هزینه استهلاکی  $O(n)$  است.

**توجه:** تخصیص هزینه استهلاکی به هر عمل، دقت می‌خواهد. اگر  $C_i$  هزینه واقعی عمل  $i$ ام و

$$\hat{C}_i \geq \sum_{i=1}^n C_i \quad \text{اگر } i \text{ام باشد آنگاه باید}$$

### روش پتانسیل

روش قبلی، اعتبار را به عنصر تخصیص می‌داد ولی این روش، اعتبار را به صورت پتانسیل به کل ساختمان داده تخصیص می‌دهد.

فرض کنید  $D_i$  وضعیت اولیه ساختمان داده‌ای است که می‌خواهیم  $n$  عمل روی آن انجام دهیم.  $C_i$  هزینه واقعی عمل  $i$ ام،  $D_i$  وضعیت ساختمان داده است پس از انجام عمل  $i$ ام روی  $D_{i-1}$ . تابع پتانسیلی تعریف می‌کنیم به اسم  $\phi$  که به هر وضعیت ساختمان داده، یک عدد حقیقی نسبت می‌دهد:

$$\phi(D_i) = \text{یک عدد حقیقی}$$

بنابراین هزینه استهلاکی  $(\hat{C}_i)$  عمل  $i$ ام برابر است با:

$$\hat{C}_i = C_i + \phi(D_i) - \phi(D_{i-1})$$

پس کل هزینه استهلاکی برابر است با:

$$\sum_{i=1}^n \hat{C}_i = \sum_{i=1}^n (C_i + \phi(D_i) - \phi(D_{i-1})) = \sum_{i=1}^n C_i + \phi(D_n) - \phi(D_0)$$

مثلاً فرض کنید در عملیات پشته (push, pop, Multipop)، تابع پتانسیل  $\phi$  را برابر عناصر موجود در پشته فرض کنیم. در حالت پشته خالی  $D_0$ ، که حالت شروع است  $\phi(D_0) = 0$ . از آنجایی که تعداد عناصر موجود در پشته عددی نامنفی است پس همیشه  $\phi(D_i) \geq 0 = \phi(D_0)$ . پس هزینه کل مستهلک شده برای  $n$  عمل با توجه به تابع  $\phi$ ، یک کران بالا برای هزینه واقعی است. حال هزینه مستهلک شده را برای push و pop و Multipop به دست می‌آوریم.

اگر عمل  $i$ ام روی پشته  $\text{push}$  باشد و در حال حاضر تعداد  $s$  عنصر در پشته باشد آنگاه:

$$\phi(D_i) - \phi(D_{i-1}) = (s+1) - s = 1$$

بنابراین هزینه مستهلک شده  $\text{push}$  برابر است با:

$$\hat{c}_i = c_i + \phi(D_i) - \phi(D_{i-1}) = 1 + 1 = 2$$

به همین ترتیب نشان دهید که هزینه مستهلک شده  $\text{pop}$  و  $\text{Multi-pop}$  برابر صفر است. پس هزینه مستهلک شده هر عمل  $O(1)$  است. بنابراین از آنجایی که هزینه مستهلک شده کران بالای هزینه واقعی است پس هزینه واقعی  $n$  عمل برابر  $O(n)$  است.

## خلاصه فصل

- ۱- هدف ما، محاسبه زمان اجرای الگوریتم‌هاست. زمان اجرای یک کد را برابر تعداد اجرای همه دستورات آن کد تعریف می‌کنیم. و از آنجایی که معمولاً مرتبه زمان اجرا برای ما اهمیت دارد، باید تعداد اجرای جمله اصلی را در یک کد محاسبه کنیم.
- ۲- در برخی الگوریتم‌ها زمان اجرا در حالات مختلف، عوض می‌شود. سه حالتِ بهترین، متوسط و بدترین مطرح می‌شود. مثلاً در جستجوی خطی در  $A[1..n]$  بهترین حالت این است که عنصر مورد جستجو با یک مقایسه پیدا شود. بدترین حالت این است که عنصر مورد جستجو، عنصر آخر آرایه باشد یا اصلاً در آرایه نباشد که نیاز به  $n$  مقایسه دارد. برای محاسبه حالت متوسط نیز باید امید ریاضی محاسبه کنیم.
- ۳- در آنالیز استهلاکی (سرشکنی)، دنباله‌ای از عملیات را روی یک ساختمان داده انجام می‌دهیم و می‌خواهیم بررسی کنیم متوسط یا میانگین زمان اجرای این عملیات در بدترین حالت چقدر است. برای این منظور سه تکنیک وجود دارد که ساده‌ترین تکنیک، آنالیز تجمعی است که زمان اجرای همه عملیات را در بدترین حالت، محاسبه کرده و جمع می‌کنیم و نتیجه را به تعداد عملیات انجام شده تقسیم می‌کنیم.

## تمرین

۱- کوچکترین مقدار  $n$  که به ازای آن الگوریتمی که زمانش  $۱۰۰n^۲$  می باشد از الگوریتمی که زمانش  $۲^n$  می باشد، سریعتر است را بیابید.

۲- زمان اجرای تکه برنامه های زیر از چه مرتبه ای است؟

a) for( $i=۱$  to  $n$ )

for( $j=۱$  to  $i^۲$ )

if( $j \bmod i == ۰$ )

for( $k=۱$  to  $j$ )

write(\*)

b) for( $i=۱$  to  $n$ )

if(odd( $i$ )) for( $j=i$  to  $n$ )  $x = x + ۱$

else for ( $j=۱$  to  $i$ )  $y = y + ۱$

c)  $i = ۳$

if( $n == ۲$  or  $n == ۳$ ) return true

if( $n \bmod ۲ == ۰$ ) return false

while ( $i^۲ \leq n$ )

if( $n \bmod i == ۰$ ) return false else  $i = i + ۲$

return true

## [سؤالات چهارگزینه‌ای فصل دوم]

۱- در یک الگوریتم، چند مرحله اول از پیچیدگی  $O(n)$ ، چند مرحله بعدی از پیچیدگی  $O(n^4)$  و چند مرحله آخر از پیچیدگی  $O(n^2)$  است. پیچیدگی کل الگوریتم چقدر است؟  
(علوم کامپیوتر – ۸۶)

(۱)  $O(n)$     (۲)  $O(n^3)$     (۳)  $O(n^4)$     (۴)  $O(n^5)$

۲- کدام گزینه تعداد مراحل برنامه زیر را به درستی بیان می‌کند؟ (ساختمان داده‌ها – دولتی ۸۳)

```
void sum (int m, int n, float S[ ][ ])
{
    int i, j;
    for (j = 0; j < m; j++)
    {
        S[n-1][j] = 0;
        for (i = 0; i < n-1; i++)
            S[n-1][j] += S[i][j];
    }
}
```

(۱)  $2m + 2n$   
(۲)  $mn^2 + m^2n$   
(۳)  $m(2n+1) + 1$   
(۴)  $m(2n+1) - 1$

۳- پیچیدگی زمانی قطعه برنامه زیر چیست؟ (مبانی نرم‌افزار – آزاد ۷۷)

```
for i := 1 to n do
    for j := 1 to i do
        for k := 1 to n do
            x := x + 1;
```

(۱)  $O(n^2)$   
(۲)  $O(n^3)$   
(۳)  $O(2^n)$   
(۴)  $O(n^2 \lg n)$

۴- مرتبه بزرگی (Order of magnitude) قطعه کد زیر چیست؟ (تخصصی نرم‌افزار آزاد – ۸۰)

```
k := 0;
for i := 1 to n do
begin
    for j := 1 to m do
        k := k + 1;
    j := 1;
    while j < n do
    begin
        k := k + 1;
        j := 2j;
    end
end;
```

(۱)  $\theta(n^2)$   
(۲)  $\theta(nm)$   
(۳)  $\theta(nm + n^2)$   
(۴)  $\theta(nm + n \log n)$

۵- درجه الگوریتم زیر چیست؟

(تخصصی هوش مصنوعی - آزاد ۸۲)

```
for i ← ۰ to n do
  {j ← i; while j ≠ ۰ do j ← j div ۲}
```

$$n^2 \quad (1) \quad n \log n \quad (2) \quad n + \log n \quad (3) \quad n \quad (4)$$

۶- مرتبه زمانی قطعه کد زیر چیست؟

(ساختمان گسسته مهندسی کامپیوتر - آزاد ۸۵)

```
i := ۲
while i ≤ n do
begin
  i := i۲
  x := x + ۱
end
```

$$\theta(\log n) \quad (1) \\ \theta(\log(\log n)) \quad (2) \\ \theta(n) \quad (3) \\ \theta(n \log n) \quad (4)$$

۷- مرتبه زمان قطعه کد زیر چیست؟

(مهندسی فناوری اطلاعات - آزاد ۸۵)

```
j := n
while j ≥ ۱ do
begin
  for i := ۱ to n do
    x := x + ۱
  j := ⌊ $\frac{j}{۲}$ ⌋
end
```

$$\theta(n \log n) \quad (1) \\ \theta(\log n) \quad (2) \\ \theta(n) \quad (3) \\ \theta(n^2) \quad (4)$$

۸- قطعه کد زیر را در نظر بگیرید، جمله  $x := x + ۱$  چند بار اجرا می شود؟

(تخصصی نرم افزار - آزاد ۸۰)

```
for i := ۱ to n do
  for j := i to n do
  begin
    k := j; r := ۱;
    while (r < k) do
    begin
      x := x + ۱;
      r := r + ۱
    end
  end
end ;
```

$$\frac{n^4 - 9n^2 - 8n}{6} \quad (1) \\ \frac{n^3 - 2n + 8}{4} \quad (2) \\ \frac{n^3 - n}{3} \quad (3) \\ \frac{n^3 - ۱۱n^2 + 9n}{6} \quad (4)$$

۹- مقدار محاسبه شده برای TOTAL را تعیین کنید. (علوم کامپیوتر - ۷۹)

```

TOTAL := ۰;
for i := ۱ to N do
begin
  k := N;
  while (k > ۱) do
  begin
    k := k DIV ۲;
    TOTAL := TOTAL + ۱
  end;
end;

```

(۱)  $N(\log_2^N - ۱)$   
 (۲)  $N \log_2^N$   
 (۳)  $N(\log_2^N + ۱)$   
 (۴)  $\log_2^N + ۱$

۱۰- در برنامه زیر تعداد دفعات تکرار دستورالعمل شماره ۳ کدام است؟ (علوم کامپیوتر - ۸۰)

```

۱ for(k = ۰; k <= n - ۱; k++)
۲   for(i = ۱; i <= n - k; i++)
۳     a[i][i + k] = k;

```

$$(۱) \quad n^2 \quad (۲) \quad \frac{n(n+۱)}{۲} \quad (۳) \quad \frac{n^2}{۲} \quad (۴) \quad \frac{n(n-۱)}{۲}$$

۱۱- پیچیدگی زمان الگوریتم زیر کدام است؟ (علوم کامپیوتر - ۸۱)

```

sum = ۰;
for(i = ۰; i < n; i++)
  for(j = ۰; j < i; j++)
    for(k = ۰; k < ۳; k++)
      sum++;

```

(۱)  $O(n^3)$   
 (۲)  $O(n)$   
 (۳)  $O(n \log n)$   
 (۴)  $O(n^2)$

۱۲- مرتبه زمانی الگوریتم زیر چیست؟ (علوم کامپیوتر - ۸۵)

```

for(i = ۱; i <= n; i = i + ۱)
  for(j = ۱; j <= n; j = j + i)
    x = x + ۱;

```

(۱)  $\theta(n)$  (۲)  $\theta(n^2)$  (۳)  $\theta(n^3)$  (۴)  $\theta(n \log n)$

۱۳- میزان زمان لازم برای اجرای قطعه برنامه زیر چگونه برآورد می‌شود؟ (IT - ۸۳)

```

for i ← ۱ to n
  for j ← n downto i
    for k ← ۱ to n^2
      sum ← sum + A

```

(۱)  $O(n^3)$  (۲)  $\theta(n^3)$  (۳)  $\omega(n^4)$  (۴)  $\theta(n^4)$



۱۴- مرتبه زمانی شبه کد زیر چیست؟

(۸۶ IT)

```
for(i = ۱; i <= n; i++)
  for(j = ۱; j <= n; j++)
  {
    x++;
    n--;
  }
```

$n$  (۱)

$n^2$  (۲)

$\log n$  (۳)

$n \log n$  (۴)

۱۵- مرتبه زمانی شبه کد زیر چیست؟

(۸۴ IT)

```
for(i = ۱; i <= n; i++)
{
  for(j = ۱; j <= n; j++)
    x++;
  n--;
}
```

$n$  (۱)

$n^2$  (۲)

$\log n$  (۳)

$n \log n$  (۴)

۱۶- پس از اجرای قطعه کد زیر، مقدار نهایی  $x$  چه مقدار خواهد بود؟

(۸۵ IT)

```
x = ۰;
for(i = ۱; i <= n; i++) {
  for(j = ۱; j <= n; j++) x++;
  j = ۱;
  while (j < n) {
    x++; j = j * ۲;
  }
}
```

$n \lceil \log_2 n \rceil$  (۱)

$n^2 + \lceil \log_2 n \rceil$  (۲)

$n^2 + n \lceil \log_2 n \rceil$  (۳)

$n(1 + \lfloor \log_2 n \rfloor)$  (۴)

۱۷- زمانی مصرفی قطعه برنامه زیر در بهترین و بدترین حالت چه مقداری است و مربوط به چه

مواردی می‌باشد؟

```
for i ← ۲ to n do
  if k mod i = ۰ then
    for j ← i to n do
       $\ell \leftarrow \ell * k$ 
```

(۱) در بهترین حالت زمان خطی است و این مربوط به ورودی است که  $k$  به  $n$  بخش‌پذیر نباشد و

بدترین حالت زمانی است که  $k$  به کلیه اعداد  $۱$  تا  $n$  بخش‌پذیر باشد در اینصورت زمان  $۲n$  است.

(۲) در بهترین حالت زمان ضریب ثابتی از  $n$  است و این حالت مربوط به مواردی است که  $k$

مضربی از  $n$  باشد و بدترین زمان  $n^2$  است که مربوط به باقی حالات می‌شود.

(۳) در هر حال و در کلیه موارد زمان مصرفی  $n^2$  است.

(۴) اگر  $k$  ضریبی از  $n!$  باشد در بدترین حالت قرار می‌گیریم که زمان مصرفی مربوطه  $n^2$

است و بهترین حالت، زمانی است که  $k$  به هیچکدام از اعداد  $۲$  تا  $n$  بخش‌پذیر نباشد در

این صورت زمان مصرفی خطی است.

۱۸- پیچیدگی زمان اجرای حلقه زیر چه می‌باشد؟ (علوم کامپیوتر - ۸۴)

$\text{while}(n > 0) \left\{ n = \frac{n}{10}; \right\}$

$O(1)$  (۱)       $O\left(\frac{n}{10}\right)$  (۲)       $O(n)$  (۳)       $O(\log n)$  (۴)

۱۹- شمار فراوانی کد زیر در بدترین حالت چیست؟ (مهندسی فناوری اطلاعات - آزاد ۸۵)

$i := 1; j := n;$   $\lceil \log(n+1) \rceil$  (۱)  
 repeat  $2 + 3 \lceil \log(n+1) \rceil$  (۲)  
      $k := \frac{(i+j)}{2}$   $\log n$  (۳)  
     if  $a[k] \leq x$  then  $i := k + 1$   $2 + 4 \lceil \log(n+1) \rceil$  (۴)  
     else  $j := k - 1$   
 until  $i > j$

۲۰- در تکه برنامه زیر processa چند بار اجرا می‌شود؟ (علوم کامپیوتر - ۸۴)

for (int i = 0; i < N - i; ++i)  
   for (int i = 0; i < N - i; ++i)  
     { /\* Process a \*/ }

$\frac{(N+1)^2}{4}$  (۴)       $\frac{N+1}{2}$  (۳)       $\frac{N^2}{4}$  (۲)       $\frac{n}{2}$  (۱)

۲۱- یک آرایه از اعداد صحیح به صورت  $A[1..m]$  مفروض است، بطوری که  $\sum_{i=1}^m A[i] = S$

می‌باشد. در این صورت درجه اجرای الگوریتم زیر کدامیک از گزینه‌ها است؟

(تخصصی هوش مصنوعی آزاد ۸۱)

$T := 0;$   $O(s^2)$  (۱)  
 for  $i := 1$  to  $m$  do  $O(m+s)$  (۲)  
   for  $j := 1$  to  $A[i]$  do  $O(m^2)$  (۳)  
      $T := T + 1;$   $O(ms)$  (۴)

۲۲- کامپیوتری در واحد زمان مساله‌ای به اندازه ۱۶ را که الگوریتم آن از مرتبه زمانی  $n^{2^n}$  است حل می‌کند. اگر سرعت کامپیوتر ۱۳۱۰۷۲ برابر گردد این کامپیوتر همان مساله را با چه

اندازه‌ای در واحد زمان حل خواهد کرد؟ (ساختمان داده‌ها - مهندسی کامپیوتر ۸۶)

$16 + 17 + \log 17$  (۱)       $16 * 17 * \log 17$  (۲)  
 $32$  (۳)       $16 + \log 131072$  (۴)

۲۳- فرض کنید که میانگین زمان اجرای یک الگوریتم تصادفی A بر روی ورودی‌های به اندازه‌ی

n برابر  $\theta(n^2)$  است. چند تا از گزاره‌های زیر درست هستند؟ (نرم‌افزار ۹۲)

ممکن است داده‌ی ورودی‌ای باشد که A آن را در زمان  $\Omega(n^{3n})$  حل کند.

ممکن است داده‌ی ورودی‌ای باشد که A آن را در زمان  $\theta(n)$  حل کند.

ممکن است داده‌ی ورودی‌ای باشد که A آن را در زمان  $O(1)$  حل کند.

(۱) ۰ (۲) ۱ (۳) ۲ (۴) ۳

۲۴- دنباله‌ای از  $2^n$  عمل بر روی داده ساختاری انجام می‌شود. هزینه‌ی عمل iام برابر i است.

اگر i توانی از ۲ باشد، وگرنه برابر ۱ است. میانگین هزینه یک عمل دلخواه (یعنی مجموع

هزینه‌ها تقسیم بر تعدادشان) به کدام گزینه نزدیک‌تر است؟ (تخصصی نرم‌افزار ۸۹)

(۱) ۲ (۲) n (۳) ۳ (۴)  $2n$

۲۵- آرایه‌ی A به طول n داده شده است. درایه‌های A در ابتدا صفرند. عمل درج روبه‌رو را n بار

بر روی A انجام می‌دهیم. هزینه‌ی سرشکنی هر عمل درج (یعنی مجموع هزینه‌ی n عمل

درج تقسیم بر n) کدام است؟ بهترین گزینه را انتخاب کنید. (نرم‌افزار ۹۱)

|                                                   |                  |
|---------------------------------------------------|------------------|
| INSERT(x)                                         | $O(1)$ (۱)       |
| ۱ $n \leftarrow n + 1$                            | $O(\lg n)$ (۲)   |
| ۲ $t \leftarrow x$                                | $O(n)$ (۳)       |
| ۳ for $i \leftarrow 0$ to $\lfloor \lg n \rfloor$ | $O(\lg^2 n)$ (۴) |
| ۴ do if $A[i] \neq 0$                             |                  |
| ۵ then $t \leftarrow t + A[i]$                    |                  |
| ۶ $A[i] \leftarrow 0$                             |                  |
| ۷ else $A[i] \leftarrow t$                        |                  |
| ۸ stop and return                                 |                  |

۲۶- هزینه‌ی زمانی تکه برنامه‌ی زیر کدام است؟ (۱ IT ۹۱)

|                 |                  |                |
|-----------------|------------------|----------------|
| int i = n;      | $O(\lg^2 n)$ (۲) | $O(\lg n)$ (۱) |
| while (i > 1) { | $O(n^2)$ (۴)     | $O(n)$ (۳)     |
| i /= ۲;         |                  |                |
| j = i;          |                  |                |
| while (j > 1)   |                  |                |
| j /= ۳;         |                  |                |
| }               |                  |                |

۲۷- رویه زیر را در نظر بگیرید: (تخصصی نرم افزار دولتی ۸۳)

این الگوریتم قرار است مقدار  $x^k$  را محاسبه کند. مقدار مستقل از حلقه (Loop invariant)

این الگوریتم چیست؟ (یعنی چه عبارتی همواره در ابتدای حلقه درست است؟)

function power (x , k)

y ← x

i ← k

a ← 1

while i > 0

do if i is odd

then a ← a × y

y ← y × y

i ←  $\left\lfloor \frac{i}{2} \right\rfloor$

return a

$$y^i a = x^k \quad (۱)$$

$$y^i a^i = x^k \quad (۲)$$

$$a (ay)^i = x^k \quad (۳)$$

$$y (ay)^i = x^k \quad (۴)$$

۲۸- یک شمارنده k بیتی در ابتدا مقدار صفر دارد. این شمارنده را  $2^k$  بار و هر بار به مقدار ۱

واحد افزایش می‌دهیم. مجموع تعداد تغییرات بیت‌های این شمارنده (از ۰ به ۱ و برعکس

(تخصصی هوش مصنوعی دولتی ۸۵)

چند تاست؟)

$$\theta(n^2) \quad (۲)$$

$$\sum_{i=1}^k \frac{n}{2^i} \quad (۱)$$

$$\theta(n \log n) \quad (۴)$$

$$\sum_{i=1}^k \frac{n}{2^{i-1}} \quad (۳)$$

۲۹- الگوریتم زیر را در نظر بگیرید:

y ← 0

i ← n

while i ≥ 0 do

y ← a<sub>i</sub> + x . y

i ← i-1

(علوم کامپیوتر – ۸۲)

مقدار y در شروع هر عبارت while برابر است با:

$$y = \sum_{k=0}^i a_k x^k \quad (۲)$$

$$y = \sum_{k=0}^{n-i} a_k x^k \quad (۱)$$

$$y = \sum_{k=0}^n a_k x^k \quad (۴)$$

$$y = \sum_{k=0}^{n-(i+1)} a_{k+i+1} x^k \quad (۳)$$

۳۰-  $n$  نفر با ترتیب تصادفی به صف وارد یک باجه می‌شوند. مسئول باجه قرار است اندازه‌ی قد بلندقدترین این افراد را به دست آورد. مسئول باجه یک برگه دارد که در ابتدا بر روی آن صفر نوشته است. او قد هر فرد که وارد باجه می‌شود را اندازه‌گیری می‌کند و اگر این قد از عدد نوشته شده بر روی برگه بزرگ‌تر باشد، عدد برگه را خط می‌زند و به جای قد فرد ورودی را می‌نویسد. به این کار او عمل «جابه‌جایی» می‌گوییم. اگر هر نفر با احتمال یکسان بتواند بلندقدترین فرد باشد، در انتها تعداد میانگین عمل جابه‌جایی چند تاست؟ بهترین پاسخ را برای  $n$  بزرگ انتخاب کنید.

(۹۵ – IT)

$$\log_2 n \quad (۱) \quad n/2 \quad (۲) \quad (n-1)/2 \quad (۳) \quad \ln n \quad (۴)$$

## پاسخنامه سؤال‌های چهارگزینه‌ای فصل دوم

- ۱- گزینه (۳). اگر چندین مرحله پشت سرهم و نه تو در تو باشند، زمان اجرای آنها با هم جمع می‌شود. و طبق قضیه ماکزیمم‌گیری زمان اجرا، متناظر با زمان اجرای کندترین مرحله است:  
 $O(n + n^4 + n^2) = O(n^4)$
- ۲- گزینه (۳). تعریف متغیر، دستور اجرایی نیست به شرطی که مقدار دهی اولیه نشده باشد. جدول زیر تعداد اجرای هر دستور را نشان می‌دهد.  
 دقت کنید حلقه‌ای که از  $m$  تا  $n$  (شامل  $m$  و  $n$ ) باشد،  $n - m + 2$  و داخل حلقه  $n - m + 1$  بار اجرا می‌شوند.

| تعداد بار اجرا  | دستور                 |
|-----------------|-----------------------|
| $m + 1$         | for(j = 0...)         |
| $m$             | s[n - 1][j] = 0       |
| $m(n)$          | for(i = 0...)         |
| $m(n - 1)$      | s[n - 1][j] + s[i][j] |
| $m(2n + 1) + 1$ | مجموع هم دستورات      |

- ۳- گزینه (۲). تعداد اجرای دستور  $x := x + 1$  برابر است با:  

$$\sum_{i=1}^n \sum_{j=1}^i \sum_{k=1}^n 1 = \sum_{i=1}^n \sum_{j=1}^i n = n \sum_{i=1}^n \sum_{j=1}^i 1 = n \sum_{i=1}^n i = n \frac{n(n+1)}{2} = \theta(n^3)$$
- البته در این تست از نماد  $O$  استفاده شده و گزینه ۳ هم غلط نیست ولی بهترین گزینه ۲ است.

- ۴- گزینه (۴). حلقه‌های ۲ و ۳ پشت سرهم هستند پس زمان آنها با هم جمع می‌شوند که  $m + \lg n$  می‌شود. هر دوی این حلقه‌ها داخل حلقه ۱ هستند که زمان حلقه ۱ یعنی  $n$  باید در  $m + \lg n$  ضرب شود. پس جواب  $\theta(n(m + \lg n))$  است.

```

۱. for i := 1 to n do
  begin
    ۲. for j := 1 to m do
      k := k + 1
    j := 1
    ۳. while j < n do
      begin
        k := k + 1
        j := 2j
      end
    end
  end
end

```

→  $\theta(n)$

→  $\theta(m)$

→  $\theta(\lg n)$

۵- گزینه (۲). حلقه for خطی است یعنی از مرتبه  $n$  است. حلقه while لگاریتمی است یعنی از مرتبه  $\lg n$  است و چون تودرتو هستند در هم ضرب می‌شوند و جواب  $n \lg n$  است.

۶- گزینه (۲). دستورات داخل حلقه به ازای مقادیر  $i$  برابر  $2, 2^2, 2^4, 2^8, \dots$  اجرا می‌شوند. فرم مقادیر  $i$  که باعث اجرا می‌شود به صورت  $2^{2^k}$  است که  $k$  تعداد اجرای دستورات داخل حلقه است و باید  $2^{2^k} \simeq n$  که حلقه تمام شود پس از طرفین  $2^{2^k} \simeq n$  لگاریتم بگیریم  $2^k \simeq \lg n$ ، یک بار دیگر لگاریتم بگیریم  $k \simeq \lg \lg n$  که گزینه ۲ صحیح است.

۷- گزینه (۱). حلقه while لگاریتمی و حلقه for خطی است و چون تودرتو هستند ضرب می‌شوند پس مرتبه زمان  $\theta(n \lg n)$  است.

۸- گزینه (۳). به ازای  $n=1$  دستور  $x := x + 1$  اصلاً اجرا نمی‌شود که اگر گزینه‌ها را با  $n=1$  بررسی کنید فقط گزینه ۳ به ازای  $n=1$  برابر صفر می‌شود. برای یافتن تعداد اجرای  $x := x + 1$  از سیگما استفاده می‌کنیم. توجه کنید حلقه while مقدار  $r$  را از ۱ تا  $k-1$

تغییر می‌دهد که  $k$  همان  $j$  است. پس این حلقه معادل  $\sum_{r=1}^{j-1}$  می‌باشد. پس تعداد اجرای

$x := x + 1$  برابر است با:

$$\begin{aligned} \sum_{i=1}^n \sum_{j=i}^n \sum_{r=1}^{j-1} 1 &= \sum_{i=1}^n \sum_{j=i}^n (j-1) = \sum_{i=1}^n \frac{(i-1+n-1)(n-i+1)}{2} \quad (*) \\ &= \sum_{i=1}^n \frac{(n+i-2)(n-i+1)}{2} = \sum_{i=1}^n \frac{n^2 - ni + n + ni - i^2 + i - 2n + 2i - 2}{2} \\ &= \frac{1}{2} \sum_{i=1}^n (n^2 - i^2 - n + 3i - 2) = \frac{1}{2} \left[ \sum_{i=1}^n (n^2 - n - 2) - \sum_{i=1}^n i^2 + 3 \sum_{i=1}^n i \right] \\ &= \frac{1}{2} \left[ (n^2 - n - 2) - \frac{n(n+1)(2n+1)}{6} + 3 \frac{n(n+1)}{2} \right] \\ &= \frac{1}{2} \left[ \frac{n(6n^2 - 6n - 12 - (n+1)(2n+1) + 9(n+1))}{6} \right] = \frac{n^3 - n}{3} \end{aligned}$$

(\*) در تصاعد حسابی هر جمله برابر است با جمله قبلی بعلاوه یک مقدار ثابت به نام قدر نسبت. مجموع جملات تصاعد حسابی برابر است با:  $\frac{(\text{تعداد جملات}) \cdot (\text{جمله آخر} + \text{جمله اول})}{2}$

$$\text{پس } \sum_{j=i}^n (j-1) \text{ برابر است با } \frac{((i-1) + (n-1))(n-i+1)}{2}.$$

۹- گزینه (۲). حلقه for خطی است و حلقه while لگاریتمی است پس گزینه ۴ نمی‌تواند صحیح باشد. از آنجایی که جواب دقیق خواسته شده است می‌توان به ازای بعضی مقادیر  $N$ ، گزینه‌ها را آزمایش کرد. گزینه‌ها به ازای  $N$ هایی که توانی از ۲ هستند  $(1, 2, 4, 8, \dots)$  معنی دارند. مثلاً به ازای  $N=1$  دستور  $ToTAL := ToTAL + 1$  اصلاً اجرا نمی‌شود و مقدارش صفر می‌ماند که فقط گزینه ۲ به ازای  $N=1$  برابر صفر می‌شود.

۱۰- گزینه (۲). به ازای  $n=1$  دستور مورد نظر فقط یک بار اجرا می‌شود که گزینه‌های ۳ و ۴ نمی‌توانند صحیح باشند. به ازای  $n=2$  دستور مورد نظر، ۳ بار اجرا می‌شود که گزینه ۲ جواب است. ولی برای رسیدن به جواب می‌توان از سیگما استفاده کرد:

$$\sum_{k=0}^{n-1} \sum_{i=1}^{n-k} 1 = \sum_{k=0}^{n-1} (n-k) = \sum_{k=0}^{n-1} n - \sum_{k=0}^{n-1} k = n^2 - \frac{n(n-1)}{2} = \frac{n(n+1)}{2}$$

۱۱- گزینه (۴). تعداد اجرای دستور  $sum++$  برابر  $3 \times \frac{n(n-1)}{2}$  است که از مرتبه  $\theta(n^2)$  است در واقع حلقه سوم اصلاً به  $n$  وابسته نیست و ثابت است.

۱۲- گزینه (۴). گام حلقه دوم برابر  $i$  می‌باشد.  $j$  از ۱ تا  $n$  با گام  $i$  تغییر می‌کند پس جمله  $x = x + 1$  به تعداد  $\frac{n}{i}$  بار اجرا می‌شود که  $i$  خود از ۱ تا  $n$  متغیر است بنابراین تعداد کل

اجرای دستور  $x = x + 1$  برابر  $\sum_{i=1}^n \frac{n}{i} \simeq n \ln n$  است که از مرتبه  $\theta(n \lg n)$  است.

۱۳- گزینه (۴). 
$$\sum_{i=1}^n \sum_{j=i}^n \sum_{k=1}^j 1 = \sum_{i=1}^n \sum_{j=i}^n n^2 = n^2 \sum_{i=1}^n \sum_{j=i}^n 1 = n^2 \sum_{i=1}^n (n-i+1)$$

$$= n^2 \left[ \sum_{i=1}^n (n+1) - \sum_{i=1}^n i \right] = n^2 \left[ n(n+1) - \frac{n(n+1)}{2} \right] = \frac{n^3(n+1)}{2} = \theta(n^4)$$

البته مرتبه این کد مشخص است که  $\theta(n^4)$  است و نیازی به بررسی دقیق نبود.

۱۴- گزینه (۱). اگر دستور  $--n$  نبود، مرتبه  $\theta(n^2)$  می‌شد. به ازای  $i=1$  دستور  $++x$  به تعداد  $\frac{n}{1}$  بار اجرای می‌شود چون به ازای هر  $++j$ ، دستور  $--n$  اجرا می‌شود و هرگاه  $j$  و  $n$  به هم برسند تمام می‌شود. به ازای  $i=2$  دستور  $++x$  به تعداد  $\frac{n}{2}$  بار اجرا می‌شود. به ازای  $i=3$ ، دستور  $++x$  به تعداد  $\frac{n}{3}$  بار اجرا می‌شود و به همین ترتیب، پس تعداد اجرای این دستور برابر  $n \left( \frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \dots \right) \simeq n$  است.



۱۵- گزینه (۲). به ازای  $i=1$  دستور  $x++$  به تعداد  $n$  بار اجرا می‌شود. به ازای  $i=2$  به تعداد  $n-1$  بار اجرا می‌شود. به ازای  $i=3$  به تعداد  $n-2$  بار اجرا می‌شود و به همین ترتیب تا وقتی  $i$  حدود  $\frac{n}{4}$  شود که دستور  $x++$  به تعداد حدوداً  $\frac{n}{4}$  بار اجرا می‌شود و برنامه تمام می‌شود. پس تعداد اجرا برابر است با:

$$(n-1) + (n-2) + \dots + \frac{n}{4} \simeq \frac{((n-1) + \frac{n}{4}) \times \frac{n}{4}}{2} = \theta(n^2)$$

۱۶- گزینه (۳). به ازای  $n=1$  دستور  $x++$  فقط یک بار (در حلقه for داخلی) اجرا می‌شود که گزینه ۱ غلط است (گزینه ۱ به ازای  $n=1$  صفر می‌شود) به ازای  $n=2$  دستور  $x++$  شش بار اجرا می‌شود (۴ بار در حلقه for داخلی و ۲ بار در حلقه while) پس گزینه ۲ غلط است چون به ازای  $n=2$  برابر ۵ می‌شود. به ازای  $n=3$  دستور  $x++$  ۱۵ بار اجرا می‌شود (۹ بار در حلقه for داخلی و ۶ بار حلقه while) پس گزینه ۳ صحیح است. البته واضح است که مرتبه زمانی این کد  $\theta(n(n + \lg n))$  است که گزینه ۳ یا ۴ چنین هستند. و با توجه به اینکه حلقه while،  $\lceil \lg n \rceil$  بار اجرا می‌شود گزینه ۳ صحیح است.

۱۷- گزینه (۴). اگر  $k \bmod i$  به ازای همه  $i$ های ۲ تا  $n$  صفر شود یعنی اگر  $k$  مضربی از  $n!$  باشد آنگاه حلقه for داخلی به ازای هر  $i$  اجرا می‌شود و مرتبه  $\theta(n^2)$  می‌شود. ولی اگر  $k \bmod i$  هیچگاه صفر نشود، آنگاه حلقه for داخلی اصلاً اجرا نمی‌شود و مرتبه  $\theta(n)$  می‌شود.

۱۸- گزینه (۴). با توجه به اینکه  $n$  هر بار به ۱۰ تقسیم می‌شود، تعداد اجرا برابر  $\log_{10} n$  است و چون پایه لگاریتم اگر ثابت باشد در رشد آن تاثیری ندارد پس مرتبه  $O(\lg n)$  یعنی گزینه ۴ است.

۱۹- گزینه (۴). شمار فراوانی یعنی مجموع تعداد اجرای همه دستورات. دستورات  $i:=1$  و  $j:=n$  هر کدام یک بار اجرا می‌شوند. دستورات داخل حلقه حدود  $\lg n$  بار (هر کدام) اجرا می‌شوند. یعنی دستورات  $k:=\frac{i+j}{4}$  و  $a[k] \leq x$  و  $i:=k+1$  یا  $j:=k-1$  و  $j>i$  که تعدادشان ۴ تا است هر کدام لگاریتمی اجرا می‌شوند. پس تعداد اجرای همه خطوط حدوداً  $2 + 4 \lg n$  است که گزینه ۴ می‌تواند صحیح باشد. البته می‌توان به ازای  $n=1$  بررسی کرد که هر یک از دستورات  $i:=1$  و  $j:=n$  و  $k:=\frac{i+j}{4}$  و  $a[k] \leq x$  و  $i:=k+1$  یا  $j:=k-1$  و  $j>i$  یک بار اجرا می‌شوند یعنی جمعاً ۶ بار اجرا می‌شوند که گزینه ۴ می‌تواند صحیح باشد.

۲۰- گزینه (۴و۲). در حلقه دوم، متغیر  $i$  یک متغیر جدید است و ارتباطی به متغیر  $i$  در حلقه اول ندارد. در واقع می‌توان این تکه برنامه را به شکل زیر نوشت:

```
for (i = ۰; i < N - i; ++ i)
  for (j = ۰; j < N - j; ++ j)
    { /* processa */ }
```

اگر  $N$  زوج باشد آنگاه processa به تعداد  $\frac{N}{۲} \times \frac{N}{۲} = \frac{N^2}{۴}$  اجرا می‌شود زیرا حلقه اول از  $i = ۰$  تا  $i < \frac{N}{۲}$  و حلقه دوم از  $j = ۰$  تا  $j < \frac{N}{۲}$  اجرا می‌شوند که هر کدام  $\frac{N}{۲}$  بار وارد بدنه حلقه می‌شوند. ولی اگر  $N$  فرد باشد، آنگاه processa به تعداد  $\frac{N+1}{۲} \times \frac{N+1}{۲} = \frac{(N+1)^2}{۴}$  اجرا می‌شود. که گزینه‌های ۲ و ۴ صحیح هستند!

۲۱- گزینه (۲). اگر مثلاً فرض کنیم عناصر آرایه  $A$  همگی صفر هستند آنگاه حلقه دوم هیچگاه اجرا نمی‌شود زیرا  $for\ j := ۱\ to\ ۰$  قابل اجرا نیست بنابراین دستور  $T := T + ۱$  اصلاً اجرا نمی‌شود و جمله اصلی  $for\ i := ۱\ to\ m$  است که  $m$  بار اجرا می‌شود و مرتبه  $O(m)$  می‌باشد. ولی اگر عناصر آرایه  $A$  بزرگتر از صفر باشند آنگاه دستور  $T := T + ۱$  جمله اصلی می‌باشد و به تعداد  $A[۱] + A[۲] + \dots + A[m]$  یعنی به تعداد  $S$  بار  $(\sum_{i=1}^m A[i] = S)$  اجرا می‌شود که از مرتبه  $O(S)$  می‌باشد. پس این تکه برنامه یا از مرتبه  $m$  یا  $S$  است پس از مرتبه  $O(m + S)$  است.

۲۲- گزینه (۳). وقتی گفته می‌شود مرتبه زمانی  $n.۲^n$  است یعنی زمان اجرا  $c.n.۲^n$  است که  $c$  ثابت است. پس طبق صورت سوال به ازای  $n = ۱۶$  زمان برابر ۱ (واحد زمان) می‌باشد یعنی  $۱ = c.(۱۶).۲^{۱۶}$ . در نتیجه  $c = ۲^{-۲۰}$ . حال اگر سرعت کامپیوتر  $۲^{۱۷} = ۱۳۱۰۷۲$  برابر شود بنابراین زمان اجرا باید به  $۲^{۱۷}$  تقسیم شود. یعنی زمان اجرا در کامپیوتر جدید  $\frac{c.n.۲^n}{۲^{۱۷}}$  می‌باشد که طبق صورت سوال این نیز باید برابر واحد زمان یعنی ۱ باشد یعنی  $\frac{c.n.۲^n}{۲^{۱۷}} = ۱$  و چون  $c = ۲^{-۲۰}$  داریم:

$$\frac{۲^{-۲۰} \times n \times ۲^n}{۲^{۱۷}} = ۱ \rightarrow n \times ۲^n = ۲^{۳۷} \rightarrow \boxed{n = ۳۲}$$

۲۳- گزینه (۴). وقتی زمان حالت متوسط  $\theta(n^۲)$  است. بهترین حالت از مرتبه  $O(n^۲)$  و بدترین حالت از مرتبه  $\Omega(n^۲)$  است. یعنی بهترین حالت ممکن است از مرتبه  $\theta(n^۲)$  یا

کمتر باشد و بدترین حالت ممکن است از مرتبه  $\theta(n^2)$  بیشتر باشد. پس هر سه گزاره می‌توانند صحیح باشند.

۲۴- گزینه (۳).

| شماره عمل | ۱ | ۲ | ۳ | ۴ | ۵ | ۶ | ۷ | ۸ | ۹ | ... | $2^n - 1$ | $2^n$ |
|-----------|---|---|---|---|---|---|---|---|---|-----|-----------|-------|
| هزینه     | ۱ | ۲ | ۱ | ۴ | ۱ | ۱ | ۱ | ۸ | ۱ |     | ۱         | $2^n$ |

$$\begin{aligned} \text{میانگین هزینه یک عمل دلخواه} &= \frac{\text{جمع هزینه‌ها}}{\text{تعداد عملیات}} = \frac{(2^0 + 2^1 + 2^2 + \dots + 2^n) + (2^n - (n+1))}{2^n} \\ &= \frac{2^{n+1} - 1 + 2^n - n - 1}{2^n} = \frac{3 \times 2^n - n - 2}{2^n} \xrightarrow{n \rightarrow \infty} 3 \end{aligned}$$

لازم به ذکر است که  $2^n$  عمل وجود دارد که تعداد  $n+1$  تای آن دارای شماره‌ای هستند که توانی از ۲ است ( $2^0$  و  $2^1$  و ... و  $2^n$ ) و سایر عملیات که شماره‌شان توانی از ۲ نیست تعدادشان  $2^n - (n+1)$  است که هزینه هر کدام برابر یک است.

۲۵- گزینه (۱). اولین عمل درجه فقط  $A[0]$  را تغییر می‌دهد. دومین عمل درج  $A[0]$  و  $A[1]$  را تغییر می‌دهد. سومین عمل درج فقط  $A[0]$  را تغییر می‌دهد. چهارمین عمل درج،  $A[0]$  و  $A[1]$  و  $A[2]$  را تغییر می‌دهد و به همین ترتیب. هزینه این الگوریتم دقیقاً شبیه عمل INC است. با  $n$  عمل درج،  $A[0]$ ،  $n$  بار تغییر می‌کند.  $A[1]$ ،  $\frac{n}{2}$  بار،  $A[2]$ ،  $\frac{n}{4}$  بار ... تغییر می‌کنند پس کل هزینه  $n$  عمل برابر  $2n \simeq n + \frac{n}{2} + \frac{n}{4} + \dots$  است پس هزینه سرشکنی هر عمل برابر ۲ یعنی  $\theta(1)$  است.

۲۶- گزینه (۲). تعداد اجرای حلقه while داخل را باید محاسبه کنیم. این حلقه به ازای هر  $i$  به تعداد  $\log_2^i$  بار اجرا می‌شود، مقادیر  $i$  عبارتند از  $\frac{n}{2}$  و  $\frac{n}{4}$  و  $\frac{n}{8}$  و ... و  $\frac{n}{2^k}$  پس جواب:

$$\begin{aligned} &\log_2^{\frac{n}{2}} + \log_2^{\frac{n}{4}} + \log_2^{\frac{n}{8}} + \dots + \log_2^{\frac{n}{2^k}} \\ &= \log_2^n - \log_2^2 + \log_2^n - \log_2^4 + \dots + \log_2^n - \log_2^{2^k} \\ &= k \cdot \log_2^n - (\log_2^{2 \times 4 \times \dots \times 2^k}) = k \cdot \log_2^n - \log_2^{\frac{k(k+1)}{2} 2} \\ &\text{و } k \simeq \log_2^n \text{ پس تعداد اجرا از مرتبه } O(\log^2 n) \text{ است.} \end{aligned}$$

۲۷- گزینه (۱). با مثال بررسی می‌کنیم. فرض کنید  $k = 5$  باشد:

$(y(ay))^i \neq x^k$  گزینه ۴ نادرست  $\Rightarrow a \leftarrow 1, i = 5, y \leftarrow x$  : بار اول

$\Rightarrow i = 2, y^2 = x^2, y \leftarrow y^2 = x^2$  : بار دوم

$(a(ay))^i \neq x^k$  گزینه ۳ نادرست و  $(y^i a^i \neq x^k)$  گزینه ۲ نادرست

پس گزینه ۱ صحیح است. مرحله ۳ را نیز می‌نویسیم هر چند لازم نیست:

$(x^4 x = x^k) \Rightarrow y^i a = x^k, i = 1, a = x$  : بار سوم

۲۸- گزینه (۳). کم ارزش‌ترین بیت هر بار عوض می‌شود ( $2^k$  بار)، بیت بعدی یک در میان عوض

می‌شود ( $2^{k-1}$  بار) ... با ارزش‌ترین بیت ۲ بار عوض می‌شود پس:

$$2^1 + 2^{k-1} + 2^k = \text{مجموع تغییر بیت‌ها}$$

اگر  $2^k = n$  فرض شود آنگاه:

$$\text{مجموع تغییر بیت‌ها} = n + \frac{n}{2} + \frac{n}{2^2} + \dots + \frac{n}{2^{k-1}} = \sum_{i=0}^{k-1} \frac{n}{2^i} = \sum_{i=1}^k \frac{n}{2^{i-1}}$$

۲۹- گزینه (۳). فرض کنید  $n = 2$ . بار اول که وارد حلقه می‌شویم  $i = 2$  و  $y = 0$  است. داخل

حلقه  $y = a_2$  و  $i = 1$  می‌شود پس بار دوم که می‌خواهیم وارد حلقه شویم  $y = a_2$  و  $i = 1$

است. و داخل حلقه  $y = a_1 + a_2 x$  و  $i = 0$  می‌شود پس بار سوم که می‌خواهیم وارد حلقه

شویم  $y = a_1 + a_2 x$  و  $i = 0$  است و داخل حلقه  $y = a_0 + a_1 x + a_2 x^2$  می‌شود و

$i = -1$ . بار چهارم وارد حلقه نمی‌شویم. پس مقادیر  $y$  در شروع حلقه عبارت است از:

| $i$ | ۱     | ۲             |
|-----|-------|---------------|
| $y$ | $a_2$ | $a_1 + a_2 x$ |

در گزینه‌ها اگر بجای  $n = 2$  و  $i = 1$  قرار دهیم باید  $y = a_2$  شود که فقط گزینه ۳ می‌شود.

۳۰- گزینه (۴). سؤال تکراری است. نفر  $i$ ام به احتمال  $\frac{1}{i}$  قدبلندترین در بین  $i$  نفر اول است

پس جواب  $\sum_{i=1}^n \frac{1}{i} \approx \ln n$  است.



## فصل ۳

روابط بازگشتی – تحلیل الگوریتم‌های  
بازگشتی – تقسیم و غلبه

بسیاری از الگوریتم‌ها و توابع، بازگشتی هستند یعنی خود را فراخوانی می‌کنند. برای تحلیل این الگوریتم‌ها و یافتن زمان اجرای آنها نیاز است که حل روابط بازگشتی را بدانیم. یک رابطه بازگشتی برای یک دنباله مثل  $a_0, a_1, a_2, \dots$  رابطه‌ای است که  $a_n$  را به حسب جملات قبلی بیان کند. مثلاً رابطه بازگشتی  $a_n = a_{n-1} + a_{n-2}$  جمله  $n$ ام را بر حسب دو جمله قبلی بیان می‌کند، که گویند این رابطه مرتبه ۲ است. پس اگر  $a_n$  تا عمق  $a_{n-k}$  وابسته باشد گویند رابطه بازگشتی مرتبه  $k$  است. یک رابطه بازگشتی مرتبه  $k$  نیاز به  $k$  شرط اولیه دارد. مثلاً رابطه  $a_n = a_{n-1} + a_{n-2}$  نیاز به دو شرط اولیه دارد که مثلاً  $a_0 = 0$  و  $a_1 = 1$ . حال با این دو شرط اولیه می‌توان همه جملات دنباله را نوشت:  $a_2 = a_1 + a_0 = 1$  و  $a_3 = a_2 + a_1 = 2$  و  $a_4 = a_3 + a_2 = 3$  و ...

منظور از حل رابطه بازگشتی یافتن یک فرمول صریح بر حسب  $n$  برای  $a_n$  است. روش‌های مختلفی برای حل روابط بازگشتی وجود دارد که بررسی می‌کنیم.

**مثال ۱:** مطلوبست حل رابطه بازگشتی مرتبه اول  $a_n = a_{n-1} + n$  با شرط اولیه  $a_0 = 0$ .

**پاسخ:** یکی از روش‌های حل برخی روابط بازگشتی، جایگذاری است که خود دو نوع است: جایگذاری از پایین و جایگذاری از بالا. جایگذاری از پایین یعنی از خود  $a_0$  شروع کنیم و  $a_1, a_2, a_3, \dots$  را بیابیم و در صورت امکان فرمولی برای  $a_n$  بیابیم. در این مثال  $a_0 = 0$  و  $a_1 = a_0 + 1 = 0 + 1 = 1$  و  $a_2 = a_1 + 2 = 0 + 1 + 2 = 3$  و به همین ترتیب نتیجه می‌شود که

$$a_n = 0 + 1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

جایگذاری از بالا یعنی از خود  $a_n$  شروع کنیم که  $a_n = a_{n-1} + n$  حال  $a_{n-1}$  را از خود رابطه بیابیم که  $a_{n-1} = a_{n-2} + n - 1$  و در  $a_n$  جایگذاری کنیم و به همین ترتیب  $a_{n-2}$  را بیابیم و جایگذاری کنیم تا وقتی که به شروط اولیه برسیم:

$$\begin{aligned} a_n &= a_{n-1} + n = (a_{n-2} + n - 1) + n = a_{n-2} + (n - 1) + n \\ &= (a_{n-3} + n - 2) + (n - 1) + n = a_{n-3} + (n - 2) + (n - 1) + n \\ &= \dots = a_0 + 1 + 2 + 3 + \dots + (n - 2) + (n - 1) + n = 0 + 1 + 2 + \dots + n = \frac{n(n+1)}{2} \end{aligned}$$

بنابراین حل رابطه  $a_n = a_{n-1} + n$  با شرط اولیه  $a_0 = 0$  برابر  $a_n = \frac{n(n+1)}{2}$  است که از مرتبه  $\theta(n^2)$  است.

**مثال ۲:** مطلوب است حل رابطه بازگشتی مرتبه اول  $T(n) = 2T(n-1) + 1$  با شرط اولیه  $T(1) = 1$ .

**پاسخ:** با روش جایگذاری از پایین داریم:

$$T(1) = 1 \rightarrow T(2) = 2T(1) + 1 = 3 \rightarrow T(3) = 2T(2) + 1 = 7 \rightarrow T(4) = 2T(3) + 1 = 15$$

ظاهراً می‌توان حدس زد که  $T(n) = 2^n - 1$  (البته این حدس نیاز به اثبات دارد).

با روش جایگذاری از بالا داریم:

$$\begin{aligned} T(n) &= 2T(n-1) + 1 = 2[2T(n-2) + 1] + 1 = 2^2T(n-2) + 2 + 1 \\ &= 2^2[2T(n-3) + 1] + 2 + 1 = 2^3T(n-3) + 2^2 + 2 + 1 \\ &= \dots = 2^{n-1}T(n - (n-1)) + 2^{n-2} + 2^{n-3} + \dots + 2^2 + 2 + 1 \\ &= 2^{n-1}T(1) + 2^{n-2} + \dots + 2 + 1 = 2^{n-1} + 2^{n-2} + \dots + 2 + 1 = 2^n - 1 = \theta(2^n) \end{aligned}$$

**مثال ۳:** با فرض اینکه  $n$  توانی از ۲ است مطلوبست حل رابطه بازگشتی  $T(n) = 2T\left(\frac{n}{2}\right) + n - 1$

با شرط اولیه  $T(1) = 0$ .

**پاسخ:** با جایگذاری از بالا داریم:

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n - 1 = 2\left[2T\left(\frac{n}{4}\right) + \frac{n}{2} - 1\right] + n - 1 \\ &= 2^2T\left(\frac{n}{4}\right) + (n - 2) + (n - 1) = 2^2\left[2T\left(\frac{n}{8}\right) + \frac{n}{4} - 1\right] + (n - 2) + (n - 1) \\ &= 2^3T\left(\frac{n}{8}\right) + (n - 4) + (n - 2) + (n - 1) = \dots \\ &= 2^kT\left(\frac{n}{2^k}\right) + (n - 2^{k-1}) + (n - 2^{k-2}) + \dots + (n - 2) + (n - 1) \end{aligned}$$

باید  $\frac{n}{2^k} = 1$  یعنی  $n = 2^k$  پس  $k = \lg n$ . و با توجه به اینکه  $T(1) = 0$

پس:

$$T(n) = 2^k \times 0 + (n - 2^{k-1}) + (n - 2^{k-2}) + \dots + (n - 2) + (n - 1) \\ = nk - (2^{k-1} + 2^{k-2} + \dots + 1) = nk - (2^k - 1) = n \cdot \lg n - n + 1 \quad \blacksquare$$

روش جایگذاری برای روابط بازگشتی محدودی می‌تواند جواب بیابد و خیلی قدرتمند نیست. مثلاً اگر رابطه بازگشتی مرتبه ۲ یا بیشتر باشد، روش جایگذاری اصلاً مناسب نیست. در ادامه روش‌های دیگری را برای حل روابط بازگشتی ارائه می‌دهیم.

### حل روابط بازگشتی خطی همگن / ناهمگن ضریب ثابت

می‌خواهیم روابط بازگشتی به شکل  $a_n = c_1 a_{n-1} + c_2 a_{n-2} + \dots + c_k a_{n-k} + f(n)$  که  $c_i$  ها ( $1 \leq i \leq k$ ) ثابت هستند را حل کنیم. در این فرم روابط نحوه بازگشت به این گونه است که از  $n$  کم می‌شود یعنی  $a_n$  به  $a_{n-1}$  و  $a_{n-2}$  و ... و  $a_{n-k}$  وابسته است. فرم نوشته شده مرتبه  $k$  است (البته به شرطی که  $c_k \neq 0$ ). اگر  $f(n) = 0$  باشد گویند رابطه همگن و در غیر این صورت ناهمگن است. مثلاً  $a_n = 2a_{n-1} + a_{n-2} + 5$  یک رابطه بازگشتی خطی ناهمگن مرتبه ۳ با ضرایب ثابت است. یا رابطه  $a_n = 2a_{n-4}$  یک رابطه خطی همگن مرتبه ۴ با ضرایب ثابت است ( $a_n$  به  $a_{n-4}$  وابسته است).

برای حل اینگونه روابط، معادله مشخصه تشکیل می‌دهیم. برای قسمت همگن معادله مشخصه به شکل  $x^k - c_1 x^{k-1} - c_2 x^{k-2} - \dots - c_k = 0$  است. برای قسمت ناهمگن  $(f(n))$ ، اگر  $f(n)$  به فرم  $b^n \cdot p(n)$  باشد که  $b$  ثابت است و  $p(n)$  چند جمله‌ای درجه  $d$  است، آنگاه عبارت  $(x - b)^{d+1}$  را در معادله مشخصه قسمت همگن ضرب می‌کنیم. مثلاً معادله مشخصه برای رابطه بازگشتی  $a_n = 4a_{n-1} + 4a_{n-2} + 3^n(2n+1)$  به شکل  $(x^2 - 4x - 4)(x - 3)^2 = 0$  است (قسمت ناهمگن  $3^n(2n+1)$  است که  $b = 3$  و درجه چند جمله‌ای  $2n+1$  برابر  $d = 1$  است). و یا معادله مشخصه برای رابطه بازگشتی  $a_n = 9a_{n-2} + 2^n(3n-6) + 1^n(2n^2 + n + 1)$  عبارت است از:

$$(x^2 - 9)(x - 2)^2(x - 1)^3 = 0.$$

زیرا قسمت همگن  $a_n = 9a_{n-2}$  مرتبه ۲ است و معادله آن  $x^2 - 9$  می‌باشد و قسمت ناهمگن از دو تا  $b^n p(n)$  تشکیل شده است که اولی  $2^n(3n-6)$  است که  $b = 2$  و  $d = 1$  و دومی  $1^n(2n^2 + n + 1)$  است که  $b = 1$  و  $d = 2$  می‌باشد. بعد از اینکه معادله مشخصه تشکیل دادیم، این معادله را حل می‌کنیم و ریشه‌های معادله مشخصه را می‌یابیم. اگر دو ریشه



حقیقی متمایز  $x_1$  و  $x_2$  داشت آنگاه جواب رابطه بازگشتی  $a_n = \alpha_1(x_1)^n + \alpha_2(x_2)^n$  است. اگر سه تا ریشه حقیقی متمایز  $x_1$  و  $x_2$  و  $x_3$  داشت آنگاه جواب رابطه  $a_n = \alpha_1(x_1)^n + \alpha_2(x_2)^n + \alpha_3(x_3)^n$  است. اگر معادله مشخصه ریشه مضاعف  $x_1 = x_2$  داشته باشد آنگاه جواب رابطه  $a_n = (\alpha_1 + \alpha_2 n)(x_1)^n$  است. اگر ریشه مرتبه ۳ یعنی  $x_1 = x_2 = x_3$  باشد آنگاه جواب رابطه  $a_n = (\alpha_1 + \alpha_2 n + \alpha_3 n^2)(x_1)^n$  است. اگر معادله مشخصه ریشه موهومی  $x_1, x_2 = a \pm bi$  داشته باشد آنگاه جواب  $a_n = (\sqrt{a^2 + b^2})^n [\alpha_1 \cos n\theta + \alpha_2 \sin n\theta]$  است که  $\theta = \arctan\left(\frac{b}{a}\right)$ . برای یافتن  $\alpha_1$  و  $\alpha_2$  و  $\alpha_3$  و ... از شروط اولیه کمک می‌گیریم.

**مثال ۴:** مطلوبست حل رابطه بازگشتی مرتبه ۲،  $a_n = a_{n-1} + a_{n-2}$  با شروط اولیه  $a_1 = 1, a_0 = 0$ .

**پاسخ:** این رابطه بازگشتی فیبوناچی است. معادله مشخصه  $x^2 - x - 1 = 0$  است که ریشه‌های این معادله  $\frac{1 \pm \sqrt{5}}{2}$  است پس جواب  $a_n = \alpha_1 \left(\frac{1+\sqrt{5}}{2}\right)^n + \alpha_2 \left(\frac{1-\sqrt{5}}{2}\right)^n$  است. برای یافتن  $\alpha_1$  و  $\alpha_2$ ، شروط اولیه را اعمال می‌کنیم:

$$۱) a_0 = \alpha_1 \left(\frac{1+\sqrt{5}}{2}\right)^0 + \alpha_2 \left(\frac{1-\sqrt{5}}{2}\right)^0 = \alpha_1 + \alpha_2 = 0$$

$$۲) a_1 = \alpha_1 \left(\frac{1+\sqrt{5}}{2}\right) + \alpha_2 \left(\frac{1-\sqrt{5}}{2}\right) = 1$$

از معادله (۱)،  $\alpha_2 = -\alpha_1$  در معادله ۲ جایگذاری کنید:

$$\alpha_1 \left(\frac{1+\sqrt{5}}{2}\right) - \alpha_1 \left(\frac{1-\sqrt{5}}{2}\right) = 1 \rightarrow \alpha_1 (\sqrt{5}) = 1 \rightarrow \alpha_1 = \frac{1}{\sqrt{5}}, \alpha_2 = -\frac{1}{\sqrt{5}}$$

بنابراین حل رابطه بازگشتی فیبوناچی  $a_n = \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2}\right)^n - \frac{1}{\sqrt{5}} \left(\frac{1-\sqrt{5}}{2}\right)^n$  است که از

مرتبه  $a_n \in \theta \left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right)$  است که حدود  $a_n \in \theta((1/6)^n)$  یعنی نمایی است.

**مثال ۵:** مطلوبست حل رابطه بازگشتی مرتبه اول ناهمگن  $a_n = a_{n-1} + n$  با شرط اولیه  $a_0 = 0$ .

**پاسخ:** در مثال ۱ این رابطه را با جایگذاری حل کردیم. معادله مشخصه  $(x-1)(x-1)^2 = 0$  است. دقت کنید قسمت ناهمگن  $n = 1^n$  است که  $b=1$  و  $d=1$ . معادله مشخصه دارای

ریشه‌های ۱ و ۱ و ۱ است پس  $a_n = (\alpha_1 + \alpha_2 n + \alpha_3 n^2)(1)^n$ . نیاز به سه شرط اولیه داریم با توجه به  $a_0 = 0$  و  $a_n = a_{n-1} + n$  داریم  $a_1 = a_0 + 1 = 1$  و  $a_2 = a_1 + 2 = 3$  پس:

$$\left\{ \begin{array}{l} a_0 = \alpha_1 + \alpha_2 \times 0 + \alpha_3 \times 0 = 0 \\ a_1 = \alpha_1 + \alpha_2 + \alpha_3 = 1 \\ a_2 = \alpha_1 + 2\alpha_2 + 4\alpha_3 = 3 \end{array} \right\} \rightarrow \alpha_1 = 0, \alpha_2 = \frac{1}{2}, \alpha_3 = \frac{1}{2}$$

$$a_n = 0 + \frac{n}{2} + \frac{n^2}{2} = \frac{n(n+1)}{2} \quad \text{بنابراین}$$

**مثال ۶:** مطلوبست شکل کلی جواب رابطه بازگشتی  $a_{n+1} = 4a_{n-1} + 2n + 1 + 5 \times 2^n + n \times 2^n$

**پاسخ:** قسمت همگن مرتبه ۲ است (اختلاف  $n+1$  و  $n-1$  برابر ۲ است) پس مشخصه

قسمت همگن  $(x^2 - 4) = 0$  است. قسمت ناهمگن از دو تا  $b^n p(n)$  تشکیل شده است.

یکی  $(2n+1)2^n$  که  $b=d=1$  و دیگری  $2^n(n+5)$  که  $b=2$  و  $d=1$  (توجه کنید که حتماً

از  $2^n$  فاکتور بگیرید). پس معادله مشخصه کل رابطه  $(x^2 - 4)(x-1)^2(x-2)^2 = 0$  است

که ریشه‌های این معادله  $1, 1, 2, 2, -2, -2$  است پس جواب این رابطه

$$a_n = \alpha_1(-2)^n + (\alpha_2 + \alpha_3 n + \alpha_4 n^2)(2)^n + (\alpha_5 + \alpha_6 n)(1)^n$$

از مرتبه  $a_n \in \theta(n^2 2^n)$  است البته به شرطی که  $\alpha_4 \neq 0$ . ■

می‌توان برخی روابط بازگشتی را با تغییر متغیر به روابط بازگشتی خطی ضریب ثابت تبدیل

کرد و با معادله مشخصه حل کرد.

**مثال ۷:** با فرض اینکه  $n$  توانی از ۲ است. مطلوبست حل رابطه  $T(n) = 4T\left(\frac{n}{2}\right) - 4T\left(\frac{n}{4}\right)$  با

$$T(2) = 3 \quad \text{و} \quad T(1) = 1$$

**پاسخ:** قرار می‌دهیم  $n = 2^m$  پس:  $T(2^m) = 4T(2^{m-1}) - 4T(2^{m-2})$ . حال تعریف می‌کنیم

$$a_m = T(2^m), \quad \text{بنابراین} \quad a_m = 4a_{m-1} - 4a_{m-2}$$

ریشه‌های ۲ و ۲ است، در نتیجه  $a_m = (\alpha_1 + \alpha_2 m)(2)^m$  و چون  $n = 2^m$  پس  $m = \lg n$

بنابراین  $T(n) = (\alpha_1 + \alpha_2 \lg n)(n)$  و با توجه به شروط اولیه  $T(1) = (\alpha_1 + 0)(1) = 1$  و

$$T(2) = (\alpha_1 + \alpha_2)(2) = 3 \quad \text{در نتیجه} \quad \alpha_1 = 1 \quad \text{و} \quad \alpha_2 = \frac{1}{2}$$

$$T(n) = (1 + \frac{1}{2} \lg n)n \quad \text{است که از مرتبه} \quad T(n) \in \theta(n \lg n)$$

**مثال ۸:** مطلوبست حل رابطه بازگشتی غیر خطی  $a_n = a_{n-1} \cdot a_{n-2}^2$  با شروط اولیه  $a_0 = 1, a_1 = 2$ .

**پاسخ:** از طرفین رابطه، لگاریتم پایه ۲ می‌گیریم و تعریف می‌کنیم  $\lg(a_n) = b_n$   
 $\lg(a_n) = \lg(a_{n-1}) + 2\lg(a_{n-2}) \rightarrow b_n = b_{n-1} + 2b_{n-2}, b_0 = \lg a_0 = 0,$   
 $b_1 = \lg a_1 = \lg 2 = 1 \rightarrow x^2 - x - 2 = 0 \rightarrow$  ریشه:  $-1, 2 \rightarrow b_n = \alpha_1(2)^n + \alpha_2(-1)^n$   
 $\rightarrow b_0 = \alpha_1 + \alpha_2 = 0, b_1 = 2\alpha_1 - \alpha_2 = 1 \rightarrow \alpha_1 = \frac{1}{3}, \alpha_2 = -\frac{1}{3} \rightarrow b_n = \frac{1}{3}(2)^n - \frac{1}{3}(-1)^n$   
 $\lg a_n = b_n \rightarrow a_n = 2^{b_n} \rightarrow a_n = 2^{\left(\frac{1}{3}(2)^n - \frac{1}{3}(-1)^n\right)}$

**مثال ۹:** مطلوبست حل رابطه بازگشتی  $n \cdot a_n = 2(n-1) \cdot a_{n-1} - (n-2)a_{n-2}$  با شرایط اولیه  $a_0 = a_1 = 1$ .

**پاسخ:** ضرایب ثابت نیستند و نمی‌توان معادله مشخصه تشکیل داد. ولی اگر تعریف کنیم  $b_n = n \cdot a_n$  آنگاه رابطه داده شده را می‌توان نوشت  $b_n = 2b_{n-1} - b_{n-2}$  و شروط اولیه  $b_0 = 0, a_0 = 1$  و  $b_1 = 1, a_1 = 1$ . حال می‌توان معادله مشخصه نوشت و  $b_n$  را یافت و سپس با توجه به  $b_n = n \cdot a_n$  آنگاه  $a_n = \frac{1}{n} b_n$

$$x^2 - 2x + 1 = 0 \rightarrow \text{ریشه‌ها: } 1, 1 \rightarrow b_n = (\alpha_1 + \alpha_2 n)(1)^n \rightarrow b_0 = \alpha_1 = 0$$

$$b_1 = \alpha_1 + \alpha_2 = 1 \rightarrow \alpha_1 = 0, \alpha_2 = 1 \rightarrow b_n = n \rightarrow a_n = \frac{1}{n} b_n = 1$$

و پس حل رابطه مذکور  $a_n = 1$  است.

**مثال ۱۰:** مطلوبست حل رابطه بازگشتی  $a_n = 3na_{n-1} - 2n(n-1)a_{n-2}$  و  $a_0 = 1$  و  $a_1 = 2$ .

**پاسخ:** ضرایب ثابت نیستند. طرفین را به  $n!$  تقسیم کنید و تعریف کنید  $b_n = \frac{a_n}{n!}$  و

$$b_1 = \frac{a_1}{1!} = 2 \text{ و } b_0 = \frac{a_0}{0!} = 1$$

$$\frac{a_n}{n!} = 3 \frac{a_{n-1}}{(n-1)!} - 2 \frac{a_{n-2}}{(n-2)!} \rightarrow b_n = 3b_{n-1} - 2b_{n-2}$$

$$\rightarrow x^2 - 3x + 2 = 0 \rightarrow b_n = \alpha_1(2)^n + \alpha_2(1)^n \rightarrow b_0 = \alpha_1 + \alpha_2 = 1, b_1 = 2\alpha_1 + \alpha_2 = 2$$

$$\rightarrow \alpha_1 = 1, \alpha_2 = 0 \rightarrow b_n = 2^n \rightarrow a_n = n! \cdot b_n \rightarrow a_n = n! \cdot 2^n \quad \blacksquare$$

در بسیاری از تست‌ها و مسائل ما حل کامل رابطه بازگشتی را نیاز نداریم و فقط می‌خواهیم مرتبه رابطه را بدست آوریم. برای این منظور نکات و روشهای مختلفی را مطرح می‌کنیم.

### نکته

رابطه بازگشتی به صورت  $T(n) = aT(n-b) + c$  که  $a, b, c$  ثابت مثبت هستند را در نظر بگیرید. اگر  $a \neq 1$  آنگاه  $T(n) \in \theta(a^{\frac{n}{b}})$  و اگر  $a = 1$  و  $c > 0$  آنگاه  $T(n) \in \theta(n)$ . به عنوان مثال به روابط زیر توجه کنید:

$$T(n) = 2T(n-1) + 1 = \theta(2^n) \quad \text{و} \quad T(n) = 4T(n-2) + 3 = \theta(4^{\frac{n}{2}}) = \theta(2^n) \\ T(n) = T(n-2) + 3 = \theta(n)$$

برای اثبات این نکته می‌توانید از معادله مشخصه یا روش جایگذاری استفاده کنید.

**قضیه Master (اساسی):** رابطه بازگشتی به صورت  $T(n) = aT(\frac{n}{b}) + f(n)$  که  $a, b, c$  ثابت هستند و  $a \geq 1$  و  $b > 1$  را در نظر بگیرید.  $\frac{n}{b}$  می‌تواند به صورت  $\lfloor \frac{n}{b} \rfloor$  یا  $\lceil \frac{n}{b} \rceil$  یا حتی به صورت  $\frac{n}{b} \pm c$  که  $c$  ثابت است، باشد. برای یافتن مرتبه اینگونه روابط بازگشتی حالات زیر را در نظر می‌گیریم:

**حالت ۱:** اگر  $f(n) = O(n^{\log_b a - \epsilon})$  که  $\epsilon > 0$  ثابت است آنگاه  $T(n) = \theta(n^{\log_b a})$

**حالت ۲:** اگر  $f(n) = \theta(n^{\log_b a})$  آنگاه  $T(n) = \theta(f(n) \cdot \lg n) = \theta(n^{\log_b a} \cdot \lg n)$

**حالت ۳:** اگر  $f(n) = \Omega(n^{\log_b a + \epsilon})$  که  $\epsilon > 0$  ثابت است و اگر  $af(\frac{n}{b}) \leq cf(n)$  (به این

شرط regularity گویند که معمولاً نیاز به بررسی ندارد) برای ثابت  $c < 1$  و  $n$ های به اندازه کافی بزرگ آنگاه  $T(n) = \theta(f(n))$ .

**توجه:** اگر  $f(n)$  چند جمله‌ای باشد، قضیه Master را می‌توان به زبان ساده اینطور بیان کرد که مرتبه  $f(n)$  را با  $n^{\log_b a}$  مقایسه کنید اگر  $f(n)$  رشد بیشتری داشت آنگاه  $T(n) = \theta(f(n))$ . اگر  $n^{\log_b a}$  رشد بیشتری داشت آنگاه  $T(n) = \theta(n^{\log_b a})$  و اگر هم‌رشد بودند آنگاه  $T(n) = \theta(f(n) \cdot \lg n)$ .

**مثال ۱۱:** مرتبه چند رابطه بازگشتی را بدست می‌آوریم:

a)  $T(n) = 4T(\frac{n}{4}) + n$  ,  $a = 4, b = 4, f(n) = n = O(n^{\log_4 4 - \epsilon})$

که به ازای مثلاً  $\epsilon = 1$  درست است پس  $T(n) = \theta(n^2)$

$$b) \quad T(n) = \tau T\left(\frac{n}{\tau}\right) + n^{\tau}, \quad a = b = \tau, f(n) = n = O(n^{\log_{\tau}^{\tau}})$$

حالت ۲ برقرار است پس  $T(n) = \theta(n \cdot \lg n)$

$$c) \quad T(n) = \tau T\left(\frac{n}{\tau}\right) + n^{\tau}, \quad a = b = \tau, f(n) = n^{\tau} = \Omega(n^{\log_{\tau}^{\tau+\varepsilon}})$$

که به ازای مثلاً  $\varepsilon = 1$  درست است. حال شرط  $af\left(\frac{n}{b}\right) \leq cf(n)$  را بررسی می‌کنیم:

$$\tau f\left(\frac{n}{\tau}\right) = \tau \left(\frac{n^{\tau}}{\tau^{\tau}}\right) = \frac{1}{\tau} n^{\tau} \leq cn^{\tau}$$

که به ازای مثلاً  $c = \frac{1}{\tau}$  برقرار است پس  $T(n) = \theta(n^{\tau})$  البته چون  $f(n) = n^{\tau}$  چند جمله‌ای است نیاز به بررسی شرط فوق نیست.

$$d) \quad T(n) = \tau T\left(\frac{n}{\tau}\right) + n \cdot \lg n, \quad a = b = \tau, f(n) = n \cdot \lg n$$

ظاهراً  $n \cdot \lg n$  از  $n^{\lg n^{\tau}} = n$  رشد بیشتری دارد و باید حالت ۳ برقرار باشد ولی  $n \cdot \lg n = \Omega(n^{\log_{\tau}^{\tau+\varepsilon}})$  به ازای هیچ  $\varepsilon > 0$  درست نیست. پس نمی‌توان از قضیه Master کمک گرفت.

$$e) \quad T(n) = \tau T\left(\frac{n}{\tau}\right) + n \cdot \lg n, \quad a = \tau, b = \tau, f(n) = n \cdot \lg n = O(n^{\log_{\tau}^{\tau-\varepsilon}})$$

به ازای مثلاً  $\varepsilon = 1$  درست است پس  $T(n) = \theta(n^{\tau})$

$$f) \quad T(n) = T\left(\frac{\tau n}{\tau}\right) + 1, \quad a = 1, b = \frac{\tau}{\tau}, f(n) = 1 = \theta(n^{\log_{\frac{\tau}{\tau}}^1}) = \theta(1)$$

حالت ۲ برقرار است پس  $T(n) = \theta(\lg n)$

$$g) \quad T(n) = \tau T\left(\frac{n}{\tau}\right) + n^{\tau} \cdot \lg n, \quad a = b = \tau, f(n) = n^{\tau} \cdot \lg n = \Omega(n^{\log_{\tau}^{\tau+\varepsilon}})$$

مثلاً به ازای  $\varepsilon = 1$  درست است حال شرط  $af\left(\frac{n}{b}\right) \leq cf(n)$  را بررسی می‌کنیم.

$$\tau f\left(\frac{n}{\tau}\right) = \tau \left(\frac{n^{\tau}}{\tau^{\tau}}\right) \lg \frac{n}{\tau} \leq \frac{1}{\tau} n^{\tau} \lg n$$

پس به ازای  $c = \frac{1}{\tau} < 1$  برقرار است بنابراین حالت ۳ درست است و  $T(n) = \theta(n^{\tau} \cdot \lg n)$

$$h) \quad T(n) = \tau T\left(\frac{n}{\tau}\right) + \frac{n}{\lg n}, \quad a = b = \tau, f(n) = \frac{n}{\lg n} = O(n^{\log_{\tau}^{\tau-\varepsilon}})$$

به ازای هیچ  $\varepsilon > 0$  برقرار نیست پس نمی‌توان از قضیه Master استفاده کرد.

## نکته

اگر  $T(n) = \theta(n^{\log_b^a} \cdot \lg^{k+1} n)$  و  $T(n) = aT(\frac{n}{b}) + n^{\log_b^a} \cdot \lg^k n$  و  $k \geq 0$  ثابت است آنگاه

**مثال ۱۲:** در دو رابطه بازگشتی زیر از این نکته استفاده شده است:

$$a) \quad T(n) = \sqrt[3]{n} T\left(\frac{n}{\sqrt[3]{n}}\right) + n \cdot \lg n = \theta(n \cdot \lg^3 n)$$

$$b) \quad T(n) = \sqrt[4]{n} T\left(\frac{n}{\sqrt[4]{n}}\right) + \lg^3 n = \theta(n^{\frac{3}{4}} \cdot \lg^3 n)$$

■ (دقت کنید  $\lg n!$  با  $n \lg n$  هم‌رشد است)

برخی روابط بازگشتی با تغییر متغیر به روابط قابل حل تبدیل می‌شوند.

**مثال ۱۳:** برای یافتن مرتبه  $T(n) = \sqrt[3]{n} T(\sqrt[3]{n}) + \lg n$ ، تعریف می‌کنیم  $n = 2^m$  پس:

$$T(2^m) = \sqrt[3]{2^m} T(2^{\frac{m}{3}}) + m$$

حال  $T(2^m)$  را  $F(m)$  می‌نامیم پس:

$$F(m) = \sqrt[3]{2^m} F\left(\frac{m}{3}\right) + m$$

با قضیه Master داریم  $F(m) = \theta(m \cdot \lg m)$  حال با توجه به  $n = 2^m$  داریم  $m = \lg n$  در

$$نتیجه \quad T(n) = \theta(\lg n \cdot \lg \lg n)$$

**مثال ۱۴:** برای یافتن مرتبه  $T(n) = \sqrt[3]{n} T(\sqrt[3]{n}) + n \cdot \lg n$  ابتدا طرفین را به  $n$

تقسیم می‌کنیم  $\frac{T(n)}{n} = h(n)$  حال تعریف می‌کنیم  $\frac{T(n)}{n} = \frac{\sqrt[3]{n} T(\sqrt[3]{n})}{\sqrt[3]{n}} + \lg n$  پس

$$h(n) = \sqrt[3]{n} h(\sqrt[3]{n}) + \lg n$$

$$h(n) = \theta(\lg n \cdot \lg \lg n) \quad \text{در نتیجه}$$

$$T(n) = n \cdot h(n) = \theta(n \cdot \lg n \cdot \lg \lg n)$$

## درخت بازگشت

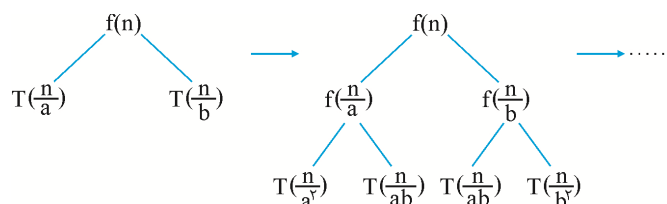
یک روش برای یافتن مرتبه برخی روابط بازگشتی، درخت بازگشت است. رابطه بازگشتی

$$T(n) = T\left(\frac{n}{a}\right) + T\left(\frac{n}{b}\right) + f(n)$$

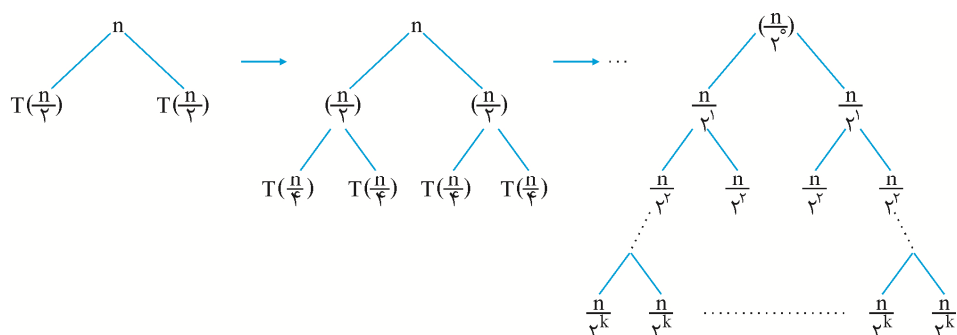
مفروض است. برای رسم درخت،  $f(n)$  را ریشه قرار می‌دهیم و  $T\left(\frac{n}{b}\right)$  و  $T\left(\frac{n}{a}\right)$  را چپ و راست ریشه. حال همین عمل را با  $T\left(\frac{n}{b}\right)$  و  $T\left(\frac{n}{a}\right)$  انجام می‌دهیم.

دقت کنید که  $T\left(\frac{n}{a}\right) = T\left(\frac{n}{a^2}\right) + T\left(\frac{n}{ab}\right) + f\left(\frac{n}{a}\right)$  و ... سپس همین عمل را با  $T\left(\frac{n}{a^2}\right)$  و ... انجام

می‌دهیم تا زمانیکه به شرط اولیه مثلاً  $T(1)$  برسیم و سپس تمام نودهای درخت را به صورت سطح به سطح با هم جمع می‌کنیم،



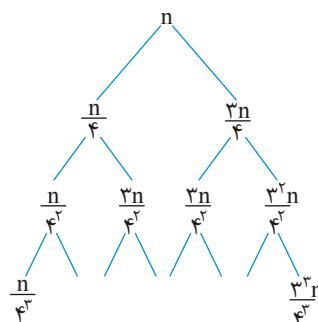
**مثال ۱۵:** مرتبه  $T(n) = 2T(\frac{n}{2}) + n = T(\frac{n}{2}) + T(\frac{n}{2}) + n$  را با درخت می‌یابیم:



درخت وقتی تمام می‌شود که  $\frac{n}{2^k} = 1$  یعنی  $k = \lg n$  این درخت دارای  $k+1$  سطح است و جمع نودهای هر سطح برابر  $n$  است پس جمع تمام نودها  $(k+1)n$  یعنی  $(\lg n + 1)n$  است پس

$$T(n) = \theta(n \cdot \lg n)$$

$$T(n) = T(\frac{n}{4}) + T(\frac{3n}{4}) + n$$



این درخت پر نیست و شاخه سمت چپ زودتر به پایان (به ۱) می‌رسد، طول شاخه سمت چپ  $\log_4 n$  و طول شاخه سمت راست  $\log_{4/3} n$  است (دقت کنید شاخه سمت راست به شکل  $\frac{3^k}{4^k} n$  حرکت می‌کند و باید

$\frac{3^k n}{4^k} = 1$  پس  $(k = \log_{\frac{4}{3}} n)$ . جمع نودهای هر سطح تا سطح  $\log_{\frac{4}{3}} n$ ، برابر  $n$  است ولی از آن بعد از  $n$  کمتر

است پس جمع همه نودها  $n \cdot \log_{\frac{4}{3}} n < T(n) < n \cdot \log_{\frac{4}{3}} n$  است پس  $T(n) = O(n \cdot \log_{\frac{4}{3}} n)$  و

$T(n) \in \Omega(n \cdot \log_{\frac{4}{3}} n)$  و چون پایه لگاریتم در رشد بی‌اهمیت است پس  $T(n) \in \theta(n \cdot \lg n)$

## نکته

رابطه بازگشتی  $T(n) = T(\alpha n) + T(\beta n) + n$  که  $\alpha$  و  $\beta$  ثابت هستند مفروض است، اگر

$\alpha + \beta = 1$  آنگاه  $T(n) = \theta(n \cdot \lg n)$  و اگر  $\alpha + \beta < 1$  آنگاه  $T(n) = \theta(n)$

**مثال ۱۷:** رابطه بازگشتی  $T(n) = T(\frac{n}{5}) + T(\frac{4n}{5}) + n$  از مرتبه  $T(n) = \theta(n)$  است چون

$\frac{1}{5} + \frac{4}{5} = \frac{5}{5} = 1 < 1$  (با درخت بازگشت اثبات کنید)

## نکته

به طور کلی  $T(n) = T(\frac{n}{a_1}) + T(\frac{n}{a_2}) + \dots + T(\frac{n}{a_k}) + \theta(n)$  اگر  $\sum_{i=1}^k \frac{1}{a_i} = 1$  آنگاه

$T(n) = \theta(n \cdot \lg n)$  و اگر  $\sum_{i=1}^k \frac{1}{a_i} < 1$  آنگاه  $T(n) = \theta(n)$

**قضیه بمب اتم!** رابطه بازگشتی  $T(n) = \sum_{i=1}^k a_i T(\frac{n}{b_i}) + f(n)$  که  $k$  ثابت،  $a_i > 0$  و  $b_i > 1$

برای هر  $i$  ثابت هستند مفروض است. مرتبه این رابطه بازگشتی از فرمول:

$$T(n) = \theta(n^p (1 + \int_1^n \frac{f(x)}{x^{p+1}} dx))$$

بدست می‌آید که  $p$  جواب منحصر به فرد معادله  $\sum_{i=1}^k \frac{a_i}{b_i^p} = 1$  است.

**مثال ۱۸:** در روابط بازگشتی زیر از این قضیه استفاده شده است.

$$a) \quad T(n) = T(\frac{3n}{4}) + T(\frac{n}{4}) + n$$

$$a_1 = a_2 = 1, \quad b_1 = \frac{4}{3}, \quad b_2 = 4, \quad f(n) = n$$

معادله  $\frac{a_1}{b_1^p} + \frac{a_2}{b_2^p} = 1$  یعنی  $(\frac{3}{4})^p + (\frac{1}{4})^p = 1$  جواب  $p = 1$  دارد.



$$T(n) = \theta(n^1 (1 + \int_1^n \frac{x}{x^2} dx)) = \theta(n \cdot \lg n)$$

پس:

توجه کنید:

$$\int_1^n \frac{x}{x^2} dx = \int_1^n \frac{1}{x} dx = \ln n$$

$$b) \quad T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{3n}{5}\right) + n$$

معادله  $\left(\frac{1}{5}\right)^p + \left(\frac{3}{5}\right)^p = 1$  جواب مشخصی ندارد ولی جوابش حتماً  $0 < p < 1$  می باشد زیرا  $\frac{1}{5} + \frac{3}{5} < 1$  است. حال انتگرال را محاسبه می کنیم.

$$\int_1^n \frac{f(x)}{x^{p+1}} dx = \int_1^n \frac{x}{x^{p+1}} dx = \int_1^n x^{-p} dx = \frac{1}{1-p} x^{1-p} \Big|_1^n = \frac{n^{1-p} - 1}{1-p} = \theta(n^{1-p})$$

$$T(n) = \theta(n^p (1 + \theta(n^{1-p}))) = \theta(n^p \cdot n^{1-p}) = \theta(n)$$

پس:

$$c) \quad T(n) = \frac{1}{4} T\left(\frac{n}{4}\right) + \frac{3}{4} T\left(\frac{3n}{4}\right) + 1$$

معادله  $\frac{1}{4} \left(\frac{1}{4}\right)^p + \frac{3}{4} \left(\frac{3}{4}\right)^p = 1$  جواب  $p = 0$  دارد پس:

$$T(n) = \theta(1 + \int_1^n \frac{1}{x} dx) = \theta(\lg n)$$

$$d) \quad T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + 1$$

معادله  $\left(\frac{1}{4}\right)^p + \left(\frac{3}{4}\right)^p = 1$  دارای جواب  $p = \lg\left(\frac{1+\sqrt{5}}{4}\right) \approx 0.69$  است (کافی است  $4^p = x$  تعریف کنید و معادله درجه ۲ را حل کنید) حاصل انتگرال را می یابیم:

$$\int_1^n \frac{1}{x^{p+1}} dx = \frac{1 - n^{-p}}{p} = \theta(1)$$

$$T(n) = \theta(n^p (1 + \theta(1))) = \theta(n^p) \approx \theta(n^{0.69})$$

پس:

### الگوریتم های بازگشتی

بسیاری از الگوریتم ها طبیعت بازگشتی دارند یعنی اگر قرار است برای  $n$  عنصر عملی انجام دهند، می توان همین عمل را بر روی تعداد کمتری از عناصر انجام داد؛ در واقع الگوریتم بازگشتی، خود را فراخوانی می کند. ما می خواهیم الگوریتم های بازگشتی را تحلیل کنیم و زمان اجرای آنها را محاسبه کنیم.

**مثال ۱۹:** با توجه به تابع بازگشتی  $f$ :

```
f(n)
{
  if(n ≤ ۱) return n * n;
  return f(n - ۱) + f(n - ۲) + n;
}
```

**الف)** مقدار  $f(4)$  را محاسبه کنید.

**ب)**  $f(4)$  چه تعداد فراخوانی دارد؟

**ج)**  $f(4)$  چه تعداد عمل جمع انجام می‌دهد؟

**د)**  $f(4)$  چه تعداد عمل ضرب انجام می‌دهد؟

**ه)** اگر  $T(n)$  تعداد فراخوانی برای  $f(n)$  باشد، برای  $T(n)$  فرمول بازگشتی بنویسید.

**و)** اگر  $T(n)$  تعداد جمع برای  $f(n)$  باشد، برای  $T(n)$  فرمول بازگشتی بنویسید.

**ز)** اگر  $T(n)$  تعداد ضرب برای  $f(n)$  باشد، برای  $T(n)$  فرمول بازگشتی بنویسید.

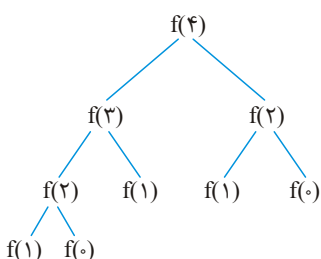
**ح)** زمان اجرای تابع  $f$  از چه مرتبه‌ای است؟

**پاسخ: الف)** برای محاسبه  $f(4)$  باید  $f(0)$  و  $f(1)$  و  $f(2)$  و  $f(3)$  را بیابیم:

$$f(0) = 0, f(1) = 1 * 1 = 1, f(2) = f(1) + f(0) + 2 = 3, f(3) = f(2) + f(1) + 3 = 7,$$

$$f(4) = f(3) + f(2) + 4 = 14$$

**ب)** درخت مقابل، تعداد فراخوانیها برای محاسبه  $f(4)$  را نشان می‌دهد، با توجه به این درخت، تعداد فراخوانیها ۹ تا است.



**ج)**  $f(n)$  اگر  $n > 1$ ، دو بار جمع را انجام می‌دهد. با توجه به

درخت فراخوانیها، ۴ تا  $f(n)$  که  $n > 1$  وجود دارد. پس ۸

بار عمل جمع انجام می‌شود.

**د)**  $f(n)$  اگر  $n \leq 1$ ، یک بار عمل ضرب را انجام می‌دهد. با

توجه به درخت فراخوانیها، ۵ تا  $f(n)$  که  $n \leq 1$  وجود

دارد، پس عمل ضرب، ۵ بار انجام می‌شود.

**ه)**  $f(n)$  باعث فراخوانی  $f(n-1)$  و  $f(n-2)$  می‌شود. اگر  $f(n)$  دارای  $T(n)$  تا فراخوانی

است، پس  $f(n-1)$  و  $f(n-2)$  به ترتیب دارای  $T(n-1)$  و  $T(n-2)$  تا فراخوانی هستند

بنابراین می‌توان نوشت  $T(n) = T(n-1) + T(n-2) + 1$  که عدد ۱ بخاطر اولین فراخوانی

یعنی فراخوانی  $f(n)$  است. این رابطه بازگشتی مرتبه ۲ است و نیاز به دو شرط اولیه دارد که

$$T(0) = T(1) = 1 \text{ زیرا به ازای } n=0 \text{ و } n=1 \text{ فقط یک فراخوانی انجام می‌شود.}$$

و) اگر فرض کنیم  $f(n)$  دارای  $T(n)$  عمل جمع است می‌توان برای  $n \geq 2$  نوشت:

$T(n) = T(n-1) + T(n-2) + 2$ . زیرا  $f(n)$  خود ۲ عمل جمع دارد و همچنین باعث فراخوانی  $f(n-1)$  و  $f(n-2)$  می‌شود که  $f(n-1)$  دارای  $T(n-1)$  عمل جمع و  $f(n-2)$  دارای  $T(n-2)$  عمل جمع است. برای  $n \leq 1$  نیز می‌توان نوشت  $T(0) = T(1) = 0$  زیرا  $f(0)$  و  $f(1)$  عمل جمع ندارند.

ز) فرض کنید  $T(n)$  تعداد عمل ضرب برای  $f(n)$  باشد. به ازای  $n \geq 2$  خود  $f(n)$  عمل ضرب ندارد و فقط باعث فراخوانی  $f(n-1)$  و  $f(n-2)$  می‌شود، پس می‌توان نوشت  $T(n) = T(n-1) + T(n-2)$ . به ازای  $n \leq 1$  نیز می‌توان نوشت  $T(0) = T(1) = 1$  زیرا  $f(0)$  و  $f(1)$  هر کدام یک عمل ضرب انجام می‌دهند.

ح) زمان اجرا برابر جمع تعداد اجرای همه خطوط است. ولی فقط مرتبه زمان اجرا خواسته شده است پس کافیت تعداد اجرای جمله اصلی را محاسبه کنیم. در این کد، جمله اصلی if است چون به ازای هر  $n$  انجام می‌شود. تعداد اجرای if نیز برابر تعداد فراخوانیهاست. و دیدیم رابطه بازگشتی تعداد فراخوانیها  $T(n) = T(n-1) + T(n-2) + 1$ ،  $T(0) = T(1) = 1$  است که اگر این رابطه را با معادله مشخصه حل کنید به  $T(n) \in \theta\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right)$  خواهید رسید. پس مرتبه این کد، نمایی است در واقع  $2^n < T(n) < 2^{\frac{n}{2}}$ .

**مثال ۲۰:** با توجه به کد زیر اگر  $T(n)$  تعداد ستاره‌های چاپ شده توسط  $f(n)$  باشد،  $T(4)$  را

بیابید. یعنی مشخص کنید به ازای  $n = 4$  چند تا ستاره (\*) چاپ می‌شود؟  
پاسخ:  $f(n-1)$  و  $f(n-2)$  به ترتیب  $T(n-1)$  و  $T(n-2)$  ستاره چاپ می‌کنند.  $write(**)$  و  $write(***)$  و  $write(*)$  به ترتیب ۲ و ۳ و ۱ ستاره چاپ می‌کنند. پس:

```
f(n)
{
  if(n > 1)
  {
    f(n-1)
    write(**)
    f(n-2)
    write(***)
    f(n-1)
    write(*)
  }
}
```

$$T(n) = T(n-1) + 2 + T(n-2) + 3 + T(n-1) + 1$$

$$= 2T(n-1) + T(n-2) + 6$$

این رابطه به ازای  $n > 1$  درست است. به ازای  $n \leq 1$  هیچ ستاره‌ای چاپ نمی‌شود، پس  $T(0) = T(1) = 0$ . برای محاسبه  $T(4)$  نیاز به  $T(2)$  و  $T(3)$  هست:

$$T(2) = 2T(1) + T(0) + 6 = 6, T(3) = 2T(2) + T(1) + 6 = 18,$$

$$T(4) = 2T(3) + T(2) + 6 = 48$$

**مثال ۲۱:** با توجه به کد زیر

```

f(n)
{
  if(n ≤ ۱) write(n)
  else{
    f(n-۲)
    write(n)
    f(n-۱)
    write(n+۱)
  }
}

```

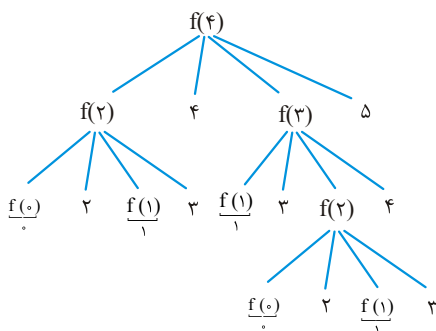
**الف)**  $f(4)$  چند تا عدد چاپ می‌کند.**ب)**  $f(4)$  به ترتیب چه اعدادی چاپ می‌کند؟

**پاسخ: الف)** اگر  $T(n)$  تعداد اعداد چاپ شده توسط  $f(n)$  باشد  
 آنگاه به ازای  $n > 1$  داریم  $T(n) = T(n-2) + 1 + T(n-1) + 1$  و  
 به ازای  $n \leq 1$  فقط یک عدد چاپ می‌شود پس  $T(0) = T(1) = 1$ ، در  
 نتیجه:

$$T(2) = T(0) + 1 + T(1) + 1 = 4$$

$$T(3) = T(1) + 1 + T(2) + 1 = 7$$

$$T(4) = T(2) + 1 + T(3) + 1 = 13$$



**ب)** به ازای  $n \leq 1$  فقط دستور  $write(n)$  اجرا می‌شود و به ازای  $n > 1$ ، دستور  $f(n-2)$  و  $write(n)$  و  $f(n-1)$  و  $write(n+1)$  اجرا می‌شوند. با درخت می‌توان ترتیب چاپ اعداد را یافت. بجای  $write(n)$  در درخت، خود  $n$  یعنی عددی که چاپ می‌شود را می‌نویسیم:

حال برگ‌های این درخت را سمت چپ (مثلاً به ترتیب پیمایش preorder) خروجی است؟  
 خروجی: ۰۲۱۳۴۱۳۰۲۱۳۴۵

**مثال ۲۲:** زمان اجرای دو تابع  $f$  و  $g$  از چه مرتبه‌ای است؟

```

f(n)
{
  if(n ≤ ۱) return n;
  return f(n-۱) + f(n-۱) + n;
}

```

```

g(n)
{
  if(n ≤ ۱) return n;
  return ۲ * f(n-۱) + n;
}

```

**پاسخ:** هر دو تابع یک عمل انجام می‌دهند ولی زمان اجرای متفاوتی دارند. در هر دو تابع جمله اصلی  $if$  است که تعداد اجرای  $if$  برابر تعداد فراخوانی‌هاست. در تابع  $f$  تعداد فراخوانی‌ها برابر

$T(n) = 2T(n-1) + 1$  می‌باشد که از مرتبه  $\theta(2^n)$  یعنی نمایی است. در تابع  $g$  تعداد فراخوانی‌ها برابر  $T(n) = T(n-1) + 1$  است که از مرتبه  $\theta(n)$  یعنی خطی است.

### تقسیم و غلبه (divide & conquer)

یک روش حل مسئله است. مسائل بزرگ به زیر مسائل کوچکتر تقسیم می‌شوند و زیر مسائل کوچک به صورت بازگشتی حل می‌شوند (یعنی آنها نیز به زیر مسائل کوچک تقسیم می‌شوند تا وقتی که مسئله آنقدر کوچک شود که حل آن بدیهی باشد) سپس با ترکیب زیر مسائل حل شده، مسئله اصلی حل می‌شود. در واقع تقسیم و غلبه با شیوه بازگشتی، مسائل را حل می‌کند.

**مثال ۲۳:** فرض کنید یک الگوریتم تقسیم و غلبه برای حل یک مسئله به اندازه  $n$  آن را به ۴ زیر مسئله به اندازه  $\frac{n}{4}$  تقسیم می‌کند (البته زیر مسائل با هم اشتراک دارند) سپس زیر مسائل را به صورت بازگشتی حل می‌کند و سپس ادغام می‌کند. اگر شکستن و ادغام کردن از مرتبه  $\theta(n^2)$  باشد، این الگوریتم تقسیم و غلبه، از چه مرتبه‌ای است.

**پاسخ:** فرض کنید  $T(n)$  زمان اجرا باشد آنگاه می‌توان نوشت  $T(n) = 4T\left(\frac{n}{4}\right) + \theta(n^2)$  که طبق قضیه master از مرتبه  $\theta(n^2 \lg n)$  است.

### ضرب ماتریس‌ها

می‌خواهیم ماتریس  $A = [a_{ij}]_{m \times n}$  را در  $B = [b_{ij}]_{n \times p}$  ضرب کنیم و ماتریس  $C = [c_{ij}]_{m \times p}$  را تولید کنیم یعنی  $C = A \times B$ . درایه  $c_{ij}$  از ماتریس  $C$  از ضرب سطر  $i$ ام ماتریس  $A$  در ستون  $j$ ام ماتریس  $B$  بدست می‌آید یعنی  $c_{ij} = a_{i1}.b_{1j} + a_{i2}.b_{2j} + \dots + a_{in}.b_{nj} = \sum_{k=1}^n a_{ik}.b_{kj}$  که  $1 \leq i \leq m$  و  $1 \leq j \leq p$ . پس برای محاسبه  $C$  به سه حلقه for نیاز است. الگوریتم عادی ضرب دو ماتریس به شکل زیر است:

```
for(i=1 to m)
  for(j=1 to p)
    for(k=1 to n)
      cij = cij + aik * bkj
```

مقدار اولیه درایه‌های ماتریس  $C$ ، صفر فرض شده است. واضح است که در این کد تعداد عمل  $+$  و تعداد عمل  $*$  هر کدام برابر  $mpn$  است. ولی اگر مقدار اولیه  $c_{ij}$  را بجای صفر،  $a_{i1} * b_{1j}$  قرار دهیم و حلقه سوم را از  $k=2$  شروع کنیم آنگاه تعداد عمل  $+$  به  $m(n-1)p$  کاهش می‌یابد. به

هر حال مرتبه کد نوشته شده برای ضرب دو ماتریس  $\theta(mnp)$  است که اگر ماتریس‌ها  $n \times n$  باشند، مرتبه  $\theta(n^3)$  می‌شود.

حال می‌خواهیم با روش تقسیم و غلبه ضرب  $C = A \times B$  را انجام دهیم. فرض می‌کنیم ماتریس‌ها  $n \times n$  هستند و  $n$  توانی از ۲ است. هر ماتریس  $n \times n$  را می‌توان به ۴ ماتریس  $\frac{n}{2} \times \frac{n}{2}$  تقسیم کرد:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}, C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

حال برای محاسبه  $C$  می‌توان روابط زیر را نوشت:

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}, C_{12} = A_{11}B_{12} + A_{12}B_{22}, C_{21} = A_{21}B_{11} + A_{22}B_{21},$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

با توجه به این روابط ضرب دو ماتریس  $n \times n$ ، به هشت ضرب (فراخوانی) دو ماتریس  $\frac{n}{2} \times \frac{n}{2}$

و همچنین ۴ عمل جمع دو ماتریس با اندازه  $\frac{n}{2} \times \frac{n}{2}$  نیاز دارد. هر فراخوانی بازگشتی  $T\left(\frac{n}{2}\right)$

زمان می‌خواهد و هر جمع ماتریس  $\frac{n}{2} \times \frac{n}{2}$  زمان می‌خواهد. پس زمان اجرای تقسیم و غلبه برای

ضرب دو ماتریس  $n \times n$  برابر  $T(n) = 8T\left(\frac{n}{2}\right) + 4\left(\frac{n^2}{4}\right)$  است که طبق قضیه master از مرتبه

$\theta(n^3)$  است و هیچ مزیتی نسبت به روش غیر بازگشتی ضرب دو ماتریس ندارد.

ولی شخصی به اسم استراسن، روابط مربوط به محاسبه  $C_{11}, C_{12}, C_{21}, C_{22}$  را طوری تعریف

کرد که بجای ۸ فراخوانی تعداد ۷ فراخوانی با اندازه  $\frac{n}{2} \times \frac{n}{2}$  انجام داد و بجای ۴ عمل جمع تعداد ۱۸

عمل جمع / تفریق ماتریس با اندازه  $\frac{n}{2} \times \frac{n}{2}$  انجام شد. پس زمان اجرای ضرب دو ماتریس با روش

تقسیم و غلبه استراسن برابر  $T(n) = 7T\left(\frac{n}{2}\right) + 18\left(\frac{n^2}{4}\right)$  است که طبق قضیه master از مرتبه

$\theta(n^{\lg 7})$  است که حدوداً برابر  $\theta(n^{2.81})$  است. روابط روش استراسن که نیاز به حفظ کردن

نیست عبارتند از:

$$C_{11} = P + S - T + V, C_{12} = R + T, C_{21} = Q + S, C_{22} = P + R - Q + U$$

که داریم:

$$P = (A_{11} + A_{22})(B_{11} + B_{22}), Q = (A_{21} + A_{22})B_{11}, R = A_{11}(B_{12} - B_{22}),$$

$$S = A_{22}(B_{21} - B_{11}), T = (A_{11} + A_{12})B_{22}, U = (A_{21} - A_{11})(B_{11} + B_{12}),$$

$$V = (A_{12} - A_{22})(B_{21} + B_{22})$$

اگر شرط پایان در الگوریتم استراسن  $n=1$  باشد یعنی آنقدر فراخوانی بازگشتی انجام شود که به ماتریس‌های یک در یک برسیم و ماتریس‌های یک در یک را با روش عادی ضرب کنیم آنگاه اگر  $T(n)$  تعداد فراخوانی برای ضرب دو ماتریس  $n \times n$  باشد می‌توان نوشت  $T(n) = 7T\left(\frac{n}{7}\right) + 1$  و  $T(1) = 1$ . مثلاً برای ضرب دو ماتریس  $4 \times 4$  تعداد فراخوانی برابر است با:

$$T(4) = 7T(2) + 1 = 7[7T(1) + 1] + 1 = 7 \times 8 + 1 = 57$$

و اگر  $T(n)$  تعداد عمل ضرب عددی دو ماتریس  $n \times n$  با روش استراسن باشد آنگاه  $T(n) = 7T\left(\frac{n}{7}\right) + 1$  و  $T(1) = 1$  مثلاً تعداد ضرب عددی برای ضرب دو ماتریس  $4 \times 4$  برابر است با:

$$T(4) = 7T(2) = 7 \times 7T(1) = 7 \times 7 \times 1 = 49$$

و همچنین اگر  $T(n)$  تعداد جمع و تفریق استراسن برای ضرب دو ماتریس  $n \times n$  باشد آنگاه  $T(n) = 7T\left(\frac{n}{7}\right) + 18\left(\frac{n^2}{4}\right)$  و  $T(1) = 0$  (چون به ازای  $n=1$  جمع و تفریق نداریم).

**توجه:** ثابت می‌شود که ضرب دو ماتریس  $n \times n$  از مرتبه  $\Omega(n^2)$  است یعنی زمان کمتر از  $n^2$  امکان‌پذیر نیست. البته هنوز کسی نتوانسته با مرتبه  $\theta(n^2)$  دو ماتریس  $n \times n$  را در هم ضرب کند ولی الگوریتم‌هایی با مرتبه  $\theta(n^{2/3})$  وجود دارد.

### ضرب دو چند جمله‌ای

می‌خواهیم دو چند جمله‌ای درجه  $n-1$  به نامهای  $p(x)$  و  $q(x)$  را در هم ضرب کنیم و چند جمله‌ای درجه  $2n-2$  به نام  $R(x)$  را تولید کنیم:

$$P(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1}, \quad q(x) = b_0 + b_1x + b_2x^2 + \dots + b_{n-1}x^{n-1},$$

$$R(x) = P(x).q(x) = c_0 + c_1x + c_2x^2 + \dots + c_{2n-2}x^{2n-2}$$

برای نمایش چند جمله‌ای‌ها می‌توان ضرایب را در آرایه ذخیره کرد. پس ۳ تا آرایه  $A[0 \dots n-1]$  و  $B[0 \dots n-1]$  و  $C[0 \dots 2n-2]$  تعریف می‌کنیم و ضرایب  $P(x)$  و  $q(x)$  را به ترتیب در  $A$  و  $B$  ذخیره می‌کنیم و ضرایب جواب  $R(x)$  را در  $C$  محاسبه می‌کنیم:

$$A \begin{matrix} 0 & 1 & 2 & & n-1 \\ a_0 & a_1 & a_2 & \dots & a_{n-1} \end{matrix}, B \begin{matrix} 0 & 1 & 2 & & n-1 \\ b_0 & b_1 & b_2 & \dots & b_{n-1} \end{matrix}$$

$$C \begin{matrix} 0 & 1 & 2 & & 2n-2 \\ c_0 & c_1 & c_2 & \dots & c_{2n-2} \end{matrix}$$

ضرایب جواب یعنی  $R(x)$  را می‌توان با روابط زیر یافت:

$$c_0 = a_0 b_0, c_1 = a_0 b_1 + a_1 b_0, c_2 = a_0 b_2 + a_1 b_1 + a_2 b_0,$$

$$c_3 = a_0 b_3 + a_1 b_2 + a_2 b_1 + a_3 b_0, \dots$$

$$c_k = \sum_{i+j=k} a_i b_j \text{ به طور کلی } c_k \text{ برابر جمع همه } a_i b_j \text{ هایی است که } i+j=k \text{ یعنی}$$

پس با فرض اینکه درایه‌های آرایه  $c$  صفر هستند می‌توان الگوریتم ضرب دو چند جمله‌ای  $R(x) = P(x) \times q(x)$  را با دو حلقه for نوشت که از مرتبه  $\theta(n^2)$  است.

```
for(i=0 to n-1)
  for(j=0 to n-1)
    c[i+j] = c[i+j] + A[i]*B[j]
```

**مثال ۲۴:** مراحل الگوریتم فوق را برای  $P(x) = 1 + x + 3x^2 - 4x^3$  و  $q(x) = 1 + 2x - 5x^2 - 3x^3$  نشان دهید.

**پاسخ:** چند جمله‌ای‌ها درجه ۳ هستند یعنی  $n-1=3$  و مقادیر آرایه‌های  $C, B, A$  عبارتند از:

$$A \begin{bmatrix} 0 & 1 & 2 & 3 \\ 1 & 1 & 3 & -4 \end{bmatrix}, B \begin{bmatrix} 0 & 1 & 2 & 3 \\ 1 & 2 & -5 & -3 \end{bmatrix}, C \begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

در اولین اجرای حلقه for اول یعنی  $i=0$ ، مقدار  $j$  برابر  $3, 2, 1, 0$  می‌شود و داریم:

$$i=0, j=0 \rightarrow c[0] = c[0] + A[0].B[0] = 1$$

$$i=0, j=1 \rightarrow c[1] = c[1] + A[0].B[1] = 2$$

$$i=0, j=2 \rightarrow c[2] = c[2] + A[0].B[2] = -5$$

$$i=0, j=3 \rightarrow c[3] = c[3] + A[0].B[3] = -3$$

$$C \begin{bmatrix} 1 & 2 & -5 & -3 & 0 & 0 & 0 \end{bmatrix}$$

در دومین اجرای حلقه for اول یعنی  $i=1$ ، مقدار  $j$  برابر  $3, 2, 1, 0$  می‌شود و داریم:

$$i=1, j=0 \rightarrow c[1] = c[1] + A[1].B[0] = 2 + 1 = 3$$

$$i=1, j=1 \rightarrow c[2] = c[2] + A[1].B[1] = -5 + 2 = -3$$

$$i=1, j=2 \rightarrow c[3] = c[3] + A[1].B[2] = -3 + -5 = -8$$

$$i=1, j=3 \rightarrow c[4] = c[4] + A[1].B[3] = 0 + -3 = -3$$

$$C \begin{bmatrix} 1 & 3 & -3 & -8 & -3 & 0 & 0 \end{bmatrix}$$



در سومین اجرای حلقه for اول یعنی  $i = 2$ ، مقدار  $j$  برابر  $3, 2, 1, 0$  می شود و داریم:

$$\begin{aligned} i = 2, j = 0 &\rightarrow c[2] = c[2] + A[2].B[0] = 0 \\ i = 2, j = 1 &\rightarrow c[3] = c[3] + A[2].B[1] = -2 \\ i = 2, j = 2 &\rightarrow c[4] = c[4] + A[2].B[2] = -18 \\ i = 2, j = 3 &\rightarrow c[5] = c[5] + A[2].B[3] = -9 \end{aligned}$$

$$C \begin{bmatrix} 1 & 3 & 0 & -2 & -18 & -9 & 0 \end{bmatrix}$$

در چهارمین و آخرین اجرای حلقه for اول یعنی  $i = 2$ ، مقدار  $j$  برابر  $3, 2, 1, 0$  می شود و داریم:

$$\begin{aligned} i = 3, j = 0 &\rightarrow c[3] = c[3] + A[3].B[0] = -6 \\ i = 3, j = 1 &\rightarrow c[4] = c[4] + A[3].B[1] = -26 \\ i = 3, j = 2 &\rightarrow c[5] = c[5] + A[3].B[2] = 11 \\ i = 3, j = 3 &\rightarrow c[6] = c[6] + A[3].B[3] = 12 \end{aligned}$$

$$C \begin{bmatrix} 1 & 3 & 0 & -6 & -26 & 11 & 12 \end{bmatrix}$$

بنابراین حاصلضرب  $p(x).q(x)$  برابر  $1 + 3x - 6x^2 - 26x^3 + 11x^4 + 12x^5$  است. حال می خواهیم با روش تقسیم و غلبه  $R(x) = p(x).q(x)$  را انجام دهیم یعنی می خواهیم ضرب دو چند جمله ای با درجه  $n-1$  را به ضرب چند جمله ای های با درجه  $\frac{n}{r}-1$  تبدیل کنیم.

برای این منظور  $p(x), q(x)$  را به شکل های زیر می نویسیم:

$$\begin{aligned} p(x) &= \underbrace{(a_0 + a_1x + \dots + a_{\frac{n}{r}-1}x^{\frac{n}{r}-1})}_{p_L(x)} + \underbrace{(a_{\frac{n}{r}} + a_{\frac{n}{r}+1}x + \dots + a_{n-1}x^{\frac{n}{r}-1})x^{\frac{n}{r}}}_{p_R(x)} \\ &= p_L(x) + p_R(x)x^{\frac{n}{r}} \\ q(x) &= \underbrace{(b_0 + b_1x + \dots + b_{\frac{n}{r}-1}x^{\frac{n}{r}-1})}_{q_L(x)} + \underbrace{(b_{\frac{n}{r}} + b_{\frac{n}{r}+1}x + \dots + b_{n-1}x^{\frac{n}{r}-1})x^{\frac{n}{r}}}_{q_R(x)} \\ &= q_L(x) + q_R(x)x^{\frac{n}{r}} \end{aligned}$$

بنابراین  $R(x) = p(x).q(x)$  را می توان به شکل زیر نوشت:

$$\begin{aligned} R(x) &= (p_L(x) + p_R(x)x^{\frac{n}{r}})(q_L(x) + q_R(x)x^{\frac{n}{r}}) \\ &= p_L(x).q_L(x) + (p_L(x).q_R(x) + p_R(x).q_L(x))x^{\frac{n}{r}} + p_R(x).q_R(x).x^n \end{aligned}$$

اگر  $T(n)$  زمان اجرای محاسبه  $p(x).q(x)$  باشد، آنگاه  $p_L(x).q_L(x)$  و  $p_L(x).q_R(x)$  و  $p_R(x).q_L(x)$  و  $p_R(x).q_R(x)$  هر یک به زمان  $T\left(\frac{n}{4}\right)$  نیاز دارند چون هر یک نیاز به فراخوانی بازگشتی دارند که بجای  $n$  مقدار  $\frac{n}{4}$  را قرار داده‌اند. همچنین عملیات جمع، و ضرب در  $x^n$  و  $x^{\frac{n}{4}}$  به زمان  $\theta(n)$  نیاز دارد چون جمع در واقع جمع دو آرایه است که از مرتبه  $\theta(n)$  است و ضرب در  $x^n$  و  $x^{\frac{n}{4}}$  نیز در واقع شیفت آرایه است که از مرتبه  $\theta(n)$  است. پس می‌توان برای زمان اجرای روش تقسیم و غلبه رابطه بازگشتی  $T(n) = 4T\left(\frac{n}{4}\right) + \theta(n)$  را نوشت که از مرتبه  $\theta(n^2)$  است و مزیتی نسبت به روش غیر تقسیم و غلبه‌ای ندارد. ولی می‌توان روش تقسیم و غلبه را طوری انجام داد که بجای ۴ تا ۳ تا فراخوانی بازگشتی انجام شود. به روابط زیر برای محاسبه  $R(x)$  توجه کنید:

$$\begin{aligned} R(x) &= p_L \cdot q_L + (p_L \cdot q_R + p_R \cdot q_L) x^{\frac{n}{4}} + p_R \cdot q_R \cdot x^n \\ &= p_L \cdot q_L + [(p_L + p_R) \cdot (q_L + q_R) - p_L q_L - p_R q_R] x^{\frac{n}{4}} + p_R q_R x^n \end{aligned}$$

همانطور که ملاحظه می‌کنید فقط سه فراخوانی بازگشتی برای  $p_L q_L$  و  $p_R q_R$  و  $(p_L + p_R) \cdot (q_L + q_R)$  نیاز است البته تعداد جمع و تفریق بیشتر شده است که تاثیری در مرتبه ندارد و حال برای زمان اجرا می‌توان رابطه  $T(n) = 3T\left(\frac{n}{4}\right) + \theta(n)$  را نوشت که از مرتبه  $\theta(n^{\lg 3})$  است که حدوداً برابر  $\theta(n^{1.58})$  است.

**مثال ۲۵:** با فرض  $p(x) = 1 + x + 3x^2 - 4x^3$  و  $q(x) = 1 + 2x - 5x^2 - 3x^3$  حاصلضرب  $R(x) = p(x).q(x)$  را با روش (a) تقسیم و غلبه معمولی  $\theta(n^2)$  و (b) تقسیم و غلبه سریع  $\theta(n^{\lg 3})$  نشان دهید.

**پاسخ: (a)**

$$p(x) = (1 + x) + (3 - 4x)x^2 = p_L + p_R \cdot x^2$$

$$q(x) = (1 + 2x) + (-5 - 3x)x^2 = q_L + q_R \cdot x^2$$

$$\begin{aligned} R(x) &= p(x).q(x) = p_L q_L + [p_L q_R + p_R q_L] x^2 + p_R q_R x^4 \\ &= (1 + x)(1 + 2x) + [(1 + x)(-5 - 3x) + (3 - 4x)(1 + 2x)] x^2 + (3 - 4x)(-5 - 3x) x^4 \\ &= (1 + 3x + 2x^2) + [(-2 - 6x - 11x^2)] x^2 + (-15 + 11x + 12x^2) x^4 \\ &= 1 + 3x - 6x^3 - 26x^4 + 11x^5 + 12x^6 \end{aligned}$$

لازم به ذکر است محاسبه مثلاً  $p_L \cdot q_L = (1 + x)(1 + 2x)$  خود نیاز به همین عملیات یعنی ۴ تا فراخوانی بازگشتی دارد که نشان نداده‌ایم.

(b)

$$\begin{aligned}
 R(x) &= p(x).q(x) = p_L q_L + [(p_L + p_R).(q_L + q_R) - p_L q_L - p_R q_R]x^2 + p_R q_R x^4 \\
 &= (1+x)(1+2x) + [(4-3x)(-4-x) - (1+x)(1+2x) - (3-4x)(-5-3x)]x^2 \\
 &\quad + (3-4x)(-5-3x)x^4 \\
 &= (1+3x+2x^2) + [-2-6x-11x^2]x^2 + (-15+11x+12x^2)x^4 \\
 &= 1+3x-6x^3-26x^4+11x^5+12x^6
 \end{aligned}$$

### ضرب اعداد بزرگ

منظور از عدد بزرگ عددی است که برای آن "نوع داده‌ای" وجود ندارد یعنی عددی که تعداد ارقامش زیاد است. می‌توان چنین عددی را با آریه نشان داد یعنی هر رقم را در یک خانه از آرایه ذخیره کرد. در این صورت تمام نکات ضرب دو عدد بزرگ همان نکات ضرب دو چند جمله‌ای است ولی مجدداً بررسی می‌کنیم. فرض کنید  $U$  و  $V$  در عدد  $n$  رقمی هستند و می‌خواهیم حاصلضرب  $U.V$  را بیابیم. می‌توان  $U$  و  $V$  را به شکل  $U = x \times 10^{\lfloor \frac{n}{2} \rfloor} + y$  و  $V = w \times 10^{\lfloor \frac{n}{2} \rfloor} + z$  نوشت که  $y$  و  $z$  هر کدام  $\lfloor \frac{n}{2} \rfloor$  رقمی و  $x$  و  $w$  هر کدام  $\lceil \frac{n}{2} \rceil$  رقمی هستند. مثلاً عدد هفت رقمی ۹۴۲۳۷۲۳ را می‌توان به شکل  $9423 \times 10^3 + 723$  نوشت. پس می‌توان حاصلضرب  $U.V$  را به شکل زیر نوشت (فرض کنید  $m = \lfloor \frac{n}{2} \rfloor$ ):

$$U.V = (x \times 10^m + y).(w \times 10^m + z) = xw \times 10^{2m} + (xz + wy) \times 10^m + yz$$

اگر ضرب  $U.V$  به زمان  $T(n)$  نیاز داشته باشد پس ضرب  $xw$  و  $xz$  و  $wy$  و  $yz$  هر کدام به زمان  $T(\frac{n}{2})$  نیاز دارند و همچنین عملیات جمع و ضرب در  $10^m$  و  $10^{2m}$  را می‌توان با مرتبه  $\theta(n)$  انجام داد پس با توجه به این رابطه تقسیم و غلبه می‌توان نوشت  $T(n) = 4T(\frac{n}{2}) + \theta(n) = \theta(n^2)$ . ولی می‌توان تقسیم و غلبه سریعتری را ارائه داد که بجای ۴ تا ۳ تا فراخوانی بازگشتی انجام دهد:

$$\begin{aligned}
 U.V &= xw \times 10^{2m} + (xz + wy) \times 10^m + yz \\
 &= xw \times 10^{2m} + [(x+y)(w+z) - xw - yz] \times 10^m + yz
 \end{aligned}$$

با توجه به این رابطه ۳ تا فراخوانی  $xw$  و  $yz$  و  $(x+y)(w+z)$  لازم است پس زمان اجرا برابر  $T(n) = 3T(\frac{n}{2}) + \theta(n)$  است که از مرتبه  $\theta(n^{\lg 3})$  است.

## مسائل معروف بازگشتی

## ۱- مجموع ۱ تا n

$$\text{sum}(n) = \underbrace{1 + 2 + \dots + n}_{\text{sum}(n-1)} + n = \text{sum}(n-1) + n, \quad \text{sum}(0) = 0$$

زمان اجرای این تابع  $\theta(n)$  است زیرا زمان اجرای این تابع با تعداد فراخوانی‌ها مساوی است که  $\theta(n)$  تاست. دقت کنید خود این تابع  $\theta(n^2)$   $1 + 2 + \dots + n = \frac{n(n+1)}{2} = \theta(n^2)$  را محاسبه می‌کند ولی این ربطی به زمان اجرای تابع ندارد.

## ۲- فاکتوریل

$$\text{fact}(n) = n! = n * (n-1)! = n * \text{fac}(n-1), \quad \text{fact}(0) = 1$$

زمان اجرای این تابع نیز  $\theta(n)$  است.

## ۳- توان

$$\text{pow}(m, n) = m^n = m * m^{n-1} = m * \text{pow}(m, n-1), \quad \text{pow}(m, 0) = 1$$

زمان اجرای این تابع نیز  $\theta(n)$  است زیرا  $\theta(n)$  بار فراخوانی دارد.

البته برای توان می‌توان تابعی با زمان  $\theta(\lg n)$  نیز نوشت. برای nهای زوج

$$\text{pow}(m, n) = (m^{\lfloor \frac{n}{2} \rfloor})^2 * m \quad \text{و برای nهای فرد} \quad \text{pow}(m, n) = (m^{\frac{n}{2}})^2$$

پس:

$$\text{pow}(m, n) = \begin{cases} \text{pow}^2(m, \frac{n}{2}) & n \neq 0 \text{ زوج} \\ m * \text{pow}^2(m, \lfloor \frac{n}{2} \rfloor) & n \text{ فرد} \\ 1 & n = 0 \end{cases}$$

زمان اجرای این تابع برای n زوج  $T(n) = T(\frac{n}{2}) + 1$  و برای n فرد  $T(n) = T(\lfloor \frac{n}{2} \rfloor) + 1$

است که در هر صورت  $\theta(\lg n)$  می‌باشد.

## ۴- خارج قسمت تقسیم

برای محاسبه  $\text{div}(m, n) = \left\lfloor \frac{m}{n} \right\rfloor$  می‌توان از تابع زیر کمک گرفت:

$$\text{div}(m, n) = \begin{cases} 0 & m < n \\ \text{div}(m - n, n) + 1 & m \geq n \end{cases}$$

تعداد فراخوانی این تابع  $\left\lfloor \frac{m}{n} \right\rfloor + 1$  است. مثلاً برای محاسبه  $\text{div}(200, 6)$  تعداد فراخوانی  $\left\lfloor \frac{200}{6} \right\rfloor + 1 = 34$  است.

## ۵- باقیمانده تقسیم

برای محاسبه  $m \bmod n$  می‌توان از تابع زیر کمک گرفت:

$$\text{mod}(m, n) = \begin{cases} m & m < n \\ \text{mod}(m - n, n) & m \geq n \end{cases}$$

تعداد فراخوانی این تابع نیز  $\left\lfloor \frac{m}{n} \right\rfloor + 1$  است.

## ۶- بزرگترین مقسوم‌علیه مشترک

$$\text{gcd}(m, n) = \begin{cases} m & n = 0 \\ \text{gcd}(n, m \bmod n) & n \neq 0 \end{cases}$$

## ۷- ترکیب

برای محاسبه ترکیب  $\binom{m}{n}$  می‌توان از فرمول پاسکال استفاده کرد  $(m \geq n)$ ،

$$\text{comb}(m, n) = \begin{cases} 1, & m = n \text{ or } n = 0 \\ \text{comb}(m-1, n-1) + \text{comb}(m-1, n), & \text{otherwise} \end{cases}$$

می‌توان بررسی کرد که تعداد فراخوانی  $\binom{m}{n} - 1$  است و تعداد باری که عمل + انجام می‌شود

$$\binom{m}{n} - 1 \text{ است.}$$

## ۸- برج‌های هانوی

سه میله با شماره‌های ۱ و ۲ و ۳ مفروض است. داخل یکی از میله‌ها  $n$  دیسک با اندازه‌های متفاوت از بزرگ به کوچک روی هم قرار گرفته‌اند (دیسک بزرگ پایین است) می‌خواهیم این  $n$  دیسک را با کمک یکی از میله‌ها به میله سوم منتقل کنیم به شرطی که اولاً هر بار فقط یک

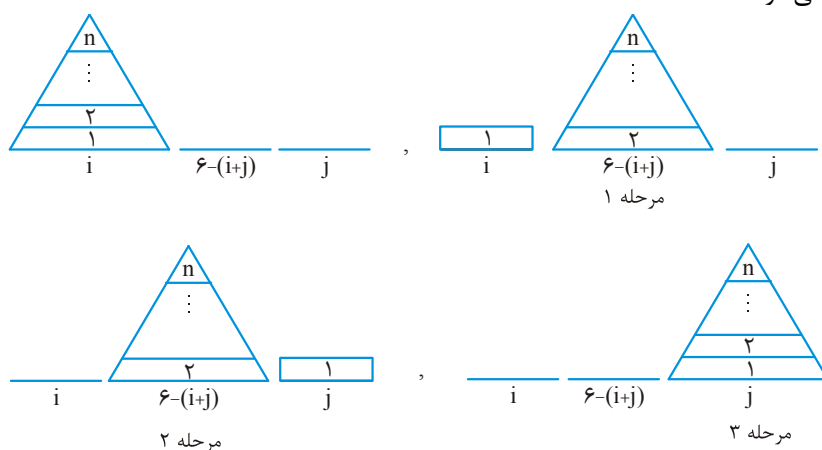
دیسک منتقل شود ثانیاً هیچگاه دیسک بزرگ روی کوچک قرار نگیرد. اگر شماره میله مبدا و مقصد به ترتیب  $i$  و  $j$  باشد، شماره میله کمکی  $6 - (i + j)$  است. برای حل این مسئله باید ۳ مرحله زیر انجام شوند:

(۱) انتقال  $n - 1$  دیسک از میله  $i$  با کمک میله  $j$  به میله  $6 - (i + j)$

(۲) انتقال یک دیسک از میله  $i$  به میله  $j$

(۳) انتقال  $n - 1$  دیسک از میله  $6 - (i + j)$  با کمک میله  $i$  به میله  $j$

دقت کنید مراحل ۱ و ۳ به صورت بازگشتی هستند یعنی هر کدام از این مراحل به ۳ مرحله تقسیم می‌شوند.



```

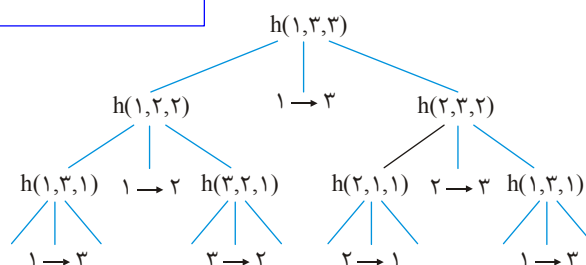
hanoi(i, j, n)
{
    if (n > 0)
    {
        hanoi(i, 6 - (i + j), n - 1);
        write(i → j);
        hanoi(6 - (i + j), j, n - 1);
    }
}

```

الگوریتم هانوی را می‌توان به شکل زیر نوشت:

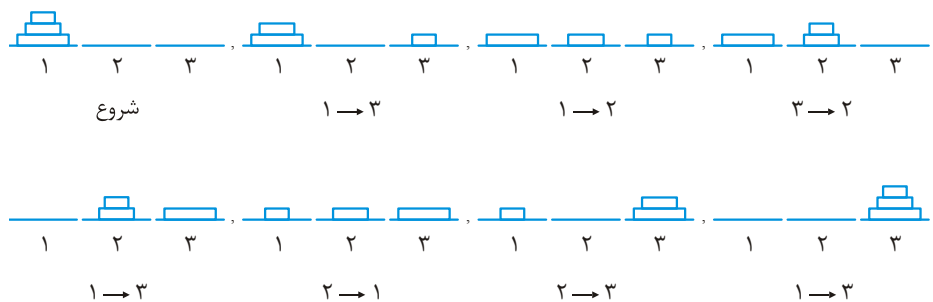
$i$  و  $j$  شماره مبدا و مقصد.  $n$  تعداد دیسک است. به ازای  $n = 3$  این الگوریتم را آزمایش می‌کنیم (نام تابع را  $h$  می‌گذاریم)

میله مبدا را ۱ و مقصد را ۳ و کمکی را ۲ در نظر می‌گیریم:



حال از چپ به راست، برگ‌ها را چاپ می‌کنیم پس خروجی عبارت است از (چپ به راست)

$1 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2, 1 \rightarrow 3, 2 \rightarrow 1, 2 \rightarrow 3, 1 \rightarrow 3$



به ازای  $n=3$ ، ۷ انتقال انجام شد. اگر  $T(n)$  تعداد انتقال به ازای  $n$  باشد، با توجه به مراحل برج‌های هانوی  $T(n) = 2T(n-1) + 1$  و  $T(0) = 0$  که با حل این رابطه  $T(n) = 2^n - 1 = \theta(2^n)$ .

## خلاصه فصل

۱- برای حل رابطه بازگشتی ناهمگن خطی ضریب ثابت مرتبه  $k$ ،

$p(n)$  هستند و  $p(n)$  چندجمله‌ای درجه  $d$  است، معادله مشخصه تشکیل می‌دهیم که عبارت است از  $(x^k - c_1 x^{k-1} - c_2 x^{k-2} - \dots - c_k)(x - b)^{d+1} = 0$ . اگر این معادله دو ریشه متمایز حقیقی  $x_1$  و  $x_2$  داشته باشد، جواب رابطه بازگشتی  $a_n = \alpha_1 (x_1)^n + \alpha_2 (x_2)^n$  است. اگر سه ریشه حقیقی متمایز  $x_1$  و  $x_2$  و  $x_3$  داشته باشد، جواب رابطه بازگشتی  $a_n = \alpha_1 (x_1)^n + \alpha_2 (x_2)^n + \alpha_3 (x_3)^n$  است. اگر ریشه مضاعف  $x_1 = x_2$  داشته باشد، جواب رابطه بازگشتی  $a_n = (\alpha_1 + \alpha_2 n)(x_1)^n$  است. اگر ریشه مرتبه ۳،  $x_1 = x_2 = x_3$  داشته باشد، جواب رابطه بازگشتی  $a_n = (\alpha_1 + \alpha_2 n + \alpha_3 n^2)(x_1)^n$  است.

۲- در رابطه بازگشتی  $T(n) = aT(n/b) + c$  که  $a$  و  $b$  و  $c$  ثابت هستند. اگر  $a \neq 1$  آنگاه

$$T(n) \in \theta(n^{\frac{\log a}{\log b}}) \text{ و اگر } a = 1 \text{ و } c > 0 \text{ آنگاه } T(n) \in \theta(n)$$

۳- در رابطه بازگشتی  $T(n) = aT(n/b) + \theta(n^k)$  که  $a$  و  $b$  و  $k$  ثابت هستند و  $a \geq 1$  و  $b > 1$ ، با

کمک قضیه Master اگر  $a > b^k$  آنگاه  $T(n) \in \theta(n^{\log_b a})$ ، اگر  $a < b^k$  آنگاه  $T(n) \in \theta(n^k)$ ، اگر  $a = b^k$  آنگاه  $T(n) \in \theta(n^k \lg n)$ .

۴- در رابطه بازگشتی  $T(n) = aT(n/b) + n^{\log_b a} \lg^k n$  که  $a$  و  $b$  و  $k$  ثابت هستند و  $a \geq 1$  و

$$T(n) \in \theta(n^{\log_b a} \lg^{k+1} n) \text{ و } b > 1 \text{ و } k \geq 0$$

۵- در رابطه بازگشتی  $T(n) = T(n/a_1) + T(n/a_2) + \dots + T(n/a_k) + \theta(n)$  اگر  $\frac{1}{a_1} + \frac{1}{a_2} + \dots + \frac{1}{a_k} = 1$

$$T(n) \in \theta(n \lg n) \text{ و اگر } \frac{1}{a_1} + \frac{1}{a_2} + \dots + \frac{1}{a_k} < 1 \text{ آنگاه } T(n) \in \theta(n)$$

۶- مرتبه رابطه بازگشتی  $T(n) = a_1 T(n/b_1) + a_2 T(n/b_2) + \dots + a_k T(n/b_k) + f(n)$  که  $a_i$  و

$b_i$ ها ثابت هستند، برابر است با

$$T(n) = \theta \left( n^p \left( 1 + \int_1^n \frac{f(x)}{x^{p+1}} dx \right) \right)$$

که  $p$  جواب معادله  $\frac{a_1}{b_1^p} + \frac{a_2}{b_2^p} + \dots + \frac{a_k}{b_k^p} = 1$  است.



۷- مرتبه ضرب دو ماتریس  $n \times n$  با روش تقسیم و غلبه‌ای استراسن برابر  $\theta(n^{\lg 7}) = \theta(n^{2.81})$  است ولی روش‌های با مرتبه کمتر از استراسن نیز وجود دارد. به طور کلی مرتبه ضرب دو ماتریس  $n \times n$   $\Omega(n^2)$  است ولی هنوز الگوریتمی با مرتبه  $\theta(n^2)$  پیدا نشده است.

۸- اگر  $T(n)$  زمان اجرای استراسن باشد آنگاه  $T(n) = 7T(\frac{n}{7}) + 18(\frac{n^2}{4})$  زیرا استراسن، ۷ فراخوانی بازگشتی و ۱۸ عمل جمع و تفریق ماتریس‌های با اندازه  $\frac{n}{7}$  را انجام می‌دهد.

۹- اگر  $T(n)$  تعداد فراخوانی استراسن باشد  $T(n) = 7T(\frac{n}{7}) + 1$  و  $T(1) = 1$  با فرض اینکه شرط پایان در استراسن،  $n = 1$  باشد.

۱۰- اگر  $T(n)$  تعداد عمل ضرب عددی استراسن باشد  $T(n) = 7T(\frac{n}{7})$  و  $T(1) = 1$  با فرض اینکه شرط پایان در استراسن،  $n = 1$  باشد.

۱۱- برای ضرب دو چندجمله‌ای با درجه  $n-1$  (یا ضرب دو عدد بزرگ  $n$  رقمی)، با روش تقسیم و غلبه عادی، نیاز به ۴ فراخوانی بازگشتی و تعدادی عمل جمع و شیفست است و زمان اجرای آن  $T(n) = 4T(\frac{n}{2}) + \theta(n) = \theta(n^2)$  است. ولی با روش تقسیم و غلبه سریع نیاز به ۳ فراخوانی بازگشتی و تعدادی عمل جمع و شیفست است و زمان اجرای آن  $T(n) = 3T(\frac{n}{2}) + \theta(n) = \theta(n^{\lg 3})$  است.

## تمرین

۱- فرض کنید  $T(n)$  برای مقادیر کوچک  $n$  ثابت است. برای هر قسمت  $T(n)$  را با نمادهای مجانبی بیان کنید.

- a)  $T(n) = ۳T\left(\frac{n}{۳}\right) + n \lg n$
- b)  $T(n) = ۵T\left(\frac{n}{۵}\right) + \frac{n}{\lg n}$
- c)  $T(n) = ۳T\left(\frac{n}{۳} + ۵\right) + \frac{n}{۳}$
- d)  $T(n) = ۲T\left(\frac{n}{۲}\right) + \frac{n}{\lg n}$
- e)  $T(n) = T\left(\frac{n}{۳}\right) + T\left(\frac{n}{۴}\right) + T\left(\frac{n}{۸}\right) + n$
- f)  $T(n) = T(n-۱) + \frac{۱}{n}$
- g)  $T(n) = T(n-۱) + \lg n$
- h)  $T(n) = T(n-۲) + ۲ \lg n$
- i)  $T(n) = \sqrt{n} T(\sqrt{n}) + n$
- j)  $T(n) = T(n-۱) + T\left(\frac{n}{۳}\right) + n$
- k)  $T(n) = ۲T\left(\frac{n}{۴}\right) + \sqrt{n}$
- l)  $T(n) = T(\sqrt{n}) + ۱$
- m)  $T(n) = ۲T(\sqrt{n}) + ۱$
- n)  $T(n) = T(n-۱) + n$

۲- مساله نزدیکترین زوج نقاط: برای یافتن نزدیکترین فاصله بین  $n$  نقطه روش معمولی آن است

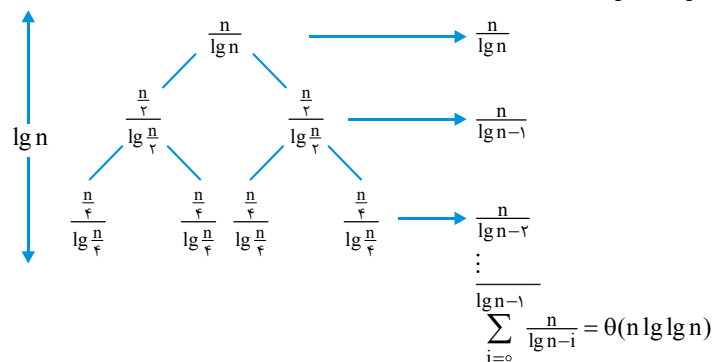
که  $\left(\frac{n}{۲}\right)$  حالت را بررسی کنیم که از مرتبه  $\theta(n^۲)$  است. روش تقسیم و غلبه‌ای ارائه دهید که رابطه بازگشتی زمان اجرای آن  $T(n) = ۲T\left(\frac{n}{۲}\right) + \theta(n)$  باشد و در نتیجه مرتبه زمانی آن  $\theta(n \lg n)$  باشد.

۳- مساله تورنمنت بازی‌ها: فرض کنید می‌خواهیم یک مسابقه دوره‌ای بین  $n = 2^m$  تیم برگزار کنیم به گونه‌ای که هر تیم در هر روز بیش از یک بازی انجام ندهد. می‌توان ثابت کرد حداقل  $n-1$  روز انجام کل بازی‌ها، طول می‌کشد. الگوریتمی برای زمان‌بندی این بازی‌ها ارائه شده است که زمان آن  $T(n) = 2T(\frac{n}{2}) + an^2$  و در نتیجه از مرتبه  $\theta(n^2)$  می‌باشد. الگوریتم را بیابید و بررسی کنید.

### پاسخ تمرین

- a)  $T(n) = \theta(n^{\lg 2})$   
 b) مشابه d بررسی کنید  
 c)  $T(n) = \theta(n \cdot \lg n)$   
 d)  $T(n) = \theta(n \cdot \lg \lg n)$   
 e)  $T(n) = \theta(n)$   
 f)  $T(n) = \theta(\lg n)$   
 g)  $T(n) = \theta(n \lg n)$   
 h)  $T(n) = \theta(n \lg n)$   
 i)  $T(n) = \theta(n \cdot \lg \lg n)$   
 j) بررسی کنید  
 k)  $T(n) = \theta(\sqrt{n} \cdot \lg n)$   
 l)  $T(n) = \theta(\lg \lg n)$   
 m)  $T(n) = \theta(\lg n)$   
 n)  $T(n) = \theta(n^2)$

اثبات d) درخت بازگشت



دقت کنید  $۱ + \frac{۱}{۲} + \dots + \frac{۱}{n} = \theta(\lg n)$  پس  $۱ + \frac{۱}{۲} + \dots + \frac{۱}{\lg n} = \theta(\lg \lg n)$

اثبات (i) طرفین را به n تقسیم می‌کنیم.

$$\frac{T(n)}{n} = \frac{T(\sqrt{n})}{\sqrt{n}} + ۱$$

تعریف می‌کنیم  $h(n) = \frac{T(n)}{n}$  پس:

$$h(n) = h(\sqrt{n}) + ۱$$

تغییر متغیر  $n = ۲^m$  پس:

$$h(۲^m) = h\left(۲^{\frac{m}{۲}}\right) + ۱$$

تعریف می‌کنیم  $F(m) = h(۲^m)$  پس:

$$F(m) = F\left(\frac{m}{۲}\right) + ۱ = \theta(\lg m)$$

در نتیجه  $h(n) = \theta(\lg \lg n)$  بنابراین:

$$T(n) = \theta(n \lg \lg n)$$

## [سؤال‌های چهارگزینه‌ای فصل سوم]

۱- جواب رابطه بازگشتی  $T(n) = 8T\left(\frac{n}{9}\right) + n \lg n$  کدام است؟

(تخصصی هوش مصنوعی دولتی ۸۴)

$$\theta(\lg n) \quad (۱) \quad \theta(n) \quad (۲) \quad \theta(n^2 \lg n) \quad (۳) \quad \theta(n \lg n) \quad (۴)$$

۲- تابع  $T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \log n$  را در نظر بگیرید. کدام یک از موارد زیر برای  $T(n)$  درست است؟

(تخصصی نرم افزار دولتی ۸۵)

$$\begin{aligned} (۱) \quad T(n) &= O(\log \log n) & (۲) \quad T(n) &= O(\log n \cdot \log \log n) \\ (۳) \quad T(n) &= O(n \cdot \log \log n) & (۴) \quad T(n) &= O(n \cdot \log n \cdot \log n) \end{aligned}$$

۳- رابطه‌های بازگشتی زیر برای اعداد صحیح  $n > 2$  تعریف شده‌اند و داریم  $T(0) = T(1) = 1$  کدام یک از این روابط جواب چند جمله‌ای (polynomial) ندارد؟

(تخصصی داده‌ها - مهندسی کامپیوتر ۸۶)

$$\begin{aligned} (۱) \quad T(N) &= 2T(N-2) + 1 & (۲) \quad T(n) &= T\left(\left\lceil \frac{\sqrt{N}}{2} \right\rceil\right) + 8N + 1 \\ (۳) \quad T(N) &= 2T\left(\left\lceil \frac{N}{2} \right\rceil\right) + N^2 & (۴) \quad T(N) &= T(N-1) + N^2 \end{aligned}$$

۴- در مورد رابطه بازگشتی زیر کدام گزینه صحیح است؟ (تخصصی نرم افزار - مهندسی کامپیوتر ۸۶)

$$T(n) = 4T(\sqrt{n}) + 1, \quad T(2) = 1$$

$$\begin{aligned} (۱) \quad T(n) &= \frac{1}{3}(\lg n)^2 - \frac{4}{3} & (۲) \quad T(n) &= \frac{4}{3}(4)^n - \frac{1}{3} \\ (۳) \quad T(n) &= \frac{4}{3}(\lg n)^2 - \frac{1}{3} & (۴) \quad T(n) &= \frac{4}{3}(n^2) - \frac{1}{3} \end{aligned}$$

۵- جواب فرمولی بازگشتی  $T(n) = 2T\left(\frac{n}{2}\right) + \log n!$  برابر است با: (علوم کامپیوتر - ۸۲)

$$\theta(n^2) \quad (۱) \quad \theta(n \lg n) \quad (۲) \quad \theta(n \log^2 n) \quad (۳) \quad \theta(n^2 \log n) \quad (۴)$$

۶- جواب تابع بازگشتی زیر کدام است؟  $T(n) = 1 \circ T\left(\frac{n}{99}\right) + \lg(n!)$ . (علوم کامپیوتر - ۸۶)

$$\begin{aligned} (۱) \quad T(n) &= \theta(n^2) & (۲) \quad T(n) &= \theta(n^{\log_{99} 1}) \\ (۳) \quad T(n) &= \theta(n \lg n) & (۴) \quad T(n) &= \theta(n \lg^2 n) \end{aligned}$$

۷- مرتبه زمانی الگوریتم با تابع زمانی زیر برابر کدام گزینه است؟ (۸۴ IT)

$$T(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ 3T(n-1) + 4T(n-2) & \text{otherwise} \end{cases}$$

$n^4 \log n$  (۴)       $2^n \log n$  (۳)       $4^n$  (۲)       $n^2$  (۱)

۸- دنباله  $T: \mathbb{Z}^{\geq 0} \rightarrow \mathbb{R}$  با معادله بازگشتی زیر تعریف شده است. مرتبه پیچیدگی  $T$  به ساده‌ترین شکل ممکن (با استفاده از نماد  $\theta$ ) کدام است؟

(IT ساختمان گسسته - ۸۶ و ساختمان داده مهندسی کامپیوتر ۹۰)

$$T(n) = \begin{cases} n & \text{if } n = 0 \text{ or } n = 1 \\ \sqrt{\frac{1}{2}T^2(n-1) + \frac{1}{2}T^2(n-2) + n} & \text{otherwise} \end{cases}$$

$$T(n) \in \theta(n^2) \quad (۲) \qquad T(n) \in \theta(n) \quad (۱)$$

$$T(n) \in \theta(n \log n) \quad (۴) \qquad T(n) \in \theta(\log n) \quad (۳)$$

۹- کدام یک از عبارات زیر جواب رابطه بازگشتی  $T(n) \leq T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + n$  است؟

(تخصصی نرم افزار دولتی ۸۵)

$$T(n) \leq \sum_{i=0}^{\log_{\Delta} n} \left(\frac{\gamma}{10}\right)^i n \quad (۲) \qquad T(n) \leq \sum_{i=0}^{\log_{10} n} \left(\frac{\gamma}{10}\right)^i n \quad (۱)$$

$$T(n) \leq \sum_{i=0}^{\log_{\frac{\gamma}{10}} n} \left(\frac{9}{10}\right)^i n \quad (۴) \qquad T(n) \leq \sum_{i=0}^{\log_{\frac{9}{10}} n} \left(\frac{9}{10}\right)^i n \quad (۳)$$

۱۰- رابطه بازگشتی روبرو داده شده است:

$$G_2 = 4, G_1 = 2, G_0 = 1$$

$$G_n = G_{n-1} + 2G_{n-2} + G_{n-3}$$

برای  $n \geq 3$  کدام یک از گزینه‌های زیر بهترین جواب این رابطه است؟

(تخصصی نرم افزار دولتی ۸۴)

$$G_n \leq 4^{n+1} \quad (۴) \qquad G_n \leq 2^{n+1} \quad (۳) \qquad G_n \leq 4^n \quad (۲) \qquad G_n \leq 2^n \quad (۱)$$

۱۱- اگر معادله زمانی بازگشتی برای یک الگوریتم برابر با:  $T(n) = T\left(\frac{2n}{3}\right) + (\log n)^2$  باشد،

زمان مصرفی الگوریتم برابر است با:

$$\theta(n^2) \quad (۱) \quad \theta(n^{\frac{2}{3}}) \quad (۲) \quad \theta(n \lg n) \quad (۳) \quad \theta((\lg n)^3) \quad (۴)$$

۱۲- اگر فرمول بازگشتی زیر پیچیدگی زمانی یک الگوریتم را نشان دهد، در آن صورت عمق

درخت بازگشتی این فرمول کدام می‌باشد؟

(علوم کامپیوتر ۸۸)

$$\begin{cases} T(1) = 1 & \lg_{\frac{1}{2}} n \quad (۲) & \lg_{\frac{1}{3}} n \quad (۱) \\ T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + O(n) & \lg_3 n \quad (۴) & \lg_2 n \quad (۳) \end{cases}$$

۱۳- در فرمول بازگشتی  $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ ,  $a \geq 1$ ,  $b > 1$  درخت بازگشتی تولید شده

(علوم کامپیوتر ۸۸)

به وسیله این فرمول کدام است؟

$$(۱) \text{ درخت } a\text{-ary با } n^{\lg a} \text{ برگ و عمق } \log_b^n$$

$$(۲) \text{ درخت غیر باینری با عمق } n^{\log_b a} \text{ برگ و عمق } \lg n$$

$$(۳) \text{ درخت کامل باینری با } 2^{\log_b a} \text{ برگ و عمق } \log_b^n$$

$$(۴) \text{ درخت کامل } a\text{-ary با } n^{\log_b a} \text{ برگ و عمق } \log_b^n$$

۱۴- کدام یک از موارد زیر در مورد تابع بازگشتی  $h(A)$  درست است؟

(علوم کامپیوتر ۸۸)

$$h(A) = \begin{cases} 0, & A = 0 \\ 1 + h(A-1), & A \text{ is odd} \\ 1 + h\left(\frac{A}{2}\right), & \text{otherwise} \end{cases}$$

$$h(A) \in \Omega(\log_2^A), h(A) \in O(A) \quad (۲) \quad h(A) \in \Theta(A) \quad (۱)$$

$$h(A) \in \Omega(A), h(A) \in O(\log_2^A) \quad (۴) \quad h(A) \in \Theta(\log_2^A) \quad (۳)$$

۱۵- مقدار تابع زیر به ازای  $n \geq 2$  چقدر است؟

(تخصصی هوش مصنوعی دولتی ۸۳ و IT ۸۷)

$$\begin{aligned} &\text{function } g(n) && 5^n - 6^n \quad (۱) \\ &\text{begin} && 3^n - 2^n \quad (۲) \\ &\text{if } n \leq 1 \text{ then} && 3^n + 2^n \quad (۳) \\ &\quad g := n; && 5^n + 6^n \quad (۴) \\ &\text{else} && \\ &\quad g := 5 * g(n-1) - 6 * g(n-2); && \\ &\text{end;} \end{aligned}$$

۱۶- فرض کنید  $n$  توانی از ۲ باشد. الگوریتم زیر چه عددی را به عنوان خروجی برمی‌گرداند؟  
(ساختمان داده - دولتی ۸۴)

```

function Test(n)
if n = ۱ then return ۱
return ۲ * Test( $\frac{n}{۲}$ ) + ۶ * n - ۱;

```

$$\begin{aligned} (۱) & \quad ۶n \log_2^n \\ (۲) & \quad ۶n \log_2^n + ۱ \\ (۳) & \quad ۶n \log_2^n - ۱ \\ (۴) & \quad ۶n \end{aligned}$$

۱۷- تابع زیر را در نظر بگیرید. فرض کنید که  $T(n)$  تعداد سطرهایی است که در اثر اجرای این تابع روی صفحه، نمایش داده خواهند شد و داریم  $n \leq ۱$ ،  $T(n) = ۱$  کدام یک از تعاریف بازگشتی زیر صحیح است؟  
(تخصصی نرم افزار دولتی ۸۳)

```

function concow(n)
if n ≤ ۱ then
    writeln('STOP')
    return ۱
else
    for i ← ۱ to n do
        writeln(concow (n div ۲))
    return n

```

$$\begin{aligned} (۱) \quad T(n) &= \sum_{i=1}^n T\left(\left\lceil \frac{i}{۲} \right\rceil\right) \\ (۲) \quad T(n) &= nT\left(\left\lceil \frac{n}{۲} \right\rceil\right) + n \\ (۳) \quad T(n) &= nT\left(\left\lfloor \frac{n}{۲} \right\rfloor\right) \\ (۴) \quad T(n) &= T\left(\left\lceil \frac{n}{۲} \right\rceil\right) + n \end{aligned}$$

۱۸- زیر برنامه روبرو را در نظر می‌گیریم:

```

procedure Time(x)
{
if (x < ۸) write ("Message")
else {
    Time( $\lfloor \frac{x}{۲} \rfloor$ )
    Time( $\lfloor \frac{x}{۴} \rfloor$ )
    Time( $\lfloor \frac{x}{۸} \rfloor$ )
}
}

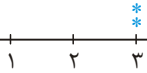
```

اگر  $t(x)$  تعداد دفعات چاپ پیغام Message روی صفحه برای فراخوانی  $\text{time}(x)$  باشد کدام یک از موارد زیر صحیح است؟  
(تخصصی هوش مصنوعی دولتی ۸۵)

$$\begin{aligned} (۱) \quad & t(۹) = ۴; t(۱۶) = ۵; t(۳۶) = ۸ \\ (۲) \quad & t(۹) = ۳; t(۱۶) = ۵; t(۳۶) = ۹ \\ (۳) \quad & t(۹) = ۳; t(۱۶) = ۶; t(۳۶) = ۸ \\ (۴) \quad & t(۹) = ۴; t(۱۶) = ۶; t(۳۶) = ۹ \end{aligned}$$



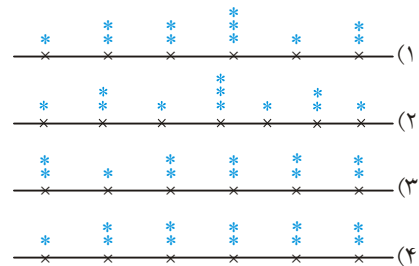
۱۹- الگوریتم روبرو را در نظر بگیرید. فرض کنید الگوریتم  $g$ ، با تعریف  $g(x, y)$ ، در محل  $x, y$

تا علامت ستاره می‌نویسد. به عنوان مثال  $g(3, 2) =$   خروجی الگوریتم

(تخصصی نرم افزار دولتی ۸۵)

$f(0, 8, 3)$  کدام است؟

```
f(int a, int b, int c)
{
  int m = (a + b) / 2;
  if (c > 0)
  {
    g(m, c);
    f(a, m, c - 1);
    f(m, b, c - 1);
  }
}
```



۲۰- کدام یک از شرایط زیر برای مقادیر صحیح و مثبت  $a_1$  تا  $a_k$  تضمین می‌کند که  $\theta(n)$

حل رابطه بازگشتی  $T(n) = \sum_{i=1}^k T(\frac{n}{a_i}) + \theta(n)$  است. (تخصصی هوش مصنوعی دولتی ۸۳)

$$\sum_{i=1}^k \frac{1}{a_i} > k \quad (۴) \quad \sum_{i=1}^k \frac{1}{a_i} < 1 \quad (۳) \quad \sum_{i=1}^k \frac{1}{a_i} = k \quad (۲) \quad \sum_{i=1}^k \frac{1}{a_i} = 1 \quad (۱)$$

۲۱- فرض کنید زمان اجرای الگوریتمی روی  $n$  ورودی،  $T(n)$  بوده که به صورت زیر تعریف

می‌شود. زمان اجرای الگوریتم مزبور برابر کدام گزینه است؟ (مهندسی IT آزاد ۸۴)

$$T(n) = \begin{cases} 1 & n = 1 \\ n + T(n-1) & n \geq 2 \end{cases} \quad \begin{matrix} O(n \log n) & (۲) \\ O(n) & (۱) \\ O(n^{\frac{3}{2}}) & (۳) \\ O(n^2) & (۴) \end{matrix}$$

۲۲- الگوریتم مقابل جمع ۱ تا  $n$  را محاسبه می‌کند، زمان اجرای آن کدام است؟ (مشابه دولتی - ۷۴)

$$f(n) = \begin{cases} 1 & n = 1 \\ n + f(n-1) & n > 1 \end{cases} \quad \begin{matrix} O(n \log n) & (۲) \\ O(n) & (۱) \\ O(n^{\frac{3}{2}}) & (۳) \\ O(n^2) & (۴) \end{matrix}$$

۲۳- مرتبه زمانی الگوریتم بازگشتی زیر چیست؟ (فراخوانی به صورت  $n$  و ۱)  $\text{algor}$  می‌باشد)

(مهندسی فناوری اطلاعات - آزاد ۸۵)

```
procedure algor (i, j)
  if i = j then return
  k := floor((i+j)/2)
  call algor(i, k)
  call algor(k+1, j)
end algor
```

$$\begin{matrix} \theta(\log n) & (۱) \\ \theta(n) & (۲) \\ \theta(n \log n) & (۳) \\ \theta(n^2) & (۴) \end{matrix}$$

۲۴- تابع بازگشتی زیر را در نظر بگیرید ( $n$  توانی از ۲ است)

```
int Fun۲(int n)
{
    if (n <= ۱)
        return ۱;
    else
        return Fun۲( $\frac{n}{۲}$ ) +  $\frac{n}{۲}$ 
```

(مهندسی فناوری اطلاعات - آزاد ۸۵)

زمان اجرای تابع فوق چیست؟

$$O(\log_2^n) \quad (۱) \quad O(n) \quad (۲) \quad O(n \log n) \quad (۳) \quad O(\frac{n}{۲}) + ۱ \quad (۴)$$

۲۵- الگوریتم بازگشتی برای مشخص نمودن عنصر  $n$ ام دنباله فیبوناچی به صورت زیر تعریف شده است: اگر  $T(n)$  پیچیدگی زمانی الگوریتم فیبوناچی فرض شود، در اینصورت کدام

(مهندسی فناوری اطلاعات - آزاد ۸۶)

یک از روابط زیر درست می‌باشد؟

```
int fib(int n)
{
    if (n <= ۱)
        return n;
    else
        return fib(n-۱) + fib(n-۲);
}
```

$$T(n) > ۲^{\frac{n}{۲}} \quad (۱)$$

$$T(n) > n^۲ \quad (۲)$$

$$T(n) = ۲T(n-۲) + ۱ \quad (۳)$$

$$T(n) = ۲T(n-۱) + ۱ \quad (۴)$$

۲۶- فرض کنید  $A(n)$  تعداد ستاره‌هایی باشد که توسط الگوریتم زیر چاپ می‌شود. برای

(علوم کامپیوتر - ۸۵)

$n \geq ۳$  کدام رابطه برای الگوریتم درست است؟

```
void mg(int n)
{
    if (n >= ۳)
    {
        print "*";
        int k = n;
        k = k - ۱;
        mg(k);
        k = k - ۱;
        mg(k);
        print "*";
    }
}
```

$$A(n) = ۲ + A(n-۱) + A(n-۲) \quad (۱)$$

$$A(n) = ۲ + ۲A(n-۱) \quad (۲)$$

$$A(n) = A(n-۱) + ۲A(n-۲) \quad (۳)$$

$$A(n) = A(n-۱) + ۲A(n-۲) + ۲ \quad (۴)$$

(علوم کامپیوتر - ۸۵)

۲۷- مقدار  $F(۱, ۱) = ?$ 

```

int F(int n , int i)
{
if(i <= n)
    return F(n , i + ۱) + i ;
else
    return ۰ ;
}

```

(۱) ۱۰

(۲) ۱۱

(۳) ۵۴

(۴) ۵۵

(علوم کامپیوتر - ۸۵)

۲۸- خروجی برنامه زیر برابر است با:

```

int pp (int n)
{
if (n == ۰) return ۰ ;
else return ۲n + pp (n - ۱) - ۱ ;
}

```

(۱)  $\frac{n^2}{2}$ (۲)  $n^2$ (۳)  $(n-1)^2$ (۴)  $\frac{n(n-1)}{2}$ ۲۹- برنامه زیر برای انتقال  $n$  حلقه از برج هانوی (برهما) استفاده می‌شود. کدام گزینه دستور

(علوم کامپیوتر - ۸۰)

حذف شده را مشخص می‌کند:

```

procedure Tower (n : integer ; from , help , to : char) ;
begin
if n = ۱ then
    writeln('move a disk from' , from , 'to' , to)
else begin
    Tower (n - ۱ , from , to , help) ;
    Writeln ('move a disk from' , from , 'to' , to) ;
    دستور حذف شده
end
end ;

```

(۱) Tower (n-۱ , from , to , help);

(۲) Writeln ('move a disk from' , help , 'to' , to);

(۳) Tower (n-۱ , help , from , to)

(۴) Writeln ('move a disk from' , from , 'to' , help);

۳۰- مساله برج هانوی را با سه میله  $A$  و  $B$  و  $C$  را در نظر می‌گیریم. در اینجا هیچ حلقه‌ای رانمی‌توان مستقیماً از  $A$  به  $B$  یا از  $B$  به  $A$  منتقل کرد. یعنی چنین انتقال‌هایی تنها به کمک

میله C انجام می‌پذیرند. اگر در ابتدا n حلقه در میله A داشته باشیم و  $T(n)$  حداقل تعداد عملیات لازم برای انتقال n حلقه از A به B باشد،  $T(n)$  با کدام رابطه مشخص می‌شود؟ (فرض کنید  $T(1)=2$ ) (یادآوری می‌شود که در هیچ مرحله‌ای نمی‌توان حلقه‌ای را روی حلقه‌ای کوچکتر از خودش قرار داد و در هر مرحله فقط یک حلقه جابجا می‌شود).

(مهندسی کامپیوتر دولتی ۸۱)

$$T(n) = 6T(n-1) + 3 \quad (1) \quad T(n) = 3T(n-1) + 2 \quad (2)$$

$$T(n) = T(n-1) + T(n-2) + 1 \quad (3) \quad T(n) = T(n-1) + T(n-2) + 2 \quad (4)$$

۳۱- حالت خاصی از مسئله برج‌های هانوی را در نظر بگیرید. ۱۰۰ سکه با شماره‌ها (و اندازه‌های) ۱ تا ۱۰۰ موجودند. از این تعداد  $n_1$  عدد در میله شماره ۱ (با شماره دلخواه) به ترتیب از بزرگ به کوچک (از پایین به بالا)، و بقیه در میله شماره ۳ با همین نظم و ترتیب قرار دارند. می‌خواهیم با حفظ مقررات برج‌هانوی، همه سکه‌ها را به میله شماره ۳ منتقل کنیم. حداقل تعداد حرکات در بدترین حالت حرکت سکه‌ها چقدر است؟

(تخصصی نرم افزار - مهندسی کامپیوتر دولتی ۸۶)

$$2^{100} - 1 \quad (1) \quad 2^{100} + 1 \quad (2) \quad 2^{101} - 1 \quad (3) \quad 2^{101} + 1 \quad (4)$$

۳۲- اگر  $T(n)$  تعداد ستاره‌های چاپ شده توسط  $Mystery(n)$  باشد، کدام یک از عبارت‌های

(۸۷ IT)

زیر رابطه بازگشتی  $T(n)$  را به درستی نشان می‌دهند؟

|                                   |                                                |
|-----------------------------------|------------------------------------------------|
| <code>void Mystery(int n){</code> | $T(n) = 1 + T(n-3) + 2 + T(n-4) + 1 \quad (1)$ |
| <code>if (n &gt;= 2){</code>      | $T(n) = 5T(n-1) + 4T(n-2) \quad (2)$           |
| <code>    Mystery(n-1);</code>    | $T(n) = 3T(n-1) + 4T(n-2) \quad (3)$           |
| <code>    Print "***";</code>     | $T(n) = 2T(n-1) + T(n-2) + 7 \quad (4)$        |
| <code>    Mystery(n-2);</code>    |                                                |
| <code>    Print "****";</code>    |                                                |
| <code>    Mystery(n-1);</code>    |                                                |
| <code>}</code>                    |                                                |
| <code>}</code>                    |                                                |

۳۳- پیچیدگی زمانی الگوریتم زیر از چه درجه‌ای است؟

|                                                                   |                             |
|-------------------------------------------------------------------|-----------------------------|
| <code>int F(int n, int m) {</code>                                | $n \quad (1)$               |
| <code>    if (n &lt;= 1    m &lt;= 1) return 1;</code>            | $\log m \quad (2)$          |
| <code>    return n + m * F(n-1, <math>\frac{m}{4}</math>);</code> | $\min(n, \log m) \quad (3)$ |
| <code>}</code>                                                    | $\max(n, \log m) \quad (4)$ |

(علوم کامپیوتر ۸۹)

۳۴- تابع بازگشتی زیر را در نظر بگیرید:

```
function DC (a, n);
begin
  if n = ۱ then return (a);
  if n is even then return (DC(a, n/۲) * DC(a, n/۲))
  return (a * DC(a, n - ۱))
end;
```

مقدار این تابع برای  $a = ۲$  و  $n = ۶$  چیست؟ (مبانی نرم افزار آزاد ۷۷)

۱۲ (۱) ۶۴ (۲) ۱۶ (۳) ۳۲ (۴)

۳۵- تابع زیر داده شده است، تابع چه نتیجه‌ای را برمی‌گرداند؟ (تخصصی نرم افزار آزاد ۷۹)

```
function find (A, B : integer) : integer;
begin
  if B = ۱ then find := A
  else find := find(A, B - ۱) + A
end;
```

(۱)  $A * (B - ۱)$   
 (۲)  $A * B$   
 (۳)  $A + B$   
 (۴)  $A^B$

۳۶- تابع بازگشتی زیر را در نظر بگیرید:

```
int test (int n)
{
  if (n <= ۲);
    return ۱;
  else
    return test (n - ۲) * test (n - ۲);
}
```

زمان اجرای تابع فوق برابر است با: (مهندسی کامپیوتر دولتی ۷۵)

(۱)  $O(n^۲)$  (۲)  $O(n \log n)$  (۳)  $O(۲^{\frac{n}{۲}})$  (۴)  $O(۲^n)$

۳۷- یک مساله با استفاده از الگوریتم تقسیم و غلبه حل شده است. در این مساله هر مساله

بزرگ به ۵ مساله کوچکتر شکسته شده و اندازه هر مساله شکسته شده  $\frac{1}{۳}$  مساله بزرگتر

می‌باشد. الگوریتم شکستن و ترکیب دارای زمان  $O(n)$  می‌باشند. مرتبه بزرگی این

الگوریتم چیست؟ (مبانی نرم افزار آزاد ۷۹)

(۱)  $\theta(n \log_۳^۵)$  (۲)  $\theta(n^۳)$  (۳)  $\theta(n^{\log_۳^۵})$  (۴)  $\theta(n^{\frac{۵}{۳}})$

۳۸- برای حل یک مساله به اندازه  $n$  با الگوریتم تقسیم و غلبه سه روش به شرح زیر امکان پذیر است:

روش اول: حل ۳ زیر مساله به اندازه  $\frac{n}{۳}$  و ترکیب آنها با هزینه  $\theta(n^2 \sqrt{n})$

روش دوم: حل ۴ زیر مساله به اندازه  $\frac{n}{۴}$  و ترکیب آنها با هزینه  $\theta(n^2)$

روش سوم: حل ۵ زیر مساله به اندازه  $\frac{n}{۵}$  و ترکیب آنها با هزینه  $\theta(n \log n)$

کدام روش دارای هزینه کمتری است؟ (۸۶ IT)

(۱) روش اول (۲) روش دوم

(۳) روش سوم (۴) هر سه روش یکسانند.

۳۹- الگوریتم ضرب ماتریس استراسن (Strassen) به این صورت عمل می‌کند که ابتدا دو ماتریس

$n \times n$  مفروض  $A$  و  $B$  را به عنوان ورودی دریافت نموده و ماتریس خروجی، که حاصل ضرب  $A$  و

$B$  را تولید می‌کند. تقسیم بندی زیر ماتریس‌ها در این روش به صورت زیر است:

$$\frac{n}{۲} \downarrow \begin{bmatrix} C_{۱۱} & C_{۱۲} \\ C_{۲۱} & C_{۲۲} \end{bmatrix} = \begin{bmatrix} A_{۱۱} & A_{۱۲} \\ A_{۲۱} & A_{۲۲} \end{bmatrix} \times \begin{bmatrix} B_{۱۱} & B_{۱۲} \\ B_{۲۱} & B_{۲۲} \end{bmatrix}$$

تقسیمات متوالی تا رسیدن به دو ماتریس  $۱ \times ۱$  ادامه می‌یابد. در این صورت پیچیدگی

زمانی حالت معمول تعداد جمع‌ها تفریق‌ها در الگوریتم استراسن برابر است با: (فرض کنید

$n > ۱$  و  $n$  توانی از ۲ می‌باشد) (تخصصی نرم افزار - مهندسی کامپیوتر آزاد ۸۶)

$$\begin{cases} T(n) = n \left( T\left(\frac{n}{۲}\right) + \frac{n^2}{۲} \right) \\ T(۱) = ۰ \end{cases} \quad (۲) \quad \begin{cases} T(n) = ۷ \left( T\left(\frac{n}{۲}\right) + \left(\frac{n}{۲}\right)^2 \right) \\ T(۱) = ۱ \end{cases} \quad (۱)$$

$$\begin{cases} T(n) = ۷ \left( T\left(\frac{n}{۴}\right) + ۹ \frac{n^2}{۴} \right) \\ T(۱) = ۱ \end{cases} \quad (۴) \quad \begin{cases} T(n) = ۷ T\left(\frac{n}{۲}\right) + ۱۸ \left(\frac{n}{۲}\right)^2 \\ T(۱) = ۰ \end{cases} \quad (۳)$$

۴۰- اگر دو ماتریس  $۴ \times ۴$  با روش ضرب استراسن (Strassen) در یکدیگر ضرب شوند برای

ضرب این دو ماتریس چند ضرب عددی صورت می‌گیرد؟ (مبانی نرم افزار آزاد ۷۹)

(۱) ۲۸ (۲) ۴۹ (۳) ۷ (۴) ۱۱

۴۱- اگر در ضرب ماتریس‌ها به روش استراسن (strassen) مساله کوچک ضرب ماتریس‌های

$۲ \times ۲$  باشد. برای ضرب دو ماتریس  $۸ \times ۸$  چند ضرب عددی صورت می‌پذیرد؟ (۸۵ IT)

۵۷ (۱) ۳۴۳ (۲) ۳۹۲ (۳) ۵۱۲ (۴)

۴۲- اگر در ضرب ماتریس‌ها به روش استراسن (strassen) مساله کوچک ضرب ماتریس‌های

$۲ \times ۲$  باشد. با چند فراخوانی بازگشتی الگوریتم استراسن عمل ضرب دو ماتریس  $۸ \times ۸$

انجام می‌پذیرد؟ (۸۶ IT)

۳۴۳ (۱) ۵۷ (۲) ۴۹ (۳) ۷ (۴)

۴۳- اگر روشی برای ضرب ماتریس‌های  $۲ \times ۲$  با ۶ عمل ضرب یا تقسیم و ۲ عمل جمع یا تفریق

داشته باشیم و این روش را با استفاده از تقسیم و غلبه برای ضرب ماتریس‌های  $n \times n$  به کار

ببریم (روش استراسن)، مرتبه ضرب ماتریسی حاصل برابر است با: (علوم کامپیوتر ۷۹)

$\theta(n^2)$  (۱)  $\theta(n^2 \log n)$  (۲)

$\theta(n^2 \log^{1/2} n)$  (۳)  $\theta\left(n^{\log_2 6}\right)$  (۴)

۴۴- ماتریس‌های  $A_0, A_1, A_2, \dots$  را با تعریف زیر در نظر بگیرید: (تخصصی هوش مصنوعی ۸۹)

$A_0$ ، یک ماتریس به شکل  $1 \times 1$

$A_k$ ، یک ماتریس به شکل  $2^k \times 2^k$   $A_k : k > 0$   $A_k = \begin{bmatrix} A_{k-1} & A_{k-1} \\ A_{k-1} & -A_{k-1} \end{bmatrix}$

فرض کنید  $V$  یک بردار ستونی به طول  $n = 2^k$  است. برای ضرب  $A_k \cdot V$  زمان مصرفی کدام مورد است؟

$\theta(n \cdot \lg n)$  (۱)  $\theta(n)$  (۲)  $\theta(k \cdot \lg n)$  (۳)  $\theta(n^2 \cdot \lg n)$  (۴)

۴۵- برای ضرب دو عدد  $A$  و  $B$  به ترتیب با  $n$  و  $4n$  رقم برای  $10000 \leq n \leq 50000$ :

(تخصصی نرم افزار دولتی ۸۳)

(۱) بهتر است از روش معمولی و کلاسیک برای ضرب آنها استفاده کرد.

(۲) بهتر است از الگوریتم تقسیم و حل استفاده کرد و فرقی نمی‌کرد به چه ترتیب

(۳) بهتر است پس از قسمت کردن  $B$  به ۴ بخش  $n$  رقمی هر کدام از این بخش‌ها را در  $A$  و با روش تقسیم و حل ضرب کند.

(۴) بهتر است پس از اضافه کردن  $3n$  صفر به سمت چپ  $A$ ، عدد حاصل را با کمک الگوریتم تقسیم و حل در  $B$  ضرب کند.

۴۶- مسئله‌ی کلاسیک ضرب دو چند جمله‌ای  $A = \sum_{i=0}^n a_i x^i$  و  $A = \sum_{i=0}^n b_i x^i$  محاسبه‌ی

$C = A \times B = \sum_{k=0}^{2n} c_k x^k$  را در نظر بگیرید که با داشتن  $n+1$  ضریب  $a_i$  و  $n+1$  ضریب  $b_j$ ، هدف محاسبه‌ی  $2n+1$  ضریب  $c_k$  است. راه حل مبتنی بر «تقسیم و حل» هر دو مسئله را به دو قسمت تقسیم می‌کند و مسئله‌ی اصلی را حل می‌کند. زمان اجرای این راه حل از رابطه‌ی بازگشتی  $T(n) = 3T\left(\frac{n}{2}\right) + \Theta(n)$  به دست می‌آید که جواب آن  $O(n \log_2^3)$  است. در صورتی که  $B' = \sum_{j=\frac{n}{2}}^n b_j c^j$  باشد (یعنی نیمی از ضرایب آن صفر

باشند)، کدام یک از گزینه‌های زیر زمان اجرای الگوریتم تقسیم و حل برای به دست آوردن ضرائب چند جمله‌ای  $C = A \times B'$  خواهد بود؟ (تخصصی هوش مصنوعی ۸۹)

(۱) رابطه‌ی بازگشتی راه حل تغییر می‌کند، اما جواب آن همان  $O(n \log_2^3)$  است.

(۲) رابطه‌ی بازگشتی راه حل تغییر نمی‌کند، و جواب هم همان  $O(n \log_2^3)$  است.

(۳) رابطه‌ی بازگشتی به  $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$  تغییر می‌کند، ولی جواب آن  $O(n \lg n)$  است.

(۴) رابطه‌ی بازگشتی به  $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$  تغییر می‌کند که جواب آن  $O(n \lg n)$  است.

۴۷- شمار فراوانی تابع بازگشتی زیر چیست؟ (ساختان داده‌ها - مهندسی کامپیوتر - آزاد ۸۵)

```
type Element List = array [1 .. m] of integer ;
function rsum (Var a : Element List ; n : integer) : real ;
begin
  if n <= 0 then rsum := 0
  else
    rsum := rsum (a , n-1) + a[n] ;
  end ;
```

(۱)  $n(m+1) + 2$  (۲)  $2n + 2$  (۳)  $2n$  (۴)  $n(m+1)$

۴۸- تابع زیر برای محاسبه بزرگترین مقسوم علیه مشترک دو عدد نوشته شده است. مرتبه

زمانی الگوریتم کدام است؟ (ساختان داده‌ها - دولتی ۸۰)

```
int gcd (int a , int b)
{
  if (b == 0)
    return a ;
  else
    return gcd (b , a mod b)
}
```

(۱)  $O(\log_{\frac{a}{b}} \frac{a}{b})$  (۲)  $O(\log_b^a)$  (۳)  $O(\log_{\frac{a}{b}}^a)$  (۴)  $O(\log_{\frac{a}{b}}^{a-b})$



۴۹- در روال بازگشتی زیر مقدار (۳ و ۵) Rec کدام است؟ (ساختمان داده‌ها - دولتی ۸۰)

```
int Rec (int p , int q)
{
    int R ;
    if (q <= 0) return 1 ;
    R = Rec (p , q/2) ;
    R = R * R ;
    if (q mod 2 == 0)
        return R ;
    else
        return R * p ;
}
```

(۱) ۱۵ (۲) ۲۵ (۳) ۷۵ (۴) ۱۲۵

۵۰-  $f(x)$  زمان اجرای الگوریتمی مطابق رابطه بازگشتی زیر محاسبه شده است. کدام گزینه

(تخصصی هوش مصنوعی دولتی ۸۴)

صحیح است؟

$$f(n) = \begin{cases} a & \text{if } n=1 \\ bn^2 + n.f(n-1) & \text{else} \end{cases}$$

$$f(n) = \theta(n!) \quad (۲)$$

$$f(n) = \theta(2^n) \quad (۱)$$

$$f(n) = \theta(n!)^2 \quad (۴)$$

$$f(n) = \theta(2^{n!}) \quad (۳)$$

۵۱- یک کامپیوتر موازی با  $\frac{n}{۲}$  پردازنده مفروض است. اگر بخواهیم با استفاده از این کامپیوتر

اعداد موجود در یک آرایه  $n$  عنصری را جمع نماییم. زمان مورد نیاز چیست؟

(تخصصی نرم افزار مهندسی کامپیوتر - آزاد ۸۵)

$$(۱) O(\log_2^n) \quad (۲) O(\log_2^n) \quad (۳) O(n \log_2^n) \quad (۴) O\left(\frac{n}{۲}\right)$$

۵۲- رویه زیر را در نظر بگیرید:

```
procedure f (a : string ; k , n : integer) ;
begin
    if k=n then print (a)
    else
        for i :=k to n do
            begin
                swap (a[k] , a[i]) ;
                f (a , k+1 , n) ;
            end;
        end f.
```

تابع print رشته a را چاپ و تابع swap جای دو عنصر را عوض می‌کند. کدام رابطه زیر درباره زمان f درست است؟ (آزاد ۸۳)

$$\theta(n \cdot n!) \quad (۴) \quad \theta(n!) \quad (۳) \quad O(n^k) \quad (۲) \quad o(n!) \quad (۱)$$

۵۳- با توجه به اینکه مقادیر آرایه A به ازای هر i ، کوچکتر مساوی n است یعنی  $A[i] \leq n$ .

مرتبه زمانی الگوریتم زیر چیست؟ (علوم کامپیوتر - ۸۵)

$$\begin{array}{ll} f(A, i) & O(n) \quad (۱) \\ \{ & \\ n := \text{length}(A); & O(n^2) \quad (۲) \\ \text{for } j := 1 \text{ to } A[i] & O(n \log n) \quad (۳) \\ \quad k := k+1; & \\ \text{if } A[i]+i \leq n & O(n^2 \log n) \quad (۴) \\ \quad \quad f(A, A[i]+i) & \\ \} & \end{array}$$

۵۴- مقدار  $F(10 \text{ و } 1) = ?$  (علوم کامپیوتر - ۸۵)

$$\begin{array}{ll} \text{int } F(\text{int } n, \text{int } i) & ۱۰ \quad (۱) \\ \{ & \\ \text{if } (i \leq n) & ۱۱ \quad (۲) \\ \quad \text{return } F(n, i+1)+i; & ۵۴ \quad (۳) \\ \text{else} & ۵۵ \quad (۴) \\ \quad \text{return } 0; & \\ \} & \end{array}$$

۵۵- اگر رابطه بازگشتی یک الگوریتم تقسیم و غلبه به صورت زیر باشد آنگاه مرتبه زمانی این

الگوریتم کدام است؟ (علوم کامپیوتر ۹۱)

$$\begin{array}{ll} T(n) = 2T\left(\frac{n}{4}\right) + \sqrt{n} & \\ O(n) \quad (۲) & O(\sqrt{n}) \quad (۱) \\ O(\sqrt{n} \log \sqrt{n}) \quad (۴) & O(n \log n) \quad (۳) \end{array}$$

۵۶- رابطه بازگشتی زیر را در نظر بگیرید:

$$\begin{cases} T(n) = \frac{3}{2}T\left(\frac{n}{2}\right) - \frac{1}{2}T\left(\frac{n}{4}\right) - \frac{1}{n}, n \geq 3 \\ T(1) = 1, T(2) = \frac{3}{2} \end{cases}$$

$T(n) \in \theta(?)$  خواهد بود؟

$$\theta\left(\frac{\log n}{n}\right) \quad (۴) \quad \theta(n \log n) \quad (۳) \quad \theta\left(\frac{n}{2^n}\right) \quad (۲) \quad \theta(\log n) \quad (۱)$$

۵۷- تابع ذیل را در نظر بگیرید. تعداد صدا زدن‌های بازگشتی (recursive calls) چقدر است؟

(تخصصی هوش مصنوعی آزاد ۸۲)

```
Function c (n, k)
If k=0 or k=n then return 1
Else return c (n-1, k-1)+c(n-1, k)
```

$$\binom{n}{k} \quad (۱) \quad \binom{n}{k} - ۱ \quad (۲) \quad \binom{n}{k} - ۲ \quad (۳) \quad \binom{n}{k} \quad (۴)$$

۵۸- الگوریتم زیر را جهت محاسبه ضریب دو جمله‌ای که در آن  $k$  و  $n$  مثبت و  $k \leq n$  است در نظر بگیرید.

```
int comb (int n, int k)
{
    If ((k == 0) or (n == k))
        return 1;
    else
        return comb (n-1, k-1)+comb (n-1, k)
}
```

در این صورت تعداد جملاتی که الگوریتم فوق برای تعیین  $\binom{n}{k}$  محاسبه می‌کند چیست؟

(مهندسی فناوری اطلاعات – آزاد ۸۵)

$$\binom{n}{k} - ۱ \quad (۱) \quad \binom{n}{k-1} + ۱ \quad (۲) \quad \binom{n-1}{k-1} + ۱ \quad (۳) \quad \binom{n-1}{k} + ۳ \quad (۴)$$

۵۹- در الگوریتم زیر (۶ و ۳)  $F$  چقدر است؟

(تخصصی نرم افزار آزاد ۸۱)

```
int F(int m, int n)
    if (m=1 or n=0 or m=n)
        return (1);
    else
        return (F (m-1, n)+F (m-1, n-1))
end F.
```

$$۲۰ \quad (۱) \quad ۱۰ \quad (۲) \quad ۱۸ \quad (۳) \quad ۱۵ \quad (۴)$$

۶۰- الگوریتم زیر  $\binom{n}{m}$  را برای اعداد صحیح  $n \geq m$  محاسبه می‌کند:

```
Function combination (n, m: integer) : integer;
Begin
    if (m=n) or (m=0) then return (1)
    else return (combination (n-1, m) + combination (n-1, m-1))
end;
```

برای  $n > m$  دلخواه عمل + چند بار اجرا می‌شود؟ (مهندسی کامپیوتر دولتی ۷۷)

$$\binom{n}{m} \quad (۱) \quad nm \quad (۲) \quad \binom{n}{m} - ۱ \quad (۳) \quad n(n-m) \quad (۴)$$

۶۱- برای محاسبه حاصل چند جمله‌ای زیر به روش تقسیم و حل چه مرتبه زمانی لازم است؟

(علوم کامپیوتر ۸۹)

$$P_{n-1}(x) = a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \dots + a_1x + a_0$$

$$O(n \lg n) \quad (۱) \quad O(\lg n) \quad (۲) \quad O(n) \quad (۳) \quad O(n^2) \quad (۴)$$

۶۲- دو تابع  $P_1$  و  $P_2$  داریم که با دریافت آرایه‌ی  $n$  بیتی  $x$ ، اولی  $y_1 = A_1x$  و دومی

$y_2 = A_2x$  را محاسبه می‌کند که  $A_1$  و  $A_2$  ماتریس‌های بیتی  $n \times n$  هستند. این دو تابع به ترتیب آرایه‌های  $n$  بیتی  $y_1$  و  $y_2$  را برمی‌گردانند. ما اطلاعات ماتریس‌ها را نداریم اما می‌دانیم که  $A_1$  و  $A_2$  دقیقاً مانند هم هستند به جز در یک درایه‌ی  $(i, j)$  و اندیس‌های  $i$  و  $j$  را هم نمی‌دانیم. ما می‌توانیم  $P_1$  و  $P_2$  را با مقادیر مختلف  $x$  فراخوانیم و خروجی آن دو را با هم مقایسه کنیم. با چند بار فراخوانی می‌توانیم اندیس‌های  $i$  و  $j$  را بیابیم؟

(نرم‌افزار - ۹۳)

$$O(\lg n) \quad (۱) \quad O(n) \quad (۲) \quad O(n \lg n) \quad (۳) \quad O(n^2) \quad (۴)$$

۶۳- فرض کنید دو عدد  $a$  و  $b$  بیتی را می‌توان با هزینه‌ای برابر  $O(\max\{a, b\})$  جمع کرد.

می‌خواهیم  $n$  عدد ۱ بیتی (۰ یا ۱) را با هم جمع کنیم. هزینه‌ی این کار بسته به ترکیب داده‌ی ورودی ممکن است متفاوت باشد. هزینه‌ی این کار در بهترین و بدترین حالت کدام است؟

(هوش مصنوعی - ۹۳)

$$(۱) \text{ بهترین: } O(\lg n) \text{ و بدترین: } O(n)$$

$$(۲) \text{ بهترین: } O(n) \text{ و بدترین: } O(n \lg n)$$

$$(۳) \text{ بهترین: } O(\lg n) \text{ و بدترین: } O(n \lg n)$$

$$(۴) \text{ بهترین: } O(n) \text{ و بدترین: } O(n)$$

۶۴- در مسئله‌ی ژوزفوس  $n$  نفر با شماره‌های ۱ تا  $n$  به صورت ساعت‌گرد دور یک میز دوار

نشسته‌اند و با الگوریتم زیر یکدیگر را می‌کشند. اسلحه ابتدا در دست فرد شماره‌ی ۱ است. الگوریتم از شماره‌ی ۱ شروع می‌کند و در جهت ساعت‌گرد در هر مرحله از یک نفر زنده عبور کرده و فرد زنده‌ی بعدی خود را می‌کشد. این الگوریتم تا زمانی که یک نفر باقی بماند ادامه پیدا می‌کند. شماره‌ی فرد زنده‌ی آخر را  $f(n)$  می‌نامیم. مثلاً اگر  $n = ۹$ ، به

ترتیب شماره‌های ۲، ۴، ۶، ۸، ۱، ۵، ۹ و ۷ کشته شده و ۳ زنده می‌ماند. یعنی  $f(9) = 3$ .

کدامیک از رابطه‌های بازگشتی زیر درست است؟ (۹۳-IT)

$$f(1392) = 2f(696) + 1 \quad (2) \quad f(1392) = 2f(696) - 1 \quad (1)$$

$$f(1393) = f(1392) - 2 \quad (4) \quad f(1393) = 2f(696) - 1 \quad (3)$$

۶۵- کدام گزینه حل رابطه‌ی بازگشتی زیر  $(T(n, k))$  است؟ (۹۳-IT)

$$T(n, k) = T(n_1, \lfloor \frac{k}{r} \rfloor) + T(n_2, \lfloor \frac{k}{r} \rfloor) + nk \quad (n = n_1 + n_2)$$

$$T(n, 1) = T(1, k) = 1$$

$$O(kn \log k) \quad (4) \quad O(kn) \quad (3) \quad O(kn \log n) \quad (2) \quad O(kn^2) \quad (1)$$

۶۶- از روش تقسیم و حل برای محاسبه‌ی حاصل ضرب دو عدد  $n$  بیتی  $A$  و  $B$  بدین شکل عمل

می‌کنیم. ابتدا قرار می‌دهیم  $A = 2^{\frac{n}{2}} A_1 + A_0$  و  $B = 2^{\frac{n}{2}} B_1 + B_0$  که در آن  $A_0, A_1, B_0, B_1$

و  $B_1$  اعداد  $\frac{n}{2}$  بیتی هستند. در این صورت  $AB = A_1 B_1 2^n + (A_1 B_0 + A_0 B_1) 2^{\frac{n}{2}} + A_0 B_0$

است که برای محاسبه‌ی آن ابتدا به طور بازگشتی (به همان صورت گفته شده)  $A_0 B_0$ ,

$A_1 B_0$ ,  $A_0 B_1$  و  $A_1 B_1$  را جداگانه محاسبه کرده و بعد با عمل شیفت و جمع اعداد  $n$  بیتی

$AB$  را با توجه به فرمول فوق محاسبه می‌کنیم. با فرض این که  $n$  توانی از ۲ است، زمان

اجرای الگوریتم فوق کدام است؟ بهترین گزینه را انتخاب کنید. (۹۳-IT)

$$O(n \log n) \quad (4) \quad O(n^2 \log n) \quad (3) \quad O(n^2) \quad (2) \quad O(n^{\log_2 3}) \quad (1)$$

۶۷- جواب رابطه‌ی بازگشتی زیر کدام است؟ (علوم کامپیوتر - ۹۳)

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{3}\right) + T\left(\frac{n}{4}\right) + n^2$$

$$T(n) = \theta(n^2) \quad (2) \quad T(n) = \theta(n) \quad (1)$$

$$T(n) = \theta(n^2 \log n) \quad (4) \quad T(n) = \theta(n \log n) \quad (3)$$

۶۸- کدام گزینه در مورد رابطه‌ی بازگشتی  $T(n)$  درست است؟

$$T(n) = \begin{cases} \lambda T\left(\frac{n}{r}\right) + \theta(1) & , n^r > m \\ m & , n^r \leq m \end{cases}$$

(علوم کامپیوتر - ۹۳)

( $m$  یک متغیر مستقل از  $n$  است.)

$$T(n) \in \theta\left(\left(\frac{n}{\sqrt{m}}\right)^3\right) \quad (2) \quad T(n) \in \theta\left(\frac{n}{\sqrt{m}}\right) \quad (1)$$

$$T(n) \in \theta\left(\log \frac{n}{\sqrt{m}}\right) \quad (4) \quad T(n) \in \theta\left(\frac{n^2}{\sqrt{m}}\right) \quad (3)$$

۶۹- فرض کنید  $C = \{1, \dots, 100\}$  مجموعه‌ی  $A_1$  تا  $A_n$  را در نظر بگیرید ( $n \leq 100$ ) که در ابتدا  $A_i = \{i\}$ ، رابطه‌ی  $R$  با ۱۵۰ عضو بر روی  $C$  نیز داده شده است. هر بار یکی از عناصر  $(a, b) \in R$  را به دلخواه انتخاب می‌کنیم، فرض کنید  $a \in A_i$  و  $b \in A_j$  اگر  $i \neq j$ ، در آن صورت  $A_j$  را در  $A_i$  ادغام می‌کنیم، یعنی قرار می‌دهیم  $A_j \leftarrow A_i \cup A_j$ ، حداکثر تعداد عمل ادغام چند تا است؟

(نرم‌افزار - ۹۴)

(۱) ۱۴۹ (۲) ۱۵۰ (۳) ۱۰۰ (۴) ۹۹

۷۰- رویه‌ی زیر  $a$  را به توان  $b$  می‌رساند (هر دو عدد صحیح هستند):

FASTPOWER( $a, b$ )

۱ if  $b == 1$

۲ return  $a$

۳  $c = a \times a$

۴  $\text{ans} = \text{FASTPOWER}(c, \lfloor b/2 \rfloor)$

۵ if  $b$  is odd

۶  $\text{ans} = a \times \text{ans}$

۷ return  $\text{ans}$

(۹۴ - IT)

تعداد دفعات فراخوانی این رویه چیست؟

(۱)  $O(\sqrt{b})$  (۲)  $O(\log b)$  (۳)  $O(b \log b)$  (۴)  $O(b)$

(۹۴ - IT)

۷۱- کدام گزینه حل تابع بازگشتی زیر است؟

$$T(n) = 7T\left(\left\lfloor \frac{n}{4} \right\rfloor\right) + O(\log n), \quad T(1) = 1$$

(۱)  $O(\log n)$  (۲)  $O(\log^2 n)$  (۳)  $O(\sqrt{n})$  (۴)  $O(\sqrt{n} \log n)$

۷۲- تابع  $M: \mathbb{Z}^+ \rightarrow \mathbb{Z}$  به صورت زیر تعریف می‌شود. کدام گزینه صحیح است؟ (علوم کامپیوتر - ۹۴)

$$M(n) = \begin{cases} n-10, & \text{if } n > 100 \\ M(M(n+11)), & \text{if } n \leq 100 \end{cases}$$

(۱) برای تمام  $n \leq 100$  مقدار  $M(n)$  برابر ۹۰ خواهد بود.

(۲) برای حداقل نیمی از  $n$ ‌های کوچکتر از ۱۰۰ مقدار  $M(n)$  برابر ۹۱ خواهد بود.

(۳) برای نیمی از  $n$ ‌های کوچکتر از ۱۰۰ جواب ندارد.

(۴) برای تمام  $n \leq 101$  مقدار  $M(n)$  برابر ۹۱ خواهد بود.

۷۳- جواب رابطه بازگشتی زیر چیست؟

(علوم کامپیوتر - ۹۴)

$$T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\log n}$$

- (۱)  $O(n)$  (۲)  $O(\log n)$   
(۳)  $O(n \log n)$  (۴)  $O(n \log(\log n))$

۷۴- جواب رابطه بازگشتی  $T(n) = T\left(\frac{n}{4}\right) + O(\log^2 n)$  کدام است؟

(دکتری علوم کامپیوتر - ۹۵)

- (۱)  $O(\log n)$  (۲)  $O(\log^2 n)$  (۳)  $O(\log^3 n)$  (۴)  $O(\log^4 n)$

۷۵- کدام گزینه در مورد رابطه بازگشتی  $T(n) = T(c_1 n) + T(c_2 n) + f(n)$  به طوری که

$f(n)$  یک تابع برحسب  $n$  و همچنین  $c_1, c_2 > 0$ ، درست است؟ (دکتری علوم کامپیوتر - ۹۵)

- (۱) اگر  $c_1 + c_2 = 1$  در این صورت  $T(n) = O(f(n))$   
(۲) اگر  $c_1 + c_2 = 1$  در این صورت  $T(n) = O(\log n \times O(f(n)))$   
(۳) اگر  $c_1 + c_2 < 1$  در این صورت  $T(n) = O(\log n \times O(f(n)))$   
(۴) اگر  $c_1 + c_2 > 1$  در این صورت  $T(n) = O(f(n))$

۷۶- مقدار تابع زیر به ازای  $n \geq 1$  چقدر است؟

(دکتری علوم کامپیوتر - ۹۵)

function  $F_n(n)$

```
{
  if  $n \leq 0$  return ۱;
  else
    return  $F_n(n-1) + 2^n + 4^n$ ;
}
```

- (۱)  $3^n - 2^n$   
(۲)  $2^n - 4^n$   
(۳)  $-\frac{7}{3} + \frac{1}{3} 4^{n+1} + 2^{n+1}$   
(۴)  $\frac{7}{3} + \frac{1}{3} 4^{n+1} + 2^n$

### پاسخنامه سؤال‌های چهارگزینه‌ای فصل سوم

۱- گزینه (۴). طبق قضیه Master،  $a=8$  و  $b=9$  و  $n^{\log_a b}$  رشد کمتری از  $n \lg n$  دارد

( $\log_a$  از یک کمتر است)، و شرایط قضیه Master برقرار است پس  $T(n) = \Theta(n \lg n)$ .

۲- گزینه (۲). تغییر متغیر می‌دهیم.

$$n = 2^m \Rightarrow \sqrt{n} = 2^{\frac{m}{2}}, \lg n = m$$

$$\Rightarrow T(2^m) = 2T(2^{\frac{m}{2}}) + m \xrightarrow{T(2^m)=S(m)} S(m) = 2S\left(\frac{m}{2}\right) + m$$

$$\Rightarrow S(m) = O(m \log m) \xrightarrow{m=\lg n} T(n) = O(\log n \log \log n)$$

۳- گزینه (۱). در رابطه بازگشتی  $T(n) = aT(n-b) + c$  اگر  $a, b, c$  ثابت مثبت باشند و

$a \neq 1$  آنگاه  $T(n) = \Theta(a^{\frac{n}{b}})$  پس گزینه ۱ از مرتبه  $\Theta(2^{\frac{n}{2}})$  یعنی نمایی است. گزینه ۴ با

جایگذاری برابر است با

$$T(n) = T(n-1) + n^2 = T(n-2) + (n-1)^2 + n^2 = T(0) + 1^2 + 2^2 + \dots + n^2 = \Theta(n^3)$$

گزینه ۲ و ۳ طبق قضیه Master به ترتیب از مرتبه  $\Theta(n)$  و  $\Theta(n^2)$  هستند.

۴- گزینه (۳). در این نوع تست‌ها که جواب دقیق بر حسب  $n$  در گزینه‌ها آمده است می‌توان به

ازای  $n$ های مختلف گزینه‌ها را آزمایش کرد. مثلاً به ازای  $n=2$  طبق صورت سوال

$T(2)=1$  حال در گزینه ۱،  $T(2)=-1$  و در گزینه ۲،  $T(2)=21$  و در گزینه ۳،

$T(2)=1$  و در گزینه ۴،  $T(2)=5$  که فقط گزینه ۳ می‌تواند صحیح باشد.

ولی برای حل دقیق تعریف می‌کنیم  $n = 2^m$  و داریم:

$$T(2^m) = 4T(2^{\frac{m}{2}}) + 1$$

حال تعریف می‌کنیم  $F(m) = T(2^m)$  پس  $F\left(\frac{m}{2}\right) = T(2^{\frac{m}{2}})$  در نتیجه:

$$F(m) = 4F\left(\frac{m}{2}\right) + 1, F(1) = T(2) = 1$$

حال می‌توان با جایگذاری  $F(m)$  را یافت:

$$F(m) = 4F\left(\frac{m}{2}\right) + 1 = 4\left[4F\left(\frac{m}{4}\right) + 1\right] + 1 = 4^2 F\left(\frac{m}{4}\right) + 4 + 1$$

$$= 4^2 \left[4F\left(\frac{m}{8}\right) + 1\right] + 4 + 1 = 4^3 F\left(\frac{m}{8}\right) + 4^2 + 4 + 1 = \dots = 4^k F\left(\frac{m}{4^k}\right) + 4^{k-1} + \dots + 4 + 1$$



$\frac{m}{r^k}$  باید برابر ۱ باشد و چون  $F(1) = 1$  داریم:

$$F(m) = r^k(1) + r^{k-1} + \dots + r + 1 = \frac{r^{k+1} - 1}{r - 1} = \frac{1}{r-1} [r(r^k) - 1]$$

و چون  $\frac{m}{r^k} = 1$  پس  $k = \lg m$  در نتیجه:

$$F(m) = \frac{1}{r-1} [r m^{\lg r} - 1]$$

حال با توجه به اینکه  $n = r^m$  بنابراین  $m = \lg n$  در نتیجه:

$$T(n) = \frac{1}{r-1} [r \lg^{\lg r} n - 1]$$

۵- گزینه (۳).  $\lg n!$  با  $n \lg n$  هم رشد است پس:

$$T(n) = r T\left(\frac{n}{r}\right) + n \lg n = \theta(n \lg^{\lg r} n)$$

یادآوری می‌کنیم رابطه  $T(n) = a T\left(\frac{n}{b}\right) + n^{\log_b a} \lg^k n$  که  $a, b$  ثابت و  $k \geq 0$  از مرتبه  $\theta(n^{\log_b a} \lg^{k+1} n)$  است.

۶- گزینه (۲).  $\lg n!$  با  $n \lg n$  هم رشد است و  $n^{\log_{99} 100}$  از  $n \lg n$  رشد بیشتری دارد. چون  $\log_{99} 100$  کمی از یک بزرگتر است و شرایط قضیه Master برقرار است.

۷- گزینه (۲). معادله مشخصه تشکیل می‌دهیم:

$$x^2 - 3x - 4 = 0 \xrightarrow{-1, 4} T(n) = \alpha_1(4)^n + \alpha_2(-1)^n$$

شرط اولیه را اعمال می‌کنیم:

$$\begin{cases} T(0) = \alpha_1 + \alpha_2 = 0 \\ T(1) = 4\alpha_1 - \alpha_2 = 1 \end{cases} \rightarrow \alpha_1 = \frac{1}{5}, \alpha_2 = -\frac{1}{5}$$

$$T(n) = \frac{1}{5}(4^n) - \frac{1}{5}(-1)^n = \theta(4^n) \text{ بنابراین}$$

۸- گزینه (۱). طرفین رابطه بازگشتی را به توان ۲ می‌رسانیم و تعریف می‌کنیم  $T^{\vee}(n) = a_n$  بنابراین:

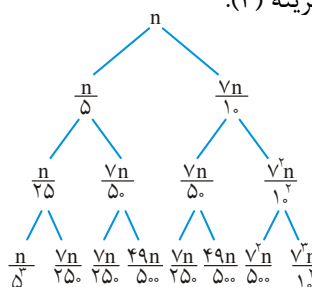
$$T^{\vee}(n) = \frac{1}{r} T^{\vee}(n-1) + \frac{1}{r} T^{\vee}(n-2) + n \rightarrow a_n = \frac{1}{r} a_{n-1} + \frac{1}{r} a_{n-2} + 1^n(n)$$

معادله مشخصه این رابطه عبارت است از  $(x^2 - \frac{1}{r}x - \frac{1}{r})(x-1)^2 = 0$  که دارای ریشه‌های

$$a_n = (\alpha_1 + \alpha_2 n + \alpha_3 n^2)(1)^n + \alpha_4 \left(-\frac{1}{r}\right)^n \text{ است بنابراین جواب این رابطه برابر } -\frac{1}{r}, 1, 1, 1$$

است که از مرتبه  $a_n = \theta(n^2)$  است (البته به شرطی که  $\alpha_3 \neq 0$  که با توجه به شروط اولیه چنین است) حال با توجه به اینکه  $a_n = T^2(n)$  پس  $T^2(n) = \theta(n^2)$  در نتیجه  $T(n) = \theta(n)$ .

گزینه (۳). -۹



$$\text{جمع سطح } 0 = n = \left(\frac{4}{5}\right)^0 n$$

$$\text{جمع سطح } 1 = \frac{n}{5} + \frac{4n}{5} = \frac{4n}{5} = \left(\frac{4}{5}\right)^1 n$$

$$\text{جمع سطح } 2 = \frac{n}{25} + \frac{4n}{25} + \frac{4n}{25} + \frac{16n}{25} = \frac{16n}{25} = \left(\frac{4}{5}\right)^2 n$$

$$\text{جمع سطح } 3 = \frac{n}{125} + \frac{4n}{125} + \frac{4n}{125} + \frac{16n}{125} + \frac{4n}{125} + \frac{16n}{125} + \frac{16n}{125} + \frac{64n}{125} = \left(\frac{4}{5}\right)^3 n$$

حال باید ارتفاع درخت را بیابیم. ارتفاع این درخت حداکثر  $\log_{5/4} n$  می‌باشد، پس:

$$T(n) \leq \left(\frac{4}{5}\right)^0 n + \left(\frac{4}{5}\right)^1 n + \left(\frac{4}{5}\right)^2 n + \left(\frac{4}{5}\right)^3 n + \dots + \left(\frac{4}{5}\right)^{\log_{5/4} n} = \sum_{i=0}^{\log_{5/4} n} \left(\frac{4}{5}\right)^i n$$

گزینه (۲). در این سوال، حل رابطه بازگشتی مطلوب نیست. معادله مشخصه

$$x^3 - x^2 - 2x - 1 = 0$$

سوال فقط برای جملات یک کران بالا می‌خواهد.

با توجه به اینکه  $G_{n-3} < G_{n-2} < G_{n-1}$  پس داریم:

$$G_n = G_{n-1} + 2G_{n-2} + G_{n-3} < G_{n-1} + 2G_{n-1} + G_{n-1} = 4G_{n-1} \approx 4^n$$

یادآوری می‌کنیم رابطه  $T(n) = aT(n-b) + c$  که  $c, b, a$  ثابت مثبت و  $a \neq 1$  از مرتبه

$\theta(a^{n/b})$  است. در ضمن در این سوال می‌توان برای  $G_n$  کران پایین نیز یافت:

$$G_n = G_{n-1} + 2G_{n-2} + G_{n-3} > G_{n-3} + 2G_{n-3} + G_{n-3} = 4G_{n-3} \approx 4^{n/3}$$

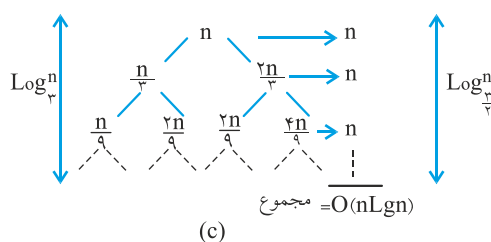
پس  $4^{n/3} < G_n < 4^n$ .

۱۱- گزینه (۴). یادآوری می‌کنیم  $T(n) = aT\left(\frac{n}{b}\right) + n^{\log_b a} \cdot \lg^k n$  که  $k \geq 0$  از مرتبه  $\theta(n^{\log_b a} \cdot \lg^{k+1} n)$  است.

$$T(n) = T\left(\frac{n}{3}\right) + 1(\log n)^2$$

$$a=1, b=\frac{3}{2}, n^{\log_b a} = n^{\log_{\frac{3}{2}} 1} = n^0 = 1 \rightarrow T(n) = \theta(\log n)^2$$

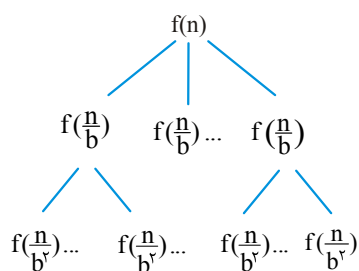
۱۲- گزینه (۲). درخت بازگشت به شکل مقابل است. اندازه شاخه سمت چپ  $\log_{\frac{3}{2}} n$  و شاخه سمت راست  $\log_{\frac{3}{2}} n$  است که ارتفاع درخت برابر فاصله ریشه تا دورترین برگ است و چون شاخه سمت راست بلندتر است پس ارتفاع درخت برابر  $\log_{\frac{3}{2}} n$  است.



۱۳- گزینه (۴).

$$T(n) = aT\left(\frac{n}{b}\right) + f(n) = \underbrace{T\left(\frac{n}{b}\right) + \dots + T\left(\frac{n}{b}\right)}_{a \text{ بار}} + f(n)$$

در درخت بازگشت هر نود داخلی دارای  $a$  زیر درخت است ( $a$  عدد طبیعی  $a \geq 1$  است). پس



درخت  $a$ -ary ( $a$  تایی) است. برای یافتن ارتفاع یا

عمق درخت باید  $\frac{n}{b^k} = 1$  پس  $n = b^k$  در نتیجه

$k = \log_b n$  ارتفاع درخت است. تعداد برگ در درخت

بر  $a$  تایی، برابر  $a$  به توان ارتفاع است. پس تعداد برگ

$a^{\log_b n}$  است که با  $n^{\log_b a}$  برابر است.

۱۴- گزینه (۳). اگر  $A$  زوج باشد که با هر فراخوانی نصف می‌شود. ولی اگر  $A$  فرد باشد از  $A$  یک واحد کم شده زوج می‌شود و در فراخوانی بعدی نصف می‌شود. پس با هر مقدار  $A$ ، تعداد فراخوانیها لگاریتمی است.

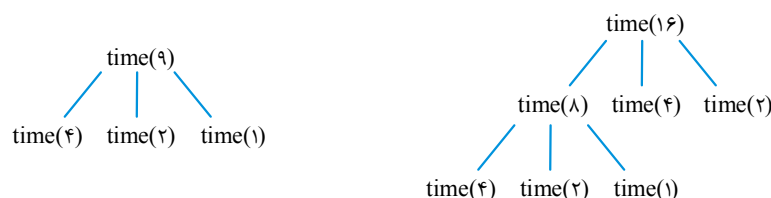
۱۵- گزینه (۲). در واقع می‌خواهیم رابطه بازگشتی  $g(n) = 5g(n-1) - 6g(n-2)$  با شروط اولیه  $g(0) = 0$  و  $g(1) = 1$  را حل کنیم. از آنجایی که جواب دقیق در گزینه‌ها هست می‌توان آزمایش کرد مثلاً به ازای  $n=1$  فقط گزینه ۲ برابر  $g(1) = 1$  می‌شود. برای حل دقیق می‌توان معادله مشخصه نوشت:

$$x^2 - 5x + 6 = 0 \xrightarrow{2,3} g(n) = \alpha_1(3)^n + \alpha_2(2)^n \xrightarrow[\alpha_1=1, \alpha_2=-1]{\text{اعمال شروط اولیه}} g(n) = 3^n - 2^n$$

۱۶- گزینه (۲). باید رابطه بازگشتی  $\text{Test}(n) = 2\text{Test}\left(\frac{n}{2}\right) + 6n - 1$  با شرط اولیه  $\text{Test}(1) = 1$  را حل کنیم. طبق قضیه Master مرتبه این رابطه  $\theta(n \lg n)$  است پس گزینه ۴ نمی‌تواند صحیح باشد. و چون گزینه‌ها جواب دقیق را می‌خواهند می‌توان آزمایش کرد. به ازای  $n=1$  فقط گزینه ۲ برابر  $\text{Test}(1) = 1$  می‌شود، بنابراین گزینه ۲ صحیح است. برای حل دقیق می‌توان از روش جایگذاری از بالا استفاده کرد.

۱۷- گزینه (۲). تعداد سطرها برابر تعداد باری است که `writeln` اجرا می‌شود. اگر فرض کنیم  $T(n)$  تعداد `concow(n)` سطر چاپ می‌کند پس  $T\left(\frac{n}{2}\right)$  سطر چاپ می‌کند. پس  $\text{writeln}(\text{concow}(n \div 2))$  تعداد  $T\left(\frac{n}{2}\right) + 1$  سطر چاپ می‌کند و به خاطر حلقه `for` می‌توان نوشت:  $T(n) = n\left(T\left(\frac{n}{2}\right) + 1\right)$  که گزینه ۲ صحیح است در ضمن شرط اولیه  $T(1) = 1$  می‌باشد. در این تست اگر تعداد باری که `stop` چاپ می‌شود، مطلوب بود جواب گزینه ۳ می‌شد چون به ازای  $n > 1$ ، `stop` چاپ نمی‌شود و فقط فراخوانی بازگشتی انجام می‌شود که  $T(n) = n.T\left(\frac{n}{2}\right)$  است.

۱۸- گزینه (۲). به ازای هر  $x \geq 8$ ، سه فراخوانی بازگشتی انجام می‌شود. و به ازای  $x < 8$  یک بار پیام چاپ می‌شود، با توجه به درخت فراخوانی  $\text{time}(9)$  و  $\text{time}(16)$  نتیجه بگیرید که  $t(9) = 3$  و  $t(16) = 5$  که گزینه ۲ صحیح است.



$$\begin{array}{c}
 f(\circ, \wedge, \vee) \\
 \swarrow \quad \downarrow \quad \searrow \\
 g(\vee, \vee) \quad f(\circ, \vee, \vee) \quad f(\vee, \wedge, \vee) \\
 \swarrow \downarrow \searrow \quad \swarrow \downarrow \searrow \quad \swarrow \downarrow \searrow \\
 g(\vee, \vee) \quad f(\circ, \vee, \vee) \quad f(\vee, \vee, \vee) \quad g(\vee, \vee) \quad f(\vee, \vee, \vee) \quad f(\vee, \wedge, \vee) \\
 \swarrow \downarrow \searrow \quad \swarrow \downarrow \searrow \quad \swarrow \downarrow \searrow \quad \swarrow \downarrow \searrow \quad \swarrow \downarrow \searrow \quad \swarrow \downarrow \searrow \\
 g(\vee, \vee) \quad f(\circ, \vee, \vee) \quad f(\vee, \vee, \vee) \quad g(\vee, \vee) \quad f(\vee, \vee, \vee) \quad f(\vee, \wedge, \vee) \quad g(\vee, \vee) \quad f(\vee, \vee, \vee) \quad f(\vee, \wedge, \vee)
 \end{array}$$

۲. گزینه (۳). در رابطه  $T(n) = \sum_{i=1}^k T\left(\frac{n}{a_i}\right) + \theta(n)$  اگر  $\sum \frac{1}{a_i}$  برابر ۱ شود مرتبه  $\theta(n \lg n)$  و

زیرا  $\frac{1}{\delta} + \frac{\gamma}{1} = \frac{9}{1} < 1$  و  $T(n) = T\left(\frac{n}{\delta}\right) + T\left(\frac{\gamma n}{1}\right) + \theta(n) = \theta(n)$  زیرا

$T(n) = n + T(n-1) = \frac{n(n+1)}{2} = \theta(n^2)$  : T(n) را حل کنیم:

```
f(n)
{
  if(n == 1) return 1;
  return n + f(n - 1);
}
```

۲۳- گزینه (۲). اگر  $T(n)$  زمان اجرای  $\text{algo}(i, n)$  باشد، پس زمان اجرای  $\text{algo}(i, k) = \text{algo}\left(i, \left\lfloor \frac{i+n}{2} \right\rfloor\right)$  و  $\text{algo}(k+1, j) = \text{algo}\left(\left\lfloor \frac{j+n}{2} \right\rfloor + 1, n\right)$  هر کدام  $T\left(\frac{n}{2}\right)$  است. بنابراین می‌توان نوشت  $T(n) = 2T\left(\frac{n}{2}\right) + 1 = \theta(n)$ .

۲۴- گزینه (۱). با توجه به اینکه هر بار فراخوانی بازگشتی با  $\frac{n}{4}$  انجام می‌شود می‌توان نوشت  $T(n) = T\left(\frac{n}{4}\right) + 1$  که طبق قضیه Master،  $T(n) = \theta(\lg n)$ . لازم به ذکر است که می‌توان از نماد  $O$  نیز استفاده کرد.

۲۵- گزینه (۱). برای پیچیدگی زمانی این الگوریتم می‌توان نوشت  
 $T(n) = T(n-1) + T(n-2) + 1$  و  $T(0) = T(1) = 1$  و با توجه به اینکه  
 $T(n-2) < T(n-1)$  (به ازای  $n > 2$ ) پس:

$$T(n) = T(n-1) + T(n-2) + 1 < T(n-1) + T(n-1) + 1 = 2T(n-1) + 1 \approx 2^n$$

$$T(n) = T(n-1) + T(n-2) + 1 > T(n-2) + T(n-2) + 1 = 2T(n-2) + 1 \approx 2^{\frac{n}{2}}$$

۲۶- گزینه (۱). اگر  $mg(n)$ ، تعداد  $A(n)$  ستاره چاپ می‌کند پس  $mg(n-1)$  و  $mg(n-2)$  به ترتیب  $A(n-1)$  و  $A(n-2)$  ستاره چاپ می‌کنند. و همچنین  $print$  نیز یک ستاره چاپ می‌کند پس می‌توان نوشت:

$$A(n) = 1 + A(n-1) + A(n-2) + 1 = A(n-1) + A(n-2) + 2$$

۲۷- گزینه (۴).

$$F(1, 1) = F(1, 2) + 1 = F(1, 3) + 2 + 1 = F(1, 4) + 3 + 2 + 1$$

$$= \dots = F(1, 9) + 8 + 7 + \dots + 1 = F(1, 10) + 9 + 8 + 7 + \dots + 1$$

$$= F(1, 11) + 10 + 9 + 8 + 7 + \dots + 1 = 0 + 10 + 9 + 8 + 7 + \dots + 1 = \frac{10 \times 11}{2} = 55$$

$$pp(n) = 2n + pp(n-1) - 1 = pp(n-1) + 2n - 1 \quad \text{گزینه (۲).}$$

با توجه به این رابطه بازگشتی،  $pp(n)$  برابر جمع اعداد فرد است چون:

$$pp(0) = 0, \quad pp(1) = pp(0) + 2 \times 1 - 1 = 1, \quad pp(2) = pp(1) + (2 \times 2 - 1) = 1 + 3,$$

$$pp(3) = pp(2) + (2 \times 3 - 1) = 1 + 3 + 5, \dots, \quad pp(n) = 1 + 3 + 5 + \dots + (2n - 1)$$

حال این مجموع یک تصاعد حسابی است که  $n$  تا جمله دارد و حاصلجمع آن برابر است با:

$$1 + 3 + 5 + \dots + (2n - 1) = \frac{(1 + 2n - 1) \times n}{2} = n^2$$

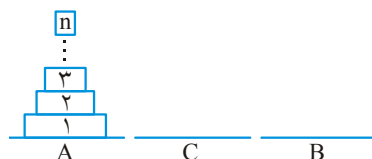
۲۹- گزینه (۳). انتقال  $n$  دیسک از  $from$  به  $to$  با کمک  $help$  سه مرحله دارد:

۱- انتقال  $n-1$  دیسک از  $from$  به  $help$  با کمک  $to$ .

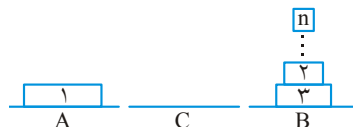
۲- انتقال یک دیسک از  $from$  به  $to$ .

۳- انتقال  $n-1$  دیسک از  $help$  با کمک  $from$  به  $to$  که این مرحله در گزینه‌ی ۳ آمده است.

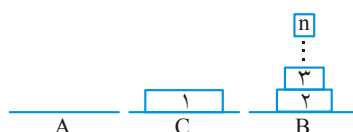
۳۰- گزینه (۲). در این سوال، میله  $A$  مبدا،  $C$  کمکی و  $B$  مقصد است:



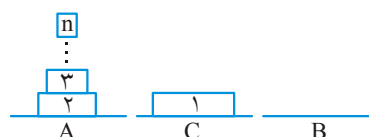
مرحله ۱: انتقال  $n-1$  دیسک از A با کمک C به B.



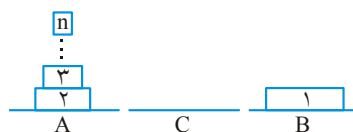
مرحله ۲: انتقال یک دیسک از C به A.



مرحله ۳: انتقال  $n-1$  دیسک از B با کمک C به A.



مرحله ۴: انتقال یک دیسک از C به B.



مرحله ۵: انتقال  $n-1$  دیسک از A با کمک C به B.



بنابراین  $T(n) = T(n-1) + 1 + T(n-1) + 1 + T(n-1) = 3T(n-1) + 2$

۳۱- گزینه (۱). بدترین حالت زمانی است که تمام سکه‌ها در میله شماره ۱ باشند و در میله شماره ۳ هیچ سکه‌ای نباشد که در این صورت مسئله همان هانوی اصلی است و  $2^{100} - 1$  انتقال لازم است.

۳۲- گزینه (۴). اگر  $Mystery(n)$  تعداد  $T(n)$  ستاره چاپ می‌کند پس  $Mystery(n-1)$  و  $Mystery(n-2)$  هر کدام  $T(n-1)$  و  $T(n-2)$  ستاره چاپ می‌کنند، همچنین  $print"****"$  و  $print"****"$  به ترتیب ۳ و ۴ ستاره چاپ می‌کنند پس:

$$T(n) = T(n-1) + 3 + T(n-2) + 4 + T(n-1) = 2T(n-1) + T(n-2) + 7$$

۳۳- گزینه (۳).  $n$  هر بار یک واحد کم می‌شود پس برحسب  $n$  زمان خطی است و چون  $m$  هر بار نصف می‌شود، برحسب  $m$  لگاریتمی است و هر کدام زودتر به ۱ یا کوچک‌تر از ۱ برسند الگوریتم خاتمه می‌یابد.

۳۴- گزینه (۲).

$$DC(2, 6) = DC(2, 3) * DC(2, 3) = 8 * 8 = 64$$

$$DC(2, 3) = 2 * DC(2, 2) = 2 * 4 = 8$$

$$DC(2, 2) = DC(2, 1) * DC(2, 1) = 2 * 2 = 4$$

۳۵- گزینه (۲). این تابع  $B$  بار  $A$  را جمع می‌کند یعنی  $A * B$

۳۶- گزینه (۳). زمان این تابع را می‌توان برابر با تعداد فراخوانیها فرض کرد. اگر  $T(n)$  تعداد فراخوانیها برای  $test(n)$  باشد، به ازای  $n \leq 2$ ،  $T(n) = 1$  و به ازای  $n > 2$ ،

$$T(n) = 2T(n-2) + 1 \quad \text{که این رابطه از مرتبه } T(n) \in \Theta(2^{\frac{n}{2}}) \text{ است.}$$

۳۷- گزینه (۳). اگر  $T(n)$  زمان اجرای الگوریتم گفته شده باشد، آنگاه می‌توان نوشت

$$T(n) = 5T\left(\frac{n}{5}\right) + O(n) \quad \text{که از مرتبه } T(n) \in \Theta(n^{\log_5 5}) \text{ است.}$$

۳۸- گزینه (۲). اگر  $T(n)$  زمان اجرا فرض شود داریم:

$$\text{روش ۱: } T(n) = 3T\left(\frac{n}{3}\right) + \Theta(n^2 \sqrt{n}) = \Theta(n^2 \sqrt{n})$$

$$\text{روش ۲: } T(n) = 4T\left(\frac{n}{4}\right) + \Theta(n^2) = \Theta(n^2 \lg n)$$

$$\text{روش ۳: } T(n) = 5T\left(\frac{n}{5}\right) + \Theta(n \lg n) = \Theta(n^{\lg 5})$$

روش‌های ۱ و ۳ از مرتبه  $\Theta(n^{2+\epsilon})$  و  $\epsilon > 0$  هستند. پس روش ۲ سریع‌تر از روش‌های ۱ و ۳ است.

۳۹- گزینه (۳). استراسن ۷ بار فراخوانی بازگشتی با اندازه  $\frac{n}{3}$  و ۱۸ بار عمل جمع و تفریق

ماتریس‌های با اندازه  $\frac{n}{3} \times \frac{n}{3}$  انجام می‌دهد، تعداد جمع و تفریق‌های استراسن به ازای

$$n > 1 \quad \text{برابر } T(n) = 7T\left(\frac{n}{3}\right) + 18\left(\frac{n}{3}\right)^2 \text{ است و به ازای } n = 1 \text{ هیچ عمل جمع و تفریق}$$

انجام نمی‌شود پس  $T(1) = 0$ .

۴۰- گزینه (۲). اگر  $T(n)$  تعداد ضرب عددی استراسن باشد، به ازای  $n > 1$  داریم

$$T(n) = 7T\left(\frac{n}{3}\right) \quad \text{و به ازای } n = 1 \text{ داریم } T(1) = 1 \text{ زیرا برای ماتریس‌های یک در یک، یک}$$

عمل ضرب عددی صورت می‌گیرد. باید  $T(4)$  را محاسبه کنیم:

$$T(4) = 7T(2) = 7 \times 7 \times T(1) = 49$$



۴۱- گزینه (۳). تعداد ضرب‌ها  $T(n) = 7T\left(\frac{n}{7}\right)$  و  $T(1) = 1$  می‌باشد ولی در این تست، به

ماتریس  $2 \times 2$  برسیم، فراخوانی تمام است و نه به ماتریسهای  $1 \times 1$ . و می‌دانیم ضرب عادی ماتریسهای  $2 \times 2$ ، هشت عمل ضرب لازم دارد یعنی  $T(2) = 8$  پس:

$$T(8) = 7T(4) = 7(7T(2)) = 7 \times 7 \times 8 = 392$$

۴۲- گزینه (۲). رابطه بازگشتی برای تعداد فراخوانی‌ها (نه تعداد ضرب یا جمع‌ها) برابر است با:

$$T(n) = 7T\left(\frac{n}{7}\right) + 1, \quad T(2) = 1 \Rightarrow T(8) = 57$$

۴۳- گزینه (۴). اگر  $T(n)$  زمان اجرای این روش باشد می‌توان نوشت

$$T(n) = 6T\left(\frac{n}{7}\right) + 28\left(\frac{n^2}{7}\right)$$

که از مرتبه  $\theta(n^{\lg 6})$  است.

$$A_k \cdot V = \begin{bmatrix} A_{k-1} & A_{k-1} \\ A_{k-1} & -A_{k-1} \end{bmatrix} \begin{bmatrix} V_1 \\ V_2 \end{bmatrix} = \begin{bmatrix} A_{k-1}V_1 + A_{k-1}V_2 \\ A_{k-1}V_1 - A_{k-1}V_2 \end{bmatrix} \quad \text{گزینه (۱).} \quad ۴۴-$$

$V_1$  و  $V_2$  دو بردار ستونی با طول  $2^{k-1}$  هستند. اگر زمان ضرب  $A_k \cdot V$  را  $T(n)$  فرض کنیم، زمان ضرب  $A_{k-1} \cdot V_1$  و  $A_{k-1} \cdot V_2$  هر کدام  $T\left(\frac{n}{2}\right)$  است. و از آنجایی که ۴ عمل ضرب با اندازه  $\frac{n}{2}$  وجود دارد که فقط ۲ تای آن متفاوت است می‌توان نوشت:

$$T(n) = 2T\left(\frac{n}{2}\right) + \theta(n) = \theta(n \lg n)$$

توجه کنید که عمل جمع و تفریق  $A_{k-1}V_1$  و  $A_{k-1}V_2$  از مرتبه  $\theta(n)$  است. ۴۵- گزینه (۳). اولاً برای ضرب اعداد بزرگ نمی‌توان از روش معمولی ضرب استفاده کرد. دو راه برای ضرب وجود دارد.

**روش ۱:** عدد  $n$  رقمی را تا  $4n$  رقم با افزودن  $3n$  رقم صفر سمت چپ آن گسترش دهیم و سپس ضرب کنیم (گزینه ۴).

**روش ۲:** عدد  $4n$  رقمی را ۴ قسمت  $n$  رقمی کنیم و هر قسمت را در عدد  $n$  رقمی ضرب کنیم و نتایج را با هم ترکیب کنیم. واضح است که زمان روش اول ۱۶ برابر زمان ضرب دو عدد  $n$  رقمی و زمان روش دوم ۴ برابر زمان ضرب دو عدد  $n$  رقمی است.

۴۶- گزینه (۲). نیمی از ضرایب  $B$  صفر است و این مسئله فقط در بار اول تقسیم چنین است. تقسیم و غلبه به این صورت است که مسئله به اندازه  $n$  به  $\frac{n}{4}$  و  $\frac{n}{4}$  به  $\frac{n}{4}$  و ... تقسیم می‌شود. در این مسئله صفر بودن نصف ضرایب  $B$  هیچ تأثیری در شکستن‌های بعدی مسئله ندارد و مسئله با همان زمان الگوریتم اصلی حل می‌شود. در ضمن در این سوال  $\log_4^3 n$  به اشتباه  $n \log_4^3$  آمده است.

۴۷- گزینه (۲). شرط if،  $n+1$  بار بررسی می‌شود. جمله  $rsum := 0$  یک بار اجرا می‌شود و جمله  $rsum := rsum(\dots)$ ،  $n$  بار اجرا می‌شود:  $2n+2 = n+1+1+n$  گام

۴۸- گزینه (۳). می‌توان نشان داد، همیشه مقسوم‌علیه از دو برابر باقیمانده بزرگتر است:

$$\begin{array}{r} a \mid b \\ \vdots \\ q \\ \hline r \end{array} \Rightarrow a = bq + r, 0 \leq r < b, q \geq 1 \Rightarrow r < bq \Rightarrow r + r < bq + r \Rightarrow 2r < a$$

بنابراین  $(a \bmod b) < \frac{a}{2}$ . یعنی می‌توان گفت تابع gcd در هر بار فراخوانی،  $a$  را تقریباً

نصف می‌کند پس مرتبه این الگوریتم  $O(\log^a)$  می‌باشد.

۴۹- گزینه (۴).

$$\text{Rec}(\Delta, r) \rightarrow R = \text{Rec}(\Delta, 1) \rightarrow R = \text{Rec}(\Delta, 0) = 1$$

مقدار  $\text{Rec}(\Delta, 0)$  برابر ۱ است پس  $R = 1$  می‌شود. سپس  $\text{Rec}(\Delta, 1)$  از پشته برداشته می‌شود و جملات باقیمانده آن اجرا می‌شوند که ابتدا  $R = 1 * 1 = 1$  شده و چون شرط  $q \bmod 2 = 0$  صحیح نیست مقدار  $1 * 5 = 5$  برمی‌گردد و  $R = 5$  می‌شود. سپس  $\text{Rec}(\Delta, 3)$  از پشته برداشته می‌شود و ابتدا  $R = 5 * 5 = 25$  شده و چون شرط if صحیح نیست  $25 * 5$  یعنی ۱۲۵ برمی‌گردد.

۵۰- گزینه (۲). روش اول:

برای سهولت فرض کنید  $b = 1$  و از روش جایگذاری با تکرار استفاده کنید:

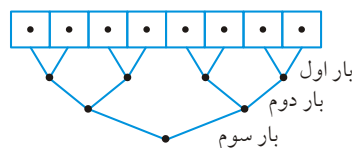
$$\begin{aligned} f(n) &= n^2 + nf(n-1) = n^2 + n[(n-1)^2 + (n-1)f(n-2)] \\ &= n^2 + n(n-1)^2 + n(n-1)f(n-2) = n^2 + n(n-1)^2 + n(n-1)[(n-2)^2 + (n-2)f(n-3)] \\ &= n^2 + n(n-1)^2 + n(n-1)(n-2)^2 + n(n-1)(n-2)f(n-3) = \dots \\ &= n^2 + n(n-1)^2 + n(n-1)(n-2)^2 + \dots + \underbrace{n(n-1)(n-2)\dots f(n-(n-1))}_{n! \times a} = \theta(n!) \end{aligned}$$

روش دوم:

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{f(n)}{n!} &= \lim_{n \rightarrow \infty} \frac{bn^2 + nf(n-1)}{n!} = \lim_{n \rightarrow \infty} \left( \frac{bn^2}{n!} + \frac{f(n-1)}{(n-1)!} \right) \\ &= \lim_{n \rightarrow \infty} \frac{f(n-1)}{(n-1)!} = \lim_{n \rightarrow \infty} \frac{b(n-1)^2 + (n-1)f(n-2)}{(n-1)!} = 0 + \lim_{n \rightarrow \infty} \frac{f(n-2)}{(n-2)!} \\ &= \dots = \frac{f(1)}{1!} = a \Rightarrow f(n) = \theta(n!) \end{aligned}$$

روش سوم: به ازای  $b = 0$  رابطه تبدیل به  $f(n) = n \cdot f(n-1)$  می‌شود که  $\theta(n!)$  است.

- ۵۱- گزینه (۲). به طور کلی با  $\frac{n}{4}$  پردازنده، اگر بخواهیم  $n$  عدد را جمع (یا ضرب یا هر عملی) کنیم.  $\lceil \lg n \rceil$  زمان لازم است.  
به ازای  $n = 8$ ،  $3$  بار ( $3$  کلاک) لازم است:



- ۵۲- گزینه (۴). اگر print از مرتبه  $\theta(1)$  باشد زمان این تابع  $\theta(n!)$  می‌شود و اگر از مرتبه  $\theta(n)$  باشد، زمان تابع  $\theta(n \cdot n!)$  می‌شود. به متن درس رجوع کنید.
- ۵۳- گزینه (۱). دستور if پایین تابع، با فراخوانی بازگشتی درواقع یک حلقه ایجاد می‌کند. حلقه for نیز یک حلقه مستقل است که  $A[i]$  بار اجرا می‌شود. اگر همه  $A[i]$  ها برابر یک باشند حلقه for یک بار اجرا می‌شود ولی فراخوانی  $n$  بار انجام می‌شود و  $O(n)$  می‌شود. اگر  $A[i]$  ها همگی  $n$  باشند، حلقه for از مرتبه  $O(n)$  می‌شود و فراخوانی بازگشتی صورت نمی‌گیرد. اگر  $A[i]$  ها برابر  $k$  باشند، حلقه for،  $k$  بار و فراخوانی ها  $\frac{n}{k}$  بار اجرا می‌شود و مرتبه  $O\left(k \times \frac{n}{k}\right) = O(n)$  می‌باشد.
- ۵۴- گزینه (۴).

$$F(10, 1) = F(10, 2) + 1$$

$$F(10, 2) = F(10, 3) + 2$$

⋮

$$F(10, 9) = F(10, 10) + 9$$

$$F(10, 10) = F(10, 11) + 10$$

$$F(10, 11) = 0$$

در نتیجه با بازگشت مقدار  $F(10, 1) = 1 + 2 + \dots + 10 = 55$  یعنی  $\frac{10 \times 11}{2}$  حاصل می‌شود.

- ۵۵- گزینه (۳ و ۴).

$$T(n) = 2T\left(\frac{n}{4}\right) + \sqrt{n} \Rightarrow n^{\log_4 2} = \sqrt{n} \Rightarrow T(n) \in \theta(\sqrt{n} \lg \sqrt{n})$$

که  $\lg \sqrt{n} = \frac{1}{2} \lg n$  پس  $T(n) \in \theta(\sqrt{n} \lg n)$ ، که گزینه ۴ صحیح است ولی در کلید گزینه ۳ نیز درست اعلام شده است که بخاطر استفاده از نماد  $O$  می‌باشد زیرا می‌توان گفت  $T(n) \in O(n \lg n)$  یعنی رشد  $T(n)$  از  $n \cdot \lg n$  بیشتر نیست.

۵۶- گزینه (۴).

$$T(n) = \frac{3}{4}T\left(\frac{n}{4}\right) - \frac{1}{4}T\left(\frac{n}{4}\right) - \frac{1}{n}, \quad n \geq 3$$

$$n = 2^m, \quad T(2^m) = F(m)$$

$$\Rightarrow F(m) = \frac{3}{4}F(m-1) - \frac{1}{4}F(m-2) - \left(\frac{1}{4}\right)^m$$

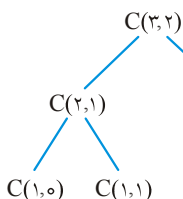
$$\Rightarrow \text{ریشه‌ها} = 1, \frac{1}{4}, \frac{1}{4} \quad \text{مشخصه: } \left(r^2 - \frac{3}{4}r + \frac{1}{4}\right)\left(r - \frac{1}{4}\right) = 0$$

$$\Rightarrow F(m) = (\alpha_1 + \alpha_2 m)\left(\frac{1}{4}\right)^m + \alpha_3 (1)^m$$

$$\Rightarrow T(n) = (\alpha_1 + \alpha_2 \lg n) \frac{1}{n} + \alpha_3 \Rightarrow T(n) \in \theta\left(\frac{\lg n}{n}\right)$$

۵۷- گزینه (۳). به عنوان مثال برای محاسبه  $C(3,2)$  به درخت

مقابل توجه کنید:



تعداد فراخوانیها برابر ۵ است یعنی  $2\binom{3}{2} - 1$ . ولی تعداد

فراخوانی بازگشتی یکی کمتر است.

۵۸- گزینه (۱). مشابه تست قبل.

۵۹- گزینه صحیح وجود ندارد.

$$F(3,6) = F(2,6) + F(2,5) = 2 + 2 = 4$$

$$F(2,6) = F(1,6) + F(1,5) = 1 + 1 = 2$$

$$F(2,5) = F(1,5) + F(1,4) = 1 + 1 = 2$$

اگر  $F(6,3)$  مطلوب بود ۲۰ جواب بود.

۶۰- گزینه (۳). برای محاسبه  $\binom{n}{m}$ ، تعداد  $\binom{n}{m}$  عدد یک با هم جمع می‌شوند پس عمل + به

تعداد  $\binom{n}{m} - 1$  نوشته می‌شود.

۶۱- گزینه (۳). با روش هورنر می‌توان با  $n$  عمل جمع و  $n$  عمل ضرب یک چندجمله‌ای درجه  $n$

را محاسبه کرد.

۶۲- گزینه (۱). ابتدا توابع  $P_1$  و  $P_2$  را با آرایه ستونی  $x_{n \times 1}$  که همه درایه‌های آن ۱ است

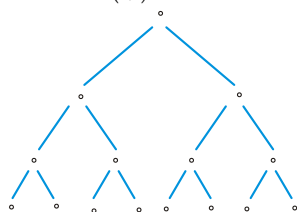
فراخوانی می‌کنیم و از حاصل ضرب  $A_1 x$  و  $A_2 x$  می‌فهمیم درایه مورد نظر در کدام سطر

قرار دارد. در مرحله بعد  $x$  را طوری تعریف می‌کنیم که  $\frac{n}{4}$  عنصر بالای آن ۱ و  $\frac{n}{4}$  عنصر

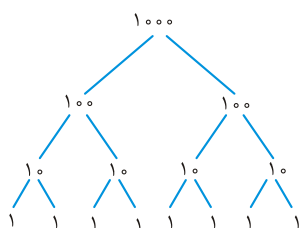
پایین آن صفر باشد، اگر  $y_1 = A_1 x$  و  $y_2 = A_2 x$  با هم برابر شدند پس درایه مورد نظر در

نیمه دوم سطر یافته شده است (ستون  $\frac{n}{4} + 1$  تا  $n$ ) و اگر  $y_1$  و  $y_2$  با هم برابر نبود یعنی درایه مورد نظر در نیم اول است. به هر حال ما مسئله را از  $n$  به  $\frac{n}{4}$  کاهش دادیم و برای تعداد فراخوانی می توان نوشت:

$$T(n) = T\left(\frac{n}{4}\right) + 1 = \theta(\lg n)$$



گزینه ۶۳- (۴). باید روش بهینه استفاده کنیم یعنی باید بیت‌ها را دو به دو با هم جمع کنیم. بهترین حالت وقتی است که همه  $n$  بیت، صفر باشند که در این صورت  $n - 1$  واحد هزینه داریم مثلاً به ازای  $n = 8$  درخت مقابل حاصل می‌شود که ۷ واحد هزینه دارد:



بدترین حالت وقتی است که همه  $n$  بیت، یک باشند. مثلاً به ازای  $n = 8$  بیت درخت مقابل حاصل می‌شود که هزینه  $11 = 4 \times 1 + 2 \times 2 + 1 \times 3$  دارد: و به طور کلی در این حالت، هزینه به ازای  $n$  بیت برابر است با:

$$\frac{n}{4} \times 1 + \frac{n}{4} \times 2 + \frac{n}{8} \times 3 + \dots + \frac{n}{4^k} \times k = \sum_{k=1}^{\lg n} \frac{n}{4^k} \times k = n \sum_{k=1}^{\lg n} \frac{k}{4^k} = n \cdot \theta(1) = \theta(n)$$

گزینه ۶۴- (۱). داریم  $f(2n) = 2f(n) - 1$  و  $f(2n+1) = 2f(n) + 1$

گزینه ۶۵- (۳).

$$T(n, k) = T\left(n_1, \frac{k}{4}\right) + T\left(n_2, \frac{k}{4}\right) + nk$$

در درخت بازگشت، نودها سطح به سطح باید جمع شود یعنی سطح اول و دوم، ... به ترتیب  $nk$ ،  $\frac{nk}{4}$ ،  $\frac{nk}{4}$  و ... است پس جمع کل  $nk + \frac{nk}{4} + \frac{nk}{4} + \dots = nk\left(1 + \frac{1}{4} + \frac{1}{4} + \dots\right)$  است که از مرتبه  $O(nk)$  است. دقت کنید ارتفاع درخت تأثیری در جواب ندارد چون تصاعد هندسی نزولی است.

$$n_1 \frac{k}{4} \quad \nwarrow \quad \nearrow \quad n_2 \frac{k}{4}$$

۶۶- گزینه (۲). رابطه گفته شده دارای ۴ فراخوانی است که زمان هر یک  $T(\frac{n}{4})$  است. دقت کنید رابطه را بهینه نکنید. در رابطه داده شده عملیات  $+$  و ضرب در  $2^n$  و در  $2^{\frac{n}{2}}$  از مرتبه  $\theta(n)$  هستند پس برای زمان اجرای  $T(n)$  می‌توان نوشت:

$$T(n) = 4T\left(\frac{n}{4}\right) + \theta(n) = \theta(n^2)$$

۶۷- گزینه (۲).

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{3}\right) + T\left(\frac{n}{4}\right) + n^2 \geq 3T\left(\frac{n}{4}\right) + n^2 = \theta(n^2)$$

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{3}\right) + T\left(\frac{n}{4}\right) + n^2 \leq 3T\left(\frac{n}{2}\right) + n^2 = \theta(n^2)$$

۶۸- گزینه (۳). درخت بازگشت تا سطحی ادامه می‌یابد که  $n > \sqrt{m}$  بیشتر است و وقتی  $n = \sqrt{m}$ ، درخت خاتمه می‌یابد. به راحتی می‌توان بررسی کرد که مرتبه  $\theta\left(\frac{n^3}{\sqrt{m}}\right)$  است.

با جایگذاری اثبات می‌کنیم:

$$\begin{aligned} T(n) &= 8T\left(\frac{n}{2}\right) + 1 = 8\left(8T\left(\frac{n}{4}\right) + 1\right) + 1 = 8^2T\left(\frac{n}{4}\right) + 8 + 1 \\ &= 8^3T\left(\frac{n}{8}\right) + 8^2 + 8 + 1 = \dots = 8^kT\left(\frac{n}{8^k}\right) + 8^{k-1} + 8^{k-2} + \dots + 8 + 1 \end{aligned}$$

۶۹- گزینه (۴). هر طور عمل ادغام را انجام دهیم، پس از ۹۹ بار ادغام، یک مجموعه ۱۰۰ عضوی حاصل می‌شود. فرض کنید ابتدا  $A_1 \leftarrow A_1 \cup A_2$ ، سپس  $A_1 \leftarrow A_1 \cup A_3$ ، سپس  $A_1 \leftarrow A_1 \cup A_4$  به همین ترتیب و در نهایت  $A_1 \leftarrow A_1 \cup A_{100}$ . پس از ۹۹ عمل ادغام مجموعه  $A_1 = \{1, 2, \dots, 100\}$  حاصل می‌شود. هر طور دیگری نیز عمل ادغام را انجام دهید، تعداد ۹۹ عمل لازم است. مثلاً فرض کنید  $A_1 \leftarrow A_1 \cup A_2$  و  $A_1 \leftarrow A_1 \cup A_3$  و ...  $A_{99} \leftarrow A_{99} \cup A_{100}$ ، تاکنون ۵۰ عمل ادغام شده است و ۵۰ مجموعه ۲ عضوی داریم. حال  $A_1 \leftarrow A_1 \cup A_3$  و  $A_5 \leftarrow A_5 \cup A_7$  و ...  $A_{97} \leftarrow A_{97} \cup A_{99}$  یعنی پس از انجام ۲۵ ادغام دیگر، اکنون ۲۵ مجموعه ۴ عضوی داریم. به همین ترتیب اگر عمل ادغام را ادامه دهید، ۹۹ عمل ادغام انجام می‌شود.

۷۰- گزینه (۲). واضح است که در هر فراخوانی  $b$  نصف می‌شود پس مرتبه این کد  $O(\lg b)$  است.

۷۱- گزینه (۳). طبق قضیه Master،  $n^{\log_2 2} = n^1 = \sqrt{n}$ ، رشد بیشتری از  $\lg n$  دارد پس  
 $T(n) \in O(\sqrt{n})$

۷۲- گزینه (۴).  $M(1 \circ 1) = 91 \rightarrow M(M(1 \circ 1)) = M(91) \rightarrow m(90) = m(91)$

$M(1 \circ 2) = 92 \rightarrow M(M(1 \circ 2)) = M(92) \rightarrow m(91) = m(92)$

$\vdots$

$M(1 \circ 0) = 100 \rightarrow M(M(1 \circ 0)) = M(100) \rightarrow m(99) = m(100)$

$M(111) = 101 \rightarrow M(M(111)) = M(101) \rightarrow m(100) = m(101)$

پس نتیجه می‌شود که  $M(90) = M(91) = \dots = m(101)$  و چون  $M(101) = 91$  بنابراین به ازای همه مقادیر  $n \leq 101$ ، مقدار  $m(n)$  برابر ۹۱ است.

۷۳- گزینه (۴). درخت بازگشت این رابطه در فصل ۳ کشیده شده است.

۷۴- گزینه (۳). می‌دانیم

$$T(n) = aT\left(\frac{n}{b}\right) + n^{\log_b a} \cdot Lg^k n \stackrel{k \geq 0}{=} \theta(n^{\log_b a} \cdot Lg^{k+1} n)$$

پس

$$T(n) = T\left(\frac{n}{4}\right) + \log^2 n = \theta(\log^3 n)$$

۷۵- گزینه (۰). کلید این تست گزینه ۲ است ولی همه گزینه‌ها نادرست هستند:

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n = \theta(n \cdot \lg n) \quad (\text{نقض گزینه ۱})$$

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n^2 = \theta(n^2) \quad (\text{نقض گزینه ۲})$$

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{4n}{5}\right) + n = \theta(n) \quad (\text{نقض گزینه ۳})$$

$$T(n) = 2T\left(\frac{2n}{3}\right) + n = \theta(n^{\log_{3/2} 2}) \quad (\text{نقض گزینه ۴})$$

۷۶- گزینه (۳). به ازای  $n=1$  مقدار تابع  $F_1(0) + 2^1 + 4^1 = 1 + 2 + 4 = 7$  می‌باشد که فقط گزینه ۳ می‌تواند صحیح باشد. البته با معادله مشخصه یا جایگذاری از بالا نیز می‌توان به راحتی رابطه را حل کرد.

## فصل ۴

## جستجو و درهم‌سازی

## جستجوی دودویی (binary search)

یادمان هست برای جستجوی خطی (linear search) عنصر با کلید  $x$  در آرایه  $A[L..U]$ ،  $x$  را با عنصر اول آرایه سپس با دومی و به همین ترتیب مقایسه کردیم که این جستجو برای آرایه  $n$  عنصری از مرتبه  $O(n)$  است. اگر آرایه  $A[L..U]$  مرتب باشد، می‌توان با سرعت بیشتری،  $x$  را جستجو کرد. ما فرض می‌کنیم آرایه  $A$  به صورت صعودی مرتب است. جستجوی دودویی، ابتدا  $x$  را با عنصر وسط آرایه مقایسه می‌کند، اگر  $x$  با عنصر وسط آرایه مساوی باشد که کار تمام است. در غیر این صورت نیمه چپ یا راست به صورت بازگشتی، جستجو می‌شوند:

$$A[L..U]: A \begin{array}{|c|c|c|} \hline L & \text{mid} & U \\ \hline \end{array}$$

جواب پیدا شده است و تمام

$$\text{mid} = \left\lfloor \frac{L+U}{2} \right\rfloor : \begin{cases} x == A[\text{mid}] \rightarrow \\ x < A[\text{mid}] \rightarrow \\ x > A[\text{mid}] \rightarrow \end{cases}$$

$A[L..\text{mid}-1]$  را جستجوی دودویی کن.  
 $A[\text{mid}+1..U]$  را جستجوی دودویی کن

توجه کنید اگر تعداد عناصر آرایه زوج باشد، دو عنصر وسط هستند که ما عنصر سمت چپ را

$\text{mid}$  گرفتیم. مثلاً در آرایه  $A \begin{array}{|c|c|c|c|} \hline 1 & 2 & 3 & 4 \\ \hline \end{array}$  اندیس عنصر وسط  $\text{mid} = \left\lfloor \frac{1+4}{2} \right\rfloor = 2$  است.

البته می‌توان برای محاسبه  $\text{mid}$  از فرمول  $\left\lceil \frac{L+U}{2} \right\rceil$  استفاده کرد و برای  $A[3]$ ،  $A[1..4]$  را عنصر وسط در نظر گرفت. الگوریتم جستجوی دودویی را می‌توان به صورت بازگشتی و غیر بازگشتی نوشت که در ادامه آمده است.



```

recbinarysearch(L, U, x)
{
  if(L > U) return NIL
  mid =  $\lfloor \frac{L+U}{2} \rfloor$ 
  if(x == A[mid]) return mid
  if(x > A[mid])
    return recbinarysearch (mid + 1, U, x)
  else
    return recbinarysearch (L, mid - 1, x)
}

```

(a)

```

iterative binarysearch (L, U, x)
{
  while(L ≤ U)
  {
    mid =  $\lfloor \frac{L+U}{2} \rfloor$ 
    if(x == A[mid]) return mid
    if(x > A[mid]) L = mid + 1
    else U = mid - 1
  }
  return NIL
}

```

(b)

الگوریتم ۱- جستجوی دودویی x در  $A[L..U]$  به صورت (a) بازگشتی (b) غیر بازگشتی. در صورت وجود x، اندیس آن در غیر این صورت NIL برگردانده می‌شود.

با توجه به الگوریتم ۱ واضح است که مرتبه جستجوی دودویی  $O(\lg n)$  است؛ زیرا اگر  $T(n)$  را حداکثر زمان اجرا فرض کنیم آنگاه با توجه به کد (a) می‌توان نوشت  $T(n) = T\left(\frac{n}{2}\right) + \theta(1)$  که از مرتبه  $\theta(\lg n)$  است. پس زمان اجرای جستجوی دودویی  $O(\lg n)$  است. در الگوریتم ۱ در هر دور اجرا دو مقایسه انجام می‌شود. یک مقایسه برای تساوی x با  $A[mid]$  و در صورت مساوی نبودن، یک مقایسه برای  $x > A[mid]$  انجام می‌شود. می‌توان الگوریتم را طوری نوشت که در هر دور فقط یک مقایسه انجام شود. الگوریتم ۲ نشان می‌دهد چگونه فقط با یک مقایسه در هر دور، x را بیابیم. البته عیب الگوریتم ۲ این است که یافتن x را به تعویق می‌اندازد ولی مزیت آن این است که در هر دور فقط یک مقایسه انجام می‌دهد:

```

binarysearch (L, U, x)
{
  if(L == U)
    if(A[L] == x) return L else return NIL
  else{
    mid =  $\lfloor \frac{L+U}{2} \rfloor$ 
    if(x < A[mid]) return binarysearch (L, mid - 1, x)
    else return binarysearch (mid, U, x)
  }
}

```

الگوریتم ۲: جستجوی دودویی x در  $A[L..U]$  که در هر دور فقط یک مقایسه صورت می‌گیرد.

اگر  $T(n)$  تعداد مقایسه کلیدها برای یافتن  $x$  در آرایه  $n$  عنصری باشد می‌توان نوشت  
 $T(n) = T\left(\frac{n}{2}\right) + 1$  و  $T(1) = 1$  که با حل این رابطه  $T(n) = \lfloor \lg n \rfloor + 1$  می‌باشد.

**مثال ۱:** در یک آرایه ۹ عنصری مرتب صعودی، با روش جستجوی دودویی، در جستجوی موفق برای یافتن هر عنصر در آرایه، چه تعداد عنصر آرایه بررسی می‌شوند؟

**پاسخ:** فرض کنید

|    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|
| ۱  | ۲  | ۳  | ۴  | ۵  | ۶  | ۷  | ۸  | ۹  |
| ۱۰ | ۲۰ | ۳۰ | ۴۰ | ۵۰ | ۶۰ | ۷۰ | ۸۰ | ۹۰ |

آرایه ۹ عنصری صعودی  $A$  است. جستجوی موفق یعنی عناصر موجود در  $A$  را جستجو کنیم. برای یافتن  $x = 50$  چون  $50$  در خانه  $5 = \left\lfloor \frac{1+9}{2} \right\rfloor$  است فقط یک عنصر (خود  $50$ ) بررسی می‌شود. برای یافتن  $x = 20$  ابتدا با  $50$  و چون  $20 < 50$  سپس  $x$  با  $A\left[\left\lfloor \frac{1+4}{2} \right\rfloor\right] = A[2]$  یعنی با  $20$  مقایسه می‌شود یعنی  $2$  عنصر بررسی می‌شوند. به همین ترتیب برای  $10$  و  $30$  هر کدام  $3$  تا، برای  $40$ ،  $4$  تا، برای  $70$ ،  $2$  تا، برای  $60$  و  $80$  هر کدام  $3$  تا و برای  $90$ ،  $4$  عدد بررسی می‌شوند.

### جستجوی دودویی در یک دنباله چرخشی

گویند دنباله  $\langle x_1, x_2, \dots, x_n \rangle$  به صورت چرخشی مرتب است اگر کوچکترین عدد در دنباله،  $x_i$  باشد و  $i$  مشخص نیست که دنباله  $\langle x_i, x_{i+1}, \dots, x_n, x_1, \dots, x_{i-1} \rangle$  صعودی مرتب باشد. مثلاً دنباله  $\langle x_1, x_2, x_3, x_4, x_5, x_6, x_7 \rangle = \langle 30, 40, 50, 10, 15, 20, 22 \rangle$  چرخشی است چون دنباله  $\langle x_4, x_5, x_6, x_7, x_1, x_2, x_3 \rangle = \langle 10, 15, 20, 22, 30, 40, 50 \rangle$  مرتب صعودی است.

در واقع اگر یک دنباله چرخشی را چندین بار به چپ چرخش دهید، مرتب صعودی می‌شود. ما می‌خواهیم در یک دنباله چرخشی، عنصر مینیمم (در دنباله فوق  $x_4 = 10$ ) را بیابیم. فرض کنید  $k$  و  $m$  دو اندیس دلخواه باشند و  $k < m$ . اگر  $x_k < x_m$  قطعاً اندیس عنصر مینیمم یعنی  $i$  نمی‌تواند در بازه  $k < i \leq m$  باشد (دقت کنید خود  $k$  ممکن است جواب باشد) و اگر  $x_k > x_m$ ، آنگاه  $i$  حتماً در بازه  $k < i \leq m$  می‌باشد. با همین ایده و شبیه جستجوی دودویی با مرتبه  $O(\lg n)$  می‌توان عنصر مینیمم را یافت. برای یافتن مینیمم در  $A[L..U]$  کافیست عنصر وسط  $A[mid]$  را با عنصر آخر  $A[U]$  مقایسه کنید. اگر  $A[mid] < A[U]$  آنگاه جستجو را با همین روش در  $A[L..mid]$  انجام دهید و در غیر این صورت جستجو را در  $A[mid+1..U]$  انجام دهید. یعنی با هر مقایسه، بازه جستجو نصف می‌شود.

### جستجوی دودویی برای یک اندیس خاص

فرض کنید آرایه  $A[L..U]$  حاوی  $n$  عدد صحیح متمایز است. می‌خواهیم اندیس  $i$  را در صورت وجود بیابیم که  $A[i] = i$ . کفایت عنصر وسط یعنی  $A[mid]$  را با خود اندیس  $mid$  مقایسه کنیم اگر  $A[mid] == mid$  که کار تمام است. اگر  $A[mid] < mid$ ، با توجه به اینکه اعداد متمایز هستند، حتماً  $A[mid-1]$  از  $mid-1$  کوچکتر است و به همین ترتیب سایر عناصر سمت چپ  $mid$  نیز چنین هستند بنابراین باید عمل جستجو را در  $A[mid+1..U]$  انجام دهیم و در غیر این صورت عمل جستجو را در  $A[L..mid-1]$  (یا  $A[L..mid]$ ) انجام دهیم. با توجه به اینکه با هر مقایسه بازه جستجو نصف می‌شود، مرتبه این الگوریتم  $O(\lg n)$  است.

### جستجوی دودویی در دنباله با طول نامشخص

فرض کنید می‌خواهیم  $x$  را در آرایه مرتب صعودی  $A[1..?]$  که طول آن نامشخص است جستجو کنیم. ما نمی‌توانیم عنصر وسط آرایه را بیابیم چون انتهای آرایه نامشخص است. ولی می‌توانیم یک مدل خاص از جستجوی دودویی را بکار ببریم، به این صورت که  $x$  را با  $A[1]$  و  $A[2]$  و  $A[4]$  و ... و  $A[i]$  و  $A[2i]$  مقایسه کنیم و اگر  $A[i] < x \leq A[2i]$  حال بازه  $A[i..2i]$  را جستجوی دودویی کنیم که مرتبه  $\Theta(\lg i)$  می‌شود. می‌توان جستجوی دودویی کلاسیک را نیز با این ایده انجام داد یعنی بجای آنکه نصف کنیم هر بار از ۱ شروع کنیم و دو برابر کنیم در این صورت مرتبه  $O(\lg i)$  می‌شود و نه  $O(\lg n)$  و اگر  $i$  خیلی کمتر از  $n$  باشد، مزیت است. ولی در این روش دو بار جستجوی دودویی صورت می‌گیرد، یکبار برای آنکه بازه  $i$  تا  $2i$  را بیابیم و یکبار هم بازه  $A[i..2i]$  را جستجوی دودویی می‌کنیم بنابراین این الگوریتم فقط در صورتی از جستجوی دودویی کلاسیک بهتر است که  $i = O(\sqrt{n})$ .

### جستجوی درونیابی (interpolation search)

در جستجوی دودویی برای یافتن  $x$  هر بار بازه جستجو را نصف می‌کنیم. در جستجوی درونیابی ما همان ابتدا سعی می‌کنیم به  $x$  نزدیک شویم. مثلاً فرض کنید یک کتاب ۱۰۰۰ صفحه‌ای در اختیار دارید و می‌خواهید به صفحه ۲۰۰ بروید، ابتدا سعی می‌کنید صفحه‌ای که حدوداً در  $\frac{1}{5}$  اول کتاب باشد باز کنید چون ۲۰۰ به ۱ نزدیکتر است نسبت به ۱۰۰۰. اگر صفحه مثلاً ۲۵۰ را باز کنید، حال بین صفحه ۱ تا ۲۵۰ را باید جستجو کنید ولی می‌دانید که برای رسیدن به صفحه ۲۰۰ باید  $\frac{1}{5}$  از ۲۵۰ به عقب برگردید. ایده جستجوی درونیابی نیز همین است و اگر توزیع اعداد در آرایه، یکنواخت باشد، روش خوبی برای جستجو است. الگوریتم زیر جستجوی درونیابی  $x$  را در  $A[L..U]$  نشان می‌دهد:

```

Int search (L, U, x)
{
  if (A[L] == x) Intsearch = L
  else if ((L == U) or (A[L] == A[U])) return NIL;
  else {Next_Guess =  $\left\lceil L + \frac{(x - A[L])(U - L)}{A[U] - A[L]} \right\rceil$ 
  if (x < A[Next_Guess]) return Intsearch (L, Next_Guess - 1, x)
  else return Intsearch (Next_Guess, U, x)}
}

```

زمان اجرای جستجوی درونیابی برای آرایه  $n$  عنصری در بدترین حالت  $\Theta(n)$  است چون ممکن است  $x$  با همه عناصر آرایه مقایسه شود. ولی در حالت متوسط می‌توان نشان داد مرتبه جستجوی درونیابی  $\Theta(\lg \lg n)$  است که اثبات این مطلب کمی دشوار است و با میانگین گرفتن روی همه حالات ممکن برای آرایه، بدست می‌آید.

### درهم‌سازی

همانطور که دیدیم جستجوی خطی با مرتبه  $O(n)$  و جستجوی دودویی با مرتبه  $O(\lg n)$  در یک آرایه  $n$  عنصری جستجو می‌کند. هدف درهم‌سازی این است که با مرتبه  $O(1)$  جستجو انجام دهد. در درهم‌سازی مستقیم (direct hash) عنصر با کلید  $k$  در خانه  $T[k]$  از آرایه  $T[0..m-1]$  ذخیره می‌شود. یعنی هر عنصر در خانه‌ای ذخیره می‌شود که کلید آن نشان می‌دهد. مشکل درهم‌سازی مستقیم این است که حافظه هدر رفته زیادی دارد. مثلاً ممکن است فقط ۴ عنصر با کلیدهای ۱۰ و ۲۰ و ۳۰۰ و ۱۰۰۰ داشته باشیم در این صورت باید آرایه  $T[0..1000]$  را تعریف کنیم که بسیاری از خانه‌های این آرایه بلااستفاده‌اند. برای رفع این مشکل از تابع درهم‌ساز (hash function) استفاده می‌کنیم. ما فرض می‌کنیم  $n$  تا عنصر داریم و می‌خواهیم در آرایه  $T[0..m-1]$  ذخیره کنیم. به آرایه  $T$ ، جدول درهم‌ساز (hash table) گویند. و به خانه‌های این جدول، حفره (Slot) گویند. پس  $n$  تا عنصر را می‌خواهیم در  $m$  حفره ذخیره کنیم. کلید این عناصر را به تابع درهم‌ساز  $h$  می‌دهیم و این تابع یک آدرس تولید می‌کند و عنصر را در آن آدرس ذخیره می‌کنیم. البته ممکن است برخورد (Collision) پیش آید، یعنی اینکه دو کلید متفاوت به تابع داده‌ایم و آدرس یکسان تولید شده است. یعنی دو کلید  $k_1$  و  $k_2$  متفاوتند ولی  $h(k_1)$  با  $h(k_2)$  مساوی است:  $k_1 \neq k_2 \rightarrow h(k_1) = h(k_2)$ .

در صورت بروز برخورد یا تصادف باید حل تصادف کنیم. دو روش کلی برای حل تصادف مطرح است: زنجیره‌سازی (Chaining) و آدرس‌دهی باز (open addressing).  
در روش **زنجیره‌سازی** هر حفره یک اشاره‌گر به ابتدای یک لیست پیوندی است. اگر کلید  $k$  را به تابع  $h$  بدهیم و آدرس  $i$  تولید شود ( $h(k) = i$ ) آنگاه عنصر با کلید  $k$  را به ابتدای لیست  $i$  درج می‌کنیم.

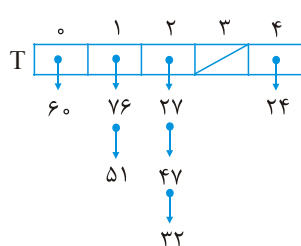
**مثال ۲:** تعداد  $n = 7$  عنصر با کلیدهای داده شده (از چپ به راست) را در جدول  $T[0..4]$  با تابع درهم‌ساز  $h(k) = k \bmod 5$  و با روش زنجیره‌سازی درج کنید.

۷۶ و ۲۷ و ۶۰ و ۵۱ و ۴۷ و ۳۲ و ۲۴: کلیدها

**پاسخ:** آدرس تولیدی برای این کلیدها طبق جدول زیر است:

| کلید        | ۲۴ | ۳۲ | ۴۷ | ۵۱ | ۶۰ | ۲۷ | ۷۶ |
|-------------|----|----|----|----|----|----|----|
| $h(k)$ آدرس | ۴  | ۲  | ۲  | ۱  | ۰  | ۲  | ۱  |

پس ابتدا ۲۴ به لیست  $T[4]$  وارد می‌شود. سپس ۳۲ به لیست  $T[2]$  وارد می‌شود سپس ۴۷ نیز به لیست  $T[2]$  وارد می‌شود و ابتدای لیست درج می‌شود (عنصر تازه وارد ابتدای لیست درج می‌شود) و به همین ترتیب، شکل ذخیره عناصر به صورت روبرو است:



در روش **آدرس‌دهی باز**، عناصر داخل خود حفره‌ها ذخیره می‌شوند. و در صورت بروز تصادف، برای عنصر تصادفی یک آدرس جدید محاسبه می‌شود که برای محاسبه آدرس جدید روش‌های مختلفی وجود دارد از جمله روش کاوش خطی (Linear probing) که از محل تصادف به سمت انتهای آرایه  $T$ ، حفره‌ها را بررسی می‌کند و اگر حفره خالی پیدا نشد، کاوش از ابتدای آرایه، ادامه می‌یابد. اگر همه  $m$  حفره بررسی شدند و حفره خالی پیدا نشد به معنی سرریز است. در روش آدرس‌دهی باز حتماً  $n \leq m$  ولی در روش زنجیره‌سازی  $n$  می‌تواند از  $m$  بزرگتر هم باشد.

**مثال ۳:** تعداد  $n = 5$  عنصر با کلیدهای داده شده (از چپ به راست) را در جدول  $T[0..4]$  با تابع درهم‌ساز  $h(k) = k \bmod 5$  و با روش کاوش خطی ذخیره کنید:

۴۱ و ۲۶ و ۳۷ و ۴۲ و ۲۳: کلیدها

**پاسخ:** آدرس تولیدی برای این کلیدها در جدول زیر آمده است:

| کلید        | ۲۳ | ۴۲ | ۳۷ | ۲۶ | ۴۱ |
|-------------|----|----|----|----|----|
| $h(k)$ آدرس | ۳  | ۲  | ۲  | ۱  | ۱  |

ابتدا ۲۳ به حفره ۳ وارد می‌شود سپس ۴۲ به حفره ۲ وارد می‌شود. سپس ۳۷ با ۴۲ تصادف می‌کند و چون حفره ۳ پر است به حفره ۴ وارد می‌شود. سپس ۲۶ به حفره ۱ وارد می‌شود و در نهایت ۴۱ با ۲۶ تصادف می‌کند و به حفره صفر وارد می‌شود:

|   |    |    |    |    |    |
|---|----|----|----|----|----|
|   | ۰  | ۱  | ۲  | ۳  | ۴  |
| T | ۴۱ | ۲۶ | ۴۲ | ۲۳ | ۳۷ |

## نکته

احتمال تصادف ۲ عنصر با هم  $\frac{1}{m}$  است. زیرا یکی از این عناصر به حفره دلخواهی وارد می‌شود و عنصر دوم با احتمال  $\frac{1}{m}$  (تعداد حفره‌ها  $m$  تاست) به همان حفره‌ای وارد می‌شود که عنصر اول وارد شده است.

## نکته

در روش زنجیره‌سازی درج یک عنصر از مرتبه  $\theta(1)$  است زیرا عنصر جدید به ابتدای لیست درج می‌شود. جستجوی یک عنصر در بدترین حالت از مرتبه  $\theta(n)$  است چون ممکن است همه  $n$  عنصر به یک حفره وارد شده باشند و مجبور باشیم همه  $n$  عنصر را بررسی کنیم. جستجو در حالت متوسط از مرتبه  $\theta(1 + \alpha)$  است که ۱ زمان اعمال تابع درهم‌ساز است و  $\alpha = \frac{n}{m}$  به فاکتور لود معروف است و برابر میانگین تعداد عناصر موجود در هر حفره است.

## خلاصه فصل

- ۱- جستجوی خطی عنصر  $x$  در  $A[1..n]$  از ابتدای آرایه جستجو می‌کند و از مرتبه  $O(n)$  است.
- ۲- جستجوی دودویی عنصر  $x$  در آرایه مرتب  $A[1..n]$  از مرتبه  $O(\lg n)$  است.
- ۳- جستجو در آرایه چرخشی از مرتبه  $O(\lg n)$  است.
- ۴- جستجوی درون‌یابی  $x$  در آرایه مرتب  $A[1..n]$  در حالت متوسط از مرتبه  $\theta(\lg \lg n)$  است ولی در بدترین حالت از مرتبه  $\theta(n)$  است.
- ۵- هدف درهم‌سازی این است که بتواند حذف و درج و جستجو را با مرتبه  $\theta(1)$  انجام دهد.
- ۶- درهم‌سازی به این صورت است که هر عنصر دارای کلید است. کلید  $k$  به تابع درهم‌ساز  $h$  داده می‌شود و این تابع آدرس  $h(k)$  را تولید می‌کند که محل ذخیره عنصر را در جدول  $T[0..m-1]$  که شامل  $m$  حفره است، نشان می‌دهد. در صورت بروز تصادف (تصادف یعنی برای دو کلید متفاوت  $k_1 \neq k_2$ ، آدرس یکسان  $h(k_1) = h(k_2)$  تولید شود) باید با یکی از روش‌های زنجیره‌سازی یا آدرس‌دهی باز، حل تصادف شود. در روش زنجیره‌سازی هر حفره مثل  $T[i]$  اشاره‌گری است به یک لیست پیوندی و اگر برای عنصر با کلید  $k$  آدرس  $h(k) = i$  تولید شود، این عنصر به لیست  $T[i]$  وارد می‌شود. در روش آدرس‌دهی باز عناصر در خود آرایه  $T$  ذخیره می‌شوند و اگر حفره  $i$  پر باشد یک حفره خالی برای عنصر جستجو می‌شود، که برای یافتن حفره خالی روش‌های مختلفی هست از جمله کاوش خطی که از محل تصادف به سمت انتهای آرایه، حفره‌ها را به ترتیب واری می‌کند و اگر حفره خالی پیدا نشد، کاوش را از ابتدای آرایه تا محل تصادف ادامه می‌دهد.
- ۷- در یک جدول درهم‌ساز با زنجیره‌سازی، با فرض اینکه تابع درهم‌ساز یکنوای ساده باشد یعنی هر آدرس در جدول را با احتمال مساوی تولید کند، آنگاه زمان متوسط جستجوی موفق و ناموفق  $\theta(1 + \alpha)$  است که  $\alpha = \frac{n}{m}$  به فاکتور لود معروف است.  $n$  تعداد عناصر و  $m$  تعداد حفره‌های جدول است.
- ۸- در یک جدول درهم‌ساز با آدرس‌دهی باز، متوسط تعداد کاوش در جستجوی ناموفق حداکثر  $\frac{1}{1-\alpha}$ ، و در جستجوی موفق حداکثر  $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$  است.

## [سؤال‌های چهار گزینه‌ای فصل چهارم]

۱- فرض کنید آرایه مورد جستجو توسط جستجوی دودویی بصورت  $(-۶, -۵, ۰, ۷, ۹, ۲۳, ۵۴, ۸۲, ۱۰۱)$  باشد. متوسط تعداد مقایسه‌های مورد نیاز برای حالت

جستجوی موفق چیست؟ (میانی نرم افزار آزاد ۷۹)

$$(۱) \frac{28}{9} \quad (۲) \frac{18}{9} \quad (۳) \frac{25}{9} \quad (۴) \frac{26}{9}$$

۲- اگر الگوریتم جستجوی دودویی را برای جستجوی عناصر آرایه  $A[1..8] = [۵, ۱۰, ۱۵, ۲۰, ۲۵, ۳۰, ۳۵, ۴۰]$  به کار ببریم، میانگین تعداد مقایسه‌ها برای

جستجوی موفق تقریباً، کدام است؟ (تخصصی نرم افزار دولتی ۸۲)

$$(۱) \frac{2}{2} \quad (۲) \frac{2}{4} \quad (۳) \frac{2}{6} \quad (۴) \frac{2}{8}$$

۳- در تست ۱، متوسط تعداد مقایسه‌های جستجوی ناموفق کدام است؟ جستجوی ناموفق یعنی عناصری که در آرایه نیستند را جستجو کنید.

$$(۱) \frac{28}{9} \quad (۲) \frac{34}{10} \quad (۳) \frac{34}{9} \quad (۴) \frac{28}{10}$$

۴- اگر  $S(n)$  متوسط تعداد مقایسه‌ها برای جستجوی موفق در یک آرایه مرتب با طول  $n$  و  $U(n)$  متوسط تعداد مقایسه‌ها برای جستجوی ناموفق در این آرایه با استفاده از روش

جستجوی دو دویی باشد. کدام یک از گزینه‌های زیر نادرست است؟ (IT ۸۳ و ۸۷)

$$(۱) S(n) = \theta(U(n)) \quad (۲) S(n) = U(n) - 1$$

$$(۳) S(n) = \left(1 + \frac{1}{n}\right)U(n) - 1 \quad (۴) \text{ موارد ۱ و ۳ صحیح است.}$$

۵- آرایه مرتب  $A$  داده شده است. تابع زیر برای جستجوی دودویی در  $A$  (از اندیس  $L$  تا  $R$ ) برای پیدا کردن  $x$  نوشته شده است.

```
function search (x : Real; L , R : integer) : boolean;
var
    M : integer;
begin
    M := (L + R) div 2;
    if (x = A[M]) then return (true);
    if (x < A[M] and (M > L)) then return (Search (x, L , M-1));
    if (x > A[M] and (M < R)) then return (Search (x, M+1 , R));
    return (False);
end;
```



کدام یک از گزینه‌های زیر در مورد الگوریتم فوق برای جستجوی دودویی در  $A[1..N]$  غلط است؟ (منظور از مقایسه، مقایسه  $x$  با یک عنصر از  $A$  است) (مهندسی کامپیوتر – دولتی ۷۷)

(۱) اگر  $x$  در  $A[1..N]$  باشد، همواره با حداکثر  $O(\log_2^N)$  پیدا می‌شود.

(۲) اگر  $x$  در  $A[1..N]$  نباشد، پاسخ False همواره با حداکثر  $O(\log_2^N)$  مقایسه بدست می‌آید.

(۳) در جستجوی ناموفق برای دو مقدار متفاوت  $x$  همواره تعداد مقایسه‌ها برابر است.

(۴) حداکثر تعداد مقایسه‌ها (چه موفق و چه ناموفق) همواره از  $O(N)$  کمتر است.

۶- رابطه بازگشتی برای  $T(n)$  تعداد مقایسه‌های لازم برای اجرای جستجوی دو تایی (binary search) در یک لیست مرتب شده با  $n$  عضو چیست؟

(ساختمان گسسته – دولتی ۸۳) و (تخصصی نرم افزار مهندسی کامپیوتر آزاد ۸۶)

$$T(n) = T\left(\frac{n}{2}\right) + 1 \quad (۲) \quad T(n) = T(n-1) + 1 \quad (۱)$$

$$T(n) = 2T(n-1) + 1 \quad (۴) \quad T(n) = 2T\left(\frac{n}{2}\right) + 1 \quad (۳)$$

۷- دستور حذف شده برنامه جستجوی دودویی زیر کدام است؟ (علوم کامپیوتر ۸۰)

procedure Binsearch (a : elementarray; x : element ; var left, right, j: integer)

var

middle : integer;

begin

if (Left <= right) then begin

middle := (Left + right) div 2;

case compare (x, a [middle]) of

'>' : Binsearch (a , x, middle + 1, right, j);

'<': -----

'=' : j := middle

end

end;

j := -1

end;

Binsearch(a, x, middle - 1, right, j); (۱)

Binsearch(a, x, middle - 1, Left, j); (۲)

Binsearch(a, x, left, middle - 1, j); (۳)

Binsearch(a, x, Left, middle, right); (۴)

۸- آرایه‌ای با تعداد عناصر  $n$  مفروض است. می‌خواهیم عنصر  $x$  را در آرایه جستجو نماییم. اگر

عنصر  $x$  با احتمال  $\frac{1}{p}$  در آرایه موجود باشد، در این صورت به طور میانگین چه تعداد از

عناصر آرایه بایستی مورد جستجو قرار گیرد؟ (مهندسی فناوری اطلاعات - آزاد ۸۶)

$$(1) \frac{1}{p} \quad (2) \frac{3}{4} \quad (3) \frac{2}{3} \quad (4) \frac{3}{5}$$

۹- یک آرایه  $n$  تایی  $A$  از اعداد صحیح، با شرط‌های زیر داده شده است:

۱-  $A(1) = x$  ،  $A(n) = y$  ،  $x < y$  ، ۲- برای هر  $i$  ،  $|A(i) - A(i+1)| \leq 1$  یک الگوریتم

کارا برای جستجوی  $w$  ( $x \leq w \leq y$ ) در آرایه  $A$  طراحی شده است. این الگوریتم اندیس

$w$  را در صورت وجود به دست می‌آورد. پیچیدگی این الگوریتم در بدترین حالت و حالت

متوسط چیست؟ (مهندسی کامپیوتر - دولتی ۷۹)

(۱) بدترین حالت  $O(n)$ ، حالت متوسط  $O(\log n)$

(۲) بدترین حالت  $O(\log n)$  و حالت متوسط  $O(n)$

(۳) هر دو حالت  $O(n)$

(۴) هر دو حالت  $O(\log n)$

۱۰-  $n$  نفر در یک کلاس ساختمان داده حضور دارند، می‌دانیم که تنها یکی از آنها در درس

ساختمان ۲۰ گرفته است، می‌خواهیم این نابغه را پیدا کنیم بدین منظور تنها می‌توانیم این

گونه عمل کنیم: «هر بار به دلخواه خود می‌بایست  $k$  (عدد دلخواهی از ۱ تا  $n$  است) نفر از

میان دانشجویان انتخاب نموده، در یک گروه قرار داده و از آن گروه سوال کنیم که آیا نابغه

در میان شما هست یا نه؟» و گروه در پاسخ تنها یک جواب بله و یا خیر می‌دهد. در بدترین

حالت ممکن به چند پرسش نیاز داریم تا بتوانیم حتماً نابغه را پیدا کنیم؟ (یعنی در واقع

پیچیدگی بهترین راه حل برحسب  $n$  از چه درجه‌ای هست؟) (علوم کامپیوتر ۸۹)

$$(1) \sqrt{n} \quad (2) n \quad (3) \log n \quad (4) n \log n$$

۱۱- آرایه مرتب  $A$  با  $n$  عدد صحیح مثبت و منفی ولی مجزا از هم داده شده است. می‌خواهیم

در صورت وجود اندیس  $i$  را به دست آوریم که  $A[i] = i$  باشد. و در صورت نبود جواب آن

را اعلام کنیم. (تخصصی هوش مصنوعی ۸۷)

(۱) می‌توان نشان داد که هر راه حل این مسئله از  $\Omega(n)$  است.

(۲) برای این مسئله الگوریتمی از مرتبه‌ی  $O(\sqrt{n})$  وجود دارد.

(۳) برای این مسئله الگوریتمی از مرتبه‌ی  $O(\lg n)$  وجود دارد.

(۴) با استفاده از درخت تصمیم می‌توان نشان داد که هر درجه‌ی پیچیدگی هر راه‌حل این

مسئله اکیداً بیش‌تر از  $\lg n$  است.

۱۲- آرایه A به طول n به صورت حلقوی مرتب است، یعنی اگر ابتدا و انتهای A را به هم بچسبانیم، از هر درایه‌ای به بعد (به صورت حلقوی) آرایه مرتب است. مثلاً آرایه

به صورت حلقوی مرتب است. بهترین

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| ۶۰ | ۷۰ | ۸۰ | ۱۰ | ۲۰ | ۳۰ | ۴۰ | ۵۰ |
|----|----|----|----|----|----|----|----|

الگوریتم برای یافتن کوچک‌ترین عنصر A از چه مرتبه‌ای است؟ (هوش مصنوعی ۹۱)

(۱)  $O(\lg \lg n)$  (۲)  $O(\lg n)$  (۳)  $O(\lg^2 n)$  (۴)  $O(n)$

۱۳- بهترین الگوریتم برای یافتن محل کوچک‌ترین عنصر در یک دنباله مرتب چرخشی صعودی n عنصری دارای چه هزینه زمانی است؟ (دنباله  $x_1, x_2, \dots, x_n$  با کوچک‌ترین عنصر  $x_i$  را دنباله مرتب چرخشی گویند اگر دنباله  $x_i, x_{i+1}, \dots, x_n, x_1, x_2, \dots, x_{i-1}$  یک دنباله مرتب صعودی باشد.) (علوم کامپیوتر ۹۱)

(۱)  $O(\sqrt{n})$  (۲)  $O(\log n)$  (۳)  $O(n)$  (۴)  $O(n \log n)$

۱۴- فرض کنید A و B دو رشته (دنباله‌ای از حروف) بوده،  $|A| = n$  و  $|B| = m$  باشد. بهترین راه حل برای یافتن بزرگترین مقدار i به طوری که  $B^i$  زیر دنباله A باشد، دارای چه مرتبه زمانی است؟

(منظور از  $B^i$  تکرار تک تک حروف آن به اندازه i بار است و منظور از «B زیر دنباله A است» این است که تک تک حروف B همان ترتیب (نه لزوماً پشت سرهم) را در A داشته باشند.) (علوم کامپیوتر ۹۱)

(۱)  $O\left(n \log\left(\frac{n}{m}\right)\right)$  (۲)  $O\left(m \log\left(\frac{n}{m}\right)\right)$   
(۳)  $O(n \log m)$  (۴)  $O(m \log n)$

۱۵- ۳۱ جعبه در کنار هم هستند. هر جعبه‌ای حاوی عددی است که تنها با باز کردن آن جعبه مشخص می‌شود. می‌خواهیم جعبه‌ای را بیابیم که عدد داخل آن از هر دو عدد جعبه‌های مجاورش (یا تنها جعبه‌ی مجاور، اگر جعبه‌ی مورد نظر در انتها یا ابتدا قرار گرفته باشد.) ناکوچکتر باشد. در بدترین حالت دست کم چند جعبه باید باز شود تا چنین جعبه‌ای یافت شود؟ (نرم افزار ۹۲)

(۱) ۳۰ (۲) ۳۱ (۳) ۱۱ (۴) ۱۲

۱۶- در جدول در همسازی (hashing) با واریسی خطی (linear probing) اگر تابع درهم‌سازی برای هفت عنصر ورودی به صورت زیر باشد، (مهندسی کامپیوتر سراسری ۸۲)

|      |   |   |   |   |   |   |   |
|------|---|---|---|---|---|---|---|
| key  | A | B | C | D | E | F | G |
| hash | ۳ | ۵ | ۳ | ۴ | ۵ | ۶ | ۳ |

کدام یک از گزینه‌های زیر نمی‌تواند حاصل درج این عناصر با هر ترتیب دلخواه در آرایه ۷ تایی  $H[0..6]$  (که در ابتدا تهی است) باشد؟

- (۱)  $H[0..6] = [E \ F \ G \ A \ C \ B \ D]$  (۲)  $H[0..6] = [C \ E \ B \ G \ F \ D \ A]$   
 (۳)  $H[0..6] = [B \ D \ F \ A \ C \ E \ G]$  (۴)  $H[0..6] = [C \ G \ B \ A \ D \ E \ F]$

۱۷- جدول درهم‌سازی (hashing table) به اندازه‌ی  $2n$  را در نظر بگیرید که تصادف به صورت لیست پیوندی حل می‌شود. فرض کنید که به تعداد  $\frac{n}{\lambda}$  عنصر در جدول هستند. با فرض آن که تابع درهم‌سازی (hashing function) با احتمال برابر هر عنصر را به یک درایه از جدول نگاشت می‌کند، متوسط تعداد عناصر در هر درایه از جدول چند است؟ (دولتی ۸۴)

- (۱)  $\frac{1}{\lambda}$  (۲)  $\frac{1}{4}$  (۳)  $\frac{1}{2}$  (۴)  $\frac{1}{16}$

۱۸- اگر آرایه  $T[0..7]$  و عناصر  $A$  تا  $E$  به آدرس‌های جدول زیر با روش کاوش خطی نگاشت شوند، متوسط تعداد مقایسه‌ها برای جستجوی موفق ( $s$ ) و ناموفق ( $u$ ) کدام است؟

| عنصر | A | B | C | D | E |
|------|---|---|---|---|---|
| آدرس | ۳ | ۵ | ۴ | ۳ | ۱ |

(۱)  $s = 2$   
 $u = 4$  (۲)  $s = 1$   
 $u = 2/5$  (۳)  $s = \frac{\lambda}{\gamma}$   
 $u = 2/5$  (۴)  $s = \frac{\lambda}{\delta}$   
 $u = \frac{19}{\lambda}$

۱۹-  $H$  یک جدول درهم‌سازی (Hash table) است که دارای  $m$  خانه است و در آن  $n$  عنصر ذخیره می‌شود. برای رفع برخورد، این جدول از روش زنجیر (collision resolution by chaining) بهره می‌جوید. متوسط و بدترین زمان جستجو برای یک عنصر چقدر است؟ (II- ۸۸)

- (۱)  $\theta\left(\frac{n}{m}\right), \theta(n)$  (۲)  $\theta\left(\frac{m}{n}\right), \theta(m)$   
 (۳)  $\theta\left(\frac{n}{m}\right), \theta\left(\frac{n}{m}\right)$  (۴)  $\theta\left(\frac{m}{n}\right), \theta\left(\frac{n}{m}\right)$

۲۰- فرض کنید که  $n$  عنصر با کلیدهای مجزا از هم را با استفاده از یک تابع در هم‌سازی ساده و یکنواخت در یک آرایه به اندازه‌ی  $m$  درج می‌کنیم. با این تابع احتمال آن که دو عنصر دلخواه و متفاوت به یک درایه نگاشت شوند برابر است. میانگین تعداد برخوردهای دو عنصر چقدر است؟ (تخصصی هوش مصنوعی ۸۸)

- (۱)  $\Theta\left(\frac{n}{m}\right)$  (۲)  $\Theta\left(\frac{m}{n}\right)$  (۳)  $\Theta\left(\frac{n^2}{m}\right)$  (۴)  $\Theta\left(\frac{n^2}{m^2}\right)$

۲۱- فرض کنید که یک جدول در هم سازی به اندازه ۸۰ (هشتاد) از روش آدرس دهی خطی باز استفاده می کند. ابتدا جدول خالی بوده است و تنها عملیات اضافه کردن و جستجو روی جدول انجام شده است. در حال حاضر وضعیت درایه های ۴۵ تا ۵۶ جدول به صورت زیر است. اعداد بالای آرایه، اندیس درایه ها و اعداد پایین آرایه خروجی تابع درهم سازی است. اگر یکبار دیگر از جدول درهم سازی خالی شروع کنیم و همان عملیات را به غیر از اضافه کردن کلید e دوباره انجام دهیم. در دایره ی ۵۰ چه کلیدی قرار می گیرد؟ (IT ۸۹)

|    |    |    |    |    |    |    |    |    |    |    |    |       |
|----|----|----|----|----|----|----|----|----|----|----|----|-------|
| ۴۵ | ۴۶ | ۴۷ | ۴۸ | ۴۹ | ۵۰ | ۵۱ | ۵۲ | ۵۳ | ۵۴ | ۵۵ | ۵۶ | i (۱) |
|    |    |    |    |    |    |    |    |    |    |    |    | f (۲) |
|    | a  | b  | c  | d  | e  | f  | g  | h  | i  | j  |    | h (۳) |
| ۴۶ | ۴۶ | ۴۶ | ۴۷ | ۴۶ | ۵۱ | ۴۷ | ۴۶ | ۴۸ | ۴۹ |    |    | g (۴) |

۲۲- فرض کنید که در ساخت جدول درهم سازی زیر از روش آدرس دهی خطی باز استفاده شده است. اگر تابع Hash به صورت باقیمانده تقسیم بر ۵ تعریف شده باشد، کدام گزینه نمی تواند ترتیب صحیح درج عناصر در جدول باشد؟ ترتیب از چپ به راست است. (IT ۸۹)

|   |    |                      |
|---|----|----------------------|
| ۰ | ۱۰ | ۱۱، ۶، ۱۰، ۷، ۱۵ (۱) |
| ۱ | ۱۱ | ۱۰، ۱۱، ۶، ۷، ۱۵ (۲) |
| ۲ | ۶  | ۱۱، ۶، ۷، ۱۰، ۱۵ (۳) |
| ۳ | ۷  | ۱۱، ۱۰، ۶، ۱۵، ۷ (۴) |
| ۴ | ۱۵ |                      |

۲۳- یک جدول در هم سازی (hashing table) ساده را در نظر بگیرید که اندازه ی آن در ابتدا ۱ است و آن درایه هم خالی است. در هم سازی هم به صورت بسته (closed) (در مقابل زنجیره ای یا chaining) است و با یک تابع در هم سازی ساده و واریسی خطی (linear probing) انجام می شود. برای درج یک عنصر x، اگر جدول جا داشته باشد، x مستقیماً درج می شود. وگرنه اندازه جدول را دو برابر می کنیم و عناصر فعلی را با هزینه ای برابر تعدادشان، به جدول جدید منتقل می کنیم و سپس x در جدول جدید درج می شود. اگر ۱۳۸۹ عنصر در این جدول درج شوند، هزینه ی کل در مجموع چقدر خواهد بود؟ (توجه کنید که هزینه در اینجا تعداد نوشتن در جدول است و ما بقیه ی هزینه ها را ثابت فرض می کنیم).

(مهندسی کامپیوتر ۹۰)

|          |          |          |          |
|----------|----------|----------|----------|
| ۳۴۴۵ (۴) | ۳۴۱۲ (۳) | ۲۰۴۸ (۲) | ۱۳۸۹ (۱) |
|----------|----------|----------|----------|

۲۴- یک ماتریس  $A$  به اندازه‌ی  $n \times n$  را در نظر بگیرید که همه‌ی عناصر سطرهای آن از چپ به راست و همه‌ی ستون‌های آن از بالا به پایین به صورت غیرنزولی مرتب هستند. می‌خواهیم با مقایسه‌ی عناصر این ماتریس با  $x$ ، در صورت وجود مکان  $x$  را در ماتریس بیابیم. حداکثر چند تا مقایسه می‌توان این کار را انجام داد؟ (تخصصی نرم‌افزار ۸۹)

(۱)  $2n$  (۲)  $n^2$  (۳)  $3n$  (۴)  $n \log n$

۲۵- ماتریس  $M_{\sqrt{n} \times \sqrt{n}}$  را با  $n$  عدد در نظر بگیرید به طوری که اعداد در سطر و ستون ماتریس صعودی مرتب شده باشند. الگوریتم زیر برای یافتن عدد  $k$  در ماتریس  $M$  داده شده است. پیچیدگی زمانی این الگوریتم چند است؟ اگر الگوریتم را تغییر دهیم و در هر ستون از binary search استفاده شود آیا زمان بهتر می‌شود یا بدتر؟ (علوم کامپیوتر ۸۷)

```
Search(K)
begin
     $r = \sqrt{n}$ ;
    for  $c = 1$  to  $\sqrt{n}$  do
        begin
            while ( $M[r, c] > k \ \& \ r > 1$ )
                 $r = r - 1$ ;
            if  $M[r, c] = k$  then return true;
        end;
    return false;
end;
```

(۱)  $O(n)$  و زمان بدتر می‌شود. (۲)  $O(\sqrt{n})$  و زمان بدتر می‌شود.

(۳)  $O(\sqrt{n})$  و زمان بهتر می‌شود. (۴)  $O(n)$  و زمان بهتر می‌شود.

۲۶- در یک جدول درهم‌سازی با روش زنجیره‌ای، با فرض استفاده از تابع درهم‌سازی ساده‌ی یکنوا، یک جستجوی موفق و یک جستجوی ناموفق به طور میانگین به چه زمانی بر حسب ضریب بارگذاری  $\alpha = \frac{n}{m}$  نیاز دارد؟ (یک جدول درهم‌ساز به اندازه  $m$  در خود  $n$  عنصر را ذخیره کرده است). (علوم کامپیوتر - ۹۳)

(۱) جستجو موفق به زمان  $O(1)$  و جستجوی ناموفق به زمان  $O(\alpha)$  نیاز دارد.

(۲) جستجو موفق به زمان  $O(1 + \alpha)$  و جستجوی ناموفق به زمان  $O(1)$  نیاز دارد.

(۳) هر دو به زمان  $O(1 + \alpha)$  نیاز دارند.

(۴) هر دو به زمان  $O(1)$  نیاز دارند.

۲۷- در روش درهم‌سازی باز با واریسی خطی، تابع درهم‌سازی برای عناصر به صورت زیر است:

key: A B C D E F

hash: ۲ ۴ ۲ ۳ ۵ ۴

اگر در ابتدا جدول درهم‌سازی با اندازه‌ی ۶ تهی باشد، به چند روش مختلف می‌توان این ۶

کلید را درج کرد تا جدول نهایی برابر  $H[0..5] = [F B C D A E]$  شود؟ (۹۴-IT)

(۱) ۰ (۲) ۲ (۳) ۴ (۴) ۸

۲۸- آرایه A با n عنصر را m چرخشی گویند اگر با m چرخش داده‌های آن مرتب شده باشند.

برای مثال آرایه  $A = \{۳۵, ۴۲, ۵, ۱۵, ۲۷, ۲۹, ۳۰\}$  یک آرایه ۲ چرخشی است. بهترین

الگوریتم برای یافتن ماکزیمم در این آرایه دارای چه مرتبه زمانی است؟ (علوم کامپیوتر - ۹۴)

(۱)  $O(\log n)$  (۲)  $O(n + m)$

(۳)  $O(m \log n)$  (۴)  $O(n \log m)$

۲۹- در الگوریتم جستجوی سه تایی (تقریباً مشابه با جستجوی دودویی) آرایه مورد نظر به سه

قسمت تقسیم می‌شود و داده مورد جستجو با این عناصر مقایسه می‌شود. کدام رابطه در

هر مرحله نشان‌دهنده‌ی عنصر  $\frac{1}{3}(m_1)$  و عنصر  $\frac{2}{3}(m_2)$  است، اگر حد پایین و بالای آرایه

$\ell$  و  $h$  باشد؟ (علوم کامپیوتر - ۹۴)

$$(۱) \quad m_1 = \frac{h + 2\ell}{3}, \quad m_2 = \frac{h - 2\ell}{3} \quad (۲) \quad m_1 = \frac{\ell + h}{3}, \quad m_2 = 2\frac{(\ell + h)}{3}$$

$$(۳) \quad m_1 = \frac{h + 2\ell}{3}, \quad m_2 = \frac{2h + \ell}{3} \quad (۴) \quad m_1 = \frac{\ell + h}{3}, \quad m_2 = \frac{2(h - \ell)}{3}$$

۳۰- فرض کنید که کلیدهای زیر با ترتیب دل‌خواهی در یک جدول درهم‌سازی به اندازه‌ی ۷ که

در ابتدا تهی است درج می‌شود. فرض کنید که مقدار اولیه‌ی تابع درهم‌سازی به صورت زیر

است و تصادم با واریسی خطی حل می‌شود:

| کلید | اندیس درهم‌سازی |
|------|-----------------|
| B    | ۳               |
| C    | ۴               |
| D    | ۵               |
| F    | ۰               |
| I    | ۳               |
| N    | ۱               |
| U    | ۱               |

چند تا از دنباله‌های زیر (از چپ به راست) می‌تواند محتوای جدول باشد؟ (۹۵ – IT)

- FUNBCDI
- CNFUIDB
- CBIDXUF
- FUIDCNB
- FUNIBCD

۱ (۱)      ۲ (۲)      ۳ (۳)      ۴ (۴)

۳۱- اگر الگوریتم جستجوی دودویی را برای جستجوی عناصر آرایه

$A[1..9] = [-1, 2, 10, 20, 25, 29, 35, 40, 50]$  به کار ببریم، میانگین تعداد مقایسه‌ها برای

جستجوی موفق و ناموفق، به طور تقریبی چقدر است؟ (علوم کامپیوتر – ۹۵)

۱) موفق ۲/۸، ناموفق ۳/۸      ۲) موفق ۲/۸، ناموفق ۲/۸  
 ۳) موفق ۲/۸، ناموفق ۳/۴      ۴) موفق ۳/۴، ناموفق ۳/۴



## پاسخنامه سؤال‌های چهارگزینه‌ای فصل چهارم

۱- گزینه (۳). در این نوع سوالات فرض می‌شود، جستجوی دودویی در هر دور فقط یک مقایسه انجام می‌دهد. یعنی ابتدا عنصر وسط آرایه بررسی می‌شود اگر با عنصر مورد نظر مساوی باشد که کار تمام است در غیر این صورت نیمه چپ یا نیمه راست آرایه بررسی می‌شود.

در آرایه مرتب  $A$  برای یافتن  $x=9$ ، فقط یک مقایسه نیاز است چون ۹ در اندیس  $\left\lfloor \frac{1+9}{2} \right\rfloor = 5$  یعنی وسط آرایه است. برای یافتن  $x=-5$ ، ۲ مقایسه نیاز است، ابتدا با  $A[5]=9$  و سپس با  $A[2]=-5$ . برای یافتن  $x=-6$ ، ۳ مقایسه به ترتیب با  $A[5]$  و  $A[2]$  و  $A[1]$  نیاز است. به همین ترتیب در جدول زیر تعداد مقایسه برای یافتن هر عنصر آمده است، که با توجه به این جدول  $S(9)$  یعنی متوسط تعداد مقایسه جستجوی موفق (Successfull) برای آرایه ۹ عنصری برابر  $\frac{25}{9}$  است.

|                |    |    |   |   |   |    |    |    |     |
|----------------|----|----|---|---|---|----|----|----|-----|
|                | ۱  | ۲  | ۳ | ۴ | ۵ | ۶  | ۷  | ۸  | ۹   |
|                | -۶ | -۵ | ۰ | ۷ | ۹ | ۲۳ | ۵۴ | ۸۲ | ۱۰۱ |
| تعداد مقایسه = | ۳  | ۲  | ۳ | ۴ | ۱ | ۳  | ۲  | ۳  | ۴   |

$$\frac{25}{9} = \text{متوسط مقایسه‌ها} \Rightarrow 25 = \text{مجموع مقایسه‌ها}$$

۲- گزینه (۳). مشابه تست قبل، زیر هر عنصر تعداد مقایسه‌ها نوشته شده است پس  $S(8) = \frac{21}{8}$  است.

|                |   |    |    |    |    |    |    |    |
|----------------|---|----|----|----|----|----|----|----|
|                | ۱ | ۲  | ۳  | ۴  | ۵  | ۶  | ۷  | ۸  |
|                | ۵ | ۱۰ | ۱۵ | ۲۰ | ۲۵ | ۳۰ | ۳۵ | ۴۰ |
| تعداد مقایسه = | ۳ | ۲  | ۳  | ۱  | ۳  | ۲  | ۳  | ۴  |

$$21 = \text{مجموع مقایسه‌ها}$$

$$\frac{21}{8} = \text{متوسط مقایسه‌ها} = 2/625$$

۳- گزینه (۲). جستجوی ناموفق در  $A$  یعنی  $A$  عناصری که در این آرایه نیستند را جستجو کنیم. برای این منظور، یک عنصر قبل از  $A[1]$ ، یک عنصر بعد از  $A[9]$  و بین هر دو عنصر، یک عنصر در نظر می‌گیریم و جستجو می‌کنیم. یعنی تعداد ۱۰ عدد در آرایه ۹ عنصری باید جستجو کنیم. مثلاً برای جستجوی ۸- تعداد ۳

مقایسه به ترتیب با  $A[5]$  و  $A[2]$  و  $A[1]$  نیاز است که مشخص شود ۸- در آرایه وجود ندارد. فرض کنید می‌خواهیم ۸- و  $-5/5$  و  $-1$  و  $2$  و  $8$  و  $12$  و  $30$  و  $60$  و  $90$  و  $110$  را جستجو کنیم. در جدول زیر اندیس‌هایی که بررسی می‌شوند به ترتیب آمده است:

| عنصر                         | ۸-      | -۵/۵    | -۱      | ۲          | ۸          | ۱۲      | ۳۰      | ۶۰      | ۹۰         | ۱۱۰        |
|------------------------------|---------|---------|---------|------------|------------|---------|---------|---------|------------|------------|
| تعداد مقایسه                 | ۳       | ۳       | ۳       | ۴          | ۴          | ۳       | ۳       | ۳       | ۴          | ۴          |
| اندیس‌های بررسی شده به ترتیب | ۵, ۲, ۱ | ۵, ۲, ۱ | ۵, ۲, ۳ | ۵, ۲, ۳, ۴ | ۵, ۲, ۳, ۴ | ۵, ۷, ۶ | ۵, ۷, ۶ | ۵, ۷, ۸ | ۵, ۷, ۸, ۹ | ۵, ۷, ۸, ۹ |

توجه کنید مثلاً برای جستجوی  $x = 90$ ، ابتدا با  $A[5]$  سپس با  $A[7]$  بعد با  $A[8]$  و در نهایت با  $A[9]$  مقایسه صورت می‌گیرد و معلوم می‌شود که ۹۰ در آرایه نیست. با توجه به این جدول، متوسط تعداد مقایسه جستجوی ناموفق در آرایه ۹ عنصری برابر است با:

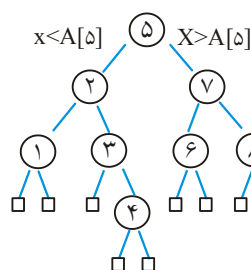
$$U(9) = \frac{3+3+3+4+4+4+3+3+3+4+4}{10} = \frac{34}{10}$$

۴- گزینه (۲). با توجه به تست‌های ۱ و ۳،  $S(9) = \frac{25}{9}$  و  $U(9) = \frac{34}{10}$  می‌باشد پس واضح است که گزینه ۲ صحیح نیست. گزینه ۳ را به ازای  $n = 9$  بررسی می‌کنیم:

$$S(9) = \left(1 + \frac{1}{9}\right) \times \frac{34}{10} - 1 = \frac{10}{9} \times \frac{34}{10} - 1 = \frac{34}{9} - 1 = \frac{25}{9}$$

اگر گزینه ۳ درست باشد آنگاه گزینه ۱ نیز درست است، زیرا با توجه به رابطه گزینه ۳،  $S(n)$  و  $U(n)$  هم مرتبه هستند.

گزینه ۳ را اثبات می‌کنیم.



می‌توان جستجوی دو دویی را با یک درخت دو دویی نشان داد. مثلاً برای آرایه ۹ عنصری  $A[1..9]$  می‌توان درخت روبرو را رسم کرد:

در این درخت (درخت تصمیم)، داخل هر نود مقدار mid نوشته شده است. مثلاً ابتدا با  $A[5]$  مقایسه می‌کنیم اگر کمتر بود به سمت چپ می‌رویم و با

$A[2]$  مقایسه می‌کنیم و اگر بیشتر بود به راست می‌رویم و با  $A[7]$  مقایسه می‌کنیم و به همین ترتیب وقتی به برگ‌ها (گره‌های مستطیلی شکل) برسیم یعنی جستجوی ناموفق و اگر به گره‌های داخلی (گره‌های دایره‌ای شکل) کار تمام شود یعنی جستجوی موفق.

$d(x)$  را برابر مسافت از ریشه به گره  $x$  تعریف می‌کنیم مثلاً  $d(5) = 0$  یا  $d(1) = 2$  ...  
 $I$  را برابر مجموع مسافت‌های گره‌های داخلی تعریف می‌کنیم:  $(۱) I = \sum_{x \text{ گره داخلی}} d(x)$

$E$  را برابر مجموع طول مسیرهای خارجی تعریف می‌کنیم  $(۲) E = \sum_{x \text{ برگ}} d(x)$

می‌توان نشان داد در درخت دودویی  $E = I + 2n$  می‌باشد ( $n$  تعداد گره‌های داخلی).  
 اگر  $S(n)$  میانگین تعداد مقایسه‌ها در جستجوی موفق باشد آنگاه

$$S(n) = \frac{\sum_{x \text{ گره داخلی}} (d(x) + 1)}{n} \quad (۳) \text{ و اگر } U(n) \text{ میانگین تعداد مقایسه‌ها در جستجوی ناموفق}$$

$$U(n) = \frac{\sum_{x \text{ گره خارجی}} d(x)}{n+1} \quad (۴) \text{ می‌باشد.}$$

از روابط ۱ و ۲ و ۳ و ۴ نتیجه می‌شود:

$$S(n) = \frac{I+n}{n}, \quad U(n) = \frac{E}{n+1}$$

$$S(n) = \left(1 + \frac{1}{n}\right) U(n) - 1$$

۵- گزینه (۳). جستجوی دودویی در بدترین حالت از مرتبه  $\theta(\lg n)$  است که می‌توان گفت همواره از مرتبه  $O(\lg n)$  است یا حتی می‌توان گفت همواره از مرتبه  $O(n)$  است (دقت کنید  $O(n)$  یعنی کمتر یا مساوی  $n$ ). پس گزینه‌های ۱ و ۲ و ۴ صحیح هستند.

برای نقض گزینه ۳، در آرایه  $A$   $\begin{matrix} 1 & 2 & 3 \\ 10 & 20 & 30 \end{matrix}$ ، یکبار  $x = 5$  و یکبار  $x = 35$  را جستجو کنید. برای جستجوی  $x = 5$ ،  $M = (1+3) \div 2 = 2$  و ابتدا مقایسه  $x = A[2]$  انجام می‌شود و سپس مقایسه  $x < A[2]$  انجام می‌شود و فراخوانی  $\text{Search}(x, 1, 1)$  انجام می‌شود و در این دور  $M = (1+1) \div 2 = 1$  و مقایسه  $x = A[1]$  و سپس مقایسه  $x < A[1]$  انجام می‌شود ولی چون شرط  $M > L$  برقرار نیست، مقایسه  $x > A[1]$  صورت می‌گیرد و چون نادرست است، مقدار  $\text{False}$  بازگردانده می‌شود. یعنی تعداد کل مقایسه  $x$  با عناصر ۵ تاست.

حال برای جستجوی  $x = 35$ ،  $M = 2$  و مقایسه  $x = A[2]$  و  $x < A[2]$  و سپس  $x > A[2]$  انجام می‌شود و فراخوانی  $\text{Search}(x, 3, 3)$  صورت می‌گیرد. در این دور  $M = 3$  و هر سه مقایسه  $x = A[3]$  و  $x < A[3]$  و  $x > A[3]$  انجام می‌شود. یعنی جمعاً ۶ مقایسه صورت می‌گیرد.

۶- گزینه (۲). با فرض اینکه در هر دور فقط یک مقایسه صورت می‌گیرد. ابتدا عنصر وسط آرایه مقایسه می‌شود، که بعد از آن آرایه نصف می‌شود و جستجو به صورت بازگشتی در نیمه چپ یا نیمه راست ادامه می‌یابد و می‌توان نوشت  $T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 1$  که با فرض اینکه  $n$  توانی از ۲ باشد می‌توان گزینه ۲ را انتخاب کرد.

۷- گزینه (۳). اگر  $x < a[\text{middle}]$  باشد باید جستجو را در نیمه چپ آرایه یعنی در  $a[\text{Left}..\text{middle}-1]$  انجام دهیم.

۸- گزینه (۲). با توجه به متن درس  $A(n) = \frac{3n}{4} + \frac{1}{4}$  می‌باشد.

۹- گزینه (۴).  $w$  را با عنصر وسط آرایه  $(A[\text{mid}])$  مقایسه کنید. اگر  $w < A[\text{mid}]$ ، جستجو را در  $A[1..\text{mid}-1]$  انجام دهید و اگر  $w > A[\text{mid}]$ ، جستجو را در  $A[\text{mid}+1..n]$  انجام دهید.

با توجه به اینکه اختلاف هر دو عنصر متوالی حداکثر ۱ است و  $A[1] < A[n]$ ، این روش در صورت وجود  $w$  حتماً آن را می‌یابد. (دقت کنید آرایه لزوماً مرتب نیست ولی اگر  $w$  وجود داشته باشد، روش ما آن را می‌یابد) مرتبه این روش  $O(\lg n)$  است.

۱۰- گزینه (۳). می‌توان با هر پرسش نصف دانشجویان را حذف کرد. ابتدا  $\frac{n}{4}$  نفر انتخاب کنید، و پرسش را مطرح کنید، اگر جواب بله شنیدید، نابغه در بین آنهاست و در غیر این صورت، نابغه در بین  $\frac{n}{4}$  نفر دیگر است. به همین ترتیب با هر پرسش تعداد افراد را نصف کنید و همین رویه را ادامه دهید. پس تعداد پرسش حداکثر برابر  $\lg n$  است.

۱۱- گزینه (۳). شبیه جستجوی دودویی، عنصر وسط آرایه را بررسی می‌کنیم اگر  $A[i] = i$  کار تمام است و اگر  $A[i] < i$  نیمه سمت چپ و اگر  $A[i] > i$  نیمه سمت راست را با همین روش بررسی می‌کنیم.

۱۲- گزینه (۲). با توجه به متن درس جستجو در دنباله چرخشی با مرتبه  $O(\lg n)$  قابل انجام است. برای یافتن مینیمم کافی است، عنصر وسط را با عنصر آخر مقایسه کنید، اگر عنصر وسط از عنصر آخر آرایه، کوچکتر باشد یعنی باید در نیمه اول آرایه دنبال مینیمم بگردیم و در غیر این صورت در نیمه دوم به دنبال مینیمم باشیم.

۱۳- گزینه (۲). مشابه تست قبل.

۱۴- گزینه (۱). بررسی اینکه یک دنباله به طول  $m$ ، زیر دنباله یک دنباله به طول  $n$  است از مرتبه  $O(n+m)$  است.

برای یافتن ماکزیمم  $i$  که  $B^i$  زیردنباله  $A$  باشد، واضح است که  $i$  نمی‌تواند از  $\frac{n}{m}$  بیشتر

شود. ابتدا  $i$  را برابر  $\lceil \frac{n}{m} \rceil$  قرار دهید اگر  $B^i$  زیر دنباله  $A$  باشد، شبیه جستجوی دودویی، مقادیر کوچکتر از  $\lceil \frac{n}{m} \rceil$  را نادیده بگیرید و اگر  $B^i$  زیر دنباله  $A$  نبود، مقادیر بیشتر از  $\lceil \frac{n}{m} \rceil$  را نادیده بگیرید و با روش دودویی این عمل را انجام دهید. تعداد بررسی‌ها برای یافتن ماکزیمم  $i$  برابر  $\lceil \lg \frac{n}{m} \rceil$  است پس زمان اجرا  $O((n+m) \lg \frac{n}{m})$  است که چون  $n > m$  پس جواب  $O(n \cdot \lg \frac{n}{m})$  است.

۱۵- گزینه (۳). فرض کنید  $\langle A, B, C \rangle$  سه جعبه کنار هم هستند که اعداد داخل آنها  $(x, y, z)$  (چپ به راست) می‌باشد. اگر  $x < y < z$  آنگاه  $y$  جواب است اگر  $x < y < z$  حتماً جوابی در سمت راست  $y$  وجود دارد، اگر  $x > y > z$  حتماً جوابی سمت چپ  $y$  وجود دارد و اگر  $x > y < z$  حتماً در سمت چپ و راست  $y$  جوابی وجود دارد. پس الگوریتم به این صورت است که سه جعبه وسط یعنی  $(15, 16, 17)$  را باز کنید، بدترین حالت می‌تواند حالتی باشد که  $x_{15} < x_{16} < x_{17}$  (منظور از  $x_i$  عدد جعبه شماره  $i$  است) پس باید سمت راست برویم و سه جعبه وسط جعبه‌های ۱۸ تا ۳۱ یعنی  $(23, 24, 25)$  را باز کنیم اگر  $x_{23} < x_{24} < x_{25}$ ، باید سمت راست برویم و سه جعبه وسط  $26..31$  یعنی  $(27, 28, 29)$  را باز کنیم اگر  $x_{27} < x_{28} < x_{29}$ ، جعبه‌های  $(30, 31)$  را باز می‌کنیم و جواب بدست می‌آید. پس در بدترین حالت ۱۱ جعبه باید باز شود.

۱۶- گزینه (۲). اگر عناصر به همان ترتیب (ABCDEFGH) چپ به راست وارد شوند، گزینه ۱ حاصل می‌شود. اگر ترتیب به صورت (ACEGBDF) باشد، گزینه ۳ بدست می‌آید (البته ترتیب‌های دیگر نیز دارد). اگر ورودی به صورت (ADEFCGB) باشد گزینه ۴ حاصل می‌شود. در گزینه ۲، ابتدا حتماً باید  $G$  وارد شود ولی بعد از آن هر عنصری وارد شود، گزینه ۲ حاصل نمی‌شود.

۱۷- گزینه (۴).

$$\frac{\frac{n}{8}}{2n} = \frac{1}{16}$$

۱۸- گزینه (۴).

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| ۰ | ۱ | ۲ | ۳ | ۴ | ۵ | ۶ | ۷ |
|   | E |   | A | C | B | D |   |

$$s = \frac{1+1+1+1+4}{5} = \frac{8}{5} \quad u = \frac{1+2+1+5+4+3+2+1}{8} = \frac{19}{8}$$

۱۹- گزینه (۱). متوسط  $\theta(1+\alpha)$  که  $\alpha = \frac{n}{m}$  که در این تست از ۱ صرف‌نظر شده و فقط در صورتی درست که  $\alpha \geq 1$ .

۲۰- گزینه (۳). میانگین تعداد برخوردها  $\frac{n(n-1)}{2m} \times \frac{1}{m} = \frac{n}{2}$  است که از مرتبه  $\theta\left(\frac{n^2}{m}\right)$

۲۱- گزینه (۴).  $a$  و  $b$  و  $c$  و  $d$  تغییر نمی‌کنند.  $f$  نیز در ۵۱ قرار می‌گیرد.  $g$  که با  $b$  تصادف می‌کند به ۵۰ وارد می‌شود.

۲۲- گزینه (۴). گزینه ۴ را بررسی می‌کنیم:

|                  |   |    |   |   |   |
|------------------|---|----|---|---|---|
|                  | ۰ | ۱  | ۲ | ۳ | ۴ |
| $11 \bmod 5 = 1$ |   | ۱۱ |   |   |   |

|                  |    |    |   |   |   |
|------------------|----|----|---|---|---|
|                  | ۰  | ۱  | ۲ | ۳ | ۴ |
| $10 \bmod 5 = 0$ | ۱۰ | ۱۱ |   |   |   |

|                 |    |    |   |   |   |
|-----------------|----|----|---|---|---|
|                 | ۰  | ۱  | ۲ | ۳ | ۴ |
| $6 \bmod 5 = 1$ | ۱۰ | ۱۱ | ۶ |   |   |

|                  |    |    |   |    |   |
|------------------|----|----|---|----|---|
|                  | ۰  | ۱  | ۲ | ۳  | ۴ |
| $15 \bmod 5 = 0$ | ۱۰ | ۱۱ | ۶ | ۱۵ |   |

|                 |    |    |   |    |   |
|-----------------|----|----|---|----|---|
|                 | ۰  | ۱  | ۲ | ۳  | ۴ |
| $7 \bmod 5 = 2$ | ۱۰ | ۱۱ | ۶ | ۱۵ | ۷ |

سایر گزینه‌ها را بررسی کنید که صحیح هستند.

۲۳- گزینه (۴). این تست حذف شد. فرض کنید هزینه برابر مجموع تعداد درجه‌هایی است که انجام می‌دهیم به علاوه تعداد عملیات کپی که بخاطر دو برابر شدن جدول انجام می‌شود:

$$= 3436 = 1024 + (1 + 2 + 4 + \dots + 1024) = 1389 + (\text{کپی‌ها}) + \text{درج‌ها} = \text{هزینه}$$

دقت کنید پس از درج عنصر  $2^i$ ، جدول ۲ برابر می‌شود و  $2^i$  عنصر جدول اول به جدول دوم کپی می‌شوند)

۲۴- گزینه (۱). با روش‌های مختلفی می‌توان جستجو کرد. مثلاً سطر اول را بررسی کنیم (حداکثر  $n$  مقایسه) با این بررسی می‌توان فهمید که عنصر  $x$  در صورت وجود، در کدام ستون است و سپس آن ستون را بررسی کنیم (حداکثر  $n-1$  مقایسه) پس جمعاً حداکثر  $2n-1$  مقایسه لازم دارد. و یا روش دوم به این صورت که  $x$  را با عنصر پایین چپ مقایسه کنیم اگر  $x$  از آن بزرگتر بود به راست حرکت کنیم در غیر این صورت بالا حرکت کنیم و به همین شیوه ادامه دهیم، این شیوه نیز حداکثر  $2n-1$  مقایسه لازم دارد به شکل زیر دقت

کنید. در این مثال  $x=40$  فرض شده است:

$$\begin{bmatrix} 10 & 20 & 30 & 40 \\ 12 & 27 & 50 & 60 \\ 20 & 31 & 61 & 70 \\ 30 & 45 & 70 & 90 \end{bmatrix}$$

۲۵- گزینه (۲). فرض کنید  $n=9$  و آرایه  $M$  به شکل زیر باشد:

$$M = \begin{bmatrix} 5 & 10 & 15 \\ 20 & 25 & 30 \\ 35 & 40 & 45 \end{bmatrix}_{3 \times 3}$$

و می‌خواهیم  $k=4$  را جستجو کنیم. به ازای  $c=1$  عناصر  $M[1,1]$  و  $M[2,1]$  و  $M[3,1]$  با  $k$  مقایسه می‌شوند و حلقه  $\text{for}$  بار دوم  $c=2$  اجرا می‌شود ولی  $\text{while}$  اجرا نمی‌شود. به ازای  $c=3$  نیز همین‌طور. می‌توان بررسی کرد که مرتبه این الگوریتم  $O(\sqrt{n})$  است ولی اگر در ستون‌ها جستجوی باینری صورت گیرد مرتبه  $O(\sqrt{n} \log n)$  می‌شود. (این ماتریس رایانگ گویند)

۲۶- گزینه (۳). یک واحد زمان تابع درهم‌ساز و زمان  $\alpha$  متوسط زمان جستجوی لیست است.

۲۷- گزینه (۴). عناصر  $C$  و  $D$  و  $E$  در آدرس طبیعی خود هستند، پس این عناصر باید ابتدا وارد شوند ولی می‌توانند به هر ترتیبی نسبت به یکدیگر باشند. بنابراین حداقل  $6!=3!$  حالت وجود دارد یعنی گزینه ۴ صحیح است. با بررسی دقیق، تعداد حالات ورود برابر ۸ است که عبارتند از: (از چپ به راست):

CDEAFB , CEDAFB , DECAFB , DCEAFB , ECDAFB , FDCAFB

CDAEFB , DCAEFB

۲۸- گزینه (۱). همانطور که در درس دیدیم جستجو در آرایه چرخشی از مرتبه  $O(\lg n)$  است.

۲۹- گزینه (۲). آرایه  $A[l..h]$  است.  $m_1 = \left\lfloor \frac{h+1}{2} \right\rfloor$  است و  $m_2 = \left\lfloor \frac{m_1+h}{2} \right\rfloor$  می‌باشد پس

$$m_2 = \frac{\frac{h+1}{2} + h}{2} = \frac{4h+1}{6}$$

۳۰- گزینه (۲). به راحتی قابل بررسی است. فقط دنباله اول و آخر درست هستند.

۳۱- گزینه (۳).

$$s(9) = \frac{3+2+3+4+1+3+2+3+4}{9} = \frac{25}{9} \approx 2/8$$

$$u(9) = \frac{3+3+3+4+4+3+3+3+4+4}{10} = \frac{34}{10}$$

## فصل ۵

## مرتبه‌های آماری و مرتب‌سازی

یافتن min و max در  $A[1..n]$ 

برای یافتن مینیمم یا ماکزیمم (یکی از دو تا) در آرایه  $n$  عنصری حداقل  $n-1$  مقایسه نیاز است. ما می‌خواهیم min و max را با هم بیابیم. برای این منظور سه روش ارائه می‌دهیم:

**روش ۱:** عنصر اول را در متغیر min و max قرار می‌دهیم و از  $A[2]$  تا  $A[n]$  بررسی می‌کنیم هر عنصر که از min کمتر باشد آن را min قرار می‌دهیم و هر عنصر که از max بیشتر باشد، آن را در متغیر max قرار می‌دهیم:

```
min = max = A[1];
for(i = 2 to n)
{
    if(A[i] < min) min = A[i];
    if(A[i] > max) max = A[i];
}
```

عمل اصلی در این کد، مقایسه است که  $2(n-1)$  بار انجام می‌شود ولی اگر قبل از if دوم، else قرار دهیم آنگاه تعداد مقایسه حداکثر  $2(n-1)$  است و در حالتی که آرایه صعودی است پیش می‌آید و حداقل تعداد مقایسه  $n-1$  است که در حالتی که آرایه نزولی است رخ می‌دهد، زیرا اگر آرایه نزولی باشد، شرط  $A[i] < min$  همواره صحیح است و شرط  $A[i] > max$  هیچگاه اجرا نمی‌شود.

**روش دوم:** با شیوه تقسیم و غلبه، آرایه را به دو نیمه  $\lceil \frac{n}{2} \rceil$  و  $\lfloor \frac{n}{2} \rfloor$  عنصری تقسیم می‌کنیم و در هر نیمه به صورت بازگشتی مینیمم و ماکزیمم می‌یابیم سپس مینیمم دو نیمه را با هم و



ماکزیمم دو نیمه را با هم مقایسه می‌کنیم تا  $\min$  و  $\max$  نهایی بدست آید. مثلاً آرایه  $n = 7$

عنصری  $\lfloor \frac{7}{2} \rfloor = 3$  را تصور کنید، مینیمم و ماکزیمم در  $A$ 

|    |    |    |    |    |   |    |
|----|----|----|----|----|---|----|
| ۱۰ | ۳۰ | ۱۷ | ۲۰ | ۱۵ | ۹ | ۲۷ |
|----|----|----|----|----|---|----|

عنصر اول یعنی در 

|    |    |    |
|----|----|----|
| ۱۰ | ۳۰ | ۱۷ |
|----|----|----|

 عبارت است از  $L_{\min} = 10$  و  $L_{\max} = 30$ ؛ و مینیمم و

ماکزیمم در  $\lceil \frac{7}{2} \rceil = 4$  عنصر آخر یعنی در 

|    |    |   |    |
|----|----|---|----|
| ۲۰ | ۱۵ | ۹ | ۲۷ |
|----|----|---|----|

 عبارت است از  $r_{\min} = 9$  و

$r_{\max} = 27$ . با مقایسه  $L_{\min}$  و  $r_{\min}$  مقدار مینیمم برابر ۹ و با مقایسه  $L_{\max}$  و  $r_{\max}$  مقدار ماکزیمم برابر ۳۰ است. الگوریتم تقسیم و غلبه برای یافتن  $\min$  و  $\max$  در  $A[L..U]$  به شکل زیر است:

```

minmax(L, U, min, max)
{
۱. if (L == U) min = max = A[L];
۲. else if (U == L + 1)
    if (A[L] < A[U]) {min = A[L]; max = A[U];}
    else {min = A[U]; max = A[L];}
۳. else{
    mid =  $\lfloor \frac{L + U}{2} \rfloor$ 
۴.    minmax(L, mid, Lmin, Lmax);
۵.    minmax(mid + 1, U, rmin, rmax);
۶.    if (Lmin < rmin) min = Lmin else min = rmin;
۷.    if (Lmax > rmax) max = Lmax else max = rmax;
}
}

```

اگر فرض کنیم  $T(n)$  تعداد مقایسه‌های بین عناصر باشد، قسمت ۱ در الگوریتم فوق، بررسی می‌کند آرایه یک عنصری است و مقایسه‌ای انجام نمی‌شود پس  $T(1) = 0$  قسمت ۲ در الگوریتم فوق بررسی می‌کند آرایه ۲ عنصری است و با یک مقایسه  $\min$  و  $\max$  را می‌یابد پس  $T(2) = 1$ . در قسمت ۳ الگوریتم، آرایه بیش از ۲ عنصری است که خطوط ۴ و ۵ با فراخوانیهای بازگشتی آرایه را به ۲ و ۱ عنصری تبدیل می‌کنند. تعداد مقایسه‌های خط ۴ برابر  $T(\lfloor \frac{n}{2} \rfloor)$  و تعداد مقایسه‌های خط ۵ برابر  $T(\lceil \frac{n}{2} \rceil)$  است و تعداد مقایسه‌های خطوط ۶ و ۷ نیز برابر ۲ (هر دو خط با هم) است بنابراین:

$$T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 2, \quad n > 2$$

$$T(1) = 0, T(2) = 1$$

لازم به ذکر است برای آرایه  $A[1..n]$  باید الگوریتم فوق را به صورت  $\text{minmax}(1, n, \min, \max)$  فراخوانی کنیم.

**مثال ۱:** برای آرایه ۲۰ عنصری الگوریتم  $\text{minmax}$  چه تعداد مقایسه بین عناصر آرایه انجام می‌دهد؟

**پاسخ:** باید  $T(20)$  را با توجه به رابطه بازگشتی گفته شده بیابیم:

$$T(20) = T(10) + T(10) + 2 = 14 + 14 + 2 = 30$$

$$T(10) = T(5) + T(5) + 2 = 6 + 6 + 2 = 14$$

$$T(5) = T(2) + T(3) + 2 = 1 + 3 + 2 = 6$$

$$T(3) = T(1) + T(2) + 2 = 0 + 1 + 2 = 3$$

**روش ۳:** روش دیگری برای یافتن  $\min$  و  $\max$  در  $A[1..n]$  موسوم به جفت کردن کلیدها،

وجود دارد. فرض کنید  $n$  زوج است. اعداد آرایه را به  $\frac{n}{2}$  جفت تقسیم کنید و در هر جفت با یک مقایسه مینیمم و ماکسیمم را بیابید تاکنون  $\frac{n}{2}$  مقایسه کرده‌ایم و  $\frac{n}{2}$  عنصر مینیمم و  $\frac{n}{2}$  عنصر ماکزیمم داریم. حال با  $\frac{n}{2} - 1$  مقایسه بین مینیمم‌ها  $\min$  را بیابید و با  $\frac{n}{2} - 1$  مقایسه بین ماکسیمم‌ها  $\max$  را بیابید. پس تعداد کل مقایسه‌ها  $2 - \frac{3n}{4} = \frac{n}{4} - 1 + \frac{n}{4} - 1$  است. حال فرض کنید  $n$  فرد است.  $n - 1$  عنصر اول آرایه را جفت کنید پس  $\frac{n-1}{2}$  جفت داریم، حال در هر جفت مینیمم و ماکسیمم را بیابید پس  $\frac{n-1}{2}$  مقایسه انجام دادیم. حال عنصر آخر آرایه را هم به دسته مینیمم‌ها و هم به دسته ماکسیمم‌ها اضافه کنید پس  $1 + \frac{n-1}{2}$  عنصر در دسته مینیمم‌ها و همین تعداد عنصر در دسته ماکسیمم‌ها وجود دارد، حال با  $1 - (\frac{n-1}{2} + 1)$  مقایسه در دسته مینیمم‌ها  $\min$  پیدا کنید. و با همین تعداد مقایسه در دسته ماکزیمم‌ها  $\max$  پیدا کنید بنابراین تعداد کل مقایسه‌ها  $\frac{3n}{4} - \frac{3}{4} = \frac{3(n-1)}{4}$  است.

#### نکته

برای یافتن  $\min$  و  $\max$  در آرایه  $n$  عنصری با هر روش مقایسه‌ای، اگر  $n$  زوج باشد تعداد مقایسه حداقل  $2 - \frac{3n}{4}$  و اگر  $n$  فرد باشد تعداد مقایسه حداقل  $\frac{3n}{4} - \frac{3}{4}$  است. به ازای هر  $n$  می‌توان گفت تعداد مقایسه حداقل  $2 - \lceil \frac{3n}{4} \rceil$  است.

الگوریتم روش ۳ برای یافتن min و max در  $A[1..n]$  با فرض اینکه  $n$  زوج است. به شکل زیر است:

```
if (A[1] < A[2]) { min = A[1]; max = A[2]; }
else { min = A[2]; max = A[1]; }
for (i = 3; i ≤ n - 1; i = i + 2)
{
    if (A[i] > A[i + 1]) swap(A[i], A[i + 1]);
    if (A[i] < min) min = A[i];
    if (A[i + 1] > max) max = A[i + 1];
}
```

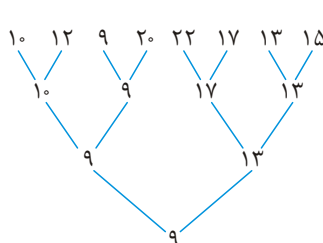
در این الگوریتم، حلقه for،  $\frac{n-2}{2}$  بار اجرا می‌شود و در هر بار اجرا، ۳ مقایسه انجام می‌شود. و همچنین یک مقایسه ابتدای الگوریتم بین  $A[1]$  و  $A[2]$  انجام می‌شود پس کل مقایسه‌ها برابر  $2 - \frac{3n}{2} + 1 = 3\left(\frac{n-2}{2}\right) + 1$  است که قبلاً نیز گفته شد.

### یافتن دومین مینیمم (یا دومین ماکزیمم)

می‌خواهیم در آرایه  $A[1..n]$  عنصر کمینه دوم را بیابیم. (بیشینه دوم یا دومین ماکزیمم نیز به طور مشابه قابل بدست آمدن است) مثلاً در 

|    |    |   |    |
|----|----|---|----|
| ۱۰ | ۲۰ | ۵ | ۱۲ |
|----|----|---|----|

 ۵ مینیمم و ۱۰ دومین مینیمم (کمینه دوم) است. یک روش برای این منظور این است که مینیمم را با  $n-1$  مقایسه بیابیم و آن را حذف کنیم و سپس با  $n-2$  مقایسه مجدداً مینیمم را بیابیم که جمعاً  $2n-3$  مقایسه انجام می‌شود. روش دیگری موسوم به تورنمنت وجود دارد که تعداد مقایسه کمتری انجام می‌دهد، البته مرتبه آن  $\theta(n)$  است و اصلاً با مرتبه کمتر از  $\theta(n)$  نمی‌توان این مسئله را حل کرد. ما فقط سعی می‌کنیم تعداد مقایسه‌ها را کمی کاهش دهیم. روش تورنمنت همانند مسابقات تک حذفی است، به این صورت که عناصر را جفت می‌کنیم و در هر جفت یک مقایسه انجام می‌دهیم و مینیمم آن جفت را برنده اعلام می‌کنیم، حال همین عمل را بین برنده‌ها انجام می‌دهیم تا زمانیکه مینیمم نهایی بدست آید، تاکنون  $n-1$  مقایسه انجام شده است. برای یافتن دومین مینیمم به این نکته توجه داریم که دومین مینیمم حتماً در مرحله‌ای با اولین مینیمم مقایسه شده و حذف شده است. حال کافیست در بین عناصری که با اولین مینیمم مقایسه شده‌اند و تعدادشان حداکثر  $\lceil \lg n \rceil$  عنصر است، مینیمم پیدا کنیم که نیاز به  $\lceil \lg n \rceil - 1$  مقایسه دارد. بنابراین روش تورنمنت برای یافتن دومین مینیمم تعداد  $n + \lceil \lg n \rceil - 2$  مقایسه انجام می‌دهد. و ثابت می‌شود با کمتر از این تعداد مقایسه نمی‌توان این مسئله را حل کرد. برای درک بهتر روش تورنمنت آرایه



$n = 8$  عنصری را 

|    |    |   |    |    |    |    |    |
|----|----|---|----|----|----|----|----|
| ۱۰ | ۱۲ | ۹ | ۲۰ | ۲۲ | ۱۷ | ۱۳ | ۱۵ |
|----|----|---|----|----|----|----|----|

 در نظر بگیرید، درخت مقابل نحوه مقایسه عناصر را نشان می‌دهد که بعد از انجام ۷ مقایسه، مینیمم یعنی ۹ بدست می‌آید. حال از بین ۳ عنصر ۲۰ و ۱۰ و ۱۳ که با ۹ مقایسه شده‌اند باید مینیمم پیدا کنیم که با ۲ مقایسه مینیمم یعنی ۱۰ پیدا می‌شود. پس ۱۰ عنصر کمینه دوم است.

### یافتن عنصر کمینه $k$ ام ( $k$ امین مینیمم)

می‌خواهیم در آرایه  $A[1..n]$  عنصر کمینه  $k$  ام را بیابیم. مثلاً  $k = 1$  یعنی اولین مینیمم،  $k = 2$  یعنی دومین مینیمم،  $k = \frac{n}{2}$  یعنی عنصر میانه (median). مثلاً در آرایه 

|    |    |   |    |    |
|----|----|---|----|----|
| ۱۰ | ۲۰ | ۷ | ۱۵ | ۳۵ |
|----|----|---|----|----|

 میانه ۱۵ (سومین مینیمم) است. یا در آرایه 

|   |    |    |   |    |    |
|---|----|----|---|----|----|
| ۹ | ۱۰ | ۲۰ | ۷ | ۱۵ | ۳۵ |
|---|----|----|---|----|----|

 که  $n = 6$  عنصری است می‌توان هم ۱۰ و هم ۱۵ را میانه فرض کرد (سومین یا چهارمین مینیمم) برای یافتن  $k$  امین مینیمم یک روش این است که آرایه را مرتب کنیم و عنصر  $k$  ام جواب است ولی این روش خوب نیست چون خواهیم دید که مرتب‌سازی مقایسه‌ای در حالت متوسط از مرتبه  $\Omega(n \lg n)$  است، حال آنکه ما می‌توانیم با مرتبه  $\theta(n)$ ، عنصر کمینه  $k$  ام (یا بیشینه  $k$  ام) را بیابیم. یک روش برای یافتن  $k$  امین مینیمم این است که با  $n - 1$  مقایسه اولین مینیمم و سپس با  $n - 2$  مقایسه اضافی، دومین مینیمم و به همین ترتیب در نهایت با  $n - k$  مقایسه اضافی،  $k$  امین مینیمم را بیابیم که تعداد مقایسه‌های این روش  $(n - 1) + (n - 2) + \dots + (n - k) = nk - \frac{k(k+1)}{2} = \theta(nk)$  است که خیلی وقتها مناسب نیست. مثلاً اگر  $k = \frac{n}{2}$  باشد، مرتبه این روش  $\theta(n^2)$  است.

روش مناسب برای یافتن  $k$  امین مینیمم این است که آرایه را پارتیشن (افراز) کنیم. پارتیشن یک عنصر (مثلاً اولین عنصر آرایه) را به عنوان محور (pivot) انتخاب می‌کند و سپس آرایه را دو قسمت می‌کند، قسمت اول عناصر کوچکتر از محور و قسمت دوم عناصر بزرگتر از محور، و خود محور را در مکان اصلی‌اش قرار می‌دهد. یعنی بعد از پارتیشن اگر محور در مکان  $i$  قرار گرفت به این معنی است که محور  $i$  امین مینیمم است. حال کافیست  $k$  را با  $i$  مقایسه کنیم اگر  $k = i$  به این معنی است که  $k$  امین مینیمم را یافته‌ایم و الگوریتم خاتمه می‌یابد. اگر  $k < i$  باید در قسمت عناصر سمت چپ محور به دنبال  $k$  امین مینیمم بگردیم. اگر  $k > i$  باید در قسمت عناصر سمت راست محور به دنبال  $k - i$  امین مینیمم بگردیم. مثلاً فرض کنید ما در آرایه  $n = 8$  عنصری

۱۰ به دنبال پنجمین مینیمم هستیم. محور را ۱۰ انتخاب می‌کنیم و آرایه را پارتیشن می‌کنیم، بعد از پارتیشن، عناصر کمتر از ۱۰ به ابتدای آرایه منتقل می‌شوند. و آرایه به 

|   |   |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|
| ۷ | ۵ | ۱۰ | ۱۵ | ۱۲ | ۲۰ | ۱۷ | ۱۳ |
|---|---|----|----|----|----|----|----|

 تبدیل می‌شود. عدد ۱۰ در مکان سوم قرار گرفته است یعنی ۱۰ سومین کوچکترین است. ما به دنباله پنجمین کوچکترین بودیم پس کافیت سمت راست ۱۰ یعنی 

|    |    |    |    |    |
|----|----|----|----|----|
| ۱۵ | ۱۲ | ۲۰ | ۱۷ | ۱۳ |
|----|----|----|----|----|

 را فراخوانی بازگشتی کنیم و دومین مینیمم را بیابیم (یعنی مجدداً ۱۵ را محور بگیریم و پارتیشن کنیم) که ۱۳ جواب می‌باشد. الگوریتم select عنصر کمینه  $k$ ام را در  $A[L..U]$  می‌یابد:

```
select(A[L..U], k)
{
  ۱. if (L == U) return A[L];
  ۲. j = partition(A[L..U]);
  ۳. i = j - L + ۱;
  ۴. if (k == i) return A[j]
  elseif (k < i) return select(A[L..j-۱], k)
  else return select(A[j+۱..U], k - i)
}
```

```
partition(A[L..U])
{
  pivot = A[L]; j = L;
  for (i = L + ۱ to U)
    if (A[i] < pivot)
    {
      j = j + ۱;
      swap(A[i], A[j]);
    }
  swap(A[L], A[j]);
  return j;
}
```

برای partition کدهای مختلفی می‌توان نوشت که یک نمونه آن را آورده‌ایم. پارتیشن برای آرایه  $n$  عنصری از مرتبه  $\theta(n)$  است، زیرا عمل اصلی در پارتیشن، مقایسه  $(A[i] < pivot)$  است که دقیقاً  $n - ۱$  بار انجام می‌شود. در مثال ۲ نحوه کار الگوریتم پارتیشن را نشان داده‌ایم. الگوریتم select در خط ۲، پارتیشن را صدا می‌زند و نقطه افراز (محل قرارگیری محور بعد از افراز) را  $j$  می‌نامد، با توجه به اینکه آرایه از اندیس  $L$  شروع شده است و از  $L$  تا  $j$  تعداد  $j - L + ۱$  عنصر است پس در خط ۳،  $i$  را برابر  $j - L + ۱$  قرار داده‌ایم. به این معنی که محور اکنون در خانه  $i$ ام قرار گرفته است. در خط ۴ نیز بررسی می‌کنیم که آیا  $k$  و  $i$  مساوی هستند یا خیر و اگر خیر، سمت چپ یا راست آن را فراخوانی بازگشتی می‌کنیم.

برای تحلیل زمان اجرای الگوریتم select، اگر با همان فراخوانی اول پارتیشن،  $k$  و  $i$  با هم مساوی شوند، الگوریتم خاتمه می‌یابد و از مرتبه  $\theta(n)$  می‌شود (بخاطر پارتیشن)، ولی اگر  $k$  و  $i$  مساوی نشوند، زمان اجرا به این بستگی دارد که فراخوانی بازگشتی روی کدام قسمت آرایه انجام می‌شود و با چند عنصر صورت می‌گیرد. مثلاً اگر هر بار بعد از افراز، محور، وسط آرایه قرار گیرد

آنگاه برای زمان اجرا می‌توان رابطه  $T(n) = T(\frac{n}{4}) + \theta(n)$  را نوشت که از مرتبه  $\theta(n)$  است. و یا اگر هر بار بعد از افراز محور در مکانی قرار گیرد که  $\frac{n}{4}$  عنصر سمت چپ آن و  $\frac{9n}{10}$  عنصر سمت راست آن قرار گیرند آنگاه می‌توان برای زمان اجرا رابطه  $T(n) = T(\frac{9n}{10}) + \theta(n)$  یا  $T(n) = T(\frac{n}{10}) + \theta(n)$  را نوشت که در هر دو صورت از مرتبه  $\theta(n)$  است. ولی حالتهایی وجود دارد که زمان اجرای الگوریتم select از مرتبه  $\theta(n^2)$  می‌شود. مثلاً فرض کنید آرایه صعودی است و بدنبال ماکزیمم ( $n$  امین مینیمم،  $k = n$ ) هستیم در این صورت زمان اجرا  $T(n) = T(n-1) + \theta(n)$  است که از مرتبه  $\theta(n^2)$  است.

**مثال ۲:** الگوریتم partition را روی آرایه  $A$  اجرا کنید یعنی به صورت  $\text{partition}(A[1..10])$  فراخوانی کنید.

|   |    |   |    |   |    |    |    |    |    |    |
|---|----|---|----|---|----|----|----|----|----|----|
|   | ۱  | ۲ | ۳  | ۴ | ۵  | ۶  | ۷  | ۸  | ۹  | ۱۰ |
| A | ۲۶ | ۵ | ۳۷ | ۱ | ۶۱ | ۱۱ | ۵۹ | ۱۵ | ۴۸ | ۱۹ |

**پاسخ:** در الگوریتم partition، عنصر اول به عنوان محور (pivot) انتخاب می‌شود سپس  $j$  را اندیس شروع قرار می‌دهد و  $i$  را از اندیس دوم تا آخر حرکت می‌دهد و  $i$  روی عناصر کوچکتر از محور توقف می‌کند، به محض توقف  $i$ ، مقدار  $j$  یک واحد زیاد شده و  $A[i]$  با  $A[j]$  تعویض می‌شود. پس از پایان حلقه for، محور ( $A[L]$ ) با  $A[j]$  تعویض می‌شود و در نهایت  $j$  به عنوان نقطه افراز اعلام می‌شود، پس  $\text{pivot} = ۲۶$  است و  $i$  روی عناصر ۵ و ۱ و ۱۱ و ۱۵ و ۱۹ توقف می‌کند یعنی وقتی مقدار  $i$  برابر ۲ و ۴ و ۶ و ۸ و ۱۰ می‌شود، مقدار  $j$  یک واحد زیاد می‌شود و تعویض  $A[i]$  و  $A[j]$  صورت می‌گیرد، پس به ترتیب  $\text{Swap}(A[۲], A[۲])$  و  $\text{Swap}(A[۴], A[۳])$  و  $\text{Swap}(A[۶], A[۴])$  و  $\text{Swap}(A[۸], A[۵])$  و  $\text{Swap}(A[۱۰], A[۶])$  انجام می‌شود. پس از پایان حلقه for نیز  $\text{Swap}(A[۱], A[۶])$  انجام می‌شود و در نهایت  $j = ۶$  به عنوان خروجی اعلام می‌شود. و شکل آرایه می‌شود

|    |   |   |    |    |      |    |    |    |    |
|----|---|---|----|----|------|----|----|----|----|
| ۱۹ | ۵ | ۱ | ۱۱ | ۱۵ | (۲۶) | ۵۹ | ۶۱ | ۴۸ | ۳۷ |
|----|---|---|----|----|------|----|----|----|----|

عناصر بیشتر از ۲۶ بعد از ۲۶ قرار گرفته‌اند. ■

همانطور که دیدیم الگوریتم select عنصر کمینه  $k$ ام را در حالت متوسط با مرتبه  $\theta(n)$  یافت. ولی بدترین حالت این الگوریتم از مرتبه  $\theta(n^2)$  است. می‌خواهیم الگوریتمی برای یافتن  $k$  امین مینیمم در آرایه  $n$  عنصری  $A$  ارائه دهیم که همواره از مرتبه  $\theta(n)$  باشد. نام این الگوریتم را sel می‌گذاریم و مراحل این الگوریتم عبارت است از:

- ۱- آرایه را به  $\lceil \frac{n}{5} \rceil$  گروه ۵ عنصری تقسیم کنید و در هر گروه میانه را بیابید. یعنی در هر گروه ۵ عنصری، سومین کوچکترین را بیابید. زمان اجرای این مرحله  $\theta(n)$  است.
  - ۲- با فراخوانی بازگشتی، میانه میانه‌های مرحله قبل را بیابید یعنی روی  $\frac{n}{5}$  میانه‌ای که یافته‌اید، فراخوانی بازگشتی کنید و میانه آنها را بیابید و اسم میانه میانه‌ها را  $x$  بگذارید. زمان این مرحله  $T(\frac{n}{5})$  است (با فرض اینکه زمان اجرای کل الگوریتم  $sel$ ،  $T(n)$  باشد)
  - ۳- با محوریت  $x$ ، آرایه  $A$  را پارتیشن کنید در این صورت عناصر کوچکتر از  $x$ ، قبل از  $x$  و عناصر بزرگتر از  $x$  بعد از  $x$  قرار می‌گیرند. زمان این مرحله  $\theta(n)$  است.
  - ۴- اگر  $x$  در مکان  $k$ ام آرایه قرار گرفته است که  $x$  عنصر کمینه  $k$ ام است و کار تمام است. در غیر این صورت سمت چپ یا سمت راست  $x$  را فراخوانی بازگشتی کنید. بررسی خواهیم کرد که زمان این مرحله در بدترین حالت  $T(\frac{7n}{10})$  است.
- می‌توان کد این الگوریتم را به شکل مقابل نوشت:

```

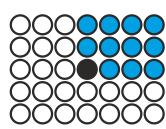
sel(A[L..u], k)
{
  n = u - L + 1;
  if (n ≤ 25) use brute force!
  else{
    m = ⌈ n/5 ⌉; B[1..m]
    (۱) for(i = 1 to m)
          B[i] = sel(A[Δi - 4...Δi], 3);
    (۲) x = sel(B[1..m], ⌈ m/3 ⌉)
    (۳) j = partition(A[L..U], x)
          i = j - L + 1
    (۴) if(k == i) return A[j]
          elseif(k < i) return sel(A[L..j-1], k)
          else return sel(A[j+1..U], k-i)
  }
}

```

این نوع الگوریتم‌ها برای  $n$ های کوچک مناسب نیستند. در این کد به ازای  $n \leq 25$  گفته شده از روش brute force استفاده شود. به این معنی که برای  $n \leq 25$  می‌توان از هر روشی استفاده کرد، مثلاً می‌توان مرتب کرد و جواب را یافت که از مرتبه  $\theta(1)$  است زیرا تعداد عناصر از ۲۵ کمتر یا مساوی است. ۴ مرحله الگوریتم با شماره‌های ۱ تا ۴ مشخص شده‌اند. در مرحله ۱ در حلقه for وقتی  $i = 1$ ، فراخوانی  $sel(A[1..5], 3)$  میانه ۵ عنصر اول  $A$  را می‌یابد و در  $B[1]$  ذخیره می‌کند، وقتی  $i = 2$ ، فراخوانی  $sel(A[6..10], 3)$  میانه ۵

عناصر دوم  $A$  را می‌یابد و در  $B[2]$  ذخیره می‌کند و به همین ترتیب میانه هر ۵ عنصر  $A$  پیدا شده و در  $B$  ذخیره می‌شود. مرحله ۲ الگوریتم میانه میانه‌ها را که در  $B$  هست را می‌یابد و در  $x$  ذخیره می‌کند و مرحله ۳ الگوریتم، پارتیشن را اجرا می‌کند با این تفاوت که محور عنصر اول  $A$  نیست بلکه  $x$  محور است. مرحله ۴ نیز بررسی می‌کند که  $x$  در کدام مکان  $A$  قرار گرفته است. زمان اجرای مرحله ۴ در بدترین حالت  $T(\frac{7n}{10})$  است زیرا ما آرایه  $A$  را به ۵ گروه ۵ عنصری تقسیم کردیم و در هر گروه میانه یافتیم که میانه هر گروه از ۲ عنصر در آن گروه کوچکتر است.

سپس میانه میانه‌ها را یافتیم (x) که x از نصف میانه‌ها یعنی از  $\frac{1}{2} \cdot \frac{n}{5} = \frac{n}{10}$  میانه کوچکتر است.



گروه ۵ عنصری

و چون هر میانه در گروه خودش از ۲ عنصر کوچکتر است پس x حداقل از

$\frac{3n}{10}$  عنصر کوچکتر است، بنابراین x حداکثر از  $\frac{7n}{10} = n - \frac{3n}{10}$  عنصر

بزرگتر است یعنی بعد از افزایش حداکثر  $\frac{7n}{10}$  عنصر سمت چپ x قرار می‌گیرند

پس مرحله ۴ در بدترین حالت روی این  $\frac{7n}{10}$  عنصر فراخوانی می‌کند. که

زمان آن  $T(\frac{7n}{10})$  می‌شود.

بنابراین زمان اجرای الگوریتم sel در بدترین حالت  $T(n) = T(\frac{n}{5}) + T(\frac{7n}{10}) + \theta(n)$  است

که از مرتبه  $\theta(n)$  است.

**مثال ۳:** در الگوریتم sel فرض کنید بجای آنکه آرایه را به  $\frac{n}{5}$  گروه ۵ عنصری تقسیم کنیم، آن را

به  $\frac{n}{3}$  گروه ۳ عنصری تقسیم کنیم و بقیه مراحل را تغییر ندهیم، در این صورت زمان اجرا را بیابید.

**پاسخ:** مرحله ۱ از مرتبه  $\theta(n)$ ، زمان مرحله ۲ برابر  $T(\frac{n}{3})$  و مرحله ۳ از مرتبه  $\theta(n)$  است.

زمان مرحله ۴ حداکثر  $T(\frac{2n}{3})$  است. زیرا  $\frac{n}{3}$  گروه وجود دارد که نصف این گروه‌ها  $\frac{n}{6}$  است و

در هر گروه حداقل ۲ عنصر هستند که x (میانه میانه‌ها) از آنها کوچکتر است پس x حداقل از

$\frac{2n}{6} = \frac{n}{3}$  عنصر کوچکتر است پس حداکثر از  $\frac{2n}{3} = n - \frac{n}{3}$  عنصر بزرگتر است. بنابراین زمان اجرا

در بدترین حالت  $T(n) = T(\frac{n}{3}) + T(\frac{2n}{3}) + \theta(n)$  است که از مرتبه  $\theta(n \lg n)$  است.

**توجه:** در برخی منابع، الگوریتم sel را دقیق‌تر و ریزتر تحلیل می‌کنند. به این صورت که مرحله ۱ و

۳ از مرتبه  $\theta(n)$  هستند و زمان مرحله ۲ برابر  $T(\lceil \frac{n}{5} \rceil)$  است و زمان مرحله ۴ در بدترین حالت

برابر  $T(\frac{7n}{10} + 6)$  است زیرا  $\lceil \frac{n}{5} \rceil$  میانه در مرحله ۱ پیدا شده است که نصف این میانه‌ها از x

بزرگتر است و در هر گروه ۲ عنصر از میانه آن گروه بزرگتر هستند (بجز احتمالاً گروه آخر چون

ممکن است کمتر از ۵ عنصر داشته باشد) پس در نصف گروه‌ها یعنی در  $\lceil \frac{1}{2} \lceil \frac{n}{5} \rceil \rceil$  گروه بجز دو

گروه (گروه آخر و گروهی که x عضو آن است) ۳ عنصر هستند که از x بزرگتر هستند پس x

حداقل از  $\frac{3n}{10} - 6 = \frac{3}{10}(\frac{n}{5} - 2) \geq 3(\frac{1}{5} \lceil \frac{n}{5} \rceil - 2)$  عنصر کوچکتر است پس x حداکثر از

$n - (\frac{3n}{10} - 6) = \frac{7n}{10} + 6$  عنصر بزرگتر است. بنابراین زمان اجرای sel در بدترین حالت

$T(n) = T(\lceil \frac{n}{5} \rceil) + T(\frac{7n}{10} + 6) + \theta(n)$  است که از مرتبه  $\theta(n)$  است.



## مرتب‌سازی

می‌خواهیم آرایه  $A[1..n]$  را به صورت صعودی مرتب کنیم. بدیهی است که روش‌های مرتب‌سازی می‌توانند به صورت نزولی هم مرتب کنند. روش مرتب‌سازی یا متعادل (پایدار، stable) است یا نا متعادل. متعادل یعنی ترتیب عناصر مساوی حفظ می‌شود. نا متعادل یعنی ممکن است ترتیب عناصر مساوی حفظ نشود. مثلاً نتیجه مرتب‌سازی آرایه  $A \begin{bmatrix} 3 & 4 & 3 & 2 \end{bmatrix}$  به صورت صعودی  $A \begin{bmatrix} 2 & 3 & 3 & 4 \end{bmatrix}$  است. ولی برخی روش‌ها ممکن است ترتیب دو تا ۳ را حفظ نکنند یعنی دومین ۳ با اولین ۳ تعویض شود یعنی اگر دومین ۳ را ۳' بنامیم ممکن است خروجی  $A \begin{bmatrix} 2 & 3' & 3 & 4 \end{bmatrix}$  باشد که به این روش‌ها نامتعادل گویند. روش‌های مرتب‌سازی یا درجا (inplace) هستند یا غیر درجا (outplace). در روش‌های درجا، حافظه کمکی مستقل از  $n$  است و ثابت است. ولی در روش‌های غیر درجا، حافظه کمکی به  $n$  وابسته است مثلاً نیاز به آرایه کمکی دارند.

روش‌های معمولی مثل selection sort (انتخابی)، bubble sort (حبابی)، و insertion sort (درجی)، در حالت متوسط و بدترین حالت از مرتبه  $\theta(n^2)$  هستند. در بهترین حالت روش‌های درجی و حبابی (بهینه شده) از مرتبه  $\theta(n)$  هستند که بهترین حالت وقتی است که آرایه از قبل صعودی مرتب باشد. این سه روش درجا هستند. حبابی و درجی متعادل هستند ولی انتخابی نامتعادل است. البته می‌توان روش‌های نامتعادل را بدون آنکه مرتبه آنها تغییر کند، متعادل کرد. برای بررسی این روش‌ها به کتاب ساختمان داده رجوع کنید. ما در این بخش به بررسی سایر روش‌های مرتب‌سازی می‌پردازیم.

## مرتب‌سازی ادغامی (mergesort)

```
mergesort(A[L..U])
{
  if(L < U)
  {
    mid =  $\lfloor \frac{L+U}{2} \rfloor$ 
    ۱. mergesort(A[L..mid])
    ۲. mergesort(A[mid+1..U])
    ۳. merge(A[L..mid], A[mid+1..U])
  }
}
```

این روش با شیوه تقسیم و غلبه، آرایه  $n$  عنصری را به دو نیمه تقسیم می‌کند و سپس هر نیمه را به صورت بازگشتی مرتب می‌کند و در نهایت دو نیمه مرتب شده را با هم ادغام (merge) می‌کند. الگوریتم ادغام، دو لیست مرتب  $n$  و  $m$  عنصری دریافت می‌کند و یک لیست مرتب  $m+n$  عنصری تولید می‌کند. می‌توان الگوریتم مرتب‌سازی ادغامی برای مرتب‌سازی آرایه  $A[L..U]$  را به شکل روبرو نوشت:

برای مرتب‌سازی آرایه  $A[1..n]$  باید این الگوریتم به صورت  $\text{mergesort}(A[1..n])$  فراخوانی شود و اگر زمان اجرای این الگوریتم  $T(n)$  فرض شود آنگاه خط شماره ۱ زمان  $T(\lfloor \frac{n}{2} \rfloor)$  و خط شماره ۲ زمان  $T(\lceil \frac{n}{2} \rceil)$  و خط شماره ۳ زمان  $\theta(n)$  صرف می‌کند (بررسی خواهیم کرد) پس زمان اجرای مرتب‌سازی ادغامی برابر  $T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + \theta(n)$  است که از مرتبه  $\theta(n \lg n)$  می‌باشد.

همانطور که گفته شد، الگوریتم merge دو لیست مرتب دریافت می‌کند و با هم ادغام می‌کند. نحوه ادغام به این صورت است که عناصر سر دو لیست با هم مقایسه می‌شوند و عنصر کوچکتر انتخاب می‌شود و این عمل تا زمانی انجام می‌شود که حداقل یکی از دو لیست خالی شود که آنگاه عناصر باقیمانده لیست دیگر به آرایه خروجی کپی می‌شوند. مثلاً ادغام دو لیست مرتب

$D[1]$  و  $E[1]$  را در نظر بگیرید. ابتدا  $D[1]$  و  $E[1]$  مقایسه می‌شود و به

|    |    |    |    |
|----|----|----|----|
| ۱  | ۲  | ۳  | ۴  |
| ۱۵ | ۲۵ | ۳۵ | ۴۵ |

|    |    |    |    |
|----|----|----|----|
| ۱  | ۲  | ۳  | ۴  |
| ۱۰ | ۲۰ | ۳۰ | ۴۰ |

همین ترتیب پس از انجام ۷ مقایسه لیست

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| ۱  | ۲  | ۳  | ۴  | ۵  | ۶  | ۷  | ۸  |
| ۱۰ | ۱۵ | ۲۰ | ۲۵ | ۳۰ | ۳۵ | ۴۰ | ۴۵ |

حاصل F

می‌شود. و یا ادغام دو لیست مرتب  $G[1]$  و  $H[1]$  را در نظر بگیرید.

|   |   |   |
|---|---|---|
| ۱ | ۲ | ۳ |
| ۶ | ۸ | ۹ |

|    |    |    |    |
|----|----|----|----|
| ۱  | ۲  | ۳  | ۴  |
| ۱۰ | ۲۰ | ۳۰ | ۴۰ |

ابتدا  $G[1]$  و  $H[1]$  مقایسه می‌شوند و  $H[1]$  انتخاب می‌شود. سپس  $G[2]$  و  $H[2]$  مقایسه می‌شوند و  $H[2]$  انتخاب می‌شود. و سپس  $G[3]$  و  $H[3]$  مقایسه می‌شوند و  $G[3]$  انتخاب می‌شود و چون لیست  $H$  خالی می‌شود، لیست  $G$  کپی می‌شود و لیست مرتب

حاصل می‌شود. بنابراین برای ادغام دو لیست مرتب  $m$  و  $n$   $I$

|   |   |   |    |    |    |    |
|---|---|---|----|----|----|----|
| ۱ | ۲ | ۳ | ۴  | ۵  | ۶  | ۷  |
| ۶ | ۸ | ۹ | ۱۰ | ۲۰ | ۳۰ | ۴۰ |

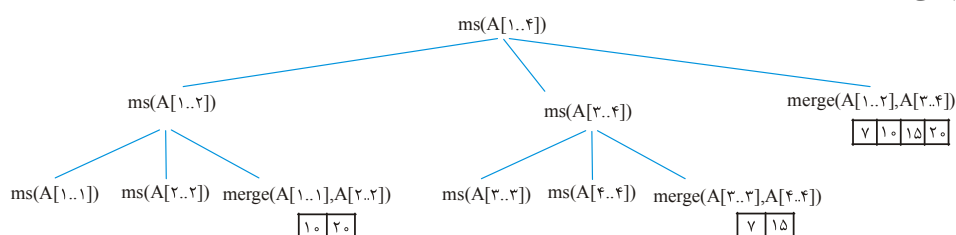
عنصری حداقل  $\min\{m, n\}$  و حداکثر  $m + n - 1$  مقایسه انجام می‌شود. در نتیجه برای ادغام دو

لیست  $\lfloor \frac{n}{2} \rfloor$  و  $\lceil \frac{n}{2} \rceil$  عنصری حداقل  $\lfloor \frac{n}{2} \rfloor$  مقایسه و حداکثر  $n - 1$  مقایسه انجام می‌شود پس

خط ۳ یعنی merge در الگوریتم mergesort از مرتبه  $\theta(n)$  است.

**مثال ۴:** الگوریتم mergesort را روی  $A = \begin{bmatrix} 20 & 10 & 7 & 15 \end{bmatrix}$  اجرا کنید.

**پاسخ:** نام mergesort را  $ms$  می‌گذاریم. فراخوانی  $ms(A[1..4])$  را انجام می‌دهیم طبق کد اگر  $L < U$  ( $4 < 1$ )، دو بار  $ms$  و یک بار  $merge$  فراخوانی می‌شوند. درخت فراخوانیها به شکل زیر است:



پس از پایان فراخوانیهای  $ms$  (پایان فراخوانی وقتی است که  $L = U$ )،  $merge$ ها از چپ به راست اجرا می‌شوند. ابتدا  $merge(A[1..1], A[2..2])$  عنصر  $A[1]$  و  $A[2]$  را ادغام می‌کند و

$A = \begin{bmatrix} 10 & 20 \end{bmatrix}$  تولید می‌کند. سپس  $merge(A[3..3], A[4..4])$  عنصر  $A[3]$  و  $A[4]$  را ادغام

می‌کند و  $A = \begin{bmatrix} 7 & 15 \end{bmatrix}$  را تولید می‌کند و سپس  $merge(A[1..2], A[3..4])$  دو لیست مرتب

$\begin{bmatrix} 10 & 20 \end{bmatrix}$  و  $\begin{bmatrix} 7 & 15 \end{bmatrix}$  را ادغام می‌کند و  $A = \begin{bmatrix} 7 & 10 & 15 & 20 \end{bmatrix}$  را تولید می‌کند.

**مثال ۵:** برای آرایه ۱۰ عنصری در الگوریتم mergesort.

(الف) چند بار فراخوانی انجام می‌شود.

(ب) چند بار تابع  $merge$  فراخوانی می‌شود.

(ج) حداکثر چند مقایسه بین عناصر آرایه صورت می‌گیرد.

**پاسخ: الف)** اگر  $T(n)$  تعداد فراخوانیها برای آرایه  $n$  عنصری باشد آنگاه

$$T(1) = 1 \text{ و } T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + 1 \text{ (برای آرایه یک عنصری، یک فراخوانی صورت می‌گیرد. باید } T(1) \text{ را بیابیم:)}$$

$$T(2) = 2T(1) + 1 = 3, \quad T(3) = T(1) + T(2) + 1 = 5, \quad T(5) = T(2) + T(3) + 1 = 9$$

$$T(10) = 2T(5) + 1 = 19$$

(ب) اگر  $T(n)$  تعداد فراخوانی تابع  $merge$  در الگوریتم mergesort باشد آنگاه به ازای  $n > 1$

$$\text{می‌توان نوشت } T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + 1 \text{ (برای آرایه یک عنصری } T(1) = 0 \text{ و)}$$

$$T(10) = 9 \text{ merge انجام نمی‌شود) حال محاسبه کنید که } T(10) = 9.$$

ج) مقایسه در تابع merge انجام می‌شود و حداکثر  $n-1$  مقایسه انجام می‌شود (برای ادغام دو لیست  $\lfloor \frac{n}{2} \rfloor$  و  $\lceil \frac{n}{2} \rceil$  عنصری) پس اگر  $T(n)$  حداکثر تعداد مقایسه‌ها باشد می‌توان نوشت

$$T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + n - 1$$

و  $T(1) = 0$  باید  $T(10)$  را محاسبه کنیم:

$$T(2) = 2T(1) + 1 = 1, \quad T(3) = T(1) + T(2) + 2 = 3, \quad T(5) = T(2) + T(3) + 4 = 8, \\ T(10) = 2T(5) + 9 = 25$$

**تست ۱:** آرایه  $n$  عنصری  $A$  را در نظر بگیرید، فرض کنید  $n = 2^k$ . الگوریتم Mergesort بر روی  $A$  (ساختمان داده‌ها- دولتی ۸۲)

(۱) در بدترین حالت  $n \log_2 n - n + 1$  مقایسه میان عناصر آرایه انجام می‌دهد.

(۲) در بدترین حالت  $n \log_2 n + n - 1$  مقایسه میان عناصر آرایه انجام می‌دهد.

(۳) در بهترین حالت  $n \log_2 n - 2n - 1$  مقایسه میان عناصر آرایه انجام می‌دهد.

(۴) در بهترین حالت  $n \log_2 n - n - 1$  مقایسه میان عناصر آرایه انجام می‌دهد.

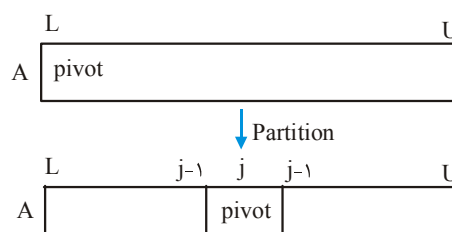
**پاسخ:** در بدترین حالت تعداد مقایسه‌های انجام شده در الگوریتم mergesort آرایه  $n$  عنصری، همانطور که دیدیم از رابطه  $T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + n - 1$  و  $T(1) = 0$  بدست می‌آید که با فرض  $n = 2^k$  می‌توان نوشت  $T(n) = 2T(\frac{n}{2}) + n - 1$ . این رابطه را در فصل ۳ مثال ۳ حل کردیم که حل آن  $T(n) = n \lg n - n + 1$  یعنی گزینه ۱ است. ■

لازم به ذکر است که مرتب‌ساز ادغامی، متعادل است ولی غیر درجاست چون در تابع merge نیاز به آرایه کمکی است (به عنوان تمرین تابع merge را پیاده‌سازی کنید).

### مرتب‌سازی سریع (quicksort)

در این روش، با استفاده از الگوریتم partition آرایه دو قسمت می‌شود، قسمت اول عناصر کوچکتر (یا مساوی) محور و قسمت دوم عناصر بزرگتر (یا مساوی) محور و سپس به صورت بازگشتی برای هر یک از دو قسمت، فراخوانی صورت می‌گیرد تا وقتی که هر قسمت یک عنصری شود. در واقع این روش نیز با شیوه تقسیم و غلبه مرتب‌سازی می‌کند:

```
quicksort(A[L..U])
{
  if(L < U)
  {
    ۱. j = partition(A[L..U])
    ۲. quicksort(A[L..j-1])
    ۳. quicksort(A[j+1..U])
  }
}
```



اگر  $T(n)$  زمان اجرای مرتب‌سازی سریع برای آرایه  $n$  عنصری باشد، و اگر فرض کنیم بعد از انجام پارتیشن، تعداد  $k$  عنصر سمت چپ محور قرار می‌گیرند می‌توان برای  $T(n)$  فرمول بازگشتی  $T(n) = T(k) + T(n-k-1) + n-1$  نوشت که  $n-1$  زمان الگوریتم partition است (خط ۱ الگوریتم) و  $T(k)$  و  $T(n-k-1)$  به ترتیب زمان اجرای خط ۲ و خط ۳ الگوریتم است. در بدترین حالت، هر بار که پارتیشن انجام می‌شود، یک سمت محور خالی می‌ماند و مثلاً  $k=0$  و می‌توان نوشت  $T(n) = T(n-1) + n-1$  که با شرط اولیه  $T(1)=0$  داریم  $T(n) = \frac{n(n-1)}{2}$ . پس در بدترین حالت مرتبه مرتب‌سازی سریع  $\theta(n^2)$  است. در بهترین حالت، هر بار که پارتیشن انجام می‌شود، آرایه نصف می‌شود و pivot وسط آرایه قرار می‌گیرد و می‌توان نوشت  $T(n) = 2T(\frac{n}{2}) + n-1 = \theta(n \lg n)$ .

برای بررسی حالت متوسط باید همه حالات را بررسی کنیم و میانگین بگیریم:

**حالت ۱:** بعد از پارتیشن هیچ عنصری قبل محور قرار نگیرد:

$$T(n) = T(0) + T(n-1) + n-1$$

**حالت ۲:** بعد از پارتیشن یک عنصر قبل از محور قرار گیرد:

$$T(n) = T(1) + T(n-2) + n-1$$

**حالت ۳:** بعد از پارتیشن دو عنصر قبل از محور قرار گیرد:

$$T(n) = T(2) + T(n-3) + n-1$$

⋮

**حالت n:** بعد از پارتیشن  $n-1$  عنصر قبل از محور قرار گیرد:

$$T(n) = T(n-1) + T(0) + n-1$$

همه این  $n$  تا حالت را با هم جمع می‌کنیم:  $n.T(n) = \sum_{k=0}^{n-1} (T(k) + T(n-k-1)) + n(n-1)$  که

این تساوی را می‌توان به صورت  $n.T(n) = 2 \sum_{k=0}^{n-1} T(k) + n(n-1)$  نوشت. برای رسیدن به

رابطه‌ای برای  $T(n)$  در حالت متوسط که فاقد سیگما باشد، کافی است بجای  $n$  در رابطه اخیر

$n-1$  قرار دهیم و دو رابطه را از هم کم کنیم:

$$n.T(n) = 2 \sum_{k=0}^{n-1} T(k) + n(n-1) \xrightarrow[\text{دهید } n-1]{\text{بجای } n \text{ قرار}} (n-1)T(n-1) = 2 \sum_{k=0}^{n-2} T(k) + (n-1)(n-2)$$

حال این دو رابطه را از هم کم کنیم:

$$n.T(n) - (n-1).T(n-1) = 2T(n-1) + 2(n-1)$$

$$\rightarrow n.T(n) = (n+1).T(n-1) + 2(n-1) \rightarrow T(n) = \frac{n+1}{n} T(n-1) + 2 \frac{(n-1)}{n}$$

و این رابطه بازگشتی زمان اجرای مرتب‌سازی سریع در حالت متوسط است. برای حل این رابطه، طرفین را به  $n+1$  تقسیم می‌کنیم و تعریف می‌کنیم  $\frac{T(n)}{n+1} = a_n$  در نتیجه:

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2(n-1)}{n(n+1)} \rightarrow a_n = a_{n-1} + \frac{2(n-1)}{n(n+1)}$$

رابطه اخیر را می‌توان با جایگذاری حل کرد ولی ما بدنبال حل دقیق نیستیم و فقط مرتبه را می‌خواهیم. می‌توان  $a_n$  را تقریباً برابر  $a_n \approx a_{n-1} + \frac{2}{n+1}$  فرض کرد چون  $\frac{n-1}{n}$  کمی از ۱ کوچکتر است. یادمان هست سری همسازه  $1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} \approx \ln n$  پس  $a_n \approx 2 \ln n$  نتیجه با توجه به اینکه  $\frac{T(n)}{n+1} = a_n$  داریم  $T(n) = (n+1)a_n = 2(n+1) \ln n$  که در نتیجه  $T(n) \in \theta(n \lg n)$ . یعنی زمان اجرای مرتب‌سازی سریع در حالت متوسط از مرتبه  $\theta(n \lg n)$  است.

لازم به ذکر است که مرتب‌سازی سریع را می‌توان به گونه‌ای پیاده‌سازی کرد که همواره (حتی بدترین حالت) از مرتبه  $O(n \lg n)$  باشد، مثلاً می‌توان میانه را به عنوان محور انتخاب کرد و آرایه را با محوریت میانه افراز کرد و این کار را به صورت بازگشتی ادامه داد که در این صورت زمان اجرای مرتب‌سازی سریع همیشه  $T(n) = 2T(\frac{n}{2}) + \theta(n) = \theta(n \lg n)$  خواهد بود. همچنین مدلی از مرتب‌سازی سریع موسوم به randomized quicksort هست که هر بار محور را به صورت تصادفی انتخابی می‌کند که در این صورت احتمال آنکه مرتبه  $\theta(n^2)$  شود بسیار ضعیف می‌باشد.

### مرتب‌سازی هرمی (heapsort)

با این مرتب‌سازی در درس ساختمان داده‌ها آشنا شده‌اید و همچنین در فصل درخت‌ها می‌توانید این روش را مطالعه کنید. فقط یادآوری می‌کنیم این روش ابتدا با  $n$  عنصر داده شده maxheap می‌سازد و سپس  $n$  بار ( $n-1$  بار هم کافی است) عمل حذف ریشه را انجام می‌دهد. این روش در حالت متوسط و بدترین حالت از مرتبه  $O(n \lg n)$  است. اگر همه عناصر آرایه مساوی باشند، مرتبه این روش  $\theta(n)$  می‌شود. این روش درجاست و نامتعادل.

### مرتب‌سازی درختی (Treesort)

با این روش نیز آشنا هستید. این روش ابتدا با  $n$  عنصر داده شده BST می‌سازد و سپس آن را به صورت inorder پیمایش می‌کند. مرتبه این روش در حالت متوسط  $\theta(n \lg n)$  است و در

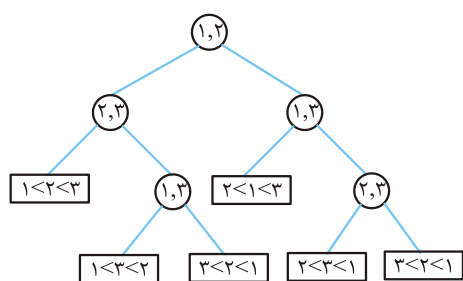
بدترین حالت  $\theta(n^2)$  است ولی می‌توان بجای AVL، BST بسازیم که در این صورت مرتبه این روش همواره  $O(n \lg n)$  می‌شود. این روش غیر درجا و نامتعادل است.

روش‌های مرتب‌سازی که تا کنون نام بردیم، مبتنی بر مقایسه هستند. یعنی عمل اصلی در این روشها مقایسه بین عناصر است. با کمک درخت تصمیم (decision tree) ثابت می‌شود که این روشها در حالت متوسط از مرتبه  $\Omega(n \lg n)$  هستند یعنی امکان ندارد روش مقایسه‌ای بتوان یافت که در حالت متوسط، بهتر از  $n \lg n$  باشد.

### درخت تصمیم

برای هر الگوریتمی که مقایسه بین عناصر انجام می‌دهد می‌توان درخت تصمیم رسم کرد. نودهای داخلی در این درخت، مقایسه بین دو عنصر را نشان می‌دهند. یال سمت چپ به این معنی است که عنصر اول از عنصر دوم کوچکتر (یا مساوی) است و یال سمت راست نشان می‌دهد که عنصر اول از عنصر دوم بزرگتر است. هر برگ این درخت یک نتیجه یا خروجی ممکن برای الگوریتم را نشان می‌دهد. اگر تعداد برگ‌های این درخت (یعنی تعداد خروجی‌های ممکن) برابر  $x$  باشد آنگاه ارتفاع حداقل برابر  $\lceil \lg x \rceil$  است.

**مثال ۶:** درخت تصمیم برای مرتب‌سازی ۳ عنصر  $A[1..3]$  را نشان دهید.



**پاسخ:** ابتدا عنصر  $A[1]$  را با  $A[2]$  مقایسه

می‌کنیم (در ریشه درخت ۱ و ۲ نوشته شده که به این معنی است) اگر  $A[1] < A[2]$  چپ و در غیر این صورت راست می‌رویم و به همین ترتیب شکل مقابل حاصل شود:

مرتب‌سازی  $n$  عنصر متمایز  $n!$  حالت دارد پس درخت تصمیم برای مرتب‌سازی دارای  $n!$

برگ است در نتیجه ارتفاع حداقل  $\lceil \lg n! \rceil$  است. ارتفاع درخت تصمیم در واقع حداقل تعداد مقایسه‌هایی را نشان می‌دهد که در حالت متوسط یا بدترین حالت باید انجام دهیم. پس مرتب‌سازی مقایسه‌ای برای مرتب کردن  $n$  عنصر در حالت متوسط یا بدترین حالت باید حداقل  $\lceil \lg n! \rceil$  مقایسه بین عناصر انجام دهد. نتیجه اینکه مرتب‌سازی مقایسه‌ای در حالت متوسط از مرتبه  $\Omega(n \lg n)$  است.

**مثال ۷:** تعداد برگ درخت تصمیم برای ادغام دو لیست مرتب  $L_1: x_1, x_2, x_3$  و

$L_2: y_1, y_2, y_3, y_4$  چند تاست؟

**پاسخ:** در واقع باید بررسی کنیم ادغام دو لیست مذکور چند حالت دارد.  $L_1$  و  $L_2$  جمعاً ۷ عنصر دارند که ۷! حالت ایجاد می‌کند ولی عناصر  $L_1$  نباید با هم تعویض شوند پس ۳! از کل جایگشت‌ها مجاز نیست. به همین ترتیب عناصر  $L_2$  نیز نباید با هم جابجا شوند. پس جواب

$$\frac{7!}{3!4!} = 35 \text{ است.}$$

### روش‌های مرتب‌سازی غیرمقایسه‌ای

عمل اصلی این روشها مقایسه نیست. معروف‌ترین این روشها عبارتند از: مرتب‌سازی شمارشی (counting sort)، مرتب‌سازی مبنایی (radix sort) و مرتب‌سازی سطلی (bucket sort) این روشها قادر نیستند هر نوع عنصری را مرتب کنند و هر کدام شرایط خاصی دارند ولی اگر شرایط این روشها برآورده شود، سرعتشان از روش‌های مقایسه‌ای معمولاً بیشتر است.

#### مرتب‌سازی شمارشی

این روش  $n$  عدد صحیح در یک بازه مشخص مثلاً  $[1..k]$  را مرتب می‌کند. این روش تعداد اعداد کوچکتر از هر عدد مثل  $x$  را می‌شمارد (بدون مقایسه) و بدین ترتیب مکان درست عدد  $x$  را می‌فهمد. مثلاً اگر تعداد ۷ عدد کمتر از ۱۵ وجود دارد یعنی ۱۵ باید در مکان هشتم قرار داده شود. الگوریتم زیر آرایه  $A[1..n]$  حاوی  $n$  عدد صحیح در بازه ۱ تا  $k$  را مرتب می‌کند:

$B$  آرایه خروجی است.  $C$  آرایه کمکی است که برای شمارش استفاده می‌شود، حلقه اول همه مقادیر  $C$  را صفر می‌کند. حلقه دوم می‌شمارد از هر کلید چند تا عنصر در  $A$  هست. پس از پایان حلقه دوم  $C[i]$  شامل تعداد آنها در  $A$  است.

```
countingsort(A[1..n], k)
{
    B[1..n], C[1..k];
    for(i = 1 to k) C[i] = 0;
    for(i = 1 to n) C[A[i]] ++;
    for(i = 2 to k) C[i] = C[i] + C[i - 1];
    for(i = n downto 1)
    {
        B[C[A[i]]] = A[i];
        C[A[i]] --;
    }
}
```

حلقه سوم می‌شمارد تا هر کلید چند تا عنصر در  $A$  هست. پس از پایان حلقه سوم، در  $C[i]$  تعداد اعداد کوچکتر یا مساوی  $i$  موجود در  $A$ ، ذخیره می‌شود. حلقه چهارم هر عنصر  $A$  را در مکان درستش در  $B$  (با کمک  $C$ ) قرار می‌دهد. تعداد  $C[A[i]]$  عنصر  $A$  هستند که از  $A[i]$  کوچکتر یا مساوی هستند پس باید  $A[i]$  را در خانه  $C[A[i]]$  از آرایه  $B$  قرار داد و بعد از این عمل  $C[A[i]]$  باید یک واحد کم شود.

مرتب‌سازی شمارشی از تعدادی حلقه که تو در تو نیستند تشکیل شده است که به  $n$  و  $k$  وابسته‌اند پس مرتبه مرتب‌سازی شمارشی  $\theta(n + k)$  است.



**مثال ۸:** آرایه  $A$  که  $n=6$  و  $k=4$  را با روش شمارشی مرتب کنید.

**پاسخ:** ابتدا  $C$  می‌شود. پس از پایان حلقه دوم  $C$  می‌شود.

$C[3]=2$  یعنی عدد ۳ دوبار در  $A$  تکرار شده است. پس از پایان حلقه سوم  $C$  می‌شود.  $C[3]=3$  یعنی ۳ تا عدد کمتر یا مساوی ۳ در  $A$  وجود دارد. پس از پایان حلقه چهارم  $B$  می‌شود.

### مرتب‌سازی مبنایی

این روش  $n$  عدد صحیح حداکثر  $d$  رقمی در مبنای  $r$  را مرتب می‌کند. این روش از رقم سمت راست شروع می‌کند و ابتدا اعداد را براساس رقم سمت راست (کم ارزش‌ترین رقم) با یک روش متعادل مرتب می‌کند. سپس نتیجه را براساس رقم دوم از سمت راست مرتب می‌کند و به همین ترتیب تا اینکه در مرحله آخر، اعداد براساس با ارزش‌ترین رقم، مرتب می‌شوند. به عنوان مثال می‌خواهیم  $n=6$  عدد حداکثر  $d=3$  رقمی در مبنای  $r=10$  را مرتب کنیم.

ابتدا بر اساس یکان به صورت متعادل مرتب می‌شوند:

| ۱   | ۲   | ۳   | ۴   | ۵   | ۶   |
|-----|-----|-----|-----|-----|-----|
| ۲۰۹ | ۱۵۸ | ۰۹۸ | ۰۲۹ | ۱۰۹ | ۱۲۳ |

سپس اعداد بر اساس دهگان با یک روش متعادل مرتب می‌شوند:

| ۱   | ۲   | ۳   | ۴   | ۵   | ۶   |
|-----|-----|-----|-----|-----|-----|
| ۱۲۳ | ۱۵۸ | ۰۹۸ | ۲۰۹ | ۰۲۹ | ۱۰۹ |

و در نهایت اعداد بر اساس صدگان با یک روش متعادل مرتب می‌شوند:

| ۱   | ۲   | ۳   | ۴   | ۵   | ۶   |
|-----|-----|-----|-----|-----|-----|
| ۰۲۹ | ۰۹۸ | ۱۰۹ | ۱۲۳ | ۱۵۸ | ۲۰۹ |

بهترین روش برای مرتب‌سازی هر رقم، مرتب‌سازی شمارشی است. پس مرتب‌سازی مبنایی از مرتبه  $\theta(d(n+r))$  است چون  $d$  بار مرتب‌سازی شمارشی را اجرا می‌کند و چون هر رقم در بازه  $[0..r-1]$  است پس مرتبه مرتب‌سازی هر رقم  $\theta(n+r)$  است.

**مثال ۹:** تعداد  $n$  عدد صحیح در بازه  $[0..n^4-1]$  را می‌خواهیم صعودی مرتب کنیم. مرتبه مرتب‌سازی چقدر است اگر

(الف) از روش مرتب‌سازی شمارشی استفاده شود.

(ب) از روش مبنایی با مبنای  $r=10$  استفاده شود.

(ج) از روش مبنایی با یک مبنای مناسب استفاده شود.

**پاسخ: الف)** ماکزیمم عدد  $n^4 - 1$  است پس مرتبه روش شمارشی  $\theta(n+k) = \theta(n+n^4) = \theta(n^4)$  است.

**ب)** اعداد در بازه گفته شده و مبنای ۱۰ حداکثر  $d = 4 \log_{10} n$  رقمی هستند پس مرتبه روش مبنایی  $\theta(d(n+r)) = \theta(n \lg n)$  است.

**ج)** اگر مبنای اعداد را  $r = n$  در نظر بگیرید آنگاه اعداد بازه گفته شده حداکثر  $d = 4$  رقمی خواهند بود و مرتبه مرتب‌سازی مبنایی  $\theta(d(n+r)) = \theta(4(n+n)) = \theta(n)$  می‌شود.

### مرتب‌سازی سطلی

این روش  $n$  عدد در یک بازه مشخص را مرتب می‌کند و وقتی عملکرد خوبی دارد که توزیع

```
bucketSort(A[۱..n])
{
  B[۰..n-۱];
  for(i=۱ to n)
    insert A[i] into list B[⌊n.A[i]⌋]
  for(i=۰ to n-۱)
    sort each B[i] with insertion sort;
  print B[۰], B[۱], ..., B[n-۱]
}
```

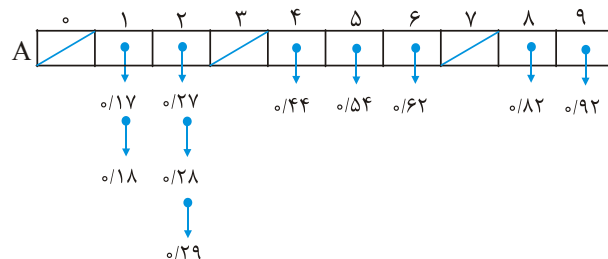
اعداد یکنواخت باشد. ما می‌خواهیم  $n$  عدد در بازه  $[۰, ۱]$  را مرتب کنیم. این روش  $n$  تا باکت (سطل) درست می‌کند. و داخل هر باکت عناصری که به هم نزدیک هستند را قرار می‌دهد و هر باکت را مرتب می‌کند: در کد مقابل  $B$  آرایه‌ای است از  $n$  تا باکت که هر باکت یک لیست پیوندی است.

**مثال ۱۰:** آرایه  $A$  را با

| ۱    | ۲    | ۳    | ۴    | ۵    | ۶    | ۷    | ۸    | ۹    | ۱۰   |
|------|------|------|------|------|------|------|------|------|------|
| ۰/۲۸ | ۰/۶۲ | ۰/۹۲ | ۰/۱۸ | ۰/۲۷ | ۰/۵۴ | ۰/۱۷ | ۰/۸۲ | ۰/۴۴ | ۰/۲۹ |

روش سطلی مرتب کنید.

**پاسخ:** در حلقه اول ابتدا  $i = 1$  و عنصر  $A[1]$  یعنی  $۰/۲۸$  به لیست  $B[2] = B[\lfloor 10 \times ۰/۲۸ \rfloor]$  وارد می‌شود. به همین ترتیب،  $۰/۶۲$  به لیست  $B[6]$  و  $۰/۹۲$  به لیست  $B[9]$  و .... و  $۰/۲۹$  به لیست  $B[2]$  وارد می‌شود و هر لیست با روش درجی مرتب می‌شود:



و در نهایت که  $B[i]$  ها چاپ شوند، اعداد مرتب می‌شوند. ■

علت اینکه در هر لیست  $B[i]$  از روش درجی استفاده می‌شود این است که تعداد عناصر هر لیست کم است و مرتب‌سازی درجی برای تعداد عناصر کم، عملکرد خیلی خوبی دارد.

مرتب‌سازی سطلی در بدترین حالت از مرتبه  $\theta(n^2)$  است چون ممکن است همه  $n$  عنصر به یک باکت وارد شوند که مرتبه الگوریتم همان مرتبه مرتب‌سازی درجی می‌شود. در بهترین حالت، در هر باکت فقط یک عنصر قرار می‌گیرد و مرتبه  $\theta(n)$  است. تحلیل حالت متوسط این روش کمی دشوار است ولی می‌توان نشان داد که حالت متوسط این روش مرتبه  $\theta(n)$  است.

## خلاصه فصل

- ۱- یافتن  $k$  امین مینیمم یا  $k$  امین ماکزیمم عنصر در آرایه  $n$  عنصری از مرتبه  $\theta(n)$  است.
- ۲- کمترین تعداد مقایسه برای یافتن مینیمم یا ماکزیمم در آرایه  $n$  عنصری برابر  $n-1$  است.
- ۳- کمترین تعداد مقایسه برای یافتن مینیمم و ماکزیمم (با هم) در آرایه  $n$  عنصری برابر  $\lceil \frac{3n}{4} \rceil - 2$  است که اگر  $n$  زوج باشد برابر  $\frac{3n}{4} - 2$  و اگر  $n$  فرد باشد برابر  $\frac{3n}{4} - \frac{3}{4}$  می‌باشد.
- ۴- کمترین تعداد مقایسه برای یافتن دومین مینیمم یا دومین ماکزیمم در یک آرایه  $n$  عنصری برابر  $n + \lceil \lg n \rceil - 2$  است.
- ۵- کمترین تعداد مقایسه برای یافتن  $k$  امین مینیمم یا  $k$  امین ماکزیمم در یک آرایه  $n$  عنصری برابر  $n + (k-1) \cdot \left\lceil \lg \frac{n}{k-1} \right\rceil - k$  است.
- ۶- روش‌های مرتب‌سازی انتخابی، حبابی و درجی برای مرتب‌سازی  $n$  عنصر در حالت متوسط و بدترین حالت از مرتبه  $\theta(n^2)$  هستند. روشهای حبابی و درجی در بهترین حالت از مرتبه  $\theta(n)$  هستند. انتخابی در بهترین حالت از مرتبه  $\theta(n^2)$  است. روشهای مرتب‌سازی ادغامی و سریع و هرمی و درختی در حالت متوسط از مرتبه  $\theta(n \lg n)$  هستند. روشهای سریع و درختی در بدترین حالت از مرتبه  $\theta(n^2)$  هستند ولی می‌توان طوری پیاده‌سازی‌شان کرد که مرتبه  $O(n \lg n)$  شود.
- ۷- اگر آرایه از قبل مرتب باشد یا تعداد عناصر آرایه کم باشد، بهترین روش مقایسه‌ای، درجی است.
- ۸- اگر همه عناصر آرایه مساوی باشند، روشهای حبابی و درجی و هرمی از مرتبه  $\theta(n)$  می‌شوند.
- ۹- تعداد مقایسه در روشهای حبابی و درجی و سریع حداکثر  $\frac{n(n-1)}{2}$  است.
- ۱۰- تعداد مقایسه در روش انتخابی همیشه  $\frac{n(n-1)}{2}$  است.
- ۱۱- تعداد جابجایی (تعویض) در روش‌های درجی و حبابی حداکثر  $\frac{n(n-1)}{2}$  است.
- ۱۲- تعداد تعویض در روش انتخابی برابر  $n-1$  است.
- ۱۳- روشهای انتخابی، حبابی، درجی، ادغامی، سریع، هرمی، و درختی، مبتنی بر مقایسه هستند. و روشهای مبتنی بر مقایسه، در حالت متوسط از مرتبه  $\Omega(n \lg n)$  هستند.

- ۱۴- روشهای انتخابی و سریع و هرمی و درختی نامتعادل و سایر روشها متعادل هستند.
- ۱۵- روشهای انتخابی و حبابی و درجی و سریع و هرمی درجا و سایر روشها غیر درجا هستند.
- ۱۶- روشهای شمارشی، مبنایی و سطلی، غیر مقایسه‌ای هستند. غیر درجا و متعادل می‌باشند.
- ۱۷- روش شمارشی  $n$  عدد صحیح که ماکزیمم آنها  $k$  است را با مرتبه  $\theta(n + k)$  مرتب می‌کند.
- ۱۸- روش مبنایی  $n$  عدد صحیح حداکثر  $d$  رقمی مبنای  $r$  را با مرتبه  $\theta(d(n + r))$  مرتب می‌کند.
- ۱۹- روش سطلی  $n$  عدد در بازه  $[0, 1)$  را با مرتبه میانگین  $\theta(n)$  مرتب می‌کند.

## تمرین

۱- می‌توان مرتب‌سازی درجی را به صورت بازگشتی نوشت. به منظور مرتب‌سازی  $A[1..n]$ ، به صورت بازگشتی، آرایه  $A[1..n-1]$  مرتب می‌شود و سپس  $A[n]$  درج می‌شود. یک رابطه بازگشتی برای زمان اجرای این نوع مرتب‌سازی درجی بنویسید.

۲- اگر چه مرتب‌سازی ادغامی در بدترین حالت  $\theta(n \log n)$  و درجی  $\theta(n^2)$  می‌باشد، ولی برای مقادیر کوچک  $n$ ، درجی از ادغامی سریعتر است.

می‌توان در مرتب‌سازی ادغامی وقتی تقسیم به اندازه کافی انجام شد، برای هر زیر لیست از مرتب‌سازی درجی استفاده کرد. فرض کنید در مرتب‌سازی ادغامی،  $\frac{n}{k}$  زیر لیست به طول  $k$  توسط درجی مرتب شوند. و سپس با هم ادغام شوند:

**الف)** نشان دهید همه  $\frac{n}{k}$  زیر لیست به طول  $k$ ، توسط مرتب‌سازی درجی با مرتبه  $\theta(nk)$  در بدترین حالت مرتب می‌شوند.

**ب)** نشان دهید، زیر لیست‌ها در بدترین حالت با مرتبه  $\theta\left(n \lg\left(\frac{n}{k}\right)\right)$  ادغام می‌شوند.

**ج)** با فرض اینکه این نوع مرتب‌سازی ادغامی از مرتبه  $\theta\left(nk + n \lg\left(\frac{n}{k}\right)\right)$  می‌باشد،  $k$  حداکثر برحسب  $n$  از چه مرتبه‌ای باشد ( $k = \theta(f(n))$ ) که مرتب‌سازی ادغامی توصیف شده، با مرتب‌سازی ادغامی استاندارد، مرتبه یکسان داشته باشند.

**د)** در عمل  $k$  چگونه تعیین می‌شود؟

۳- فرض کنید  $A[1..n]$  آرایه‌ای از  $n$  عدد متمایز باشد. اگر  $i < j$  و  $A[i] > A[j]$  در این صورت  $(i, j)$  را یک وارونگی (inversion) در  $A$  گویند.

**الف)** ۵ وارونگی در لیست  $\langle 2, 3, 8, 6, 1 \rangle$  را نام ببرید.

**ب)** چه نوع آرایه‌ای با اعداد  $\{1, 2, \dots, n\}$  بیشترین وارونگی را دارد و چند تا وارونگی دارد؟

**ج)** چه رابطه‌ای بین زمان اجرای مرتب‌سازی درجی و تعداد وارونگی‌های آرایه ورودی هست؟

**د)** الگوریتمی از مرتبه  $\theta(n \log n)$  ارائه دهید که تعداد وارونگی‌های آرایه  $n$  عنصری را مشخص کند (مرتب‌سازی ادغامی را تغییر دهید)

۴- یک آرایه  $n$  عنصری  $A$  مفروض است. می‌خواهیم با روش Selection Sort آن را مرتب کنیم به این صورت که کوچکترین عنصر را با  $A[1]$  عوض کنیم. دومین کوچکترین را با  $A[2]$

عوض کنیم و این عمل را  $n-1$  بار انجام دهیم. الگوریتم مربوطه را بنویسید و مرتبه آن را در بهترین و بدترین حالت بیابید.

۵- فرض کنید الگوریتمی فقط با مقایسه  $i$  امین کوچکترین عنصر را می‌یابد. نشان دهید که این الگوریتم می‌تواند  $i-1$  عنصر کوچک و  $n-i$  عنصر بزرگ عنصر را بدون مقایسه اضافی بیابد.

۶- آرایه مرتب  $A[1..n]$ ،  $n = 2^m$  مفروض است. یک الگوریتم برای تعیین مد و فرکانس آرایه مزبور ارائه دهید. مد آرایه، عنصری است که بیشترین تکرار را داشته باشد. فرکانس آرایه، تعداد تکرارهای مد آرایه است. مثلاً برای آرایه  $[4, 4, 4, 3, 2, 2, 1, 1]$ ، مد ۴ و فرکانس (تعداد تکرار ۴) برابر ۳ می‌باشد.

۷- الگوریتم هوارد (Howard) برای مرتب سازی عناصر آرایه  $A[1..n]$  به صورت زیر بیان می‌شود:

```
Algorithm stooge (Low, high)
if A[Low]>A[high] then A[Low] ↔ A[high]
if Low+1 < high then
{
     $k = \left\lfloor \frac{\text{high} - \text{Low} + 1}{3} \right\rfloor$ 
    stooge (Low, high -k)
    stooge (Low+k, high)
    stooge (Low, high-k)
}
```

نشان دهید مرتبه زمانی این الگوریتم  $O(n^{2/7})$  می‌باشد.

۸- آرایه مرتب شده صعودی  $A[1..n]$  که  $n = 2^m$  مفروض است. الگوریتمی برای پیدا کردن فاصله نزدیکترین عناصر بین زوج عناصر آرایه ارائه دهید. مثلاً برای آرایه  $[0, 3, 6, 9, 11, 15, 21, 25]$ ، فاصله نزدیکترین عناصر ۲ می‌باشد که مربوط به ۹ و ۱۱ است.

۹- با توجه به الگوریتم Quicksort و Partition زیر به سؤالات پاسخ دهید:

```
QUICKSORT (A, p, r) // A[p..r]
1   if (p < r) {
2       q = PARTITION (A, p, r)
3       QUICKSORT (A, p, q-1)
4       QUICKSORT (A, q+1, r)
5   }
```

فراخوانی به صورت  $\text{QUICKSORT}(A, 1, n)$  انجام می‌شود.

$\text{PARTITION}(A, p, r)$

```

۱  x = A[r]
۲  i = p - ۱
۳  for (j = p to r - ۱)
۴      if (A[j] ≤ x)
۵          {i = i + ۱
۶            exchange A[i] ↔ A[j]}
۷  exchange A[i + ۱] ↔ A[r]
۸  return i + ۱

```

(a) عملیات  $\text{PARTITION}$  روی 

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| ۲ | ۸ | ۷ | ۱ | ۳ | ۵ | ۶ | ۴ |
|---|---|---|---|---|---|---|---|

 نشان دهید.

(b) اگر همه عناصر  $A$  یکسان باشند، رویه  $\text{PARTITION}$  چه مقداری بر می‌گرداند.

(c) مرتبه  $\text{PARTITION}$  چیست؟

(d) تغییراتی ایجاد کنید که آرایه نزولی مرتب شود.

(e) اگر همه عناصر مساوی باشند مرتبه  $\text{QUICKSORT}$  چیست؟

(f) در بدترین حالت عمق پشته چقدر است؟

۱۰- نشان دهید زمان اجرای مورد انتظار (expected) مرتب‌سازی سریع عبارت است از:

$$T(n) \leq O(n) + \frac{1}{n} \sum_{i=1}^{n-1} (T(i) + T(n-i))$$

۱۱- دیدیم که مرتب‌سازی سریع، دو فراخوانی بازگشتی به خودش دارد. با فراخوانی اول، سمت چپ

محور و با فراخوانی دوم، سمت راست محور به طور بازگشتی مرتب می‌شوند. می‌توان فراخوانی دوم

را حذف کرد و به صورت حلقه نوشت. به این تکنیک  $\text{tail recursion}$  گویند که برخی کامپایلرها،

این عمل را انجام می‌دهند. تابع زیر با این روش مرتب‌سازی سریع را نشان می‌دهد:

(a) نشان دهید فراخوانی  $Q'(A, l, n)$  به درستی  $A[1..n]$  را مرتب می‌کند.

(b) در چه حالتی عمق پشته در این الگوریتم  $\theta(n)$  است

(c) تغییری در الگوریتم ایجاد کنید که عمق پشته در بدترین حالت  $\theta(\lg n)$  شود.

$Q'(A, p, r) // A[p..r]$

while  $p < r$  do

{

q =  $\text{PARTITION}(A, p, r)$

$Q'(A, p, q - 1)$

p = q + ۱

}



۱۲- برای آنکه احتمال وقوع بدترین حالت مرتب‌سازی سریع را کاهش دهیم می‌توانیم عنصر محور را به صورت تصادفی انتخاب کنیم و توابع زیر را ارائه دهیم:

RAND – PARTITION(A, p, r)

i = RANDOM(p, r)

Swap(A[r], A[i])

return PARTITION(A, p, r)

RAND – QUICKSORT(A, p, r)

if (p < r)            q = RAND – PARTITION(A, p, r)

                        RAND – QUICKSORT(A, p, q – ۱)

                        RAND – QUICKSORT(A, q + ۱, r)

در توابع فوق، در بهترین و بدترین حالت تابع RANDOM چند بار فراخوانی می‌شود؟

۱۳- زمان اجرای مرتب‌سازی سریع را می‌توان با استفاده از مرتب‌سازی درجی وقتی تقریباً مرتب است بهبود بخشید، هنگامی که QUICKSORT روی یک زیر آرایه با کمتر از k عنصر فراخوانی شد، اجازه دهید بدون مرتب‌سازی زیر آرایه بازگردد. پس از آنکه فراخوانی اولیه به QUICKSORT تمام شد، مرتب‌سازی درجی را روی کل آرایه اجرا کنید. نشان دهید مرتبه الگوریتم  $O(nk + n \lg \frac{n}{k})$  است.

۱۴- n عدد متمایز  $x_1, x_2, \dots, x_n$  با وزن‌های مثبت  $w_1, w_2, \dots, w_n$  که  $\sum_{i=1}^n w_i = 1$  مفروضند.

میان‌ه وزن‌دار، عنصری مثل  $x_k$  است به طوری که  $\sum_{x_i < x_k} w_i < \frac{1}{2}$  و  $\sum_{x_i > x_k} w_i \leq \frac{1}{2}$ .

(a) نشان دهید میان‌ه  $x_1, \dots, x_n$  همان میان‌ه وزن‌دار  $x_i$  ها است به شرطی که  $w_i = \frac{1}{n}$  برای

$i = 1, \dots, n$

(b) چگونه می‌توان میان‌ه وزن‌دار n عنصر را در بدترین حالت با کمک مرتب‌سازی در مرتبه  $O(n \lg n)$  یافت.

(c) چگونه می‌توان با استفاده از الگوریتم SEL، با مرتبه  $\theta(n)$  در بدترین حالت میان‌ه وزن‌دار n عنصر را یافت؟

مسئله مکان اداره پست (post – office Location problem) به این صورت تعریف می‌شود که n نقطه  $p_1, \dots, p_n$  با وزن‌های  $w_1, \dots, w_n$  داده شده‌اند. می‌خواهیم نقطه‌ای مثل p (که

لزوماً یکی از نقاط ورودی نیست) بیابیم بطوری که حاصل جمع  $\sum_{i=1}^n w_i \cdot d(p, p_i)$  که  $d(a, b)$

فاصله بین نقاط  $a$  و  $b$  است را می‌نیمم کند.

**(d)** نشان دهید که میانه وزن‌دار بهترین راه حل برای مسئله مکان اداره پست یک بعدی است

که در آن نقاط اعداد حقیقی هستند و  $d(a, b) = |a - b|$

**(e)** برای مسئله مکان اداره پست دو بعدی، بهترین راه حل را ارائه دهید. فرض کنید نقاط به

شکل  $(x, y)$  هستند و فاصله بین نقاط  $a = (x_1, y_1)$  و  $b = (x_2, y_2)$  عبارت است از:

$$d(a, b) = |x_1 - x_2| + |y_1 - y_2|$$

**۱۵-** دیدیم که تعداد مقایسه‌های SEL برای یافتن  $i$  امین کوچک‌ترین عنصر از بین  $n$  عنصر در

بدترین حالت  $T(n) = \theta(n)$  است ولی ثابت مخفی آن بزرگ است. اگر  $i$  کوچک باشد،

می‌توان الگوریتمی ارائه کرد که از SEL بهتر باشد.

**(a)** یک الگوریتم با  $u_i(n)$  مقایسه برای یافتن  $i$  امین کوچک‌ترین عنصر ارائه دهید که:

$$u_i(n) = \begin{cases} T(n) & \text{if } i \geq \frac{n}{2} \\ \left\lfloor \frac{n}{2} \right\rfloor + u_i\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T(\gamma i), & \text{otherwise} \end{cases}$$

**(b)** نشان دهید اگر  $i < \frac{n}{2}$  آنگاه:

$$u_i(n) = n + O\left(T(\gamma i) \lg\left(\frac{n}{i}\right)\right)$$

**(c)** نشان دهید اگر  $i$  ثابت کوچکتر از  $\frac{n}{2}$  باشد آنگاه  $u_i(n) = n + O(\lg n)$

**(d)** نشان دهید اگر  $i = \frac{n}{k}$  برای  $k \geq 2$ ، آنگاه  $u_i(n) = n + O\left(T\left(\frac{\gamma n}{k}\right) \lg k\right)$

**۱۶-** فرض کنید در هر سطح از مرتب‌سازی سریع، عناصر به نسبت  $1 - \alpha$  به  $\alpha$  قسمت می‌شوند

که  $\frac{1}{2} \leq \alpha < 1$  ثابت است. نشان دهید مینیمم عمق یک برگ در درخت بازگشت آن، تقریباً

$$-\frac{\lg n}{\lg \alpha} \text{ و ماکزیمم عمق تقریباً } -\frac{\lg n}{\lg(1 - \alpha)} \text{ است.}$$

**۱۷-** بحث کنید که برای هر ثابت  $\frac{1}{2} \leq \alpha < 1$  احتمال آنکه پارتیشن، بخش‌بندی متوازن تراز

$1 - \alpha$  به  $\alpha$  تولید کند، برابر  $1 - 2\alpha$  است.

## پاسخ تمرین

۲- الف) مرتبه مرتب‌سازی درجی برای مرتب‌سازی لیست  $k$  عنصری در بدترین حالت  $\theta(k^2)$

است پس مرتبه مرتب‌سازی  $\frac{n}{k}$  تا لیست  $k$  عنصری برابر  $\theta\left(\frac{n}{k} \cdot k^2\right) = \theta(nk)$  است.

ب) اگر بخواهیم همانند ادغام دو لیست، همه لیست‌ها را با هم ادغام کنیم، مرتبه

$\theta\left(n \cdot \frac{n}{k}\right) = \theta\left(\frac{n^2}{k}\right)$  است  $n$  بخاطر اینکه هر عنصر یک بار در لیست نهایی کپی می‌شود،

$\frac{n}{k}$  چون  $\frac{n}{k}$  لیست در هر مرحله بررسی می‌شوند که عنصر بعدی انتخاب و به لیست نهایی کپی شود).

ولی برای رسیدن به مرتبه  $\theta\left(n \cdot \lg\left(\frac{n}{k}\right)\right)$ ، باید لیست‌ها جفت جفت با هم ادغام شوند و سپس لیست‌های حاصل نیز جفت جفت ادغام شوند و به همین ترتیب، تا وقتی که فقط یک لیست بماند.

ادغام جفت جفت نیاز به زمان  $\theta(n)$  در هر مرحله دارد چون روی  $n$  عنصر عمل می‌کند، با اینکه روی زیر لیست‌ها افزاز شده است. تعداد سطوح با شروع از  $\frac{n}{k}$  لیست (هر یک  $k$  عنصر) و در نهایت ۱ لیست (با  $n$  عنصر)، برابر  $\left\lceil \lg\left(\frac{n}{k}\right) \right\rceil$  است. پس زمان کل برای ادغام  $\theta\left(n \lg\left(\frac{n}{k}\right)\right)$  است.

ج) باید  $\theta\left(nk + n \lg\left(\frac{n}{k}\right)\right) = \theta(n \lg n)$  که نتیجه می‌شود  $k$  حداکثر  $k = \theta(\lg n)$  است.

د) در عمل،  $k$  باید بزرگترین اندازه لیستی باشد که مرتب‌سازی درجی از ادغامی سریع‌تر است.

۳- الف) وارونگی‌ها عبارتند از (۵ و ۱)، (۵ و ۲)، (۴ و ۳) و (۵ و ۳) و (۴ و ۵) دقت کنید که اندیس عناصر و نه خود عناصر ذکر شده است.

$$\begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ \hline 2 & 3 & 8 & 6 & 1 \end{pmatrix}$$

ب) آرایه نزولی مرتب  $\langle n, n-1, \dots, 2, 1 \rangle$  و تعداد وارونگی‌هایش  $\frac{n(n-1)}{2}$  است.

ج) درواقع هر بار اجرای حلقه while در مرتب‌سازی درجی متناظر با کاهش یک وارونگی در آرایه است.

۴- فرض کنید  $\text{FIND-MIN}(A, r, s)$  اندیس کوچکترین عنصر در  $A$  از  $A[r]$  تا  $A[s]$  را برمی گرداند. واضح است که می توان این الگوریتم را با مرتبه  $O(s-r)$ ، پیاده سازی کرد. بنابراین الگوریتم مرتب سازی انتخابی:

Algorithm SELECTION-SORT(A)

Input:  $A = \langle a_1, a_2, \dots, a_n \rangle$

Output: Sorted A.

```

for i ← 1 to n - 1 do

```

$$j \leftarrow \text{FIND-MIN}(A, i, n)$$
$$A[j] \leftrightarrow A[i]$$

end for

با توجه به اینکه  $n-1$  بار FIND-MIN فراخوانی می‌شود مرتبه بهترین و بدترین برابر است با:

$$\theta(\sum_{i=1}^{n-1} i) = \theta(n^2)$$

(a - 9

Diagram illustrating an array structure with indices  $p$  to  $r$ . The array contains values: 2, 8, 7, 1, 3, 5, 6, 4. A blue arrow points to index  $i$  at the first cell. A box on the right shows  $x = 4$ .

$j = p$  ,  $A[p] \leq x \Rightarrow i = p$  ,  $A[p] \leftrightarrow A[p]$  : for بار اول اجرای

$j = p + 1$  ,  $A[p + 1] \leq x$      $\times$  : بار دوم اجرای

$j = p + 2$  ,  $A[p+2] \leq x$  : بار سوم اجرای for

$$j = p + r, A[p+r] \leq x \Rightarrow i = p + 1, A[p+1] \leftrightarrow A[p+r] \quad \text{بار چهارم:}$$

|   |     |     |     |     |     |   |   |
|---|-----|-----|-----|-----|-----|---|---|
| p | p+1 | p+2 | p+3 | p+4 | p+5 |   |   |
| 2 | 1   | 7   | 8   | 3   | 5   | 6 | 4 |

$$j=p+\epsilon, A[p+\epsilon] \leq x \Rightarrow i=p+\epsilon, A[p+\epsilon] \leftrightarrow A[p+\epsilon]$$

بار پنجم:

|   |     |     |     |     |     |   |   |
|---|-----|-----|-----|-----|-----|---|---|
| p | p+1 | p+2 | p+3 | p+4 | p+5 |   |   |
| 2 | 1   | 3   | 4   | 5   | 6   | 7 | 8 |

بار ششم:  $j = p + 5$  ,  $A[p + 5] \leq x$  ×

بار هفتم:  $j = p + 6 = r - 1$  ,  $A[p + 6] \leq x$  ×

از حلقه for خارج می‌شویم و خط 7 الگوریتم،  $A[p + 3]$  را با  $A[r]$  تعویض می‌کند:

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| ۲ | ۱ | ۳ | ۴ | ۷ | ۵ | ۶ | ۸ |
|---|---|---|---|---|---|---|---|

(b) در این حالت خط ۴ الگوریتم همیشه برقرار است و  $i$  هر بار اضافه می‌شود و با توجه به اینکه مقدار اولیه  $i \leftarrow p - 1$  است و در نهایت  $i + 1$  بر می‌گردد، مقدار برگشتی  $r - 1$  است.

(c) مرتبه  $\theta(n)$  است چون حلقه for برای آرایه  $n$  عنصری،  $\theta(n)$  بار اجرا می‌شود و در هر اجرا عملیات ثابت انجام می‌دهد.

(d) کفایت بجای  $\leq$  قرار دهید  $\geq$  در خط ۴.

(e) در این حالت مرتبه  $\theta(n^2)$  خواهد بود.

(f)  $\theta(n)$

(b-۱۱) در بدترین حالت عمق پشته  $\theta(n)$  است چون تعداد فراخوانی‌ها  $\theta(n)$  است.

(c) می‌توان در الگوریتم  $Q'$ ، فراخوانی بازگشتی را روی نیمه کوچک‌تر آرایه انجام داد:

دقت کنید زمان اجرا تغییر نمی‌کند و در بدترین حالت همچنان  $\theta(n^2)$  است ولی عمق پشته  $\theta(\lg n)$  می‌شود.

```

Q''(A, p, r)
while p < r do
{
  q ← PARTITION(A, p, r)
  if q - p < r - q then Q''(A, p, q - 1), p ← q + 1
  else Q''(A, q + 1, r), r ← q - 1
}

```

(۱۴-a) میانه عناصر  $x_1, \dots, x_n$  عنصری است مثل  $x_k$  که تقریباً نصف عناصر کوچک‌تر از آن و

تقریباً نصف عناصر بزرگ‌تر از آن هستند، پس اگر وزن هر عنصر  $\frac{1}{n}$  باشد آنگاه:

$$\sum_{x_i < x_k} w_i = \sum_{x_i < x_k} \frac{1}{n} = \frac{1}{n} + \frac{1}{n} + \frac{1}{n} + \dots + \frac{1}{n} \leq \frac{n}{2} \cdot \frac{1}{n} = \frac{1}{2}$$

$$\sum_{x_i > x_k} w_i = \sum_{x_i > x_k} \frac{1}{n} \leq \frac{n}{2} \cdot \frac{1}{n} = \frac{1}{2}$$

پس  $x_k$  میانه وزن‌دار  $x_1, \dots, x_n$  است که وزن‌های این نقاط  $w_i = \frac{1}{n}$  می‌باشد.

(b) ابتدا  $n$  عنصر را به صورت صعودی مرتب کنید ( $O(n \lg n)$ ) سپس از ابتدای آرایه، وزن عناصر را جمع کنید ( $O(n)$ ) تا به اولین عنصری مثل  $x_k$  برسید که اگر وزن آن را اضافه کنید، جمع از  $\frac{1}{p}$  بیشتر شود.  $x_k$  جواب است.

(c) میانه  $n$  عنصر را می‌یابیم ( $x_k$ ) و عناصر را حول آن افراز می‌کنیم، سپس جمع وزن‌های هر دو نیمه را محاسبه می‌کنیم، اگر وزن هر دو نیمه از  $\frac{1}{p}$  کمتر باشد،  $x_k$  میانه وزن‌دار است. در غیر این صورت میانه وزن‌دار در نیمه‌ای است که جمع وزن‌هایش از  $\frac{1}{p}$  بیشتر است. حال جمع وزن‌های نیمه‌ای که کمتر است را به  $x_k$  می‌افزاییم و الگوریتم را روی نیمه‌ای که جمع وزن‌هایش از  $\frac{1}{p}$  بیشتر است ادامه می‌دهیم:

```
WEIGHTED – MEDIAN(X) // X = {x1, x2, ..., xn}
if n = 1 then return x1
else if n = 2 then if w1 ≥ w2 then return x1 else return x2
else {find the median xk of X = {x1, x2, ..., xn}
partition the set X around xk
compute WL = ∑xi < xk wi and WG = ∑xi > xk wi
if WL <  $\frac{1}{p}$  and WG <  $\frac{1}{p}$  then return xk
elseif WL >  $\frac{1}{p}$  then {wk ← wk + wG, X' ← {xi ∈ X : xi ≤ xk}
return WEIGHTED – MEDIAN(X')}
else {wk ← wk + wL, X' ← {xi ∈ X : xi ≥ xk}
return WEIGHTED – MEDIAN(X')} }
```

زمان این الگوریتم در بدترین حالت:

$$T(n) = T\left(\frac{n}{p} + 1\right) + \theta(n) = \theta(n)$$

۱۶- عمق مینیمم در مسیری رخ می‌دهد که با ضریب  $\alpha$  همراه است (ضریب کوچکتر). یک

فراخوانی تعداد عناصر را از  $n$  به  $\alpha n$  کاهش می‌دهد. و  $i$  فراخوانی تعداد عناصر را به  $\alpha^i n$

کاهش می‌دهد. اگر برگ با عمق مینیمم  $m$  باشد باید داشته باشیم  $\alpha^m n = 1$  پس  $\alpha^m = \frac{1}{n}$

در نتیجه  $m = -\frac{\lg n}{\lg \alpha}$  به همین ترتیب اگر عمق ماکزیمم  $M$  باشد باید  $(1 - \alpha)^M n = 1$  که

$$M = -\frac{\lg n}{\lg(1 - \alpha)} \text{ در نتیجه}$$

## سؤال‌های چهارگزینه‌ای فصل پنجم

۱- با استفاده از الگوریتمی مناسب جهت تعیین عناصر مینیمم و ماکزیمم یک آرایه  $n$

عنصری، چه تعداد مقایسه انجام می‌پذیرد؟ (مهندسی فناوری اطلاعات – آزاد ۸۵)

$$(1) \left\lceil \frac{2n}{3} \right\rceil + 2 \quad (2) \left\lceil \frac{2n-3}{3} \right\rceil \quad (3) \lceil n \log n \rceil \quad (4) \left\lceil \frac{2n}{3} \right\rceil - 2$$

۲- هرگاه بخواهیم در یک آرایه شامل  $n$  عنصر ( $n = 2k$ ) عناصر مینیمم و ماکزیمم آن را

بدست آوریم. چه تعداد مقایسه باید انجام دهیم؟ (مهندسی فناوری اطلاعات آزاد ۸۵)

$$(1) 2k+3 \quad (2) 3k+1 \quad (3) 3k-2 \quad (4) 3k-1$$

۳- تعداد مقایسه‌های لازم برای مشخص نمودن مینیمم و ماکزیمم عناصر ذخیره شده در یک

آرایه یک بعدی شامل  $n$  عنصر را  $T(n)$  فرض می‌کنیم. در این صورت رابطه  $T(n)$  کدام

است؟ (فرض کنید  $T(1) = 0$  و  $T(2) = 1$ ) (تخصصی نرم افزار – مهندسی کامپیوتر آزاد ۸۶)

$$(1) T(n) = \frac{2T(n-2)}{2} + 1 \quad (2) T(n) = T(n-1) + T(n-2) + 2$$

$$(3) T(n) = 2T(n-2) + 1 \quad (4) T(n) = T(n-2) + 3$$

۴- الگوریتم زیر برای پیدا کردن عناصر ماکزیمم و مینیمم یک آرایه با  $n$  عنصر پیشنهادشده

است: (مهندسی کامپیوتر دولتی ۷۵)

procedure minmax ( $i, j$ : integer; var min, max: real);

var

k: integer;

min1, min2, max1, max2: real;

begin

if  $i+1=j$  then

if  $A[i] > A[j]$  then begin

min:= A[j]; max:= A[i]

end

else begin

min:= A[i]; max:= A[j]

end

else begin

k:= (i+j) div 2;

minmax (i, k, min1, max1);

minmax (k+1, j, min2, max2);

if min1 < min2 then min:= min1

else min:= min2;

if max1 < max2 then max:= max2

else max:= max1

end

end;

تعداد مقایسه‌های این الگوریتم (فقط مقایسه‌هایی که زیر آنها خط کشیده شده است) برای  $n = ۸$  چقدر است؟

۱۲ (۱)      ۱۳ (۲)      ۱۰ (۳)      ۱۱ (۴)

۵- الگوریتم زیر برای پیدا کردن اندیس‌های کوچکترین و بزرگترین عناصر در یک آرایه N تایی A داده شده است:

$\min \leftarrow ۱; \max \leftarrow ۱$

For  $i \leftarrow 2$  to  $N$  do

    Choose  $B = \text{true}$  or  $B = \text{false}$  with equal probabilities

    If  $B$  then if  $A[i] < A[\min]$  then  $\min \leftarrow i$

        else {if  $A[i] > A[\max]$

            then  $\max \leftarrow i$  }

    else if  $A[i] > A[\max]$

        then  $\max \leftarrow i$

    else if  $A[i] < A[\min]$  then  $\min \leftarrow i$

متوسط تعداد مقایسه‌های دو عنصر از A چند تا است؟ (تخصصی هوش مصنوعی دولتی ۸۲)

$\frac{۳(n-۱)}{۲}$  (۴)       $\frac{۳n}{۲}$  (۳)       $۲(n-۱)$  (۲)       $n$  (۱)

۶- الگوریتم پیدا کردن دو عنصر بیشینه و کمینه در یک آرایه با  $N$  عنصر بصورت زیر است:

MinMax (A)

۱-  $\min \leftarrow ۱$

۲-  $\max \leftarrow ۱$

۳- for  $i \leftarrow ۲$  to  $N$

۴-     do if  $A[i] < A[\min]$

۵-         then  $\min \leftarrow i$

۶-         else if  $A[i] > A[\max]$

۷-             then  $\max \leftarrow i$

حداکثر و حداقل تعداد مقایسه‌های دو عنصر از آرایه (سطر ۴ و ۶) به ترتیب برابر است با:

(علوم کامپیوتر ۸۳ و IT ۸۴)

(۱) هر دو  $n$       (۲) هر دو  $n-۱$       (۳)  $۲n$  و  $n$       (۴)  $۲(n-۱)$  و  $n-۱$



۷- رویه‌ی روبه‌رو را در نظر بگیرید.

تعداد مقایسه‌ها (تعداد < ها در سطر ۶) بر حسب  $n$  چند است؟ (هوش مصنوعی ۹۱)

FINDMAX (A, i, j)

۱  $n \leftarrow j - i + 1$

۲ if  $n = 1$

۳ then return  $A[i]$

۴ else  $m_1 \leftarrow \text{FINDMAX}\left(A, i, i + \left\lfloor \frac{n}{2} \right\rfloor - 1\right)$

۵  $m_2 \leftarrow \text{FINDMAX}\left(A, i + \left\lfloor \frac{n}{2} \right\rfloor, j\right)$

۶ if  $m_1 < m_2$

۷ then return  $m_2$

۸ else return  $m_1$

$$(1) \quad n-1 \quad (2) \quad n - \left( \left\lfloor \frac{n}{2} \right\rfloor - \left\lfloor \frac{n}{2} \right\rfloor \right) \quad (3) \quad n \quad (4) \quad \lceil \lg n \rceil$$

۸- درستی یا نادرستی گزاره‌های زیر کدام است؟ (نرم‌افزار ۹۱)

(a) از میان  $n$  نقطه در صفحه، می‌توان نزدیک‌ترین  $\sqrt{n}$  نقطه به مبدأ را در زمان  $O(n)$  به دست آورد.

(b) مرتب‌سازی ۶ عنصر با الگوریتم‌های مبتنی بر مقایسه حداقل به ۱۰ مقایسه در بدترین حالت نیاز دارد.

(۱) (a نادرست، b نادرست) (۲) (a نادرست، b درست)

(۳) (a درست، b نادرست) (۴) (a درست، b درست)

۹- کمترین مرتبه زمانی الگوریتم پیدا کردن  $i$  امین کوچکترین عنصر از میان  $n$  عنصر کدام

است؟ (IT ۸۵)

$$(1) \quad O(n^2) \quad (2) \quad O(n) \quad (3) \quad O(n \lg n) \quad (4) \quad \text{هیچکدام}$$

۱۰- دومین کوچکترین عنصر بین  $n$  عنصر را با چند مقایسه بین عناصر می‌توان به دست آورد؟

(بهترین جواب ممکن را انتخاب کنید) (تخصصی هوش مصنوعی دولتی ۸۲ و علوم کامپیوتر ۸۳)

$$(1) \quad n + \lceil \lg n \rceil - 2 \quad (2) \quad n + \lfloor \lg n \rfloor - 2$$

$$(3) \quad n + \lfloor \lg n \rfloor - 1 \quad (4) \quad n + \lceil \lg n \rceil - 1$$

- ۱۱- بهترین زمان برای انتخاب  $k$  امین عنصر کوچک ( $k-1$  عنصر کوچکتر و بقیه بزرگترند) و عدد میانه از بین  $n$  عدد نامرتب به ترتیب از راست به چپ چقدر است؟

(تخصصی نرم افزار آزاد ۸۲)

$$\begin{array}{ll} (۱) \quad O(n \log n), O(n \log n) & (۲) \quad O(n) \text{ و } O(n) \\ (۳) \quad O(n) \text{ و } O(n \log n) & (۴) \quad O(n \log n) \text{ و } O(n) \end{array}$$

- ۱۲- اگر  $n$  عنصر نامرتب داشته باشیم می‌توان  $k$  عنصر بعد از median را به صورت مرتب در پیچیدگی زمانی زیر چاپ کرد:

(علوم کامپیوتر ۸۶)

$$\begin{array}{ll} (۱) \quad O(kn) & (۲) \quad O(k \lg n) \\ (۳) \quad O(n + k \lg k) & (۴) \quad O(n + k \lg n) \end{array}$$

- ۱۳-  $n$  عدد نامرتب و نامساوی داده شده‌اند. می‌خواهیم جمع کوچکترین  $\sqrt{n}$  عددهای این اعداد را پیدا کنیم. یک الگوریتم کارا این مسئله را در چه زمانی می‌تواند حل کند؟

(تخصصی هوش مصنوعی دولتی ۸۶)

$$(۱) \quad O(\sqrt{n}) \quad (۲) \quad O(n) \quad (۳) \quad O(n \lg n) \quad (۴) \quad O(\sqrt{n} \lg n)$$

- ۱۴- آرایه  $A[1..3n]$  از اعداد داده شده است. می‌خواهیم با مقایسه‌ی اعداد آرایه، دو عدد  $x$  و  $y$  ( $x < y$ ) را به دست آوریم به طوری که  $n$  عنصر  $A$  مقداری کم‌تر از  $x$ ،  $n$  عنصر  $A$  مقداری بین  $x$  و  $y$  و  $n$  عنصر بقیه مقداری بیشتر از  $y$  داشته باشند. یک الگوریتم کارا برای حل این مسئله به میزان  $M(n)$  حافظه‌ی اضافی (علاوه بر حافظه‌ی  $A$ ) مصرف می‌کند و به زمان  $T(n)$  نیاز دارد. کدام یک، بهترین جواب برای این مسئله است؟

(تخصصی هوش مصنوعی ۸۸)

$$\begin{array}{ll} (۱) \quad M(n) = O(n) \text{ و } T(n) = O(n) & (۲) \quad M(n) = O(1) \text{ و } T(n) = O(n) \\ (۳) \quad M(n) = O(1) \text{ و } T(n) = O(n^2) & (۴) \quad M(n) = O(n) \text{ و } T(n) = O(n \lg n) \end{array}$$

- ۱۵- یک مجموعه  $A$  از  $n$  عدد مجزا از هم داده شده است. می‌خواهیم بدانیم آیا برای همه‌ی  $x, y, z \in A$  رابطه‌ی زیر برقرار است؟

(تخصصی هوش مصنوعی ۸۸)

$x + y > z$ ,  $x + z > y$ , and  $y + z > x$  بهترین راه حل این مسئله از چه مرتبه‌ای است؟

$$\begin{array}{ll} (۱) \quad O(n) & (۲) \quad O(n^2) \\ (۳) \quad O(n \log n) & (۴) \quad O(n^2 \log n) \end{array}$$

- ۱۶- برای پیدا کردن عدد میانه (median) بین  $n$  عدد در حالتی که  $n = 3$  و  $n = 5$  باشد به ترتیب حداقل چند مقایسه نیاز داریم؟ (از چپ به راست)

(علوم کامپیوتر ۸۹)

$$(۱) \quad ۳ \text{ و } ۲ \quad (۲) \quad ۱۰ \text{ و } ۳ \quad (۳) \quad ۶ \text{ و } ۲ \quad (۴) \quad ۱۰ \text{ و } ۲$$

۱۷- در راه حل کلاسیک برای یافتن عنصر میانه‌ی  $n$  عنصر، عناصر را به دسته‌های ۵ تایی تقسیم کردیم و میانه‌ی هر کدام و سپس میانه‌ی میانه‌ها را به صورت بازگشتی به دست آوردیم. این عنصر به عنوان محور برای بخش‌بندی عناصر محسوب می‌شود. اگر به جای دسته‌های ۵ تایی، از دسته‌های ۳ تایی استفاده کنیم، زمان اجرا توسط کدام رابطه‌ی بازگشتی بیان می‌شود؟

(نرم‌افزار ۹۱)

$$\begin{aligned} T(n) &\leq T\left(\frac{n}{3}\right) + cn & (۲) & \quad T(n) \leq T\left(\frac{2n}{3}\right) + cn & (۱) \\ T(n) &\leq T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + cn & (۴) & \quad T(n) \leq 2T\left(\frac{n}{3}\right) + cn & (۳) \end{aligned}$$

۱۸- فرض کنید عنصری در آرایه  $n$  عنصری، بیش از  $\frac{n}{3}$  بار تکرار شده باشد. بهترین الگوریتم برای یافتن این عنصر با هزینه حافظه مصرفی  $O(1)$  دارای چه هزینه زمانی است؟

(علوم کامپیوتر ۹۱)

$$O(n \log n) \quad (۴) \quad O(n^2) \quad (۳) \quad O(\log n) \quad (۲) \quad O(n) \quad (۱)$$

۱۹-  $n$  نقطه در صفحه‌ی مختصات دو بعدی داده شده است. می‌خواهیم  $k$  نزدیک‌ترین نقطه به مرکز مختصات را به ترتیب فاصله‌هایشان به دست آوریم. بهترین الگوریتم برای این کار از چه مرتبه‌ای است؟

(هوش مصنوعی ۹۲)

$$O(n + k \lg n) \quad (۴) \quad O(n + k \lg k) \quad (۳) \quad O(n \lg n) \quad (۲) \quad O(n) \quad (۱)$$

۲۰- دو آرایه  $x[1..n]$  و  $y[1..n]$  مرتب شده‌اند. سریع‌ترین الگوریتم برای یافتن میانه  $2n$  عضو آرایه‌های  $x$  و  $y$  کدام است؟

(نرم‌افزار دولتی ۸۵ و ۹۵)

(۱) روش استفاده از ادغام است که  $\theta(n)$  زمان نیاز دارد.

(۲) روش استفاده از ادغام است که  $\theta(\log n)$  زمان نیاز دارد.

(۳) یک الگوریتم تقسیم و حل است که نمونه‌های  $n$  تایی را به زیر نمونه‌های  $\frac{n}{3}$  تایی تبدیل می‌کند و  $\theta(\sqrt{n})$  زمان نیاز دارد.

(۴) یک الگوریتم تقسیم و حل است که نمونه‌های  $n$  تایی را به زیر نمونه‌های  $\frac{n}{3}$  تایی تبدیل می‌کند و  $\theta(\log n)$  زمان نیاز دارد.

۲۱- اگر  $X[1..n]$  و  $Y[1..n]$  دو آرایه  $n$  عدد به صورت مرتب شده باشند کمترین زمان برای پیدا کردن median (عنصر میانه) این  $2n$  عدد برابر است با:

(علوم کامپیوتر ۸۷)

$$O(\lg n) \quad (۴) \quad O(n) \quad (۳) \quad O(n \lg n) \quad (۲) \quad O(1) \quad (۱)$$

۲۲- دو آرایه‌ی مرتب هر یک به طول  $n$  از اعداد مجزا از هم داده شده است. می‌خواهیم با کمترین تعداد خواندن درایه‌های این دو آرایه میانه‌ی  $2n$  عدد موجود در این دو آرایه را به دست آوریم. کدام یک از رابطه‌های بازگشتی نحوه‌ی رفتار این الگوریتم را نشان می‌دهد؟ فرض کنید  $T(2n)$  بیشترین تعداد دسترسی به درایه‌های این دو آرایه برای حل بهینه‌ی این مسئله است.

(تخصصی هوش مصنوعی ۸۷)

$$T(2n) = 2T(n) + 2 \quad (۲)$$

$$T(2n) = T(n) + 2 \quad (۱)$$

$$T(2n) = T\left(\frac{3n}{2}\right) + 2 \quad (۴)$$

$$T(2n) = 3T\left(\frac{n}{2}\right) + 2 \quad (۳)$$

۲۳- دو آرایه‌ی مرتب  $A_1$  و  $A_2$  با مجموع تعداد  $n$  عنصر داده شده‌اند. فرض کنید که عناصر مجزا هستند. می‌خواهیم  $k$  امین عنصر  $A_1 \cup A_2$  را به دست آوریم.

(نرم‌افزار ۹۰)

این کار را می‌توان در کدام زمان زیر انجام داد؟

$$O(\lg \lg n) \quad (۴)$$

$$O(n + \lg n) \quad (۳)$$

$$O(\lg^2 n) \quad (۲)$$

$$O(\lg n) \quad (۱)$$

۲۴- رویه زیر را در نظر بگیرید:

Procedure mergesort (Low, high, A)

// A [Low.. high] is an array containing values which

represents the elements to be sorted //

begin

if Low < high then

begin

$$\text{mid} \leftarrow \left\lfloor \frac{(\text{Low} + \text{high})}{2} \right\rfloor$$

mergesort (Low, mid, A)

mergesort (mid+1, high, A)

merge (Low, mid, high, A)

end

end;

رویه merge در رویه فوق دو لیست مرتب شده  $A[\text{Low} \dots \text{mid}]$  و  $A[\text{mid} + 1 \dots \text{high}]$

را ادغام می‌کند. تعداد صداهای انجام گرفته به رویه فوق برای مرتب کردن لیست زیر

(تخصصی نرم افزار آزاد ۷۸)

چیست؟

۳۱۰، ۲۸۵، ۱۷۹، ۶۵۲، ۳۵۱، ۴۲۳، ۸۶۱، ۲۵۴، ۴۵۰، ۵۲۰

$$۲۰ \quad (۴)$$

$$۱۹ \quad (۳)$$

$$۲۸ \quad (۲)$$

$$۲۳ \quad (۱)$$

۲۵- در الگوریتم Mergesort اگر بجای آنکه هر بار لیست به دو قسمت مساوی تقسیم شود به چهار قسمت مساوی تقسیم گردد و در مرحله ترکیب این چهار لیست در یکدیگر ادغام شوند پیچیدگی زمانی الگوریتم چه خواهد شد؟  
(تخصصی نرم افزار آزاد ۸۰)

$$(1) \theta(n^{\frac{3}{4}}) \quad (2) \theta(n \log^{\frac{n}{4}}) \quad (3) \theta(n^2) \quad (4) \theta(n^2 \log^{\frac{n}{4}})$$

۲۶-  $T(n) = 2T(\frac{n}{4}) + n - 1$  این فرمول بازگشتی دقیقاً زمان Mergesort را در کدام حالت نشان می دهد؟  
(علوم کامپیوتر ۸۶) (مشابه تخصصی نرم افزار مهندسی کامپیوتر آزاد ۸۶)

$$(1) \text{ بدترین حالت} \quad (2) \text{ بهترین حالت} \quad (3) \text{ در همه حالات} \quad (4) \text{ بهترین و میانگین}$$

۲۷- اگر در الگوریتم Mergesort برای مرتب سازی لیست های زیر ۲۰ عنصر، از الگوریتم Insertion sort استفاده شود، پیچیدگی زمانی الگوریتم چه خواهد شد؟  
(IT ۸۵)

$$(1) \theta(n^2) \quad (2) \theta(n) \quad (3) \theta(n^2 \log n) \quad (4) \theta(n \log n)$$

۲۸- می خواهیم k لیست مرتب، هر کدام با  $\frac{n}{k}$  عنصر را در هم ادغام کنیم و یک لیست مرتب n تایی ایجاد نماییم. برای این کار ابتدا لیست ۱ را با لیست ۲ ادغام می کنیم، سپس لیست حاصل را با لیست ۳، ... و در پایان لیست حاصل را با لیست k ادغام می کنیم. زمان اجرای این الگوریتم در بدترین حالت بر حسب n و k چقدر است؟  
(نرم افزار ۹۱)

$$(1) O(n) \quad (2) O(nk) \quad (3) O(n \lg k) \quad (4) O(k \lg n)$$

۲۹- رویه partition در الگوریتم Quicksort به صورت زیر است:

function partition (p, r: integer): integer;

var

x, i, j: integer;

begin

x:=A[p]; i:= p-1; j:= r+1

while true do

begin

repeat j:=j-1 until A[j]<=x;

repeat i:=i+1 until A[i]>= x;

if i<j then swap (A[i] , A[j]) else return (j)

end

end

اگر تمام درایه های A[p..r] دارای مقدار یکسانی باشند، مقداری که رویه فوق برمی گرداند چقدر است؟  
(دولتی ۸۱)

$$(1) p \quad (2) r \quad (3) \left\lceil \frac{p+r}{2} \right\rceil \quad (4) \left\lfloor \frac{p+r}{2} \right\rfloor$$

۳۰- در گونه جدیدی از مرتب‌سازی سریع (quick sort)، برای انتخاب محور از میان  $n$  عنصر،  $2\sqrt{n} + 1$  عنصر اول آرایه را انتخاب می‌کنیم و با یک الگوریتم ساده مانند مرتب‌سازی درجی آنها را مرتب می‌کنیم. عنصر میانه این تعداد عنصر مرتب را به عنوان محور الگوریتم انتخاب می‌کنیم. بقیه الگوریتم مانند قبل عمل می‌کند. بدترین زمان اجرای این الگوریتم با کدام رابطه بازگشتی نشان داده می‌شود؟

(تخصصی نرم افزار - دولتی ۸۳ و تخصصی هوش مصنوعی - دولتی ۸۴)

$$T(n) = T(n-1) + n \quad (۲) \quad T(n) = 2T(\sqrt{n}) + n \quad (۱)$$

$$T(n) = T(n - \sqrt{n}) + T(\sqrt{n}) + n \quad (۴) \quad T(n) = 2T(n - \sqrt{n}) + n \quad (۳)$$

۳۱- زیر برنامه New Quick sort را برای مرتب‌سازی آرایه  $T$  به ترتیب نزولی در نظر می‌گیریم. Pivot ( $T, p, k$ ) عناصر  $T$  را حول محور  $p$  طوری می‌چرخاند که پس از اتمام کار کلیه عناصر بزرگتر از  $p$  قبل از خانه  $k$  و عناصر کوچکتر و مساوی بعد از آن در  $T$  جای می‌گیرند. ( $k$  پارامتر خروجی است). Insertion sort به روش درجی آرایه را به ترتیب نزولی مرتب می‌کند. پیچیدگی زمانی این زیر برنامه برای کدام یک از آرایه‌های زیر مطمئناً کمتر از پیچیدگی زمانی الگوریتم معمول Quick sort است؟

(تخصصی نرم افزار - دولتی ۸۴)

```
NewQuicksort (T[1 .. n])
p ← T[p]
pivot(T, p, k)
Insertionsort (T [1 .. k-1])
Insertionsort (T[k+1 .. n])
```

(۱) برای همه حالات ورودی

(۲) آرایه ابتدایی مرتب شده نزولی باشد.

(۳) آرایه ابتدایی مرتب شده صعودی باشد.

(۴)  $p$  دقیقاً میانه آرایه باشد و آرایه ابتدایی نامرتب باشد.

۳۲- رویه partition در quicksort به صورت زیر است:

```
Partition (A, p, r)
x := A[p]; i := p - 1; j := r + 1;
while (true)
  repeat j:=j-1 until A[j] ≤ x
  repeat i:=i+1 until A[i] ≥ x
  if (i < j) then swap (A[i], A[j]) else return j
end
end
```

و خود quick sort به صورت زیر است:

```
quicksort (A, p, r)
  if p < r then begin
    q := Partition (A, p, r)
    Quicksort (A, p, q)
    Quicksort (A, q+1, r)
  end
```

اگر A با n عنصر نامساوی در ابتدا برعکس مرتب باشد. تعداد تعویض‌ها دقیقاً چند

تاست؟ (تخصصی هوش مصنوعی دولتی ۸۶)

$$(1) \left\lfloor \frac{n}{2} \right\rfloor \quad (2) n \quad (3) n-1 \quad (4) 2n-2$$

۳۳- اگر در الگوریتم مرتب‌سازی سریع (Quick sort) به ترتیب صعودی، عنصر لولا (pivot) را همان عنصر اول لیست بگیریم و با استفاده از آن یک بار یک لیست مرتب نزولی و یک بار دیگر لیست مرتب صعودی را مرتب کنیم، گزینه صحیح برای مرتبه عملیات اصلی (مقایسه و جابجایی) را در این حالت انتخاب کنید. (علوم کامپیوتر ۷۹)

(۱) هر دو حالت از  $O(n \log n)$

(۲) هر دو حالت از  $O(n^2)$

(۳) برای لیست صعودی  $O(n)$  و برای لیست نزولی  $O(n^2)$

(۴) برای لیست صعودی  $O(n \log n)$  و برای لیست نزولی  $O(n^2)$

۳۴- لیست زیر را در نظر بگیرید. اگر عنصر اول لیست یعنی عدد ۹ را به عنوان لولا (pivot) اختیار کنیم کدام یک از گزینه‌های زیر می‌تواند خروجی مرحله اول الگوریتم مرتب‌سازی سریع (Quicksort) باشد: (۹, ۱۰, ۸, ۷, ۶, ۱۵, ۳) (علوم کامپیوتر ۸۰)

(۱) (۱۵ و ۳ و ۶ و ۱۰ و ۹ و ۸ و ۷) (۲) (۱۵ و ۱۰ و ۶ و ۳ و ۹ و ۸ و ۷)

(۳) (۱۵ و ۹ و ۷ و ۸ و ۳ و ۶) (۴) (۱۵ و ۱۰ و ۳ و ۹ و ۸ و ۷ و ۶)

۳۵- چنانچه در الگوریتم quicksort، الگوریتم بخش‌بندی (partition) زمان ثابت C نیاز داشته باشد، زمان اجرای مرتب‌سازی سریع در حالت تصادفی (داده‌های تصادفی) چیست؟ (هوش مصنوعی ۹۰)

$$(1) \Theta(n^2) \quad (2) \Theta(n) \quad (3) \Theta(\lg n) \quad (4) \Theta(n \lg n)$$

۳۶- می‌خواهیم یک آرایه  $A$  با  $n$  عنصر مجزا را به  $k = 2^T$  زیر آرایه  $A_k$  و ... و  $A_1$  هر یک با تعداد عناصر  $\left\lfloor \frac{n}{k} \right\rfloor$  یا  $\left\lceil \frac{n}{k} \right\rceil$  تقسیم کنیم به طوری که برای همه  $j < i$  و هر  $a_i \in A_i$  و  $a_j \in A_j$  داشته باشیم  $a_i < a_j$ . بهترین الگوریتم برای این کار چه ویژگی دارد؟

(تخصصی هوش مصنوعی دولتی ۸۳)

(۱) همیشه از  $O(n \log k)$  است. (۲) همیشه از  $O(n \log n)$  است.

(۳) در حالت متوسط از  $O(n \log k)$  است. (۴) در حالت متوسط از  $O(n)$  است.

۳۷- آرایه  $A$  تقریباً مرتب شده است یعنی برای  $i = 1, 2, \dots, n-k$  داریم  $A[i] \leq A[i+k]$  برای مرتب نمودن تمام  $n$  عضو حداقل چه مقدار زمان لازم است؟

(تخصصی هوش مصنوعی دولتی ۸۵)

(۱)  $O(n)$  (۲)  $O(n.k)$  (۳)  $O(n \log k)$  (۴)  $O(n \log n)$

۳۸- در الگوریتم پیدا کردن  $k$  امین عنصر از  $n$  عنصر، ابتدا همه عناصر را به دسته‌های ۵ تایی (به جز احتمالاً یک دسته) تقسیم می‌کنیم، میانه هر دسته را به دست می‌آوریم و سپس میانه میانه‌ها را به صورت بازگشتی پیدا می‌کنیم. این عنصر را به عنوان محور انتخاب می‌کنیم و عمل partition را بر روی عناصر آرایه انجام می‌دهیم. پس از آن همین الگوریتم را به صورت بازگشتی (و برای یک  $k$  دیگر) بر روی یکی از بخش‌ها اجرا می‌کنیم تا عنصر مورد نظر پیدا شود. زمان اجرای این الگوریتم توسط کدام یک از رابطه‌های بازگشتی زیر بیان می‌شود؟

(تخصصی نرم افزار دولتی ۸۲)

$$(۱) T(n) = T\left(\left\lceil \frac{n}{5} \right\rceil\right) + T\left(\frac{3n}{5} - 6\right) + O(n) \quad (۲) T(n) = T\left(\left\lceil \frac{n}{5} \right\rceil\right) + T\left(\frac{3n}{5} - 6\right) + O(n)$$

$$(۳) T(n) = T\left(\left\lceil \frac{n}{5} \right\rceil\right) + T\left(\frac{7n}{5} + 6\right) + O(n) \quad (۴) T(n) = T\left(\left\lceil \frac{n}{5} \right\rceil\right) + T\left(\frac{7n}{5} + 6\right) + O(n)$$

۳۹- فرض کنید  $n$  عدد صحیح داریم که تعداد تکرار آنها بسیار زیاد است به طوری که حداکثر  $O(\log n)$  عدد متفاوت در میان این اعداد وجود دارد. با توجه به این شرایط، بهترین الگوریتم مرتب‌سازی مبتنی بر مقایسه که می‌توان برای این اعداد در نظر گرفت دارای چه هزینه‌ای است؟

(تخصصی نرم افزار ۸۹)

(۱)  $\theta(n \log n)$  می‌توان الگوریتمی را یافت.

(۲)  $\theta(n)$  می‌توان پیدا کرد.

(۳)  $\theta(n \log \log n)$  می‌توان پیدا کرد.

(۴) با توجه به آنکه اثبات شده است که هیچ الگوریتمی مبتنی بر مقایسه کمتر از  $\theta(n \log n)$  را نمی‌توان پیدا کرد، عملاً در این مورد نیز هزینه  $\theta(n \log n)$  است.



۴۰- یک الگوریتم مرتب‌سازی را «خسته‌کننده» می‌گوییم اگر جای گشتی از  $n$  عنصر مجزا از هم به عنوان ورودی وجود داشته باشد که یک عنصر خاص از آن ورودی توسط الگوریتم مورد نظر در مجموع  $\Omega(n)$  بار با بقیه‌ی عناصر مقایسه کند. کدام یک از الگوریتم‌های زیر خسته‌کننده نیست؟

(تخصصی نرم‌افزار ۸۸)

Mergesort (۲)

Insertion sort (۱)

Heapsort (۴)

Bubble sort (۳)

۴۱- الگوریتم زیر نسخه بازگشتی کدام یک از الگوریتم‌های مرتب‌سازی است؟ (IT ۸۷)

```
function Sort(A[۰..n-۱], n)
  if n > ۱ then
    for k = ۱ to n-۱ do
      if A[k-۱] > A[k] then
        Swap(A, k, k-۱)
      end if
    end for
  end if
end sort
```

Insertion sort (۱)

Bubble sort (۲)

Selection sort (۳)

Shell sort (۴)

۴۲- یک آرایه  $A[۱..n]$  را  $k$  مرتب می‌گوییم اگر برای هر  $i$  که  $k < i \leq n-k$  داشته باشیم:

$$A[i-k] \leq A[i] \leq A[i+k]$$

مثلاً عناصر ۸ ۵ ۷ ۳ ۶ ۲ ۴ ۱ (از چپ به راست) یک آرایه ۲- مرتب است. در یک آرایه ۲ مرتب با  $2N$  عنصر، حداکثر اختلاف بین اندیس یک عنصر در این آرایه و اندیس همان عنصر اگر آرایه ۱ مرتب می‌بوده چقدر است؟

(تخصصی هوش مصنوعی دولتی ۸۶)

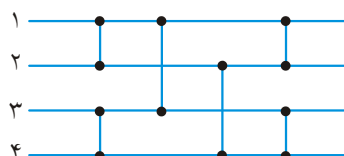
$$N \quad (۱) \quad ۲ \quad (۲) \quad \frac{N}{۲} \quad (۳) \quad ۲N-۱ \quad (۴)$$

۴۳- مدار زیر را در نظر بگیرید. هر المان این مدار می‌تواند در دو حالت

مستقیم یا زیگزاگ قرار گیرد. اگر ورودی ۱ تا ۴

باشد. با تعیین حالت‌های همه المان‌ها کدام یک از گزینه‌های زیر را می‌توان تولید کرد؟

(تخصصی هوش مصنوعی دولتی ۸۵)



(۱) ۳-۱-۴-۲

(۲) فقط ۴-۲-۱-۳

(۳) ۲-۳-۱-۴

(۴) همه موارد فوق

۴۴- الگوریتم Pancake Sort براساس رفتار آشپزها با پن یک بنا شده است. فرض کنید  $n$  عدد پن یک بر روی هم قرار گرفته‌اند و بر روی هر کدام عددی نوشته شده است که قابل رویت می‌باشد. تنها کار ممکن فروکردن کفگیر قبل از یک پن یک خاص و وارونه کردن همه پن یک‌های بالای کفگیر است. این کار را عمل «وارون» می‌گوییم که می‌تواند از هر جا انجام شود. هدف از این الگوریتم مرتب کردن صعودی اعداد روی پن یک‌ها با استفاده از کم‌ترین تعداد عمل «وارون» است. اگر تعداد  $n \geq 6$  باشد با چه تعداد عمل وارون می‌توان یقیناً اعداد را مرتب کرد؟ (کمترین گزینه را علامت بزنید.)

(تخصصی نرم‌افزار دولتی ۸۶)

$$(1) \quad n \quad (2) \quad n-1 \quad (3) \quad 2n-1 \quad (4) \quad 2n-2$$

۴۵- آرایه  $n$  تایی  $A$  حاوی جایگشتی از اعداد  $1$  تا  $n$  است. برنامه زیر این اعداد را مرتب می‌کند: بیشترین تعداد تعویض‌های برنامه فوق چند تاست؟

(تخصصی هوش مصنوعی دولتی ۸۴)

```
for i = 1 to n-1 do
while A[i] < i do
swap (A[i], A[A[i]])
```

$$(1) \quad n \quad (2) \quad n-1 \quad (3) \quad n \log n \quad (4) \quad \frac{n(n-1)}{2}$$

۴۶- احتمال این که Randomized – Quicksort به زمان  $\Omega(n^2)$  نیاز داشته باشد تا  $n$  عنصر مجزا از هم را مرتب کند چقدر است؟ (در Randomized – Quicksort، محور به صورت تصادفی و با احتمال یکسان یکی از عناصر انتخاب می‌شود و بقیه‌ی الگوریتم مانند Quicksort عادی است) بهترین جواب را انتخاب کنید.

(تخصصی هوش مصنوعی ۸۸)

$$(1) \quad \prod_{i=1}^{n-1} \frac{i}{n-i} \quad \text{حداقل} \quad (2) \quad \frac{1}{(n^n)} \quad (3) \quad \frac{1}{(n!)} \quad \text{حداقل} \quad (4) \quad \frac{1}{(n^2)} \quad \text{حداقل}$$

۴۷- می‌خواهیم آرایه‌ی  $A$  به طول  $n$  را که در آن حداکثر  $k$  عدد مجزا از هم وجود دارد و داریم  $k < \sqrt{n}$  را مرتب کنیم.

(تخصصی نرم‌افزار ۸۷)

- (۱) این کار را می‌توان با درجه  $(n \log k)$  انجام داد.
- (۲) این کار را می‌توان با درجه  $(n + k \log k)$  انجام داد.
- (۳) این کار را می‌توان با درجه  $(n)$  و مستقل از  $k$  انجام داد.
- (۴) این کار را نمی‌توان با درجه‌ی پیچیدگی‌ای کمتر از  $n \lg n$  انجام داد.

۴۸- فرض کنید تعدادی داده داریم که به صورت نزولی مرتب شده‌اند. با اجرای یک الگوریتم مرتب‌سازی صعودی روی آن، بعد از دو مرحله اجرا داده‌ها به صورت زیر درآمده است:

۵، ۱، ۲۶، ۱۹، ۱۵، ۱۱، ۷۷، ۶۱، ۵۹، ۴۸

الگوریتم اجرا شده چه نوع sort می‌باشد. (علوم کامپیوتر ۸۸)

Insertion sort (۱) Merge sort (۲) Quick sort (۳) Heap sort (۴)

۴۹- آرایه نامرتب A شامل  $n-1$  عدد صحیح و مجزا بین یک تا n است به جز یک عدد x. می‌خواهیم با یک الگوریتم کارا x را پیدا کنیم. کدام یک از الگوریتم‌های زیر درست و نسبت به بقیه سریعتر است؟ (تخصصی هوش مصنوعی دولتی ۸۵)

(۱) A را مرتب می‌کنیم و با یک پیمایش A، x را به دست می‌آوریم.  
 (۲) یک minheap بر روی A می‌سازیم. از ریشه شروع می‌کنیم و برای پیدا کردن x به یکی از زیردرخت‌های آن می‌رویم و کار را به صورت بازگشتی دنبال می‌کنیم.  
 (۳) یک درخت دودویی جستجوی متوازن بر روی عناصر A می‌سازیم و کار جستجو برای پیدا کردن x را از ریشه و سپس یکی از زیردرخت‌های آن دنبال می‌کنیم.  
 (۴) میانه A را پیدا می‌کنیم و آن را برای عمل بخش‌بندی (Partition) محور قرار می‌دهیم. بر این اساس، کار جستجو را دنبال می‌کنیم تا x بدست آید.

۵۰- آرایه  $S[1..N]$  از اعداد صحیح داده شده است. با داشتن K می‌خواهیم i و j را (در صورت وجود) پیدا کنیم که  $S[i] + S[j] = k$ . برای این کار، الگوریتم زیر پیشنهاد شده است: (تخصصی نرم‌افزار - دولتی ۸۲ و دولتی ۸۵)

```
i ← ۱ ; j ← n
while i ≤ j do
begin
  if S[i] + S[j] = k then return i, j
  if S[i] + S[j] < k then i ← i + ۱
  if S[i] + S[j] > k then j ← j - ۱
end
return not-possible
```

(۱) همواره درست کار می‌کند.

(۲) اگر S به صورت نزولی مرتب باشد درست کار می‌کند.

(۳) اگر S به صورت صعودی مرتب باشد درست کار می‌کند

(۴) برای هر حالت ورودی ممکن است هیچگاه جواب پیدا نشود.

۵۱- بر روی آرایه‌ی  $A$  با  $N$  عنصر  $A[1]$  تا  $A[N]$  که در ابتدا حاوی جای گشتی از اعداد ۱ تا  $N$  است، روبه زیر را اجرا می‌کنیم:

MAYBESORT( $A, N$ )

```

۱   $k \leftarrow 0$ 
۲  repeat
۳       $k \leftarrow k + 1$ 
۴      for  $i = 1$  to  $N$ 
۵          do  $B[i] \leftarrow A[A[i]]$ 
۶      for  $i = 1$  to  $N$ 
۷          do  $A[i] \leftarrow B[i]$ 
۸  until  $\forall 1 \leq i \leq N \ A[i] = i$ 
۹  return  $k$ 
```

(تخصصی نرم‌افزار ۹۰)

کدام یک از گزینه‌های زیر درست است؟

- (۱) مقدار خروجی دقیقاً مقدار  $N$  است.
- (۲) مقدار خروجی دقیقاً مقدار  $N-1$  است.
- (۳) خروجی این برنامه هیچ‌وقت بیش‌تر از  $\lceil \log N \rceil$  نیست.
- (۴) اجرای این برنامه ممکن است هیچ‌وقت تمام نشود.

۵۲- برای مرتب‌سازی آرایه‌ای با ۲۰۰۰ (دو هزار) عضو از الگوریتم Randomized quick sort

استفاده شده است اگر call stack برنامه در شروع الگوریتم خالی باشد و برای هر فراخوانی تابع تنها ۴ بایت آدرس برگشت در call stack قرار گیرد، در طی این فراخوانی حداکثر طول اشغال شده call-stack به طور متوسط چقدر خواهد بود؟ (علوم کامپیوتر ۸۷)

- (۱) ۴ بایت (۲) ۸ بایت (۳) ۴۴ بایت (۴) ۸۸ بایت

۵۳- فرض کنید که  $n = 2^k$  می‌خواهیم عنصر بیشینه (بزرگترین) را در یک ماتریس به اندازه‌ی

$n \times n$  بیابیم. برای این کار ماتریس را به چهار قسمت مساوی و هر کدام به اندازه‌ی  $\frac{n}{2} \times \frac{n}{2}$

تقسیم می‌کنیم. بیشینه‌ی هر کدام را به صورت بازگشتی به دست می‌آوریم و بین آنها جواب نهایی را پیدا می‌کنیم. تعداد دقیق مقایسه‌های عناصر با هم در این الگوریتم چند تاست؟ (تخصصی هوش مصنوعی ۸۹)

- (۱)  $n^2 - 1$  (۲)  $2n - 1$   
(۳)  $3(\log_2 n)$  (۴)  $3(\log_2 n - 1)$

۵۴- الگوریتم زیر آرایه‌ی  $n$  تایی  $A$  حاوی اعداد غیر تکراری  $1$  تا  $n$  را به درستی مرتب می‌کند.  
 INDEXSORT( $A$ )

```

۱ for  $i \leftarrow 1$  to  $n$ 
۲   do while  $A[i] \neq i$ 
۳     do SWAP ( $A[i], A[A[i]]$ )
  
```

در مورد تعداد جابه‌جایی (Swap) های این الگوریتم کدام یک از گزینه‌های زیر درست است؟  
 (تخصصی نرم‌افزار ۹۲)

- (۱) آرایه‌ای وجود دارد که الگوریتم دقیقاً  $n-1$  جابه‌جایی انجام دهد.
- (۲) تعداد جابه‌جایی‌ها همیشه برابر تعداد عناصری است که سر جای خودشان نیستند.
- (۳) این الگوریتم برای هر ورودی به تعداد  $n-1$  جابه‌جایی انجام می‌دهد.
- (۴) بیش‌ترین تعداد جابه‌جایی وقتی است که  $A$  برعکس مرتب شده باشد.

۵۵- برای مرتب‌سازی آرایه‌ی  $n$  عضوی  $A$  تنها مجاز به استفاده از رویه‌ی  $\text{SQRTSORT}(k)$  برای  $0 \leq k \leq n - \sqrt{n}$  هستیم. این رویه زیر آرایه‌ی  $|k + 1..k + \sqrt{n}|$  را به صورت درجا مرتب می‌کند. در بدترین حالت با چند بار فراخوانی  $\text{SQRTSORT}$  می‌توان  $A$  را مرتب کرد؟  
 (تخصصی هوش مصنوعی ۹۲)

(۱)  $O(n)$  (۲)  $O(1)$  (۳)  $O(\sqrt{n})$  (۴)  $O(\sqrt{n} \lg n)$

۵۶-  $n$  گوی باردار را در نظر بگیرید ( $n$  فرد است). هم چنین می‌دانیم که تعداد گویه‌های با بار مثبت زوج است. در هر عمل می‌توانیم دو گوی را انتخاب و به هم نزدیک کنیم و هم نوع بودن (از نظر بار) آنها را دریابیم. برای آن که بار (مثبت یا منفی) همه‌ی گوی‌ها را بیابیم. انجام دست کم چه تعداد عمل ضروری است؟  
 (تخصصی IT ۹۲)

(۱)  $2n-1$  (۲)  $\frac{n}{2}$  (۳)  $n-1$  (۴)  $\frac{3n}{2}$

۵۷- آرایه‌ای با  $n$  عنصر مجزا داده شده است. فرض کنید که  $a$  میانه‌ی این عناصر است. می‌خواهیم به تعداد  $k \leq \frac{n}{p}$  عنصر از این آرایه را انتخاب کنیم که میانه‌ی آنها هم  $a$  باشد. اگر  $k$  یک پارامتر ورودی باشد، با چه مرتبه‌ای این کار را می‌توان انجام داد؟ بهترین جواب را علامت بزنید.  
 (هوش مصنوعی - ۹۳)

(۱)  $O(n)$  (۲)  $O(nk)$  (۳)  $O(n \log k)$  (۴)  $O(n \min\{\log n, k\})$

## ۵۸- درستی گزاره‌های زیر را تعیین کنید.

الف) در هر الگوریتم مرتب‌سازی مبتنی بر مقایسه، دو عددی که اختلاف مرتبه‌ی آنها یک است، حتماً با یکدیگر مقایسه می‌شوند.

ب) در هر الگوریتم مرتب‌سازی مبتنی بر مقایسه، کوچک‌ترین و بزرگ‌ترین عدد حتماً با یکدیگر مقایسه می‌شوند.

(۹۳-IT)

۱) الف: نادرست و ب: نادرست      ۲) الف: درست و ب: درست

۳) الف: نادرست و ب: درست      ۴) الف: درست و ب: نادرست

## ۵۹- چند تا از گزینه‌های زیر درست‌اند؟ (۹۳-IT)

الف) داده ساختاری برای  $n$  عنصر وجود دارد که بتوان اعمال  $\text{Pop}$ ،  $\text{Push}$ ، یافتن مقدار عنصر کمینه‌ی موجود را هر کدام در  $O(1)$  انجام دهد.

ب) داده ساختاری برای  $n$  عنصر وجود دارد که بتوان اعمال  $\text{Pop}$ ،  $\text{Push}$ ، یافتن مقدار عنصر کمینه و یافتن مقدار عنصر بیشینه‌ی موجود را هر کدام در  $O(1)$  انجام دهد.

ج) داده ساختاری برای  $n$  عنصر وجود دارد که بتوان اعمال  $\text{Pop}$ ،  $\text{Push}$  و حذف عنصر کمینه‌ی موجود را هر کدام در  $O(1)$  انجام دهد.

۱) ۰      ۲) ۱      ۳) ۲      ۴) ۳

۶۰- ادغام دو لیست مرتب شده که هر یک شامل  $n$  عدد است با حداقل و حداکثر چند مقایسه

قابل انجام است؟ (۹۳-IT)

۱)  $n, 2n-1$       ۲)  $n-1, 2n$       ۳)  $n, 2n$       ۴)  $n-1, 2n-1$

۶۱- آرایه نامرتب  $B[1..2n+1]$  شامل اعداد حقیقی را در نظر بگیرید. می‌خواهیم آرایه

$A[1..2n+1]$  را به صورت جایگشتی از آرایه  $B$  ایجاد کنیم به طوری که:

$$A[1] \leq A[2] \geq A[3] \leq A[4] \dots \leq A[2n] \geq A[2n+1]$$

سریع‌ترین الگوریتم برای انجام این کار چه هزینه زمانی خواهد داشت؟ (علوم کامپیوتر - ۹۳)

۱)  $\theta(n)$       ۲)  $\theta(n^2)$       ۳)  $\theta(\log n)$       ۴)  $\theta(n \log n)$

۶۲- آرایه‌ای شامل  $n$  عدد متمایز به صورت نامرتب داده شده است. می‌خواهیم مجموع تعداد

$\log n$  عدد از بزرگترین اعداد موجود در این آرایه را محاسبه کنیم. بهترین الگوریتم برای

انجام این کار دارای چه مرتبه زمانی است؟ (علوم کامپیوتر - ۹۳)

۱)  $O(n)$       ۲)  $O(\log n)$       ۳)  $O(\log^2 n)$       ۴)  $O(n \log n)$

۶۳- فرض کنید  $0 < \alpha < 0.5$  یک عدد ثابت است. در ضمن، فرض کنید ورودی یک آرایه‌ی  $n$ 

عضوی باشد. الگوریتم Randomized QuickSort با احتمال یکسان یکی از عناصر آرایه

را به عنوان محور انتخاب کرده و براساس آن محور عمل «بخش‌بندی» را انجام می‌دهد، احتمال این که با این کار اندازه‌ی بخش کوچک‌تر بیش‌تر از  $\alpha n$  باشد، چقدر است؟

(هوش مصنوعی - ۹۴)

$$(1) 1 - 2\alpha \quad (2) 1 - \alpha \quad (3) \alpha \quad (4) 2 - 2\alpha$$

۶۴- فرض کنید مجموعه‌ی  $A$  شامل  $n$  عدد مرتب‌شده (صعودی) به همراه عدد  $c$  داده شده است. با چه مرتبه‌ی زمانی می‌توان فهمید که آیا دو عدد در  $A$  وجود دارند که جمع آن‌ها برابر  $c$  شود؟

(IT - ۹۴)

$$(1) O(\log n) \quad (2) O(n) \quad (3) O(n \log n) \quad (4) O(n^2)$$

۶۵- می‌خواهیم  $k$  کوچک‌ترین عنصر یک آرایه‌ی نامرتب  $n$  عضوی را از کوچک به بزرگ به دست آوریم. زمان اجرای بهترین الگوریتم برای این کار چیست؟

(IT - ۹۴)

$$(1) O(n + k \lg k) \quad (2) O(n \lg n + k) \quad (3) O(n \lg k) \quad (4) O(n + k \lg n)$$

۶۶- برای مرتب‌سازی یک آرایه  $5$  عنصری بر مبنای مقایسه، در بدترین حالت حداقل چند مقایسه نیاز داریم؟

(علوم کامپیوتر - ۹۴)

$$(1) 4 \quad (2) 5 \quad (3) 6 \quad (4) 7$$

۶۷- فرض کنید برای مرتب‌سازی  $n$  عدد از مرتب‌ساز ادغامی استفاده کرده‌ایم. چند تا از گزینه‌های زیر درست هستند؟

(دکتری کامپیوتر - ۹۵)

- هر عدد حداکثر با  $O(\log n)$  عدد مقایسه می‌شود.
- به طور متوسط هر عدد با  $O(\log n)$  عدد مقایسه می‌شود.
- حتماً عددی وجود دارد که با  $\Omega(\log n)$  عدد مقایسه شده است.

$$(1) 3 \quad (2) 2 \quad (3) 1 \quad (4) 0$$

۶۸- داده ساختار صف با سه عملیات افزودن به ابتدای صف، حذف از انتهای صف و استخراج عنصر کمینه را در نظر بگیرید. بهترین پیاده‌سازی ممکن برای این داده ساختار هر یک از سه عملیات فوق را به صورت «سرشکن» در چه زمانی پشتیبانی می‌کند؟ بهترین گزینه را انتخاب کنید.

(دکتری کامپیوتر - ۹۵)

- (۱) هر سه عملیات  $O(1)$
- (۲) هر سه عملیات  $O(\log n)$
- (۳) درج و حذف  $O(1)$ ، استخراج کمینه  $O(\log n)$
- (۴) درج و حذف  $O(\log n)$ ، استخراج کمینه  $O(n)$

۶۹- آرایه‌ای با ۸ عنصر را با quicksort می‌خواهیم مرتب کنیم. بعد از اولین تکرار، داده‌ها به صورت زیر در آمده است. کدام داده pivot بوده است؟  
 ۲, ۵, ۱, ۷, ۹, ۱۲, ۱۱, ۱۰  
 (دکتری علوم کامپیوتر - ۹۵)

(۱) ۷ یا ۹

(۲) می‌تواند ۷ شکلی باشد ولی نمی‌تواند ۹ باشد.

(۳) نمی‌تواند ۷ باشد ولی می‌تواند ۹ باشد.

(۴) نه ۷ و نه ۹

۷۰- مجموعه‌ی  $S$  با  $n \geq 2$  از اعداد نامرتب و متمایز را در نظر بگیرید. می‌خواهیم دو عدد  $x$  و  $y$  در  $S$  را طوری بیابیم که  $|x - y| \leq \frac{1}{n-1}(\max(S) - \min(S))$  باشد. بهترین الگوریتم برای یافتن  $x$  و  $y$  در  $S$  دارای چه هزینه‌ی زمانی خواهد بود؟  
 (دکتری علوم کامپیوتر - ۹۵)

(۱)  $O(n \log n)$  (۲)  $O(\log n)$  (۳)  $O(n^2)$  (۴)  $O(n)$

۷۱- یک لیست پیوندی دوطرفه داده شده است. می‌خواهیم این لیست را بدون جابجا کردن مقادیر درون گره‌ها و تنها با تغییر اشاره‌گرهای بین گره‌ها مرتب کنیم. با کدام یک از روش‌های زیر می‌توان این لیست را با کمترین تعداد تغییر اشاره‌گرها مرتب کرد؟ (IT - ۹۵)

(۱) مرتب‌سازی سریع (۲) مرتب‌سازی حبابی

(۳) مرتب‌سازی ادغامی (۴) مرتب‌سازی درجی

۷۲- آرایه‌ای  $A$  به طول  $n$  داده شده است. هدف یافتن دو اندیس  $i < j$  است که  $A[j] - A[i]$  بیشینه شود. در چه زمانی می‌توان این کار را انجام داد؟ (IT - ۹۵)

(۱)  $O(n)$  (۲)  $O(n \log n)$  (۳)  $O(n^2)$  (۴)  $O(n \log^2 n)$

۷۳- فرض کنید آرایه‌ی  $A$  دارای  $n$  عدد صحیح است، که بعضی از آنها ممکن است دوبار در آرایه ظاهر شده باشند. بهترین الگوریتم برای تعیین اینکه آرایه‌ی  $A$  دارای عناصر منحصر به فرد است، دارای چه پیچیدگی زمانی است؟ (علوم کامپیوتر - ۹۵)

(۱)  $O(n)$  (۲)  $O(\log n)$  (۳)  $O(n \log n)$  (۴)  $O(n^2)$

۷۴- در چه صورت می‌توان زمان الگوریتم quick Sort را در بدترین حالت  $O(n \log n)$  کرد؟ (علوم کامپیوتر - ۹۵)

(۱) در هر گام از الگوریتم Pivot با میانه زیر لیست آن گام مقدار بگیرد.

(۲) در هر گام از الگوریتم، Pivot به صورت تصادفی انتخاب شود.

(۳) در هر گام از الگوریتم، Pivot با عضو مینیمم زیر لیست آن گام مقدار بگیرد.

(۴) احتیاج به عمل خاصی نیست و بدترین زمان این الگوریتم  $O(n \log n)$  است.



## پاسخنامه سؤال‌های چهارگزینه‌ای فصل پنجم

- ۱- گزینه (۴). همانطور که دیدیم حداقل تعداد مقایسه برای یافتن min و max در آرایه n عنصری وقتی n زوج است برابر  $\frac{3n}{2} - 2$  و وقتی n فرد است برابر  $\frac{3n}{2} - \frac{3}{2}$  است که این دو رابطه را می‌توان به شکل  $\lceil \frac{3n}{2} \rceil - 2$  نوشت.
  - ۲- گزینه (۳). چون  $n = 2k$  پس n زوج است و تعداد مقایسه برابر  $\frac{3n}{2} - 2 = \frac{3(2k)}{2} - 2 = 3k - 2$  است.
  - ۳- گزینه (۴). با یک مقایسه  $\min_1$  و  $\max_1$  را بین ۲ عنصر اول آرایه بیابید و با  $T(n-2)$  مقایسه  $\min_2$  و  $\max_2$  را در بین  $n-2$  عنصر آخر آرایه بیابید و سپس  $\min_1$  و  $\min_2$  را با هم مقایسه کنید و  $\max_1$  و  $\max_2$  را با هم، پس  $T(n) = T(n-2) + 3$
  - ۴- گزینه (۳). فرض کنید  $T(n)$  تعداد مقایسه برای آرایه n عنصری است. با توجه به الگوریتم،  $T(2) = 1$  چون اگر  $j = i + 1$  یعنی اگر آرایه ۲ عنصری باشد فقط یک مقایسه  $(A[i] > A[j])$  صورت می‌گیرد. به ازای  $n > 2$  می‌توان نوشت  $T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 2$  چون دو فراخوانی بازگشتی انجام شده است یکی با  $\lfloor \frac{n}{2} \rfloor$  عنصر و یکی با  $\lceil \frac{n}{2} \rceil$  عنصر؛ و ۲ مقایسه  $\min_1 < \min_2$  و  $\max_1 < \max_2$  نیز انجام شده است. لازم به ذکر است فراخوانی تابع داده شده به صورت  $\min \max(1, n, \min, \max)$  می‌باشد. ما بدنبال  $T(8)$  هستیم:
- $$T(4) = 2T(2) + 2 = 4, T(8) = 2T(4) + 2 = 2 \times 4 + 2 = 10.$$
- ۵- گزینه (۴). فرض کنید  $B = \text{True}$ ، در این صورت، بهترین حالت هنگامی رخ می‌دهد که شرط  $\text{if } A[i] < A[\min]$  همیشه برقرار باشد که در این صورت،  $\text{else if}$  اجرا نخواهد شد و تعداد مقایسه‌ها  $B(n) = n - 1$  خواهد بود. بدترین حالت وقتی است که شرط  $\text{if } A[i] < A[\min]$  همیشه نادرست باشد که در این صورت اولین شرط  $\text{if } A[i] > A[\max]$  نیز هر بار اجرا می‌شود و تعداد مقایسه‌ها  $w(n) = 2(n - 1)$  می‌شود پس متوسط مقایسه‌ها  $\frac{(n-1) + 2(n-1)}{2}$  یعنی  $\frac{3(n-1)}{2}$  است. اگر فرض کنیم  $B = \text{false}$  است نیز همین وضعیت برای  $\text{else}$  مربوط به  $\text{if}$  دوم پیش می‌آید.
  - ۶- گزینه (۴). حداقل مقایسه وقتی است که شرط  $A[i] < A[\min]$  همیشه برقرار باشد که  $n - 1$  مقایسه انجام می‌دهد. بدترین حالت وقتی است که شرط  $A[i] < A[\min]$  هیچگاه برقرار نباشد که در این صورت شرط  $A[i] > A[\max]$  هر بار بررسی می‌شود و جمعاً  $2(n - 1)$  مقایسه انجام می‌شود.

- ۷- گزینه (۱). با توجه به الگوریتم وقتی آرایه یک عنصری است مقایسه‌ای انجام نمی‌شود یعنی  $T(1) = 0$ . به ازای  $n > 1$ ، دو تا فراخوانی بازگشتی انجام می‌شود که تعداد مقایسه‌های آنها به ترتیب  $T(\lfloor \frac{n}{2} \rfloor)$  و  $T(\lceil \frac{n}{2} \rceil)$  است و یک مقایسه  $(m_1 < m_2)$  انجام می‌شود پس  $T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 1$  با جایگذاری  $T(2) = 1$  و  $T(3) = 2$  و  $T(4) = 3$  و ... و  $T(n) = n - 1$ . توجه کنید که فراخوانی تابع به صورت  $\text{FINDMAX}(A, 1, n)$  می‌باشد.
- ۸- گزینه (۴). ابتدا فاصله این  $n$  نقطه تا مبدا را با مرتبه  $\theta(n)$  می‌یابیم و در آرایه  $A$  ذخیره می‌کنیم (یافتن فاصله نقطه  $(x, y)$  تا مبدا با فرمول  $\sqrt{x^2 + y^2}$  انجام می‌شود که از مرتبه  $\theta(1)$  است) سپس  $\sqrt{n}$  امین کوچکترین عنصر را در  $A$  می‌یابیم و با محوریت آن،  $A$  را افراز می‌کنیم که از مرتبه  $\theta(n)$  است. بدین ترتیب  $\sqrt{n}$  نقطه نزدیک به مبدا در ابتدای  $A$  قرار می‌گیرند. یادآوری می‌کنیم که یافتن  $k$  امین مینیمم و افراز آرایه  $n$  عنصری از مرتبه  $\theta(n)$  است.
- (b) برای مرتب‌سازی  $n$  عنصر با روش مقایسه‌ای در بدترین حالت به حداقل  $\lceil \lg n! \rceil$  مقایسه نیاز است و  $\lceil \lg 6! \rceil = 10$  پس درست است.
- ۹- گزینه (۲). با روش گفته شده در درس می‌توان آم مینیمم یا آمین ماکزیمم را با مرتبه  $\theta(n)$  یافت.
- ۱۰- گزینه (۱). با روش تورنمنت که در درس دیدیم تعداد مقایسه برای یافتن دومین مینیمم یا دومین ماکزیمم برابر  $n + \lceil \lg n \rceil - 2$  است.
- ۱۱- گزینه (۲). یافتن  $k$  امین مینیمم با مرتبه  $\theta(n)$  امکان‌پذیر است. میانه نیز حالت خاصی از همین الگوریتم است که  $k = \frac{n}{2}$  می‌باشد.
- ۱۲- گزینه (۳). کافی است  $n - k$  امین مینیمم را بیابیم و با محوریت آن آرایه را افراز کنیم که  $O(n)$  زمان می‌خواهد و سپس  $k$  عنصر انتهایی آرایه را با مرتبه  $O(k \lg k)$  مرتب کنیم. پس زمان کل از مرتبه  $O(n + k \lg k)$  است.
- ۱۳- گزینه (۲).  $\sqrt{n}$  امین مینیمم را یافته و با محوریت آن عناصر را افراز می‌کنیم که با مرتبه  $O(n)$  انجام می‌شود و سپس  $\sqrt{n}$  عنصر اول آرایه را با مرتبه  $O(\sqrt{n})$  جمع می‌کنیم بنابراین مرتبه کلی  $O(n + \sqrt{n}) = O(n)$  است.
- ۱۴- گزینه (۲).  $n$  امین مینیمم را یافته و عمل افراز را انجام می‌دهیم و سپس در بین  $2n$  عنصر آخر آرایه، مجدداً  $n$  امین مینیمم را یافته و افراز می‌کنیم. در این صورت  $n$  عنصر ابتدای آرایه همگی از  $n$  عنصر بعدی کوچکتر و  $n$  عنصر وسط نیز از  $n$  عنصر آخر آرایه کوچکتر هستند. مرتبه این عملیات  $O(n)$  است و می‌توان با حافظه کمکی ثابت این عملیات را انجام داد.

۱۵- گزینه (۱). می‌خواهیم بررسی کنیم آیا به ازای هر ۳ عنصر، جمع ۲ عنصر از سومی بیشتر است یا خیر (یعنی نامساوی مثلثی به ازای هر ۳ عنصر آرایه برقرار است یا خیر). اگر کمی دقت کنید نیاز نیست به ازای هر ۳ عنصر نامساوی مثلثی را بررسی کنید بلکه کافی است، اولین مینیمم (x) و دومین مینیمم (y) و ماکزیمم (z) را بیابید و اگر  $x + y > z$  آنگاه به ازای هر ۳ عددی نامساوی برقرار است. و اگر  $x + y > z$  آنگاه مثال نقض پیدا کرده‌ایم. پس این مسئله منجر می‌شود به مسئله یافتن اولین و دومین مینیمم و عنصر ماکزیمم که با مرتبه  $O(n)$  قابل انجام است.

۱۶- گزینه (۱). ثابت می‌شود که یافتن k امین مینیمم (یا k امین ماکزیمم) در بین n عنصر نیاز به حداقل  $n + (k-1) \left\lceil \lg \frac{n}{(k-1)} \right\rceil - k$  مقایسه دارد (اثبات دشوار است). بنابراین در بین  $n = 3$  عنصر یافتن میانه (یعنی دومین مینیمم  $k = 2$ ) حداقل  $3 - 2 = 1$  مقایسه می‌خواهد. و در بین  $n = 5$  عنصر یافتن میانه (یعنی سومین مینیمم  $k = 3$ ) نیاز به حداقل  $5 + (3-1) \left\lceil \lg \frac{5}{(3-1)} \right\rceil - 3 = 6$  مقایسه دارد.

۱۷- گزینه (۴). همانطور که در درس دیدیم، مراحل زیر باید انجام شوند:

- آرایه را به  $\frac{n}{3}$  گروه ۳ عنصری تقسیم کنیم و در هر گروه میانه را بیابیم. زمان این مرحله  $\theta(n)$  است.

- با فراخوانی بازگشتی، میانه  $\frac{n}{3}$  میانه‌ای که در مرحله قبل یافته‌ایم را بیابیم. زمان  $T(\frac{n}{3})$  است.

- با محوریت میانه میانه‌ها، عناصر را افراز کنیم. زمان  $\theta(n)$  است.

- اگر محور در محل k قرار گرفت که کار تمام است در غیر این صورت چپ یا راست محور را فراخوانی بازگشتی کنیم که زمان این مرحله در بدترین حالت  $T(\frac{2n}{3})$  است.

۱۸- گزینه (۱).  $\frac{n}{3}$  امین و  $\frac{2n}{3}$  امین مینیمم را یافته و آرایه را به ۳ قسمت افراز می‌کنیم، اگر عنصری بیش از  $\frac{n}{3}$  بار تکرار شده باشد با مقایسه عناصر در هر قسمت، متوجه می‌شویم. مرتبه این عملیات  $O(n)$  است.

۱۹- گزینه (۳). همانند تست ۸، ابتدا فاصله نقاط تا مبدا را یافته و این فاصله‌ها را در آرایه A ذخیره می‌کنیم. سپس k امین مینیمم را یافته و با محوریت آن، آرایه را افراز می‌کنیم. تاکنون زمان  $O(n)$  است. ولی چون گفته شده k نقطه نزدیک به مبدا را به ترتیب فاصله‌هایشان بیابید، حال باید k عدد ابتدایی آرایه را با زمان  $O(k \cdot \lg k)$  مرتب کنیم، که کل زمان  $O(n + k \cdot \lg k)$  است.

۲۰- گزینه (۴). عنصر وسط دو آرایه را با هم مقایسه کنید اگر مساوی بودند که جواب بدست آمده است. اگر عنصر وسط  $x$  از عنصر وسط  $y$  بزرگتر باشد، میانه یا در نیمه اول  $x$  یا در نیمه دوم  $y$  است یعنی می‌توان نیمه دوم  $x$  و نیمه اول  $y$  حذف کرد. و اگر وسط  $x$  از وسط  $y$  کوچکتر باشد، میانه یا در نیمه دوم  $x$  یا در نیمه اول  $y$  است. یعنی با یک مقایسه می‌توان نصف تعداد کل عناصر را حذف کرد. پس اگر زمان اجرا در بدترین حالت  $T(n)$  فرض شود می‌توان نوشت  $T(n) = T(\frac{n}{2}) + 1 = \theta(\lg n)$ .

۲۱- گزینه (۴). مشابه تست قبل مرتبه  $O(\lg n)$  است.

۲۲- گزینه (۱). مشابه تست ۲۰، عنصر وسط دو آرایه می‌خوانیم (پس ۲ عنصر خواندیم) و با مقایسه این ۲ عنصر می‌توان نصف کل عناصر را حذف کرد حال همین عملیات را به صورت بازگشتی روی  $n$  عنصر باقی‌مانده انجام می‌دهیم که نیاز به  $T(n)$  عمل خواندن عناصر دارد پس  $T(2n) = T(n) + 2$ .

۲۳- گزینه (۱). به طور کلی در دو آرایه مرتب  $n$  عنصری یافتن  $k$  امین مینیمم از مرتبه  $O(\lg n)$  است.

۲۴- گزینه (۳). فرض کنید  $T(n)$  تعداد فراخوانی‌های mergesort برای آرایه  $n$  عنصری باشد. با توجه به دو فراخوانی بازگشتی انجام شده می‌توان نوشت  $T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 1$  و  $T(1) = 1$  و تعداد عناصر داده شده  $n = 10$  تا تست که به راحتی می‌توان محاسبه کرد.  $T(10) = 19$ .

۲۵- گزینه (۲). تغییری نخواهد کرد زیرا اگر  $T(n)$  زمان اجرا باشد می‌توان نوشت  $T(n) = 4T(\frac{n}{4}) + \theta(n)$  که طبق قضیه master از مرتبه  $\theta(n \cdot \lg n)$  است.

۲۶- گزینه (۱). در بدترین حالت ادغام دو لیست  $\frac{n}{2}$  عنصری نیاز به  $n - 1$  مقایسه دارد. پس رابطه نوشته شده، بدترین حالت مرتب‌سازی ادغامی را نشان می‌دهد.

۲۷- گزینه (۴). چون ۲۰ عدد ثابت است، مرتبه فرقی نمی‌کند در واقع می‌توان رابطه زمان اجرا برای  $n \geq 20$  به صورت  $T(n) = 2T(\frac{n}{2}) + \theta(n)$  و برای  $n < 20$ ،  $T(n) = c$  که  $c$  ثابت است، نوشت که مرتبه این رابطه  $T(n) \in \theta(n \cdot \lg n)$  است.

۲۸- گزینه (۲). ادغام دو لیست مرتب  $m$  و  $n$  عنصری از مرتبه  $O(m + n)$  است. زمان ادغام دو لیست اول حداکثر  $\frac{n}{k} + \frac{n}{k} = \frac{2n}{k}$  است حال زمان ادغام این لیست جدید با لیست سوم حداکثر  $\frac{3n}{k}$  است و زمان ادغام این لیست جدید با لیست چهارم حداکثر  $\frac{4n}{k}$  است و به

همین ترتیب زمان آخرین ادغام حداکثر  $\frac{kn}{k}$  است. بنابراین زمان اجرای الگوریتم در بدترین حالت برابر است با:

$$\frac{2n}{k} + \frac{3n}{k} + \dots + \frac{kn}{k} = \frac{n}{k} (2 + 3 + \dots + k) = \frac{n}{k} \times \left[ \frac{k(k+1)}{2} - 1 \right] = \theta \left( \frac{n}{k} \cdot k^2 \right) = \theta(n.k)$$

۲۹- گزینه (۴). فرض کنید آرایه  $A[3..6]$  و همه عناصر آن ۲۰ باشد:

|   |    |    |    |    |
|---|----|----|----|----|
|   | ۳  | ۴  | ۵  | ۶  |
| A | ۲۰ | ۲۰ | ۲۰ | ۲۰ |

یعنی  $r=6$  و  $p=3$  الگوریتم را دنبال می‌کنیم:

$$x = A[3] = 20, \quad i = 2, \quad j = 7$$

در اولین repeat،  $j=6$  می‌شود و چون شرط  $A[j] \leq x$  درست است خارج می‌شویم در دومین repeat،  $i=3$  می‌شود و خارج می‌شود. شرط if درست است و swap اجرا می‌شود. بار بعدی اجرای while،  $j=5$  و  $i=4$  می‌شود و swap اجرا می‌شود. بار بعدی اجرای while،  $j=4$  و  $i=5$  می‌شود و مقدار  $j$  یعنی ۴ برگردانده می‌شود یعنی خروجی  $\left\lfloor \frac{3+6}{2} \right\rfloor = 4$  است.

۳۰- گزینه (۴). مرتب‌سازی درجی برای  $n$  عنصر از مرتبه  $O(n^2)$  است. پس مرتب‌سازی درجی برای  $2\sqrt{n}+1$  عنصر از مرتبه  $O((2\sqrt{n}+1)^2) = O(n)$  است. میانه  $2\sqrt{n}+1$  عنصر، عنصری است که از  $\sqrt{n}$  عنصر بزرگتر است، پس اگر با محوریت این عنصر، آرایه را پارتیشن کنیم، حداقل  $\sqrt{n}$  عنصر سمت چپ محور و حداکثر  $n - \sqrt{n}$  عنصر سمت راست محور قرار می‌گیرند و فراخوانی بازگشتی با چپ و راست محور به ترتیب زمان  $T(\sqrt{n})$  و  $T(n - \sqrt{n})$  نیاز دارد. بنابراین برای زمان این الگوریتم می‌توان نوشت:

$$T(n) = T(\sqrt{n}) + T(n - \sqrt{n}) + O(n)$$

۳۱- گزینه (۲). می‌دانیم برای آرایه نزولی در مرتب‌سازی درجی اگر بخواهیم نزولی مرتب کنیم بهترین حالت است و مرتبه آن  $O(n)$  است. در این سوال نیز بهترین حالت وقتی است که آرایه نزولی است که در این حالت،  $Insertion\ sort(T[1])$  و  $Insertion\ sort(T[2..n])$  اجرا می‌شوند که مرتبه  $O(n)$  است.

۳۲- گزینه (۱). از آنجایی که آرایه به صورت نزولی مرتب است، در اولین اجرای partition، اولین عنصر آرایه یعنی ماکزیمم به عنوان pivot انتخاب می‌شود. سپس الگوریتم پارتیشن، ماکزیمم (عنصر اول) را با عنصر آخر یعنی مینیمم تعویض می‌کند و فقط همین یک تعویض در این مرحله انجام می‌شود. سپس به صورت بازگشتی  $Quicksort(A, p, q)$  برای  $n-1$

عنصر اول آرایه مجدداً پارتیشن اجرا می‌شود ولی این بار عنصر اول، مینیمم عنصر است که به عنوان محور انتخاب می‌شود و هیچ جابجایی صورت نمی‌گیرد و فقط عنصر اول از  $n-2$  عنصر بعدی جدا می‌شود و سپس با فراخوانی بازگشتی بعدی  $\text{Quicksort}(A, q+1, r)$  روی  $n-2$  عنصر مجدداً همانند مسئله اصلی، عنصر ماکزیمم، به عنوان محور انتخاب می‌شود و یک تعویض صورت می‌گیرد. یعنی به ازای هر ۲ عنصری که از آرایه کاسته می‌شود، یک تعویض صورت می‌گیرد. پس تعداد تعویض‌ها  $\frac{n}{2}$  است.

۳۳- گزینه (۲). اگر لیست صعودی باشد، محور عنصر مینیمم است و بعد از پارتیشن، سمت چپ محور خالی می‌ماند و الگوریتم روی  $n-1$  عنصر فراخوانی بازگشتی می‌کند. اگر لیست نزولی باشد، محور، عنصر ماکزیمم است و بعد از پارتیشن، سمت راست محور خالی می‌ماند و الگوریتم روی  $n-1$  عنصر فراخوانی بازگشتی می‌کند. در هر دو صورت زمان اجرا  $T(n) = T(n-1) + \theta(n)$  است که از مرتبه  $\theta(n^2)$  است.

۳۴- گزینه (۳). بعد از پارتیشن باید عناصر قبل از محور یعنی قبل از ۹ از ۹ کوچکتر و عناصر بعد از ۹ از ۹ بزرگتر باشند که فقط گزینه ۳ چنین است.

۳۵- گزینه (۲). بعد از انجام پارتیشن فرض می‌کنیم تعداد  $k$  عنصر سمت چپ محور و تعداد  $n-k-1$  عنصر سمت راست محور قرار می‌گیرند. حال روی هر دو قسمت فراخوانی بازگشتی صورت می‌گیرد و با توجه به اینکه الگوریتم پارتیشن زمان ثابت  $C$  دارد، برای زمان اجرا می‌توان رابطه  $T(n) = T(k) + T(n-k-1) + C$  را نوشت. این رابطه در بدترین حالت به صورت  $T(n) = T(0) + T(n-1) + C = T(n-1) + C$  است که از مرتبه  $\theta(n)$  است. و در بهترین حالت این رابطه تبدیل به  $T(n) = 2T\left(\frac{n}{2}\right) + C$  می‌شود که از مرتبه  $\theta(n)$  است. پس همیشه رابطه مذکور از مرتبه  $\theta(n)$  است.

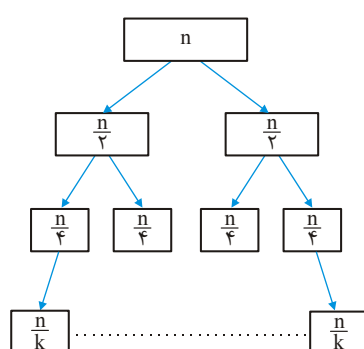
۳۶- گزینه (۱). در واقع باید الگوریتم مرتب‌سازی سریع را انجام دهیم ولی نه تا زمانی که به لیست‌های تک عنصری برسیم، بلکه تا زمانی باید فراخوانی انجام شود که به لیست‌های  $\frac{n}{k}$  عنصری برسیم. به این صورت عمل می‌کنیم که میانه را می‌بایم و با محوریت آن، آرایه را افراز می‌کنیم سپس به صورت بازگشتی روی هر دو نیمه فراخوانی انجام می‌دهیم تا زمانی که به لیست‌های  $\frac{n}{k}$  عنصری برسیم.

پس اگر  $T(n)$  زمان اجرا باشد می‌توان نوشت:

$$T(n) = \begin{cases} 2T\left(\frac{n}{2}\right) + cn & , n > 2^r \\ a & , n \leq 2^r \end{cases}$$

که  $a$  و  $c$  ثابت هستند. حال با جایگذاری داریم: ( $k = 2^r$ )

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + cn = 2\left[2T\left(\frac{n}{4}\right) + c\frac{n}{2}\right] + cn = 2^2T\left(\frac{n}{4}\right) + cn + cn \\ &= 2^2\left[2T\left(\frac{n}{8}\right) + \frac{cn}{2}\right] + cn + cn = 2^3T\left(\frac{n}{8}\right) + cn + cn + cn = \dots \\ &= 2^rT\left(\frac{n}{2^r}\right) + \underbrace{cn + cn + \dots + cn}_{r \text{ بار}} = 2^r \times a + c.n.r = k.a + c.n.\lg k = \theta(n.\lg k) \end{aligned}$$



برای درک بهتر، درخت مقابل را در نظر بگیرید  
ارتفاع این درخت  $\lg k$  است و در هر سطح زمان  
 $\theta(n)$  برای میانه‌یابی و افراز صرف می‌شود. پس  
کل زمان  $\theta(n.\lg k)$  است.

۳۷- گزینه (۳). اگر  $k = 1$  آنگاه  $A[1] \leq A[2] \leq A[3] \leq \dots \leq A[n]$  یعنی آرایه مرتب است. اگر  
 $k = 2$  آنگاه  $A[1] \leq A[3] \leq A[5] \leq \dots$  و  $A[2] \leq A[4] \leq A[6] \leq \dots$  یعنی آرایه از دو  
زیرآرایه مرتب هر یک به طول  $\frac{n}{2}$  تشکیل شده است. با شرط گفته شده در صورت سوال،  
آرایه  $A$  از  $k$  تا زیر آرایه مرتب به طول  $\frac{n}{k}$  تشکیل شده است. پس در واقع ما باید  $k$  تا  
لیست مرتب که هر یک طول  $\frac{n}{k}$  دارند را با هم ادغام کنیم که  $n$  عدد مرتب حاصل شود.  
می‌توان نشان داد مرتبه ادغام  $\Omega(n.\lg k)$  است، زیرا تعداد حالات ادغام  $k$  تا لیست مرتب  
 $\frac{n}{k}$  عنصری برابر  $\frac{n!}{\left(\left(\frac{n}{k}\right)!\right)^k}$  است. پس ارتفاع درخت تصمیم برای این منظور حداقل

$$\lg \frac{n!}{\left(\left(\frac{n}{k}\right)!\right)^k} \text{ است که برابر است با:}$$

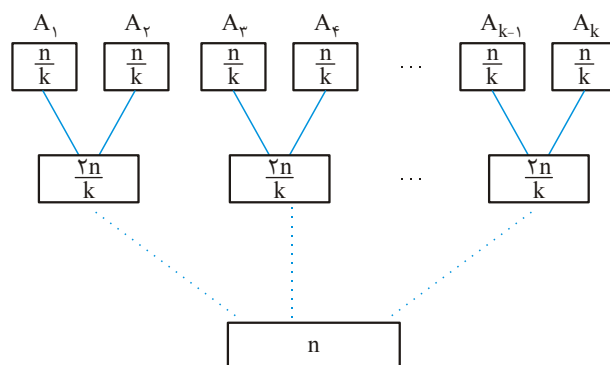
$$\lg n! - k \lg \left(\frac{n}{k}\right)! \approx n \lg n - k \frac{n}{k} \lg \frac{n}{k} = n \lg n - n(\lg n - \lg k) = n \lg k$$

پس الگوریتمی که بتواند  $k$  تا لیست مرتب  $\frac{n}{k}$  عنصری را ادغام کند از مرتبه  $\Omega(n.\lg k)$   
است. برای این منظور ما دو روش ارائه می‌دهیم:

**روش ۱:** با عناصر اول لیست‌ها یک مین هیپ بسازید، ارتفاع این هیپ  $\lg k$  است. سپس  
ریشه هیپ را حذف کرده و در خروجی قرار دهید و عنصر بعدی از لیستی که ریشه متعلق

به آن بوده را در هیپ درج کنید. جمعاً  $n$  بار باید از هیپ حذف و درج انجام دهید که مرتبه هر حذف و درج  $O(\lg k)$  است پس مرتبه کل  $O(n \lg k)$  است.

**روش ۲:** لیست ۱ و ۲ را با هم، لیست ۳ و ۴ را با هم و .... لیست  $k-1$  و  $k$  را با هم ادغام کنید. و به همین ترتیب عمل ادغام را ادامه دهید:



ارتفاع درخت فوق،  $\lg k$  است و در هر سطح زمان  $\theta(n)$  برای ادغام لازم است. پس مرتبه کل  $O(n \lg k)$  است.

۳۸- گزینه (۴). با توجه به الگوریتم sel که در درس دیدیم گزینه ۴ صحیح است.

۳۹- گزینه (۳). روش‌های مختلفی می‌توان ارائه داد. یک روش به این صورت است که با  $n$  عدد داده شده یک درخت جستجوی دودویی متوازن بسازیم ولی در هر نود تعداد تکرار آن را ذخیره کنیم. ارتفاع این درخت  $O(\lg n)$  خواهد بود و مرتبه ساخت این درخت  $O(n \lg n)$  است. سپس درخت را به صورت inorder پیمایش کنیم. مرتبه کل این عملیات  $O(n \lg n)$  است.

۴۰- گزینه (۴). Heapsort یک عنصر را حداکثر  $\lg n$  بار مقایسه می‌کند. در سه روش دیگر، جایگشتی وجود دارد که یک عنصر خاص با همه عناصر دیگر مقایسه شود.

۴۱- گزینه (۲). دو عنصر متوالی  $A[k-1]$  و  $A[k]$  با هم مقایسه می‌شوند و در صورت لزوم تعویض می‌شوند این عمل به ازای ۱ تا  $n-1$  انجام می‌شود سپس با فراخوانی  $\text{sort}(A, n-1)$  طول آرایه یک واحد کم می‌شود.

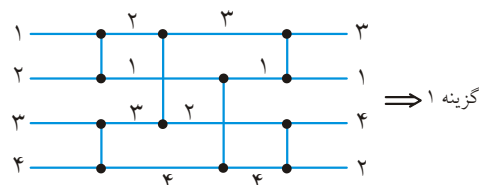
۴۲- گزینه (۱). آرایه ۱- مرتب، یک آرایه صعودی مرتب است. اما در آرایه ۲- مرتب، هر عنصر از عنصری که ۲ تا بعد از آن قرار دارد کوچک‌تر و از عنصری که ۲ تا قبل از آن قرار دارد بزرگ‌تر است. آرایه زیر یک آرایه  $2N$  عنصری ۲- مرتب است:

|   |   |       |       |       |       |       |       |     |      |
|---|---|-------|-------|-------|-------|-------|-------|-----|------|
| ۱ | ۲ | ۳     | ۴     | ۵     | ۶     | ۷     | ۸     | ... | $2N$ |
| x | y | $x+1$ | $y+1$ | $x+2$ | $y+2$ | $x+3$ | $y+3$ | ... |      |



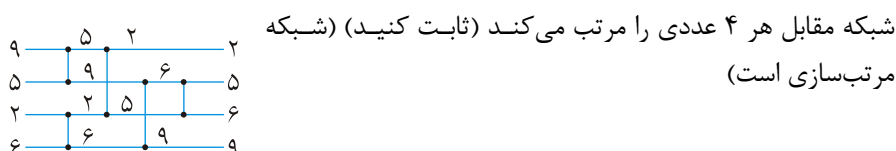
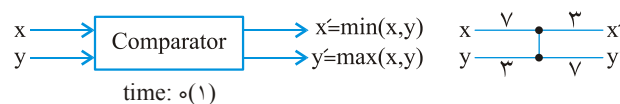
اگر در این آرایه، همه عناصر موجود در خانه‌های زوج ( $y$  و  $y+1$  و ...)، از  $x$  کوچکتر باشند آنگاه در آرایه مرتب همه قبل از  $x$  قرار می‌گیرند. و در این صورت حداکثر اختلاف اندیس بدست می‌آید که  $N$  است.

۴۳- گزینه (۴). در شکل زیر ۱ و ۲ را با هم جابجا کنید، ۳ و ۴ را مستقیم عبور دهید، ۲ و ۳ را جابجا کنید و ...



به همین ترتیب گزینه‌های ۲ و ۳ نیز حاصل می‌شوند.

**توجه:** چنین شکلهایی، شبکه‌های مقایسه‌ای نام دارند، این شبکه‌ها از یک مقایسه‌کننده و تعدادی ورودی خروجی تشکیل شده است. برخی از این شبکه‌ها می‌توانند هر ورودی را مرتب کنند که به آنها شبکه مرتب‌سازی گویند. شبکه‌های مرتب‌سازی براساس مقایسه (به صورت موازی) مرتب می‌کنند پس روشهایی مثل مرتب‌سازی شمارشی را نمی‌توان با این شبکه‌ها پیاده کرد. با شبکه مرتب‌سازی می‌توان با مرتبه  $O(\lg^2 n)$  مرتب کرد. به شکلهای زیر توجه کنید.



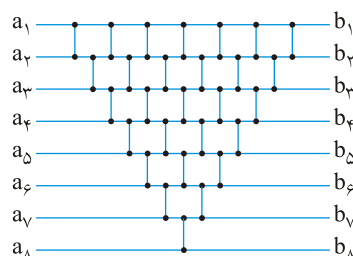
(۳ در شکل فوق) طولانی‌ترین مسیر مقایسه‌کننده‌ها = عمق = زمان اجرا

**تمرین:** فرض کنید  $n$  توانی از ۲ است. چگونه می‌توان یک شبکه مقایسه‌ای  $n$  ورودی  $n$  خروجی ساخت که عمقش  $\lg n$  باشد و مینیمم در سیم بالا و ماکزیمم در سیم پایین ظاهر شود.

**تمرین:** نشان دهید هر شبکه مرتب‌سازی با  $n$  ورودی عمقش حداقل  $\lg n$  است.

**تمرین:** نشان دهید تعداد مقایسه‌کننده‌ها در هر شبکه مرتب‌سازی  $\Omega(n \lg n)$  است.

**تمرین:** نشان دهید شبکه مرتب‌سازی زیر، به مرتب‌سازی درجی مرتبط است.



**قضیه:** هر شبکه مقایسه‌ای که  $n$  ورودی ۰ و ۱ داشته باشد و هر  $2^n$  حالت ورودی را مرتب کند، هر ورودی دلخواه را نیز مرتب می‌کند (اصل صفر - یک)

**قضیه:** اگر یک شبکه مقایسه ورودی  $a = (a_1, a_2, \dots, a_n)$  را به خروجی  $b = (b_1, b_2, \dots, b_n)$  تبدیل کند سپس برای هر تابع صعودی  $f$ ، این شبکه ورودی  $f(a) = (f(a_1), \dots, f(a_n))$  را به خروجی  $f(b) = (f(b_1), \dots, f(b_n))$  تبدیل می‌کند.

**تمرین:** ثابت کنید یک شبکه مقایسه با  $n$  ورودی به درستی دنباله  $\langle n, n-1, \dots, 1 \rangle$  را مرتب می‌کند اگر و فقط اگر  $n-1$  دنباله حاوی ۰ و ۱ زیر را مرتب کند:

$$\langle 1, 0, 0, \dots, 0 \rangle, \langle 1, 1, 0, \dots, 0 \rangle, \dots, \langle 1, 1, \dots, 1, 0 \rangle$$

**تمرین:** ثابت کنید یک شبکه مرتب‌سازی  $n$  ورودی باید شامل حداقل یک مقایسه‌کننده بین  $i$  امین و  $i+1$  امین خط برای هر  $i = 1, 2, \dots, n-1$  باشد.

**۴۴- گزینه (۴).** کفگیر را زیر کوچکترین کیک قرار می‌دهیم و وارون می‌کنیم تا کوچکترین کیک به بالا منتقل شود، سپس تمام کیکها را وارون می‌کنیم تا کوچکترین کیک پایین قرار گیرد این عمل را برای  $n-1$  کیک انجام می‌دهیم یعنی هر کیک ۲ وارون می‌خواهد پس کل عملیات وارون  $2(n-1)$  می‌باشد. البته گزینه ۴ نیز صحیح نیست زیرا وقتی  $n-2$  عنصر با  $2(n-2)$  عمل وارون مرتب شدند، ۲ عنصر باقیمانده حداکثر با یک وارون مرتب می‌شوند. یعنی  $2(n-2) + 1 = 2n - 3$  عمل وارون لازم است!

**۴۵- گزینه (۲).** اگر  $n = 2$ ، حداکثر یک تعویض انجام می‌شود. اگر  $n = 3$  باشد حداکثر ۲ تعویض و به طور کلی برای  $n$ ،  $n-1$  تعویض انجام می‌شود تا آرایه صعودی شود. مثلاً:

$$n = 3, A = \begin{bmatrix} 2 & 3 & 1 \end{bmatrix} \begin{cases} i = 1 \Rightarrow \text{swap}(A[1], A[2]), \text{swap}(A[1], A[3]) \\ i = 2 \Rightarrow \text{تعویض ندارد} \end{cases}$$

**۴۶- گزینه (۳).** تعداد جایگشت‌های  $n$  عنصر برابر  $n!$  است و احتمال اینکه در عمل پارتیشن هر بار می‌نیمم انتخاب شود  $\frac{1}{n!}$  است. البته احتمال آنکه هر بار مینی‌م یا ماکسی‌م انتخاب شوند  $\frac{2^n}{n!}$  است که این نیز بدترین حالت است.

۴۷- گزینه (۱). یک روش به این صورت است که با  $n$  عدد یک BST متوازن که در هر نودش تعداد تکرار نیز ذخیره می‌شود، بسازیم که ارتفاع این BST برابر  $\lg k$  و مرتبه ساخت آن  $n \lg k$  است و سپس آن را inorder پیمایش کنیم که مرتبه آن  $O(n)$  است پس مرتبه کل  $O(n \cdot \lg k)$  است.

۴۸- گزینه (۲).

۱ و ۵ و ۱۱ و ۱۵ و ۱۹ و ۲۶ و ۴۸ و ۵۹ و ۶۱ و ۷۷: داده ورودی  
 ۱ و ۵ و ۱۱ و ۱۵ و ۱۹ و ۲۶ و ۴۸ و ۵۹ و ۷۷ و ۶۱: مرحله اول ادغامی (ادغام تک عنصرها)  
 ۱ و ۵ و ۱۹ و ۲۶ و ۱۵ و ۱۱ و ۷۷ و ۶۱ و ۵۹ و ۴۸: مرحله دوم ادغامی (ادغام دو عنصرها)  
 ۴۹- گزینه (۴). مسأله معادل پیدا کردن  $k$  امین عنصر کوچک‌تر در آرایه‌ای نامرتب است که بهترین الگوریتم برای این منظور الگوریتم Sel است که خود از الگوریتم Partition استفاده می‌کند. مرتبه زمانی الگوریتم Sel  $O(n)$  است.  
 ۵۰- گزینه (۳). آرایه نزولی S را در نظر بگیرید:

| [۱] | [۲] | [۳] | [۴] | [۵] |
|-----|-----|-----|-----|-----|
| 5   | 4   | 3   | 2   | 1   |

اگر  $k=7$  فرض شود، در شروع  $i=1$  و  $j=5$  است و چون  $S[1]+S[5]=6 < 7$  آنگاه  $i=2$  می‌شود حال چون  $S[2]+S[5]=5 < 7$ ،  $i=3$  می‌شود و  $S[3]+S[5]=4 < 7$  و  $i=4$  می‌شود و  $S[4]+S[5]=5 < 7$  و حال  $i=5$  می‌شود و  $S[5]+S[5]=2 < 7$  و  $i=6$  می‌شود و الگوریتم خاتمه می‌یابد و اعلام می‌کند «امکان‌پذیر نیست». حال آنکه  $S[1]+S[4]=7$  پس برای آرایه نزولی ممکن است الگوریتم درست جواب ندهد.  
 ولی اگر آرایه صعودی باشد همیشه الگوریتم درست جواب می‌دهد. زیرا اگر  $S[i]+S[j]=k$  باشد که الگوریتم جواب می‌دهد و اگر  $S[i]+S[j] < k$  باشد آنگاه  $i$  افزایش می‌یابد تا وقتی که احتمالاً  $S[i]+S[j]=k$  شود و یا اعلام شود که جواب وجود ندارد. و اگر  $S[i]+S[j] > k$  باشد آنگاه  $j$  کم می‌شود تا به جواب برسد.

۵۱- گزینه (۴). در این الگوریتم ۲ حالت امکان‌پذیر است یا همان بار اول اجرای حلقه،  $k=1$  شده و آرایه مرتب می‌شود و از حلقه خارج می‌شود و یا اینکه آرایه ورودی به آرایه دیگری تبدیل می‌شود و در بار دوم آرایه جدید مجدداً به آرایه اول تبدیل می‌شود و حلقه هیچ‌گاه پایان نمی‌یابد.

۵۲- گزینه (۳). متوسط تعداد فراخوانی  $\log_2^n$  می‌باشد که چون  $n=2000$  پس تعداد فراخوانی متوسط  $\log_2^{2000}$  یعنی حدود ۱۱ بار و چون هر فراخوانی ۴ بایت از پشته را اشغال می‌کند، گزینه «۳» حاصل می‌شود.

۵۳- گزینه (۱). ابتدا با عددگذاری مسئله را حل می‌کنیم به ازای  $n=1$  هیچ مقایسه‌ای لازم نیست. به ازای  $n=2$ ، یعنی ماتریس  $2 \times 2$ ، با ۳ مقایسه می‌توان ماکزیمم یافت به ازای  $n=4$ ، یعنی ماتریس  $4 \times 4$ ، ماتریس را به ۴ ماتریس  $2 \times 2$  تقسیم کرده، در هر ماتریس  $2 \times 2$  با ۳ مقایسه ماکزیمم پیدا می‌شود حال با ۳ مقایسه از بین ۴ ماکزیمم، ماکزیمم یافت می‌شود یعنی جمعاً ۱۵ مقایسه. با بررسی گزینه‌ها، این اعداد در گزینه ۱ صدق می‌کنند. ولی حل دقیق این سوال به این صورت است که فرض کنید  $T(n)$  تعداد مقایسه‌های ماتریس  $n \times n$  باشد، ماتریس به ۴ تا ماتریس  $\frac{n}{4} \times \frac{n}{4}$  تقسیم می‌شود که در هر یک با  $T(\frac{n}{4})$  مقایسه، ماکزیمم یافت می‌شود و در نهایت با ۳ مقایسه از بین ۴ ماکزیمم، ماکزیمم نهایی یافت می‌شود پس:

$$\begin{cases} T(n) = 4T\left(\frac{n}{4}\right) + 3 = n^2 - 1 = \theta(n^2) \\ T(1) = 0 \end{cases}$$

۵۴- گزینه (۱). برای آرایه مرتب هیچ جابجایی صورت نمی‌گیرد (گزینه ۳ غلط). برای آرایه معکوس مرتب به مثال زیر توجه کنید:

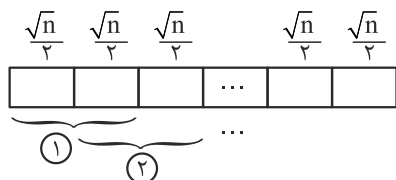
$$\begin{array}{l} A \begin{array}{cccc} 1 & 2 & 3 & 4 \\ 4 & 3 & 2 & 1 \end{array} \\ i=1: A[1] \neq 1 \rightarrow \text{swap}(A[1], A[4]) \begin{array}{cccc} 1 & 2 & 3 & 4 \\ 1 & 3 & 2 & 1 \end{array} \\ i=2: A[2] \neq 2 \rightarrow \text{swap}(A[2], A[3]) \begin{array}{cccc} 1 & 2 & 3 & 4 \\ 1 & 3 & 2 & 4 \end{array} \end{array}$$

و دیگر هیچ جابجایی انجام نمی‌شود. در واقع برای آرایه معکوس مرتب  $\left\lfloor \frac{n}{2} \right\rfloor$  جابجایی صورت می‌گیرد حال آنکه آرایه‌ای وجود دارد که  $n-1$  جابجایی انجام می‌شود مثلاً:

$$\begin{array}{l} A \begin{array}{cccc} 1 & 2 & 3 & 4 \\ 3 & 2 & 4 & 1 \end{array} \\ i=1: A[1] \neq 1 \rightarrow \text{swap}(A[1], A[2]) \begin{array}{cccc} 3 & 2 & 4 & 1 \\ 4 & 3 & 2 & 1 \end{array} \\ A[1] \neq 1 \rightarrow \text{swap}(A[1], A[3]) \begin{array}{cccc} 4 & 2 & 3 & 1 \\ 4 & 3 & 2 & 1 \end{array} \\ A[1] \neq 1 \rightarrow \text{swap}(A[1], A[4]) \begin{array}{cccc} 1 & 2 & 3 & 4 \\ 4 & 3 & 2 & 1 \end{array} \end{array}$$

و دیگر هیچ جابجایی انجام نمی‌شود.

۵۵- گزینه (۱). با حداکثر  $\theta(n)$  فراخوانی الگوریتم گفته شده، می‌توان آرایه را sort کرد. روش به این صورت است که تابع روی  $\sqrt{n}$  عنصر بعدی تابع را صدا کنیم و به همین ترتیب پس از



$2\sqrt{n} - 1$  فراخوانی  $\frac{\sqrt{n}}{2}$  عنصر بزرگ آرایه به صورت مرتب در انتهای آرایه قرار می‌گیرند. حال همین داستان را دوباره از اول تکرار می‌کنیم و این بار با  $2\sqrt{n} - 2$  فراخوانی  $\frac{\sqrt{n}}{2}$  عنصر دیگر سر جای خود قرار می‌گیرند و مجدد از اول ... تعداد کل فراخوانی برابر است با:

$$1 + 2 + \dots + (2\sqrt{n} - 1) = \frac{(2\sqrt{n} - 1)(2\sqrt{n})}{2} = \theta(n)$$

۵۶- گزینه (۳). یک گوی دلخواه برداشته و آن را با همه مقایسه کنید بدین ترتیب معلوم می‌شود چه گوی‌هایی با هم هم‌بار هستند و چون تعداد گوی‌های با بار مثبت زوج است، دسته‌ای که تعداد گوی‌های زوج دارد مثبت هستند.

۵۷- گزینه (۱). ابتدا میانه را با مرتبه  $O(n)$  می‌یابیم و با محوریت میانه، آرایه را پارتیشن می‌کنیم. حال  $\frac{k}{2}$  امین کوچکترین عنصر را در سمت چپ و در سمت راست میانه با مرتبه  $O(n)$  می‌یابیم و پارتیشن می‌کنیم اکنون  $\frac{k}{2}$  عنصر اول آرایه و  $\frac{k}{2}$  عنصر بعد از میانه  $a$ ، جمعاً  $k$  عنصر هستند که میانه آنها  $a$  است.

۵۸- گزینه (۴). الف درست و ب نادرست است.

۵۹- گزینه (۳). فقط ج نادرست است. زیرا در صورت صحیح بودن، مرتبه مرتب‌سازی مقایسه‌ای از  $n \lg n$  کمتر می‌شود. هزینه  $n$  عمل درج (هزینه ساخت) بعلاوه هزینه  $n$  عمل حذف در روش‌های مقایسه‌ای نمیتواند از  $n \lg n$  بهتر شود.

۶۰- گزینه (۱). ادغام دو لیست  $m$  و  $n$  عنصری حداقل  $\min\{m, n\}$  و حداکثر  $m + n - 1$  مقایسه می‌خواهد.

۶۱- گزینه (۱). با مرتبه  $\theta(n)$  میانه را یافته و افراز می‌کنیم سپس عناصر نیمه اول آرایه را در مکان‌های فرد و عناصر نیمه دوم را در مکان‌های زوج قرار می‌دهیم که با مرتبه  $\theta(n)$  انجام می‌شود.

۶۲- گزینه (۱).  $n - \lg n$  امین مینیمم را یافته و افراز می‌کنیم که با مرتبه  $\theta(n)$  انجام می‌شود و سپس  $\lg n$  عنصر انتهای آرایه را با مرتبه  $\theta(\lg n)$  جمع می‌کنیم. پس مرتبه کل  $\theta(n + \lg n) = \theta(n)$  است.

- ۶۳- گزینه (۱). احتمال آنکه اندازه بخش کوچکتر کمتر یا مساوی  $\alpha n$  باشد برابر  $2\alpha$  است زیرا برای آنکه این اتفاق بیافتد، محور باید اولین یا دومین یا ... یا  $\alpha n$  امین مینیمم یا ماکزیمم باشد، یعنی برای محور  $2\alpha n$  حالت وجود دارد پس احتمال آن  $\frac{2\alpha n}{n} = 2\alpha$  است. پس احتمال آنکه اندازه بخش کوچکتر، بیش از  $\alpha n$  باشد،  $1 - 2\alpha$  است.
- ۶۴- گزینه (۲). یک روش این است که  $i$  را برابر ۱ و  $j$  را برابر  $n$  قرار دهیم (فرض می‌کنیم آرایه  $A[1..n]$  است). سپس در یک حلقه،  $A[i] + A[j]$  را محاسبه کنیم، اگر برابر  $C$  بود که کار تمام است، اگر کمتر از  $C$  بود،  $i$  را افزایش دهیم و اگر بیشتر از  $C$  بود،  $j$  را کاهش دهیم و این عمل را تا زمانی انجام دهیم که  $i < j$ . مرتبه این عمل به وضوح  $O(n)$  است.
- ۶۵- گزینه (۱). کافیس  $k$  امین کوچکترین عنصر را یافته و با محوریت آن، آرایه را افراز کنیم که مرتبه این عمل  $O(n)$  است و سپس  $k$  عنصر ابتدای آرایه را مرتب کنیم که با مرتبه  $O(k \lg k)$  انجام می‌شود. پس کل عملیات از مرتبه  $O(n + k \lg k)$  است.
- ۶۶- گزینه (۴). حداقل مقایسه در بدترین حالت  $\lceil \lg 5! \rceil = 7$  است.
- ۶۷- گزینه (۲). گزاره اول غلط است. مثلاً فرض کنید دو نیمه آرایه مرتب شده‌اند و حال قرار است ادغام شوند. اگر همه عناصر نیمه اول از همه عناصر نیمه دوم بزرگتر باشند آنگاه اولین عنصر نیمه اول با همه عناصر نیمه دوم مقایسه می‌شود یعنی عنصری وجود دارد که با  $\theta(n)$  عنصر مقایسه می‌شود.
- گزاره دوم درست است چون مرتبه مرتب‌سازی ادغامی  $O(n \lg n)$  است پس به طور متوسط هریک از  $n$  عنصر با  $\lg n$  عنصر مقایسه می‌شوند.
- گزاره سوم درست است زیرا اگر همه اعداد با کمتر از  $\lg n$  عنصر مقایسه شوند آنگاه مرتبه مرتب‌سازی از  $n \lg n$  کمتر می‌شود که غیر ممکن است.
- ۶۸- گزینه (۳). به جز گزینه ۱ سایر گزینه‌ها را می‌توان ساخت زیرا هزینه ساخت بعلاوه هزینه  $n$  بار استخراج کمینه نمی‌تواند از  $n \lg n$  بهتر شود.
- ۶۹- گزینه (۱). باید همه عناصر سمت چپ محور از محور کوچکتر باشند و همه عناصر سمت راست محور از محور بزرگتر باشند که ۷ و ۹ می‌توانند محور باشند.
- ۷۰- گزینه (۴).
- ۷۱- گزینه (۴). مرتب‌سازی درجی مناسب‌ترین است.
- ۷۲- گزینه (۱). با پیمایش خطی آرایه و به عنصر  $A[k]$  که رسیدیم عنصر کمینه تا آن عنصر و تفاضل آن با کمینه یافته شده تا آن را بیابیم.

۷۳- گزینه (۱). چون اعداد صحیح هستند می‌توان با مرتبه  $O(n)$  شمارش کرد که از هر عدد چند تا وجود دارد.

۷۴- گزینه (۱). اگر در هر مرحله پارتیشن، میانه را به عنوان محور انتخاب کنیم آنگاه مرتب‌سازی سریع همواره دارای زمان  $T(n) = 2T(\frac{n}{4}) + \theta(n)$  یعنی  $\theta(n \lg n)$  خواهد بود.

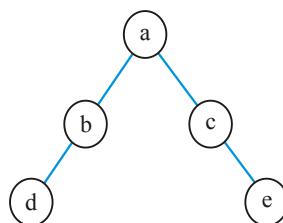
## فصل ۶

## مباحثی از درخت‌ها

## پیمایش درخت دودویی

درخت دودویی دو نوع پیمایش سطحی و عمقی دارد. پیمایش سطحی همان ترتیب ذخیره درخت در آرایه است یعنی از ریشه شروع می‌کنیم و از چپ به راست به ترتیب سطحی نودها را ملاقات می‌کنیم. برای پیاده‌سازی این نوع پیمایش از یک صف استفاده می‌شود.

**مثال ۱:** پیمایش سطحی درخت دودویی مقابل عبارت است از: a b c d e



پیمایش عمقی ۶ نوع است. اگر ملاقات ریشه را با D و حرکت به سمت چپ را با L و حرکت به راست را با R نشان دهیم، این ۶ پیمایش عبارتند از:

DLR : rT<sub>l</sub>T<sub>r</sub>

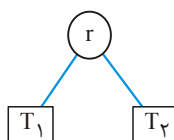
LDR : T<sub>l</sub> rT<sub>r</sub>

LRD : T<sub>l</sub>T<sub>r</sub>r

RLD : T<sub>r</sub>T<sub>l</sub>r

RDL : T<sub>r</sub>rT<sub>l</sub>

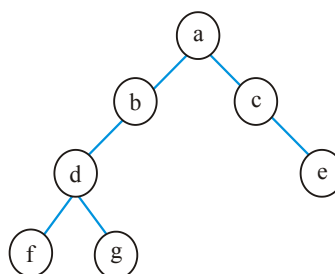
DRL : rT<sub>r</sub>T<sub>l</sub>





از این ۶ پیمایش، ۳ پیمایش اول بیشتر استفاده می‌شوند و ۳ پیمایش آخر به ترتیب معکوس ۳ پیمایش اول هستند، مثلاً RLD معکوس DLR می‌باشد.

DLR (preorder): a b d f g c e  
 LDR (inorder): f d g b a c e  
 LRD (postorder): f g d b e c a  
 RLD (reverse preorder): e c g f d b a  
 RDL (reverse inorder): e c a b g d f  
 DRL (reverse postorder): a c e b d g f



مثال ۲:

اگر هر گره درخت دودویی را ساختاری شامل داده، اشاره گر به فرزند چپ و اشاره گر به فرزند راست در نظر بگیریم: 

|        |      |        |
|--------|------|--------|
| Lchild | data | Rchild |
|--------|------|--------|

 و با توجه به اشاره گر به ریشه درخت که داده خواهد شد، پیمایش میان ترتیب درخت دودویی به شکل زیر است:

```

void inorder (pointer ptr)
{
    if (ptr != NULL) {
        1) inorder (ptr → Lchild);
        2) printf ("%d" , ptr → data);
        3) inorder (ptr → Rchild);
    }
}
  
```

به همین ترتیب می‌توان با جابجایی سطرهای ۱ و ۲ و ۳ الگوریتم فوق، سایر پیمایش‌های عمقی را نوشت.

## نکات

- ۱- همه پیمایش‌های درخت دودویی دارای مرتبه  $\theta(n)$  هستند.
- ۲- فضای لازم (پشته) برای پیمایش‌های عمقی از مرتبه  $\theta(h)$  می‌باشد که  $h$  ارتفاع درخت است و چون  $h \leq n$  پس مرتبه حافظه  $O(n)$  می‌باشد.
- ۳- با داشتن پیمایش‌های  $inorder$  و  $preorder$  یا پیمایش‌های  $inorder$  و  $postorder$  یا پیمایش‌های  $inorder$  و  $levelorder$  می‌توان درخت را به صورت منحصر به فرد رسم کرد.
- ۴- با داشتن پیمایش‌های  $preorder$  و  $postorder$  و با فرض اینکه  $k$  تا گره تک فرزندی وجود دارد می‌توان  $2^k$  تا درخت مختلف رسم کرد. بنابراین اگر گره تک فرزندی وجود نداشته باشد می‌توان درخت را منحصر به فرد رسم کرد.

- ۵- با داشتن پیمایش preorder یا postorder و با مشخص بودن برگ‌ها و با فرض اینکه گره تک فرزندی وجود ندارد می‌توان درخت را منحصر به فرد رسم کرد.
- ۶- تنها درختی که پیمایش‌های preorder و inorder آن مساوی است، درخت مورب راست است.
- ۷- تنها درختی که پیمایش‌های postorder و inorder آن مساوی است، درخت مورب چپ است.

## Heap

یک هیپ (دودویی) (به طور دقیق Maxheap) یک درخت کامل (Complete) است که مقدار هر نود آن بیشتر یا مساوی مقدار فرزندان‌ش می‌باشد.<sup>۱</sup>

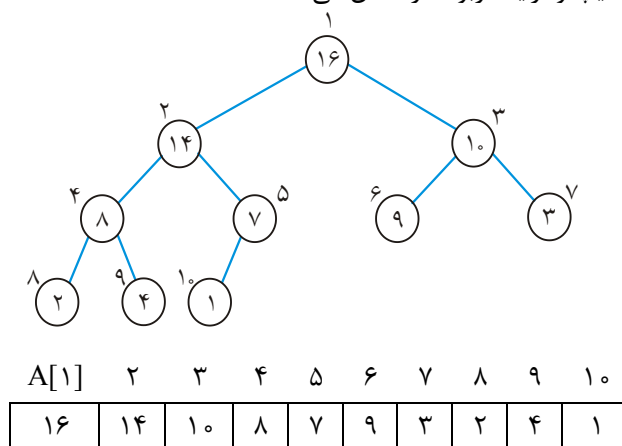
یک هیپ  $n$  نودی را می‌توان با یک آرایه  $A[1..n]$  نشان داد که روابط

$\text{PARENT}(i) \quad \text{return } \lfloor \frac{i}{2} \rfloor$  پدر و فرزندی در آرایه به شکل روبرو است:

$\text{LEFT}(i) \quad \text{return } 2i$  البته پدر  $i$  به شرطی  $\lfloor \frac{i}{2} \rfloor$  است که  $i \neq 1$ . فرزند چپ  $i$  به شرطی  $2i$

$\text{RIGHT}(i) \quad \text{return } 2i + 1$  است که  $2i \leq n$ . فرزند راست  $i$  به شرطی  $2i + 1 \leq n$  است که  $2i + 1 \leq n$  باشد.

شکل زیر یک هیپ و آرایه مربوطه را نشان می‌دهد:



تعداد عناصر موجود در heap را  $\text{heap\_size}$  و طول آرایه را  $\text{Length}$  می‌گوییم. در شکل فوق داریم  $\text{heap\_size}[A] = \text{Length}[A] = 10$ . همیشه  $\text{heap\_size} \leq \text{Length}$  می‌باشد.

### نکته

- ۱- در heap (Maxheap) بزرگترین مقدار در ریشه است و کوچکترین مقدار در یکی از برگ‌هاست. (می‌دانیم در درخت کامل با  $n$  گره،  $\lfloor \frac{n}{2} \rfloor$  برگ وجود دارد).

۱ - در Minheap مقدار هر نود کوچکتر یا مساوی مقدار فرزندان‌ش است. منظور ما از heap ، maxheap است.

- ۲- در Minheap کوچکترین نود ریشه است و بزرگترین در یکی از برگهاست.  
 ۳- یک آرایه نزولی یک هیپ است ولی عکس آن لزوماً صحیح نیست.  
 ۴- یک آرایه صعودی یک Minheap است ولی عکس آن لزوماً صحیح نیست.

## زیربرنامه HEAPIFY

این زیر برنامه به صورت  $\text{HEAPIFY}(A, i)$  فراخوانی می‌شود.  
 این زیر برنامه فرض می‌کند زیر درخت‌های با ریشه  $\text{LEFT}(i)$  و  $\text{RIGHT}(i)$  خود هیپ هستند ولی  $A[i]$  ممکن است از فرزندانش کوچکتر باشد و خاصیت هیپ را نقض کرده باشد. این زیر برنامه،  $A[i]$  را با فرزندانش مقایسه می‌کند و اگر از فرزندانش کوچکتر بود، آن را با بزرگترین فرزندش تعویض می‌کند و این عمل را آنقدر انجام می‌دهد تا خاصیت هیپ برقرار شود:

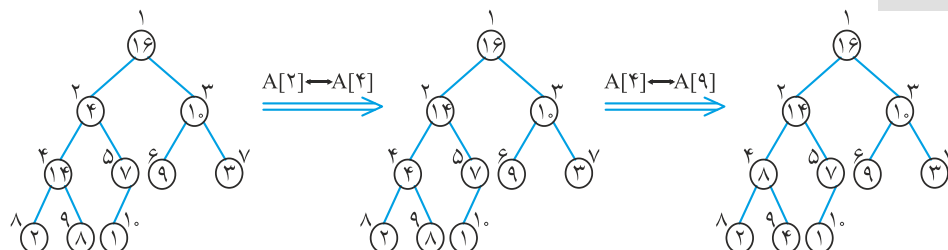
```

HEAPIFY(A, i)
{
    L = LEFT(i)
    r = RIGHT(i)
    if (L ≤ heap_size[A] and A[L] > A[i])
        Largest = L else Largest = i
    if (r ≤ heap_size[A] and A[r] > A[Largest]) Largest = r
    if (Largest ≠ i)
        {exchange A[i] ↔ A[Largest]}
        HEAPIFY(A, Largest)
}

```

با توجه به الگوریتم، در هر مرحله بزرگترین عنصر از بین  $A[\text{LEFT}(i)]$  و  $A[i]$  و  $A[\text{RIGHT}(i)]$  مشخص می‌شود و اندیسش در متغیر largest قرار می‌گیرد. اگر  $A[i]$  بزرگترین باشد آنگاه هیچ تعویضی نیاز نیست و خاصیت هیپ برقرار است و الگوریتم تمام می‌شود در غیر این صورت  $A[i]$  با بزرگترین فرزندش تعویض می‌شود و حال نود با اندیس largest ممکن است خاصیت هیپ را نقض کرده باشد که با فراخوانی بازگشتی، روال فوق دوباره انجام می‌شود.

**مثال ۳:**  $\text{HEAPIFY}(A, 2)$  را روی درخت زیر انجام می‌دهیم:



## نکته

اگر  $T(n)$  زمان اجرای الگوریتم HEAPIFY باشد، آنگاه  $T(n) \leq T\left(\frac{2n}{3}\right) + \theta(1)$ .

در این رابطه  $\theta(1)$  زمان مقایسه و تعویض نود  $i$  با فرزندانش است، و  $T\left(\frac{2n}{3}\right)$  زمان اجرای HEAPIFY روی یکی از زیر درختان نود  $i$  است، زیرا حداکثر  $\frac{2n}{3}$  گره در زیر درخت چپ  $i$  وجود دارد (بدترین حالت وقتی پیش می‌آید که نصف سطح آخر درخت پر باشد). با حل رابطه بازگشتی طبق قضیه Master داریم  $T(n) = O(\lg n)$ .

## ساخت هیپ

فرض کنید  $A[1..n]$  یک آرایه است و می‌خواهیم آن را تبدیل به هیپ کنیم. می‌دانیم عناصر  $A\left[\left(\left\lfloor \frac{n}{2} \right\rfloor + 1\right) .. n\right]$  برگ هستند. پس کفایست از  $A\left[\left\lfloor \frac{n}{2} \right\rfloor\right]$  شروع کنیم و تا  $A[1]$  رویه HEAPIFY را فراخوانی کنیم. یعنی از پایین به بالا هیپ را بسازیم:

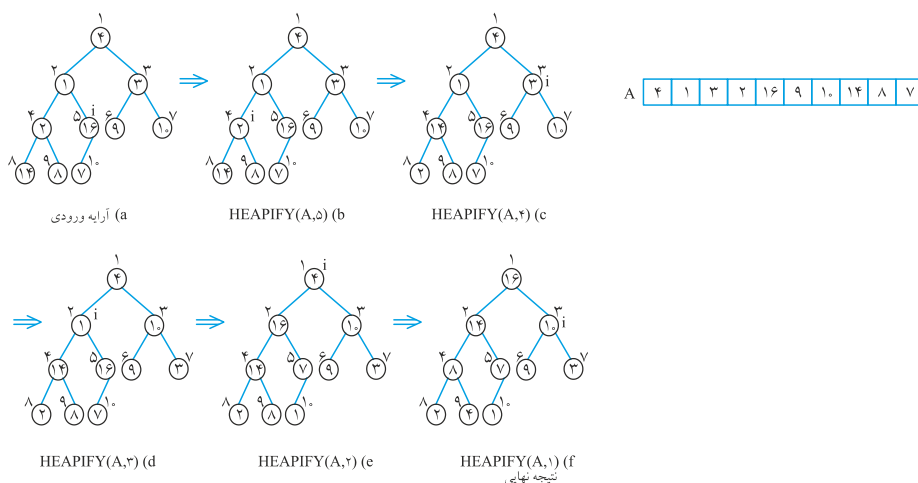
BUILD-HEAP( $A$ )

heap\_size[ $A$ ] = length[ $A$ ]

for ( $i = \left\lfloor \frac{\text{length}[A]}{2} \right\rfloor$  downto 1)

HEAPIFY( $A, i$ )

**مثال ۴:** فرض کنید آرایه ورودی  $A$  باشد. مراحل ساخت هیپ را نشان می‌دهیم.



## نکته

می‌توان نشان داد ساخت هیپ گفته شده از مرتبه خطی یعنی  $O(n)$  است.

## مرتب‌سازی heapsort

ابتدا با آرایه ورودی  $A[1..n]$  که  $n = \text{length}[A]$ ، هیپ می‌سازیم. سپس  $A[1]$  را با  $A[n]$  تعویض می‌کنیم که بزرگترین عنصر به آخر آرایه منتقل می‌شود. حال  $\text{heap\_size}[A]$  را یک واحد کم می‌کنیم که عنصر آخر از هیپ خارج شود و سپس عمل HEAPIFY را روی  $A[1]$  انجام می‌دهیم. سپس عمل فوق را روی هیپ  $n-1$  عنصری تا هیپ ۲ عنصری انجام می‌دهیم:

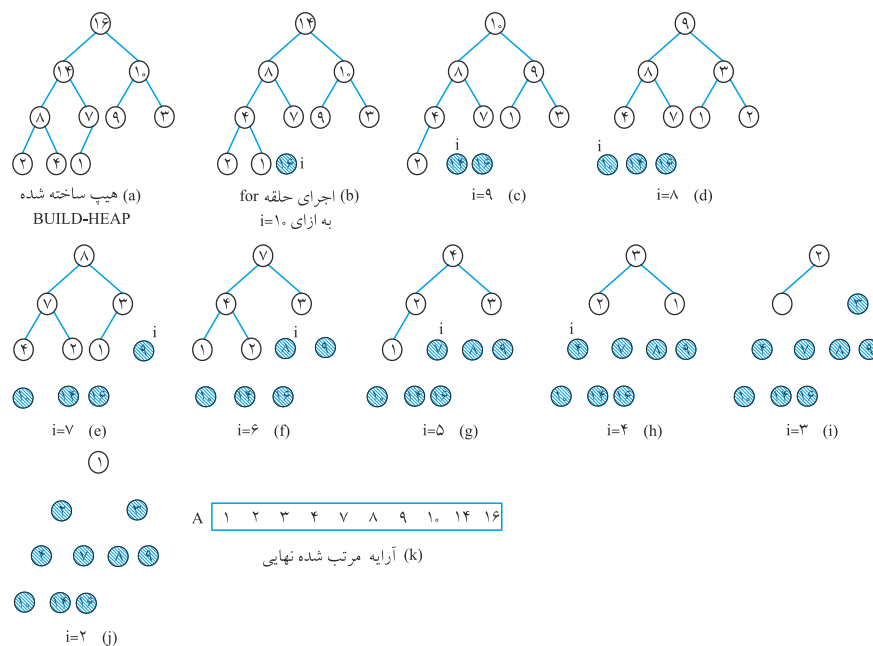
```

HEAPSORT(A)
  BUILD_HEAP(A)
  for (i=length[A] downto ۲)
    { exchange  $A[1] \leftrightarrow A[i]$ 
      heap_size[A]=heap_size[A]-۱
      HEAPIFY(A,1)
    }

```

مرتبه الگوریتم فوق  $O(n \lg n)$  است.

**مثال ۵:** در مثال ۴، هیپ ساختیم، حال می‌خواهیم حلقه for الگوریتم HEAPSORT را روی heap ساخته شده اعمال کنیم:



## صف اولویت priority queue

صف اولویت ساختمان داده‌ای برای نگهداری یک مجموعه  $S$  از عناصر است که هر عنصر دارای مقدار  $key$  می‌باشد. در صف اولویت همیشه ابتدا عنصری سرویس می‌گیرد و از صف خارج می‌شود که اولویت ( $key$ ) بیشتری دارد و اگر دو عنصر اولویت یکسان داشته باشند، ابتدا عنصری سرویس می‌گیرد که زودتر به صف وارد شده باشد.

عملیات زیر روی صف اولویت انجام می‌شود:

**INSERT ( $S, x$ )**: عنصر  $x$  را در مجموعه  $S$  درج می‌کند. این عمل به صورت  $S \leftarrow S \cup \{x\}$

نوشته می‌شود.

**MAXIMUM ( $S$ )**: عنصری که دارای بزرگترین  $key$  باشد را برمی‌گرداند.

**EXTRACT-MAX( $S$ )**: عنصری که دارای بزرگترین  $key$  باشد را حذف می‌کند و

برمی‌گرداند.

یک هیپ یک صف اولویت است. عمل MAXIMUM را می‌توان با زمان  $\theta(1)$  انجام داد، کافیس ریشه ( $A[1]$ ) را برگردانیم. عمل EXTRACT-MAX به شکل زیر است:

```
EXTRACT_MAX (A)
  if (heap_size[A] < 1) error "heap underflow"
  max = A[1]
  A[1] = A[heap_size[A]]
  heap_size[A] = heap_size[A] - 1
  HEAPIFY (A, 1)
  return max
```

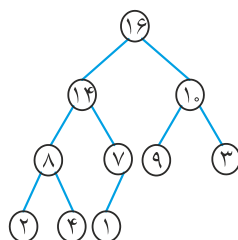
مرتبه زمانی این الگوریتم  $O(\lg n)$  می‌باشد.

برای عمل HEAP-INSERT، عنصر جدید را انتهای هیپ قرار می‌دهیم و سپس آن را با اجدادش مقایسه می‌کنیم و اگر از اجدادش بزرگتر بود، تعویض می‌کنیم:

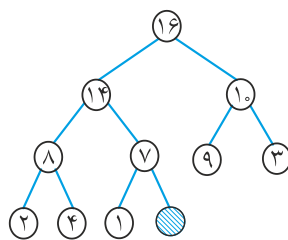
```
HEAP_INSERT (A, Key)
  heap_size[A] = heap_size[A] + 1
  i = heap_size[A]
  while (i > 1 and A[PARENT(i)] < key)
    { A[i] = A[PARENT(i)]
      i = PARENT(i) }
  A[i] = key
```

مرتبه زمانی درج در هیپ  $O(\lg n)$  می‌باشد.

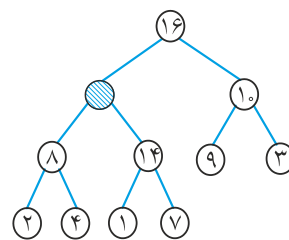
**مثال ۶:** عنصر با کلید ۱۵ را در هیپ مقابل درج می‌کنیم:



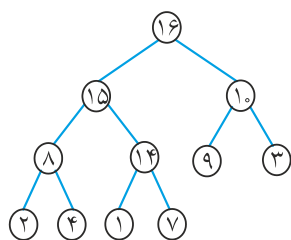
(a) هیپ اولیه



(b)  $\text{heap-size} \leftarrow \text{heap-size} + 1$



(c) ۱۵ از ۷ و ۱۴ بزرگتر است و حلقه while ۲ بار اجرا می‌شود.



(d) کلید ۱۵ درج می‌شود  
(خط ۶ الگوریتم)

### نکته

می‌توان با استفاده از الگوریتم HEAP-INSERT نیز هیپ ساخت. بنابراین الگوریتم دوم ساخت هیپ:

BUILD\_HEAP'(A)

heap\_size[A] = 1

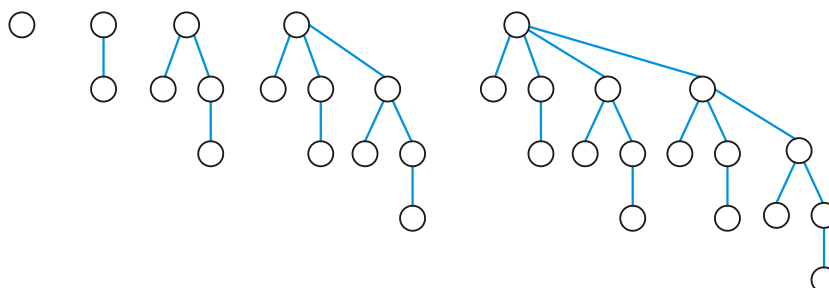
for (i = 2 to length[A])

HEAP\_INSERT(A, A[i])

می‌توان نشان داد که مرتبه این الگوریتم  $O(n \lg n)$  می‌باشد و همچنین هیپ ساخته شده با این الگوریتم و الگوریتم BUILD\_HEAP لزوماً یکسان نیستند. این روش وقتی استفاده می‌شود که اعداد ورودی همگی در اختیار نباشند و یکی یکی وارد شوند.

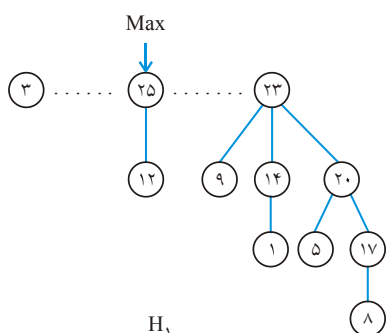
### هیپ دو جمله‌ای (Binomial heap)

ابتدا درخت دو جمله‌ای (Binomial tree) را تعریف می‌کنیم. درخت دو جمله‌ای را با  $B_i$  نشان می‌دهیم.  $B_0$  یک نود دارد و  $B_1$  از اتصال ۲ تا  $B_0$  به دست می‌آید و به همین ترتیب  $B_i$  از اتصال دو تا  $B_{i-1}$  به دست می‌آید، به این صورت که ریشه یکی را آخرین فرزند ریشه دیگری قرار دهیم:

 $B_0$  $B_1$  $B_2$  $B_3$  $B_4$ 

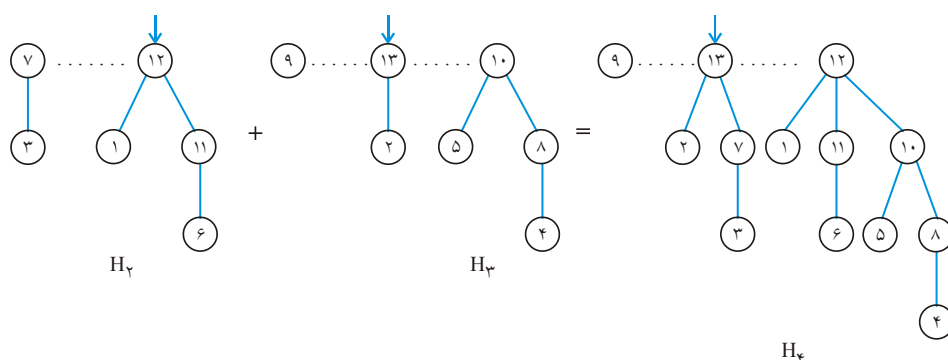
هیپ دو جمله‌ای از تعدادی درخت دو جمله‌ای با اندازه‌های مختلف ساخته شده است به طوری که ریشه درخت‌ها با لیست پیوندی (ترجیحاً دو طرفه) به هم لینک شده است (درخت با اندازه کوچک‌تر سمت چپ است) و هر درخت دارای خاصیت Maxheap است یعنی کلید هر نودش بزرگ‌تر یا مساوی فرزندانش است.

ماکزیمم در ریشه یکی از درخت‌هاست و یک اشاره‌گر به آن ایجاد می‌کنیم بنابراین یافتن ماکزیمم از مرتبه  $O(1)$  است.

 $H_1$ 

شکل مقابل یک هیپ دو جمله‌ای ۱۱ نودی را نشان می‌دهد. هیپ دو جمله‌ای  $n$  نودی از حداکثر  $\lceil \lg n \rceil$  درخت دو جمله‌ای تشکیل شده است.

ادغام دو هیپ دو جمله‌ای که جمعاً  $n$  عنصر دارند از مرتبه  $O(\lg n)$  است. از ارائه الگوریتم خودداری می‌کنیم ولی به مثال زیر توجه کنید:

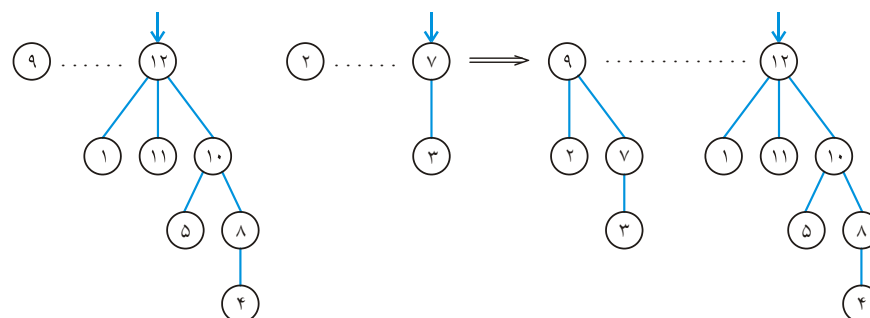
 $H_2$  $H_3$  $H_4$ 

برای حذف ماکزیمم، درخت دو جمله‌ای که ریشه آن ماکزیمم است را از هیپ دو جمله‌ای جدا می‌کنیم، ریشه آن را حذف می‌کنیم، ریشه زیر درختان ریشه را به هم لینک می‌کنیم تا هیپ دو



جمله‌ای دیگر حاصل شود و سپس هیپ‌های دو جمله‌ای حاصل را با هم ادغام می‌کنیم و مرتبه این عملیات  $O(\lg n)$  است.

مثلاً حذف ۱۳ از  $H_4$  را نشان می‌دهیم:



دقت کنید ۱۳ را حذف کردیم و زیر درختان آن را به هیپ دو جمله‌ای تبدیل کردیم و سپس دو هیپ دو جمله‌ای به دست آمده را ادغام کردیم. (۲ را فرزند ۹ قرار دادیم سپس چون ۲ تا  $B_1$  به دست آمد، ۷ را فرزند ۹ قرار دادیم)

برای درج عنصر در هیپ دو جمله‌ای نیز، آن عنصر را به عنوان یک هیپ دو جمله‌ای یک عنصری در نظر می‌گیریم و با هیپ دو جمله‌ای اصلی ادغام می‌کنیم که مرتبه این عمل نیز  $O(\lg n)$  است.

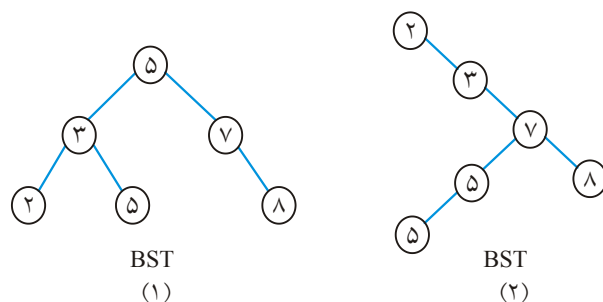
### هیپ فیبوناچی (مینیمم)

برای عملیاتی مثل درج، ادغام و کاهش یک کلید، این هیپ از هیپ کمینه دودویی عملکرد بهتری دارد. ولی عمل حذف یک کلید یا حذف مینیمم در هر دو هیپ، از یک مرتبه است. از ذکر ساختار این هیپ خودداری می‌کنیم ولی جدول زیر مرتبه برخی عملیات را برای برخی ساختار داده‌ها نشان داده است. برای هیپ فیبوناچی، مرتبه هر عمل به طور استهلاکی آمده است. در واقع این هیپ وقتی خوب است که عملیات به تعداد بار زیادی انجام شوند. مثلاً در الگوریتم پریم و دایجسترا با کمک این نوع هیپ بجای هیپ دودویی می‌توان مرتبه را از  $O(e \cdot \lg n)$  به  $O(e + n \cdot \lg n)$  کاهش داد.

| Operation     | Linked list | binary min heap | binomial min heap | fibonacci min heap |
|---------------|-------------|-----------------|-------------------|--------------------|
| is – empty    | ۱           | ۱               | ۱                 | ۱                  |
| insert        | ۱           | $\lg n$         | $\lg n$           | ۱                  |
| find min      | n           | ۱               | $\lg n$           | ۱                  |
| del – min     | n           | $\lg n$         | $\lg n$           | $\lg n$            |
| decreasekey   | n           | $\lg n$         | $\lg n$           | ۱                  |
| union (merge) | ۱           | n               | $\lg n$           | ۱                  |

### درخت جستجوی دودویی (BST) Binary Search Tree

درختی است که در هر نود آن یک کلید وجود دارد (البته اگر درخت تهی نباشد). کلید هر نود از کلید همه نودهای زیر درخت چپش بزرگتر (یا مساوی) و از کلید همه نودهای زیر درخت راستش، کوچکتر (یا مساوی) است. البته در برخی منابع، BST دارای کلیدهای متمایز است.



با توجه به تعریف، اگر BST به صورت inorder پیمایش شود، اعداد صعودی مرتب می‌شوند. پیمایش inorder دو شکل فوق عبارت است از:

inorder: ۲ ۳ ۵ ۵ ۷ ۸

مرتبه پیمایش درخت با  $n$  نود، همانطور که می‌دانیم  $\theta(n)$  است.

### جستجو در BST

برای جستجوی  $k$  ابتدا  $k$  را با کلید ریشه مقایسه می‌کنیم، اگر مساوی بود، تمام. اگر از کلید ریشه کوچکتر بود، جستجو را در زیر درخت چپ انجام می‌دهیم و در غیر این صورت در سمت راست:

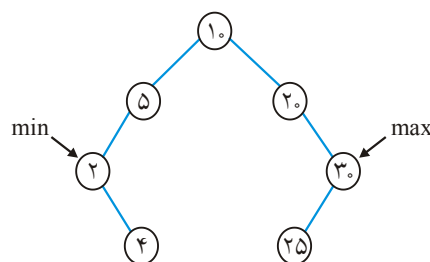
```

TREE_SEARCH (x,k)
  if (x == NIL or k == key[x]) return x
  if (k < key[x]) then return TREE_SEARCH (Left [x] , k)
  else return TREE_SEARCH (right[x] , k)
  
```

در این الگوریتم  $x$ ، اشاره‌گری است به ریشه BST و  $k$  کلیدی است که به دنبال آن هستیم. زمان این الگوریتم به ارتفاع وابسته است و از مرتبه  $O(h)$  است. می‌توان الگوریتم فوق را به صورت غیربازگشتی نیز نوشت که به عهده شما.

## یافتن min و max

برای یافتن مینیمم در BST، از ریشه باید اشاره‌گرهای چپ را دنبال کنیم تا به اولین نودی برسیم که اشاره‌گر چپش تهی است:



و برای یافتن ماکزیمم، اشاره‌گرهای راست را دنبال می‌کنیم تا به اولین نودی برسیم که راستش تهی است.

```

TREE_MIN(x)
while (left[x] ≠ NIL)
    x = left[x]
return x

```

```

TREE_MAX(x)
while (right[x] ≠ NIL)
    x = right[x]
return x

```

مرتبه این عملیات نیز  $O(h)$  است.

## یافتن عنصر بعدی و قبلی x (فرض کنید عناصر متمایز هستند)

عنصر بعدی x (SUCCESSOR(x)) کوچکترین عنصری است که کلیدش از کلید x بزرگتر است مثلاً در درخت فوق، عنصر بعدی ۲، ۴ است. یا عنصر بعدی ۴، ۵ است می‌خواهیم بدون مقایسه کلیدها، عنصر بعدی x را بیابیم. الگوریتم به شکل زیر است:

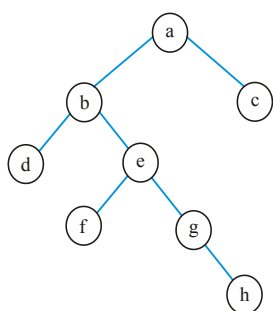
```

TREE_SUCC(x)
if (right[x] ≠ NIL) return TREE_MIN(right[x])
y = p[x] // پدر x است
while (y ≠ NIL and x == right[y])
    x = y
    y = p[y]
return y

```

در این الگوریتم، اگر x فرزند راست داشته باشد، می‌نیمم زیر درخت راست آن جواب است. اگر x فرزند راست نداشته باشد، اولین جد x، که در زیر درخت چپ آن است، جواب است:

Succ(d)=b



چون  $d$  راست ندارد، اولین جدش که  $d$  سمت چپ آن است یعنی  $b$ .

$Succ(b)=f$

چون  $b$  راست دارد، مینیمم زیر درخت راست آن یعنی  $f$ .

$Succ(h)=a$

چون  $h$  راست ندارد، اولین جدش که  $h$  سمت چپ آن است یعنی  $a$ .

مرتبه الگوریتم  $Succ$  و به همین ترتیب  $PRED$  (عنصر قبلی)

نیز  $O(h)$  است.

**توجه:** اگر  $BST$  را  $inorder$  پیمایش کنید،  $Succ$  هر عنصر، بعد از آن نوشته می‌شود، پیمایش  $inorder$  درخت فوق:

$inorder : d \ b \ f \ e \ g \ h \ a \ c$

**توجه:** عنصر قبلی  $x$  یا  $PRED(x)$  را نیز شبیه  $Succ(x)$  خودتان بررسی کنید.

## درج و حذف

برای درج ( $INSERT$ )، از ریشه شروع به مقایسه می‌کنیم (جستجو)، تا محل مناسب برای درج پیدا شود:

$TREE\_INSERT(T, z)$  // درخت  $T$  و  $z$  را می‌خواهیم درج کنیم

$y = NIL$

$x = root[T]$

$NIL \neq while(x$

$\{y = x$

$if \ (key[z] < key[x]) \ x = left[x]$

$else \ x = right[x]\}$

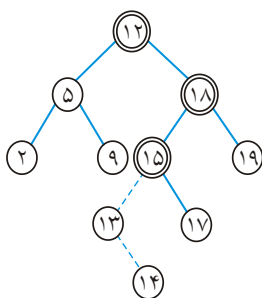
$p[z] = y$

$if \ (y == NIL) //$  درخت تهی است

$root[T] = z$

$else \ if \ (key[z] < key[y]) \ left[y] = z$

$else \ right[y] = z$



مثلاً در درخت مقابل به ترتیب ۱۳ و ۱۴ را درج می‌کنیم:

برای درج ۱۳، با ۱۲ و ۱۸ و ۱۵ مقایسه می‌کنیم.

و برای درج ۱۴، با ۱۲ و ۱۸ و ۱۵ و ۱۳ مقایسه می‌کنیم.

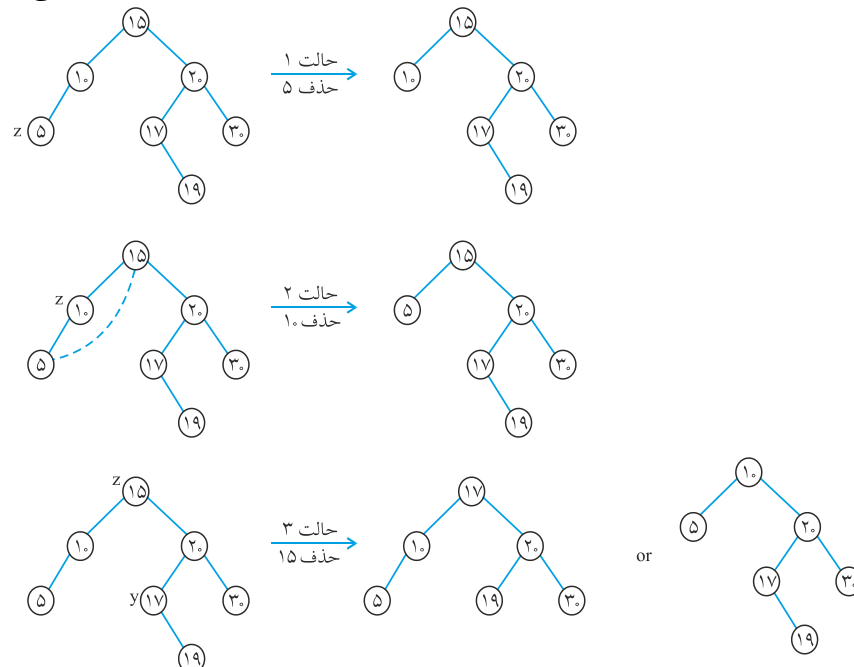
مرتبه درج نیز  $O(h)$  است.

## نکته

ترتیب درج مهم است یعنی درج جابجاپذیر نیست.

برای حذف نود Z سه حالت وجود دارد:

- ۱- اگر Z فرزند نداشته باشد، آن را حذف می‌کنیم یعنی اشاره‌گر پدرش را تهی می‌کنیم.
- ۲- اگر Z یک فرزند داشته باشد، آن را حذف می‌کنیم و تک فرزندش را بجای آن قرار می‌دهیم یعنی پدر Z اشاره می‌کند به فرزند Z.
- ۳- اگر Z دو فرزند داشته باشد، آنگاه داده موجود در  $y = \text{Succ}(z)$  (یا  $\text{Pred}(z)$ ) را بجای Z قرار می‌دهیم.



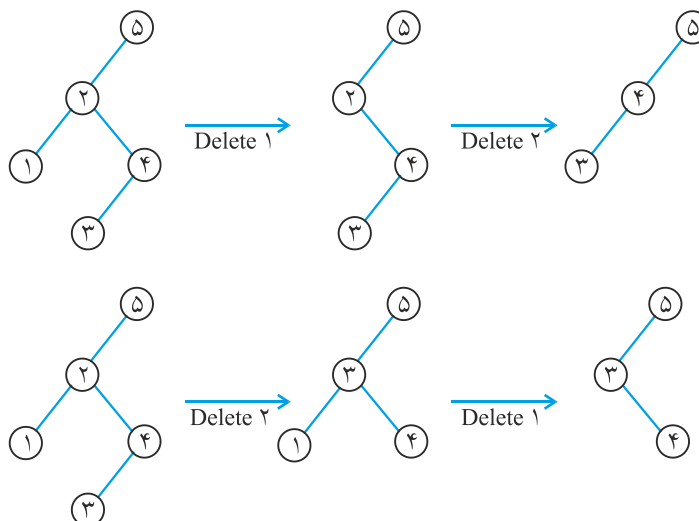
```

TREE_DELETE (T, z)
  if (left[z] == NIL or right[z] == NIL) y = z
  else y = TREE_SUCC(z)
  NIL) if (left[y] x = left [y]
  else x = right [y]
  NIL) if (x p[x] = p[y]
  if (p[y] == NIL) root[T] = x
  else if (y == left[p[y]]) left[p[y]] = x
  else right [p[y]] = x
  key[z] = key[y], copy y's Data into z (y ≠ z) if
  return y
  
```

مرتبه این الگوریتم نیز  $O(h)$  است.

## نکته

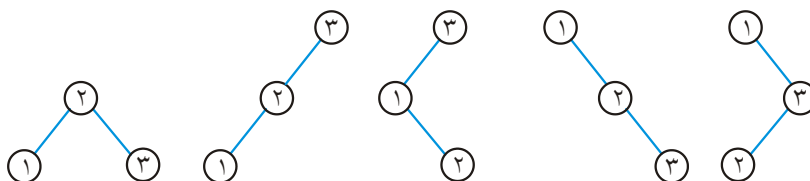
۱- عمل حذف نیز همانند درج، جایپذیر نیست:



۲- عملیات گفته شده روی BST از مرتبه  $O(h)$  بود. حال اگر BST مورب باشد یا شبیه مورب باشد، مرتبه  $O(n)$  می‌شود (بدترین حالت) ولی اگر BST متوازن باشد و یا تا حدودی متوازن باشد مرتبه  $O(\lg n)$  می‌شود (حالت متوسط)

۳- تعداد BST ها با  $n$  کلید متمایز برابر عدد کاتالان یعنی  $\frac{1}{n+1} \binom{2n}{n}$  است. مثلاً با ۳ عدد ۱ و ۲ و ۳

تعداد پنج BST وجود دارد:

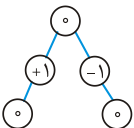


۴- می‌توان با  $n$  عدد ورودی و با درج‌های متوالی، ساخت BST که مرتبه آن  $O(n.h)$  است که در بدترین حالت (وقتی اعداد ورودی مرتب باشند)  $O(n^2)$  می‌شود و در حالت متوسط مرتبه  $O(n \lg n)$  است. ولی اگر  $n$  عدد در اختیار باشند می‌توان ابتدا آنها را مرتب کرد و سپس عنصر وسط آنها را به عنوان ریشه در نظر گرفت و سپس با  $\frac{n}{2}$  نود سمت چپ و  $\frac{n}{2}$  نود سمت راست ریشه همین عمل را بازگشتی انجام دهیم که مرتبه این روش همیشه  $O(n \lg n)$  است. ولی اگر عملیات حذف و درج قرار باشد زیاد انجام شود این روش مناسب نیست.

## AVL

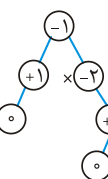
BST متوازن را AVL گویند. درخت متوازن (Balanced) در اینجا منظور درختی است که فاکتور توازن هر نود مثل  $X$  برابر  $0$  یا  $+1$  یا  $-1$  باشد. فاکتور توازن نود  $X$  برابر است با تعداد

سطوح زیر درخت چپ  $X$  منهای تعداد سطوح زیر درخت راست  $X$ . مثلاً



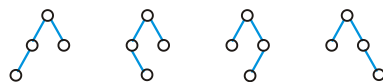
متوازن است

(داخل هر نود فاکتور توازن آن نوشته شده است). ولی درخت



متوازن نیست و نودی

که کنار آن علامت ضربدر است، توازن را برهم زده است.

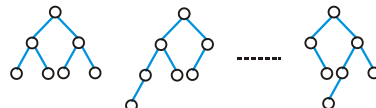


**مثال ۷:** با ۴ نود چند درخت متوازن می توان ساخت؟

**پاسخ:** تعداد ۴ درخت متوازن با ۴ نود می تون ساخت.

**مثال ۸:** با ۷ نود چند درخت متوازن می توان ساخت؟

**پاسخ:** ۱۷ درخت متوازن با ۷ نود وجود دارد که تعدادی از آنها رسم شده اند.



## نکته

اگر  $N(h)$  حداقل تعداد نود لازم برای ساخت درخت متوازن به ارتفاع  $h$  باشد، آنگاه  $N(0) = 1$  و

$N(1) = 2$  و  $N(h) = N(h-1) + N(h-2) + 1$ . که نتیجه می شود.  $N(h) = \theta\left(\left(\frac{1+\sqrt{5}}{2}\right)^h\right)$  پس

ارتفاع درخت متوازن از مرتبه  $\theta(\lg n)$  است. ■

الگوریتم های پایه یعنی جستجو، درج، حذف، یافتن Min و Max و یافتن عنصر بعدی و قبلی  $X$ ، همگی

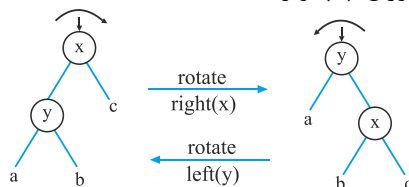
در AVL از مرتبه  $O(\lg n)$  هستند. ■

برای ساخت AVL، همانند ساخت BST، کلیدهای ورودی را به ترتیب درج می کنیم. ولی با

هر عمل درج باید فاکتور توازن اجداد نود جدید را بررسی می کنیم و اگر فاکتور توازن نودی  $\pm 2$

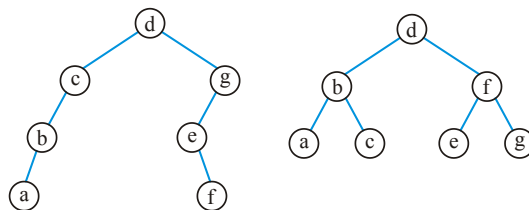
باشد باید با عمل چرخش (rotate)، توازن را ایجاد کنیم.

دو نوع دوران وجود دارد: دوران چپ و راست:

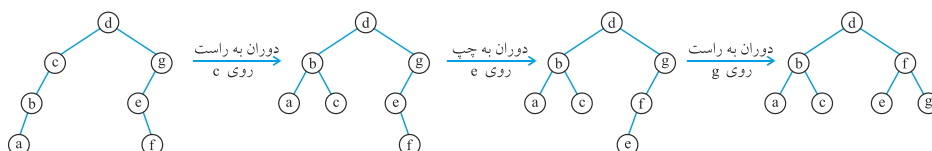


برای انجام دوران به راست روی نود  $x$ ، حتماً  $x$  باید فرزند چپ داشته باشد که در این صورت، فرزند چپ  $x$  (در شکل فوق  $y$  است)، پدر  $x$  می‌شود و  $x$  فرزند راست  $y$  می‌شود. در ضمن زیر درخت راست  $y$  (یعنی  $b$  در شکل فوق)، زیر درخت چپ  $x$  می‌شود. دوران به چپ نیز به همین ترتیب قابل بیان است. به طور کلی در هر عمل دوران، ۳ اشاره‌گر تغییر می‌کنند. اشاره‌گرهایی که در دوران راست  $x$  عوض می‌شوند، عبارتند از لینک چپ  $x$ ، لینک راست  $y$ ، لینک پدر  $x$ . بنابراین عمل دوران از مرتبه  $\theta(1)$  است.

**مثال ۹:** با چند عمل دوران درخت سمت چپ به درخت سمت راست تبدیل می‌شود؟



**پاسخ:** ۳ عمل دوران لازم است. دوران  $c$  به راست. دوران  $e$  به چپ و سپس دوران  $g$  به راست.

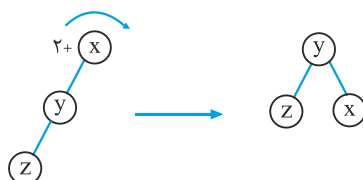


هنگام ساخت AVL، ۴ حالت پیش می‌آید. فرض کنید  $Z$  نود جدید است که درج کرده‌ایم.  $x$

اولین جد نود  $Z$  است که توازن آن  $\pm 2$  شده است. و  $y$  فرزند  $x$  است که در مسیر درج قرار دارد:

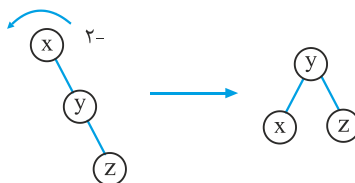
**حالت ۱:**  $Z$  به زیر درخت چپ  $y$  اضافه شده و  $y$  فرزند چپ  $x$  است و توازن  $x$  برابر  $+2$  شده

است در این حالت،  $x$  را دوران به راست می‌دهیم.

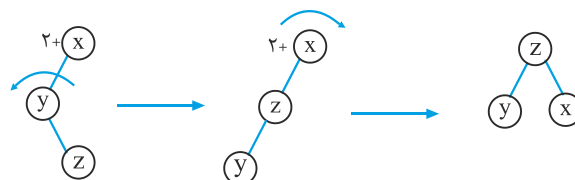




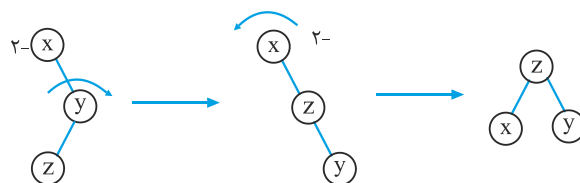
**حالت ۲:** Z به زیر درخت راست Y اضافه شده و Y فرزند راست X است و توازن X برابر  $-2$  شده است. در این حالت، X را دوران به چپ می‌دهیم:



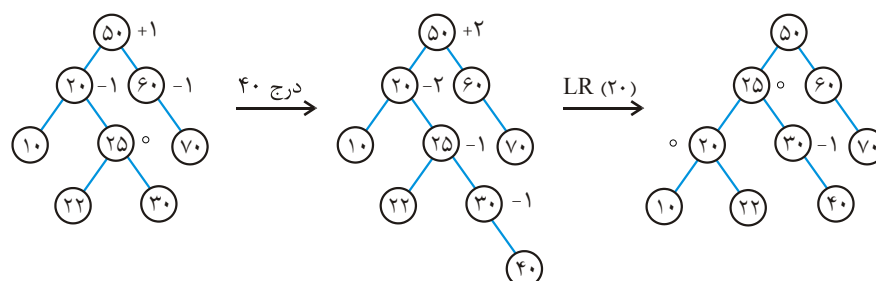
**حالت ۳:** Z به زیر درخت راست Y اضافه شده و Y فرزند چپ X است و توازن X برابر  $+2$  شده است. در این حالت ابتدا Y را به چپ و سپس X را به راست دوران می‌دهیم:



**حالت ۴:** Z به زیر درخت چپ Y اضافه شده و Y فرزند راست X است و توازن X برابر  $-2$  شده است. در این حالت ابتدا Y را به راست و سپس X را به چپ دوران می‌دهیم:

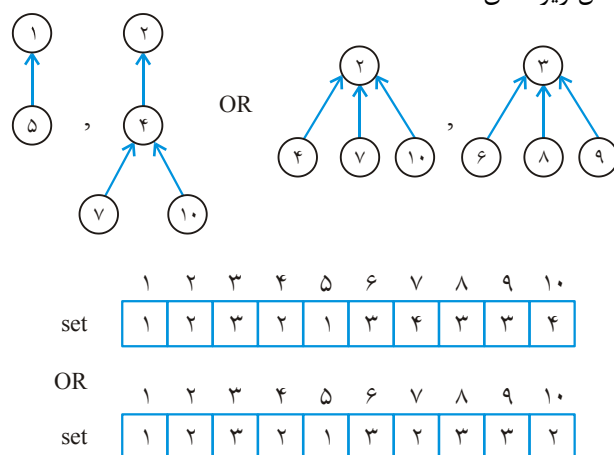


**مثال ۱۰:** در AVL داده شده،  $40$  درج شده است. کنار برخی نودها فاکتور توازن آن نود، نوشته شده است. پس از عمل درج، فاکتور توازن نود  $20$  برابر  $-2$  و فاکتور توازن نود  $50$  برابر  $+2$  شده است. ولی اولین جد نود  $40$  یعنی  $20$  مهم است. پس  $X=20$  و  $Y=25$  و  $Z=40$ . که حالت ۲ است. و باید  $20$  را دوران چپ (LR: Leftrotate) دهیم:



## مجموعه‌های مجزا

می‌خواهیم تعدادی مجموعه مجزا مثل  $A_1 = \{1, 5\}$ ,  $A_2 = \{2, 4, 7, 10\}$ ,  $A_3 = \{3, 6, 8, 9\}$  را نمایش دهیم. هر مجموعه را با یک نماینده (معمولاً کوچک‌ترین عضو آن مجموعه) نشان می‌دهیم، مثلاً مجموعه  $A_1$  را با عضو ۱ یا مجموعه  $A_2$  را با عضو ۲ می‌شناسیم. می‌توان هر مجموعه را با یک درخت ریشه‌دار نشان داد که هر نود به پدرش اشاره می‌کند، مثلاً مجموعه‌های  $A_1$  و  $A_2$  و  $A_3$  را می‌توان به شکل زیر نشان داد:



در واقع درخت‌ها را می‌توان با آرایه Set ذخیره کرد. اگر  $\text{Set}[i]=i$  آنگاه  $i$  ریشه درخت و در واقع نماینده مجموعه مورد نظر است. و اگر  $\text{Set}[i]=j \neq i$  آنگاه  $j$  پدر  $i$  در یکی از درخت‌ها است. دو عمل روی مجموعه‌ها، پیاده‌سازی می‌کنیم:  $\text{find}(x)$  نماینده مجموعه‌ای را برمی‌گرداند که  $x$  عضو آن است، مثلاً  $\text{find}(7)$  برابر ۲ است.  $\text{Merge}(a, b)$  (union(a, b)) دو مجموعه با نماینده‌های  $a, b$  ( $a \neq b$ ) را ادغام می‌کند:

```

find (x)
r = x
while (set[r] ≠ r) r = set[r]
return r

merge (a, b)
if (a < b) set[b] = a else set[a] = b

```

مرتبه  $\text{find}$  در بدترین حالت  $\theta(N)$  است (  $\text{set}[1..N]$  فرض می‌شود) و مرتبه  $\text{merge}$  ثابت است. در الگوریتم  $\text{merge}$ ، از بین  $a, b$  آنکه کوچک‌تر است ریشه قرار می‌گیرد. ولی بهتر است  $\text{Merge}$  را طوری تغییر دهیم که آنکه ارتفاعش کمتر است، زیر درخت دیگری قرار گیرد، زیرا در این صورت ارتفاع درخت حاصل خیلی زیاد نخواهد شد و عمل  $\text{find}$  طولانی نخواهد شد. الگوریتم  $\text{merge}'$  با این ایده ارائه می‌شود:

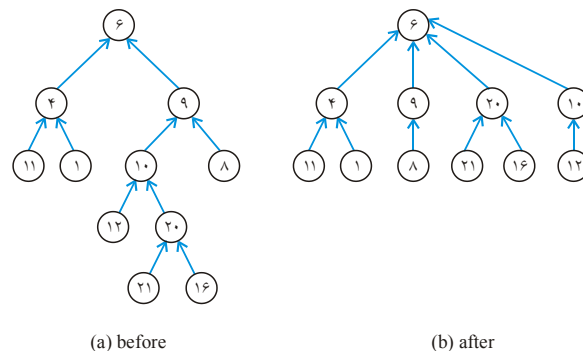
merge'(a, b)

if ( $h[a] = h[b]$ )

{  $h[a] = h[a] + 1$  ,  $set[b] = a$ }

else if ( $h[a] > h[b]$ )  $set[b] = a$  else  $set[a] = b$ }

آرایه  $h[1..N]$  که در ابتدا عناصرش صفر هستند، ارتفاع هر نود را ذخیره می‌کند. اگر  $a$  ریشه یک درخت باشد،  $h[a]$  ارتفاع آن درخت را نشان می‌دهد. با  $merge'$  اگر  $k$  عمل ادغام پشت سر هم روی مجموعه‌های تک عضوی انجام شود، ارتفاع درخت حاصل حداکثر  $\lfloor \lg k \rfloor$  است. با کمک تکنیکی موسوم به فشردسازی مسیر (path compression) می‌توان  $find$  را بهبود بخشید، به این صورت که در عمل  $find(x)$  که از  $x$  به سمت ریشه حرکت می‌کنیم، هر نودی که در مسیرمان دیدیم، اشاره‌گر به پدرش را به ریشه درخت تغییر دهیم تا ارتفاع درخت کاهش یابد. در شکل زیر عمل  $find'(20)$  روی درخت (a) انجام شده و درخت (b) حاصل شده است:



(a) before

(b) after

الگوریتم  $find'$  با تکنیک فشردسازی مسیر ارائه می‌شود:

$find'(x)$

$r = x$

while ( $set[r] \neq r$ )  $r = set[r]$  //  $r$  ریشه درخت است.

$i = x$

while ( $i \neq r$ )

{  $j = set[i]$ ,  $set[i] = r$ ,  $i = j$  }

return  $r$

#### نکته

تحلیل الگوریتم‌های  $merge'$  و  $find'$  بسیار دشوار است. اگر آرایه  $set[1..N]$  باشد و در ابتدا  $N$  مجموعه تک عضوی داشته باشیم، و اگر  $n$  فراخوانی به  $find'$  و  $m \leq N-1$  فراخوانی به  $merge'$  انجام دهیم و فرض کنیم  $C = n + m$ ، زمان اجرا در بدترین حالت  $\theta(C \cdot \alpha(C, N))$  است که  $\alpha(C, N)$  رشد بسیار ناچیزی دارد و برای اعداد خیلی بزرگ در حدود ۳ است.

## خلاصه فصل

- ۱- الگوریتم‌های پیمایش درخت شامل  $n$  نود، همگی از مرتبه  $\theta(n)$  است.
- ۲- با داشتن پیمایش‌های میان ترتیب و پیش ترتیب یا میان ترتیب و پس ترتیب یا میان ترتیب و سطح ترتیب درخت دودویی منحصر به فرد است.
- ۳- با داشتن پیمایش‌های پیش ترتیب و پس ترتیب اگر  $K$  تا گره تک فرزندی وجود داشته باشد، می‌توان  $2^k$  تا درخت دودویی متفاوت رسم کرد.
- ۴- عملیاتِ درج، حذف یک نود مشخص، افزایش یا کاهش کلید یک عنصر در هیپ  $n$  نودی از مرتبه  $O(\lg n)$  است.
- ۵- یافتن ماکزیمم در هرم بیشینه (Maxheap)  $n$  نودی از مرتبه  $\theta(1)$  و یافتن مینیمم از مرتبه  $\theta(n)$  است زیرا مینیمم در یکی از  $\lceil \frac{n}{2} \rceil$  برگ است.
- ۶- ساخت هیپ  $n$  نودی از مرتبه  $\theta(n)$  است.
- ۷- ادغام دو هیپ  $m$  و  $n$  عنصری از مرتبه  $\theta(m+n)$  است.
- ۸- عملیات جستجو، درج، حذف، یافتن Min و Max، یافتن عنصر بعدی و قبلی  $x$  در BST از مرتبه  $O(h)$  است که اگر BST متوازن باشد (مثل AVL) از مرتبه  $O(\lg n)$  است.
- ۹- می‌توان مجموعه‌های مجزا را با آرایه پیاده‌سازی کرد و با درخت ریشه‌دار نمایش داد. اگر تعدادی عمل find و union (merge) انجام شود، با تکنیک‌های گفته شده در درس، مرتبه سرشکن شده هر عمل  $O(\alpha)$  است که  $\alpha$  تابعی است با رشد بسیار ناچیز.

## تمرین

- ۱- چگونه می‌توان با صف اولویت، صف FIFO و پشته ساخت.
- ۲- یک الگوریتم با زمان  $O(n \lg k)$  ارائه دهید که  $k$  لیست مرتب را با هم ادغام کند و لیست نهایی  $n$  عنصری را بسازد (از یک min heap برای  $k$ -way merging استفاده کنید)

## [سؤال‌های چهارگزینه‌ای فصل ششم]

۱- در یک درخت باینری دلخواه پیچیدگی زمان ۳ پیمایش Preorder ، Postorder و inorder به ترتیب برابر است با:

(علوم کامپیوتر ۸۲)

- (۱)  $O(n)$  ,  $O(\log n)$  ,  $O(n)$  (۲)  $O(n^2)$  ,  $O(n)$  ,  $O(n^2)$   
 (۳)  $O(n)$  ,  $O(n)$  ,  $O(n)$  (۴)  $O(n)$  ,  $O(\log n)$  ,  $O(n^2)$

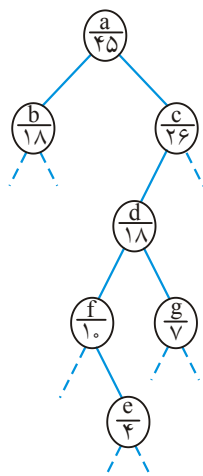
۲- چه تعداد درخت دودویی برچسب‌دار متفاوت با  $n$  گره و با برچسب‌های ۱ تا  $n$  که دارای ترتیب‌های یکسان در دو روش پس‌ترتیب و بین‌ترتیب می‌باشند، وجود دارند؟

(علوم کامپیوتر ۸۳)

- (۱) ۰ (۲) ۱ (۳)  $n$  (۴)  $n!$

۳- در شکل زیر قسمتی از یک درخت دودویی نشان داده شده است. در زیر برچسب هر گره عددی نوشته شده است که تعداد کل گره‌های موجود در زیر درخت آن گره را نشان می‌دهد (با احتساب خود آن گره). اگر درخت مفروض را بطور inorder پیمایش کنیم،  $f$  چندمین خروجی خواهد بود؟

(علوم کامپیوتر ۸۳)



- (۱) ۲۵ امین  
 (۲) ۲۶ امین  
 (۳) ۲۹ امین  
 (۴) ۳۰ امین

۴- الگوریتم پیمایش درخت در زمان  $O(d)$  اجرا می‌شود.  $d$  چه می‌باشد؟

(علوم کامپیوتر ۸۴)

- (۱) تعداد برگ‌های درخت (۲) عمق درخت  
 (۳) درجه درخت (۴) تعداد گره‌های درخت

۵- شرط اصلی برای اینکه بتوان از روی دو پیمایش داده شده یک درخت باینری، درخت اصلی را ساخت چیست؟  
(علوم کامپیوتر ۸۴)

- (۱) تمام داده‌ها در درخت متمایز باشند.
- (۲) فقط ریشه از بقیه متمایز باشد.
- (۳) داده‌های زیر درخت چپ ریشه از داده‌های زیر درخت راست متمایز باشند ولی در خود زیر درخت‌های چپ و راست می‌تواند داده تکراری وجود داشته باشد.
- (۴) داده‌های موجود در برگ‌های درخت متمایز باشند.

۶- در یک درخت باینری T با n گره در چه حالتی اولین گره در پیمایش Postorder مشابه آخرین گره در پیمایش inorder است؟  
(علوم کامپیوتر ۸۵)

- (۱) زیر درخت چپ T خالی باشد.
- (۲) T دارای ارتفاع  $n-1$  باشد.
- (۳) زیر درخت راست T خالی باشد.
- (۴) T حداکثر ۳ گره داشته باشد.

۷- خروجی الگوریتم زیر برای یک درخت باینری

```
Void Rco (ptr p)
{
    if (P!=Null)
        Print (P → Data);
        Rco (P → right);
        Rco (P → left);
}
```

معادل کدام یک از پیمایش‌های زیر می‌باشد؟  
(علوم کامپیوتر ۸۶)

- (۱) inorder
- (۲) reverse preorder
- (۳) reverse postorder
- (۴) معادل هیچ پیمایشی نمی‌باشد.

۸- حاصل پیمایش inorder یک درخت دودویی با ۷ گره به ترتیب از چپ به راست عبارت است از c و b و e و d و a و f و g و حاصل پیمایش Preorder همین درخت به ترتیب از چپ به راست عبارت است از c و e و d و b و g و f و a با توجه به مفروضات در این درخت گره c فرزند کدام گره است؟  
(IT ۸۶)

- (۱) a
- (۲) b
- (۳) e
- (۴) d

۹- پس از حذف (یا اضافه کردن) یک عنصر، می‌توان هرم (Heap) را مجدداً .....  
(علوم کامپیوتر ۸۱)

- (۱) در زمان  $n^2$  بازسازی کرد.
- (۲) در زمان  $n \log n$  بازسازی کرد.
- (۳) در زمان  $\log n$  بازسازی کرد.
- (۴) هیچکدام

۱۰- عمق یک گره در درایه  $i$  در یک heap با  $n$  گره که با آرایه پیاده‌سازی شده است چقدر است؟ (۸۳ IT و علوم کامپیوتر ۸۱)

$$(1) \left\lfloor \frac{i}{2} \right\rfloor \quad (2) \left\lceil \frac{i}{2} \right\rceil \quad (3) i-1 \quad (4) \left\lfloor \log_2^i \right\rfloor$$

۱۱- اگر آرایه 

|    |    |    |   |    |    |    |    |
|----|----|----|---|----|----|----|----|
| ۱۰ | ۱۵ | ۲۲ | ۴ | ۱۱ | ۲۳ | ۱۹ | ۱۴ |
|----|----|----|---|----|----|----|----|

 نمایش یک درخت باینری باشد، و آنرا به یک min-heap تبدیل نماییم در آن صورت محتوای آرایه برابر خواهد بود:

(علوم کامپیوتر ۸۳)

$$(1) ۴, ۱۰, ۱۵, ۱۹, ۱۱, ۲۳, ۲۲, ۱۴ \quad (2) ۴, ۱۰, ۱۴, ۱۵, ۱۱, ۱۹, ۲۲, ۲۳$$

$$(3) ۴, ۱۰, ۱۱, ۱۴, ۱۵, ۱۹, ۲۲, ۲۳ \quad (4) ۴, ۱۰, ۱۹, ۱۴, ۱۱, ۲۳, ۲۲, ۱۵$$

۱۲- فرض کنید که ماکزیمم - هیپ حاوی اعداد متمایز ۱ تا ۱۰۲۳ است. حداکثر چند تا از اعداد بیش‌تر از ۱۰۰۰ می‌توانند در پایین‌ترین سطح درخت قرار گیرند؟ (علوم کامپیوتر ۸۳)

$$(1) ۱۰ \quad (2) ۱۲ \quad (3) ۱۳ \quad (4) ۱۴$$

۱۳- ماکزیمم تعداد مقایسه برای min-heap کردن یک max-heap با  $n$  گره برابر است با: (علوم کامپیوتر ۸۴)

$$(1) O(n + \log n) \quad (2) O(\log n) \quad (3) O(n \log n) \quad (4) O(2n)$$

۱۴- اگر در یک min-heap دلخواه جای زیر درخت‌های چپ و راست تعویض شود در آن صورت کدام گزینه صحیح است؟ (علوم کامپیوتر ۸۵)

- (۱) همیشه ساختار min-heap از بین می‌رود.
- (۲) ساختار min-heap تبدیل به max-heap می‌شود.
- (۳) در برخی از شرایط ساختار min-heap از بین می‌رود.
- (۴) در هیچ شرایطی ساختار min-heap از بین نمی‌رود.

۱۵- اگر بخواهیم دو min-heap به تعداد  $m$  و  $n$  را با هم merge کنیم پیچیدگی زمانی آن برابر است با: (علوم کامپیوتر ۸۵)

$$(1) m \log mn \quad (2) m \log n + n \log m$$

$$(3) n \log n + m \log m \quad (4) m \cdot \log n + \log m$$

۱۶- در یک آرایه دلخواه با ۱۲ عنصر چند مقایسه حداکثر نیاز می‌باشد تا این آرایه تبدیل به یک minheap شود؟ (علوم کامپیوتر ۸۶)

$$(1) ۶ \quad (2) ۱۰ \quad (3) ۱۲ \quad (4) ۱۸$$



۱۷- در یک heap با  $n$  گره که در یک آرایه نمایش داده شده باشد گره  $i$  در زیر درخت چپ درخت، متناظر با گره  $j$  ام در زیر درخت راست درخت می‌باشد به طوری که:

(علوم کامپیوتر ۸۵)

$$\begin{aligned} j &= i + \left\lfloor \frac{\log_2 i}{2} \right\rfloor + 1 & (1) \\ \text{if } j > n \quad j &= \frac{j}{2}; \end{aligned}$$

$$\begin{aligned} j &= i + \left\lfloor \frac{\log_2 i}{2} \right\rfloor + 1 & (2) \\ \text{if } j > n \quad j &= \frac{j+1}{2} \end{aligned}$$

۱۸- می‌خواهیم دو هیپ مینیمم با اندازه‌های  $n$  و  $m$  را در یک هیپ به اندازه  $n+m$  ادغام کنیم فرض کنید که هیپ خروجی می‌تواند به صورت درختی باشد، با فرض  $n > m$ ، کدام یک از گزینه‌های زیر همیشه درست است؟

(IT ۸۴)

- (۱) این کار را همیشه می‌توان در  $O(\lg^2 n)$  انجام داد.
- (۲) این کار را همیشه می‌توان در  $O(\lg n)$  انجام داد.
- (۳) این کار را می‌توان در  $O(\lg n)$  انجام داد، و هیپ خروجی لزوماً به صورت آرایه‌ای نیست.
- (۴) این کار را همیشه می‌توان در  $O(\lg^2 n)$  انجام داد، ولی هیپ خروجی لزوماً به صورت آرایه‌ای نیست.

۱۹- به یک Min-Heap خالی به ترتیب گره‌هایی با کلیدهای (از راست به چپ) ۷۰ و ۴۵ و ۵۰ و ۴۲ و ۴۵ و ۵۵ و ۴۰ و ۷۵ اضافه شده است. سپس سه عمل حذف (Delete) بر روی Min-Heap انجام شده است. Min-Heap حاصل مطابق کدام گزینه خواهد بود؟

(ساختمان داده IT ۸۴)

$$\begin{aligned} (1) & [45, 50, 55, 70, 75] & (2) & [45, 50, 55, 75, 70] \\ (3) & [45, 55, 50, 70, 75] & (4) & [45, 55, 50, 75, 70] \end{aligned}$$

۲۰- کدام یک از گزینه‌های زیر نمی‌تواند یک بازنمایی از یک Max-Heap با آرایه باشد (ترتیب عناصر از چپ به راست است)

(ساختمان داده IT ۸۵)

$$\begin{aligned} (1) & 1 \ 4 \ 3 \ 6 \ 10 \ 5 \ 20 & (2) & 1 \ 2 \ 4 \ 3 \ 17 \ 17 \ 20 \\ (3) & 1 \ 2 \ 4 \ 3 \ 10 \ 5 \ 20 & (4) & 1 \ 2 \ 4 \ 3 \ 18 \ 10 \ 20 \end{aligned}$$

۲۱- به یک Min-heap خالی گره‌هایی با کلیدهای (به ترتیب از راست به چپ): ۴۵ و ۵۵ و ۴۰ و ۲۲ و ۷۵ و ۷۰ و ۴۵ و ۵۰ و ۲ اضافه شده است. min-heap حاصل کدام گزینه است؟  
(ساختمان داده IT ۸۶)

- (۱) ۵۰ و ۵۵ و ۴۵ و ۷۰ و ۷۵ و ۴۰ و ۴۵ و ۲۲ و ۲
- (۲) ۵۵ و ۷۰ و ۴۰ و ۴۵ و ۷۵ و ۵۰ و ۲۲ و ۴۵ و ۲
- (۳) ۵۵ و ۵۰ و ۴۵ و ۷۰ و ۷۵ و ۴۵ و ۴۰ و ۲۲ و ۲
- (۴) ۵۵ و ۵۰ و ۴۵ و ۷۰ و ۷۵ و ۴۰ و ۴۵ و ۲۲ و ۲

۲۲- سومین کوچکترین کلید در یک Min Heap با کلیدهای متمایز در درایه‌هایی با چه اندیس‌هایی می‌تواند باشد؟  
(ساختمان داده IT ۸۶)

- (۱) ۱ و ۲ و ۳
- (۲) ۲ و ۳ و ۴ و ۵ و ۶ و ۷
- (۳) ۴ و ۵ و ۶ و ۷
- (۴) ۱ و ۲ و ۳ و ۴ و ۵ و ۶ و ۷

۲۳- آرایه زیر یک heap است. برای درج عدد ۹۵ در آرایه به گونه‌ای که آرایه نهایی نیز وضعیت heap داشته باشد، چند عمل exchange (تعویض دو کمیت) لازم است؟  
(دولتی ۸۰)

- |     |          |
|-----|----------|
| ۱۰۰ | (۱) دو   |
| ۹۰  | (۲) چهار |
| ۸۲  | (۳) شش   |
| ۸۵  | (۴) هشت  |
| ۷۴  |          |
| ۷۵  |          |
| ۷۳  |          |
| ۶۸  |          |
| ۷۰  |          |

۲۴- آرایه n خانه‌ای A را در نظر می‌گیریم که برای ذخیره‌سازی عناصر یک درخت دودویی کامل مورد استفاده قرار گرفته. فرض کنیم الگوریتمی کارا برای بررسی این که این درخت دودویی یک heap است یا خیر در اختیار داریم. پیچیدگی این الگوریتم در بدترین حالت چقدر است؟  
(دولتی ۸۱)

- (۱)  $O(n)$
- (۲)  $O(n^2)$
- (۳)  $O(\log n)$
- (۴)  $O(n \log n)$

۲۵- یک max-heap با n عنصر به صورت آرایه پیاده‌سازی شده است. مناسب‌ترین گزینه برای پیدا کردن عنصر مینیمم در این ساختمان داده کدام است؟  
(ساختمان داده‌ها دولتی ۸۱)

(۱) این کار را همواره می‌توان با  $O(\log n)$  مقایسه بین عناصر heap انجام داد.

(۲) این کار به حداکثر  $\frac{n}{2}$  مقایسه بین عناصر heap نیاز دارد.

(۳) این کار ممکن است به  $n-1$  مقایسه بین عناصر heap نیاز داشته باشد.  
 (۴) تنها در صورتی که heap عناصر تکراری نداشته باشد، می‌توان این کار را با  $O(\log n)$  مقایسه بین عناصر heap انجام داد.

۲۶- یک max-heap با  $N$  عنصر متمایز را در نظر بگیرید که با یک آرایه پیاده‌سازی شده است (بزرگترین عنصر در درایه اول قرار دارد). چهارمین بزرگترین عنصر در کدام یک از درایه‌های زیر می‌تواند قرار گیرد؟ (ساختمان داده‌ها دولتی ۸۲)

- (۱) ۲ یا ۳ (۲) ۸ تا ۱۵ (۳) ۴ و ۵ و ۶ و ۷ (۴) همه موارد فوق

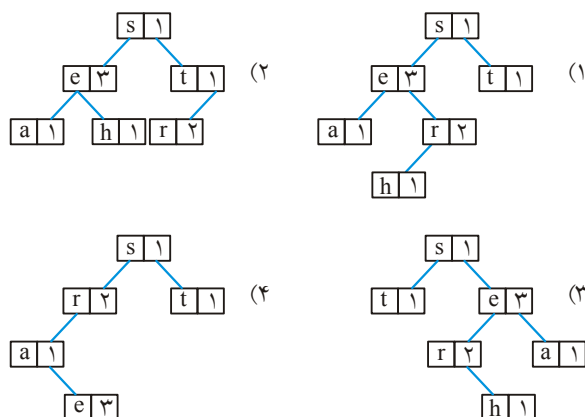
۲۷- یک ماکزیمم - هیپ حاوی ۶۴ عنصر با کلیدهای متفاوت ۱ تا ۶۴ است. بزرگترین عددی که می‌تواند در برگ واقع در آخرین سطح این هیپ قرار گیرد، کدام یک از اعداد زیر می‌تواند باشد؟ (در ماکزیمم هیپ هر عنصر از فرزندانش کوچک‌تر نیست) (ساختمان داده‌ها دولتی ۸۳)

- (۱) ۵۹ (۲) ۵۸ (۳) ۵۷ (۴) ۵۶

۲۸- به منظور Sort صعودی یک آرایه از heap sort استفاده نموده و max heap می‌سازیم. در مورد زمان اجرای این الگوریتم کدام گزینه صحیح است؟ (تخصصی هوش مصنوعی دولتی ۸۴)

- (۱) اگر آرایه از ابتدا به صورت صعودی مرتب شده باشد زمان مورد نیاز  $\theta(n)$  است.  
 (۲) اگر آرایه از ابتدا به صورت نزولی مرتب شده باشد زمان مورد نیاز  $\theta(n)$  است.  
 (۳) اگر آرایه از ابتدا به صورت نزولی مرتب شده باشد زمان مورد نیاز  $\theta(n^2 \log n)$  است.  
 (۴) اگر آرایه از ابتدا به صورت صعودی مرتب شده باشد زمان مورد نیاز  $\theta(n \log n)$  است.

۲۹- درخت جستجوی دودویی حاصل از لیست (s,e,a,r,e,h,t,r,e) که علاوه بر نویسه‌ها (characters)، تعداد آنها را نیز ذخیره می‌کند برابر است با ..... (علوم کامپیوتر ۸۰)



۳۰- اگر اعداد ۱۰ و ۷ و ۶ و ۴ و ۱۲ و ۹ و ۱ و ۲ و ۸ و ۵ و ۳ از چپ به راست در یک درخت

دودویی جستجوی تهی درج شوند، ارتفاع درخت حاصل چند است؟ (علوم کامپیوتر ۸۱)

(۱) ۳ (۲) ۴ (۳) ۶ (۴) ۵

۳۱- فرض کنید اعداد ۱ تا ۱۰۰۰ در یک درخت دودویی مرتب ذخیره شده‌اند و ما می‌خواهیم

عدد ۳۶۳ را پیدا کنیم. کدامیک از ترتیب‌ها (از چپ به راست) نمی‌تواند بیانگر دسترسی

به عناصر درخت در این جستجو باشد؟ (علوم کامپیوتر ۸۲)

(۱) ۳۶۳ و ۲۴۵ و ۹۱۲ و ۲۴۰ و ۹۱۱ و ۲۰۲ و ۹۳۵

(۲) ۳۶۳ و ۳۶۲ و ۲۵۸ و ۸۹۸ و ۲۴۴ و ۹۱۱ و ۲۲۰ و ۹۲۴

(۳) ۳۶۳ و ۳۹۷ و ۳۴۴ و ۳۳۰ و ۳۹۸ و ۴۰۱ و ۲۵۲ و ۲

(۴) ۳۶۳ و ۳۷۸ و ۳۸۱ و ۳۸۲ و ۲۶۶ و ۲۱۹ و ۳۷۸ و ۳۹۹ و ۲

۳۲- یک درخت جستجوی باینری با کلمه "SEARCH" (از چپ به راست) بسازید. تعداد

متوسط مقایسه برای پیدا کردن یک کاراکتر در این درخت برابر است با: (علوم کامپیوتر ۸۳)

(۱) ۲/۸۳ (۲) ۲/۴ (۳) ۲/۶ (۴) ۶

۳۳- در یک درخت جستجوی باینری کدام عمل در  $O(1)$  انجام می‌شود؟ (علوم کامپیوتر ۸۳)

(۱) بررسی تهی بودن درخت (۲) پیدا کردن کمترین مقدار

(۳) حذف یک عنصر (۴) درج کردن یک عنصر جدید

۳۴- در یک درخت جستجوی باینری با  $n$  گره، تعداد مقایسه برای جستجوی یک عنصر حداکثر

برابر است با: (علوم کامپیوتر ۸۴)

(۱)  $O\left(\frac{n}{2}\right)$  (۲)  $O(n)$  (۳)  $O(\sqrt{n})$  (۴)  $O(\log n)$

۳۵- برای یک درخت جستجوی باینری که در گره‌های آن تعدادی عدد ذخیره شده است، با

داشتن پیمایش preorder آن کدام جمله صحیح است؟ (علوم کامپیوتر ۸۴)

(۱) فقط با یک پیمایش نمی‌توان درخت را ساخت.

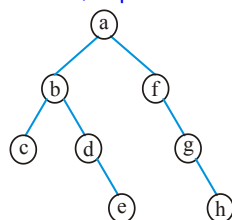
(۲) می‌توان درخت را در  $O(n^2)$  ساخت.

(۳) می‌توان درخت را در  $O(n \log n)$  ساخت.

(۴) می‌توان درخت را در  $O(n)$  ساخت.

۳۶- در درخت جستجوی باینری زیر با حذف ریشه کدام یک از عناصر زیر جایگزین می‌شود؟

(علوم کامپیوتر ۸۴)



(۱) b یا f

(۲) c یا h

(۳) e یا f

(۴) h یا e

۳۷- اگر عناصر یک درخت جستجوی باینری را به صورت inorder پیمایش کنیم و در داخل

یک استک قرار دهیم و سپس عناصر استک را خارج کنیم و یک درخت جستجوی باینری

(علوم کامپیوتر ۸۵)

بسازیم درخت حاصل چه درختی خواهد بود؟

(۱) درخت باینری تغییر نمی‌کند.

(۲) درخت باینری مورب به چپ

(۳) درخت باینری مورب به راست

(۴) زیر درخت‌های چپ و راست درخت باینری تعویض می‌شود.

۳۸- با داده‌های زیر یک درخت جستجوی باینری می‌سازیم:

۱۵۰۰ و ۱۲۰۰ و ۱۰۵۰ و ۱۰۰۰ و ۱۱۰۰ و ۱۰۲۵ و ۱۴۰۰ و ۱۳۰۰

اگر عدد ۱۰۲۵ را از درخت حذف کنیم در آن صورت پیمایش preorder درخت پس از

(علوم کامپیوتر ۸۵)

حذف برابر است با:

(۱) ۱۵۰۰ و ۱۲۰۰ و ۱۴۰۰ و ۱۱۰۰ و ۱۰۰۰ و ۱۰۵۰ و ۱۳۰۰

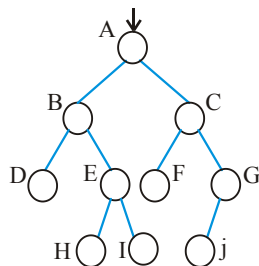
(۲) ۱۴۰۰ و ۱۵۰۰ و ۱۲۰۰ و ۱۰۵۰ و ۱۱۰۰ و ۱۰۰۰ و ۱۳۰۰

(۳) ۱۵۰۰ و ۱۴۰۰ و ۱۲۰۰ و ۱۱۰۰ و ۱۰۵۰ و ۱۰۰۰ و ۱۳۰۰

(۴) ۱۵۰۰ و ۱۴۰۰ و ۱۲۰۰ و ۱۱۰۰ و ۱۰۰۰ و ۱۰۵۰ و ۱۳۰۰

۳۹- اگر T یک درخت جستجوی دودویی به صورت زیر باشد که در هر گره آن یک عدد صحیح

ذخیره شده است چهارمین کوچکترین عنصر آن در کدام گره قرار دارد؟ (علوم کامپیوتر ۸۶)



(۱) D

(۲) I

(۳) H

(۴) E

۴۰- بهترین جواب را علامت بزنید. ساختن یک درخت دودویی جستجو برای  $n$  عنصر داده شده حتماً از مرتبه زیر است؟  
(ساختمان داده‌ها IT ۸۴)

(۱)  $\theta(n^2)$  (۲)  $\Omega(n^2)$  (۳)  $\theta(n \log n)$  (۴)  $\Omega(n \log n)$

۴۱- یک درخت جستجوی دودویی ۱۵۰ گره دارد. کدام یک از موارد زیر صحیح است؟  
(ساختمان داده‌ها IT ۸۵)

(۱) این درخت حداقل ۶ سطح و حداکثر ۸ سطح دارد.

(۲) این درخت حداقل ۶ سطح و حداکثر ۷ سطح دارد.

(۳) این درخت حداقل ۷ سطح و حداکثر ۸ سطح دارد.

(۴) این درخت حداقل ۷ سطح و حداکثر ۱۵۰ سطح دارد.

۴۲- کدام گزینه پیمایش Preorder یک درخت جستجوی دودویی BST با پیمایش Postorder به صورت زیر است؟  
(ساختمان داده‌ها IT ۸۶ مشابه دولتی ۸۱)

Postorder : ۵ و ۶ و ۱۵ و ۱۰ و ۲۳ و ۲۴ و ۲۲ و ۲۶ و ۲۰

(۱) ۲۰ و ۲۶ و ۲۲ و ۲۴ و ۱۰ و ۶ و ۵ و ۱۵ و ۲۳ و ۲۰

(۲) ۲۶ و ۲۳ و ۲۴ و ۲۲ و ۲۰ و ۱۵ و ۱۰ و ۵ و ۶

(۳) ۲۲ و ۲۳ و ۲۴ و ۲۶ و ۱۵ و ۵ و ۶ و ۱۰ و ۲۰

(۴) ۲۳ و ۲۴ و ۲۲ و ۲۶ و ۱۵ و ۵ و ۶ و ۱۰ و ۲۰

۴۳- در یک درخت دودویی جستجوی T، اگر x برگ و y پدر x باشد، کدام گزینه در مورد مقادیر  $a = \text{key}[x]$  و  $b = \text{key}[y]$  درست است؟  
(دولتی ۸۰)

(۱) b بزرگترین کلید در T است که کوچکتر از a باشد.

(۲) a کوچکترین کلید در T است که بزرگتر از b باشد.

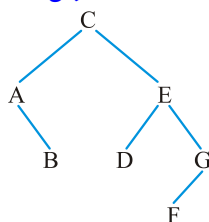
(۳) b کوچکترین کلید در T است که بزرگتر از a باشد.

(۴) هیچکدام از گزینه‌های فوق همواره درست نیست.

۴۴- با n عنصر متفاوت، چند درخت دودویی جست و جوی متفاوت به ارتفاع  $n-1$  وجود دارد؟  
(ساختمان داده‌ها دولتی ۸۱)

(۱) ۱ (۲) ۲ (۳)  $n!$  (۴)  $2^{n-1}$

۴۵- کدام یک از دنباله‌های زیر که هر یک نشان‌دهنده ترتیب درج عناصر از چپ به راست در یک درخت دودویی جستجو تهی است، درخت زیر را تولید نمی‌کند؟ (ساختمان داده‌ها دولتی ۸۳)



(۱) C A E G D B F

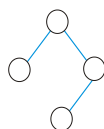
(۲) C A B E G D F

(۳) C E G A F D B

(۴) C E B G A D F

۴۶- چند حالت عناصر با کلیدهای  $a < b < c < d$  را می‌توان وارد یک درخت دودویی جست و

(ساختمان داده‌ها دولتی ۸۴)



جوی تهی کرد تا درختی به شکل زیر ایجاد شود؟

(۱) ۴      (۲) ۳  
(۳) ۲      (۴) ۱

۴۷- گزینه صحیح را انتخاب کنید.

(علوم کامپیوتر ۸۰)

(۱) حداکثر عمق درخت جستجوی باینری برابر با  $O(\log n)$  است.

(۲) حداکثر عمق درخت باینری heap برابر با  $O(n)$  است.

(۳) حداکثر عمق درخت جستجوی باینری برابر با  $O(n)$  است.

(۴) حداکثر عمق درخت باینری heap و نصف باینری جستجو برابر  $O(\log n)$  است.

۴۸- پیچیدگی زمان تبدیل یک heap با  $n$  عنصر به یک درخت جستجو باینری حداقل برابر

(علوم کامپیوتر ۸۲)

است با:

(۱)  $O(\log n)$       (۲)  $O(n)$       (۳)  $O(n^2)$       (۴)  $O(n \log n)$

۴۹- کدام یک از عبارات زیر همیشه صحیح است؟

(ساختمان داده‌ها IT ۸۵)

(۱) کوچکترین گره در یک درخت جستجوی دودویی همیشه برگ است.

(۲) درج در یک درخت دودویی جستجو می‌تواند در زمان حداکثر  $O(1)$  انجام شود.

(۳) پیمایش میان ترتیب یک Max heap عناصر موجود در آن را به صورت مرتب طی می‌کند.

(۴) پیمایش بین ترتیب یک درخت دودویی جستجو عناصر موجود در آن را به صورت مرتب طی می‌کند.

۵۰-  $n$  عدد در آرایه  $T$  ذخیره شده‌اند. برای  $T$ ، الگوریتم ذکر شده را اجرا می‌کنیم. اگر  $t$  تعداد

دفعات اجرای حلقه repeat باشد، کدام یک از گزاره‌های زیر صحیح است با فرض این که

(ساختمان داده‌ها - مهندسی کامپیوتر ۸۶)

$k = \lceil \lg n \rceil$ .

Procedure  $M(T[1..n])$

(۱)  $t \leq 2 \times 2^{k-2} + 3 \times 2^{k-3} + \dots + k$

for  $i \leftarrow \lfloor \frac{n}{2} \rfloor$  downto ۱ do

(۲)  $t \leq 3 \times 2^{k-2} + 4 \times 2^{k-3} + \dots + k$

$R \leftarrow i$

(۳)  $t \leq 3 \times 2^{k-2} + 4 \times 2^{k-3} + \dots + (k+1)$

Repeat

(۴)  $t \leq 2 \times 2^{k-1} + 3 \times 2^{k-2} + \dots + (k+1)$

$j \leftarrow R$

if  $2j \leq n$  and  $T[2j] > T[R]$  then

$R \leftarrow 2j$

if  $2j+1 \leq n$  and  $T[2j+1] > T[R]$  then

$R \leftarrow 2j+1$

swap  $T[j], T[R]$

until  $j = R$

۵۱- چند درخت جستجوی باینری با عمق ۴ می‌توان با این داده‌ها ساخت؟ (ریشه در سطح

(علوم کامپیوتر ۸۷)

شماره ۱) {۱۶ و ۱۰ و ۵ و ۴ و ۱}

$$C_5 = \frac{1}{6} \binom{10}{5} (4)$$

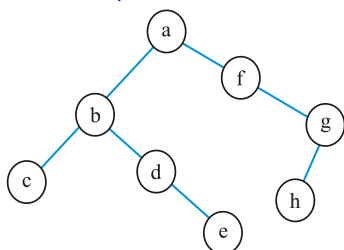
۵! (۳)

۲۵ (۲)

۲۲ (۱)

(علوم کامپیوتر ۸۷)

۵۲- نمایش درخت زیر به کدام صورت درست می‌باشد؟



(۱) (a,b(c,d),e,f(g(h)))

(۲) (a(b(c,d),e),f(g(h)))

(۳) (a(b(c,d(e),f(g(h))))

(۴) (a(b(c,d(e)),f(g(h))))

۵۳- اگر یک درخت جستجوی باینری با n گره داشته باشیم کمترین پیچیدگی زمان پیدا کردن

عنصر میانه (median) برای این n عدد در داخل درخت چقدر می‌باشد؟ (علوم کامپیوتر ۸۷)

(۴)  $O(n \lg n)$

(۳)  $O(n^2)$

(۲)  $O(n)$

(۱)  $O(1)$

۵۴- پیچیدگی زمان ساخت یک heap از حل کدام فرمول بازگشتی زیر حاصل می‌شود؟

(علوم کامپیوتر ۸۷)

(۱)  $T(n) = T\left(\frac{n}{2}\right) + O(1)$

(۲)  $T(n) = T(n-1) + T(n-2) + O(n)$

(۳)  $T(n) = 2T\left(\frac{n}{2}\right) + O(n)$

(۴)  $T(n) = T(n-K+1) + T(K) + O(1), 1 < K < n$

۵۵- اگر ارتفاع یک گره را فاصله طولانی‌ترین مسیر آن تا برگ در نظر بگیریم، حداکثر چند گره

(۸۷ IT)

با ارتفاع h در یک heap تشکیل شده از n عنصر وجود دارد؟

(۴)  $\left\lceil \frac{n}{2^h} \right\rceil$

(۳)  $\left\lceil \frac{n}{2^{h+1}} \right\rceil$

(۲)  $\frac{n+1}{2^h}$

(۱)  $\left\lceil \frac{n+1}{2^h} \right\rceil$

۵۶- در پیمایش غیر بازگشتی درخت باینری به صورت preorder در بهترین حالت گره‌های با

درجه ۰، ۱ و ۲ هر کدام چند بار وارد استک می‌شوند؟ (جواب از چپ به راست)

(علوم کامپیوتر ۸۸)

(۴) ۲, ۲, ۲

(۳) ۰, ۲, ۲

(۲) ۰, ۱, ۲

(۱) ۰, ۰, ۱



۵۷- اگر دنباله‌ی زیر یک max-heap را بخواهد نشان دهد کدام اعداد باید با چه مقادیر تعویض شوند؟

(علوم کامپیوتر ۸۸)

۱۱، ۷، ۵، ۱، ۱۰، ۳، ۶، ۱۴، ۱۷، ۲۳

(a به b تغییر نماید یعنی  $a \rightarrow b$ )

(۱)  $6 \rightarrow 12, 3 \rightarrow 15$  (۲)  $10 \rightarrow 11, 1 \rightarrow 8$

(۳)  $6 \rightarrow 9, 3 \rightarrow 10$  (۴)  $7 \rightarrow 8, 14 \rightarrow 15$

۵۸- فرض کنید T یک درخت دودویی کامل با n گره و به ارتفاع lgn است. می‌خواهیم یک مسیر از یک رأس u به یک رأس دیگر به نام v پیدا کنیم. گره‌های u و v داده شده‌اند و می‌دانیم که هر گره از این درخت به گره‌های فرزند و گره‌ی پدر دسترسی دارد.

(تخصصی نرم‌افزار ۸۸)

(۱) این کار را می‌توان در  $O(\lg^2 n)$  انجام داد.

(۲) این کار را نمی‌توان در کمتر از  $O(n)$  انجام داد.

(۳) سریع‌ترین روش استفاده از الگوریتم دایکسترا است.

(۴) این کار متناسب با ارتفاع درخت و با  $O(\lg n)$  امکان‌پذیر است.

۵۹- خروجی الگوریتم زیر برای درخت باینری داده شده کدام است؟

(علوم کامپیوتر ۸۹)

IPP(Tree \*T)

{ if T {

cout << T → data;

IPP(T → left);

cout << T → data;

IPP(T → right);

cout << (T → data);

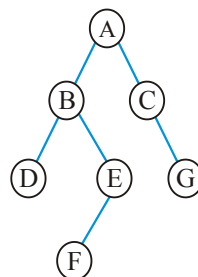
}

ABDDDBEFFFEEBACCGGGCA (۲)

ABCDEF CGDDBFEACG (۱)

ABDEF CGABDEF CGABEF CG (۴)

ABDDBEFFEBACGGCA (۳)



۶۰- در الگوریتم heapify یک آرایه دلخواه با n عنصر،  $A[i]$  در heap در عمق a خواهد بود و تعداد مقایسه‌ها برای گنجاندن این عنصر در بدترین حالت b خواهد بود. مقدار a و b چیست؟

(علوم کامپیوتر ۸۹)

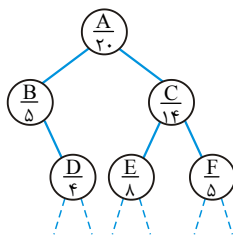
(۲)  $a = \lceil \lg i \rceil, b = 2 \lceil \lg i \rceil$

(۱)  $a = \left\lceil \lg \frac{n}{i} \right\rceil, b = 2 \left\lceil \lg \frac{n}{i} \right\rceil$

(۴)  $a = \lfloor \lg n \rfloor, b = 2 \lfloor \lg n \rfloor$

(۳)  $a = \lceil \lg n \rceil, b = 2 \lceil \lg n \rceil$

۶۱- در شکل زیر قسمتی از یک درخت دودویی نشان داده شده است. در زیر برچسب هر گره عددی نوشته شده است که تعداد کل گره‌های موجود در زیر درخت آن گره به علاوه خود آن گره را نشان می‌دهد. در هر یک از پیمایش‌های preorder و postorder گره F چندمین گره خواهد بود؟

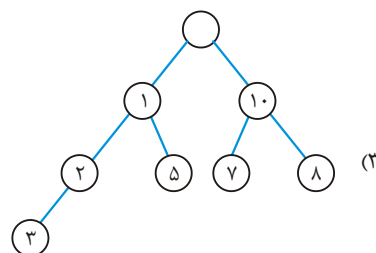
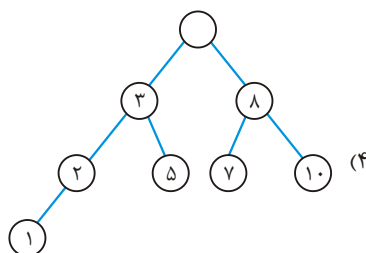
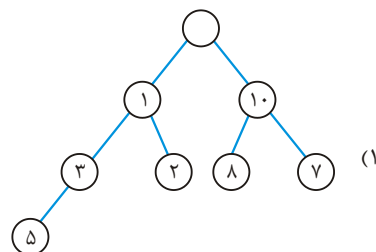
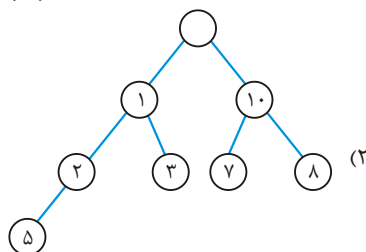


- (۱) ۱۵ ام در preorder و ۱۹ در postorder  
 (۲) ۱۶ ام در preorder و ۱۸ در postorder  
 (۳) ۱۶ ام در preorder و ۱۵ در postorder  
 (۴) ۱۹ ام در preorder و ۱۸ در postorder

۶۲- ساختار Deap حاصل از اعداد زیر که به ترتیب از چپ به راست وارد می‌شوند چیست؟

(علوم کامپیوتر ۹۱)

۵, ۱, ۷, ۲, ۳, ۸, ۱۰



۶۳- فرض کنید دو هرم مینیمم (min-heap) هر یک دارای n عدد داده شده باشند. ادغام این دو هرم در یک آرایه به صورت صعودی دارای چه مرتبه زمانی است؟

(علوم کامپیوتر ۹۱)

(۲)  $O(\log n)$

(۱)  $O(n)$

(۴)  $O(n \log^2 n)$

(۳)  $O(n \log n)$

۶۴- تابع  $\text{aproc}()$  بصورت زیر تعریف شده است،  $\text{swap}(\circ, \circ)$  تابعی است که مقدار عملوندهایش را جابجا می‌کند.

(علوم کامپیوتر ۸۳)

```
void aproc (int * A, int n)
{
    If (n>1)
        if  $\left( A[n] < A\left[\frac{n}{2}\right] \right)$ 
        {
            swap  $\left( A[n], A\left[\frac{n}{2}\right] \right);$ 
            aproc  $\left( A, \frac{n}{2} \right);$ 
        }
}
```

با اجرای حلقه

```
For (i=1; i<=n; ++i)
    aproc (A,i);
```

(علوم کامپیوتر ۸۳)

چه مقداری در  $A[1]$  خواهد بود؟

(۱) مقدار میانه عناصر  $A$

(۲) کمترین مقدار در بین عناصر  $A$

(۳) بیشترین مقدار در بین عناصر  $A$

(۴)  $A[1]$  تغییر نمی‌کند و مقدار آن همان مقدار قبل از اجرای برنامه است.

۶۵- یک درخت دودویی جست‌وجوی متوازن شامل  $n$  عدد متمایز داده شده است. فرض کنید

که به دلیل وجود نوپز عدد داخل یکی از گره‌ها تغییر می‌کند. با چه مرتبه‌ی زمانی می‌توان تشخیص داد که آیا درخت جدید هم‌چنان یک درخت دودویی جست‌وجوی معتبر هست یا خیر؟ بهترین گزینه را انتخاب کنید.

(IT - ۹۳)

(۱)  $O(\log n)$  (۲)  $O(n)$  (۳)  $O(n \log n)$  (۴)  $O(n^2)$

۶۶- یک هرم بیشینه (max-heap) به اندازه‌ی  $m$  و یک هرم کمینه (min-heap) به اندازه‌ی  $n$

موجودند. بهترین الگوریتم برای ادغام این دو و ایجاد یک هرم بیشینه از کدام مرتبه است؟

(IT - ۹۳)

(۱)  $O(n + m)$  (۲)  $O(n \log m + m \log n)$  (۳)  $O(n \log n + m \log m)$  (۴)  $O(\min\{n \log m, m \log n\})$

۶۷- چند تا از گزینه‌های زیر در مورد یک عبارت ریاضی E با عملگرهای دودویی و یکانی (unary) درست‌اند؟

الف) تنها با داشتن نگارش پیشوندی (prefix) E می‌توان در  $O(n)$  درخت عبارت آن را به دست آورد.

ب) از نگارش پیشوندی E می‌توان مستقیماً و در  $O(n)$  نگارش پسوندی (postfix) آن را به دست آورد.

ج) از نگارش میانوندی با پرانتز کامل (infix) E می‌توان در  $O(n)$  درخت عبارت آن را به دست آورد. (IT-۹۳)

۰ (۱) ۱ (۲) ۲ (۳) ۳ (۴)

۶۸- فرض کنید یک هرم کمینه شامل n عدد متمایز داده شده باشد. پنجمین کوچکترین عدد در کدام یک از درایه‌ها نمی‌تواند قرار بگیرد؟ (علوم کامپیوتر-۹۳)

۱ تا ۵ (۱) ۸ تا ۱۵ (۲) ۱۶ تا ۲۱ (۳) ۳۲ تا ۶۳ (۴)

۶۹- کدام یک از آرایه‌های زیر تشکیل یک minmax-heap را می‌دهد؟ (علوم کامپیوتر-۹۳)

۱) ۷, ۷۰, ۴۰, ۳۰, ۹, ۱۰, ۱۵, ۴۵, ۵۰, ۳۰, ۲۰, ۱۲

۲) ۷, ۷۰, ۴۰, ۱۵, ۹, ۱۰, ۳۰, ۴۵, ۵۰, ۳۰, ۲۰, ۱۲

۳) ۷, ۷۰, ۴۰, ۳۰, ۱۵, ۱۰, ۹, ۴۵, ۳۰, ۲۰, ۱۲

۴) ۷, ۷۰, ۴۰, ۳۰, ۹, ۱۰, ۱۵, ۱۲, ۵۰, ۳۰, ۲۰, ۴۵

۷۰- با چه هزینه زمانی می‌توان تشخیص داد یک درخت جستجوی دودویی BST (Binary Search Tree) با n گره، یک درخت AVL است یا نه؟

(درخت AVL یک BST است که در آن فاکتور توازن هر گره برابر، -۱، ۰ و +۱ می‌باشد و منظور از فاکتور توازن هر گره، اختلاف سطح (و یا ارتفاع) زیر درخت چپ آن گره با سطح (و یا ارتفاع) زیردرخت راست آن گره می‌باشد). (علوم کامپیوتر-۹۳)

۱)  $\theta(n)$  ۲)  $\theta(n^2)$  ۳)  $\theta(\log n)$  ۴)  $\theta(n \log n)$

۷۱- فرض کنید یک هرم کمینه با یک درخت متوازن پیاده‌سازی شده است. کدام گزینه نادرست است؟ (IT-۹۴)

۱) بزرگ‌ترین عنصر حتماً در یک برگ ذخیره شده است.

۲) سومین کوچک‌ترین عنصر حتماً فرزند ریشه است.

۳) دومین بزرگ‌ترین عنصر لزوماً در برگ ذخیره نمی‌شود.

۴) هر سه گزینه‌ی بالا

۷۲- برچسب هر گره از یک درخت دودویی یکی از حروف الفبای انگلیسی است. می‌دانیم هر حرف حداکثر دو بار به عنوان برچسب استفاده شده است. اگر رشته‌ی حاصل از پیمایش پیش ترتیب قرینه‌ی رشته‌ی حاصل از پیمایش پس ترتیب باشد، در مورد این درخت چه می‌توان گفت؟ (۹۴ - IT)

- (۱) پیمایش میان ترتیب درخت آینه‌ای است. (۲) درخت متوازن است.  
(۳) چنین درختی با حداقل ۶ گره وجود ندارد. (۴) هیچ‌کدام

۷۳- درخت قرمز سیاه با ۱۰۲۳ عنصر، دست‌کم چند گره قرمز باید داشته باشد؟ (۹۴ - IT)

(۱) ۰ (۲) ۱۰ (۳) ۵ (۴) ۱۱

۷۴- فرض کنید گره  $x$  باید بعد از گره  $n$  در یک لیست یک سویه درج شود. کدام گزینه به درستی اشاره‌گرها را مقداره‌ی می‌کند. (ترتیب عملیات‌ها از چپ به راست است و فرض کنید  $next[n]$  وجود دارد.) (۹۴ - IT)

- (۱)  $next[n] = x; next[x] = next[n];$   
(۲)  $next[n] = x; next[x] = next[next[n]];$   
(۳)  $next[x] = n; next[n] = x;$   
(۴)  $next[x] = next[n]; next[n] = x;$

۷۵- فرض کنید صف  $Q$  با یک آرایه‌ی حلقوی به اندازه  $m$  پیاده‌سازی شده است که اندیس‌های آن از ۰ تا  $m-1$  است و عناصر آن به صورت چرخه‌ای و در جهت ساعت‌گرد ذخیره شده‌اند. مؤلفه‌های  $front(Q)$  و  $rear(Q)$  به ترتیب اندیس اولین عنصر و عنصر بعد از آخرین عنصر صف  $Q$  را ذخیره می‌کنند. تعداد عناصر داخل صف و شرط پر بودن صف کدام گزینه زیر است؟ (۹۴ - IT)

- (۱)  $front(Q) = rear(Q)$  و  $rear(Q) - front(Q) + 1 \bmod m$   
(۲)  $front(Q) = rear(Q)$  و  $rear(Q) - front(Q) \bmod m$   
(۳)  $front(Q) = rear(Q) + 1 \bmod m$  و  $rear(Q) - front(Q) + 1 \bmod m$   
(۴)  $front(Q) = rear(Q) + 1 \bmod m$  و  $rear(Q) - front(Q) \bmod m$

۷۶- کدام گزینه در مورد درخت‌های دودویی صحیح می‌باشد؟ (تعداد کل گره‌ها بزرگتر یا مساوی یک فرض می‌شود.) (علوم کامپیوتر - ۹۴)

- (۱) همواره تعداد برگ‌ها و تعداد گره‌های تک فرزندی دو عدد متوالی می‌باشند.  
(۲) همواره تعداد برگ‌ها و گره‌های تک فرزندی برای درخت‌های کامل، دو عدد متوالی می‌باشند.

۳) همواره تعداد برگ‌ها و تعداد گره‌های دو فرزندی دو عدد متوالی می‌باشند.

۴) همواره تعداد گره‌های تک فرزندی و تعداد گره‌های دو فرزندی، دو عدد متوالی می‌باشند.

۷۷- در یک صف حلقوی به طول  $n$ ، مقدار متغیری که ابتدای صف را نشان می‌دهد ( $f$ ) چگونه

به روزرسانی می‌شود؟ (علوم کامپیوتر - ۹۴)

$$f = f + 1 \quad (۱) \quad f = (f + 1) \% n \quad (۲)$$

$$f = (f + 1) \% (n - 1) \quad (۳) \quad f = (f - 1) \% n \quad (۴)$$

۷۸- سه آرایه مرتب شده  $A_1, A_2, A_3$  با  $n$  عنصر از اعداد حقیقی متمایز داریم، می‌خواهیم از

روی مجموعه  $A_1 \cup A_2 \cup A_3$  یک درخت جستجوی دودویی متوازن بسازیم. بهترین

الگوریتم دارای چه مرتبه زمانی است؟ (علوم کامپیوتر - ۹۴)

$$O(n^2) \quad (۱) \quad O(n) \quad (۲) \quad O(n \log n) \quad (۳) \quad O(n^2 \log n) \quad (۴)$$

۷۹- اگر اعداد ۱ تا  $n$  را به ترتیب تصادفی در یک درخت جست و جوی دودویی درج کنیم، کدام

رابطه بازگشتی در مورد میانگین ارتفاع این درخت صحیح است؟ (دکتری کامپیوتر - ۹۵)

$$h(n) = \frac{1}{n} \sum_{i=1}^n (h(i-1) + h(n-i)) \quad (۱)$$

$$h(n) = \frac{1}{n} \sum_{i=1}^n (h(i-1).h(n-i)) \quad (۲)$$

$$h(n) = 1 + \frac{1}{n} \sum_{i=1}^n (h(i-1).h(n-i)) \quad (۳)$$

$$h(n) = 1 + \frac{1}{n} \sum_{i=1}^n \max \{h(i-1), h(n-i)\} \quad (۴)$$

۸۰- یک درخت دودویی (درخت ریشه‌دار که هر گره داخلی حداکثر دو فرزند دارد) را در نظر

بگیرید که در هر گره آن یک زوج مرتب  $(x, y)$  نگهداری می‌شود. این درخت برحسب درایه

اول (یعنی  $x$ ) یک درخت دودویی جست و جو و بر حسب درایه دوم یک هرم کمینه است

(یعنی مقدار  $y$  هر گره از مقدار  $y$  فرزنداناش کمتر است) به این درخت، درخت خوب گوییم. به

ازای  $n$  جفت  $(x_i, y_i)$  داده شده چند تا از گزینه‌های زیر درست‌اند؟ (دکتری کامپیوتر - ۹۵)

- درخت خوب آن یکتا است.

- لزوماً درخت خوب ندارد.

- درخت خوبی وجود دارد که ارتفاع آن از مرتبه  $O(\log n)$  است.

- جست و جوی یک زوج داده شده در درخت خوب همیشه از مرتبه  $O(\log n)$  است.

$$(۱) \quad ۳ \quad (۲) \quad ۲ \quad (۳) \quad ۱ \quad (۴) \quad ۰$$

۸۱- چند تا از گزاره‌های زیر همیشه درست‌اند؟ (دکتری کامپیوتر - ۹۵)

- بدون تغییر دیگری در درخت، گره‌های هر درخت دودویی جست و جو را می‌توان با رنگ‌های قرمز و سیاه رنگ کرد طوری که درخت حاصل قرمز - سیاه شود.
- بدون تغییر دیگری در درخت، گره‌های هر درخت دودویی جست و جو با  $n$  عنصر و ارتفاع حداکثر  $2 \log n$  را می‌توان با رنگ‌های قرمز و سیاه رنگ کرد طوری که درخت حاصل قرمز - سیاه شود.
- یک درخت دودویی جست و جوی کاملاً متوازن را می‌توان فقط با رنگ کردن گره‌هایش به صورت قرمز - سیاه در آورد.

(۱) ۰ (۲) ۱ (۳) ۲ (۴) ۳

۸۲- دو پیمایش  $\text{Pre order} = \{A, B, C, D\}$  و  $\text{Postorder} = \{A, B, C, D\}$  از درختی در دسترس هستند. تعداد درخت‌های پوشش‌دهنده این دو پیمایش چند تا است؟ (دکتری علوم کامپیوتر - ۹۵)

(۱) صفر (۲) ۱ (۳) ۴ (۴) ۱۴

۸۳- در یک درخت جست و جوی دودویی شامل اعداد  $\{1, 2, 3, \dots, 2n\}$  در یک برگ قرار گرفته است. (دکتری علوم کامپیوتر - ۹۵)

اگر  $x$  اشاره‌گر به گره حاوی  $n$  باشد، مقدار برگشتی  $\text{ancest}(x)$  چیست؟

```
int ancest (node * x) {
    node * p = x -> parent;
    while (p != NULL && x == p -> right) {
        x = p;
        p = x -> parent;
    }
    return p -> value;
}
```

(۱) ۱ (۲)  $n-1$  (۳)  $n+1$  (۴)  $2n$

۸۴- تعداد درخت‌های جستجوی دودویی با  $n$  ( $n \geq 2$ ) کلید متمایز، توسط کدامیک از روابط بازگشتی زیر می‌توانند تولید شوند؟ (دکتری علوم کامپیوتر - ۹۵)

$$C_n = \frac{4n-2}{n+1} C_{n-1} \quad (۲) \quad C_n = \frac{4n-1}{n+1} C_{n-1} \quad (۱)$$

$$C_n = \frac{4n-2}{n-1} C_{n-1} \quad (۴) \quad C_n = \frac{5n-3}{n+1} C_{n-1} \quad (۳)$$

۸۵- اگر  $F_n$ ،  $n$  امین عدد فیبونیچی (  $F_0 = 0$ ،  $F_1 = 1$  و  $F_n = F_{n-1} + F_{n-2}$  ) باشد و  $G_h$  کمینه تعداد گره‌های یک درخت AVL با ارتفاع  $h$  باشد ( $G_0 = 1$  و  $G_1 = 2$ ) آنگاه:

(دکتری علوم کامپیوتر - ۹۵)

$$G_h = F_{h+3} - 1 \quad (۱) \quad G_h = F_{h-1} + 1 \quad (۲)$$

$$G_h = F_{h+3} + 1 \quad (۳) \quad G_h = F_{h-1} + 3 \quad (۴)$$

۸۶- با داشتن هر یک از پیمایش پسوندی (Postorder)، میانوندی (Inorder) و پیشوندی (Preorder) به تنهایی، کدام گزینه تعداد درخت‌های دودویی را که می‌توان ساخت، نشان می‌دهد؟

(علوم کامپیوتر - ۹۵)

$$T(n) = \frac{1}{n} \binom{2n}{n} \quad (۱) \quad T(n) = \frac{1}{n+1} \binom{2n}{n} \quad (۲)$$

$$T(n) = \frac{1}{n} \binom{2n-2}{n-1} \quad (۳) \quad T(n) = \frac{1}{n+1} \binom{2n-2}{n-1} \quad (۴)$$

۸۷- فرض کنید  $a$  و  $b$  اشاره‌گر به گره وسط، به ترتیب در لیست‌های  $L_1$  و  $L_2$  هستند. با فرض

اینکه  $L_1$  و  $L_2$  لیست‌های پیوندی یکطرفه دایره‌ای هستند. فراخوانی  $\text{change}(a, b)$  چه

تغییراتی بر روی این لیست‌ها پدید می‌آورد؟ (علوم کامپیوتر - ۹۵)

```
void change(link *a, link *b){
    link *temp = a->next;
    a->next = b->next;
    b->next = temp;
}
```

(۱) دو لیست از تعویض نیمه دوم لیست‌ها با یکدیگر به دست می‌آید.

(۲) یک لیست پیوندی دایره‌ای از اتصال لیست‌ها به دست می‌آید.

(۳) در عمل باعث می‌شود لیست‌های  $L_1$  و  $L_2$  با یکدیگر جابجا شوند.

(۴) چهار لیست پیوندی دایره‌ای جدید به وجود می‌آیند، که هر یک شامل نیمه‌ای از لیست‌های اولیه است.

۸۸- در رابطه با درختان پر (full) و کامل (complete)، کدام عبارت صحیح است؟

(علوم کامپیوتر - ۹۵)

(۱) هر درخت باینری یا پر است یا کامل

(۲) هیچ درخت باینری پر و کامل نیست.

(۳) هر درخت باینری کامل، درخت باینری پر نیز است.

(۴) هر درخت باینری پر، درخت باینری کامل نیز است.



۸۹- کدام گزاره (گزاره‌ها) در مورد درخت AVL صحیح است؟ (علوم کامپیوتر - ۹۵)

گزاره ۱: اگر  $F_n$ ،  $\ln$  امین عدد فیبوناچی  $(F_n = F_{n-1} + F_{n-2}, F_1 = 1, F_0 = 0)$  باشد، کمینه تعداد گره‌های یک درخت AVL با ارتفاع  $h$  برابر خواهد بود با  $F_{h+2} - 1$ .

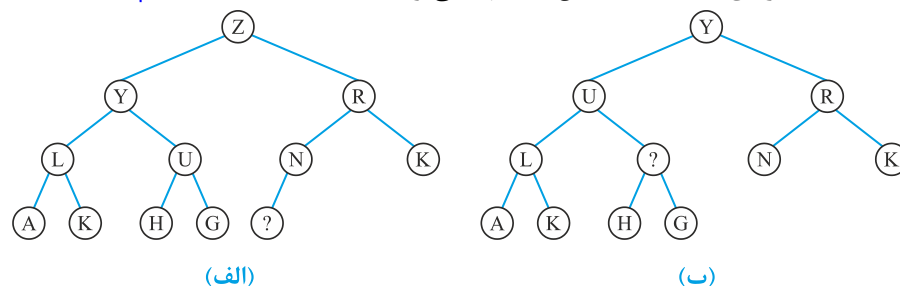
گزاره ۲: یک درخت AVL با ارتفاع ۴، حداقل ۱۲ گره دارد.

گزاره ۳: تعداد درخت‌های ممکن AVL با کلیدهای ۱، ۲ و ۳ برابر یک درخت می‌باشد.

(۱) فقط گزاره ۱ (۲) فقط گزاره ۱ و ۲ (۳) هیچکدام از گزاره‌ها صحیح نیستند. (۴) هر سه گزاره صحیح هستند.

۹۰- عمل حذف ماکزیمم روی درخت هیپ (heap) (الف) انجام شده و درخت (ب) تولید شده

است. داده گره‌ای که با (؟) مشخص شده چه می‌تواند باشد؟ (علوم کامپیوتر - ۹۵)



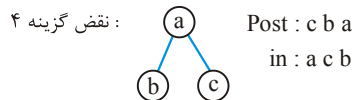
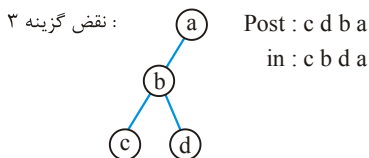
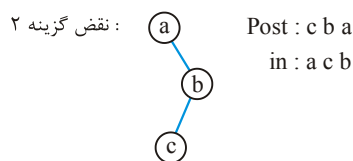
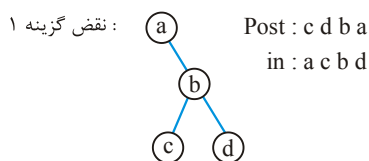
(۲) یکی از H, I, J, K, L, M, N  
(۴) M

(۱) یکی از I, J, K, L, M  
(۳) I

## پاسخنامه سؤال‌های چهارگزینه‌ای فصل ششم

- ۱- گزینه (۳). همه پیمایش‌ها  $O(n)$  هستند.
- ۲- گزینه (۴). درختی که پیمایش‌های  $inorder$  و  $postorder$  آن یکسان است، مورب چپ است و درخت مورب چپ برچسب‌دار با  $n$  نود به تعداد  $n!$  است.
- ۳- گزینه (۱). ابتدا زیر درخت چپ  $a$  که شامل ۱۸ نود است پیمایش می‌شود و سپس خود  $a$  ملاقات می‌شود. پس از آن زیردرخت چپ  $f$  که شامل ۵ نود است و سپس خود  $f$  :  
 $18 + 1 + 5 + 1 = 25$

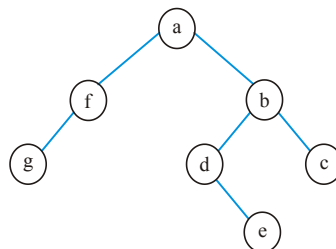
- ۴- گزینه (۴). پیمایش‌های درخت از مرتبه تعداد نودهای درخت هستند.
- ۵- گزینه (۱). اگر داده‌ها تکراری باشند درخت‌های متفاوتی می‌توان ساخت.
- ۶- گزینه (۴). کلید این سؤال گزینه ۲ اعلام شده ولی با چند مثال می‌توان نشان داد هیچ گزینه‌ای صحیح نیست. درخت مورب راست جواب این تست است که در گزینه‌ها نیست.



- ۷- گزینه (۳). این پیمایش ابتدا ریشه سپس زیردرخت راست و در نهایت چپ را ملاقات می‌کند که معکوس  $postorder$  است.

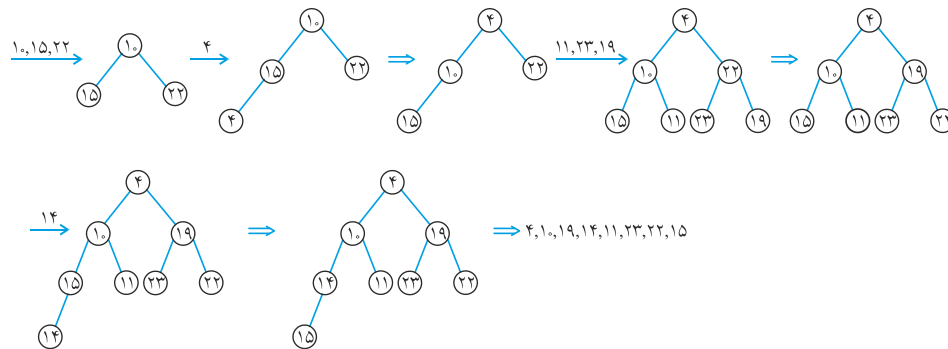
۸- گزینه (۲).

$inorder : g, f, a, d, e, b, c$   
 $preorder : a, f, g, b, d, e, c$

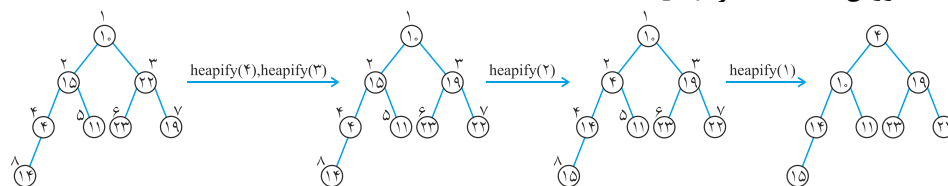


- ۹- گزینه (۳). حذف و درج در هیپ از مرتبه  $O(Lgn)$  است.

- ۱۰- گزینه (۴). عمق گره فاصله آن تا ریشه است. پس عمق ریشه (گره ۱) صفر است و عمق گره‌های ۲ و ۳ برابر ۱ است. عمق گره‌های ۴ و ۵ و ۶ و ۷ برابر ۲ است که گزینه ۴ حاصل می‌شود.
- ۱۱- گزینه (۴). دو روش تبدیل به minheap وجود دارد. روش ۱: درج:



روش ۲: استفاده از heapify :



در این تست، جواب هر دو روش یکسان شد، ولی لزوماً همیشه این اتفاق نمی‌افتد. بنابراین به طور پیش فرض از روش ۲ استفاده کنید مگر اینکه گفته شود، اعداد یکی یکی وارد می‌شوند که در این صورت از روش ۱ استفاده کنید. در کل مرتبه روش ۱،  $O(n \lg n)$  و روش ۲،  $O(n)$  است.

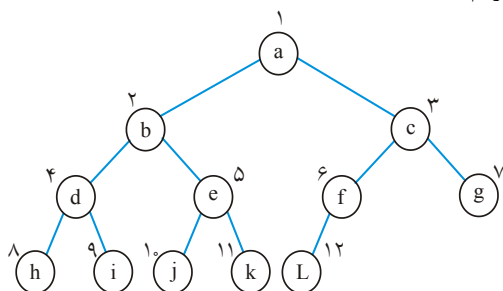
- ۱۲- گزینه (۴). درخت دارای ۱۰ سطح است و حداقل از ۲۳ عدد بیش از ۹،۰۰۰ عدد در سطوح ۱ تا ۹ هستند پس حداکثر  $23 - 9 = 14$  عدد بیش از ۱۰۰۰ در سطح ۱۰ می‌توانند باشند.

- ۱۳- گزینه (۴). درواقع باید هیپ ساخته شود که از مرتبه  $O(n)$  است. گزینه ۱ نیز می‌تواند صحیح باشد چون معادل  $O(n)$  است.

- ۱۴- گزینه (۳). اگر درخت پر باشد، حاصل Minheap است ولی در غیر این صورت درخت حاصل کامل نیست.

- ۱۵- گزینه (۱). از هیپ  $m$  عنصری حذف می‌کنیم و در هیپ  $n$  عنصری درج:  $m \log m + m \log n = m \log mn$  البته ادغام دو هیپ که جمعاً  $m + n$  عنصر دارند  $O(m + n)$  است! و طراح محترم با روش نامناسب، عمل ادغام انجام داده است. در این تست فرض شده است که  $m < n$ .

۱۶- گزینه (۴) با عمل heapify، هیپ می‌سازیم:



۱ مقایسه: heapify (۶)

۲ مقایسه: heapify (۵)

۲ مقایسه: heapify (۴)

۳ مقایسه: heapify (۳)

۴ مقایسه: heapify (۲)

۶ مقایسه: heapify (۱)

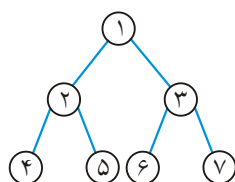
جمع مقایسه‌ها ۱۸ می‌شود.

۱۷- گزینه (۲). با توجه به درخت مقابل اگر  $i = 4$  باشد

$j = 6$  متناظر آن است. که در گزینه‌های ۲ و ۳ صدق می‌کند.

ولی اگر گره‌های ۶ و ۷ وجود نداشته باشد آنگاه متناظر گره ۴ گره

۳ می‌باشد (متناظر پدرش) پس اگر  $j$  وجود نداشت باید  $\left\lfloor \frac{j}{2} \right\rfloor$  را



بیابیم که گزینه ۲ حاصل می‌شود.

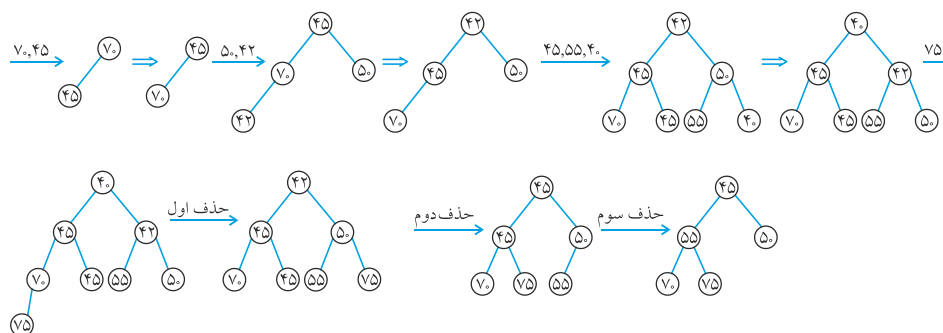
۱۸- گزینه (۲ و ۳). می‌توان آخرین نود یکی از هیپ‌ها را برداشته و یک نود جدید ایجاد کنیم و

سپس ریشه دو هیپ را به ترتیب چپ و راست نود جدید قرار دهیم و روی آن عمل

heapify انجام دهیم که از مرتبه  $O(\lg n)$  می‌شود. البته درخت حاصل لزوماً کامل نخواهد

بود و به شکل درختی است.

۱۹- گزینه (۳).

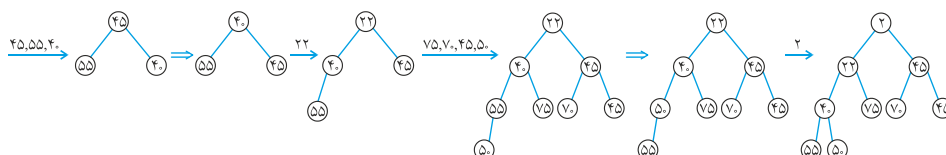


۲۰- گزینه (۱).

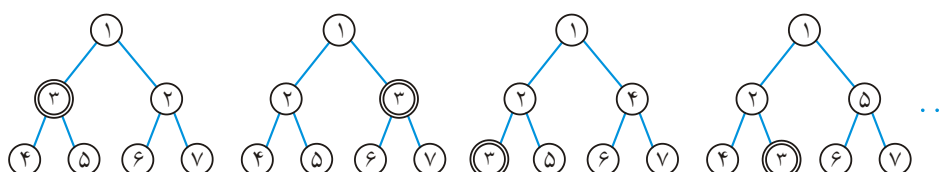
|      |   |    |   |   |   |   |
|------|---|----|---|---|---|---|
| A[۱] | ۲ | ۳  | ۴ | ۵ | ۶ | ۷ |
| ۲۰   | ۵ | ۱۰ | ۶ | ۳ | ۴ | ۱ |

$A[2]$  از فرزند چپ خود یعنی  $A[4]$  کوچکتر است.

۲۱- گزینه (۱).



۲۲- گزینه (۲). به minheap های زیر دقت کنید. عدد ۳، سومین کوچکترین است:



۳ می‌تواند در اندیس ۲ تا ۷ قرار گیرد.

۲۳- گزینه (۱).

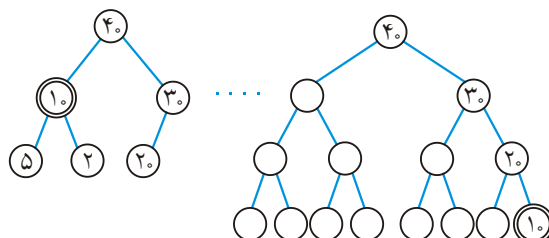
|      |    |    |    |    |    |    |    |    |    |
|------|----|----|----|----|----|----|----|----|----|
| A[۱] | ۲  | ۳  | ۴  | ۵  | ۶  | ۷  | ۸  | ۹  | ۱۰ |
| ۱۰۰  | ۹۰ | ۸۲ | ۸۵ | ۷۴ | ۷۵ | ۷۳ | ۶۸ | ۷۰ |    |

عدد ۹۵ را در خانه ۱۰ قرار می‌دهیم با پدرش یعنی ۷۴ تعویض می‌شود سپس با A[۲] یعنی ۹۰ تعویض می‌شود.

۲۴- گزینه (۱). از اندیس ۱ تا  $\left\lfloor \frac{n}{2} \right\rfloor$  باید بررسی کنیم که هر نود از فرزندانش کمتر نباشد.

۲۵- گزینه (۲). مینیمم در یکی از برگ‌هاست و تعداد برگ‌ها  $\left\lceil \frac{n}{2} \right\rceil$  می‌باشد.

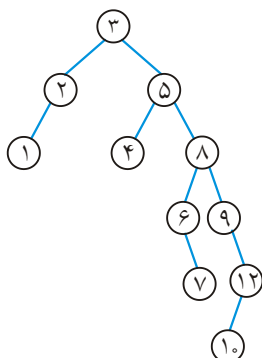
۲۶- گزینه (۴). چهارمین بزرگترین می‌تواند در درایه ۲ تا ۱۵ باشد (یعنی سطح دوم و سوم و چهارم) به هیپ‌های زیر دقت کنید، عدد ۱۰ چهارمین بزرگترین است:



۲۷- گزینه (۲). تعداد سطوح این درخت ۷ است. در سطح آخر، بزرگترین عددی که می‌تواند قرار گیرد هفتمین ماکزیمم یعنی ۵۸ است.

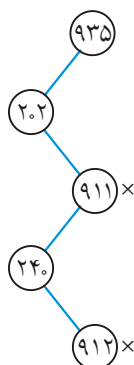
۲۸- گزینه (۴). heapsort همواره از مرتبه  $\theta(n \lg n)$  است. (البته به شرطی که اعداد متمایز باشند!)

۲۹- گزینه (۱).  $s$  ریشه،  $e$  از  $s$  کمتر است (طبق ترتیب حروف الفبا) و سمت چپ ریشه قرار می‌گیرد و ...



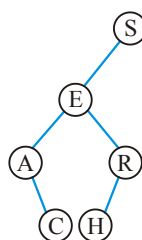
۳۰- گزینه (۳). اگر سطح ریشه را یک بگیریم ولی اگر ارتفاع را یک واحد کمتر از تعداد سطوح بگیریم ارتفاع این درخت ۵ است. در تست‌های اخیر ارتفاع این درخت را ۵ فرض کنید.

۳۱- گزینه (۱).



۳۲- گزینه (۱).

| کاراکتر      | S                             | E | A | R | C | H |
|--------------|-------------------------------|---|---|---|---|---|
| تعداد مقایسه | ۱                             | ۲ | ۳ | ۳ | ۴ | ۴ |
| مجموع مقایسه | $17 = 1 + 2 + 3 + 3 + 4 + 4$  |   |   |   |   |   |
| متوسط        | $\frac{17}{6} = 2\frac{5}{6}$ |   |   |   |   |   |



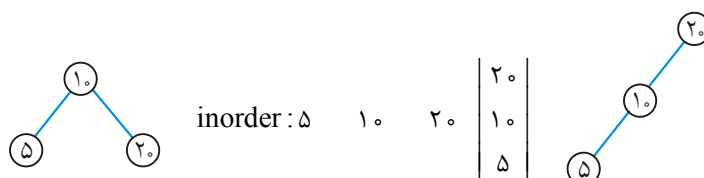
۳۳- گزینه (۱). سایر گزینه‌ها از مرتبه  $O(h)$  هستند.

۳۴- گزینه (۲). اگر درخت مورب باشد، حداکثر مقایسه  $O(n)$  است.

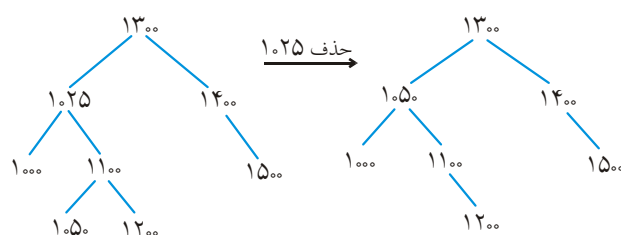
۳۵- گزینه (۴). در BST اگر فقط یک پیمایش preorder یا postorder یا Levelorder موجود باشد، درخت منحصر به فرد است.

۳۶- گزینه (۳). مینیمم زیر درخت راست (f) یا ماکزیمم زیر درخت چپ (e) را می‌توان جایگزین کرد.

۳۷- گزینه (۲).

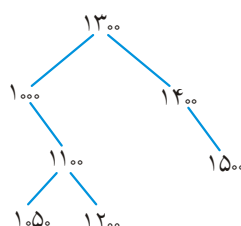


۳۸- گزینه (۴).



Pre : ۱۳۰۰ و ۱۰۵۰ و ۱۰۰۰ و ۱۱۰۰ و ۱۲۰۰ و ۱۴۰۰ و ۱۵۰۰

البته می‌توان پس از حذف ۱۰۲۵، ۱۰۰۰ را جایگزین آن کرد:



Pre : ۱۳۰۰ و ۱۰۰۰ و ۱۱۰۰ و ۱۰۵۰ و ۱۲۰۰ و ۱۴۰۰ و ۱۵۰۰

که در گزینه‌ها نیست.

۳۹- گزینه (۴). inorder پیمایش کنید، چهارمین خروجی جواب است.

D B H (E) ...

۴۰- گزینه (۴). بهترین حالت را  $\Omega$  نمایش می‌دهیم. درواقع ساخت BST با  $n$  عنصر متمایز از

مرتبه  $n \lg n$  یا بیشتر است.

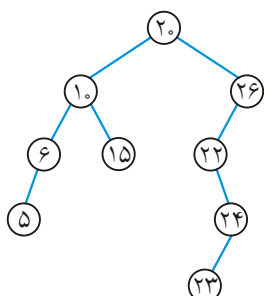
۴۱- گزینه صحیح وجود ندارد.

$$\text{حداقل سطوح} = \left\lceil \log_2^{150} \right\rceil + 1 = 8$$

$$\text{حداکثر سطوح} = 150$$

کلید گزینه ۴ اعلام شده که اگر بجای تعداد سطوح ارتفاع گفته می‌شد، حداقل ارتفاع ۷ و

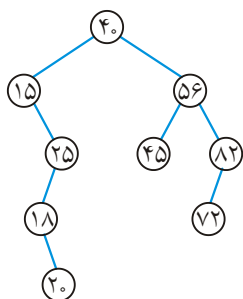
حداکثر ارتفاع ۱۴۹ است.



۴۲- گزینه (۴). می‌دانیم پیمایش inorder یک BST، لیست

صعودی ایجاد می‌کند و با توجه به پیمایش in و post می‌توان درخت را رسم کرد:

Pre : ۲۰ و ۱۰ و ۶ و ۵ و ۱۵ و ۲۶ و ۲۲ و ۲۴ و ۲۳



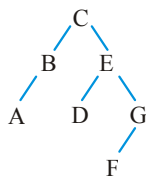
۴۳- گزینه (۴). درخت زیر را در نظر بگیرید.

اگر  $a = 72$  و  $b = 82$ ، گزینه ۱ و ۲ نادرست هستند.

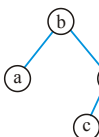
اگر  $a = 20$  و  $b = 18$ ، گزینه ۳ نادرست است.

۴۴- گزینه (۴). چون به جز ریشه،  $n-1$  عنصر دیگر، هر یک دو حالت دارند: چپ یا راست.

۴۵- گزینه (۴). درخت گزینه ۴ عبارت است از:



۴۶- گزینه (۲). درخت به شکل است که ۳ ترتیب ورود وجود دارد:



b , a , d , c

b , d , a , c

b , d , c , a

۴۷- گزینه (۳). حداکثر عمق BST برابر  $O(n)$  و عمق هیپ همواره  $O(\lg n)$  است.

۴۸- گزینه (۴). درواقع باید BST بسازیم که حداقل از مرتبه  $\lg n$  است.

۴۹- گزینه (۴).

۵۰- گزینه (۴). این الگوریتم، ساخت هیپ را با بهترین روش نشان می‌دهد. این الگوریتم برای

تمام نودهای داخلی بررسی می‌کند آیا خاصیت هیپ را نقض کرده‌اند یا خیر.



هیپ  $n$  عنصری دارای ارتفاع  $K = \lfloor \lg n \rfloor$  است که در سطح آخر آن حداکثر  $2^k$  نود وجود دارد و همگی برگ هستند و حلقه repeat برای آنها اجرا نمی‌شود. در سطح ماقبل آخر حداکثر  $2^{k-1}$  نود وجود دارد که همگی لزوماً داخلی نیستند و برای نودهای داخلی در این سطح، حلقه repeat حداکثر ۲ بار اجرا می‌شود. در سطح ماقبل آخر،  $2^{k-2}$  نود وجود دارد که برای هر یک حداکثر ۳ بار repeat اجرا می‌شود و به همین ترتیب پس اگر  $t$  تعداد اجرای حلقه repeat باشد آنگاه:

$$t \leq 2 \times 2^{k-1} + 3 \times 2^{k-2} + \dots + (k+1) \times 2^0$$

با کمک این نامساوی می‌توان نشان داد مرتبه ساخت هیپ  $O(n)$  است:

$$t \leq -2^k + 2^k + 3 \times 2^{k-1} + 3 \times 2^{k-2} + \dots + (k+1) \times 2^0$$

$$t < -2^k + 2^{k+1} (2^{-1} + 2 \times 2^{-2} + 3 \times 2^{-3} + \dots)$$

$$t < -2^k + 2^{k+1} \sum_{n=1}^{\infty} n \left(\frac{1}{2}\right)^n = -2^k + 2^{k+1} \times 2 = 3 \times 2^k$$

$$\Rightarrow t < 3n \Rightarrow t \in O(n)$$

(توجه: می‌دانیم  $\sum_{k=1}^{\infty} x^k = \frac{1}{1-x}$  وقتی  $|x| < 1$  حال اگر از طرفین مشتق بگیریم و در  $x$

ضرب کنیم داریم  $\sum_{k=1}^{\infty} kx^k = \frac{x}{(1-x)^2}$  و اگر  $x = \frac{1}{2}$  و  $k = n$  آنگاه:

$$\left( \sum_{n=1}^{\infty} n \left(\frac{1}{2}\right)^n = \frac{\frac{1}{2}}{\left(1-\frac{1}{2}\right)^2} = 2 \right)$$

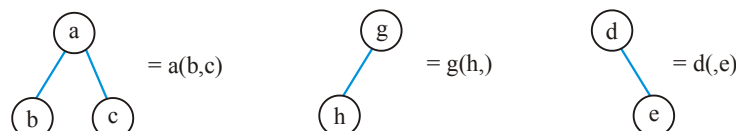
۵۱- گزینه «؟». با اعداد ۱۶، ۱۰، ۵، ۴، ۱ می‌توان BSTهای با ۳، ۴، ۵ سطح ساخت. تعداد

BSTهای با ۵ سطح برابر  $2^4 = 16$  است. تعداد BSTهای با ۳ سطح برابر  $\binom{4}{2} = 6$  است و

تعداد کل BSTها برابر  $\binom{10}{5} = 42$  است پس تعداد BSTها با ۴ سطح برابر است با:

$20 = 42 - (16 + 6)$  که ظاهراً هیچ گزینه‌ای صحیح نیست. کلید طراح گزینه ۱ است.

۵۲- گزینه (۴). فرم پرانتزی درخت خواسته شده است به نمونه‌های زیر توجه کنید:



۵۳- گزینه (۲). کافیت BST به صورت inorder پیمایش شود و این عمل تا نصف تعداد گره‌ها انجام شود. دقت کنید که inorder یک BST، عناصر را صعودی می‌کند. در ضمن همه پیمایش‌های درخت  $O(n)$  هستند.

۵۴- گزینه (۴). یک آرایه  $n$  عنصری فرض کنید اگر عنصر اول، ماکزیمم باشد که ریشه هیپ است و داریم:

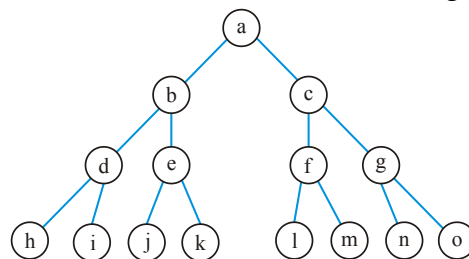
$$T(n) = T(n-1) + T(0)$$

اگر عنصر دوم ماکزیمم باشد پس عنصر دوم ریشه می‌شود و داریم:

$$T(n) = T(n-2) + T(2)$$

به همین ترتیب.

۵۵- گزینه (۳). به عنوان مثال:



با توجه به تعریف تست ارتفاع نودهای  $b$  و  $c$  برابر ۲ است یعنی با ارتفاع ۲، حداکثر ۲ نود وجود دارد که گزینه «۳» صادق است:

$$n = 15, h = 2 \Rightarrow \left\lceil \frac{15}{2^3} \right\rceil = 2$$

۵۶- گزینه (۱). درجه ۲ها وارد استک می‌شوند ولی اگر درخت مثلاً مورب چپ باشد، درجه صفرها و یک‌ها وارد استک نمی‌شوند.

۵۷- گزینه (۱).

|   |    |    |    |   |   |    |   |   |   |    |
|---|----|----|----|---|---|----|---|---|---|----|
|   | ۱  | ۲  | ۳  | ۴ | ۵ | ۶  | ۷ | ۸ | ۹ | ۱۰ |
| A | ۲۳ | ۱۷ | ۱۴ | ۶ | ۳ | ۱۰ | ۱ | ۵ | ۷ | ۱۱ |

عنصر  $A[4] = 6$  از فرزند راستش یعنی  $A[9] = 7$  کوچکتر است که اگر با عددی مثل ۱۲ جایگزین شود درست می‌شود. عنصر  $A[5] = 3$  از فرزندش  $A[10] = 11$  کوچکتر است که اگر با عنصری مثل ۱۵ جایگزین شود، درست می‌شود.

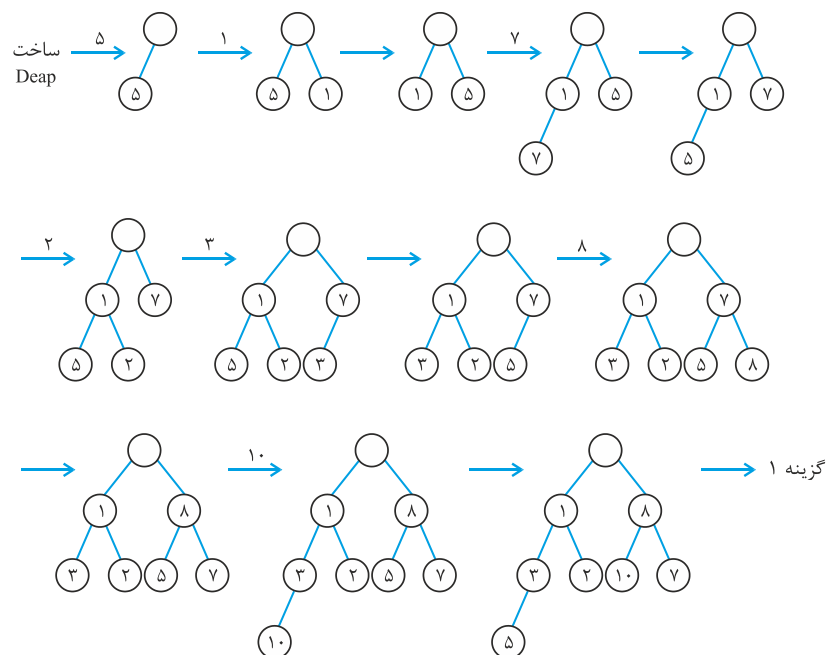
۵۸- گزینه (۴). فاصله گره‌های  $u$  و  $v$  تا ریشه حداکثر  $\lg n$  است و کافی است این فاصله طی شود.

۵۹- گزینه (۲). تابع را با ریشه درخت فراخوانی می‌کنیم،  $IPP(A)$ ، باعث چاپ  $A$  می‌شود. سپس  $IPP(B)$  فراخوانی می‌شود که  $B$  چاپ می‌شود سپس  $IPP(D)$ ، سه بار  $D$  چاپ می‌کند و به همین ترتیب، پس گزینه ۲ صحیح است.

۶۰- گزینه (۱).  $A[1]$  در عمق  $\lfloor \lg n \rfloor$  است (عمق یک نود در این تست برابر حداکثر فاصله آن نود تا برگ است که در واقع ارتفاع نود است).  $A[2]$  و  $A[3]$  در عمق  $\lfloor \lg n \rfloor - 1$  هستند و ... تعداد مقایسه‌ها در بدترین حالت ۲ برابر  $\lfloor \lg \frac{n}{1} \rfloor$  است. چون هر نود با هر دو فرزندش مقایسه می‌شود.

۶۱- گزینه (۲). در preorder ابتدا A و همه نودهای زیر درخت چپ A که تعدادشان ۶ نو است ملاقات می‌شوند سپس C و همه چپ آن که تعدادشان ۹ تاست و سپس F ملاقات می‌شود پس F نود شماره  $6 + 9 + 1 = 16$  است. در postorder ابتدا چپ A که ۵ نود است پس چپ C که ۸ نود است و سپس زیر درختان F که ۴ نود است و در نهایت F، پس F نود شماره  $5 + 8 + 5 = 18$  است.

۶۲- گزینه (۱). Deap یک درخت دودویی کامل است که ریشه آن خالی است، زیر درخت چپ ریشه Minheap و زیردرخت راست ریشه Maxheap است و هر نود در زیر درخت چپ ریشه از نود نظیرش در زیردرخت راست ریشه کوچکتر یا مساوی است (اگر نظیر نود وجود نداشت، باید نود با نظیر پدرش مقایسه شود)



۶۳- گزینه (۳). ساخت هیپ از مرتبه  $O(n)$  است ولی بعد از آن باید عملیات حذف ریشه انجام شود که از مرتبه  $O(n \lg n)$  است. (heapsort)

۶۴- گزینه (۲). فرض کنید 

|    |    |   |    |
|----|----|---|----|
| ۱  | ۲  | ۳ | ۴  |
| ۱۰ | ۴۰ | ۵ | ۲۰ |

 یعنی  $n = 4$ . در حلقه for ابتدا  $\text{aproc}(A, 1)$  فراخوانی می‌شود که عملی انجام نمی‌شود. بار بعد  $\text{aproc}(A, 2)$  فراخوانی می‌شود که چون  $A[n] < A\left[\frac{n}{2}\right]$  درست نیست، عملی انجام نمی‌شود. بار بعد  $\text{aproc}(A, 3)$  فراخوانی می‌شود و چون  $A[3] < A[1]$ ، آرایه به شکل 

|   |    |    |    |
|---|----|----|----|
| ۱ | ۲  | ۳  | ۴  |
| ۵ | ۴۰ | ۱۰ | ۲۰ |

 تبدیل می‌شود و  $\text{aproc}(A, 1)$  فراخوانی می‌شود که عملی انجام نمی‌شود. آخرین فراخوانی در حلقه for به صورت  $\text{aproc}(A, 4)$  است و چون  $A[4] < A[2]$  آرایه به صورت 

|   |    |    |    |
|---|----|----|----|
| ۱ | ۲  | ۳  | ۴  |
| ۵ | ۲۰ | ۱۰ | ۴۰ |

 تبدیل می‌شود و  $\text{aproc}(A, 2)$  عملی انجام نمی‌دهد. در نهایت در  $A[1]$  عنصر min قرار دارد. رویه  $\text{aproc}$  درواقع با آرایه  $A$ ، min heap می‌سازد.

۶۵- گزینه (۲). در بدترین حالت  $\theta(n)$  عنصر باید بررسی شوند. یک روش این است که درخت را به صورت میان ترتیب پیمایش کنیم، اگر حاصل به صورت صعودی مرتب بود یعنی درخت دودویی جستجو معتبر است. در غیر این صورت معتبر نیست.

۶۶- گزینه (۱). ادغام دو هیپ دودویی از هر نوع از مرتبه خطی است. درواقع باید با  $n + m$  عنصر یک هیپ ساخته شود که مرتبه آن  $O(n + m)$  است.

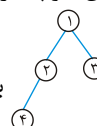
۶۷- گزینه (۴). هر ۳ عبارت جملات صحیحی هستند.

۶۸- گزینه (۴). پنجمین کوچکترین می‌تواند در هر یک از درایه‌ها ۲ تا ۳۱ باشد پس نمی‌تواند در درایه ۱ یا ۳۲ تا ۶۳ باشد. گزینه ۱ نیز گزینه مناسبی نیست.

۶۹- گزینه (۱). ریشه باید min باشد، سطح دوم max باشند، سطح سوم min باشد و به همین ترتیب هر نودی که در سطح  $\min(\max)$  است باید از همه نواده‌هایش کوچکتر (بزرگتر) باشد. که فقط گزینه ۱ چنین است. به کتاب آبی ساختمان داده پوران پژوهش رجوع کنید.

۷۰- گزینه (۱). باید به ازای هر نود بررسی شود که اختلاف تعداد سطح زیر درخت چپ و راست آن نود چند است. با مرتبه  $\theta(n)$  این عمل امکان‌پذیر است.

۷۱- گزینه (۲). سومین کوچکترین عنصر می‌تواند فرزند ریشه باشد مثلاً با اعداد ۱ تا ۴، هرم



کمینه می‌تواند باشد که عدد ۳ سومین کوچکترین عنصر است. بزرگترین عنصر

حتماً در یک برگ است. دومین بزرگترین می‌تواند پدر اولین بزرگترین باشد پس می‌تواند برگ نباشد.

- ۷۲- گزینه (۴). یک درخت مورب با  $n \geq 6$  گره که همه برچسب‌های آن متفاوت است، در نظر بگیرید. پیمایش پیش ترتیب این درخت، قرینه پیمایش پس ترتیب آن است و چنین درختی نقض گزینه‌های ۱ تا ۳ است.
- ۷۳- گزینه (۱). چنین درختی می‌تواند درخت پر باشد و اصلاً گره قرمز نداشته باشد. به کتاب آبی ساختمان داده پوران پژوهش رجوع کنید.
- ۷۴- گزینه (۴). دو اشاره‌گر باید عوض شوند. ابتدا باید لینک گره  $x$  ( $next[x]$ ) به نود بعد از  $n$  اشاره کند:  $next[x] = next[n]$ . سپس لینک نود  $n$  به نود  $x$  اشاره کند:  $next[n] = x$ .
- ۷۵- گزینه (۴). اگر  $f < r$ ، تعداد عناصر داخل برابر  $r - f$  است و اگر  $f > r$ ، تعداد عناصر داخل صف  $m - (f - r)$  است، که می‌توان در هر حالت، تعداد عناصر صف را برابر  $(r - f + m) \bmod m$  در نظر گرفت که گزینه‌ها همگی نادرست هستند. صف حلقوی وقتی پر است که فقط یک خانه خالی باشد که می‌توان شرط پر بودن را  $f = (r + 1) \bmod m$  نوشت. کلید اولیه گزینه ۴ می‌باشد.
- ۷۶- گزینه (۳). در درخت دودویی همواره  $n_0 = n_1 + 1$  یعنی تعداد برگ از تعداد گره دو فرزند، یک واحد بیشتر است. بین تعداد گره تک فرزندی و سایر گره‌ها هیچ رابطه‌ای وجود ندارد.
- ۷۷- گزینه (۲). اگر  $f$  انتهای آرایه باشد باید به ابتدای آرایه مقدار دهی شود، در غیر این صورت باید یک واحد به جلو برود پس  $f = (f + 1) \bmod n$  صحیح است که همان  $\% \bmod$  است.
- ۷۸- گزینه (۲). سه آرایه را با مرتبه  $O(n)$  ادغام می‌کنیم و سپس عنصر وسط لیست مرتب حاصل را ریشه قرار می‌دهیم و به طور بازگشتی به زیر درخت چپ و راست ریشه، همین عمل را انجام می‌دهیم پس  $T(n) = 2T\left(\frac{n}{2}\right) + \theta(1) = \theta(n)$ .
- ۷۹- گزینه (۴). ارتفاع درخت را  $h(n)$  می‌نامیم. فرض کنید عدد  $i$  ریشه است. در زیر درخت چپ اعداد ۱ تا  $i - 1$  قرار می‌گیرند که ارتفاع زیر درخت چپ برابر  $h(i - 1)$  و تعداد  $n - i$  عدد در زیر درخت راست قرار می‌گیرند که ارتفاع زیر درخت راست برابر  $h(n - i)$  می‌باشد و ماکزیمم بین این دو مقدار، تعیین کننده ارتفاع است. حال  $i$  می‌تواند از ۱ تا  $n$  تغییر کند و میانگین گرفته شود. در این سوال، ارتفاع درخت یک نودی، یک در نظر گرفته شده است.
- ۸۰- گزینه (۳). اگر همه  $x_i$  ها متمایز باشند و  $y_i$  ها نیز متمایز باشند آنگاه درخت خوب (این درخت در واقع treap است) منحصر به فرد است ولی در غیر این صورت لزوماً چنین نیست (گزاره اول غلط).

گزاره دوم غلط است چون حداقل یک درخت خوب وجود دارد و روش ساخت این درخت به این صورت است که عنصر تازه وارد را همانند درج در  $T$  با توجه به  $x_i$  درج کنیم و پس از درج اگر مقدار  $y_i$  عنصر جدید از  $y_i$  پدرش کوچکتر باشد باید پدر عنصر جدید را دوران دهیم. ارتفاع درخت خوب می‌تواند از  $\lg n$  تا  $n$  تغییر کند پس گزاره سوم درست و گزاره چهارم غلط است.

۸۱- گزینه (۲). شرط آنکه بتوان گره‌ها را قرمز سیاه کرد آن است که تعداد سطوح زیر درخت چپ هر نود حداکثر ۲ برابر (حداقل نصف) تعداد سطوح زیر درخت راست آن نود باشد که فقط گزاره سوم صحیح است.

۸۲- گزینه (۱). چنین درختی وجود ندارد.

۸۳- گزینه (۳). این الگوریتم  $\text{succ}(x)$  را بر می‌گرداند یعنی اولین جد  $x$  که  $x$  در زیر درخت چپ آن واقع است. پس جواب  $n+1$  (عدد بعد از  $n$ ) می‌باشد.

۸۴- گزینه (۲). تعداد BST ها با اعداد ۱ تا  $n$  برابر عدد  $n$ ام کاتالان  $(C_n)$  است که می‌دانیم

$$C_{n-1} = \frac{1}{n} \binom{2n-2}{n-1} \quad C_n = \frac{1}{n+1} \binom{2n}{n} \quad \text{پس } C_0 = 1 \quad \text{و} \quad C_n = \sum_{k=1}^n C_{k-1} \cdot C_{n-k}$$

حال نسبت  $\frac{C_n}{C_{n-1}}$  را می‌یابیم:

$$\begin{aligned} \frac{C_n}{C_{n-1}} &= \frac{n}{n+1} \times \frac{\binom{2n}{n}}{\binom{2n-2}{n-1}} = \frac{n}{n+1} \times \frac{(2n)!(n-1)!(n-1)!}{(2n-2)!n!n!} \\ &= \frac{n}{n+1} \times \frac{2n(2n-1)}{n \times n} = \frac{2n-1}{n+1} \end{aligned}$$

۸۵- گزینه (۱). می‌دانیم  $G_h = G_{h-1} + G_{h-2} + 1$  پس اعداد دنباله‌های  $F$  و  $G$  عبارتند از:

$$F: F_0 = 0, F_1 = 1, F_2 = 1, F_3 = 2, F_4 = 3, F_5 = 5, F_6 = 8, F_7 = 13, \dots$$

$$G: G_0 = 1, G_1 = 2, G_2 = 4, G_3 = 7, G_4 = 12, \dots$$

با مقایسه این دو دنباله در می‌یابیم که

$$G_4 = F_7 - 1 \quad \text{مثلاً} \quad G_h = F_{h+3} - 1$$

۸۶- گزینه (۲). با داشتن هر پیمایش حاوی  $n$  عنصر به تعداد  $c_n = \frac{1}{n+1} \binom{2n}{n}$  درخت متفاوت

می‌توان کشید.

۸۷- گزینه (۲). دو لیست با هم یکی می‌شوند.

۸۸- گزینه (۴). درخت دودویی پر درختی است که تک فرزندی ندارد و همه برگ‌ها هم سطح هستند. درخت دودویی کامل درختی است که همه سطوح ماقبل آخر پر هستند و در سطح آخر برگ‌ها از چپ قرار می‌گیرند.

۸۹- گزینه (۴). هر سه گزاره صحیح هستند که از متن درس می‌دانیم.

۹۰- گزینه (۲). عنصر  $N$  حتماً از  $N$  کوچکتر یا مساوی و از  $H$  بزرگتر یا مساوی است.

## گراف فصل ۷

### گراف

در درس ساختمان گسسته با تعاریف گراف آشنا شده‌اید. گراف  $G$  از دو مجموعه  $V$  و  $E$  تشکیل شده است که  $V$  مجموعه رئوس و  $E$  مجموعه یالها یا لبه‌ها است.

$E$  از تعدادی زوج تشکیل شده است. اگر زوج‌ها زوج مرتب باشند معمولاً به صورت  $(a, b)$  نشان می‌دهیم و گراف جهت‌دار (digraph) می‌شود یعنی به شکل  $a \rightarrow b$  می‌کشیم. و اگر زوج‌ها نامرتب باشند معمولاً به صورت  $\{a, b\}$  یا  $ab$  یا در برخی منابع به صورت  $(a, b)$  نشان می‌دهیم و گراف غیر جهت‌دار می‌شود یعنی به شکل  $a \text{ --- } b$  می‌کشیم. مثلاً گراف  $G = (V, E)$  که  $V = \{a, b, c, d\}$  و  $E = \{ab, bc, cd, ad, bd\}$  غیرجهت‌دار است که ۴ رأس و ۵ یال دارد و

می‌توان به شکل  کشید. در این گراف  $\deg(b) = \deg(d) = 3$  و  $\deg(c) = \deg(a) = 2$  و چون ماکزیمم درجه ۳ است گویند گراف درجه ۳ است. گراف  $G = (V, E)$  که  $V = \{a, b, c, d\}$  و  $E = \{(a, b), (b, c), (c, b), (d, c), (a, d), (b, d)\}$  جهت‌دار است که ۴ رأس و ۶ یال دارد و

می‌توان به شکل  کشید. درجه ورودی  $a$  صفر است و درجه خروجی  $a$  برابر ۲ است.

**گراف ساده** به گرافی گویند که selfloop (طوقه) و multiedge (لبه چندگانه) ندارد. دو

گرافی که در بالا کشیدیم ساده هستند. ولی گراف  ساده نیست، یال  $\{b, b\}$  یک طوقه است و سه یالی که بین  $a$  و  $d$  هستند، یالهای چندگانه می‌باشند.



گرافی را **همبند** (متصل connected) گویند که بین هر دو رأس آن **حداقل یک مسیر** وجود داشته باشد. (البته اگر گراف جهت‌دار باشد، بدون در نظر گرفتن جهت بین هر دو رأس حداقل یک مسیر باشد). گراف‌های فوق همگی همبند هستند ولی گراف  $\triangle$ ،  $\uparrow$ ،  $\searrow$ ، ناهمبند و متشکل از ۳ مولفه است.

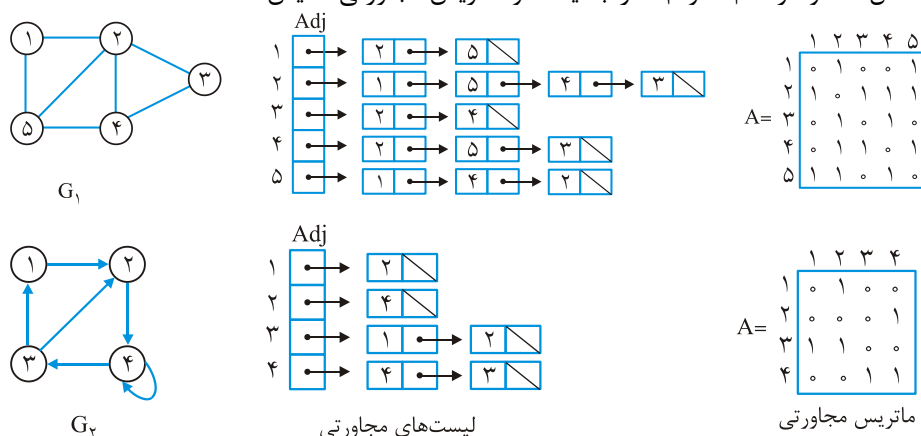
گرافی که سیکل ندارد را **جنگل** گویند و اگر همبند باشد، **درخت** گویند، منظور از سیکل (دور) این است که از یک رأس شروع کنید و به همان رأس باز گردید. مثلاً در گراف  $\begin{matrix} a & b \\ \swarrow & \searrow \\ d & c \end{matrix}$ ، دنباله  $a \rightarrow b \rightarrow d \rightarrow a$  یک سیکل به طول ۳ است. تعداد رئوس را با  $|V|$  یا  $n$  یا  $p$  نشان می‌دهند و به آن مرتبه گراف گویند. تعداد یالها را با  $|E|$  یا  $e$  یا  $q$  نشان می‌دهند و به آن اندازه گراف گویند. تعداد یالهای گراف ساده حداقل صفر و حداکثر برای گراف غیر جهت‌دار  $\binom{n}{2} = \frac{n(n-1)}{2}$  و برای جهت‌دار  $n(n-1)$  است. و اگر گراف همبند باشد، حداقل تعداد یال  $n-1$  است. گرافی را **متراکم** (شلوغ dense) گویند که تعداد یال آن از مرتبه  $\theta(n^2)$  باشد یعنی تعداد یال آن به تعداد یال گراف کامل نزدیک باشد. گرافی را **خلوت** (اسپارس) گویند که تعداد یال آن از مرتبه  $O(n)$  باشد.

برای نمایش گراف چندین روش مطرح است که ما دو روش متداول را بررسی می‌کنیم:

**ماتریس مجاورتی** (adjacency matrix) و **لیست‌های مجاورتی** (adjacency lists).

**ماتریس مجاورتی** یک ماتریس  $n \times n$  است که درایه‌های آن نشان می‌دهند چه رئوسی با هم مجاور هستند یعنی بین چه رئوسی یال هست. **لیست‌های مجاورتی** از  $n$  تا لیست پیوندی تشکیل شده است و همه رئوسی که با رأس  $i$  مجاور هستند، در لیست  $i$  قرار می‌گیرند.

شکل ۱، دو گراف  $G_1$  و  $G_2$  را با لیست و ماتریس مجاورتی نمایش داده است.



شکل ۱.

ترتیب قرار گرفتن رئوس در ماتریس و لیست‌های مجاورتی اهمیتی ندارد.

حافظه مصرفی ماتریس  $n^2$  و حافظه مصرفی لیست برای گراف غیر جهت‌دار  $n + 2e$  و برای گراف جهت‌دار  $n + e$  است. پس می‌توان گفت حافظه مصرفی ماتریس  $\theta(n^2)$  و حافظه مصرفی لیست  $\theta(n + e)$  است. اگر گراف خلوت باشد، حافظه مصرفی لیست  $\theta(n)$  می‌شود که از ماتریس بهتر است. اگر گراف متراکم باشد، حافظه مصرفی لیست  $\theta(n^2)$  می‌شود که با ماتریس هم مرتبه است ولی چون در لیست اشاره گرهای وجود دارد، ماتریس بهتر است. لازم به ذکر است اگر گراف همبند باشد آنگاه  $\theta(n) \leq e \leq \theta(n^2)$  و می‌توان  $\theta(n + e)$  را با  $\theta(e)$  نشان داد. اگر بخواهیم بررسی کنیم رأس  $i$  با  $j$  مجاور است یا خیر در ماتریس از مرتبه  $\theta(1)$  است چون فقط کافی است یک درایه از ماتریس بررسی شود. ولی در لیست از مرتبه  $O(\deg(i))$  است چون باید لیست  $i$  پیمایش شود و وجود نود  $j$  در آن بررسی شود. اگر بخواهیم همه رئوس مجاور با رأس  $i$  را چاپ کنیم در ماتریس از مرتبه  $\theta(n)$  و در لیست از مرتبه  $\theta(\deg(i))$  می‌باشد.

### پیمایش گراف

برای گراف دو نوع پیمایش مطرح است: عمقی (DFS: Depth First Search) و سطحی (BFS: Breadth First Search).

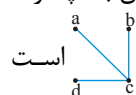
### پیمایش عمقی (DFS)

همانطور که از اسم این پیمایش پیداست، جستجو را هر چه عمیق‌تر باید انجام دهیم. از یک رأس شروع می‌کنیم و یکی از رئوس مجاورش را ملاقات می‌کنیم. حال همین عمل را با رأس جدید انجام می‌دهیم. اگر به رأسی رسیدیم که امکان ادامه وجود نداشت یعنی همه رئوس مجاورش قبلاً ملاقات شده بودند، یک مرحله به عقب برمی‌گردیم یعنی به پدر آن رأس برمی‌گردیم، یعنی به رأسی باز می‌گردیم که از طریق آن رأس جدید را ملاقات کرده‌ایم. اگر باز هم امکان ادامه وجود نداشت یک مرحله به عقب باز می‌گردیم، به همین ترتیب آنقدر بازگشت می‌کنیم تا به یکی از اجداد رأس جدید برسیم که بتوان از آن رأس، رأس جدیدی را ملاقات کرد. الگوریتم این روش بازگشتی است و از یک پشته استفاده می‌کند. پیمایش DFS برای گراف منحصر به فرد نیست.

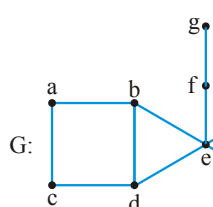
مثلاً در گراف  می‌توان از  $a$  شروع کرد و بعد  $b$  را ملاقات کرد و سپس با کمک  $b$  رأس  $c$  را

ملاقات کرد و سپس با کمک  $c$  رأس  $d$  را ملاقات کرد و نتیجه  است که یک درخت است و چون شامل همه رئوس است به آن درخت پوشا یا فراگیر (spanning tree) گویند.

یا می‌توان از  $a$  شروع کرد و سپس  $c$  را ملاقات کرد و با کمک  $c$  رأس  $b$  را ملاقات کرد از رأس  $b$  امکان ادامه وجود ندارد چون همه رئوس مجاورش ( $a$  و  $c$ ) قبلاً ملاقات شده‌اند، پس به پدر  $b$  یعنی به رأس  $c$  بازگشت می‌کنیم و با کمک  $c$ ، رأس  $d$  را ملاقات می‌کنیم و نتیجه است که درخت پوشاست.

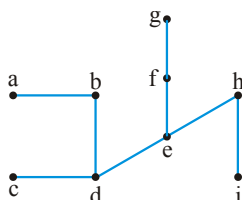


**مثال ۱:** با در نظر گرفتن اولویت حروف الفبا، پیمایش DFS گراف  $G$  شکل ۲ را بیابید.



شکل ۲.

**پاسخ:** با توجه به صورت سوال از رأس  $a$  شروع می‌کنیم. با کمک رأس  $a$  می‌توان  $b$  یا  $c$  را ملاقات کرد که  $b$  را ملاقات می‌کنیم با کمک  $b$  می‌توان  $d$  یا  $e$  را ملاقات کرد که  $d$  ملاقات می‌شود. رأس  $d$  می‌تواند  $c$  یا  $e$  را ملاقات کند که  $c$  ملاقات می‌شود. رئوس مجاور رأس  $c$  یعنی  $a$  و  $d$  هر دو قبلاً ملاقات شده‌اند پس باید به پدر  $c$ ، یعنی به  $d$  بازگشت کنیم و با کمک  $d$  رأس  $e$  سپس با کمک  $e$  رأس  $f$  و با کمک رأس  $f$  رأس  $g$  ملاقات می‌شوند. از  $g$  امکان ادامه نیست، دو بار بازگشت می‌کنیم و با کمک  $e$  رأس  $h$  و در نهایت با کمک  $h$  رأس  $i$  ملاقات می‌شود. شکل درخت پوشای حاصل در شکل ۳ آمده است:



شکل ۳: درخت پوشای حاصل از پیمایش DFS گراف شکل ۲ با در نظر گرفتن

اولویت حروف الفبا  $a b d c e f g h i$

پیمایش DFS لزوماً همه رئوس را ملاقات نمی‌کند مثلاً برای گراف  $a \rightarrow b \leftarrow c$  اگر از رأس  $a$  شروع کنیم، فقط  $a$  و  $b$  قابل ملاقات هستند و  $c$  ملاقات نمی‌شود. ولی به خاطر کاربردهایی که برای DFS مطرح می‌شود، ما می‌خواهیم همه رئوس را ملاقات کنیم، برای این منظور تا زمانی که همه رئوس ملاقات نشده‌اند، پیمایش را برای رأسی که ملاقات نشده، فراخوانی می‌کنیم. بنابراین ابتدا از  $a$  شروع می‌کنیم و  $b$  ملاقات می‌شود و سپس از  $c$  شروع می‌کنیم که فقط  $c$  ملاقات می‌شود و شکل حاصل از پیمایش  $a \rightarrow b$  است که درخت نیست ولی پوشاست که به آن جنگل پوشا گویند.

الگوریتم پیمایش عمقی در ادامه آمده است. این الگوریتم از دو تابع DFS و DFS\_VISIT تشکیل شده است. تابع DFS به همه رئوس رنگ سفید (WHITE) نسبت می‌دهد که یعنی رأس هنوز ملاقات نشده است، سپس یک رأس سفید را به تابع DFS\_VISIT ارسال می‌کند و این تابع آن رأس را خاکستری (GRAY) می‌کند به این معنی که رأس ملاقات شد و این تابع از رأس دریافتی شروع می‌کند و تا جایی که بتواند رئوس را ملاقات و خاکستری می‌کند. وقتی یک رأس تمام شد یعنی همه رئوس مجاورش ملاقات شدند، آن رأس سیاه (BLACK) می‌شود. اگر تابع DFS\_VISIT نتواند رأس جدیدی ملاقات کند، و رأسی باشد که هنوز ملاقات نشده و سفید است، تابع DFS آن رأس را به DFS\_VISIT ارسال می‌کند. یعنی همانطور که گفته شد، این دو تابع تمام رئوس را ملاقات می‌کنند.

**DFS(G)**

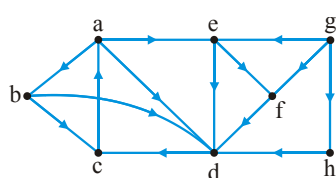
```
{for (each  $u \in V$ ) {color[u] = WHITE ;  $\pi[u]$  = NIL}
time = ۰ ;
for (each  $u \in V$ ) if (color [u] = WHITE ) DFS_VISIT(u) ;}
```

**DFS\_VISIT(u)**

```
{color[u] = GRAY ; time = time + ۱ ; d[u] = time ;
for (each  $v \in \text{Adj}[u]$ ) if (color[v] == WHITE) { $\pi[v]$  = u ; DFS_VISIT(v) ;}
color[u] = BLACK ; time = time + ۱ ; f[u] = time ;}
```

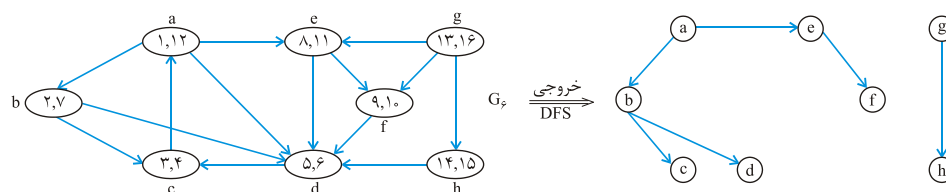
متغیر time یک متغیر سراسری است. وقتی رأس u ملاقات می‌شود (خاکستری می‌شود) به u یک عدد به نام زمان ملاقات (discovery time) نسبت داده می‌شود (d[u]) و وقتی رأس u تمام می‌شود یعنی همه رئوس مجاورش ملاقات شده‌اند و رنگ رأس u سیاه می‌شود، به رأس u عددی به نام زمان پایان (finish time) نسبت داده می‌شود (f[u]) زمان ملاقات و زمان پایان برای هر رأس دو عدد متمایز هستند و از ۱ شروع می‌شوند و برای هر رأس u رابطه  $۱ \leq d[u] < f[u] \leq 2n$  برقرار است. مثلاً در گراف شکل ۲ مثال ۱، ابتدا a ملاقات می‌شود پس  $d[a] = ۱$  سپس b و بعد d و بعد c ملاقات می‌شوند پس  $d[b] = ۲$  و  $d[d] = ۳$  و  $d[c] = ۴$ . از رأس c امکان ادامه نیست پس  $f[c] = ۵$ . حال رأس e ملاقات می‌شود سپس f و بعد g، بنابراین  $d[e] = ۶$  و  $d[f] = ۷$  و  $d[g] = ۸$ ، رأس g پایان یافته است (سیاه شده است). پس  $f[g] = ۹$  و به همین ترتیب  $f[f] = ۱۰$ ، سپس h و i ملاقات می‌شوند. پس  $d[h] = ۱۱$  و  $d[i] = ۱۲$ . رأس i پایان یافته پس  $f[i] = ۱۳$  و سپس  $f[h] = ۱۴$  و بعد  $f[e] = ۱۵$  و بعد  $d[b] = ۱۶$  و  $f[b] = ۱۷$  و  $f[a] = ۱۸$ .

**مثال ۲:** با در نظر گرفتن اولویت حروف الفبا، پیمایش عمقی را روی گراف شکل ۴ اجرا کنید برای هر رأس مقادیر زمان ملاقات و زمان پایان را مشخص کنید.



شکل ۴.

**پاسخ:** از رأس a شروع می‌کنیم سپس b و بعد c ملاقات می‌شود پس  $d[a]=1$  و  $d[b]=2$  و  $d[c]=3$ . از c امکان ادامه نیست پس تمام می‌شود و  $f[c]=4$  به پدر c یعنی b بازگشت می‌کنیم و رأس d را ملاقات می‌کنیم، پس  $d[d]=5$ ، از d امکان ادامه نیست پس  $f[d]=6$  به b بازگشت می‌کنیم که از b امکان ادامه نیست پس  $f[b]=7$  به a بازگشت می‌کنیم و رأس e را ملاقات می‌کنیم پس  $d[e]=8$  از طریق e رأس f ملاقات می‌شود پس  $d[f]=9$ . از f امکان ادامه نیست پس  $f[f]=10$  به e بازگشت می‌کنیم که امکان ادامه نیست پس  $f[e]=11$  به a بازگشت می‌کنیم که از a امکان ادامه نیست. چون همه رئوس مجاور a ملاقات شده‌اند پس  $f[a]=12$ . حال تابع DFS\_VISIT را با رأس g فراخوانی می‌کنیم و g ملاقات می‌شود پس  $d[g]=13$  و سپس h ملاقات می‌شود که  $d[h]=14$ . از h امکان ادامه نیست پس  $f[h]=15$  به g بازگشت می‌کنیم که از g امکان ادامه نیست پس  $f[g]=16$ . در شکل ۵ داخل هر نود (d, f) نوشته شده است که d زمان ملاقات و f زمان پایان است. همچنین جنگل پوشای حاصل از پیمایش نیز نشان داده شده است.



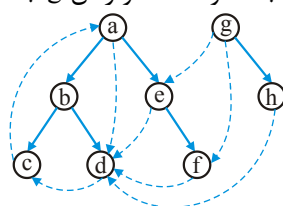
شکل ۵.

در الگوریتم پیمایش عمقی آرایه‌ای به اسم  $\pi$  تعریف شده است که  $\pi[u]$  یعنی پدر u، یعنی رأسی که از طریق آن رأس u ملاقات می‌شود. با داشتن این آرایه می‌توان جنگل پوشای حاصل از پیمایش را نمایش داد.

با توجه به پیمایش عمقی یالهای گراف را می‌توان ۴ نوع در نظر گرفت: یال درختی (tree edge)، یال پشتی (back edge)، یال پیش‌رو (forward edge) و یال عبوری یا ضربدری (cross edge) یا صلیبی.

**یال درختی** یالی است که الگوریتم، آن را طی می‌کند یعنی ملاقات می‌کند و سه نوع یال دیگر یالهایی هستند که در گراف می‌باشند ولی در جنگل پوشای حاصل نیستند. با توجه به شکل ۵، در

جنگل پوشای حاصل یالهای  $(a,b)$  و  $(a,e)$  و  $(e,f)$  و  $(b,d)$  و  $(b,c)$  و  $(g,h)$  درختی هستند برای تشخیص نوع سایر یالها بهتر است جنگل پوشای حاصل را به شکل درختهای ریشه‌دار بکشیم که در شکل ۶، جنگل شکل ۵ را مجدداً کشیده‌ایم و یالهایی که پیمایش نشده‌اند را به صورت یالهای خط چین نشان داده‌ایم در این جنگل، رأس  $a$  جد رؤس  $f,d,c,e,b,a$  است. رأس  $b$  جد  $d,c,b$  است و رأس  $e$  جد  $e$  و رأس  $f$  است و رأس  $g$  جد  $g$  و  $h$  است.

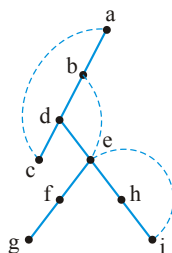


شکل ۶. جنگل حاصل از پیمایش عمقی گراف شکل ۴ با توجه به اولویت حروف الفبا.

**یال پستی** یالی است از نواده به جد، پس یال  $(c,a)$  پستی است. **یال پیش‌رو**، یالی است از جد به نواده پس یال  $(a,d)$  پیش‌رو است. سایر یالهایی که ملاقات نشده‌اند، **عبوری** هستند. پس یالهای  $(g,e)$  و  $(g,f)$  و  $(h,d)$  و  $(e,d)$  و  $(f,d)$  و  $(d,c)$  عبوری هستند.

**مثال ۳:** انواع یالها را در گراف غیر جهت‌دار شکل ۲ با توجه به پیمایش عمقی و با در نظر گرفتن اولویت حروف الفبا مشخص کنید.

**پاسخ:** پیمایش عمقی گراف شکل ۲، در شکل ۳ کشیده شده است که ما آن را به صورت یک درخت ریشه‌دار در شکل ۷ نمایش داده‌ایم و یالهایی که ملاقات نشده‌اند را با خط چین نشان داده‌ایم.



شکل ۷. درخت پوشای حاصل از پیمایش عمقی گراف شکل ۲.

یالهای  $ab$  و  $bd$  و  $dc$  و  $de$  و  $ef$  و  $fg$  و  $eh$  و  $hi$  درختی هستند. یالهای  $ac$  و  $be$  و  $ei$  یالهایی هستند که پیمایش نشده‌اند و دو سر هر کدام از این یالها جد و نواده هستند. مثلاً  $a$  جد  $c$  است ( $c$  نواده  $a$  است) این یالها یا پستی هستند یا پیش‌رو. ولی قرار داد شده است که به این یالها، یال پستی گفته شود. بنابراین گراف غیر جهت‌دار، با توجه به پیمایش عمقی، یال پیش‌رو ندارد و همچنین یال عبوری نیز نمی‌تواند داشته باشد (چرا؟).

**نتیجه:** با توجه به پیمایش عمقی، در گراف غیر جهت‌دار یال پیش‌رو و عبوری وجود ندارد.

#### نکته

زمان‌های ملاقات  $d$  (discovery time) و پایان  $f$  (finish time) دارای خاصیت پرانتزی هستند. زمان ملاقات یعنی پرانتز باز و زمان پایان یعنی پرانتز بسته. مثلاً اگر  $d[u]$  را به شکل " $u$ "  $f[u]$  را به شکل " $u$ " نشان دهیم آنگاه فرم پرانتزی کامل شکل ۵ به صورت  $(a(b(cc)(dd)b)(e(ff)e)a)(g(hh)g)$  است.

**قضیه:** برای هر دو رأس  $u$  و  $v$  با توجه به پیمایش عمقی، فقط یکی از حالات زیر امکان دارد:

(۱)  $d[v] < d[u] < f[u] < f[v]$  یعنی  $v$  ابتدا ملاقات شده سپس  $u$  ملاقات شده و تمام شده و سپس  $v$  تمام شده است. این نشان می‌دهد که در جنگل پوشای حاصل از پیمایش،  $v$  جد  $u$  است (یا  $u$  نواده  $v$  است) اگر برای  $v$  از ( ) و برای  $u$  از [ ] استفاده کنیم، این حالت معادل فرم پرانتزی ( [ ] ) است.

(۲)  $d[u] < d[v] < f[v] < f[u]$  یعنی در جنگل پوشای حاصل از پیمایش،  $u$  جد  $v$  است (یا  $v$  نواده  $u$  است) که این حالت معادل فرم پرانتزی ( [ ] ) است.

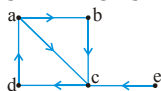
(۳)  $d[v] < f[v] < d[u] < f[u]$  یعنی در جنگل پوشای حاصل از پیمایش،  $u$  و  $v$  هیچ یک جد یا نواده یکدیگر نیستند و اگر در گراف، احتمالاً بین آنها یال باشد حتماً یال  $(u, v)$  است و نه  $(v, u)$ ، که در این صورت یال  $(u, v)$  یال عبوری است. این حالت معادل فرم پرانتزی ( [ ] ) است.

(۴)  $d[u] < f[u] < d[v] < f[v]$  یعنی در جنگل پوشای حاصل از پیمایش،  $u$  و  $v$  هیچ یک جد و نواده یکدیگر نیستند و اگر احتمالاً در گراف اصلی، بین  $u$  و  $v$  یالی باشد آن یال حتماً  $(v, u)$  است (و نه  $(u, v)$ ) که یال عبوری می‌شود. این حالت معادل فرم پرانتزی ( [ ] ) است.

**نتیجه:** حالت  $d[u] < d[v] < f[u] < f[v]$  هیچگاه رخ نمی‌دهد زیرا فرم پرانتزی ( [ ] ) خطاست.

#### نکته

در پیمایش عمقی، هر یال حداقل یک بار بررسی می‌شود و تصمیم گرفته می‌شود که یال پیمایش شود یا خیر. می‌توان نوع یال  $(u, v)$  را وقتی اولین بار این یال بررسی می‌شود، با توجه به رنگ  $v$ ، تشخیص داد. اگر رنگ  $v$  سفید باشد به این معنی است که  $v$  هنوز ملاقات نشده است پس باید ملاقات شود بنابراین یال  $(u, v)$  درختی می‌شود. اگر رنگ  $v$  خاکستری باشد یعنی  $v$  قبلاً ملاقات شده ولی تمام نشده است پس یال  $(u, v)$  پشتی است. اگر رنگ  $v$  سیاه باشد، یال  $(u, v)$  یا پیش‌رو یا عبوری است که در این حالت اگر



$d[u] < d[v]$  یال پیش‌رو است و اگر  $d[u] > d[v]$  یال عبوری است. مثلاً در گراف

ابتدا همه رئوس سفید هستند. می‌خواهیم با توجه به اولویت حروف الفبا پیمایش عمقی انجام دهیم. از  $a$  شروع می‌کنیم و  $a$  خاکستری می‌شود، یال  $(a, b)$  را بررسی می‌کنیم و چون  $b$  سفید است پس این یال

پیمایش می‌شود و  $b$  خاکستری می‌شود، پس یال  $(a, b)$  درختی است. به همین ترتیب  $(b, c)$  و  $(c, d)$  نیز درختی هستند و  $b$  و  $c$  و  $d$  خاکستری می‌شوند. حال یال  $(d, a)$  بررسی می‌شود و چون  $a$  خاکستری است پس این یال پیمایش نمی‌شود پس یال  $(d, a)$  پشتی است. رئوس  $d$  و  $c$  و  $b$  سیاه می‌شوند. یال  $(a, c)$  را بررسی می‌کنیم چون رنگ  $c$  سیاه است پس این یال پیمایش نمی‌شود و چون  $d[a] < d[c]$  پس یال  $(a, c)$  پیش‌رو است. حال رأس  $e$  را خاکستری می‌کنیم و یال  $(e, c)$  را بررسی می‌کنیم و چون رنگ  $c$  سیاه است و  $d[e] > d[c]$  پس یال  $(e, c)$  عبوری است.

**قضیه:** شرط لازم و کافی برای آنکه گراف سیکل داشته باشد آن است که پیمایش عمقی یال پشتی تولید کند. البته تعداد یالهای پشتی لزوماً با تعداد سیکل‌ها مساوی نیست. ولی اگر پیمایش عمقی یال پشتی تولید نکند. حتماً گراف بدون سیکل (acyclic) است و برعکس.

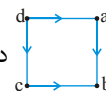
### برخی کاربردهای پیمایش عمقی عبارتند از:

#### ۱- تشخیص اینکه گراف همبند است یا خیر و اگر ناهمبند است، چند مولفه دارد.

برای این منظور کافی است از یک رأس پیمایش عمقی را اجرا کنید، اگر همه رئوس ملاقات شدند یعنی گراف همبند است. در غیر این صورت ناهمبند است. برای شمارش تعداد مولفه‌ها می‌توان تعداد باری که تابع DFS-VISIT را صدا می‌زنیم، بشماریم. البته برای گراف جهت‌دار، ابتدا جهت‌ها را نادیده بگیرید.

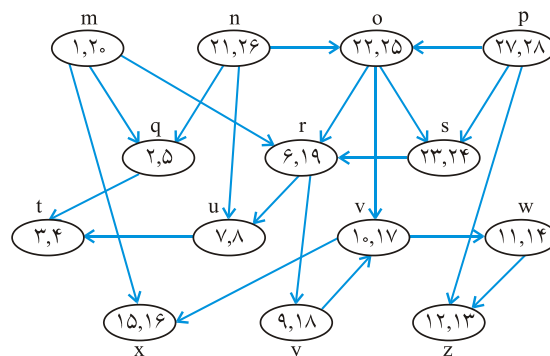
#### ۲- یافتن ترتیب توپولوژیکی

در گراف بدون سیکل جهت‌دار (directed acyclic graph: dag) مرتب سازی توپولوژیکی یعنی رئوس گراف را به ترتیبی پشت سر هم بچینیم که رعایت پیش‌نیازی شود یعنی هر رأسی که می‌نویسیم یا درجه ورودی‌اش صفر باشد و یا اگر صفر نیست رئوس وارد شونده به آن راس، قبلاً نوشته شده باشند. فقط گراف بدون سیکل جهت‌دار (dag) دارای ترتیب توپولوژیکی است. مثلاً



گراف دارای ترتیب‌های توپولوژیکی «dacb» و «dcab» (از چپ به راست) است. برای یافتن ترتیب توپولوژیکی با کمک پیمایش عمقی، روی گراف، پیمایش عمقی را به هر ترتیبی اجرا کنید و سپس رئوس را به ترتیب نزولی  $f$ ها (finish time) چاپ کنید یعنی ابتدا رأسی را چاپ کنید که  $f$  آن ماکزیمم است. مثلاً شکل ۸ یک dag را نشان می‌دهد که روی آن به ترتیب حروف الفبا پیمایش عمقی انجام شده است و داخل هر نود  $(d, f)$  نوشته است و با توجه به اینکه رأس  $p$  بزرگترین زمان پایان را دارد، ترتیب توپولوژیکی این گراف از چپ به راست pnosmryvxwzqt است. البته این گراف ترتیب‌های توپولوژیکی دیگری نیز دارد که می‌توان پیمایش‌های عمقی مختلفی انجام داد و سایر ترتیب توپولوژیکی‌ها را یافت.

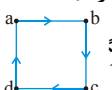
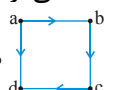




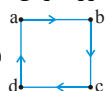
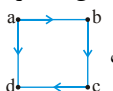
شکل ۸.

### ۳- یافتن مولفه‌های متصل قوی گراف جهت‌دار:

یک گراف جهت‌دار را متصل قوی (همبند قوی strongly connected) گویند اگر به ازای هر دو رأس  $x$  و  $y$  هم از  $x$  به  $y$  به  $y$  از  $x$  مسیری وجود داشته باشد. یعنی از هر رأسی بتوان به

همه رئوس دسترسی داشت. مثلاً  متصل قوی است ولی  متصل قوی نیست.

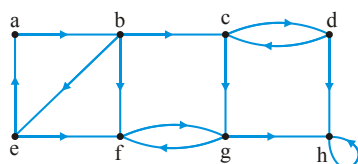
رئوس یک گراف جهت‌دار را می‌توان به مولفه‌های متصل قوی افراز کرد. هر مولفه متصل قوی بزرگترین مجموعه از رئوس گراف است که این رئوس از یکدیگر قابل دسترسی باشند. مثلاً گراف

 فقط یک مولفه متصل قوی  $\{a, b, c, d\}$  دارد و گراف  دارای ۴ مولفه متصل قوی

قوی  $\{a\}$  و  $\{b\}$  و  $\{c\}$  و  $\{d\}$  است. دقت کنید باید همه رئوس را به مجموعه‌هایی افراز کنید که عناصر داخل هر مجموعه با هم متصل قوی باشند پس اگر رأسی با هیچ رأس دیگری مسیر دوطرفه نداشت آن رأس خود یک مولفه متصل قوی است. برای یافتن مولفه‌های متصل قوی با کمک پیمایش عمقی، الگوریتم به این صورت است که روی گراف پیمایش عمقی را به هر ترتیبی که مایل هستید اجرا کنید و زمان‌های  $d$  و  $f$  برای هر رأس را ثبت کنید و سپس گراف را ترانهاده کنید یعنی جهت یالها را عوض کنید، حال روی گراف ترانهاده، پیمایش عمقی را مجدداً اجرا کنید ولی این بار به ترتیب نزولی  $f$ ها، تابع DFS-VISIT را صدا کنید. هر فراخوانی به DFS-VISIT یک مولفه متصل قوی را می‌یابد.

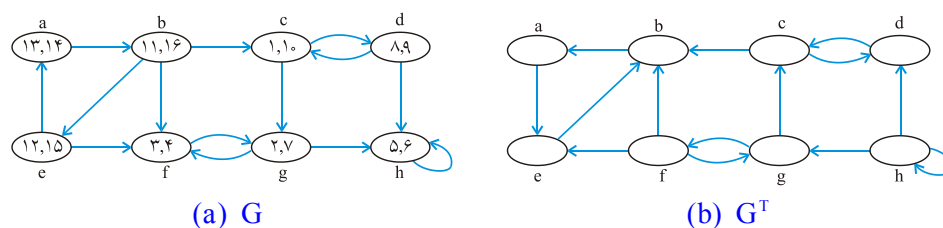
**مثال ۴:** مولفه‌های متصل قوی گراف شکل ۹ را بیابید.

**پاسخ:** با کمی دقت واضح است که مولفه‌های متصل قوی گراف  $G$   $\{a, b, e\}$  و  $\{c, d\}$  و  $\{f, g\}$  و  $\{h\}$  هستند. ولی



شکل ۹. گراف G

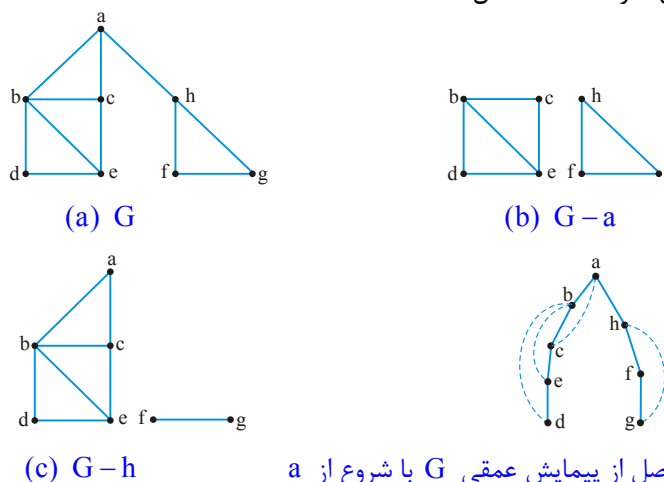
می‌خواهیم با الگوریتم گفته شده، مولفه‌ها را بیابیم. ابتدا روی گراف  $G$  پیمایش عمقی انجام می‌دهیم که ما به ترتیب حروف الفبا انجام می‌دهیم (به هر ترتیب دلخواهی می‌توان انجام داد) در شکل ۱۰ قسمت (a) پیمایش انجام شده و داخل هر نود  $(d, f)$  نوشته شده است. حال گراف را ترانواده می‌کنیم که در شکل ۱۰ قسمت (b)  $G^T$  کشیده شده است. حال روی  $G^T$  و به ترتیب نزولی  $f, h$ ، تابع DFS-VISIT را صدا می‌زنیم یعنی ابتدا DFS-VISIT(b) که  $\{e, a, b\}$  ملاقات می‌شوند که یک مولفه است. سپس DFS-VISIT(c) که  $\{d, c\}$  ملاقات می‌شوند سپس تابع را با  $g$  صدا می‌زنیم که مولفه  $\{f, g\}$  حاصل می‌شود و در نهایت تابع را با  $h$  صدا می‌زنیم و مولفه  $\{h\}$  حاصل می‌شود. لازم به ذکر است اگر رأس  $h$  طوقه نداشت نیز مؤلفه‌ها تغییر نمی‌کردند.



شکل ۱۰.

#### ۴- یافتن نقاط انفصال (articulation points) یا رأس برشی در گراف غیر جهت‌دار

نقطه انفصال رأسی است که اگر از گراف حذف شود (یا لایه‌ای متصل به آن نیز حذف می‌شوند)، گراف ناهمبند شود و یا اگر گراف ناهمبند است، تعداد مولفه‌های آن بیشتر شود. مثلاً در گراف  $G$  شکل ۱۱، رؤس  $a$  و  $h$  نقاط انفصال هستند.

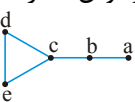
(d) درخت حاصل از پیمایش عمقی  $G$  با شروع از  $a$ 

شکل ۱۱.

اگر روی  $G$  پیمایش عمقی با شروع از رأس  $x$  انجام دهید آنگاه درخت ریشه‌داری با ریشه  $x$  حاصل می‌شود. اگر رأس  $x$  در درخت حاصل بیش از یک فرزند داشته باشد آنگاه  $x$  نقطه انفصال است و در غیر این صورت نقطه انفصال نیست. حال فرض کنید  $y$  رأسی غیر ریشه در درخت حاصل از پیمایش باشد، شرط لازم و کافی برای آنکه  $y$  نقطه انفصال باشد آن است که  $y$  فرزندی مثل  $z$  داشته باشد طوری که از  $z$  یا نواده‌های  $z$  هیچ یال پشتی به هیچ از اجداد محض  $y$  وجود نداشته باشد. مثلاً درخت حاصل از پیمایش عمقی گراف  $G$  شکل ۱۱-a در شکل ۱۱-d آمده است.

رأس  $a$  برشی است چون بیش از یک فرزند دارد. رأس  $h$  برشی است چون از  $f$  و از  $g$  یال پشتی به اجداد محض  $h$  وجود ندارد.

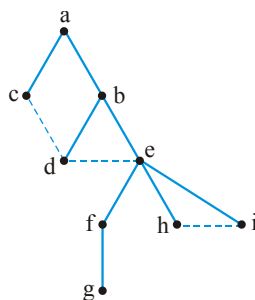
### ۵- یافتن پل (bridge)

در گراف غیر جهت‌دار، پل یالی است که اگر از گراف حذف شود (بدون حذف رئوس) گراف ناهمبند شود. و اگر ناهمبند است، تعداد مولفه‌های آن بیشتر شود. مثلاً در گراف  مثلاً در گراف  $ab$  و  $bc$  پل هستند. در واقع پل یالی است که عضو هیچ سیکلی نباشد. با کمک پیمایش عمقی می‌توان پل‌های گراف را یافت.

### پیمایش سطحی (BFS)

از یک رأس شروع می‌کند و آن را ملاقات می‌کند. سپس همه رئوس مجاور رأس شروع را ملاقات می‌کند. و همین عمل را با رأس ملاقات شده بعدی انجام می‌دهد. در واقع BFS از یک صف استفاده می‌کند. ابتدا رأس شروع را وارد صف می‌کند و تا زمانی که صف خالی نشده است، عنصر سر صف را حذف می‌کند و ملاقات می‌کند و همه رئوس مجاورش را به صف اضافه می‌کند. مثلاً برای گراف شکل ۲، با اولویت حروف الفبا، ابتدا  $a$  وارد صف می‌شود.  $a$  از صف حذف شده ملاقات می‌شود و  $b$  و  $c$  وارد صف می‌شوند.  $b$  از صف حذف شده ملاقات می‌شود و  $d$  و  $e$  وارد صف می‌شوند.  $c$  از صف حذف شده ملاقات می‌شود ولی رأسی وارد صف نمی‌شود.  $d$  از صف حذف می‌شود ملاقات می‌شود ولی رأسی وارد صف نمی‌شود.  $e$  از صف حذف شده ملاقات می‌شود و  $f$  و  $h$  و  $i$  وارد صف می‌شوند.  $f$  از صف حذف شده ملاقات می‌شود و  $g$  وارد صف می‌شود.  $h$  و  $i$  به ترتیب از صف حذف شده ملاقات می‌شوند و صف خالی می‌شود. پس ترتیب ملاقات رئوس از چپ به راست  $abcdefghig$  است و شکل درخت پوشای حاصل از این پیمایش به صورت شکل ۱۲ می‌باشد. در این شکل یالهایی که ملاقات نشده‌اند را با خط‌چین نشان داده‌ایم. همانطور که مشاهده می‌کنید یالهای  $cd$  و  $de$  و  $hi$  همگی یال عبوری هستند و سایر یالها، درختی هستند. پیمایش

سطحی به هیچ وجه یال پیش رو تولید نمی‌کند و در گراف غیر جهت‌دار یال پشتی نیز تولید نمی‌کند. لازم به ذکر است که پیمایش سطحی همانند عمقی، منحصر به فرد نیست.



شکل ۱۲. درخت پوشای پیمایش سطحی شکل ۲ با اولویت حروف الفبا.

**تست ۱:** کدام گزینه نمی‌تواند پیمایش سطحی گراف شکل ۲ باشد. (چپ به راست)؟

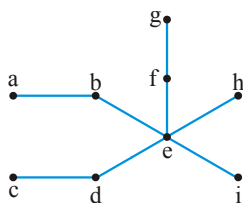
edfihbhcga (۲)

efihdbgcga (۱)

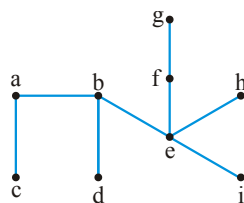
beadifhbcg (۴)

acbedfhigh (۳)

**پاسخ:** گزینه (۳). در گزینه ۳ ابتدا رأس a ملاقات شده و سپس c و b ملاقات شده‌اند حال باید با کمک c رأس d ملاقات شود (c زودتر از b وارد شده است و زودتر حذف می‌شود) که چنین نشده است و e ملاقات شده است که اشتباه است. شکل درخت پوشای سایر گزینه‌ها را می‌کشیم:



گزینه ۱ و ۲



گزینه ۴

با کمک BFS می‌توان کوتاهترین مسیرها را نسبت به رأس شروع در گراف بدون وزن بدست آورد. مثلاً در گراف شکل ۲ اگر رأس شروع (مبدا) را a فرض کنیم آنگاه  $d[a] = 0$  و  $d[b] = d[c] = 1$  و  $d[d] = d[e] = 2$  و  $d[f] = d[h] = d[i] = 3$  و  $d[g] = 4$  می‌باشد. d مخفف distance است و  $d[x]$  یعنی کمترین تعداد یال برای رسیدن از رأس مبدا به رأس x. مثلاً  $d[g] = 4$  چون کوتاهترین مسیر از a به g مسیر  $a \rightarrow b \rightarrow e \rightarrow f \rightarrow g$  است که شامل ۴ یال است.

الگوریتم پیمایش سطحی را می‌توان به شکل زیر نوشت:

```
BFS(s)
for(each vertex  $u \in V - \{s\}$ )
    { color[u] = WHITE; d[u] =  $\infty$ ;  $\pi[u]$  = NIL }
color[s] = BLACK ; d[s] = 0 ;  $\pi[s]$  = NIL ; Q =  $\phi$  ; ENQUEUE (Q,s);
while (Q  $\neq \phi$ )
{
    u = DEQUEUE(Q);
    for (each vertex v adjacent to u)
        if(color[v] = WHITE)
            {color[v] = BLACK ; d[v] = d[u] + 1 ;  $\pi[v]$  = u ; enqueue(Q, v)}
    color[u] = BLACK
}
```

در این کد  $d[x]$  برابر فاصله (distance) رأس مبدا (s) تا رأس x است. یعنی  $d[x]$  برابر حداقل تعداد یال در مسیر از s به x است.  $\pi[x]$  پدر x است یعنی رأسی که x را ملاقات کرده است برای رأس شروع  $\pi[s] = \text{NIL}$  است و اگر رأسی مثل x باشد که از رأس شروع به x دسترسی وجود نداشته باشد  $\pi[x] = \text{NIL}$  می‌باشد. در این کد اگر همه رئوس از رأس شروع قابل دسترسی باشند، نتیجه یک درخت پوشاست که با داشتن آرایه  $\pi$  می‌توان این درخت را ترسیم کرد. الگوریتم `printpath` با بررسی آرایه  $\pi$ ، کوتاهترین مسیر از رأس شروع s به رأس v را چاپ می‌کند:

```
printpath (s,v)
{
    if (v == s) print(s)
    else if ( $\pi[v] == \text{NIL}$ ) print("nopath from" s "to" v);
    else {printpath (s,  $\pi[v]$ ); print (v) ; }
```

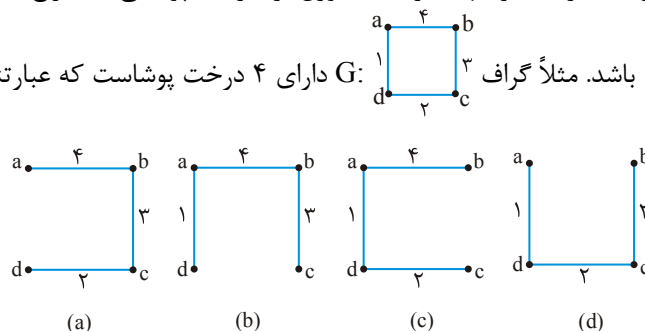
#### نکته

مرتبه الگوریتم‌های DFS و BFS برابر حافظه تخصیص داده شده به گراف است. زیرا این دو الگوریتم رئوس و یالها را بررسی می‌کنند. بنابراین اگر گراف با ماتریس مجاورتی پیاده‌سازی شود، مرتبه این الگوریتم‌ها  $\theta(n^2)$  و اگر گراف با لیست‌های مجاورتی پیاده‌سازی شود، مرتبه این الگوریتم‌ها  $\theta(n + e)$  است. البته اگر BFS لزوماً همه رئوس را ملاقات نکند، بجای نماد  $\theta$  از نماد O استفاده می‌کنیم. یادآوری می‌کنیم اگر گراف همبند باشد، می‌توان  $\theta(n + e)$  را با  $\theta(e)$  برابر گرفت. اگر گراف خلوت باشد،  $\theta(n + e)$  با  $\theta(n)$  یکسان است. و اگر گراف متراکم (شلوغ) باشد،  $\theta(n + e)$  با  $\theta(n^2)$  برابر است.

### درخت پوشای مینیمم (Minimum Spanning Tree: MST)

می‌خواهیم از یک گراف غیر جهت‌دار همبند وزن‌دار، درخت پوشایی استخراج کنیم که جمع

وزنهایش مینیمم باشد. مثلاً گراف  $G$  دارای ۴ درخت پوشاست که عبارتند از:



که وزن درخت پوشای (d) برابر  $1 + 2 + 3 = 6$  است که کمترین وزن را دارد و درخت پوشای مینیمم گراف  $G$  ( $MST(G)$ ) است.

توجه کنید اگر در گراف، وزن همه یالها متمایز باشد آنگاه  $MST$  منحصر به فرد است ولی اگر وزن یکسان وجود داشته باشد ممکن است شکل‌های مختلفی برای  $MST$  حاصل شود که البته

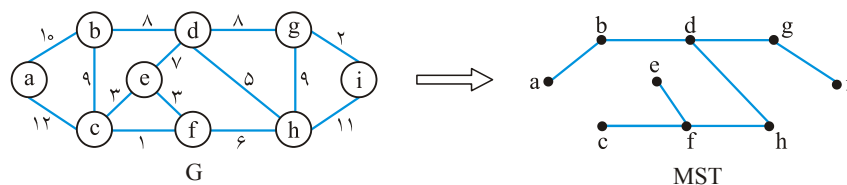
جمع وزن همگی مساوی است. مثلاً گراف  $G$  فقط یک  $MST$  به شکل  $G$  دارد.

گراف  $G$  دارای دو  $MST$  است،  $G$  و  $G$ . است که جمع وزنهای هر دو برابر ۶ است.

معروف‌ترین الگوریتم‌های یافتن  $MST$  عبارتند از: کراسکال، پریم و سولین (بروفکا)، ما در این کتاب به بررسی الگوریتم‌های کراسکال و پریم می‌پردازیم.

### الگوریتم کراسکال (kruskal)

یک الگوریتم حریصانه است. (با الگوریتم‌های حریصانه در فصل ۸ آشنا خواهید شد). یالها به ترتیب صعودی وزنشان مرتب می‌شوند سپس به ترتیب، یالها بررسی می‌شوند و اگر سیکل تشکیل نشود (اگر یال safe باشد)، انتخاب می‌شود در غیر این صورت آن یال صرفنظر (reject) می‌شود. مراحل اجرای کراسکال را روی گراف  $G$  شکل ۱۳ مشاهده کنید:



شکل ۱۳

|             | (c,f) | (g,i) | (e,f) | (c,e)  | (d,h) | (f,h) | (e,d)  | (b,d) | (d,g) | (b,c)  | (g,h)  | (a,b) | (h,i)  | (a,c)  |
|-------------|-------|-------|-------|--------|-------|-------|--------|-------|-------|--------|--------|-------|--------|--------|
| safe/reject | safe  | safe  | safe  | reject | safe  | safe  | reject | safe  | safe  | reject | reject | safe  | reject | reject |

در جدول فوق، یالها به ترتیب صعودی مرتب شده و بررسی شده‌اند و یک یال به شرطی انتخاب شده که سیکل تشکیل نشود.

با انتخاب (a,b) درخت پوشا بدست می‌آید و دیگر نیاز به بررسی یالهای باقی مانده نیست. اگر یال (c,e) قبل از (e,f) بررسی می‌شد، بجای (e,f) یال (c,e) انتخاب می‌شد.

برای بررسی تشکیل سیکل، می‌توان از مجموعه‌ها کمک گرفت. ابتدا هر رأس خود یک مجموعه مجزا است، سپس یک یال به شرطی انتخاب می‌شود که رئوسش عضو یک مجموعه نباشند. با انتخاب یال مثلاً (x,y) مجموعه‌های شامل x و y اجتماع گرفته می‌شوند. برای گراف G شکل ۱۳ می‌توان مراحل را با مجموعه‌ها نشان داد:

$\{a\} \{b\} \{c\} \{d\} \{e\} \{f\} \{g\} \{h\} \{i\}$   
 $\xrightarrow{(c,f)} \{a\} \{b\} \{d\} \{e\} \{cf\} \{g\} \{h\} \{i\}$   
 $\xrightarrow{(g,i)} \{a\} \{b\} \{d\} \{e\} \{cf\} \{gi\} \{h\}$   
 $\xrightarrow{(e,f)} \{a\} \{b\} \{d\} \{cef\} \{gi\} \{h\}$   
 $\xrightarrow{(c,e)} e, c$  عضو یک مجموعه هستند پس این یال صرفنظر می‌شود  
 $\xrightarrow{(d,h)} \{a\} \{b\} \{dh\} \{cef\} \{gi\}$   
 $\xrightarrow{(f,h)} \{a\} \{b\} \{cdefh\} \{gi\}$   
 $\xrightarrow{(e,d)} e, d$  عضو یک مجموعه هستند پس این یال صرفنظر می‌شود  
 $\xrightarrow{(b,d)} \{a\} \{bcdefh\} \{gi\}$   
 $\xrightarrow{(d,g)} \{a\} \{bcdefghi\}$   
 $\xrightarrow{(b,c)} b, c$  عضو یک مجموعه هستند پس این یال صرفنظر می‌شود  
 $\xrightarrow{(g,h)} g, h$  عضو یک مجموعه هستند پس این یال صرفنظر می‌شود  
 $\xrightarrow{(a,b)} \{abcdefghi\}$

شبه کد الگوریتم کراسکال را می‌توان به شکل زیر نوشت:

#### KRUSKAL(G, W)

```

A =  $\phi$ 
۱. for (each vertex  $v \in V$ )
    MAKE-SET( $v$ )
۲. Sort E into nondecreasing order by weight W
۳. for (each  $(u, v) \in E$  taken from the sorted list)
    if (FIND( $u$ )  $\neq$  FIND( $v$ ))
         $\{A = A \cup \{(u, v)\}; \text{UNION}(u, v)\}$ 
return A

```

قسمت (۱) الگوریتم با هر رأس یک مجموعه یک عضوی می‌سازد، قسمت (۲) یال‌ها را صعودی مرتب می‌کند، قسمت (۳) یک یال را به شرطی انتخاب می‌کند که دو سرش عضو یک مجموعه نباشد و سپس آن دو مجموعه را اجتماع می‌گیرد.

در این الگوریتم FIND( $u$ ) مجموعه‌ای که  $u$  عضو آن است را برمی‌گرداند. مرتبه کراسکال  $O(e \cdot \lg e)$  است (بخاطر مرتب‌سازی یالها) و چون  $\lg e = O(\lg n)$ ، مرتبه کراسکال را می‌توان  $O(e \cdot \lg n)$  نوشت. البته اگر یالها از قبل مرتب باشند مرتبه کراسکال  $O(e \cdot \alpha(n))$  می‌باشد. (در واقع در الگوریتم کراسکال، زمان  $O(n)$  برای حلقه for اول (مقداردهی اولیه) لازم است، زمان  $O(e \cdot \lg e)$  برای مرتب‌سازی یالها و زمان  $O(e \cdot \alpha(n))$  برای عملیات روی مجموعه که در نتیجه مرتبه کراسکال  $O(n + e \cdot \lg e + e \cdot \alpha(n)) = O(e \cdot \lg e)$  می‌شود).

#### نکته

اگر وزن یالها عدد صحیح در بازه ۱ تا  $n$  باشد آنگاه می‌توان یالها را با روش مرتب‌سازی شمارشی و با مرتبه  $O(n + e) = O(e)$  مرتب کرد که در این صورت مرتبه کراسکال  $O(e \cdot \alpha(n))$  خواهد بود.

**توجه:**  $\alpha(n)$  یک تابع با رشد بسیار ناچیز است و برای اعداد  $n$  بسیار بزرگ  $\alpha(n) \leq 4$  (برای بررسی بیشتر به فصل ۶ بخش مجموعه‌های مجزا رجوع کنید)

### الگوریتم پریم (Prim)

یک الگوریتم حریصانه است. از رأس گفته شده شروع می‌کنیم، رأسی را ملاقات می‌کنیم که با رأس شروع مجاور است و وزن یالش مینیمم است. در مرحله بعدی رأسی را ملاقات می‌کنیم که با حداقل یکی از رئوس ملاقات شده، مجاور است و وزن یالش مینیمم است، به همین ترتیب در هر مرحله یک رأس ملاقات نشده را ملاقات می‌کنیم. در واقع پریم به صورت پیوسته یالها را انتخاب می‌کند.

مراحل پریم روی گراف  $G$  شکل ۱۳ با شروع از رأس  $a$ ، به ترتیب از چپ به راست:  
 $(a, b), (b, d), (d, h), (h, f), (f, c), (c, e)$  or  $(f, e), (d, g), (g, i)$



شبه کد پریم به صورت زیر است:

```

PRIM(G, W, r)
for (each  $u \in V$ )
    {key[u] =  $\infty$  ;  $\pi[u] = \text{NIL}$ }
key[r] = 0 ; Q = V ;
while (Q  $\neq \emptyset$ )
    {u = EXTRACT - MIN(Q)
    for (each  $v \in \text{Adj}[u]$ )
        if ( $v \in Q$  and  $w(u, v) < \text{key}[v]$ )
            { $\pi[v] = u$  ;  $\text{key}[v] = w(u, v)$ }
    }

```

W تابع وزن یالهاست و r رأس شروع است. در این الگوریتم، Q یک صف اولویت است که ابتدا همه رؤوس در آن قرار می‌گیرند. کلید هر رأس در این صف بجز کلید r ابتدا برابر  $\infty$  می‌شود و کلید r برابر صفر می‌شود. در حلقه while، ابتدا تابع EXTRACT-MIN، رأس r را از صف برداشته به u منتقل می‌کند و سپس کلید همه رؤوسی که با u مجاور هستند را برابر وزن یالشان به u قرار می‌دهد و پدرشان را u قرار می‌دهد (  $\pi(v)$  یعنی پدر رأس v که اگر u ملاقات شده باشد و u با v مجاور باشد، پدر رأس v، رأس u است).

به همین ترتیب در حلقه while هر بار رأسی از صف انتخاب می‌شود که کلیدش (وزن یالش) به مینیمم باشد و با یکی از رؤوس قبلاً انتخاب شده، مجاور باشد. پس از پایان الگوریتم، یالهای  $(v, \pi[v])$  که  $v \neq r$  تشکیل‌دهنده درخت پوشای مینیمم هستند.

مرتبه پریم بستگی به پیاده‌سازی صف اولویت دارد. اگر صف اولویت را با binary minheap پیاده‌سازی کنیم، مرتبه پریم  $O(n \lg n + e \lg n) = O(e \lg n)$  است که از نظر مجانبی با کراسکال یکسان است. اگر از هیپ فیبوناچی برای پیاده‌سازی صف اولویت استفاده کنیم، مرتبه پریم  $O(e + n \lg n)$  می‌شود.

**تمرین:** اگر گراف را با ماتریس مجاورتی نمایش دهیم، پیاده‌سازی ساده‌ای از پریم ارائه دهید که مرتبه آن  $O(n^2)$  است.

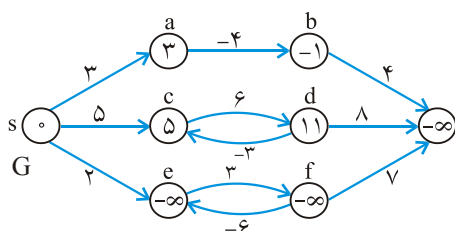
#### نکته

برای گراف خلوت (گرافی که تعداد یالهایش کم است  $e = O(n)$ ) پیاده‌سازی پریم با هیپ باینری از هیپ فیبوناچی بهتر است ولی برای گراف‌های انبوه یا شلوف  $(e = \theta(n^2))$  پیاده‌سازی با هیپ فیبوناچی سریعتر است.

### کوتاهترین مسیرهای هم مبدا (Single-Source Shortest Paths)

در گراف وزن دار جهت‌دار  $G = (V, E)$  می‌خواهیم از یک رأس مثل S کوتاهترین مسیرها را به سایر رؤوس بیابیم. الگوریتم BFS نیز کوتاهترین مسیر را از یک رأس به سایر رؤوس در گرافی که وزن ندارد یا وزن همه یالهای آن مساوی است، می‌یابد.

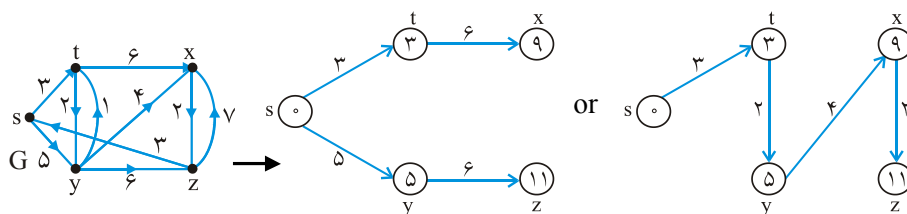
اگر گراف سیکل منفی نداشته باشد (ولی یال منفی می‌تواند داشته باشد) کوتاهترین مسیر از رأس  $S$  به همه رئوس، یک عدد حقیقی است ولی اگر سیکل منفی در گراف باشد که از رأس  $S$  قابل دسترسی باشد، کوتاهترین مسیر از  $S$  به برخی رئوس  $-\infty$  می‌باشد (یعنی کوتاهترین مسیر وجود ندارد). در گراف  $G$  شکل ۱۴ داخل هر نود، طول کوتاهترین مسیر از  $S$  به آن رأس، نوشته شده است:



شکل ۱۴.

از آنجایی که سیکل  $[e, f, e]$  منفی است  $(3 + (-6) = -3)$ ، طول کوتاهترین مسیر از  $S$  به  $e$  و  $f$  و  $g$  که از این سیکل عبور می‌کند،  $-\infty$  است.

کوتاهترین مسیر از  $S$  به یک رأس دیگر لزوماً منحصر به فرد نیست چون ممکن است چند مسیر متفاوت، با طول‌های یکسان وجود داشته باشد. در گراف  $G$  شکل ۱۵ کوتاهترین مسیرها از  $S$  به سایر رئوس مشخص شده است:



شکل ۱۵.

کوتاهترین مسیر از  $S$  به هر رأسی، فاقد سیکل است. در واقع اگر کوتاهترین مسیرها را از  $S$  به سایر رئوس رسم کنیم، یک درخت ریشه‌دار با ریشه  $S$  بدست می‌آید. که به آن  $SPT(s)$  (Shortest Path Tree) گویند.

الگوریتم‌های یافتن کوتاهترین مسیرهای هم مبدأ، همگی از دو تابع زیر استفاده می‌کنند:

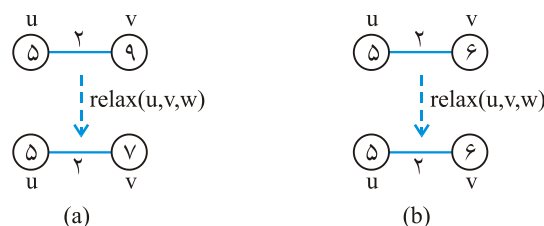
**INITIALIZE( $G, s$ )**

```
for (each vertex  $v \in V$ )
{  $d[v] = \infty$ ;  $\pi[v] = NIL$  }
 $d[s] = 0$ .
```

**RELAX( $u, v, W$ )**

```
if ( $d[v] > d[u] + w(u, v)$ )
{  $d[v] = d[u] + w(u, v)$ ;  $\pi[v] = u$  }
```

$d[v]$  طول مسیر از رأس  $S$  به  $v$  است که در ابتدا برای همه رئوس بجز  $S$  برابر  $\infty$  است و برای  $S$ ، صفر است. الگوریتم‌های یافتن کوتاهترین مسیر، تابع RELAX را بارهای مختلف فرا می‌خوانند و تابع RELAX بررسی می‌کند که مسیر  $S$  به  $v$  از  $u$  عبور کند بهتر است یا خیر. اگر طول مسیر از  $S$  به  $u$  بعلاوه وزن یال  $u$  به  $v$  کمتر از طول مسیر از  $S$  به  $v$  باشد آنگاه  $d[v]$  مقدار جدید دریافت می‌کند و پدر رأس  $v$ ، رأس  $u$  قرار داده می‌شود ( $\pi[v] \leftarrow u$ ). شکل زیر ۲ مثال از عمل RELAX را نشان می‌دهد، که در یکی  $d[v]$  تغییر کرده و در دیگری خیر:



در (a) ابتدا  $d[u]=5$  و  $d[v]=9$  و  $w(u,v)=2$  پس  $d[v] > d[u] + w(u,v)$  در نتیجه مقدار  $d[v]$  برابر ۷ می‌شود. ولی در (b) ابتدا  $d[u]=5$  و  $d[v]=6$  و  $w(u,v)=2$  پس  $d[v] < d[u] + w(u,v)$  تغییر نمی‌کند. در تمام الگوریتم‌هایی که در ادامه خواهید دید، فرض کرده‌ایم گراف با لیست مجاورتی نمایش داده می‌شود.

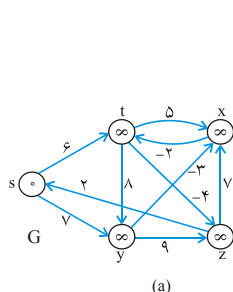
### الگوریتم بلمن فورد

```
BELLMAN_FORD(G,W,s)
۱. INITIALIZE(G,s)
۲. for (i = ۱ to n-۱)
    for (each edge (u,v) ∈ E)
        RELAX(u,v,W)
۳. for (each edge (u,v) ∈ E)
    if (d[v] > d[u] + w(u,v))
        return FALSE
return TRUE
```

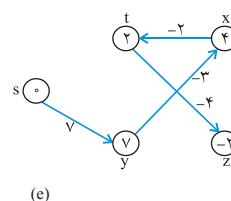
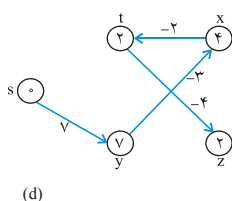
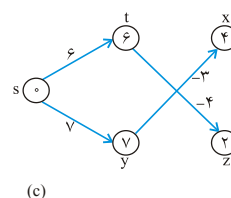
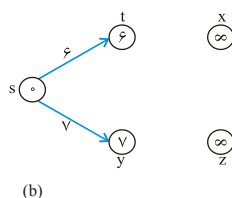
برای هر گرافی جواب را می‌یابد، حتی اگر گراف یال با وزن منفی داشته باشد. اگر گراف سیکل منفی داشته باشد که از رأس  $S$  قابل دسترسی باشد، این الگوریتم متوجه می‌شود و اعلام می‌کند. این الگوریتم  $n-1$  بار همه یالها را RELAX می‌کند. در واقع ثابت می‌شود که اگر  $n-1$  بار همه یالها RELAX شوند کوتاهترین مسیرها نسبت به  $S$  بدست می‌آیند.

دو حلقه for اول،  $n-1$  بار عمل RELAX را هر بار

روی همه یالها انجام می‌دهد. حلقه for آخر بررسی می‌کند اگر در گراف سیکل منفی که از  $S$  قابل دسترسی است وجود داشته باشد، FALSE برمی‌گرداند. شکل زیر به ترتیب اجرای این الگوریتم را روی گراف  $G$  شکل ۱۶ نشان می‌دهد. این گراف دارای ۵ رأس است پس ۴ بار روی همه یالها باید عمل RELAX انجام شود. ابتدا  $d$  برای همه رئوس  $\infty$  و برای  $S$  برابر صفر است:



شکل ۱۶.



۴ بار RELAX روی همه یالها اجرا شد. ترتیب اجرای RELAX هر بار در این مثال به ترتیب زیر است:

ترتیب :  $(t, x)(t, y)(t, z)(x, t)(y, x)(y, z)(z, x)(z, s)(s, t)(s, y)$

البته هر ترتیب دلخواه دیگری را می‌توان برای اجرای RELAX در نظر گرفت، در نتیجه نهایی تأثیری ندارد. مثلاً در شکل (b) نتیجه RELAX(s, t, w) عبارت است از:

$$d[s]=0, d[t]=\infty, w(s, t)=6 \Rightarrow d[t] > d[s] + w(s, t) \Rightarrow d[t]=6$$

و یا در قسمت (e) اجرای RELAX(t, z, w) را دقت کنید:

$$d[t]=6, d[z]=\infty, w(t, z)=-4 \Rightarrow d[z] > d[t] + w(t, z) \Rightarrow d[z]=-2$$

#### نکته

مرتبه بلمن فورد  $O(n \cdot e)$  است. البته اگر گراف با ماتریس مجاورتی ساخته شود، مرتبه  $O(n^3)$  است.

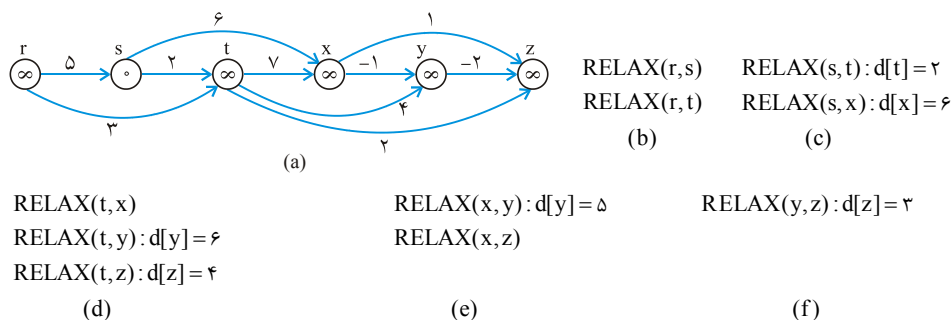
**تمرین:** الگوریتم بلمن فورد را روی گراف شکل ۱۶ و با مبدأ z اجرا کنید. ترتیب اجرای RELAX را همان ترتیب قبل در نظر بگیرید.

**تمرین:** در گراف شکل ۱۶ وزن یال  $(z, x)$  را به ۴ تغییر دهید و الگوریتم را با مبدأ S مجدداً اجرا کنید.

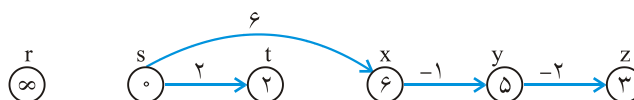
### یافتن کوتاهترین مسیرهای هم مبدأ در گراف جهت‌دار بدون سیکل

در یک گراف جهت‌دار بدون سیکل (directed acyclic graph: dag) سیکل منفی وجود ندارد (حتی اگر یال منفی وجود داشته باشد) و اگر رئوس را به ترتیب توپولوژیکی در نظر بگیریم و سپس روی یالها یک بار عمل RELAX را انجام دهیم کوتاهترین مسیرهای هم مبدأ با مرتبه  $\theta(n + e)$  بدست می‌آید. پس ابتدا INITIALIZE می‌کنیم سپس رئوس را به ترتیب توپولوژیکی RELAX می‌کنیم.

این الگوریتم روی گراف شکل زیر اجرا شده است، رئوس گراف از چپ به راست طبق ترتیب توپولوژیکی مرتب هستند. رأس مبدأ S فرض می‌شود، داخل هر نود مقدار d نوشته شده است:



نتیجه نهایی با در نظر گرفتن پدر هر نود ( $\pi$ ) به شکل زیر است:



### الگوریتم دایجسترا (Dijkstra)

یک الگوریتم حریصانه است.

اگر گراف یال منفی نداشته باشد، با این الگوریتم می‌توان کوتاهترین مسیرها از مبدأ s را سریعتر از بلمن فورد یافت. در این الگوریتم، رئوس در یک صف اولویت قرار می‌گیرند (کلید رئوس همان d (distance) است) سپس رأسی مثل u که d آن کمترین است از صف انتخاب می‌شود و به مجموعه‌ای تحت عنوان A منتقل می‌شود، سپس برای همه یالهایی که از u خارج می‌شوند، عمل RELAX انجام می‌شود:

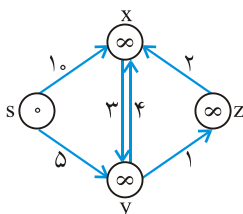
```

DIJKSTRA(G, W, s)
INITIALIZE(G, s)
A =  $\phi$ ; Q = V
while (Q  $\neq \phi$ )
{
    u  $\leftarrow$  EXTRACT - MIN(Q)
    A = A  $\cup$  {u}
    for (each vertex v  $\in$  Adj[u])
        RELAX(u, v, w)
}

```

در این کد ابتدا به رئوس مقدار اولیه داده می‌شود، سپس رئوس به صف Q اضافه می‌شوند، سپس در حلقه while مینیمم رأس حذف می‌شود و خروجی‌های آن RELAX می‌شوند تا وقتی که صف Q خالی شود. البته اگر فقط یک رأس در صف باقی بماند، می‌توان الگوریتم را خاتمه داد. A ترتیب انتخاب رئوس را مشخص می‌کند.

به عنوان مثال:



(a)  $G$

$$Q = \{s, x, y, z\}$$

$$A = \{ \}$$

(b)  $u \leftarrow s$

$$A \leftarrow \{s\}, Q = \{x, y, z\}$$

$$\text{RELAX}(s, x) : d[x] = 10$$

$$\text{RELAX}(s, y) : d[y] = 5$$

while بار اول اجرای

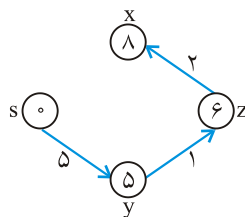
(c)  $u \leftarrow y$

$$A = \{s, y\}, Q = \{x, z\}$$

$$\text{RELAX}(y, x) : d[x] = 9$$

$$\text{RELAX}(y, z) : d[z] = 6$$

while بار دوم اجرای



(d)  $u \leftarrow z$

$$A = \{s, y, z\}, Q = \{x\}$$

$$\text{RELAX}(z, x) : d[x] = 8$$

while بار سوم اجرای

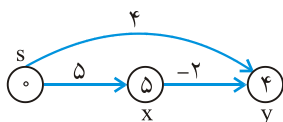
(e)  $u \leftarrow x$

$$A = \{s, y, z, x\}, Q = \{ \}$$

while بار چهارم اجرای

#### نکته

اگر صف اولویت با هیپ باینری پیاده‌سازی شود، مرتبه دایجسترا  $O(e \lg n)$  است و اگر با هیپ فیبوناچی پیاده‌سازی شود، مرتبه دایجسترا  $O(n \lg n + e)$  است. (البته اگر گراف با ماتریس مجاورتی ساخته شود، مرتبه  $O(n^2)$  است)



**توجه:** برای گرافی که یال منفی دارد، دایجسترا لزوماً جواب درست نمی‌دهد، مثلاً برای گراف مقابل نتیجه دایجسترا نشان داده شده است که نادرست است:

طبق دایجسترا،  $d[x]=5$  و  $d[y]=4$  در صورتی که جواب درست  $d[x]=5$  و  $d[y]=3$  است. البته به شرطی که حلقهٔ while فقط دو بار اجرا شود، یعنی تا زمانی اجرا شود که در صف یک عنصر در این مثال یعنی  $x$  باقی بماند.

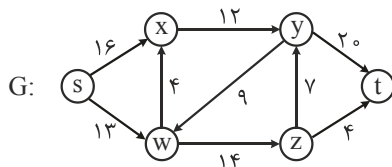
**تمرین:** دایجسترا را روی گراف شکل ۱۵ یک بار با مبداء  $s$  و یک بار با مبداء  $z$  اجرا کنید.

**توجه:** برای یافتن کوتاهترین مسیرها بین هر جفت راس، می‌توان الگوریتم بلمن فورد را  $n$  بار اجرا کرد و (هر بار با یک مبداء جدید). اگر گراف با ماتریس مجاورتی پیاده‌سازی شود، مرتبه  $\theta(n^4)$  می‌شود ولی در فصل ۹، الگوریتم فلویید ارائه شده است که می‌تواند با مرتبه  $\theta(n^3)$  کوتاهترین مسیرها بین هر جهت رأس را بیابد.

**توجه:** برای یافتن کوتاهترین مسیر بین هر دو رأس در گراف جهت‌دار وزن‌دار می‌توان  $n$  بار بلمن فورد یا  $n$  بار دایجسترا (اگر وزن‌ها نامنفی باشند) اجرا کرد یا از الگوریتم‌های شبه ضرب ماتریسی، فلویید وارشال، جانسون استفاده کرد که دو تای اول در فصل ۹ پوشش داده شده‌اند. الگوریتم جانسون در فیلم آموزشی شرح داده شده است.

### شار (جریان) بیشینه (max flow)

گراف جهت‌دار وزن‌دار  $G=(V,E)$  با وزن‌های مثبت داده شده است که به وزن‌ها ظرفیت یال گوئیم در این گراف دو رأس مشخص  $s$  و  $t$  وجود دارند که درجه ورودی  $s$  و درجه خروجی  $t$  صفر است. به چنین گرافی شبکه جریان (flow network) گوئیم.



می‌خواهیم بررسی کنیم با توجه به ظرفیت یال‌ها حداکثر چه میزان شار یا جریان (جریان آب، برق و ...) می‌توان در یک لحظه از  $s$  به  $t$  فرستاد. برای درک بهتر گراف  $G$  یک شبکه جریان است.

یال  $(s,x)$  حداکثر می‌تواند ۱۶ واحد جریان از خود عبور دهد چون ظرفیت این یال ۱۶ است. در رئوس میانی یعنی  $x$  و  $y$  و  $w$  و  $z$  امکان ذخیره جریان وجود ندارد یعنی میزان جریان وارد شده به یک رأس با میزان جریان خروجی آن برابر است. پس با اینکه ظرفیت  $(s,x)$  برابر ۱۶ است ولی با توجه به ظرفیت  $(x,y)$ ، این یال بیشتر از ۱۲ واحد جریان نباید منتقل کند. رأس  $s$  تولید کننده جریان و رأس  $t$  انبارکننده آن است. مثلاً با توجه به ظرفیت یال‌های خروجی از  $s$  می‌توانیم حداکثر  $16+13=29$  واحد جریان تولید کنیم و با توجه به ظرفیت یال‌های ورودی  $t$  می‌توان حداکثر  $20+4=24$  واحد جریان ذخیره کرد پس  $s$  نمی‌تواند ۲۹ واحد جریان تولید کند بلکه حداکثر می‌تواند ۲۴ واحد جریان تولید کند. توجه کنید تمام جریان تولیدی توسط  $s$  در لحظه در

$t$  باید ذخیره شود. ما می‌خواهیم بررسی کنیم حداکثر جریان چقدر است؟ در این مثال خواهیم دید که جواب ۲۳ واحد است.

**قضیه:** شار بیشینه برابر است با ظرفیت برش کمینه.

این قضیه می‌خواهد کار ما را ساده‌تر کند. دقت کنید مسئله یافتن شار بیشینه است و این قضیه می‌گوید شار بیشینه برابر است با ظرفیت برش کمینه. اما برش (cut) یعنی چه؟ کلاً در هر گرافی برش یعنی افراز رئوس به دو مجموعه  $(S, T)$ . در گراف شبکه شار برش یک ویژگی دیگر دارد و آن اینکه حتماً  $s \in S$  و  $t \in T$  در گراف  $G$  فوق چند برش را مشخص می‌کنیم:

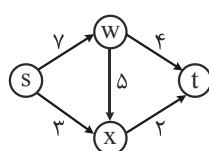
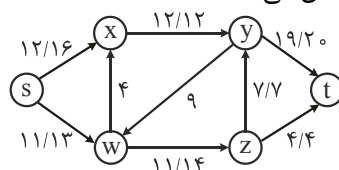
$$C_1 : (\{s, x, w\}, \{y, z, t\})$$

$$C_2 : (\{s, w, x, y, z\}, \{t\})$$

$$C_3 : (\{s, x, w, z\}, \{y, t\})$$

یال‌های یک برش یال‌هایی هستند مثل  $(a, b)$  که  $a \in S$  و  $b \in T$ . مثلاً یال‌هایی برش  $C_1$  عبارت‌اند از:  $(x, y)$  و  $(w, z)$ . یال‌های برش  $C_2$  عبارت‌اند از:  $(y, t)$  و  $(z, t)$ . یال‌های برش  $C_3$  عبارت‌اند از:  $(x, y)$  و  $(z, y)$  و  $(z, t)$  (دقت کنید یال  $(y, w)$  یال این برش نیست). ظرفیت یک برش برابر مجموع ظرفیت یال‌های برش است پس ظرفیت برش‌های  $C_1, C_2, C_3$  به ترتیب برابر  $26 = 12 + 14$  و  $24 = 20 + 4$  و  $23 = 12 + 7 + 4$  می‌باشد. برش کمینه برشی است که کمترین ظرفیت را دارد. می‌توان بررسی کرد که برش کمینه در این مثال (در بین همه برش‌ها)  $C_3$  است که ظرفیت ۲۳ دارد و طبق این قضیه شار بیشینه برابر ۲۳ می‌شود.

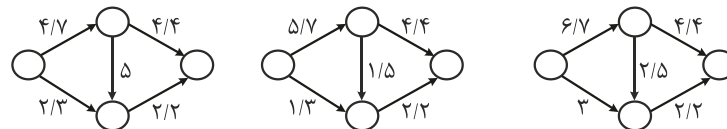
البته برای یافتن شار بیشینه الگوریتم‌هایی وجود دارند که یکی از آن‌ها را در ادامه خواهیم دید. در این مثال سوالی مطرح می‌شود و اینکه ما این ۲۳ واحد شار را چگونه از  $s$  به  $t$  ارسال کنیم (البته الگوریتمی که خواهیم گفت به این سوال نیز پاسخ می‌دهد) با آزمایش و خطا می‌توان طبق شکل مقابل شار فرستاد. روی هر یال دو عدد به صورت  $f/c$  نوشته شده که  $c$  ظرفیت یال و  $f$  میزان شار ارسالی روی آن یال را نشان می‌دهد.



اگر میزان شار روی یالی صفر است نوشته نشده است مثلاً شار یال  $(w, x)$  صفر است. لازم به ذکر است که عدد شار بیشینه منحصر به فرد است ولی نحوه ارسال شار بیشینه لزوماً منحصر به فرد نیست. در شکل مقابل شار بیشینه برابر ۶ است (برش  $(\{s, w, x\}, \{t\})$  کمینه است)



که می‌توان به صورت‌های مختلف این ۶ واحد شار را ارسال کرد:



حال به بیان روشی موسوم به فورد فالکرسون (FF) برای یافتن شار بیشینه می‌پردازیم. این روش با مفاهیم شبکه پسماند (residual network) و مسیر افزونه (augmenting path) سروکار دارد. ایده روش این است که ابتدا شار همه یال‌ها را صفر قرار می‌دهد سپس به تدریج شار را افزایش می‌دهد تا حدی که مطمئن شود به شار بیشینه رسیده است، و در نهایت شار بیشینه برابر است با میزان شاری که از  $s$  خارج می‌شود یا میزان شاری که به  $t$  وارد می‌شود. شبکه یا گراف پسماند  $G_f$  را با نشان می‌دهیم و به این صورت به دست می‌آید که هر یال در  $G$  با حداکثر ۲ یال

جایگزین می‌شود. مثلاً یال  $(x \xrightarrow{f/c} y)$  که ظرفیت  $c$  و شار  $f$  دارد به صورت  $(x \xrightarrow{f} y) \xleftarrow{c-f} (x)$  در

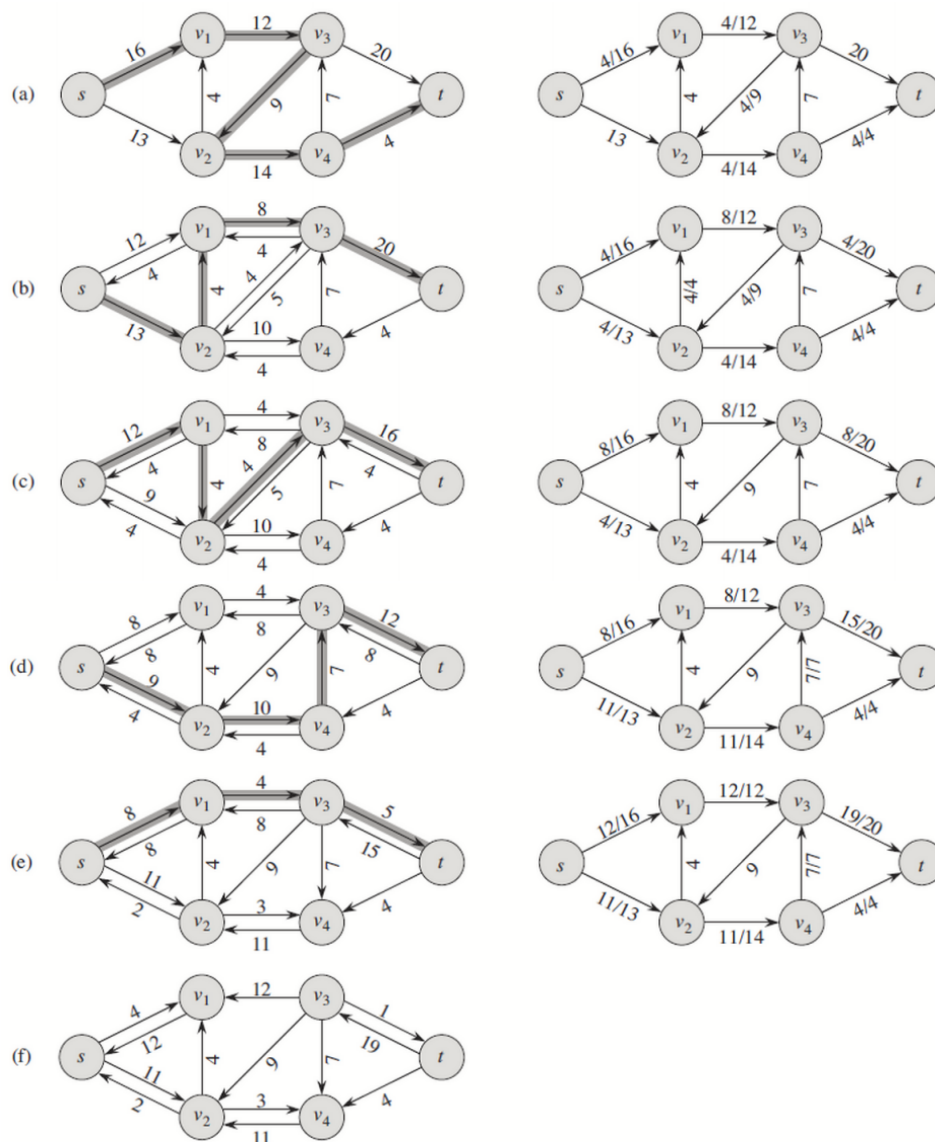
$G_f$  جایگزین می‌شود. در واقع مفهوم این دو یال این است که می‌توان  $c-f$  واحد شار بیشتری از  $x$  به  $y$  ارسال کرد و همچنین می‌توان  $f$  واحد شار ارسالی از  $x$  به  $y$  را کنسل کرد. مثلاً  $(x \xrightarrow{4/7} y)$  در  $G$  به این معنی است که یال  $(x, y)$  ظرفیت ۷ دارد و ۴ واحد شار روی آن در

جریان است حال این یال در  $G_f$  تبدیل به دو یال  $(x \xrightarrow{4} y) \xleftarrow{3} (x)$  می‌شود که به این معنی است

که می‌توان ۳ واحد شار بیشتر از  $x$  به  $y$  ارسال کرد و می‌توان ۴ واحد از شار  $x$  به  $y$  را کنسل کرد. در ضمن اگر مقدراً  $f$  یا  $c-f$  صفر شوند، یال مربوطه کشیده نمی‌شود. مسیر افزونه نیز هر مسیر ساده‌ای از  $s$  به  $t$  در  $G_f$  است.

روش FF به این صورت است که ابتدا شار همه یال‌ها را صفر قرار می‌دهد سپس از روی گراف  $G$  گراف پسماند  $G_f$  را می‌سازد (که البته مرحله اول  $G_f$  همان  $G$  است) سپس یک مسیر دلخواه از  $s$  به  $t$  در  $G_f$  می‌یابد که به مسیر افزونه معروف است.

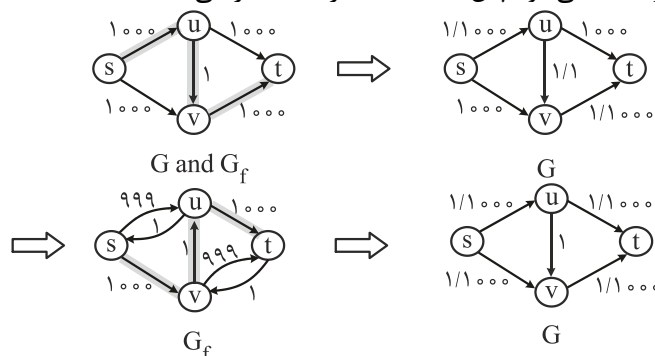
ظرفیت کم ظرفیت‌ترین یال این مسیر را در نظر می‌گیرد که فرض کنید این ظرفیت برابر  $C_f$  است حال در گراف  $G$  به شار یال‌های موافق این مسیر  $C_f$  را می‌افزاید و از شار یال‌های مخالف این مسیر  $C_f$  را کم می‌کند و مجدد از روی گراف  $G$  گراف  $G_f$  را می‌سازد و همین عملیات را تکرار می‌کند تا زمانی که در  $G_f$  هیچ مسیری از  $s$  به  $t$  وجود نداشته باشد. برای درک بهتر به شکل زیر توجه کنید:



در سمت چپ هر قسمت گراف پسماند  $G_f$  به همراه مسیر افزونه آن نشان داده شده است که در قسمت (a) همان  $G$  است. در سمت راست هر قسمت نتیجه به روز کردن شار یال‌های مسیر افزونه روی  $G$  نشان داده شده است. مثلاً در قسمت (a) مسیر افزونه پررنگ شده که کم ظرفیت‌ترین یال ظرفیت  $C_f = 4$  دارد و حال به شار یال‌های این مسیر ۴ واحد اضافه شده که در سمت راست قسمت (a) نشان داده شده. سپس در قسمت (b) مجدد  $G_f$  به دست آمده به این

صورت که مثلاً یال  $(S, V_1)$  تبدیل به  $(S, V_1)$  شده است: و به همین ترتیب کار ادامه پیدا می‌کند. در قسمت (c) توجه کنید که مسیر افزونه  $s \rightarrow v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow t$  دارای  $C_f = 4$  است که به شار یال‌های  $(s, v_1), (v_3, t)$  که موافق این مسیر هستند ۴ واحد کم شده است. در قسمت (e) سمت راست جواب به دست آمده زیرا در قسمت (f) که گراف  $G_f$  کشیده شده هیچ مسیری از  $s$  به  $t$  وجود ندارد.

اگر شار بیشینه برابر  $f^*$  باشد (در مثال فوق  $f^* = 23$ ) آن‌گاه زمان اجرای روش FF برابر  $O(e \cdot f^*)$  است زیرا در هر مرحله یک مسیر افزونه از  $s$  به  $t$  می‌یابد که یافتن مسیر با روش‌هایی مثل DFS یا BFS امکان‌پذیر است و مرتبه یافتن مسیر روی گراف همبند  $O(e)$  است و در بدترین حالت در هر مرحله فقط یک واحد به میزان شار اضافه می‌شود و بنابراین حداکثر، روش FF ممکن است  $f^*$  بار مسیر افزونه بیابد. مثلاً در گراف زیر  $f^* = 2000$  است که ۱۰۰۰ واحد آن از طریق مسیر  $s \rightarrow u \rightarrow t$  و ۱۰۰۰ واحد از طریق مسیر  $s \rightarrow v \rightarrow t$  منتقل می‌شود حال اگر FF مسیر  $s \rightarrow u \rightarrow v \rightarrow t$  را به عنوان مسیر افزونه انتخاب کند فقط یک واحد به شار یال‌های  $(v, t), (s, u)$  اضافه می‌شود و در مرحله بعد اگر مسیر  $s \rightarrow v \rightarrow u \rightarrow t$  را انتخاب کند فقط یک واحد به شار  $(s, v)$  و  $(u, t)$  اضافه می‌شود و اگر همچنین نحوه انتخاب را ادامه دهد در هر مرحله فقط یک واحد به شار اضافه می‌شود پس باید ۲۰۰۰ مرحله مسیریابی کند.



ولی اگر FF در مرحله اول مسیر  $s \rightarrow u \rightarrow t$  را انتخاب کند ۱۰۰۰ واحد به شار اضافه می‌شود و اگر مرحله دوم مسیر  $s \rightarrow v \rightarrow t$  را به عنوان مسیر افزونه انتخاب کند ۱۰۰۰ واحد دیگر اضافه می‌شود و کار تمام می‌شود.

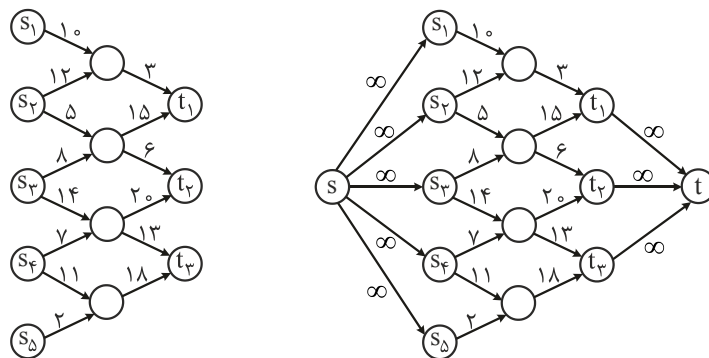
### نکته

الگوریتمی موسوم به ادموند کارپ وجود دارد که با روش فورد فالکرسون (FF) شار بیشینه را می‌یابد ولی در هر مرحله مسیر افزونه را با کمک الگوریتم BFS پیدا می‌کند یعنی مسیر با کمترین تعداد یال از  $s$  به  $t$

را به عنوان مسیر افزونه انتخاب می‌کند. ثابت می‌شود این الگوریتم حداکثر  $ne$  بار مسیریابی می‌کند و هر مرحله با زمان  $n + e = e$  است که مرتبه ادموند کارپ  $O(n.e^2)$  می‌شود.

## نکته

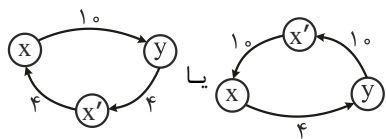
اگر گراف  $G$  دارای چندین منبع چندین مقصد باشد می‌توان این گراف را گرافی فقط با یک مبدأ و یک مقصد تبدیل کرد و سپس با روش FF شار بیشینه یافت کافی است یک رأس مبدأ به اسم  $s$  به گراف اضافه کنیم و از  $s$  به همه مبدأها یک یال با ظرفیت  $\infty$  در نظر بگیریم. برای مقصد نیز به همین ترتیب در شکل زیر یک نمونه آمده است.



## نکته

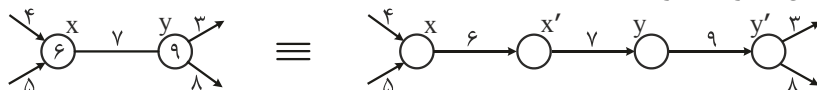
اگر در گراف  $G$  یال‌های ضد موازی (antiparallel) وجود داشت می‌توان با افزودن رأس این یال‌ها را از بین برد مثلاً  $(x, y)$  و  $(y, x)$  را ضد موازی گوئیم که نباید در گراف  $G$  باشند و

اگر بودند می‌توان معادل در نظر گرفت و سپس با FF شار بیشینه یافت.



## نکته

اگر در گراف  $G$  رئوس نیز ظرفیت داشتند می‌توان گراف را به حالت استاندارد تبدیل کرد یعنی به گرافی که فقط یال‌ها ظرفیت دارند.



## خلاصه فصل

- ۱- تعداد رئوس گراف را با  $n$  یا  $|V|$  و تعداد یال را با  $e$  یا  $|E|$  نشان می‌دهیم.
- ۲- تعداد یال گراف ساده حداکثر  $\frac{n(n-1)}{2}$  و اگر جهت‌دار باشد حداکثر  $n(n-1)$  است که در هر دو صورت از مرتبه  $\theta(n^2)$  است. حداقل تعداد یال صفر و اگر گراف همبند باشد حداقل  $n-1$  است.
- ۳- گراف متراکم (dense) گرافی است که تعداد یال آن  $\theta(n^2)$  (یعنی زیاد) است و گراف خلوت گرافی است که تعداد یال آن  $O(n)$  (یعنی کم) است.
- ۴- گراف را می‌توان با روش‌های مختلفی از جمله ماتریس و لیست مجاورتی پیاده‌سازی کرد. حافظه مصرفی ماتریس  $\theta(n^2)$  و حافظه مصرفی لیست  $\theta(n+e)$  است. اگر گراف خلوت باشد حافظه مصرفی لیست  $\theta(n)$ ، اگر متراکم باشد  $\theta(n^2)$  و اگر گراف همبند باشد، حافظه لیست  $\theta(e)$  است.
- ۵- الگوریتم‌های DFS و BFS گراف را پیمایش می‌کنند و اگر همه رئوس ملاقات شوند، خروجی یک جنگل پوشاست. اگر گراف با لیست مجاورتی پیاده‌سازی شود، مرتبه این دو الگوریتم  $O(n+e)$  و در صورت پیاده‌سازی با ماتریس، مرتبه  $O(n^2)$  است.
- ۶- با BFS و DFS می‌توان وجود سیکل در گراف را تشخیص داد.
- ۷- در گراف ۴ نوع یال وجود دارد که عبارتند از: درختی، پستی، پیش‌رو، عبوری، که با کمک پیمایش می‌توان این یال‌ها را تشخیص داد.
- ۸- کاربردهای مهم DFS عبارتند از: - یافتن ترتیب توپولوژیکی DAG. - یافتن مؤلفه‌های متصل قوی در گراف جهت‌دار. - یافتن نقاط انفصال و پل‌های گراف.
- ۹- کاربردهای مهم BFS عبارتند از: - یافتن کوتاه‌ترین مسیرهای هم مبدأ در گراف بدون وزن. - تشخیص دو بخشی بودن گراف.
- ۱۰- با الگوریتم‌های کراسکال و پریم و سولین می‌توان از یک گراف وزن‌دار غیرجهت‌دار همبند، درخت پوشای مینیمم (MST) بدست آورد.
- ۱۱- مرتبه کراسکال  $O(e \cdot \lg n)$  است. اگر گراف با ماتریس مجاورتی پیاده‌سازی شود. مرتبه پریم  $O(n^2)$  است. اگر گراف با لیست مجاورتی پیاده‌سازی شود و از هیپ دودویی برای ساخت صف اولویت استفاده شود مرتبه پریم  $O(e \cdot \lg n)$  است و اگر از هیپ فیبوناچی استفاده شود، مرتبه پریم  $O(e + n \cdot \lg n)$  است.

- ۱۲- برای یافتن کوتاهترین مسیرهای هم مبدأ در گراف جهت‌دار وزن‌دار، الگوریتم‌های بلمن فورد و دایجسترا مطرح شده‌اند. بلمن فورد برای گراف جهت‌دار با یال منفی به درستی جواب می‌دهد و سیکل منفی را تشخیص می‌دهد و مرتبه آن با ماتریس  $\theta(n^3)$  و با لیست  $\theta(n.e)$  است. دایجسترا برای گراف با وزن‌های مثبت جواب می‌دهد و مرتبه آن دقیقاً مثل پریم است.
- ۱۳- در DAG می‌توان با کمک DFS، کوتاهترین مسیرهای هم مبدأ را یافت.
- ۱۴- یافتن کوتاهترین مسیر بین هر دو رأس در گراف جهت‌دار وزن‌دار با اجزای  $n$  بار بلمن فورد (زمان  $n \times ne$ )،  $n$  بار دایجسترا (اگر وزن‌ها نامنفی باشند و زمان  $n \times (e + n \lg n)$ ) امکان‌پذیر است. ولی الگوریتم معروف فلویید وارشال با زمان  $n^3$  این مسئله را حل می‌کند. الگوریتم جانسون نیز وجود دارد که وزن‌ها را ابتدا نامنفی می‌کند و سپس  $n$  بار دایجسترا می‌زند که مرتبه آن  $n(e + n \lg n)$  است.
- ۱۵- برای یافتن شار بیشینه روش فورد فاکلرسون  $f^*$  بار حداکثر مسیریابی می‌کند که  $f^*$  اندازه شار بیشینه است. الگوریتم ادموند کارپ حداکثر  $n.e$  بار مسیریابی می‌کند. زمان هر مسیریابی  $n + e$  است.

## تمرین

- ۱- ترانهاده گراف جهت‌دار  $G$  از معکوس کردن یالهای  $G$  به دست می‌آید ( $G^T$ ). الگوریتمی برای محاسبه  $G^T$  از روی  $G$  که با لیست مجاورتی و ماتریس مجاورتی نشان داده شده است، بیابید.
- ۲- لیست مجاورتی گراف چندگانه  $G$  داده شده است، الگوریتمی از مرتبه  $O(n + e)$  ارائه دهید که تمام لبه‌های چندگانه را به یک لبه تبدیل کند و تمام خود حلقه‌ها را حذف کند و در نهایت یک گراف ساده تولید کند.
- ۳- مربع گراف  $G$  شامل یالهایی مثل  $(u, w)$  است به طوری که  $(u, v)$  و  $(v, w)$  در  $G$  باشند، الگوریتمی برای یافتن  $G^2$  بیابید.
- ۴- در گراف جهت‌دار  $G$ ، اگر رأسی با درجه ورودی  $n-1$  و خروجی صفر وجود داشته باشد، به آن  $universal\ sink$  گویند، اگر گراف با ماتریس مجاورتی پیاده شود، الگوریتمی با مرتبه  $O(n)$  برای یافتن چنین رأسی در صورت وجود ارائه دهید.
- ۵- ماتریس برخورد گراف جهت‌دار  $G$  ماتریس  $B = [b_{ij}]_{n \times e}$  است که:
 
$$b_{ij} = \begin{cases} -1 & \text{if edge } j \text{ Leaves vertex } i \\ 1 & \text{if edge } j \text{ enters vertex } i \\ 0 & \text{other wise} \end{cases}$$
 درایه‌های  $B \cdot B^T$  نشان‌دهنده چه چیزی هستند؟
- ۶- قطر درخت  $T$  برابر ماکزیمم کوتاه‌ترین فاصله‌های بین رئوس است، الگوریتمی برای محاسبه قطر یک درخت ارائه دهید.
- ۷- نشان دهید یال  $(u, v)$ : (با توجه به پیمایش DFS)
  - (a) درختی یا پیش‌رو است اگر و فقط اگر  $d[u] < d[v] < f[v] < f[u]$
  - (b) پشتی است اگر و فقط اگر  $d[v] < d[u] < f[u] < f[v]$
  - (c) عبوری است اگر و فقط اگر  $d[v] < f[v] < d[u] < f[u]$
- ۸- با مثال نقض نشان دهید جمله مقابل صحیح نیست: «اگر یک مسیر از  $u$  به  $v$  در گراف جهت‌دار  $G$  وجود داشته باشد و اگر  $d[u] < d[v]$  (با توجه به پیمایش DFS)، آنگاه در جنگل حاصل از DFS،  $v$  نواده  $u$  است.
- ۹- مثال نقضی برای جمله مقابل ارائه دهید: «اگر یک مسیر از  $u$  به  $v$  در گراف جهت‌دار  $G$  باشد آنگاه هر پیمایش DFS باید منجر به  $d[v] \leq f[u]$  شود.
- ۱۰- توضیح دهید چگونه ممکن است در یک گراف جهت‌دار، رأس  $u$  درجه ورودی و خروجی مخالف صفر داشته باشد ولی در جنگل حاصل از DFS، تک باشد.

۱۱- می‌توان DFS را طوری تغییر داد که مولفه‌های هم‌بند گراف غیرجهت‌دار  $G$  را مشخص کند، کافیت یک شمارنده در نظر بگیریم که تعداد مولفه‌های هم‌بند را مشخص کند، الگوریتم ارائه دهید.

۱۲- یک الگوریتم با زمان خطی ارائه دهید که یک گراف جهت‌دار بدون سیکل و دو رأس  $s$  و  $t$  دریافت کند و تعداد مسیرهای از  $s$  به  $t$  را بشمارد.

۱۳- الگوریتمی از مرتبه  $O(n)$  ارائه دهید که مشخص کند گراف غیر جهت‌دار  $G$ ، سیکل دارد یا خیر.

۱۴- روش دیگر برای مرتب‌سازی توپولوژیکی گراف جهت‌دار بدون سیکل این است که به طور مداوم، رأس با درجه ورودی صفر را یافته چاپ کرده و آن رأس و یالهایی که از آن خارج می‌شوند را از گراف حذف کنیم. پیاده‌سازی این ایده را با زمان  $O(n + e)$  انجام دهید.

۱۵- گراف جهت‌دار  $G$  نیمه هم‌بند (semiconnected) است اگر برای هر جفت رأس  $u, v$  یا از  $u$  به  $v$  و یا برعکس مسیر باشد، الگوریتمی از مرتبه  $O(n + e)$  ارائه دهید که مشخص کند  $G$  نیمه هم‌بند است یا خیر.

۱۶- گراف غیرجهت‌دار هم‌بند  $G$  را در نظر بگیرید. نقطه انفصال (articulation) رأسی است که حذف آن گراف را ناهم‌بند کند. پل، یالی است که حذف آن گراف را ناهم‌بند کند. فرض کنید  $G_\pi = (V, E_\pi)$  درخت حاصل از DFS روی گراف  $G$  باشد:

(a) نشان دهید ریشه  $G_\pi$  نقطه انفصال  $G$  است اگر و فقط اگر دارای دو فرزند در  $G_\pi$  باشد

(b) اگر  $v$  یک رأس غیرریشه در  $G_\pi$  باشد، نشان دهید  $v$  نقطه انفصال در  $G$  است اگر و فقط اگر  $v$  فرزندی مثل  $s$  داشته باشد بطوری که از  $s$  یا هیچ یک از نواده‌های  $s$ ، یال پشتی به اجداد محض  $v$  وجود نداشته باشد.

(c) تعریف می‌کنیم.

$$\text{Low}[v] = \min \begin{cases} d[v], \\ d[w] : (u, w) \text{ is a back edge for some descendant } u \text{ of } v \end{cases}$$

نشان دهید چگونه می‌توان  $\text{Low}[v]$  را برای همه رئوس  $v \in V$  در زمان  $O(e)$  بدست آورد.

(d) چگونه می‌توان همه نقاط انفصال را با زمان  $O(e)$  یافت.

(e) نشان دهید یک یال از  $G$ ، پل است اگر و فقط اگر در هیچ سیکل ساده از  $G$  نباشد

(f) چگونه می‌توان همه پل‌ها را با زمان  $O(e)$  بدست آورد.

۱۷- الگوریتمی از مرتبه  $O(e)$  برای یافتن مدار اویلری (در صورت وجود) در گراف جهت‌دار هم‌بند بیابید.

۱۸- فرض کنید  $G$  یک گراف جهت‌دار باشد، به هر رأس  $G$  یک برچسب از ۱ تا  $n$  نسبت می‌دهیم.

$R(u)$  مجموعه رئوسی است که از  $u$  قابل دسترسی هستند. الگوریتمی از مرتبه  $O(n + e)$  ارائه دهید که رأسی که از  $u$  قابل دسترسی است و می‌نیم برچسب را دارد بیابید.



۱۹- اگر وزن یالها عدد صحیح از ۱ تا  $n$  باشد، زمان کراسکال را چگونه می‌توان بهبود بخشید؟ اگر وزن یالها صحیح ۱ تا  $w$  که  $w$  ثابت است باشد، چطور؟

۲۰- نشان دهید اگر  $G$  یک گراف غیرجهت‌دار هم‌بند باشد و وزن همه یالها متمایز باشد آنگاه درخت پوشای می‌نیم منحصراً به فرد است ولی دومین بهترین درخت پوشای می‌نیم لزوماً منحصراً به فرد نیست.

۲۱- نشان دهید از ۳ الگوریتم زیر،  $a$  و  $c$  درخت پوشای می‌نیم تولید می‌کنند ولی  $b$  خیر.

- a. MAYBE – MST-A ( $G, w$ )
  - 1 sort the edges into nonincreasing order of edge weights  $w$
  - 2  $T \leftarrow E$
  - 3 for each edge  $e$ , taken in nonincreasing order by weight
  - 4     do if  $T - \{e\}$  is a connected graph
  - 5         then  $T \leftarrow T - e$
  - 6 return  $T$
- b. MAYBE- MST-B( $G, w$ )
  - 1  $T \leftarrow \phi$
  - 2 for each edge  $e$ , taken in arbitrary order
  - has no cycles  $T \cup \{e\}$      do if
  - 4     then  $T \leftarrow T \cup e$
- c. MAYBE-MST – C ( $G, w$ )
  - 1  $T \leftarrow \phi$
  - 2 for each edge  $e$ , taken in arbitrary order
  - 3 do  $T \leftarrow T \cup \{e\}$
  - 4     if  $T$  has a cycle  $c$
  - 5         then let  $e'$  be a maximum – weight edge on  $c$
  - 6          $T \leftarrow T - \{e'\}$
  - 7 return  $T$

۲۲- یک درخت پوشای گلوگاه (bottleneck) از گراف غیر جهت‌دار  $G$  یک درخت پوشا است که یال با بیشترین وزنش، در بین همه درخت‌های پوشای  $G$ ، می‌نیم باشد. می‌گوییم که مقدار یک درخت پوشای گلوگاه، وزن یالی است که بیشترین وزن را دارد.

(a) نشان دهید که یک درخت پوشای می‌نیم یک درخت پوشای گلوگاه است.

(b) یک الگوریتم با زمان خطی ارائه دهید که گراف  $G$  و عدد صحیح  $b$  را دریافت کند و مشخص کند آیا مقدار درخت پوشای گلوگاه حداکثر  $b$  است یا خیر.

(c) از الگوریتم قسمت b به عنوان یک سابروتین برای الگوریتم مسئله درخت پوشای گلوگاه با زمان خطی استفاده کنید.

## پاسخ تمرین

۱- اگر گراف با ماتریس مجاورتی  $A$  نشان داده شود، با مرتبه  $\theta(n^2)$ ،  $A^T$  را بدست می‌آوریم و اگر گراف با لیست مجاورتی پیاده‌سازی شود می‌توان با مرتبه  $\theta(n + e)$  ترانهاده گراف را یافت.

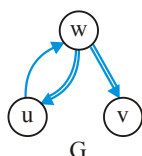
۶- اگر  $x$  نودی با عمق  $d(x)$  باشد، قطر  $D(x)$ :

$$D(x) = \begin{cases} \max \{ \max_i \{ D(x \cdot \text{child}_i) \}, \max_{ij} \{ d(x \cdot \text{child}_i) + d(x \cdot \text{child}_j) \} + 2 \} & \text{نود داخلی } x \\ 0 & \text{برگ } x \end{cases}$$

با برنامه‌نویسی پویا و با مرتبه خطی می‌توان قطر را به دست آورد.

۸- واضح است یک مسیر از  $u$  به  $v$  در  $G$  وجود دارد. یال‌های پر رنگ پیمایش DFS را نشان می‌دهد.  $d[u] < d[v]$  ولی  $v$  نواده  $u$  در DFS حاصل نیست.

|   | d | f |
|---|---|---|
| w | ۱ | ۶ |
| u | ۲ | ۳ |
| v | ۴ | ۵ |



۹- در تمرین قبل، یک مسیر از  $u$  به  $v$  در  $G$  هست و  $d[v] > f[u]$

۱۰- دقت کنید از رأس  $w$  شروع کردیم.

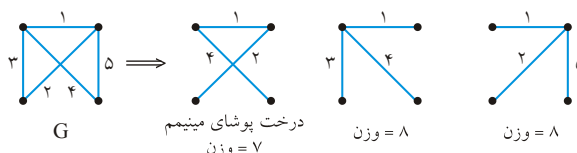
|   | d | f |
|---|---|---|
| w | ۱ | ۲ |
| u | ۳ | ۴ |
| v | ۵ | ۶ |



۱۳- گراف غیر جهت‌دار  $G$  سیکل ندارد اگر و فقط اگر DFS آن یال پشتی نداشته باشد. DFS را اجرا می‌کنیم اگر  $n$  یال مجزا دیدیم، یال پشتی وجود دارد پس سیکل وجود دارد. مرتبه الگوریتم  $O(n)$  است و نه  $O(n + e)$ . زیرا اگر ما  $n$  یال دیدیم کار تمام است و سیکل دارد.

۱۹- می‌دانیم الگوریتم کراسکال،  $O(n)$  زمان برای مقداردهی اولیه،  $O(e \log e)$  زمان برای مرتب‌سازی یالها و  $O(e \cdot \alpha(n))$  زمان برای عملیات مربوط به مجموعه‌های مجزا صرف می‌کند. اگر وزن یالها صحیح ۱ تا  $n$  باشد می‌توان با مرتب‌سازی شمارشی، یالها را با زمان  $O(n + e) = O(e)$  مرتب کرد که در نتیجه زمان کراسکال  $O(n + e + e \cdot \alpha(n)) = O(e \cdot \alpha(n))$  است. اگر وزن یالها صحیح ۱ تا  $w$  باشد نیز همین وضعیت پیش می‌آید.

۲۰- گراف  $G$  دارای دو دومین بهترین درخت پوشای می‌نیم است.



## سؤال‌های چهارگزینه‌ای فصل هفتم

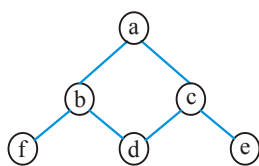
۱- در یک گراف هم‌بند، بدون جهت، بدون حلقه و بدون یال چندگانه، کدام یک از نامساوی‌های زیر ممکن است برقرار نباشد؟  
(هوش مصنوعی ۹۰)

$$\begin{aligned} (1) \quad |V| &\geq \frac{|E|}{2} & (2) \quad |E| &\leq |V|^2 \\ (3) \quad |E| &\geq |V| - 1 & (4) \quad |V| &\leq |E| + 1 \end{aligned}$$

۲- در یک گراف جهت‌دار، بدون دور و بدون یال چندگانه که با بی‌جهت گرفتن یال‌ها گراف هم‌بند است، کدام یک از نامساوی‌های زیر ممکن است برقرار نباشد؟  
(IT ۹۱)

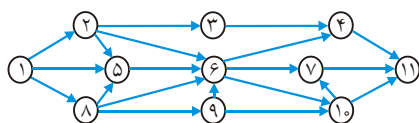
$$(1) \quad |E| \leq |V|^2 \quad (2) \quad |E| \geq |V| - 1 \quad (3) \quad |V| \leq |E| + 1 \quad (4) \quad |V| \geq \frac{|E|}{2}$$

۳- کدام گزینه نمی‌تواند خروجی پیمایش اول عمق (dfs) گراف زیر باشد؟ (چپ به راست)  
(علوم کامپیوتر ۸۰)



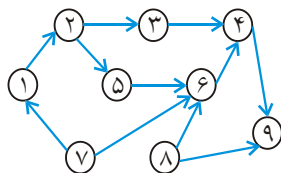
- (۱) a b d c e f
- (۲) a b d c f e
- (۳) a c d b f e
- (۴) a b f d c e

۴- در جستجوی عمق اول (DFS) گراف جهت‌دار زیر، فرض کنید که گره ۱، گره شروع باشد و گره‌های مجاور یک گره به ترتیب مقدار عددی‌شان ملاقات می‌شوند. کدام گزینه (از چپ به راست) ترتیب گره‌های ملاقات شده را نشان می‌دهد؟  
(دولتی ۸۱)



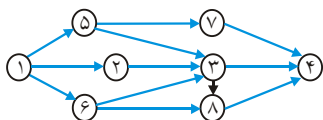
- (۱) ۱, ۲, ۳, ۴, ۱۱, ۵, ۶, ۷, ۱۰, ۸, ۹
- (۲) ۱, ۲, ۸, ۵, ۳, ۹, ۶, ۴, ۱۱, ۱۰, ۷
- (۳) ۱, ۲, ۵, ۸, ۳, ۹, ۶, ۴, ۱۰, ۷, ۱۱
- (۴) ۱, ۲, ۳, ۴, ۵, ۶, ۷, ۸, ۹, ۱۰, ۱۱

۵- یک topological sort مربوط به گراف زیر را انتخاب کنید:  
(علوم کامپیوتر ۸۱)



- (۱) ۱, ۲, ۳, ۴, ۵, ۶, ۷, ۸, ۹
- (۲) ۷, ۱, ۲, ۸, ۶, ۴, ۹, ۵, ۳
- (۳) ۸, ۷, ۱, ۲, ۳, ۴, ۶, ۵, ۹
- (۴) ۸, ۷, ۱, ۲, ۵, ۳, ۶, ۴, ۹

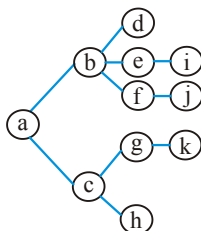
۶- اگر راس‌های گراف زیر را به صورت topological مرتب کنیم. چند دنباله متفاوت به دست می‌آید؟  
(تخصصی نرم‌افزار دولتی ۸۲)



۶ (۱)      ۱ (۲)

۲ (۳)      ۴۸ (۴)

۷- با اجرای یک پیمایش DFS بر روی یک گراف بدون جهت همبند مثل  $G$  درخت DFS بصورت شکل زیر بدست آمده است، با فرض اینکه  $a$  رأس مبدا پیمایش است و دانستن اینکه درجه  $a$  برابر ۴ است، کدام گزاره درست است؟  
(علوم کامپیوتر ۸۳)



(۱)  $G-a$  سه مولفه همبندی دارد.

(۲)  $G$  دارای دور نمی‌باشد.

(۳)  $G-a$  همبند است.

(۴)  $G-b$  نمی‌تواند همبند باشد.

۸- اگر دنباله درجه یک گراف بدون جهت ۲ و ۲ و ۳ و ۳ و ۳ باشد در آن صورت تعداد یال‌های پشتی (back edge) این گراف در جستجوی DFS برابر است با: (علوم کامپیوتر ۸۴)

۲ (۱)      ۳ (۲)      ۴ (۳)      ۵ (۴)

۹- الگوریتم تشخیص اینکه یک گراف جهت‌دار  $G(V,E)$  بدون دور است کدام پیچیدگی می‌باشد؟  
(علوم کامپیوتر ۸۷)

(۱)  $O(|V| \times |E|)$       (۲)  $O(|V|)$       (۳)  $O(|E|)$       (۴)  $O(|V| + |E|)$

۱۰- ماتریس مجاورت یک گراف جهت‌دار با مجموعه رأس‌های  $\{A, B, C, D, E, F\}$  به صورت زیر داده شده است:

|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | ۰ | ۱ | ۱ | ۱ | ۰ | ۰ |
| B | ۱ | ۰ | ۰ | ۰ | ۱ | ۰ |
| C | ۱ | ۰ | ۰ | ۱ | ۰ | ۰ |
| D | ۰ | ۰ | ۰ | ۰ | ۰ | ۰ |
| E | ۱ | ۰ | ۰ | ۱ | ۰ | ۰ |
| F | ۱ | ۰ | ۱ | ۱ | ۰ | ۰ |

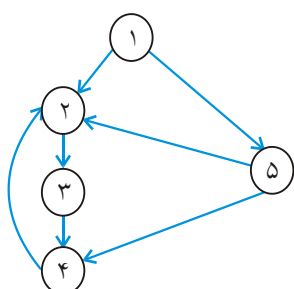
کدام یک از گزینه‌های زیر یک جزء قویاً هم‌بند این گراف است؟

- (۱)  $\{A, B, C, F\}$  (۲)  $\{A, B, C, E\}$   
(۳)  $\{A, B, C, D\}$  (۴)  $\{A, B, C, D, E\}$

۱۱- پیمایش DFS گراف جهت‌دار زیر به ترتیب از چپ به راست ۵، ۴، ۳، ۲، ۱ می‌باشد. یال

(علوم کامپیوتر ۸۸)

$< ۵, ۲ >$  چه یالی می‌باشد؟



(۱) Back edge

(۲) Cross edge

(۳) Tree edge

(۴) Forward edge

۱۲- کدام یک از الگوریتم‌ها در داخل خود الگوریتم DFS را دوبار صدا می‌کند؟ (علوم کامپیوتر ۸۸)

(۱) Topological sort (۲) پیدا کردن مؤلفه‌های مرتبط

(۳) پیدا کردن مؤلفه‌های قویاً مرتبط (۴) پیدا کردن دور در یک گراف

۱۳- گراف جهت‌دار با مجموعه گره‌های  $V = \{a, b, c, d, e\}$  و مجموعه یال‌های

$E = \{(a, c), (a, e), (c, d), (d, e), (e, b), (e, c)\}$  را در نظر بگیرید. اگر این گراف با روش

پیمایش عمق اول، پیمایش شود به ترتیب از چپ به راست، تعداد کمان‌های درخت

(tree-edges)، کمان‌های برگشتی (back-edges)، کمان‌های ضربدری (cross edges) و

کمان جلورو (forward edges) برابر کدام است؟ (IT ۸۸)

- (۱)  $(4, 0, 1, 1)$  (۲)  $(4, 1, 1, 0)$  (۳)  $(3, 2, 1, 0)$  (۴)  $(4, 1, 0, 1)$

۱۴- گراف بدون جهت  $G(V, E)$  مفروض است. می‌خواهیم مشخص کنیم که آیا گراف شامل

حلقه (سیکل) است یا نه. کوچکترین حد بالای زمان اجرای سریع‌ترین الگوریتم برای حل

مسئله کدام است؟ (IT ۸۸)

- (۱)  $O(|E|)$  (۲)  $O(|V| \cdot |E|)$  (۳)  $O(|V|)$  (۴)  $O(|V| + |E|)$

۱۵- گراف بدون جهت  $G$  با  $n$  رأس و  $e$  یال را «تک - دوره» می‌گوییم، اگر هم‌بند باشد و فقط

یک عدد دور داشته باشد. با چه سرعتی می‌توان درخت فراگیر کمینه‌ی یک گراف تک -

دوره با وزن‌های مثبت را به دست آورد؟ (تخصصی هوش مصنوعی ۸۸)

- (۱)  $O(n^2)$  (۲)  $O(e + n)$  (۳)  $O(e + n \lg n)$  (۴)  $O(e \times n)$

۱۶- مسئله‌ی پیدا کردن همه‌ی طولانی‌ترین مسیرها از یک رأس داده شده در یک گراف DAG

(جهت‌دار و بدون دور) را در چه زمانی می‌توان حل کرد؟ (تخصصی نرم‌افزار ۸۸)

$$O(|V| + |E|) \quad (۲)$$

$$O(|V| \parallel |E|) \quad (۱)$$

(۴) راه حل چند جمله‌ای ندارد.

$$O(|V| \log |E|) \quad (۳)$$

۱۷- اگر  $d[u]$  نخستین زمان ملاقات گره  $u$  در جستجوی عمق اول و  $f[v]$  آخرین زمان ملاقات

گره  $v$  و یال  $(u, v)$  یک Cross edge باشد، کدام رابطه درست است؟ (تخصصی نرم‌افزار ۸۸)

$$d[v] < f[v] < d[u] < f[u] \quad (۲)$$

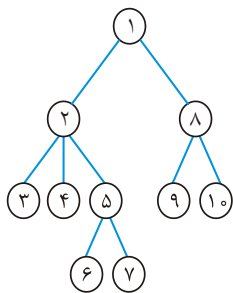
$$d[v] < d[u] < f[u] < f[v] \quad (۱)$$

$$d[v] < d[u] < f[v] < f[u] \quad (۴)$$

$$d[u] < f[u] < d[v] < f[v] \quad (۳)$$

۱۸- اگر درخت حاصل از جستجوی DFS یک گراف درخت زیر باشد یال  $a$  می‌تواند Forward

edge و یال  $b$  می‌تواند یک cross edge باشد. یال‌های  $a$  و  $b$  کدامند؟ (علوم کامپیوتر ۸۹)



$$a = (1, 4), b = (6, 7) \quad (۱)$$

$$a = (3, 4), b = (9, 10) \quad (۲)$$

$$a = (6, 1), b = (6, 2) \quad (۳)$$

$$a = (9, 10), b = (7, 1) \quad (۴)$$

۱۹- الگوریتم dfs به روی گراف بدون جهت  $G$  اجرا شده است. زمان ورود و خروج از هر رأس به

صورت مقابل است. کدام یال در گراف  $G$  وجود ندارد؟ (تخصصی هوش مصنوعی ۸۹)

|      | a  | b  | c  | d  | e | f  | g | h  | i  |
|------|----|----|----|----|---|----|---|----|----|
| ورود | ۱  | ۹  | ۸  | ۷  | ۵ | ۴  | ۲ | ۱۲ | ۱۳ |
| خروج | ۱۸ | ۱۰ | ۱۱ | ۱۶ | ۶ | ۱۷ | ۳ | ۱۵ | ۱۴ |

$$fh \quad (۱)$$

$$di \quad (۲)$$

$$bd \quad (۳)$$

$$ch \quad (۴)$$

۲۰- فرض کنید  $G$  یک گراف جهت‌دار باشد، کدام یک از شرایط زیر کافی است تا تضمین کند

در جستجوی DFS، برای هر دو رأس  $V$  و  $W$  در  $G$  اگر یک مسیر از  $V$  به  $W$  وجود داشته

باشد  $DFS(W)$  سریعتر از  $DFS(V)$  پایان می‌یابد؟ (علوم کامپیوتر ۸۵)

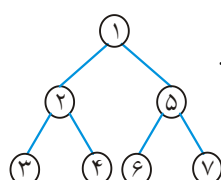
(۲) همبند باشد.

(۱) همبند نباشد.

(۳)  $G$  دارای یک Cycle جهت‌دار نباشد. (۴)  $G$  دارای هیچ Cycle جهت‌دار نباشد.

۲۱- اگر در پیمایش DFS در گراف غیرجهت دار  $G = (V, E)$  درخت فراگیر آن برابر با درخت

زیر باشد در آن صورت کدام گزینه صحیح است؟ در گراف  $G = (V, E)$ : (علوم کامپیوتر ۸۴)

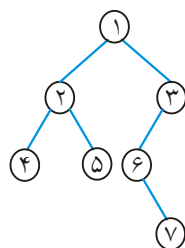


- (۱) بین دو رأس ۵ و ۲ می تواند یال وجود داشته باشد.
- (۲) بین دو رأس ۶ و ۲ و دو رأس ۷ و ۶ می تواند یال وجود داشته باشد.
- (۳) بین دو رأس ۱ و ۴ می تواند یال وجود داشته باشد.
- (۴) بین دو رأس ۷ و ۶ می تواند یال وجود داشته باشد.

۲۲- اگر در پیمایش BFS یک گراف غیرجهت دار  $G = (V, E)$  درخت فراگیر حاصل برابر با

درخت زیر باشد در آن صورت در گراف  $G = (V, E)$  کدام گزینه صحیح است؟

(علوم کامپیوتر ۸۴)



- (۱) بین دو رأس ۵ و ۴ می تواند یال وجود داشته باشد.
- (۲) بین دو رأس ۵ و ۱ می تواند یال وجود داشته باشد.
- (۳) بین دو رأس ۷ و ۳ و دو رأس ۴ و ۱ می تواند یال وجود داشته باشد.
- (۴) بین دو رأس ۷ و ۱ و دو رأس ۵ و ۱ می تواند یال وجود داشته باشد.

۲۳- فرض کنید  $G$  یک گراف غیرجهت دار بدون وزن باشد. کدام یک از روش های جستجو زیر

در گراف  $G$ ، گره های  $G$  را به صورتی ملاقات می کند که گره هایی که فاصله آنها از گره

شروع کمتر است زودتر ملاقات شوند؟ (علوم کامپیوتر ۸۵)

شروع کمتر است زودتر ملاقات شوند؟

(۱) روش BFS

(۲) روش DFS

(۳) روش های BFS و DFS

(۴) با پیمایش های BFS و DFS نمی توان نتیجه ای بدست آورد.

۲۴- کدام گزاره نادرست است؟ (علوم کامپیوتر ۸۶)

(۱) الگوریتم DFS از استک استفاده می کند.

(۲) الگوریتم DFS برای پیدا کردن topological sort استفاده می شود.

(۳) پیچیدگی زمان الگوریتم DFS در یک گراف سریع تر از BFS است.

(۴) در الگوریتم DFS یک گراف اگر Back edge وجود نداشته باشد گراف بدون دور است.

۲۵- اگر گراف  $G = (V, E)$  به صورت لیست مجاورت (Adjacency List) پیاده سازی شده

(ساختمان داده ها IT ۸۶)

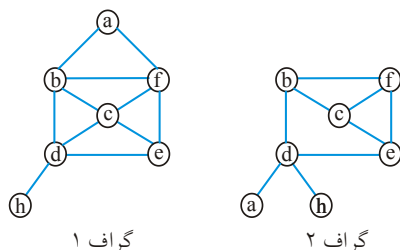
باشد، پیچیدگی پیمایش Breadth-First چیست؟

- (۱)  $O(|V|^2)$  (۲)  $O(|V| \cdot |E|)$  (۳)  $O(|V|^2 \cdot |E|)$  (۴)  $O(|V| + |E|)$

۲۶- دو گراف زیر و دو پیمایش DFS و BFS روبرو مفروض هستند:

DFS : cbfedha

BFS : cbfedah



(۱) این دو پیمایش برای هر دو گراف درست هستند.

(۲) این دو پیمایش برای هر دو گراف غلط است.

(۳) DFS داده شده برای هر دو درست است ولی

BFS فقط برای گراف ۱ درست است.

(۴) BFS برای هر دو درست است ولی DFS

فقط برای گراف ۲ درست است.

۲۷- در جستجوی BFS یک گراف جهت دار  $G = (V, E)$  در طبقه بندی یال ها در زمان پیمایش

(علوم کامپیوتر ۸۸)

کدام نوع از یال ها به هیچ وجه ایجاد نمی شود؟

(۱) back edge (۲) tree edge (۳) cross edge (۴) forward edge

۲۸- در صورتی که  $d[v]$  نخستین زمان ملاقات گره  $v$  در جستجوی سطح اول باشد، کدام

عبارت در مورد جستجوی سطح اول یک گراف جهت دار نادرست است؟

(تخصصی هوش مصنوعی ۸۷ و ۸۸)

(۱) برای هر Back edge  $(u, v)$  داریم،  $0 \leq d[v] \leq d[u]$

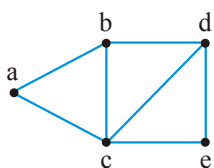
(۲) برای هر Cross edge  $(u, v)$  داریم،  $d[v] \leq d[u] + 1$

(۳) تعداد Tree Edge های حاصل کمتر از بقیه یال ها هستند.

(۴) هیچ forward edge وجود ندارد.

۲۹- کدام مورد نمی تواند ترتیب اجرای الگوریتم BFS روی گراف روبه رو باشد؟

(تخصصی هوش مصنوعی ۸۹)



(۱) cedab

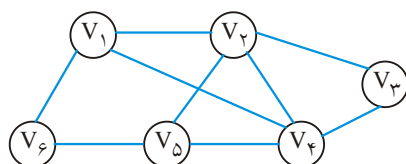
(۲) bcdae

(۳) abced

(۴) acbde

۳۰- در گراف زیر جستجوی BFS از رأس  $V_1$  منجر می شود به .....

(علوم کامپیوتر ۷۹)



(۱)  $V_1, V_2, V_3, V_4, V_5, V_6$

(۲)  $V_1, V_6, V_4, V_2, V_5, V_3$

(۳)  $V_1, V_6, V_5, V_4, V_2, V_3$

(۴)  $V_1, V_6, V_4, V_2, V_3, V_5$



۳۱- فرض کنید یک نوع الگوریتم پیمایش گراف بر روی دو گراف ساده همبند  $G_1$  و  $G_2$  اجرا

شده است و  $T_1 = a_1 \dots a_n$  و  $T_2 = b_1 \dots b_m$  نشاندهنده دنباله راسهای ملاقات شده به

ترتیب در  $G_1$  و  $G_2$  هستند کدام گزاره نادرست است؟ (علوم کامپیوتر ۸۳)

(۱) اگر  $T_1$  و  $T_2$  از پیمایش BFS بدست آمده باشند، آنگاه  $T = T_1 T_2$  نشاندهنده یک

پیمایش BFS در گرافی است که از اتصال راسهای  $a_n$  و  $b_1$  بدست آمده است.

(۲) اگر  $T_1$  و  $T_2$  از پیمایش BFS بدست آمده باشند، آنگاه  $T = T_1 T_2$  نشاندهنده یک

پیمایش BFS در گرافی است که از اتصال راسهای  $a_1$  و  $b_1$  بدست آمده است.

(۳) اگر  $T_1$  و  $T_2$  از پیمایش DFS بدست آمده باشند، آنگاه  $T = T_1 T_2$  نشاندهنده یک

پیمایش DFS در گرافی است که از اتصال راسهای  $a_1$  و  $b_1$  بدست آمده است.

(۴) اگر  $T_1$  و  $T_2$  از پیمایش DFS بدست آمده باشند، آنگاه  $T = T_1 T_2$  نشاندهنده یک

پیمایش DFS در گرافی است که از اتصال راسهای  $a_n$  و  $b_1$  بدست آمده است.

۳۲- می‌خواهیم قطر یک درخت آزاد  $G = (V, E)$  (گراف بدون جهت و غیرسیکلی و بدون

وزن) را پیدا کنیم. قطر بیشترین فاصله دو گره در گراف است. این کار در بهترین حالت با

چه مرتبه‌ای می‌تواند انجام شود. (تخصصی نرم‌افزار دولتی ۸۶)

$$(۱) O(|V| + |E|) \quad (۲) O(|V|)$$

$$(۳) O(|V| \lg |V|) \quad (۴) O(|V|^3)$$

۳۳- پیچیدگی زمان پیدا کردن قطر یک گراف  $G(V, E)$  برابر است با: (علوم کامپیوتر ۸۷)

$$(۱) O(|V|^2) \quad (۲) O(|V|^2 + |V| \cdot |E|)$$

$$(۳) O(|V|^2 \cdot |E|) \quad (۴) O(|V| + |E|)$$

۳۴- چه تعداد از گزاره‌های زیر درست هستند؟ (IT ۹۱)

■ با هزینه  $O(|V| + |E|)$  می‌توان وجود یا عدم وجود دور اویلری در یک گراف

$G = (V, E)$  را تشخیص داد.

■ یک گراف دور اویلری دارد اگر و تنها اگر برای هر رأس  $v$  رابطه‌ی

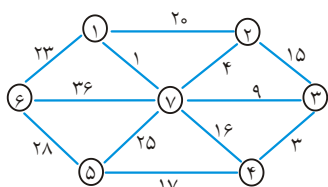
$\text{indegree}(v) = \text{outdegree}(v)$  برقرار باشد.

■ درخت فراگیر کمینه برای یک گراف وزن دار با وزن‌های متمایز یک‌تاست. اما عکس آن درست

نیست.

$$(۱) ۰ \quad (۲) ۱ \quad (۳) ۲ \quad (۴) ۳$$

۳۵- هزینه درخت پوشای مینیمم (Minimum Spanning Tree) گراف زیر چیست؟ (آزاد ۷۷)



۶۸ (۱)

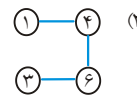
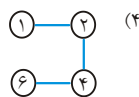
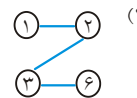
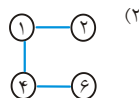
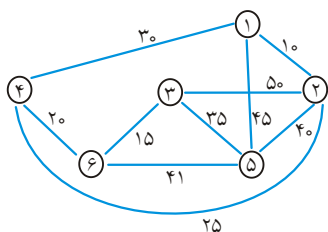
۴۱ (۲)

۸۱ (۳)

۵۷ (۴)

۳۶- اگر برای پیدا کردن درخت پوشای مینیمم زیر از الگوریتم Prim استفاده کنیم کدامیک از گزینه‌های زیر درخت حاصله در انتهای مرحله سوم این الگوریتم را به ما می‌دهد؟

(آزاد ۷۸ و آزاد ۷۹)



۳۷- برای یافتن درخت پوشای حداقل یک گراف خلوت کدامیک از الگوریتم‌های زیر مناسب‌تر

(تخصصی نرم افزار آزاد ۸۰)

است؟

Dijkstra (۴)

kruskal (۳)

prim (۲)

Floyd (۱)

۳۸- الگوریتم ذیل درخت پوشا با کمترین هزینه را بدست می‌آورد. منطق الگوریتم کدامیک از

(تخصصی هوش مصنوعی آزاد ۸۱)

گزینه‌های زیر است؟

$T \leftarrow \emptyset$ ;

While T contains less than  $n-1$  edge and E not empty do

begin

choose an edge  $(v, w)$  from E of Lowest cost;

delete  $(v, w)$  from E;

if  $(v, w)$  does not create a cycle in T

then add  $(v, w)$  to T

else discard  $(v, w)$

end;

if T contains fewer than  $n-1$  edge then print ("no Spanning tree");

سولین (۴)

وارشال (۳)

کروسکال (۲)

پریم (۱)

۳۹- گراف همبندی بدون جهت با  $n$  گره و  $n-2$  لبه داریم. کدام یک از الگوریتم‌های زیر برای

(تخصصی نرم افزار آزاد ۸۲)

تولید درخت پوشا با حداقل هزینه بر روی گراف مناسب‌تر است؟

هیچکدام (۴)

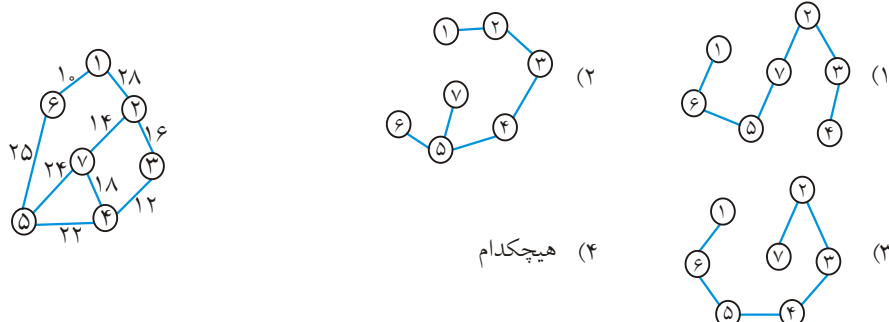
Sollin (۳)

kruskul (۲)

prim (۱)

(آزاد ۸۴)

۴۰- درخت پوشای با حداقل هزینه گراف زیر کدام است؟



۴۱- مرتبه زمانی الگوریتم‌های kruskal و prim برای یافتن درخت پوشای کمینه یک گراف با

(مهندسی کامپیوتر آزاد ۸۵)

n رأس و e یال به ترتیب از راست به چپ کدام است؟

(۱)  $O(n^2)$ ,  $O(e \log e)$  (۲)  $O(e \log e)$  و  $O(n^2)$ (۳)  $O(n^2)$  و  $O(n^2)$  (۴)  $O(e \log e)$  و  $O(n^2)$ 

۴۲- یک گراف بسیار پراکنده شامل n گره مفروض است. می‌خواهیم با استفاده از الگوریتم

kruskal کوچکترین درخت پوشای این گراف را به دست می‌آوریم. در این صورت

(تخصصی نرم افزار آزاد ۸۶)

پیچیدگی زمانی این الگوریتم برابر است با:

(۱)  $O(n^2)$  (۲)  $O(n^2 \log n)$  (۳)  $O(n)$  (۴)  $O(n \log n)$ 

۴۳- یک گراف بسیار متصل شامل n گره مفروض است. هرگاه بخواهیم با استفاده از الگوریتم

kruskal کوچکترین درخت پوشای این گراف را به دست می‌آوریم. در این صورت

(تخصصی هوش مصنوعی آزاد ۸۶)

پیچیدگی زمانی این الگوریتم برابر است با:

(۱)  $O(n^2 \log n)$  (۲)  $O(n \log n)$  (۳)  $O(n^2)$  (۴)  $O\left(\frac{n^2}{2}\right)$ 

۴۴- گراف وزن‌دار زیر را در نظر می‌گیریم، مجموع وزن‌های درخت مینیمال آن چقدر است؟

(ساختار گسسته دولتی ۸۱)



(۱) ۲۲

(۲) ۲۴

(۳) ۲۵

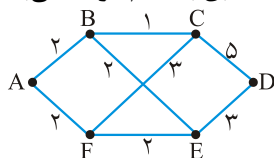
(۴) ۲۰

۴۵- گرافی با ۱۰ رأس در نظر می‌گیریم که گرافی کامل می‌باشد و رأس‌های آن از یک تا ده شماره‌گذاری شده‌اند. اگر وزن هر یال، از تابع زیر به دست آید، مجموع وزن‌های یال‌های درخت فراگیر مینیمال این گراف کدام است؟ (ساختمان گسسته دولتی ۸۲)

$$w_{ji} = w_{ij} = \begin{cases} i+j & i+j \geq 5 \\ i^2 + j^2 & i+j < 5 \end{cases}$$

|        |        |
|--------|--------|
| ۷۵ (۲) | ۷۶ (۱) |
| ۶۵ (۴) | ۶۶ (۳) |

۴۶- گراف بدون جهت و وزن‌دار روبرو را در نظر می‌گیریم. اگر نقطه A را به عنوان مبدا انتخاب کنیم و الگوریتم‌های کروسکال و پریم را جداگانه بر روی این گراف اجرا کنیم، لبه (یال)‌های انتخاب شده به ترتیب مطابق با کدام گزینه می‌باشد؟ (تخصصی نرم افزار دولتی ۸۲)



- (۱) ED و AF و FE و BE و BC: کروسکال (۲) ED و FE و AF و BA و BC: کروسکال  
 ED و BC و BE و FE و AF: پریم ED و BC و EB و FE و AF: پریم  
 (۳) ED و AB و AF و FE و BC: کروسکال (۴) FE و ED و AB و BE و BC: کروسکال  
 ED و BC و FE و AF و AB: پریم ED و AB و BC و FE و AF: پریم

۴۷- کدام گزینه در مورد الگوریتم‌های کروسکال و پریم برای ایجاد درخت پوشای کمینه درست است؟ (تخصصی هوش مصنوعی دولتی ۸۲)

- (۱) زمان اجرای هر دو الگوریتم روی گراف‌های یکسان، مساوی است.
- (۲) هر دو الگوریتم روی گراف‌های یکسان، درخت پوشای یکسان تولید می‌کنند.
- (۳) مجموع طول اضلاع درخت پوشا در هر دو الگوریتم یکسان است.
- (۴) هر دو الگوریتم با رشد و بهم پیوستن یک جنگل از درخت‌ها، درخت پوشا را تولید می‌کند.

۴۸- در یک گراف همبند بدون جهت و وزن دار کدام یک از گزینه‌های زیر در مورد درخت پوشای کمینه ایجاد شده توسط الگوریتم‌های prim و kruskal درست است؟ (تخصصی هوش مصنوعی دولتی ۸۴)

- (۱) با هر وزنی این دو الگوریتم درخت‌های یکسانی تولید می‌کند.
- (۲) درخت تولید شده توسط الگوریتم prim مستقل از رأس شروع در آن الگوریتم است.
- (۳) اگر وزن یال‌ها مجزا باشند، درخت‌های تولیدی این دو الگوریتم یکسان هستند.
- (۴) اگر وزن همه یال‌ها برابر باشند، این دو الگوریتم درخت‌های یکسانی را تولید می‌کنند.

۴۹- فرض کنیم که  $T$  درخت پوشای مینیمم برای گراف  $G$  باشد. اگر مقادیر برخی از یال‌های  $G$

کاهش پیدا کنند و گراف حاصل را  $G'$  بنامیم در این صورت: (تخصصی هوش مصنوعی دولتی ۸۵)

(۱)  $T$  در هر حال یک درخت پوشای مینیمم از  $G'$  خواهد بود.

(۲) اگر یال‌هایی که مقدارشان کاهش پیدا کرده همگی متعلق به  $T$  باشند آنگاه  $T$  یک

درخت پوشای مینیمم برای  $G'$  خواهد بود.

(۳) حتی اگر فقط برخی از یال‌هایی که مقدارشان کاهش یافته متعلق به  $T$  باشند،  $T$  یک

درخت پوشای مینیمم از  $G'$  خواهد بود.

(۴)  $T$  یک درخت پوشای مینیمم از  $G'$  خواهد بود اگر یال‌هایی که مقدارشان کاهش پیدا

کرده همگی متعلق به  $T$  باشند و نیز کلیه یال‌های متعلق به  $T$  کاهش یافته باشند.

۵۰-  $T$  درخت فراگیر کمینه برای یک گراف  $G = (V, E)$  است. وزن یک یال  $e \in E$  را کاهش

می‌دهیم. با چه مرتبه‌ای می‌توان درخت فراگیر کمینه گراف جدید را بدست آورد؟

(تخصصی نرم افزار دولتی ۸۵)

$$O(|V| + |E|) \quad (۲) \quad O(|V|) \quad (۱)$$

$$O(|V| \log |E|) \quad (۴) \quad O(|E| \log |V|) \quad (۳)$$

۵۱- یک گراف  $G = (V, E)$  با یک یال  $e = (u, v)$  را در نظر بگیرید. می‌خواهیم درخت فراگیر

کمینه (MST) برای این گراف پیدا کنید که حتماً شامل  $e$  باشد. کدام یک از راه‌های زیر

همیشه درست است؟

(تخصصی هوش مصنوعی دولتی ۸۶)

(۱)  $e$  را حذف می‌کنیم (بدون حذف  $u$  و  $v$ )، MST گراف حاصل را بدست می‌آوریم و سپس

$e$  را اضافه می‌کنیم.

(۲)  $u$  و  $v$  را در هم ادغام می‌کنیم و MST گراف حاصل را به دست می‌آوریم. سپس  $e$  را

اضافه می‌کنیم.

(۳) همه یال‌های متصل به  $u$  و  $v$  را حذف می‌کنیم. MST گراف حاصل را به دست می‌آوریم

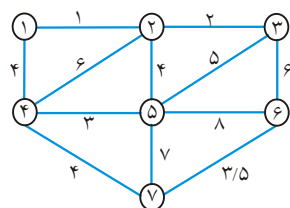
و سپس  $e$  را اضافه می‌کنیم.

(۴) این مسئله راه حل چند جمله ای ندارد.

۵۲- الگوریتم kruskal در مرحله چهارم خود، کدام کمان از گراف را به عنوان کمان درخت

(IT ۸۳)

پوشای با حداقل هزینه در نظر می‌گیرد؟



(۱ و ۴) (۱)

(۴ و ۵) (۲)

(۶ و ۷) (۳)

(۴) هیچکدام

۵۳- کدام یک از گزاره‌های ذیل، در مورد محاسبه درخت پوشای کمین برای یک گراف هم بند (Connected)، درست است؟ (IT ۸۴)

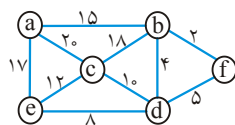
(۱) در روش prim در هر مرحله، هم بندی درخت رعایت می‌شود و زمان مصرفی الگوریتم prim و الگوریتم kruskal یکسان است.

(۲) در روش prim در هر مرحله، هم بندی درخت، رعایت می‌شود، اما در روش kruskal مرحله پایانی، این امر اتفاق می‌افتد. اما زمان مصرفی روش prim، بهتر است.

(۳) در روش kruskal، در هر مرحله، هم بندی درخت رعایت می‌شود، اما در روش prim مرحله پایانی، این امر اتفاق می‌افتد، اما زمان مصرفی روش prim، بهتر است.

(۴) در روش kruskal، در هر مرحله، هم بندی درخت رعایت نمی‌شود، اما در پایان الگوریتم، درخت هم بند است، اما زمان مصرفی آن از روش prim بهتر است.

۵۴- ترتیب انتخاب یال‌ها در الگوریتم kruskal بر روی گراف زیر کدام است؟ (از چپ به راست) (IT ۸۵)



(۱) (b, f) (b, d) (e, d) (c, d) (a, b)

(۲) (a, b) (b, f) (b, d) (f, d) (c, d)

(۳) (b, f) (b, d) (e, d) (e, c) (a, e)

(۴) هیچکدام

۵۵- ترتیب انتخاب یال‌ها در الگوریتم prim بر روی گراف تست قبل با شروع از a کدام مورد است (از چپ به راست)؟

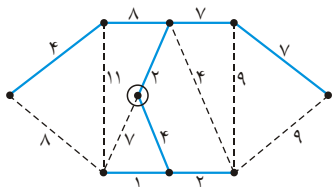
(۱) (b, f) (b, d) (e, d) (c, d) (a, b)

(۲) (a, b) (b, f) (f, d) (d, e) (e, c)

(۳) (a, b) (b, f) (b, d) (d, e) (d, c)

(۴) هیچکدام

۵۶- در گراف روبه رو لبه‌های پر رنگ، درخت پوشای حداقل را با استفاده از روش prim نمایش می‌دهد. گره‌ای که با دایره مشخص شده نقطه شروع است. در این حالت آخرین گره‌ای که به درخت پوشا متصل شده است دارای چه وزنی است؟ (IT ۸۶)



(۱) ۸

(۲) ۷

(۳) ۴

(۴) ۲

۵۷- فرض کنید گراف  $G = (V, E)$  یک گراف بی جهت و  $M$  یک درخت فراگیر مینیمم برای  $G$

(علوم کامپیوتر ۸۲)

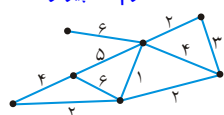
باشد، کدام عبارت درست است؟

- (۱) مسیر  $M$  بین هر جفت رأس  $V_1$  و  $V_2$  کوتاهترین مسیر است.
- (۲) مسیر  $M$  بین هر جفت رأس  $V_1$  و  $V_2$  کوتاهترین مسیر نمی باشد.
- (۳) تمام مسیرهای موجود در  $M$  کوتاهترین مسیر می باشند.
- (۴) بین تمام رئوس در  $M$  مسیری وجود ندارد.

۵۸- وزن مینیمم درخت فراگیر را در گراف زیر بیابید به طوری که درجه هر رأس از درخت

(ساختمان گسسته علوم کامپیوتر ۸۳)

بیشتر از ۲ نباشد؟



- |     |    |
|-----|----|
| (۱) | ۱۶ |
| (۲) | ۱۷ |
| (۳) | ۱۹ |
| (۴) | ۲۱ |

۵۹- در گراف وزن دار  $G = (V, E)$  که  $w_e$  وزن یال  $e$  می باشد اگر درخت فراگیر مینیمم این

گراف  $\sum_{e \in T} w_e$  را مینیمم می کند در آن صورت الزاماً  $\sum_{e \in T} w_e^2$  نیز مینیمم می گردد و

(علوم کامپیوتر ۸۶)

بلعکس. گزاره بالا صحیح است یا نادرست؟

(۱) خیر

(۲) بله

(۳) فقط برای گراف های بدون دور صحیح است.

(۴) فقط در صورتی که  $w_e > 0$  به ازای هر یال باشد صحیح است.

۶۰- درخت پوشای مینیمم برای گراف وزن دار  $G$ ، در چه صورت یکتا می باشد؟ (علوم کامپیوتر ۸۷)

(۱) در صورتی که گراف  $G$  یک درخت باشد.

(۲) در صورتی که در گراف  $G$  هیچ دو یالی دارای وزن یکسان نباشند.

(۳) در صورتی که گراف  $G$  غیر جهتدار و مرتبط باشد.

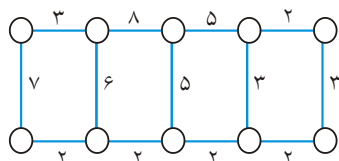
(۴) موارد ۱ و ۲

۶۱- یک شرکت در نظر دارد بین ۱۰ اتاق شرکت یک شبکه LAN ایجاد نماید. گراف روبه رو

اتاق هایی که امکان سیم پیچی مستقیم بین آنها وجود دارد را نمایش می دهد. وزن هر لبه هزینه سیم کشی بین آن دو اتاق است. اگر هدف انجام سیم کشی با حداقل هزینه باشد،

(۸۷ IT)

کدام الگوریتم زیر برای حل این مسئله مناسب است؟



(۱) Prim

(۲) Floyd

(۳) Kruskal

(۴) Dijkstra

۶۲- الگوریتم زیر را بر روی یک گراف همبند بدون جهت و وزن دار  $G = (V, E)$  در نظر بگیرید:

تا وقتی که گراف  $G$  دوری به نام  $C$  دارد این کار را تکرار کن

یال  $e$  با بیشترین وزن در  $C$  را به دست آور و آن را از  $G$  حذف کن (نقصی نرم افزار ۸۷)

(۱) این الگوریتم ممکن است ختم نشود.

(۲) گراف حاصل ممکن است همبند نباشد.

(۳) در انتها گراف حاصل یک درخت فراگیر کمینه برای گراف اولیه است.

(۴) در انتها گراف حاصل درخت فراگیر برای  $G$  اولیه است ولی لزوماً کمینه نیست.

۶۳- پیچیدگی زمان  $O(|V| + |E|)$  برای یک گراف  $G(V, E)$  به ازای کدام الگوریتم صادق

نمی باشد؟

(علوم کامپیوتر ۸۸)

(۱) پیدا کردن مؤلفه های قویاً مرتبط (۲) پیدا کردن درخت پوشا

(۳) Topological sort (۴) پیدا کردن درخت پوشا کمینه

۶۴- فرض کنید در گراف وزن دار  $G = (V, E)$  یال  $e \in E$  سنگین ترین یال در یک دور موجود

در گراف باشد، در این صورت آیا درخت فراگیر کمینه شامل این یال می باشد یا خیر؟

(علوم کامپیوتر ۸۸)

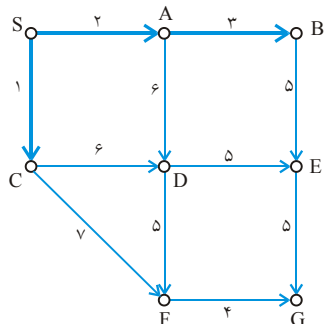
(۱) خیر - هیچ وقت (۲) بله - همیشه

(۳) در موارد خاص بله (۴) مشخص نمی باشد.

۶۵- فرض کنید که الگوریتم Prim روی گراف شکل زیر اجرا شده است. یال های پر رنگ

(۸۸ IT)

یال هایی است که تا به حال پردازش شده اند. یال بعدی کدام یال است؟



(۱) یال FG

(۲) یال CF

(۳) یال CD

(۴) یال BE

(تخصصی هوش مصنوعی ۸۸)

۶۶- چند تا از عبارت ها صحیح هستند؟

- BFS از پشته استفاده می کند.

- درخت پوشای بیشینه را می توان در  $O(E \log V)$  ساخت.

- یک درخت پوشای کمینه ممکن است شامل یال با بیشترین وزن باشد.

- درخت عمق اول و سطح اول یک گراف مانند هم اند.

(۱) صفر (۲) ۱ (۳) ۲ (۴) ۳



۶۷- «درخت فراگیر گلوگاه» (د.ف. گلوگاه) T در یک گراف بدون جهت و وزن دار G یک درخت فراگیر (د.ف) است که وزن سنگین ترین یال آن نسبت به هر درخت فراگیر (نه لزوماً کمینه‌ی) دیگر کمینه باشد. می‌گوییم که «مقدار» یک درخت فراگیر گلوگاه، وزن سنگین ترین یال آن درخت است. می‌دانیم که درخت فراگیر کمینه (د.ف. کمینه) و درخت فراگیر بیشینه (د.ف. بیشینه) به ترتیب درخت فراگیر با کم‌ترین و بیش‌ترین وزن (مجموع وزن یال‌های آن) ممکن هستند. کدام یک از گزینه‌های زیر درست است؟ (تخصصی نرم‌افزار ۸۹)

- ۱) هر د.ف. گلوگاه یک د.ف. کمینه است. ۲) هر د.ف. کمینه یک د.ف. گلوگاه هم هست.
- ۳) هر د.ف. بیشینه یک د.ف. گلوگاه است. ۴) هر د.ف. گلوگاه یک د.ف. بیشینه است.

۶۸- فرض کنید G یک گراف بدون جهت باشد که هیچ دو یال آن دارای وزن یکسان نیستند کدام گزینه در مورد گراف G صحیح است؟ (دومین زیردرخت فراگیر مینیمم G، یکی از زیردرخت‌های فراگیر G است که فقط وزن زیردرخت فراگیر مینیمم از وزن آن کمتر باشد). (تخصصی نرم‌افزار ۸۹)

- ۱) زیردرخت فراگیر مینیمم و دومین زیردرخت فراگیر مینیمم، هیچ یک لزوماً یکتا نیستند.
- ۲) زیردرخت فراگیر مینیمم و دومین زیردرخت فراگیر مینیمم هر دو یکتا هستند.
- ۳) زیردرخت فراگیر مینیمم لزوماً یکتا نیست، اما دومین زیردرخت فراگیر مینیمم یکتا است.
- ۴) زیردرخت فراگیر مینیمم یکتا است، اما دومین زیردرخت فراگیر مینیمم لزوماً یکتا نیست.

۶۹- چند تا از گزاره‌های زیر درست هستند؟

- I. اگر در یک گراف وزن دار بدون جهت، یال  $e = (u, v)$  در درخت فراگیر کمینه باشد، حتماً دو رأس وجود دارد که کوتاه‌ترین مسیر بین آن دو از e می‌گذرد.
- II. مسئله‌ی یافتن همه‌ی کوتاه‌ترین مسیرها از یک رأس به بقیه‌ی رأس‌ها را در یک گراف وزن دار بدون جهت و هم‌بند با مجموعه‌ی یال‌های E را می‌توان در  $O(|E| + |V|)$  حل کرد.

III. در یک گراف وزن دار و ساده بدون جهت با وزن‌های نامساوی، یال با سبک‌ترین وزن و یال دومین سبک‌ترین وزن هر دو در درخت فراگیر کمینه هستند. (نرم‌افزار ۹۰)

- ۱) ۰ ۲) ۱ ۳) ۲ ۴) ۳

۷۰- در یک گراف با وزن‌های صحیح بزرگ‌تر از ۱، فرض کنید که وزن هر یال را یک واحد زیاد کنیم. در آن صورت چند تا از گزاره‌های زیر درست‌اند؟ (هوش مصنوعی ۹۰)

- I. برش کمینه‌ی  $(S, T)$  در هر دو گراف یکی است.
  - II. درخت فراگیر کمینه‌ی هر دو گراف یکی است.
  - III. کوتاه‌ترین مسیر بین دو رأس مشخص در دو گراف شامل یال‌های یکسان هستند.
- ۱) ۱ ۲) ۰ ۳) ۲ ۴) ۳

۷۱- «قدرت» یک درخت فراگیر در یک گراف  $G$  برابر وزن سنگین‌ترین یال در آن درخت است. می‌خواهیم در گراف داده‌شده‌ی  $G$ ، از میان تمام درخت‌های فراگیر، آن را انتخاب کنیم که قدرتش از همه کم‌تر باشد. خروجی کدام یک از الگوریتم‌های پریم یا کروسکال همواره کم‌قدرت‌ترین درخت فراگیر است؟

(نرم‌افزار ۹۱)

- (۱) پریم  
(۲) کروسکال  
(۳) هر دو  
(۴) هیچ‌کدام

۷۲- در یک گراف جهت‌دار  $G = (V, E)$  وزن هر کدام از یال‌ها یک عدد صحیح، مثبت و حداکثر  $C$  است؛ که  $C$  یک عدد ثابت است. می‌خواهیم طول کوتاه‌ترین مسیرها از یک رأس به نام  $S$  را تا بقیه‌ی رأس‌های  $G$  به دست آوریم. کدام یک از گزینه‌های زیر مرتبه‌ی یک الگوریتم کارا برای حل این مسئله است؟

(هوش مصنوعی ۹۱)

- (۱)  $O(|E| \lg |V|)$   
(۲)  $O(|V| \lg |V| + |E|)$   
(۳)  $O(|V| + |E|)$   
(۴)  $O((|V| + |E|) \lg |E|)$

۷۳- دورترین رأس از یک رأس داده‌شده‌ی  $v$  در یک گراف بدون وزن، رأسی است که کوتاه‌ترین فاصله‌ی آن تا  $v$  بیش‌ترین باشد. کدام یک از روش‌های زیر برای یافتن این دورترین رأس مناسب‌تر و سریع‌تر است؟

(هوش مصنوعی ۹۱)

- (۱) دایکسترا  
(۲) مرتب‌سازی توپولوژیکی  
(۳) DFS  
(۴) BFS

۷۴- گراف بدون جهت و وزن‌دار  $G = (V, E)$  با وزن‌های مثبت و منفی داده‌شده است. فرض کنید که  $w_{\min}$  و  $w_{\max}$  به ترتیب وزن‌های سبک‌ترین و سنگین‌ترین یال‌های این گراف است.

(نرم‌افزار ۹۲)

چند تا از گزاره‌های زیر در مورد درخت فراگیر کمینه‌ی  $G$  (MST) درست هستند؟

■ اگر  $G$  بیش‌تر از  $|V| - ۱$  یال داشته باشد، و تنها یک یال، با وزن  $w_{\max}$  وجود داشته باشد. در آن صورت نمی‌تواند در هیچ MST از  $G$  باشد.

■ یالی با وزن  $w_{\min}$  حتماً در MST هست.

■ اگر  $G$  یک دور (cycle) داشته باشد که این دور حاوی تنها یک یال با وزن  $w_{\min}$  است، حتماً یالی از هر MST است.

- (۱) ۰ (۲) ۱ (۳) ۲ (۴) ۳

۷۵- درستی دو گزاره‌ی زیر را مشخص کنید. (هوش مصنوعی ۹۲)

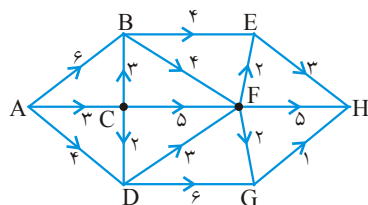
الف) در گراف بدون جهت  $G = (V, E)$  با وزن‌های مثبت و منفی (ولی بدون دور منفی)، یال  $x = (u, v) \in E$  در درخت فراگیر کمینه است اگر و فقط اگر دو رأس  $x, y \in V$  باشند که کوتاه‌ترین مسیر از  $x$  به  $y$  در  $G$  از یال  $e$  عبور کند.

ب) اگر وزن‌های یال‌های گراف نامساوی باشند، درخت فراگیر کمینه یکتا (unique) است.

- (۱) الف: نادرست، ب: نادرست  
(۲) الف: درست، ب: نادرست  
(۳) الف: نادرست، ب: درست  
(۴) الف: درست، ب: درست

۷۶- گراف جهت‌دار (digraph) زیر را در نظر بگیرید.

(آزاد ۷۷)

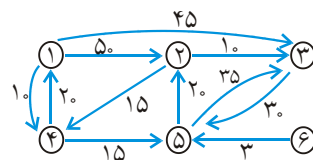


کوتاه‌ترین مسیر بین A و H دارای چه فاصله‌ای است؟

- (۱) ۹  
(۲) ۱۱  
(۳) ۸  
(۴) ۱۰

۷۷- گراف زیر را در نظر بگیرید. کوتاه‌ترین مسیر از گره ۱ به گره‌های ۲ و ۳ به ترتیب چند

(تخصصی هوش مصنوعی آزاد ۸۱)



- (۱) ۴۵ و ۶۰  
(۲) ۵۰ و ۶۰  
(۳) ۴۵ و ۴۵  
(۴) ۴۵ و ۳۵

۷۸- در گراف تست قبل کوتاه‌ترین مسیر از گره ۱ به سایر گره‌ها با استفاده از الگوریتم کوتاه

(تخصصی نرم افزار آزاد ۸۶)

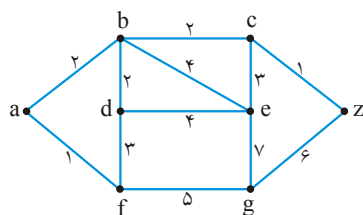
ترین مسیر چیست؟

- (۱)  $[45, 45, 10, 25, \infty]$   
(۲)  $[50, 45, 10, 25, \infty]$   
(۳)  $[50, 60, 10, 25, \infty]$   
(۴)  $[45, 60, 10, 25, \infty]$

۷۹- در گراف زیر کوتاه ترین مسیر بین رئوس a و z برطبق الگوریتم Dijkstra دارای چه وزنی

(مهندسی فناوری اطلاعات آزاد ۸۵)

است؟



- (۱) ۵  
(۲) ۳  
(۳) ۱۲  
(۴) ۱۸

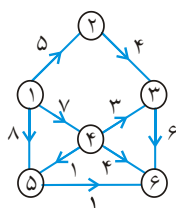
۸۰- کدام عبارت در مورد الگوریتم دایکسترا بر روی گراف وزن دار  $G = (V, E)$  نادرست

است؟ (گراف  $G = (V, E)$  وزن دار است) (دولتی ۸۱ و تخصصی هوش مصنوعی دولتی ۸۳)

- (۱) پیچیدگی الگوریتم می تواند  $O(|E| \log |V|)$  باشد.
- (۲) اگر وزن هیچ یالی منفی نباشد، الگوریتم درست کار می کند.
- (۳) تنها اگر گراف دارای دور با وزن منفی نباشد الگوریتم درست کار می کند.
- (۴) هم برای گراف های بدون جهت درست کار می کند و هم برای گراف های جهت دار.

۸۱- ترتیب رئوس حاصل از اجرای الگوریتم دایکسترا (Dijkstra) برای تعیین کوتاهترین مسیر

در گراف جهت دار به قرار ..... است. (علوم کامپیوتر ۷۹)



- (۱) ۱ و ۵ و ۳ و ۴ و ۲ و ۶
- (۲) ۱ و ۵ و ۳ و ۶ و ۴ و ۲
- (۳) ۱ و ۵ و ۴ و ۶ و ۳ و ۲
- (۴) ۱ و ۳ و ۶ و ۵ و ۴ و ۲

۸۲- در گراف  $G = (V, E)$  اگر وزن بعضی از یال ها منفی باشد برای پیدا کردن کوتاه ترین

مسیر بین دو رأس  $s$  و  $t$  به تمام وزن یال ها مقدار ثابت  $c$  اضافه کنیم به طوری که وزن تمام

یال های منفی مثبت شود در آن صورت الگوریتم Dijkstra ..... (علوم کامپیوتر ۸۶)

- (۱) کوتاه ترین مسیر بین  $s$  و  $t$  را محاسبه می کند.
- (۲) کوتاه ترین مسیر بین  $s$  و  $t$  را محاسبه نمی کند.
- (۳) کوتاه ترین مسیر بین  $s$  و  $t$  را محاسبه می کند فقط در صورتی که گراف مذکور  $G$  دارای دور منفی باشد.
- (۴) فقط در صورتی که گراف مذکور  $G$  دور منفی نداشته باشد کوتاه ترین مسیر بین  $s$  و  $t$  حاصل می شود.

۸۳- شما باید الگوریتمی طراحی کنید که «مسیر ساده با کمترین تعداد یال» را بین دو رأس

مفروض در یک گراف بیابد. شما الگوریتم خود را براساس کدام یک از الگوریتم های زیر

ارائه می کنید؟ (IT ۸۶)

- |                          |                          |
|--------------------------|--------------------------|
| (۱) الگوریتم kruskal     | (۲) الگوریتم Dijkstra    |
| (۳) جستجوی عمق اول (DFS) | (۴) جستجوی سطح اول (BFS) |

۸۴- ماتریس  $B$  از روی گراف  $G = (V, E)$  به صورت زیر بدست آمده است:

$$b_{ij} = \begin{cases} -1 & \text{اگر یال } j, \text{ گره } i \text{ را ترک کند.} \\ 1 & \text{اگر یال } j, \text{ به گره } i \text{ وارد شود.} \\ 0 & \text{در غیر این صورت} \end{cases}$$

ماتریس  $C = B \cdot t_B$  (  $t_B$  ، ترانهاده B است) چه را نشان می‌دهد؟ می‌توان:

(تخصصی نرم‌افزار دولتی ۸۶)

(۱) با مقدار  $C_{ij}$  تعداد دورهای بین گره‌های  $V_i$  و  $V_j$  را تعیین کرد.

(۲) با مقدار  $C_{ij}$  تعداد یالهای از  $V_j$  به  $V_i$  را تعیین کرد.

(۳) با مقدار  $C_{ij}$  تعداد یالهای از  $V_i$  به  $V_j$  را تعیین کرد.

(۴) با مقدار  $C_{ij}$  تعداد یالهای بین گره  $i$  و  $j$  را تعیین کرد.

۸۵- در یک گراف  $N$  گره با شماره‌های ۱ تا  $N$  موجود است از هر گره  $i$  به تمامی گره‌هایی که

شماره آن از  $i$  بزرگتر است یک یال وجود دارد تعداد مسیرهای ممکن از گره ۱ به  $N$  را

محاسبه کنید.

(تخصصی هوش مصنوعی دولتی ۸۶)

$$(1) N-1 \quad (2) N \quad (3) 2^{N-2} \quad (4) \sum_{i=1}^N i$$

۸۶- کدام یک از گزینه‌های زیر غلط است؟

(تخصصی هوش مصنوعی دولتی ۸۵)

(۱) وجود یک یال Cross در پیمایش عمق اول معرف عدم وجود سیکل هامیلتونی است.

(۲) وجود یک یال Cross در پیمایش عمق اول نشانگر وجود گره مفصلی در گراف است.

(۳) وجود یک یال Cross در پیمایش عمق اول را نمی‌توان دلالت بر عدم وجود مسیر هامیلتونی دانست.

(۴) وجود یک یال Cross در پیمایش عمق اول معرف عدم وجود مسیر اولری در گراف است.

۸۷- به دست آوردن تعداد سیکل‌های ساده در یک گراف مسطح با الگوریتمی با درجه

..... قابل محاسبه است.

(دولتی ۸۱)

$$(1) 1 \quad (2) n \quad (3) n^2 \quad (4) 2^n$$

۸۸- قطعه کد زیر نسخه‌ای از الگوریتم جستجوی عمق - نخست (depth first search) را در

یک گراف نشان می‌دهد. فرض کنید گراف مورد نظر جهت‌دار بوده و دارای حلقه نیست.

فرض کنید که Mystery یک فیلد در هر رأس است.

(۸۷ IT)

```
void DFS(v)
{
    v.Visited = true;
    v.Mystery = 0;
    for (each w such that v → w) {
        if (!w.Visited) DFS(w);
        v.Mystery = max(v.Mystery, 1 + w.Mystery);
    }
}
```

این الگوریتم با قطعه کد زیر فراخوانی می‌شود:

```
for(each v ∈ V) {
    v.Visited = false;
    v.Mystery = ۰;
}
for(each v ∈ V)
    if(!v.Visited)DFS(v);
```

برای هر رأس  $v$  کدام یک از گزینه‌های زیر عدد ذخیره شده در فیلد  $Mystery$  را پس از اجرای الگوریتم توصیف می‌کند؟ (۸۷ IT)

(۱) طول طولانی‌ترین مسیر در گراف که به  $v$  ختم می‌شود.

(۲) طول طولانی‌ترین مسیر در گراف که از  $v$  آغاز می‌شود.

(۳) طول کوتاه‌ترین مسیر در گراف از  $v$  با درجه ورودی صفر به  $v$ .

(۴) طول کوتاه‌ترین مسیر در گراف از  $v$  به یک رأس با درجه خروجی صفر.

۸۹- فرض کنید که  $A$  ماتریس مجاورت گراف  $G$  باشد، کدام ماتریس تمامی زوج‌های مرتبی از گره‌ها را نشان می‌دهند که به طول دقیقاً  $k$  به هم متصل شده‌اند؟ (توجه:  $V$  نشان‌دهنده جمع دودویی دو ماتریس است.) (۸۸ IT)

$$\sum_{j=1}^k A^j \quad (۴) \quad A^{k-1} \quad (۳) \quad A^k \quad (۲) \quad A^{k+1} \quad (۱)$$

۹۰- برای ۴ الگوریتم زیر برای گراف با  $n$  رأس و  $O(n)$  یال ۶ زمان اجرا به ترتیب از  $A$  تا  $F$  داده شده است:

۱- دایکسترا، ۲- پریم، ۳- بلمن فورد، ۴- همه‌ی کوتاه‌ترین مسیرها از یک رأس در یک DAG (گراف جهت‌دار و بدون دور)

(A)  $O(\log n)$ , (B)  $O(n)$ , (C)  $O(n \log n)$ , (D)  $O(n^2)$ , (E)  $O(n^2 \log n)$ , (F)  $O(n^3)$

بهترین گزینه برای زمان اجرای هر الگوریتم کدام یک می‌باشد؟ (تخصصی هوش مصنوعی ۸۸)

(۱) 1:D, 2:C, 3:F, 4:C (۲) 1:C, 2:D, 3:F, 4:F

(۳) 1:D, 2:D, 3:D, 4:B (۴) 1:C, 2:C, 3:D, 4:B

## ۹۱- کدام یک از گزاره‌های زیر درست و کدام یک نادرست است؟

الف) برای به دست آوردن طولانی‌ترین مسیر از یک رأس  $s$  به رأس  $t$  در یک گراف جهت‌دار و وزن‌دار بدون دور (DAG) می‌توان وزن همه‌ی یال‌ها را منفی کرده و با استفاده از الگوریتم فلوید کوتاه‌ترین مسیر بین  $s$  و  $t$  را در گراف جدید به دست آورد.

ب) اگر گراف  $G$  شامل یک مسیر یکتا (unique) با نام  $P$  از رأس  $s$  به رأس  $t$  باشد و درخت فراگیر کمینه‌ی  $G$  هم یکتا و با نام  $T$  باشد، در آن صورت هر یال موجود در  $P$  در  $T$  هم هست.

(نرم‌افراز - ۹۳)

- (۱) الف: نادرست، ب: نادرست (۲) الف: درست، ب: نادرست  
(۳) الف: درست، ب: درست (۴) الف: نادرست، ب: درست

۹۲- فرض کنید  $A$  ماتریس مجاورت یک گراف وزن‌دار و جهت‌دار  $G$  با  $n$  رأس است که در آن

درایه‌ی  $A[i, j]$  برابر با وزن یال  $i$  به  $j$  در صورت وجود است، اگر این یال موجود نباشد قرار می‌دهیم  $A[i, j] = 0$ . ماتریس  $A^k = \underbrace{A \times A \times \dots \times A}_k$  را در نظر بگیرید. فرض کنید که در

ضرب دو ماتریس به جای ضرب دو عدد عمل جمع آن دو به جای جمع عمل  $\min$  انجام شود. درایه‌ی  $A^k[i, j]$  چه عددی را نشان می‌دهد؟

(نرم‌افراز - ۹۳)

- (۱) مجموع وزن‌های همه‌ی مسیرهای از رأس  $i$  به رأس  $j$  که دقیقاً از  $k$  یال عبور کرده باشد.  
(۲) وزن کوتاه‌ترین مسیر از رأس  $i$  به رأس  $j$  که حداکثر از  $k$  یال عبور کرده باشد.  
(۳) وزن کوتاه‌ترین مسیر از رأس  $i$  به رأس  $j$  که حداکثر از  $k$  یال عبور کرده باشد.  
(۴) عددی غیر از گزینه‌های بالا

۹۳- گراف وزن‌دار و بدون جهت  $G = (V, E)$  و یک درخت فراگیر کمینه از آن داده شده است.

اگر به این گراف یک یال جدید با وزن نامنفی اضافه شود، درخت فراگیر کمینه‌ی جدید را در چه مرتبه‌ای می‌توان به دست آورد؟

(هوش مصنوعی - ۹۳)

- (۱)  $O(|V| + |E|)$  (۲)  $O(|V|)$  (۳)  $O(|V| \log |E|)$  (۴)  $O(|E| \log |V|)$

۹۴- فرض کنید گراف همبند و بدون جهت  $G$  حداقل دارای ۳ رأس است. می‌دانیم ترتیب رؤیت

رأس‌ها در جست‌وجوی عمق اول (DFS) و جست‌وجوی سطح اول (BFS) از یک رأس مشخص یکسان شده است. کدام گزینه غلط است؟

(هوش مصنوعی - ۹۳)

- (۱) گراف  $G$  می‌تواند گراف کامل باشد.  
(۲) قطر گراف  $G$  حداکثر ۲ است.  
(۳) گراف  $G$  می‌تواند یک گراف دوبخشی کامل باشد.  
(۴) گراف  $G$  حتماً یک درخت یا یک گراف کامل است.

**۹۵-** گراف  $G$  هم‌بند، بدون جهت، با وزن‌های متمایز و دارای دست کم ۳ رأس است. می‌دانیم درخت کوتاهترین مسیر (Shortest Path Tree) برای رأس مشخص  $s$  یکتا بوده و به شکل یک درخت ستاره‌ای به مرکزیت  $s$  است. کدام گزینه زیر صحیح است؟ درخت کوتاهترین مسیر را بدون جهت در نظر بگیرید. (۹۳-IT)

(۱) درخت پوشای کمینه نیز همین درخت خواهد بود.

(۲) در همه‌ی زیرگراف‌های مثلثی که یک رأس آن  $s$  است، نامساوی مثلثی برقرار است.

(۳) درجه‌ی رأس  $s$  در درخت پوشای کمینه بزرگ‌تر از یک خواهد بود.

(۴) درخت کوتاهترین مسیر برخی رأس‌های دیگر نیز می‌تواند ستاره‌ای شود.

**۹۶-** فرض کنید گراف  $G$  هم‌بند و بدون جهت است. می‌دانیم ترتیب یال‌های خروجی (یا به عبارتی ترتیب پیدا کردن یال‌های درخت پوشای کمینه) در الگوریتم‌های پریم و کروسکال یکسان شده است. در مورد گراف  $G$  چه می‌توان گفت؟ (۹۳-IT)

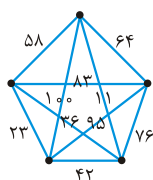
(۱) گراف  $G$  حتماً یک درخت است.

(۲) مجموعه یال‌های  $G$  که حداکثر وزن  $k$  (یک عدد مثبت دلخواه) دارند تشکیل یک گراف هم‌بند می‌دهند.

(۳) وزن یال‌های گراف  $G$  حتماً متمایز است.

(۴) گراف  $G$  حتماً یک گراف کامل است.

**۹۷-** در الگوریتم Kruskal برای گراف زیر، آخرین یال اضافه شده به درخت دارای چه وزنی است؟ (علوم کامپیوتر - ۹۳)



(۱) ۳۶

(۲) ۵۸

(۳) ۶۴

(۴) ۷۶

**۹۸-** فرض کنید گراف بدون جهت  $G = (V, E)$  وجود داشته باشد. می‌خواهیم بزرگترین زیرگراف در آن را پیدا کنیم به طوری که درجه هر گره در این زیرگراف بزرگتر یا مساوی  $k$  باشد. بهترین الگوریتم برای انجام این کار چه هزینه‌ای خواهد داشت؟ (فرض کنید گراف  $G$  به صورت ماتریس مجاورت ذخیره شده باشد). (علوم کامپیوتر - ۹۳)

(۱)  $O(n)$  (۲)  $O(n^2)$  (۳)  $O(n \log n)$  (۴)  $O(n^3)$

**۹۹-** اگر گراف  $G$  دارای  $O(n)$  یال باشد، آنگاه می‌تواند با چند رنگ، رنگ آمیزی شود؟ (علوم کامپیوتر - ۹۳)

(۱)  $O(n)$  (۲)  $O(\sqrt{n})$  (۳)  $O(n^2)$  (۴)  $O(n \log n)$



۱۰۰- چه تعداد از مسئله‌های زیر بر روی یک گراف بدون جهت و بی‌وزن  $G = (V, E)$  را

می‌توان با استفاده از الگوریتم عمق - اول (DFS) در زمان خطی  $O(|V| + |E|)$  حل کرد؟

- بررسی آن که  $G$  دو بخشی است.
- بررسی آن که  $G$  حاوی یک دور ساده است.
- یافتن تعداد اجزای هم‌بند  $G$

• با دریافت دو رأس  $u$  و  $x$  یافتن مسیری بین  $u$  و  $x$  با کم‌ترین تعداد یال (نرم‌افزار - ۹۴)

(۱) ۱ (۲) ۲ (۳) ۳ (۴) ۴

۱۰۱- چه تعداد از گزاره‌های زیر در مورد درخت فراگیر کمینه (MST) یک گراف ساده، بدون

جهت، وزن دار و هم‌بند  $G$  درست است؟ وزن یال‌ها لزوماً متفاوت نیست.

- اگر وزن سبک‌ترین یال بین هر برش در گراف یکتا باشد، MST هم یکتا خواهد بود.
- اگر وزن همه‌ی یال‌ها نابرابر باشد، MST یکتا است.

• اگر وزن یال  $c = (u, v)$  برابر باشد با بیشینه‌ی سبک‌ترین یال در همه‌ی مسیرهای بین

$u$  و  $v$ ، در آن صورت  $c$  در MST خواهد بود. (هوش مصنوعی - ۹۴)

(۱) ۰ (۲) ۱ (۳) ۲ (۴) ۳

۱۰۲- گراف جهت‌دار زیر با ۱۰۰ رأس را در نظر بگیرید:

$$v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_{100}$$

فرض کنید وزن همه‌ی یال‌ها برابر ۱ است. می‌خواهیم با استفاده از الگوریتم بلمن - فورد طول همه‌ی کوتاه‌ترین مسیرها را از رأس  $v_1$  به بقیه‌ی رأس‌ها بیابیم. الگوریتم در هر «مرحله» همه‌ی یال‌ها را با ترتیب دلخواهی مورد بررسی قرار می‌دهد. اگر در یک مرحله از این کار کوتاه‌ترین فاصله‌ی  $v_1$  تا همه‌ی رأس‌ها نسبت به مرحله‌ی قبل تغییر نکرده باشد، الگوریتم متوقف می‌شود. تعداد مراحل این الگوریتم به ترتیب بررسی یال‌ها وابسته است.

کمینه و بیشینه‌ی تعداد مرحله‌ها در این مسئله چند تا است؟ (هوش مصنوعی - ۹۴)

(۱) ۱۰۰ و ۱۰۰۰۰ (۲) ۲ و ۱۰۰ (۳) ۱۰۰ و ۱۰۰ (۴) ۲ و ۹۹

۱۰۳- هرم فیبوناچی داده ساختاری است که اعمال Extract-Min و Decrease-Key را بر روی یک

مجموعه‌ی  $n$  عضوی به ترتیب زمان‌های  $O(\lg n)$  و  $O(1)$  انجام می‌دهد. اگر از این داده ساختار برای یافتن کوتاه‌ترین مسیر بین دو رأس مشخص در گراف  $G = (V, E)$  با استفاده از

الگوریتم Dijkstra استفاده کنیم، هزینه‌ی کل کار چقدر خواهد بود؟ (هوش مصنوعی - ۹۴)

(۱)  $O(E \lg V + V)$  (۲)  $O((V + E) \lg V)$

(۳)  $O(V \lg V + E \lg E)$  (۴)  $O(V \lg V + E)$

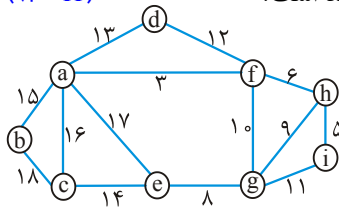
۱۰۴- چند تا از گزاره‌های زیر در مورد هم‌بندی قوی یک گراف جهت‌دار  $G = (V, E)$  درست است؟

- اگر  $G$  دور نداشته باشد، تعداد اجزای هم‌بند قوی آن برابر  $V$  است.
- اگر یک یال از  $G$  حذف شود، تعداد اجزای هم‌بند قوی  $G$  حداکثر ۲ واحد کم می‌شود.
- در الگوریتم یافتن اجزای هم‌بندی قوی  $G$ ، می‌توان به جای الگوریتم DFS از BFS استفاده کرد.

(۹۴ - IT)

۰ (۱)      ۱ (۲)      ۲ (۳)      ۳ (۴)

۱۰۵- مجموع وزن یال‌های درخت فراگیر کمینه‌ی گراف زیر چند است؟



۶۸ (۱)

۷۲ (۲)

۷۴ (۳)

۷۷ (۴)

۱۰۶- فرض کنید  $K_n$  یک گراف کامل و بدون جهت با  $n$  گره باشد که در آن  $n$  عددی زوج است.

یال‌های گراف  $K_n$  را به چند درخت پوشا می‌توان افراز کرد؟

(علوم کامپیوتر - ۹۴)

$\frac{n}{2}$  (۴)       $n-1$  (۳)       $n^{n-2}$  (۲)       $\frac{n-1}{2}$  (۱)

۱۰۷- درختی با  $n$  گره وجود دارد به طوری که درجه گره‌های آن  $d_1, d_2, \dots, d_n$  می‌باشند.

کدام یک از موارد زیر حاصل  $d_1 + d_2 + \dots + d_n$  را نشان می‌دهد؟

(علوم کامپیوتر - ۹۴)

$n-1$  (۴)       $n-2$  (۳)       $2n-1$  (۲)       $2n-2$  (۱)

۱۰۸- فرض کنید  $T$  یک درخت پوشای کمینه از گراف  $G$  باشد، در رابطه با  $T$  همه‌ی موارد زیر

(علوم کامپیوتر - ۹۴)

صحیح‌اند بجز:

(۱)  $T$  یکتا نیست.

(۲)  $T$  شامل تمام گره‌های  $G$  می‌باشد.

(۳) هر مسیر در  $T$  بین دو رأس  $s$  و  $t$  یک کوتاهترین مسیر در  $G$  نیست.

(۴) برای هر جفت رأس  $s$  و  $t$ ، کوتاهترین مسیر بین  $s$  و  $t$  در  $G$  همان مسیر بین  $s$  و  $t$  در  $T$  است.

۱۰۹- کدام گزینه در مورد گزاره‌های زیر صحیح است؟ در گزاره‌های زیر،  $P$  یک مسیر «ساده» و

(دکتری کامپیوتر - ۹۵)

$G$  یک گراف جهت‌دار وزن‌دار است.

الف) اگر  $P$  یک کوتاهترین مسیر در  $G$  باشد، آنگاه هر زیر مسیر از  $P$  نیز یک کوتاه‌ترین

مسیر در  $G$  است.

ب) اگر  $P$  یک بلندترین مسیر در  $G$  باشد، آنگاه هر زیر مسیر از  $P$  نیز یک بلندترین مسیر

در  $G$  است.

- (۱) (الف) درست، (ب) نادرست  
(۲) (الف) درست، (ب) درست  
(۳) (الف) نادرست، (ب) نادرست  
(۴) (الف) نادرست، (ب) درست

۱۱۰- فرض کنید  $T$  یک درخت فراگیر کمینه از گراف وزن دار  $G$  باشد. چند تا از گزاره‌های زیر

همیشه درست‌اند؟ (دکتری کامپیوتر - ۹۵)

- اگر  $v$  یک رأس از  $G$  باشد، آنگاه  $T - \{v\}$  یک درخت فراگیر کمینه از  $G - \{v\}$  است.  
اگر  $v$  یک برگ از  $T$  باشد، آنگاه  $T - \{v\}$  یک درخت فراگیر کمینه از  $G - \{v\}$  است.  
اگر  $e$  یک یال از  $T$  باشد، آنگاه  $T - \{e\}$  یک جنگل شامل دو درخت  $T_1, T_2$  است، طوری که به ازای  $i = 1, 2$  یک درخت فراگیر کمینه از گراف القایی  $G$  روی رأس‌های  $T_i$  است.
- (۱) ۳ (۲) ۲ (۳) ۱ (۴) ۰

۱۱۱- گراف وزن دار  $G$  با تابع وزن  $f$  روی یال‌ها را در نظر بگیرید. می‌خواهیم با تغییر وزن هر یال

$e$  از  $f(e)$  به  $f'(e)$  به گراف جدید  $G'$  برسیم، طوری که به ازای هر دو رأس  $u$  و  $v$ ، کوتاه‌ترین مسیر بین  $u$  و  $v$  در  $G$  برابر کوتاه‌ترین مسیر بین  $u$  و  $v$  در  $G'$  باشد. کدام یک از توابع زیر این ویژگی را دارند؟ در گزینه‌های زیر  $c$  یک عدد ثابت مثبت، و  $g$  یک تابع وزن دلخواه روی رأس‌های گراف است. (دکتری کامپیوتر - ۹۵)

$$(۱) f'(e) = f(e) - c$$

$$(۲) f'(e) = f(e) + c$$

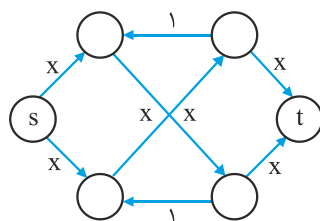
$$(۳) f'(e) = f(e) + g(u) + g(v) \text{ (با فرض } e = (u, v) \text{)}$$

$$(۴) f'(e) = f(e) - g(u) + g(v) \text{ (با فرض } e = (u, v) \text{)}$$

۱۱۲- الگوریتم فورد - فالکرسن برای یافتن شار بیشینه را در نظر بگیرید. با فرض این‌که این

الگوریتم در هر بار عملیات افزایش شار، در مسیر افزایشی انتخاب شده بین  $s$  و  $t$  بیشترین مقدار شار ممکن را ارسال می‌کند، در گراف زیر حداکثر چند بار ممکن است عملیات افزایش شار انجام شود؟ منظور از  $x$  در گراف زیر یک عدد صحیح بزرگ‌تر از ۱ است.

(دکتری کامپیوتر - ۹۵)



$$(۱) 2x$$

$$(۲) x$$

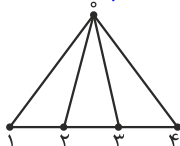
$$(۳) 2$$

$$(۴) 1$$

۱۱۳- در گراف وزن دار فاصله دو رأس به صورت طول کوتاه ترین مسیر بین آن دو رأس تعریف می شود. قطر گراف نیز برابر بیشترین فاصله بین جفت رأس های گراف تعریف می شود. بهترین الگوریتم برای یافتن قطر یک درخت ریشه دار با  $n$  گره، دارای چه مرتبه زمانی است؟  
(دکتری علوم کامپیوتر - ۹۵)

$$(۱) O(n \log n) \quad (۲) O(\log n^2) \quad (۳) O(n^2) \quad (۴) O(n)$$

۱۱۴- گراف Fan زیر را برای  $n=4$  در نظر بگیرید. به طور کلی گراف Fan همواره دارای  $n+1$  گره و ساختاری مشابه شکل زیر دارد. اگر  $t_n$  تعداد درخت های پوشای (Spanning) گراف Fan با  $n+1$  گره باشد، و  $t_0=1$  در این صورت کدام یک از روابط زیر برای  $n \geq 1$  صادق است؟  
(دکتری علوم کامپیوتر - ۹۵)



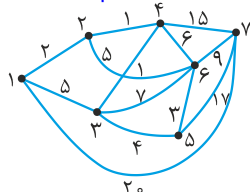
$$(۱) t_{n+1} = 2t_n + 1 \quad (۲) t_{n+1} = 2t_n + 2$$

$$(۳) t_{n+1} = t_n + \sum_{i=0}^n t_i \quad (۴) t_{n+1} = t_n + 2 \sum_{i=0}^n t_i$$

۱۱۵- تعداد درختان پوشا (spanning) برای یک گراف با  $2n$  رأس که از اتصال دو گراف کامل  $(K_n)$  با یک یال به دست می آید چقدر است؟  
(دکتری علوم کامپیوتر - ۹۵)

$$(۱) n^{n-4} \quad (۲) n^{2n-4} \quad (۳) 2n^{n-2} \quad (۴) 2n^{2n-2}$$

۱۱۶- الگوریتم دایکسترا را بر روی گراف زیر و برای یافتن همه ی کوتاه ترین مسیرها از رأس شماره ۱ اجرا کنید. رأس شماره ۵ چندمین رأسی است که کوتاه ترین مسیرش از رأس مبدا به دست می آید؟ رأس مبدا نیز در این ترتیب لحاظ می شود.  
(نرم افزار - ۹۵)



$$(۱) 3$$

$$(۲) 4$$

$$(۳) 5$$

$$(۴) 6$$

۱۱۷- چند تا از گزاره های زیر درست اند؟  
(نرم افزار - ۹۵)

• ناحیه های ایجاد شده بین  $n$  دایره روی یک صفحه را می توان با دو رنگ رنگ آمیزی کرد، طوری که هیچ دو ناحیه ی مجاور هم رنگ نباشند. دو ناحیه مجاورند اگر یک کمان مشترک بین آنها باشد.

• هر هزینه ی پستی بیش از ۷ ریال را می توان با تمبرهای ۳ ریالی و ۵ ریالی انجام داد.

• در گراف جهت دار شبکه ی شار اگر هر یال را با یال بدون جهت و با همان ظرفیت تعویض کنیم، مقدار شار بیشینه تغییر نمی کند.

$$(۱) 0 \quad (۲) 1 \quad (۳) 2 \quad (۴) 3$$

۱۱۸- در چه مرتبه‌ای می‌توان «قطر» یک DAG را به دست آورد؟ قطر حداکثر طول مسیر بین دو رأس در گراف است.

(هوش مصنوعی - ۹۵)

$$(۱) O(|V|) \quad (۲) O(|V| + |E|) \quad (۳) O(|V| \log |E|) \quad (۴) O(|V|^2)$$

۱۱۹- گراف ساده و وزن دار  $G = (V, E)$  را در نظر بگیرید. وزن یال‌های این گراف نامنفی است و  $M$  زیردرختی فراگیر با کم‌ترین وزن در این گراف است. هم‌چنین می‌دانیم که  $P$  کوتاه‌ترین مسیر بین دو رأس  $u$  و  $v$  است. فرض کنید که به جای وزن هر یال، مجذور وزن آن را قرار می‌دهیم. مثلاً، اگر وزن یالی ۳ بود وزن آن را با ۹ می‌کنیم. وضعیت گزاره‌های زیر کدام است؟

(هوش مصنوعی - ۹۵)

(الف) در گراف جدید، همان  $P$  قبلی لزوماً کوتاه‌ترین مسیر بین  $u$  و  $v$  است.

(ب) در گراف جدید، همان  $M$  قبلی لزوماً زیردرخت فراگیر با کم‌ترین وزن است.

(۱) الف: درست، ب: درست. (۲) الف: نادرست، ب: درست.

(۳) الف: درست، ب: نادرست. (۴) الف: نادرست، ب: نادرست.

۱۲۰- چند تا از گزاره‌های زیر درست‌اند؟

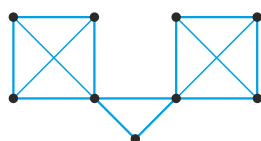
(الف) با این تغییر می‌توان از الگوریتم دایکسترا برای گراف‌های با وزن منفی استفاده کرد. به وزن همه‌ی یال‌ها یک عدد ثابت بزرگ اضافه کرده و الگوریتم دایکسترا را بر روی گراف حاصل اجرا می‌کنیم. مسیرهای به دست آمده کوتاه‌ترین مسیرها در گراف اصلی هستند.

(ب) یک درخت پوشای کمینه هیچ گاه شامل یال با بیش‌ترین وزن نیست.

(ج) وجود یا عدم وجود تور اویلری در یک گراف را می‌توان در زمان خطی تشخیص داد.

(۱) ۰ (۲) ۱ (۳) ۲ (۴) ۳

۱۲۱- تعداد درخت‌های پوشا (spanning) برای گراف زیر، چند تا است؟



(۱) ۴۸

(۲) ۲۵۶

(۳) ۵۱۲

(۴) ۷۶۸

۱۲۲- در یک گراف همبند وزن دار  $G$  با  $n$  رأس و  $m$  یال که وزن تمام یال‌ها مجزا است، می‌خواهیم الگوریتمی طراحی کنیم که برای یال داده شده  $e$  مشخص کند که آیا  $e$  جزء درخت پوشای مینیمم (minimum spanning tree) است یا نه. مرتبه زمانی الگوریتم چیست؟

(علوم کامپیوتر - ۹۵)

$$(۱) O(m \log n) \quad (۲) O(m \log m) \quad (۳) O(2^n) \quad (۴) O(n + m)$$

## پاسخنامه سؤال‌های چهارگزینه‌ای فصل هفتم

- ۱- گزینه (۱). در گراف هم بند، بدون جهت، بدون حلقه (بدون طوقه) و بدون یال چندگانه (multiedge) بین تعداد رئوس ( $n$ ) و تعداد یالها ( $e$ ) رابطه  $\frac{n(n-1)}{2} \leq e \leq n-1$  برقرار است. پس چون  $n-1 \leq e$  گزینه ۳ صحیح است. از  $n-1 \leq e$  نتیجه می‌شود  $n \leq e+1$  و نتیجه می‌شود  $n \leq e^2 + 1$  پس گزینه ۴ نیز صحیح است. از  $e \leq \frac{n(n-1)}{2}$  نتیجه می‌شود  $e \leq \frac{n^2}{2}$  و نتیجه می‌شود  $e \leq n^2$  پس گزینه ۲ نیز صحیح است. برای نقض گزینه ۱، گراف  $K_6$  را در نظر بگیرید که  $n=6$  و  $e = \frac{6 \times 5}{2} = 15$  و  $n \geq \frac{e}{2}$  یعنی  $6 \geq \frac{15}{2}$  نادرست است.
- ۲- گزینه (۴). در گراف جهت‌دار همبند بدون سیکل (dag) و بدون یال چندگانه رابطه  $\frac{n(n-1)}{2} \leq e \leq n-1$  برقرار است. و داریم:

$$n-1 \leq e \leq \frac{n(n-1)}{2} \Rightarrow n-1 \leq e \quad (\text{گزینه ۲})$$

$$\Rightarrow n-1 \leq e \Rightarrow n \leq e+1 \Rightarrow n \leq e^2 + 1 \quad (\text{گزینه ۳})$$

$$e \leq \frac{n^2 - n}{2} \Rightarrow e \leq n^2 - n \Rightarrow e \leq n^2 \Rightarrow \quad (\text{گزینه ۱})$$

برای نقض گزینه ۴ گراف کامل غیرجهت‌دار  $K_6$  را در نظر بگیرید و به یالهای آن جهت دلخواه دهید، طوری که dag شود.

- ۳- گزینه (۲). در پیمایش dfs، از هر رأسی می‌تون شروع کرد. همه گزینه‌ها از  $a$  شروع کرده‌اند. در گزینه ۲، بعد از  $a$  رأس  $b$  و سپس  $d$  و سپس  $c$  ملاقات شده است، تا اینجا درست است ولی بعد از  $c$ ، رأس  $f$  ملاقات شده که اشتباه است و باید  $e$  ملاقات می‌شد.

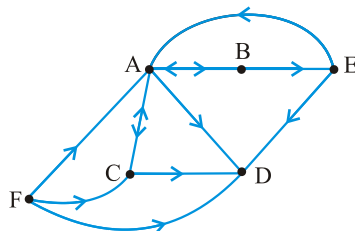
- ۴- گزینه (۱). از رأس ۱، امکان ملاقات رئوس ۲ و ۵ و ۸ می‌باشد که رأس ۲ ملاقات می‌شود از رأس ۲، رأس ۳ و بعد ۴ و بعد ۱۱ ملاقات می‌شوند (پس گزینه ۱ صحیح است) از رأس ۱۱ امکان ادامه نیست و به رأس ۲ بازگشت می‌کنیم و از رأس ۲، رأس ۵ و سپس ۶ و سپس ۷ ملاقات می‌شود. از رأس ۷ به ۶ بازگشت می‌کنیم و رأس ۱۰ ملاقات می‌شود. از رأس ۱۰ به رأس ۱ بازگشت می‌کنیم و رأس ۸ و سپس ۹ ملاقات می‌شوند.

- ۵- گزینه (۴). مرتب‌سازی توپولوژیکی یعنی، ملاقات رئوس طبق ترتیب پیش‌نیازی. مثلاً در گراف داده شده نمی‌توان اول ۱ را ملاقات کرد زیرا رأس ۷ پیش‌نیاز رأس ۱ است بنابراین گزینه ۱ صحیح نیست.

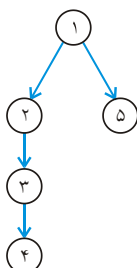
گزینه ۲ نیز صحیح نیست زیرا رأس ۶ نمی‌تواند قبل از ۵ ملاقات شود.

گزینه ۳ نیز صحیح نیست زیرا رأس ۴ نمی‌تواند قبل از ۶ ملاقات شود.

- ۶- گزینه (۴). تعدادی از این دنباله‌ها را می‌نویسیم:  
 ... و  $۱۵۲۶۷۳۸۴, ۱۵۷۲۶۳۸۴, ۱۵۲۷۶۳۸۴, ۱۲۵۶۷۳۸۴, ۱۲۶۵۷۳۸۴, ۱۶۲۵۷۳۸۴, ۱۶۵۲۷۳۸۴, ۱۵۲۶۷۳۸۴$   
 به هر حال ۵ و ۲ و ۶ حتماً بعد از ۱ باید ملاقات شوند و ترتیب خودشان مهم نیست که ۳! حالت می‌دهد. همچنین ۷ را می‌توان قبل از ۲ و ۶ و ۳ یا بعد از هر کدام ملاقات کرد و ... به هر حال تعداد از ۶ بیشتر است.
- ۷- گزینه (۴). در گراف  $G$  درجه  $a$  برابر ۴ است. یعنی گراف  $G$  حداقل ۲ یال بیشتر از درخت رسم شده دارد. در نتیجه گراف  $G$  حتماً دور دارد (به درخت هر یالی اضافه شود دور تشکیل می‌شود). اگر از  $G$ ،  $a$  را حذف کنیم آنگاه گراف ناهمبند ۲ مولفه‌ای می‌شود پس گزینه‌های ۱ و ۲ و ۳ نادرست هستند. اگر از گراف  $G$ ،  $b$  را حذف کنیم، حتماً ناهمبند می‌شود، زیرا درجه  $a$  برابر ۴ است و نمی‌توانند همه قسمت‌های گراف را به هم متصل نگه دارد.
- ۸- گزینه (۲). جمع درجات رئوس برابر  $۱۶ = ۲ + ۲ + ۳ + ۳ + ۳ + ۳$  است و از آنجایی که جمع درجات ۲ برابر تعداد یالهاست پس گراف ۸ یال دارد. تعداد رئوس گراف نیز ۶ است (در دنباله داده شده ۶ عدد هست)، پس در درخت حاصل از پیمایش، ۵ یال وجود دارد (تعداد یال‌ها در درخت یک واحد از تعداد رئوس کمتر است) بنابراین  $۳ = ۵ - ۸$  یال پشته‌ای هستند. لازم به ذکر است که پیمایش DFS گراف غیر جهت دار، فقط یالهای درختی و پشته‌ای تولید می‌کند.
- ۹- گزینه (۴). با الگوریتم DFS می‌توان وجود یا عدم وجود دور در هر گرافی را تشخیص داد. شرط لازم و کافی برای آنکه سیکل داشته باشیم آن است که DFS، یال پشته‌ای تولید کند. پس مرتبه تشخیص دور  $O(n + e)$  است. که اگر گفته شود گراف همبند است می‌توان  $O(e)$  گفت.
- ۱۰- گزینه (۲). می‌توان با اجرای الگوریتم DFS طبق آنچه در درس گفته شد، مولفه‌های متصل قوی را یافت. ولی با توجه به شکل گراف به راحتی می‌توان با کمی توجه و دقت، مولفه‌ها را یافت.



مولفه‌های همبند قوی این گراف  $\{ABCE\}$  و  $\{D\}$  و  $\{F\}$  هستند.

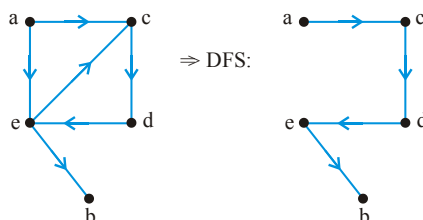


۱۱- گزینه (۲). درخت پوشای حاصل به شکل مقابل است و رئوس ۵ و ۲ هیچ یک نواده یا جد یکدیگر نیستند.

۱۲- گزینه (۳). با توجه به متن درس، یکبار روی خود گراف الگوریتم پیمایش عمیقی اجرا می‌شود. و سپس روی ترانهاده گراف به ترتیب نزولی زمانهای پایان (f) پیمایش عمقی انجام می‌شود.

۱۳- گزینه (۴). گراف مذکور دارای پیمایش‌های عمقی مختلفی است و در هر کدام یالها عوض می‌شوند. در این تست طبق اولویت حروف الفبا پیمایش را انجام می‌دهیم:

tree: ac, cd, de, eb  
back: ec  
forward: ae



۱۴- گزینه (۳). الگوریتم یافتن دور در گراف با اعمال تغییراتی روی DFS قابل پیاده‌سازی است. در بهترین حالت اگر فرض کنیم درجه هر گره در گراف ۲ باشد، مرتبه الگوریتم  $O(|V|)$  خواهد بود.

۱۵- گزینه (۲). در گراف همبندی که فقط یک دور دارد،  $n = e$  است. کافی است آن دور را در گراف بیابیم که با پیمایش DFS امکان پذیر است و سپس سنگین‌ترین یال آن دور را حذف کنیم. مرتبه این عملیات  $O(n + e)$  است ولی با کمی دقت بیشتر می‌توان  $O(n)$  نوشت چون  $n = e$  است، که در گزینه‌ها نیست.

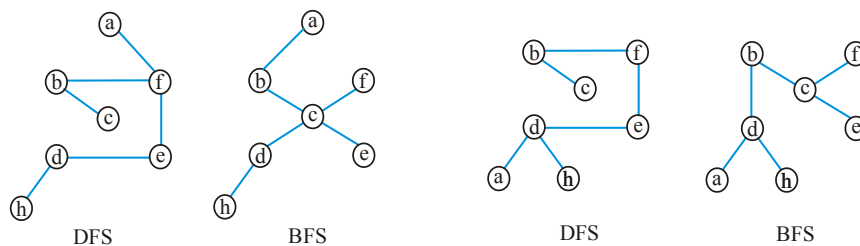
۱۶- گزینه (۲). به طور کلی پیدا کردن طولانی‌ترین مسیرها در گراف، راه حل چند جمله‌ای ندارد. ولی در DAG با مرتبه  $O(n + e)$  این عمل قبل انجام است. یک روش این است که همه وزن‌ها را قرینه کنیم و سپس به ترتیب توپولوژیکی، عمل relax را اجرا کنیم و کوتاهترین مسیرها را بیابیم.

۱۷- گزینه (۲). در متن درس بحث شده است. یادآوری می‌کنیم اگر  $d[v] < f[v] < d[u] < f[u]$  آنگاه یال  $(u, v)$  در صورت وجود، یال cross است. و اگر  $d[u] < f[u] < d[v] < f[v]$  آنگاه یال  $(v, u)$  در صورت وجود، یال cross است.



- ۱۸- گزینه (۱). همانطور که می‌دانیم، DFS گراف غیرجهت‌دار یال forward و cross ندارد پس باید فرض کنیم در این تست گراف و درخت حاصل جهت‌دار هستند (جهت‌ها را از بالا به پایین فرض می‌کنیم) در این صورت یال (۱۴ و forward و یال (۶ و cross است. در ضمن یال‌های (۴ و ۳) و (۱۰ و ۹) در صورت وجود، cross هستند. یال‌های (۱ و ۶) و (۷ و ۲) در صورت وجود، back هستند.
- ۱۹- گزینه (۴). در DFS گراف غیرجهت‌دار، یال cross وجود ندارد. یال xy یا yx در صورتی cross است که  $d(x) < f(x) < d(y) < f(y)$  یا  $d(x) < f(x) < d(y) < f(y)$  (منظور از  $d(x)$  زمان ورود x و  $f(x)$  زمان خروج x است) با توجه به جدول، یال ch امکان‌پذیر نیست چون  $d(c) < f(c) < d(h) < f(h)$  ( $۸ < ۱۱ < ۱۲ < ۱۵$ ).
- ۲۰- گزینه (۴). در گراف  $v \rightarrow w \rightarrow x$ ، اجرای DFS(v) خروجی vwx تولید و اجرای DFS(w) خروجی wx تولید می‌کند و DFS(w) سریع‌تر است. حال اگر از x به v یال وجود داشته باشد (سیکل داشته باشد)، DFS(w) خروجی wxv تولید می‌کند که از DFS(v) سریع‌تر نیست. (فرض کرده‌ایم DFS فقط یک بار فراخوانی می‌شود و لزوماً همه رئوس ملاقات نمی‌شوند)
- ۲۱- گزینه (۳). پیمایش DFS گراف غیر جهت‌دار، یال cross تولید نمی‌کند. پس یال‌های  $\{۲, ۵\}$  و  $\{۲, ۶\}$  و  $\{۶, ۷\}$  نمی‌توانند وجود داشته باشند. علاوه بر این اگر مثلاً فرض کنیم یال  $\{۲, ۵\}$  وجود داشته است، آنگاه باید رأس ۵ را از طریق رأس ۲ ملاقات می‌کردیم نه اینکه به رأس ۱ بازگشت کنیم.
- ۲۲- گزینه (۱). پیمایش BFS گراف غیر جهت‌دار، یال پشتی و پیش‌رو ندارد. پس یال‌های  $\{۱, ۵\}$  و  $\{۳, ۷\}$  و  $\{۱, ۴\}$  و  $\{۱, ۷\}$  نمی‌توانند وجود داشته باشند. ولی یال  $\{۴, ۵\}$  در صورت وجود، یال عبوری است که امکان وجودش هست. برای درک بهتر فرض کنید یال  $\{۱, ۵\}$  وجود داشته باشد، پس باید از طریق رأس ۱ رأس ۵ ملاقات شود و نه از طریق رأس ۲.
- ۲۳- گزینه (۱). پیمایش BFS می‌تواند کمترین تعداد یال هر رأس تا رأس مبدا را بیابد. در واقع در این پیمایش رأسی که به رأس شروع نزدیکتر است، زودتر ملاقات می‌شود.
- ۲۴- گزینه (۳). پیمایش عمقی از پشت به پیمایش سطحی از صف استفاده می‌کند. یکی از کاربردهای پیمایش عمقی، ترتیب توپولوژیکی است. زمان اجرای هر دو پیمایش عمقی و سطحی یکسان است، بالیست  $O(n + e)$  و با ماتریس  $O(n^2)$  است. وجود یال پشتی در DFS شرط لازم و کافی برای وجود سیکل در گراف است.
- ۲۵- گزینه (۴). با لیست، مرتبه BFS برابر  $O(|V| + |E|)$  و با ماتریس  $O(|V|^2)$  است.

۲۶- گزینه (۱). پیمایش را می‌توان از هر رأسی شروع کرد و با هر اولییتی رئوس را ملاقات کرد. هر دو پیمایش داده شده برای هر دو گراف درست است. شکل درخت پوشای حاصل از دو روش روی دو گراف را رسم می‌کنیم.



گراف ۱

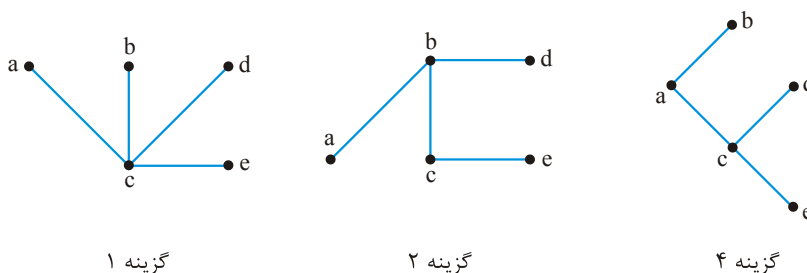
گراف ۲

۲۷- گزینه (۴). در BFS گراف جهت‌دار یال forward وجود ندارد و در BFS گراف غیرجهت‌دار یال forward و back وجود ندارد.

۲۸- گزینه (۳). در پیمایش سطحی،  $d[v]$  برابر کمترین تعداد یال رأس شروع تا رأس  $v$  است (distance). اگر یال  $(u, v)$  یال پشتی باشد، به این معنی است که رأس  $v$  زودتر از  $u$  ملاقات شده است پس  $d[v] \leq d[u]$  می‌باشد. اگر یال  $(u, v)$  یال عبوری باشد به این معنی است یا دو رأس  $u$  و  $v$  تا رأس شروع هم فاصله هستند ( $d[v] = d[u]$ ) و یا رأس  $a$  فقط یک واحد از  $v$  به رأس شروع نزدیکتر است. ( $d[v] = d[u] + 1$ ).

تعداد یالهای درختی و پشتی و عبوری و پیش رو به طور کلی هیچ رابطه‌ای با هم ندارد. فقط می‌توان گفت در پیمایش سطحی گراف جهت‌دار، یال پیش رو وجود ندارد. و در پیمایش سطحی گراف غیر جهت‌دار یال پیش رو و پشتی وجود ندارد.

۲۹- گزینه (۳).



گزینه ۱

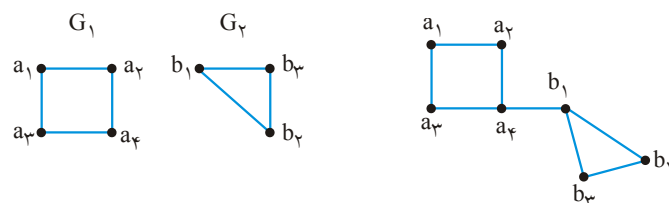
گزینه ۲

گزینه ۴

در گزینه ۳ پس از ملاقات  $a$  و  $b$  و  $c$  ابتدا باید  $d$  از طریق  $b$  ملاقات شود و سپس  $e$  از طریق  $c$ .

۳۰- گزینه (۲). بعد از رأس  $V_1$  باید  $V_2$  و  $V_4$  و  $V_6$  ملاقات شوند پس گزینه‌های ۱ و ۳ صحیح نیستند. در گزینه ۴، بعد از  $V_1$ ، رأس  $V_6$  و بعد  $V_4$  و بعد  $V_2$  ملاقات شده است حال باید با کمک  $V_6$ ، رأس  $V_5$  را ملاقات کرد پس گزینه ۴ صحیح نیست.

۳۱- گزینه (۲). دو گراف  $G_1$  و  $G_2$  را در نظر بگیرید. پیمایش سطحی  $G_1$  و  $G_2$  به ترتیب  $T_1 = a_1 a_2 a_3 a_4$  و  $T_2 = b_1 b_2 b_3$  است. حال اگر طبق گزینه ۱ رأس آخر  $G_1$  در پیمایش BFS را به رأس اول  $G_2$  متصل کنیم. گرافی بدست می‌آید که پیمایش BFS آن  $T = a_1 a_2 a_3 a_4 b_1 b_2 b_3 = T_1 T_2$  است و گزینه ۱ صحیح است.



طبق گزینه ۲، اگر رأس شروع دو گراف را هم متصل کنیم گراف مقابل حاصل می‌شود که پیمایش BFS آن  $T_1 T_2 \neq a_1 a_2 a_3 b_1 a_4 b_2 b_3$  است. و گزینه ۲ نادرست است به همین ترتیب گزینه‌های ۳ و ۴ قابل بررسی هستند.

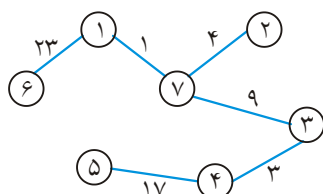
۳۲- گزینه (۲). برای یافتن قطر باید کوتاهترین فاصله بین هر دو رأس را بیابیم و سپس بین این فاصله‌ها ماکزیمم بگیریم. در بهترین حالت مرتبه این عمل  $O(n + e)$  است و چون در درخت  $e = \theta(n)$ ، پس جواب  $O(n)$  است.

۳۳- گزینه (۲). برای یافتن قطر در گراف،  $n$  بار و هر بار با مبدأ یک رأس جدید، پیمایش سطحی اجرا می‌کنیم و بین فاصله‌های بدست آمده، ماکزیمم فاصله، قطر گراف است. مرتبه این عملیات  $O(n(n + e))$  است.

۳۴- گزینه (۳). جمله اول صحیح است اگر گراف بالیست مجاورتی پیاده‌سازی شود، با مرتبه  $O(n + e)$  می‌توان اویلری بودن را تشخیص داد. جمله دوم صحیح نیست زیرا شرط همبندی را ذکر نکرده است. شرط لازم و کافی برای آنکه گراف جهت‌دار، اویلری باشد، آن است که همبند باشد و برای هر رأس  $v$  رابطه  $\text{in deg}(v) = \text{out deg}(v)$  برقرار باشد. شرط لازم و کافی برای آنکه گراف غیر جهت‌دار، اویلری باشد، آن است که همبند باشد و درجه همه رئوس زوج باشد.

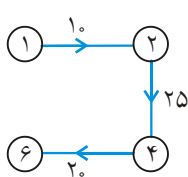
جمله سوم درست است، اگر MST یکتا باشد نمی‌توان نتیجه گرفت وزن‌ها متمایز هستند.





۳۵- گزینه (۴). شکل MST این گراف، با هر الگوریتمی به صورت مقابل است که جمع وزنهای آن برابر ۵۷ است.

۳۶- گزینه (۴). می‌توان از هر رأسی شروع کرد. از رأس ۱ شروع می‌کنیم. سبک‌ترین یال مجاور با رأس ۱، یال با وزن ۱۰ است پس بعد از ۱ رأس ۲ ملاقات می‌شود. سبک‌ترین یال مجاور با رئوس ۱ یا ۲، یال با وزن ۲۵ است پس رأس ۴ ملاقات می‌شود. و سپس رأس ۶ ملاقات می‌شود.

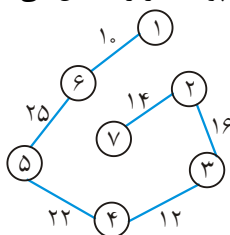


۳۷- گزینه (۳). برای گراف‌های خلوت، کراسکال از پریم بهتر است.

۳۸- گزینه (۲). الگوریتم داده شده، تا زمانی که  $n-1$  یال انتخاب شود، سبک‌ترین یال‌ها را به شرطی که سیکل تشکیل نشود، انتخاب می‌کند که همان کراسکال است.

۳۹- گزینه (۴). گرافی که  $n$  رأس و  $n-2$  یال دارد، همبند نیست و درخت پوشا ندارد.

۴۰- گزینه (۳). با هر الگوریتمی درخت پوشای زیر حاصل می‌شود.

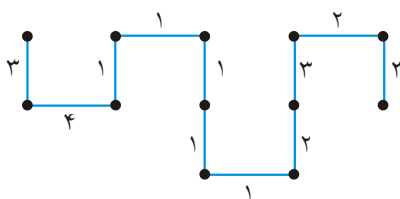


۴۱- گزینه (۴). مرتبه کراسکال همیشه  $O(e \cdot \lg n) = O(e \cdot \lg e)$  است. مرتبه پریم با ماتریس

مجاورتی  $O(n^2)$  است. در تست‌های دانشگاه آزاد، معمولاً پیش فرض ساخت گراف، ماتریس مجاورتی است. ولی در تست‌های دانشگاه دولتی، پیش فرض، لیست مجاورتی است.

۴۲- گزینه (۴). برای گراف خلوت یا پراکنده مرتبه کراسکال  $O(n \cdot \lg n)$  می‌شود، زیرا  $e \in O(n)$  است.

۴۳- گزینه (۱). برای گراف بسیار متصل،  $e \in \theta(n^2)$  است پس مرتبه کراسکال  $O(n^2 \cdot \lg n)$  است.



۴۴- گزینه (۱). این گراف چندین MST دارد که هزینه همگی برابر ۲۲ است. یکی از MST ها به شکل مقابل است.

۴۵- گزینه (۳). با توجه به رابطه داده شده وزن برخی یال‌ها در جدول زیر آمده است:

| یال (i, j)   | (۱,۲) | (۲,۳) | (۲,۴) | (۲,۵) | (۱,۳) | (۱,۴) | (۲,۶) | (۱,۵) | (۱,۶) | ... |
|--------------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-----|
| وزن $w_{ij}$ | ۵     | ۵     | ۶     | ۷     | ۱۰    | ۵     | ۸     | ۶     | ۷     | ... |

یال‌های (۱,۲), (۲,۳), (۱,۴), (۱,۵), (۱,۶), (۱,۷), (۱,۸), (۱,۹), (۱,۱۰) بنابراین هزینه MST این گراف برابر است با:

۴۶- گزینه (۲ و ۱). در الگوریتم کراسکال، اول باید BC انتخاب شود. بعد از آن می‌توان AB یا AF یا BE یا FE را انتخاب کرد. به هر حال از بین یالهای AB و AF و BE و FE باید ۳ یال انتخاب شوند بعد از انتخاب ۳ یال از بین این ۴ یال، باید ED انتخاب شود که گزینه ۴ نادرست است.

در الگوریتم پریم اگر از A شروع کنیم می‌توان B یا F را ملاقات کرد. اگر B را ملاقات کنیم باید C را ملاقات کنیم که گزینه ۳ غلط است. گزینه‌های ۱ و ۲ هر دو صحیح هستند ولی کلید سنجش ۲ است.

۴۷- گزینه (۳). با ماتریس زمان اجرای کراسکال  $\theta(\log n)$  و پریم  $\theta(n^2)$  است. اگر گراف وزنهای مساوی داشته باشد، ممکن است درخت حاصل از دو روش یکسان نباشد ولی جمع وزنهای درخت حاصل همیشه یکسان و مینیمم است.

۴۸- گزینه (۳). اگر وزنهای یکسان باشد ممکن است درخت‌های حاصل یکسان نباشند ولی جمع وزنهای همیشه یکسان است.

۴۹- گزینه (۲). اگر یالی که مقدارش کاهش یافته متعلق به درخت T نباشد، و آن را به درخت T اضافه کنیم یک دور ایجاد می‌شود و با حذف بزرگترین یال از آن دور، درخت جدیدی ایجاد می‌شود که مینیمم است و با درخت قبلی متفاوت است. گزینه ۴ نادرست است چون لازم نیست کلیه یالهای متعلق به T کاهش یافته باشند.

۵۰- گزینه (۱). اگر یال e متعلق به درخت T باشد خود T کمینه است. برای آنکه بفهمیم e متعلق به T است یا نه باید یالهای درخت T را که  $|V|-1$  است بررسی کنیم و مرتبه آن  $O(|V|)$  است. اگر e متعلق به T نباشد باید e را به T اضافه کنیم و یک یال حذف کنیم که این عمل نیز از مرتبه  $O(|V|)$  است.

۵۱- گزینه (۲). دو روش برای این منظور وجود دارد.

**روش ۱:** e را حذف کنیم (بدون حذف u و v)، MST گراف حاصل را بیابیم، سپس e را به MST اضافه کنیم که یک سیکل ایجاد می‌شود. حال از این سیکل، سنگین‌ترین یال را حذف کنیم.

**روش ۲:** u و v را ادغام کنیم یعنی دو رأس u و v تبدیل به یک رأس x می‌شوند، سپس MST را بیابیم. و در نهایت رأس x را به یال (u, v) تبدیل می‌کنیم.

۵۲- گزینه (۳). کراسکال به ترتیب یالهای سبک را به شرطی که سیکل تشکیل نشود، انتخاب می‌کند پس از چپ به راست یالهای انتخابی عبارتند از:

$(1,2), (2,3), (4,5), (6,7), \dots$

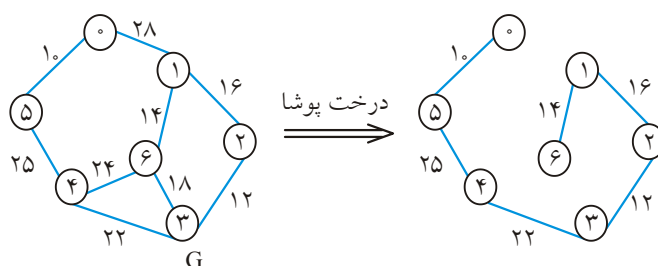
۵۳- گزینه (۲). با ماتریس الگوریتم پریم از مرتبه  $O(n^2)$  و کراسکال از مرتبه  $O(e \log e)$  است. در گراف خلوت، کراسکال  $O(n \log n)$  می‌شود و در گراف‌های خیلی متصل مرتبه کراسکال  $O(n^2 \log n)$  می‌شود. پس اگر گراف خیلی متصل باشد، پریم از کراسکال بهتر است. اگر خلوت یا متراکم بودن گراف معلوم نباشد، با لیست پیاده‌سازی‌هایی از پریم وجود دارد که از کراسکال سریع‌تر است.

۵۴- گزینه (۱). به ترتیب یالهای سبک به شرطی که سیکل تشکیل نشود، انتخاب می‌شوند.

۵۵- گزینه (۳).

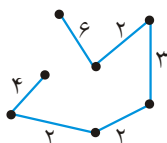
۵۶- گزینه (۳). به ترتیب یال‌هایی با وزن ۲ و ۴ و ۱ و ۲ و ۷ و ۸ و در نهایت ۴ انتخاب می‌شوند.

۵۷- گزینه (۲). در گراف  $G$ ، فاصله رأس ۳ تا ۶ برابر ۱۸ است ولی در درخت پوشای مینیمم این فاصله ۴۲ است:



پس گزینه‌های ۱ و ۳ نادرست هستند.

۵۸- گزینه (۳).

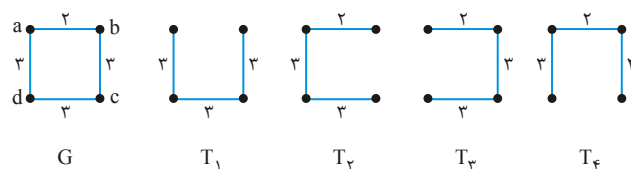


۵۹- گزینه (۴). اگر یال منفی وجود داشته باشد، این جمله لزوماً درست نیست.

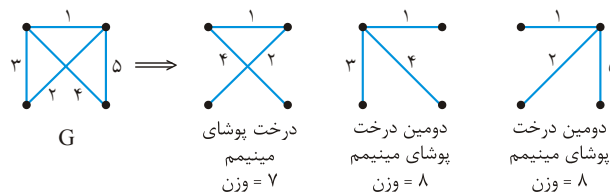
۶۰- گزینه (۴). اگر گراف  $G$  خود درخت باشد که همان درخت پوشای مینیمم است و اگر وزن همه یالها متفاوت باشد نیز درخت حاصل یکتاست.

۶۱- گزینه (۳). هدف یافتن درخت پوشای مینیمم است که گزینه ۱ و ۳ جواب می‌دهند. ولی چون گراف خلوت است، کراسکال مناسب است (گراف ۱۰ رأس دارد و ۱۳ یال)

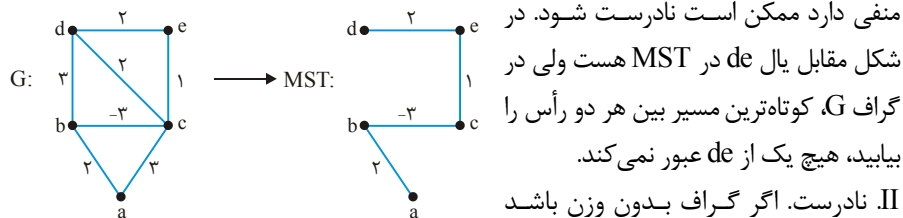
- ۶۲- گزینه (۳). با این عمل هیچ سیکلی باقی نمی ماند و شکل حاصل درخت پوشاست و چون بالهای با وزن ماکزیمم حذف شده اند، درخت پوشای کمینه حاصل می شود.
- ۶۳- گزینه (۴). یافتن درخت پوشای کمینه توسط هیچ الگوریتمی  $O(|V| + |E|)$  نیست.
- ۶۴- گزینه (۱). در یک سیکل یال با بیشترین وزن حذف می شود، البته اگر وزن همه یال ها متمایز باشد. مثلاً مثلی که وزن همه یال هایش ۵ است، هر سه یال سنگین ترین هستند ولی حتماً دو یال در MST انتخاب می شود.
- ۶۵- گزینه (۴). وزن یال BE برابر ۵ است و از AD و CD و CF کوچکتر است.
- ۶۶- گزینه (۳). جمله اول و چهارم نادرست هستند.
- ۶۷- گزینه (۲). برای درک سوال گراف G و ۴ درخت پوشای آن رسم شده اند:



- این ۴ زیر درخت همگی گلوگاه هستند چون وزن سنگین ترین یال آنها ۳ است. ولی  $T_1$  درخت فراگیر کمینه نیست. یعنی هر درخت گلوگاهی لزوماً کمینه نیست. ولی درخت فراگیر کمینه، حتماً گلوگاه هست.
- ۶۸- گزینه (۴). همانطور که ملاحظه می شود، دومین درخت پوشای مینیمم لزوماً یکتا نیست. ولی در صورت متمایز بودن وزن ها، درخت پوشای مینیمم یکتاست

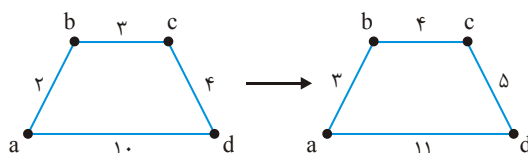


- ۶۹- گزینه (۲). I از نظر طراح این جمله درست انتخاب شده است ولی این جمله برای گرافی که یال منفی دارد ممکن است نادرست شود. در شکل مقابل یال de در MST هست ولی در گراف G، کوتاه ترین مسیر بین هر دو رأس را بیابید، هیچ یک از de عبور نمی کند. II نادرست. اگر گراف بدون وزن باشد این جمله درست است.

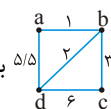


- III درست. چون گراف ساده و با وزن های متفاوت است حتماً کوچکترین و دومین کوچکترین یال در MST هستند.

۷۰- گزینه (۱). II صحیح است. III نادرست است. در شکل زیر در  $G$  کوتاه‌ترین مسیر از  $a$  به  $d$ ،  $a \rightarrow b \rightarrow c \rightarrow d$  است ولی در  $G'$ ،  $a \rightarrow d$  است.



گزاره I نیز نادرست است، در گراف  $\Delta/\Delta$  برش کمینه  $(\{b\}, \{a, c, d\})$  است که وزن  $1+2+3=6$  دارد. حال اگر به وزن هر یال یک واحد اضافه کنیم، برش کمینه  $(\{a\}, \{b, c, d\})$  است که وزن  $2+6/5=8/5$  دارد.

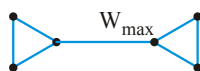


۷۱- گزینه (۳). هر دو الگوریتم درخت پوشای مینیمم پیدا می‌کنند که همین درخت کم‌قدرت‌ترین درخت پوشاست.

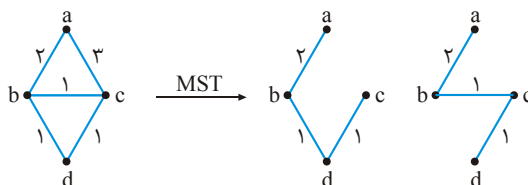
۷۲- گزینه (۳). ظاهراً باید از الگوریتم دایجسترا استفاده کنیم. ولی می‌توان هر یال با وزن  $x$  را به  $x$  تا یال با وزن ۱ تبدیل کرد. در این صورت گرافی به دست می‌آید که وزن همه یال‌هایش ۱ است و تعداد رئوس و یال‌هایش حداکثر  $C$  برابر گراف قبلی است. حال می‌توان از الگوریتم BFS استفاده کرد که مرتبه آن  $O(C(n+e))$  است که چون  $C$  ثابت است مرتبه  $O(n+e)$  است.

۷۳- گزینه (۴). با BFS می‌توان کوتاه‌ترین فاصله‌ها را از یک نقطه شروع به دست آورد و سپس ماکزیمم آنها را یافت.

۷۴- گزینه (۲). جمله اول نادرست است زیرا ممکن است یال با وزن  $w_{\max}$  یال پل (bridge) باشد که حتماً باید در MST باشد. مانند گراف مقابل:

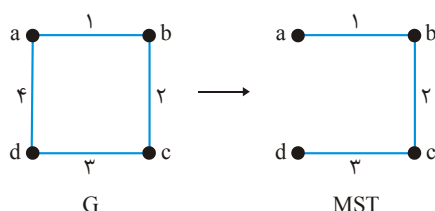


جمله دوم درست است. طبق کراسکال یکی از  $w_{\min}$  ها حتماً انتخاب می‌شود. جمله سوم کمی ابهام دارد ولی با توجه به گراف مقابل سیکل  $abc$  فقط یک یال با وزن (یال  $bc$ )  $w_{\min} = 1$  دارد ولی یال  $bc$  لزوماً در همه MST ها نیست.





۷۵- گزینه (۳). الف) نادرست است. در گراف G کوتاهترین مسیر a به d از یال ad می‌گذرد ولی



این یال در MST نیست.

ب) درست است. اگر وزن یالها متمایز باشد،

آنگاه MST منحصر به فرد است ولی عکس

این مطلب درست نیست.

۷۶- گزینه (۴).

$$A \rightarrow D \rightarrow F \rightarrow G \rightarrow H$$

$$4 + 3 + 2 + 1 = 10$$

۷۷- گزینه (۳).

$$1 \rightarrow 4 \rightarrow 5 \rightarrow 2$$

$$10 + 15 + 20 = 45$$

$$1 \rightarrow 3 \quad 45$$

۷۸- گزینه (۱).

$$a \rightarrow b \rightarrow c \rightarrow z \quad 2 + 2 + 1 = 5$$

۷۹- گزینه (۱).

۸۰- گزینه (۳). با لیست اگر الگوریتم دایکسترا از heap دودویی استفاده کند، مرتبه آن

$O(e \log v)$  خواهد بود. این الگوریتم برای گرافهای جهت‌دار و بدون جهت که یال منفی

ندارند درست کار می‌کند.

۸۱- گزینه (۴). با توجه به گزینه‌ها رأس شروع ۱ است و انتخاب سایر رئوس طبق دایکسترا به

راحتی انجام می‌شود.

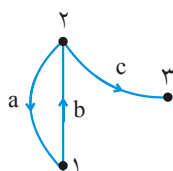
۸۲- گزینه (۲). با این عمل ممکن است کوتاهترین مسیرها بین رئوس تغییر کند.

۸۳- گزینه (۴). BFS می‌تواند کوتاهترین مسیرهای هم مبدأ برحسب تعداد یال را بیابد.

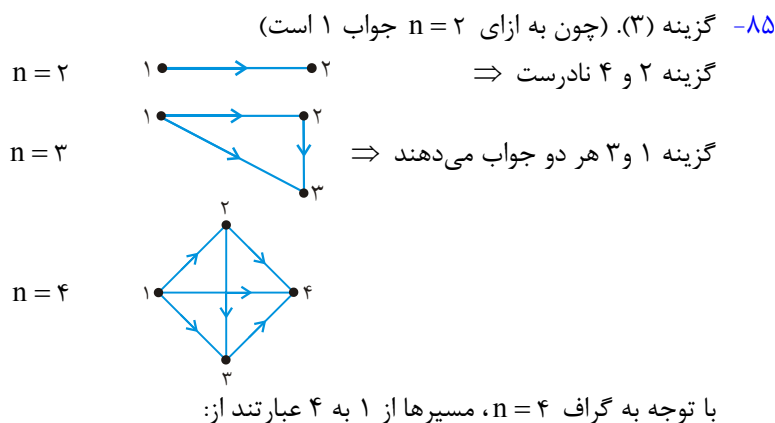
۸۴- گزینه (۴). اگر  $i = j$  درایه  $c_{ij}$  برابر جمع درجات ورودی و خروجی رأس نظیر را نشان

می‌دهد و اگر  $i \neq j$ ، قرینه  $c_{ij}$  برابر تعداد یالهای بین رئوس  $i$  و  $j$  است.

$$B = \begin{bmatrix} 1 & 1 & -1 & 0 \\ -1 & 1 & -1 & \\ 0 & 0 & 1 & \end{bmatrix}$$



$$t_B = \begin{bmatrix} 1 & -1 & 0 \\ -1 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix}, \quad B \cdot t_B = \begin{bmatrix} 1 & 2 & -2 & 0 \\ -2 & 3 & -1 & \\ 3 & -1 & 1 & \end{bmatrix}$$



$[1, 4], [1, 2, 4], [1, 3, 4], [1, 2, 3, 4]$

که گزینه ۳ جواب می‌دهد.

۸۶- گزینه (۴ و ۲). اگر در پیمایش DFS یا cross تولید شود آنگاه گراف سیکل همیلتنی

ندارد. وجود یا عدم وجود یال cross ربطی به مسیر یا مدار اولیه ندارد.

۸۷- گزینه (۱). طبق فرمول اویلر برای گراف‌های مسطح (به کتاب ساختمان گسسته پوران

پژوهش رجوع کنید) رابطه  $r = e - n + 2$  برقرار است که  $r$  تعداد نواحی است. یکی از

نواحی ناحیه نامتناهی است و سایر نواحی، سیکل هستند.

یعنی تعداد سیکل‌ها یکی از تعداد نواحی کمتر است مثلاً در (۳)

گراف زیر ۳ ناحیه وجود دارد (  $r = n - e + 2 = 3$  ) و ۲ سیکل. پس با  $O(1)$  می‌توان تعداد سیکل‌ها را محاسبه کرد.

۸۸- گزینه (۲). به عنوان مثال الگوریتم را روی گراف  $1 \rightarrow 2 \rightarrow 3$  اجرا کنید در این

صورت mystery برای رأس ۱ برابر ۲ و برای رأس ۲ برابر ۱ می‌شود.

|   |      |      |      |         |
|---|------|------|------|---------|
|   | V[1] | V[2] | V[3] |         |
| V | T    | T    | T    | Visited |
|   | 2    | 1    | 0    | mystery |

۸۹- گزینه (۲). درایه‌های  $A^k$  برابر تعداد مسیرهای به طول  $k$  بین رئوس گراف است.

۹۰- گزینه (۴). به‌طور پیش فرض گراف را با لیست مجاورتی بسازیم. در این تست گراف خلوت

است (  $e \in O(n)$  ) پس پریم و دایجسترا (  $O(n \cdot \lg n)$  )، بلمن فورد (  $O(n^2)$  )، مسیر در DAG

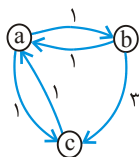
مرتبه  $O(n)$  است.

۹۱- گزینه (۳). الف) درست. چون دور نداریم پس با منفی کردن یالها دور منفی ایجاد نمی‌شود و

اگر کوتاهترین مسیر بین  $s$  و  $t$  را بیابیم در واقع طولانی‌ترین مسیر بین  $s$  و  $t$  در گراف اصلی

را یافته‌ایم. البته فلوید الگوریتم خوبی برای این منظور نیست و ترتیب توپولوژیکی (طبق درس) بهتر است. در درخت همان مسیر P است.

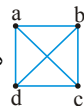
۹۲- گزینه (۴). در کلید اولیه گزینه‌های ۲ و ۳ درست اعلان شد ولی در صورت سوال اشکال هست. اگر از رأس i به رأس j یالی نباشد آنگاه  $A[i, j] = \infty$  آنگاه گزینه ۲ و ۳ صحیح می‌باشد و چون این نکته ذکر نشده است، گزینه ۴ کلید نهایی درست اعلام شد.



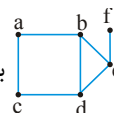
$$A^2 = \begin{matrix} & \begin{matrix} a & b & c \end{matrix} \\ \begin{matrix} a \\ b \\ c \end{matrix} & \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 3 \\ 1 & \infty & 0 \end{bmatrix} \end{matrix} \times \begin{matrix} & \begin{matrix} a & b & c \end{matrix} \\ \begin{matrix} a \\ b \\ c \end{matrix} & \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 3 \\ 1 & \infty & 0 \end{bmatrix} \end{matrix} = \begin{matrix} & \begin{matrix} a & b & c \end{matrix} \\ \begin{matrix} a \\ b \\ c \end{matrix} & \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 2 \\ 1 & 2 & 0 \end{bmatrix} \end{matrix}$$

۹۳- گزینه (۲). یال جدید را به درخت فراگیر کمینه اضافه کنید با این عمل یک دور ایجاد می‌شود و تعداد یالها با تعداد رئوس برابر می‌شود، حال از دور ایجاد شده یال با وزن ماکزیمم را حذف کنید که زمان این عمل به تعداد یالها بستگی دارد که با تعداد رئوس برابر است یعنی از مرتبه  $O(|V|)$  است. دقت کنید گزینه ۱ به این علت صحیح نیست که  $|E|$  تعداد یال گراف اولیه است که با  $|V|$  برابر نیست.

۹۴- گزینه (۴ و ۲). در گراف کامل ترتیب ملاقات رئوس توسط DFS و BFS می‌تواند یکسان

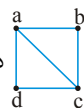


باشد. مثلاً در گراف  $K_4$  هم BFS و هم DFS می‌توانند  $\langle a b c d \rangle$  (چپ به راست) باشند. قطر یعنی ماکزیمم کمترین فاصله‌ها. مثلاً قطر گراف کامل ۱ است. یا قطر



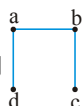
گراف برابر ۳ (فاصله a و f) است. گزینه ۲ غلط است زیرا مثلاً در گراف

۳ گزینه ۳ است. BFS و DFS یکسان هستند حال آنکه قطر این گراف برابر ۳ است. گزینه ۳ صحیح است زیرا گراف G می‌تواند مثلاً  $K_{1,n}$  باشد که BFS و DFS آن یکسان است. گزینه

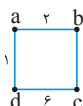


۴ غلط است مثلاً گراف درخت و گراف کامل نیست ولی BFS و DFS آن می‌تواند  $\langle a b c d \rangle$  باشد. (چپ به راست).

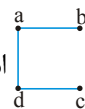
۹۵- گزینه (۴). درخت پوشای کمینه (MST) ارتباطی با درخت کوتاهترین مسیر (SPT) ندارد.



مثلاً گراف ۱ را در نظر بگیرید. MST این گراف است ولی SPT این گراف

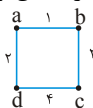


با مبدأ  $d$ ، است. اگر درجه رأس  $S$  در خود گراف یک باشد، حتماً در SPT و در MST نیز یک خواهد بود پس گزینه ۳ غلط است. SPT برای هر رأس ممکن است ستاره‌ای شود که گزینه ۴ صحیح است.



گزینه ۲. در هر نوع گرافی از نظر شکل، با توجه به رأس شروع ممکن است ترتیب کراسکال و پریم یکسان شود یعنی گراف لزوماً درخت یا گراف کامل نیست. مثلاً در گراف

ترتیب ملاقات یالها توسط کراسکال  $ab$  و  $ad$  و  $bc$  است. و اگر پریم از  $a$  شروع کند، همین ترتیب را دارد. لزوماً وزن یالها متمایز نیستند. یعنی چه وزن‌ها متمایز و چه نامتمایز باشند، ممکن است ترتیب کراسکال با پریم یکسان شود.



گزینه ۳. به ترتیب ۱۱ و ۲۳ و ۳۶ و ۶۴ انتخاب می‌شوند.

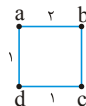
گزینه ۴.

گزینه ۲. گراف  $n$  رأسی که  $O(n)$  یال دارد با حداکثر  $O(\sqrt{n})$  رنگ، قابل رنگ‌آمیزی است.

گزینه ۳. سه گزاره اول را می‌توان با DFS حل کرد. برای تشخیص دو بخشی بودن، به رأس شروع شماره ۰ دهید. و اگر رأس با شماره ۰ رأس  $x$  را ملاقات کرد، به  $x$  شماره ۱ نسبت دهید و اگر رأس با شماره ۱ رأس  $x$  را ملاقات کرد، به  $x$  شماره ۰ دهید. پس از پایان الگوریتم اگر دو سریالی هم شماره باشد، یعنی گراف دو بخشی نیست.

در مورد گزاره دوم، شرط لازم و کافی برای وجود دور در گراف آن است که پیمایش DFS، یال پشتی تولید کند. در مورد گزاره سوم، ابتدا روی گراف DFS را اجرا کنید و زمان شروع و پایان برای هر رأس را ثبت کنید و سپس روی ترانواده گراف (جهت یالها را عوض کنید) مجدداً پیمایش DFS اجرا کنید ولی به ترتیب نزولی زمان‌های پایان اجرا کنید و مولفه‌های متصل قوی گراف جهت‌دار، بدست می‌آیند.

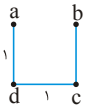
گزینه ۳. گزاره‌های اول و دوم صحیح هستند. برش یعنی افراز رئوس  $V$  به دو مجموعه جدا از هم  $S$  و  $V-S$ . همیشه سبک‌ترین یال برش، عضو MST است. منظور از یال برش، یالی است که رئوس مجموعه  $S$  را به  $V-S$  متصل می‌کند. گزاره سوم صحیح نیست. مثلاً در



گراف ۱ مسیره‌ای از  $a$  و  $b$  عبارتند از  $a \rightarrow b$  و  $a \rightarrow d \rightarrow c \rightarrow b$  که در اولی

سبک‌ترین یال وزن ۲ دارد و در دومی، وزن سبک‌ترین یال برابر ۱ است، بیشینه این‌ها برابر

۲ است ولی یال با وزن ۲ در MST نیست. زیرا MST این گراف ۱ است.

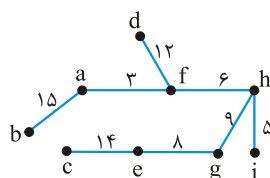


۱۰۲- گزینه (۲). کمینه تعداد مراحل وقتی است که یالها به ترتیب ابتدا  $(v_1, v_2)$  سپس  $(v_2, v_3)$  و ... و  $(v_{99}, v_{100})$  بررسی (relax) شوند، در این صورت همان بار اول، کوتاهترین مسیرها بدست می‌آید و بار دوم تغییر نمی‌کند. بیشینه تعداد مراحل وقتی است که یالها بر عکس بررسی شوند. یعنی ابتدا  $(v_{98}, v_{99})$  و به همین ترتیب در نهایت  $(v_1, v_2)$ . در این صورت ۹۹ بار همه یالها relax می‌شوند  $(n-1)$  بار و تغییر ایجاد می‌شود. ولی بار صدم هیچ تغییری ایجاد نمی‌شود.

۱۰۳- گزینه (۴). این نکته را در درس دیده‌اید و نیاز است به خاطر بسپارید.

۱۰۴- گزینه (۲). فقط گزاره اول درست است. گراف  $n$  راسی فاقد دور، دقیقاً دارای  $n$  مولفه همبند قوی است یعنی هر رأس خود یک مولفه متصل قوی است. گزاره دوم غلط است، یک گراف  $n$  راسی که به شکل یک دور به طول  $n$  است، تصور کنید. این گراف دارای یک مولفه است، حال اگر یک یال از این گراف حذف شود آنگاه شامل  $n$  مولفه (هر رأس) می‌شود. وقتی یالی از گراف حذف می‌شود. مولفه‌ها یا تغییر نمی‌کند و یا زیاد می‌شود. گزاره سوم غلط است و فقط با DFS این عمل امکان‌پذیر است.

۱۰۵- گزینه (۲). با هر الگوریتمی مثل کراسکال یا پریم، شکل MST این گراف به صورت زیر است که هزینه‌اش ۷۲ است:



$$3 + 5 + 6 + 8 + 9 + 12 + 14 + 15 = 72$$

۱۰۶- گزینه (۴). گراف  $k_n$  دارای  $\frac{n(n-1)}{2}$  یال است و درخت پوشای آن نیاز به  $n-1$  یال دارد پس می‌توان  $k_n$  را به  $\frac{n}{2}$  درخت پوشا، افراز کرد.

۱۰۷- گزینه (۱). مجموع درجات ۲ برابر تعداد یالها است و درخت  $n$  راسی دارای  $n-1$  یال است، پس حاصلضرب  $d_1 + d_2 + \dots + d_n$  برابر  $2n-2 = 2(n-1)$  است.

۱۰۸- گزینه (۴). اگر وزن یکسان در گراف باشد آنگاه لزوماً یکتا نیست.  $T$  حتماً شامل همه رئوس گراف  $G$  هست. کوتاهترین مسیر بین دو راس در  $T$  لزوماً با  $G$  یکسان نیست.

۱۰۹- گزینه (۱). برای کوتاه‌ترین مسیر، اصل بهینگی برقرار است و برای طولانی‌ترین مسیر، این اصل برقرار نیست. برای مطالعه به بحث برنامه‌نویسی پویا، کتاب طراحی الگوریتم پوران پژوهش مراجعه کنید.

۱۱۰- گزینه (۲). گزاره اول نادرست است مثلاً ممکن است  $G - v$  ناهمبند باشد و اصلاً MST نداشته باشد مثلاً



برای گراف  $G$ ،  $T$  درخت پوشای کمینه است ولی  $G - v$  ناهمبند است و  $T - v$  درخت پوشای کمینه آن نیست.

۱۱۱- گزینه (۴). تغییر وزن در این گزینه  $g$  هر چه باشد، مسیرها را تغییر نمی‌دهد.

۱۱۲- گزینه (۱). شار بیشینه  $2x$  است و اگر هر بار مسیری شامل وزن یک انتخاب شود سپس از  $2x$  مرحله به جواب می‌رسیم.

۱۱۳- گزینه (۴). الگوریتم BFS را با مبدأ ریشه درخت اجرا کرده و سپس مجدد BFS را از دورترین برگ اجرا می‌کنیم که در این اجرا، دورترین برگی که حاصل می‌شود، فاصله‌اش تا برگ مبدأ قطر درخت است.

۱۱۴- گزینه (۳). پاسخ از کتاب نارنجی گسسته پاسخ تست ۵ فصل ۱۱ صفحه ۲۲۹ برداشته شود.

۱۱۵- گزینه (۲).  $k_n$  دارای  $n^{n-2}$  درخت پوشاست و گراف گفته شده در سوال دارای  $n^{2n-4} = n^{n-2} \times n^{n-2}$  درخت پوشاست.

۱۱۶- گزینه (۴). به راحتی می‌توان بررسی کرد که رأس اول ۱ و رأس ششم ۵ است.

۱۱۷- گزینه (۳). دو گزاره اول با استقراء قابل اثبات هستند و گزاره سوم نادرست است.

۱۱۸- گزینه (۲). برای یافتن قطر یک DAG کافی است ترتیب توپولوژیکی بیابیم بیشترین فاصله را طبق ترتیب محاسبه کنیم.

۱۱۹- گزینه (۲). کوتاه‌ترین مسیرها ممکن است تغییر کند و MST تغییر نمی‌کند.

۱۲۰- گزینه (۲). الف و ب نادرست هستند که بسیار تکراری هستند. ج درست است. با مرتبه خطی  $(O(n + e))$  می‌توان همبند بودن و درجات رؤس را بررسی کرد.

۱۲۱- گزینه (۴). تعداد درخت پوشای  $k_n$  برابر  $n^{n-2}$  است (در درس گسسته می‌خوانید). تعداد درخت پوشای گراف داده شده  $4^2 \times 3^1 \times 4^2 = 768$  است.

۱۲۲- گزینه (۴). فرض کنید  $e = \{x, y\}$  یعنی رئوس دو سریال  $e$  رأس‌های  $x$  و  $y$  هستند. از رأس  $x$  پیمایش عمقی اجرا کنید ولی فقط یالهایی را در این پیمایش در نظر بگیرید که دارای وزن کمتر یا مساوی  $e$  هستند اگر و فقط اگر پس از پایان،  $x$  و  $y$  به هم متصل شدند آنگاه  $e$  نمی‌تواند در MST باشد زیرا در گراف سیکلی وجود دارد که  $e$  بیشینه آن است. مرتبه این عملیات خطی است.

## فصل ۸

## روش‌های حریصانه (greedy)

## مقدمه

الگوریتم حریصانه یا آزمند، به ترتیب داده‌ها را گرفته و هر بار عنصری را که طبق معیار معینی «بهترین» به نظر می‌رسد، بدون توجه به انتخاب‌های قبلی و انتخاب‌هایی که در آینده انجام خواهد شد، انتخاب می‌کند. این الگوریتم‌ها بسیار ساده هستند و معمولاً برای مسائل بهینه‌سازی کاربرد دارند. در این الگوریتم‌ها، مجموعه‌ای وجود دارد و باید از آن مجموعه، مرحله به مرحله اعضا را انتخاب (select) کرد. با هر انتخاب بررسی می‌شود که آیا عضو انتخاب شده، امکان رسیدن به جواب نهایی را دارد یا خیر (feasible). با هر انتخاب بررسی می‌شود که آیا به جواب نهایی رسیده‌ایم یا خیر (Solution).

شکل کلی این الگوریتم‌ها به صورت کد زیر می‌باشد:

```
Set greedy (set C)
{
  S =  $\phi$  ;
  while (!solution(S) && C !=  $\phi$ )
  {
    x = select(C) ;
    C = C - {x} ;
    if (feasible (S  $\cup$  {x}))
      S = S  $\cup$  {x} ;
  }
  if (solution (S))
    return (S) ;
  else
    return ( $\phi$ ) ;
}
```

C مجموعه انتخاب‌های ممکن است، S مجموعه جواب است که در ابتدا تهی است، Solution بررسی می‌کند که جواب حاصل شده است یا خیر، select یک عضو از مجموعه C انتخاب می‌کند و آن عضو را از C حذف می‌کند، feasible بررسی می‌کند که آیا امکان اضافه شدن x به جواب وجود دارد یا خیر. اگر امکان داشت x به مجموعه جواب S اضافه می‌شود. اگر جواب حاصل شد S خروجی است در غیر این صورت  $\phi$  برگشت داده می‌شود.

برای آنکه با مفهوم الگوریتم‌های حریصانه بیشتر آشنا شویم یک مثال مطرح می‌کنیم.



**مثال ۱:** فرض کنید فروشنده‌ای می‌خواهد بقیه پول شما را که ۳۶ واحد است پرداخت کند.

او می‌خواهد تعداد سکه‌های پرداختی حداقل باشد. فرض کنید سکه‌های موجود عبارتند از ۲۵ و ۱۰ و ۵ و ۱ واحدی. فروشنده ابتدا بزرگترین سکه موجود یعنی ۲۵ واحدی را انتخاب می‌کند (بهینه محلی). با انتخاب یک سکه ۲۵ واحدی دیگر جواب حاصل نخواهد شد بنابراین او یک سکه ۱۰ واحدی انتخاب می‌کند تاکنون ۳۵ واحد انتخاب شده. با انتخاب سکه ۱۰ واحدی دیگر و ۵ واحدی جواب حاصل نمی‌شود، بنابراین فروشنده سکه ۱ واحدی را انتخاب می‌کند. بنابراین مجموعه جواب این مسئله {۱ و ۱۰ و ۲۵} می‌باشد که جواب بهینه نیز هست.

حال فرض کنید بجای سکه‌های ۲۵ و ۱۰ و ۵ و ۱ واحدی، سکه‌های ۱۲ و ۱۰ و ۵ و ۱ واحدی وجود می‌داشت و بقیه پول شما ۱۶ واحد باشد. باروش حریصانه یک سکه ۱۲ واحدی و ۴ سکه ۱ واحدی دریافت می‌کنید یعنی ۵ سکه. حال آنکه جواب بهینه در این حالت شامل یک سکه ۱۰ واحدی، یک سکه ۵ واحدی و یک سکه ۱ واحدی است یعنی فقط ۳ سکه.

پس روش حریصانه برای خرد کردن سکه لزوماً جواب صحیح نمی‌دهد و بستگی به نوع سکه‌ها دارد.

در این مثال، مجموع انتخاب‌های ممکن یعنی C، سکه‌های موجود می‌باشد. تابع Select هر بار یک سکه انتخاب می‌کند. تابع feasible با هر انتخاب بررسی می‌کند که آیا انتخاب انجام شده امکان پذیر هست یا خیر. اگر امکان پذیر نباشد آن انتخاب برای همیشه طرد می‌شود. تابع Solution با هر انتخاب بررسی می‌کند که آیا جواب حاصل شده است یا خیر.

الگوریتم‌هایی که به روش حریصانه بررسی می‌شوند عبارتند از:

- ۱- مساله کوله پشتی غیرصفر و یک ۲- کدهافمن ۳- زمان‌بندی بر مبنای کمینه کردن زمان کل ۴- انتخاب (زمان‌بندی) فعالیت‌ها ۵- زمان‌بندی فعالیت‌ها با مهلت معین
- الگوریتم‌های حریصانه دیگری مثل کراسکال (راشال) - پریم - دایجسترا در فصل گراف بررسی شدند.

### ۱- مساله کوله‌پشتی غیرصفر و یک (کوله پشتی کسری)

یک کوله پشتی که حداکثر ظرفیت آن جرم M است در نظر بگیرید. n تاشی با جرم‌های  $w_1, w_2, \dots, w_n$  و ارزش‌های  $p_1, p_2, \dots, p_n$  مفروضند. می‌خواهیم کوله پشتی را با اشیاء پر کنیم طوری که حداکثر سود (ارزش) بدست آید. در ضمن می‌توان از یک شی کسری را انتخاب کرد. بنابراین اگر فرض کنیم از شی i، کسر  $x_i$  انتخاب شده است ( $0 \leq x_i \leq 1$ ) هدف

$$\text{ماکزیمم کردن عبارت } \sum_{i=1}^n p_i x_i \text{ به شرطی که } \sum_{i=1}^n w_i x_i \leq M \text{ باشد.}$$

**مثال ۲:** کوله پشتی با ظرفیت جرم  $M = 20$  و ۳ شی که جرم و ارزش آنها داده شده است مفروضند. کوله پشتی را می‌خواهیم پر کنیم طوری که ارزش بدست آمده ماکزیمم باشد.

$$(w_1, w_2, w_3) = (18, 15, 10)$$

$$(p_1, p_2, p_3) = (25, 24, 15)$$

**پاسخ:** برای این مساله می‌توان با روش حریصانه اشیاء را انتخاب کرد ولی ابتدا باید یک حدس خوب برای انتخاب اشیاء بیابیم. ۳ پیشنهاد وجود دارد:

**الف)** چون ارزش اشیاء مهم است پس بهتر است اشیاء را به ترتیب بیشترین ارزش انتخاب کنیم به عنوان مثال شی ۱ را کامل انتخاب کنیم که در این صورت ظرفیت باقی مانده کوله پشتی ۲ خواهد بود بنابراین کسر  $\frac{2}{15}$  از شی ۲ را انتخاب کنیم پس:

$$(x_1, x_2, x_3) = \left(1, \frac{2}{15}, 0\right) \Rightarrow \text{ارزش بدست آمده} = 1 \times 25 + \frac{2}{15} \times 24 = 28\frac{2}{3}$$

**ب)** چون وزن اشیاء تاثیر منفی دارد بهتر است اشیاء را به ترتیب از کمترین وزن انتخاب کنیم. یعنی شی ۳ را کامل انتخاب کنیم که ظرفیت باقیمانده ۱۰ خواهد شد و از شی ۲ کسر  $\frac{10}{15}$  را انتخاب کنیم.

$$(x_1, x_2, x_3) = \left(0, \frac{2}{3}, 1\right) \Rightarrow \text{ارزش بدست آمده} = 0 \times 25 + \frac{2}{3} \times 24 + 1 \times 15 = 31$$

**ج)** بهتر است اشیاء را به ترتیب نزولی  $\frac{p_i}{w_i}$  انتخاب کنیم یعنی آن شی را اول انتخاب کنیم که  $\frac{p_i}{w_i}$  آن بیشترین است:

$$\left(\frac{p_1}{w_1}, \frac{p_2}{w_2}, \frac{p_3}{w_3}\right) = \left(\frac{25}{18}, \frac{24}{15}, \frac{15}{10}\right)$$

پس شی ۲ را باید کامل انتخاب کنیم که ظرفیت باقی مانده ۵ خواهد بود و سپس از شی ۳ کسر  $\frac{1}{3}$  را انتخاب می‌کنیم تا کوله پشتی پر شود پس:

$$(x_1, x_2, x_3) = \left(0, 1, \frac{1}{3}\right) \Rightarrow \text{ارزش بدست آمده} = 0 \times 25 + 1 \times 24 + \frac{1}{3} \times 15 = 31\frac{1}{3}$$

همانطور که ملاحظه می‌شود حدس سوم بهترین است و می‌توان ثابت کرد که حدس سوم همیشه به جواب بهینه خواهد رسید. شبه کد مسأله کوله‌پشتی کسری در ادامه آمده است.

```

knapsack (p[۱..n], w[۱..n], x[۱..n])
pwsort (p, w, n)
for (i = ۱ to n) x[i] = ۰;
rc = M;
for (i = ۱ to n)
{
    if (w[i] > rc) break;
    x[i] = ۱;
    rc = rc - w[i];
}
if (i <= n)
    x[i] =  $\frac{rc}{w[i]}$ ;
}

```

n تعداد اشیاء، M ظرفیت کوله و متغیر rc ظرفیت باقیمانده کوله را نشان می‌دهند. آرایه‌های p و w به ترتیب ارزش اشیاء، وزن اشیاء و کسر انتخابی از اشیاء را نشان می‌دهند. تابع pwsort  $\frac{p}{w}$  ها را به صورت نزولی مرتب می‌کند. یعنی

$$\frac{p[۱]}{w[۱]} \geq \frac{p[۲]}{w[۲]} \geq \dots \geq \frac{p[n]}{w[n]}$$

در این الگوریتم اشیاء از ۱ تا n شماره‌گذاری شده‌اند. rc ظرفیت باقیمانده کوله است که در ابتدا rc = M می‌باشد. اشیاء یکی یکی انتخاب می‌شوند تا اینکه یا اشیاء تمام شوند و یا با گذاشتن شی جاری از ظرفیت مجاز کوله، تجاوز کنیم. در وضعیت

آخر، کسر  $\frac{rc}{w[i]}$  از شی i (شیئی که افزودن کامل آن امکان‌پذیر نیست) به کوله افزوده می‌شود.

مرتبه زمانی الگوریتم، به مرتب‌سازی بستگی دارد که برابر  $O(n \lg n)$  می‌باشد.

**مثال ۳:** چگونه می‌توان با کمک میانه‌گیری، مسئله کوله پشتی کسری را با زمان خطی  $O(n)$  حل کرد؟

**پاسخ:** میانه عناصر  $\frac{p_i}{w_i}$  را با مرتبه خطی بیابید و m بنامید. عناصر را ۳ دسته کنید:

G: عناصری که  $\frac{p_i}{w_i}$  آنها از m بزرگ‌تر است.

E: عناصری که  $\frac{p_i}{w_i}$  آنها با m مساوی است.

L: عناصری که  $\frac{p_i}{w_i}$  آنها از m کوچک‌تر است.

سپس  $W_G$  را برابر جمع وزن عناصر موجود در G و  $W_E$  را برابر جمع وزن عناصر موجود در E تعریف کنید.

- اگر  $W_G > M$ ، آنگاه از G فعلاً عنصری انتخاب نمی‌شود و روی عناصر موجود در G، الگوریتم را به صورت بازگشتی فراخوانی کنید (با ظرفیت مجاز کوله برابر M. عناصر E و L را حذف کنید)

- در غیر این صورت ( $W_G \leq M$ )، تمام عناصر موجود در G را انتخاب کنید و از مجموعه E حداکثر تعدادی عناصر با محدودیت ظرفیت کوله برابر  $M - W_G$  انتخاب کنید.

- اگر  $W_G + W_E \geq M$  یعنی کوله پر شده است و کار تمام است.
- در غیر این صورت ( $W_G + W_E < M$ )، بعد از انتخاب عناصر  $G$  و  $E$  به طور کامل، روی  $L$  با محدودیت ظرفیت  $M - W_G - W_E$  فراخوانی بازگشتی انجام دهید.
- تحلیل:** هر فراخوانی بازگشتی، زمان خطی دارد و خودش ممکن است یک فراخوانی بازگشتی دیگر با حداکثر نصف اندازه انجام دهد:

$$T(n) \leq T\left(\frac{n}{2}\right) + \theta(n) \Rightarrow T(n) = O(n)$$

## ۲- کد هافمن

اگر بخواهیم متنی شامل  $n$  کاراکتر را به صورت باینری کد کنیم،  $\lceil \log_2^n \rceil$  بیت به ازای هر کاراکتر نیاز است مثلاً برای کدگذاری حروف الفبای انگلیسی (۲۶ حرف)، ۵ بیت نیاز است. در این حالت هر کاراکتر دارای طول بیتی یکسان خواهد بود. مثلاً  $a = 00000$  و  $b = 00001$  و ... به راحتی می‌توان چنین متنی را از حالت کد خارج کرد (decode) زیرا هر ۵ بیت نشان دهنده یک کاراکتر است.

کد هافمن یک کد با طول متغیر است. به این معنی که به هر کاراکتر رشته بیتی با طول متفاوت نسبت داده می‌شود. مثلاً  $a = 00$  و  $b = 101$  و  $c = 1000$  و ...

این تخصیص بیت باید دارای خاصیت پیشوند آزاد (prefix-free) باشد. یعنی هیچ کاراکتری نباید پیشوند هیچ کاراکتر دیگری باشد. مثلاً  $a = 00$  و  $b = 001$  نادرست است. هافمن با استفاده از درخت دودویی کدی پیشوند آزاد بهینه تولید می‌کند. هافمن براساس تعداد تکرار کاراکترها، به آنها بیت نسبت می‌دهد.

کاراکترهایی که فراوانی آنها زیاد است، رشته بیتی کوچک دریافت می‌کنند. با یک مثال روش هافمن را توضیح می‌دهیم.

**مثال ۴:** فرض کنید متنی شامل حروف  $a$  و  $b$  و  $c$  و  $d$  و  $e$  و  $f$  می‌باشد و فراوانی این حروف در جدول زیر آمده است، می‌خواهیم کدی بهینه برای این حروف تولید کنیم.

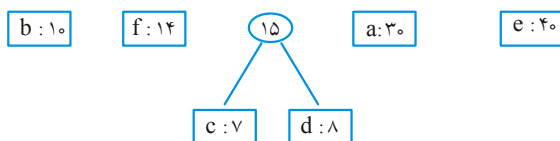
| کاراکتر | a  | b  | c | d | e  | f  |
|---------|----|----|---|---|----|----|
| فراوانی | ۳۰ | ۱۰ | ۷ | ۸ | ۴۰ | ۱۴ |

**پاسخ:** ابتدا دو کاراکتر با کمترین فراوانی انتخاب می‌شوند و با آنها یک درخت ساخته می‌شود و جمع فراوانی آنها در ریشه درخت نوشته می‌شود و آن دو فراوانی از جدول حذف می‌شوند و جمع فراوانی آنها به جدول اضافه می‌شود. سپس دو فراوانی مینیمم دیگر انتخاب می‌شوند و به همین ترتیب. مراحل اجرا در ادامه آمده است.

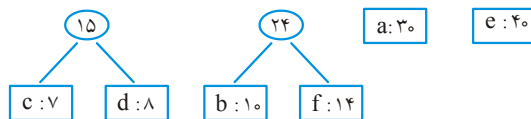
الف) فراوانی‌ها را به ترتیب صعودی مرتب می‌کنیم:

c: ۷      d: ۸      b: ۱۰      f: ۱۴      a: ۳۰      e: ۴۰

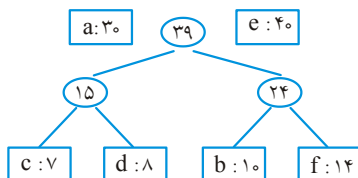
ب) c و d انتخاب می‌شوند و مجموع فراوانی آنها یعنی ۱۵ بعد از ۱۰ قرار می‌گیرد:



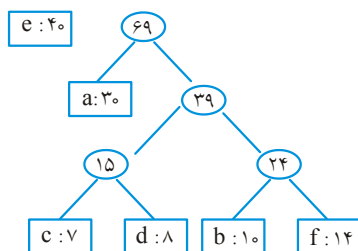
ج) حال b و f انتخاب می‌شوند و با آنها درخت ساخته می‌شود:



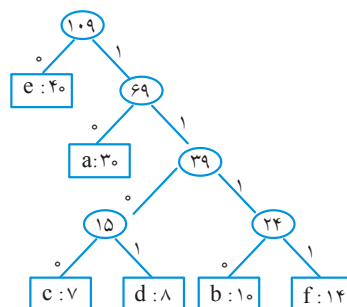
د) با ۱۵ و ۲۴ درخت ساخته می‌شود:



ه) با ۳۰ و ۳۹ درخت ساخته می‌شود:



و) در نهایت با ۴۰ و ۶۹ درخت می‌سازیم و بعد از ساخت درخت، روی یال‌های چپ و روی یال‌های راست ۱ می‌نویسیم. بدین ترتیب کد هر کاراکتر بدست می‌آید:



کد حروف مختلف عبارتند از:  $e = 0$  و  $a = 10$  و  $c = 1100$  و  $d = 1101$  و  $b = 1110$  و  $f = 1111$ . الگوریتم هافمن برای عناصر با فراوانی مینیمم از minheap استفاده می‌کند. به این صورت که با فراوانی‌ها یک minheap (صف اولویت) می‌سازد و با عمل حذف ریشه، مینیمم را انتخاب می‌کند. کد این الگوریتم به شکل زیر است:

#### HUFFMAN(C)

```
{
  n = |C| // مجموعه الفباست که n نوع کاراکتر دارد
  Q = C // صف اولویت است که با minheap و با مرتبه O(n) ساخته می‌شود
  for (i = 1 to n - 1)
  {
    allocate a new node z
    // نود z ساخته می‌شود و دو تا مینیمم x و y چپ و راست z قرار می‌گیرند.
    Left(z) = x = extract_min(Q)
    right(z) = y = extract_min(Q)
    f(z) = f(x) + f(y) // f(x) فراوانی x است.
    insert (Q, z)
  }
}
```

مرتبه این الگوریتم  $O(n \lg n)$  است. زیرا ساخت هیپ  $O(n)$  است و سپس  $n-1$  بار ۲ عمل حذف و ۱ عمل درج در هیپ انجام می‌شود که مرتبه هر حذف و درج  $O(\log n)$  است.

#### نکته

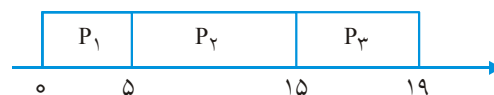
در درخت هافمن، نود یک فرزندی وجود ندارد، چون در غیر این صورت کد بهینه تولید نمی‌شود.

**تمرین:** الگوریتم هافمن را به کلمه کد مبنای ۳ تعمیم دهید (یعنی از سمبول‌های ۰ و ۱ و ۲ استفاده کنید)

**تمرین:** فرض کنید یک فایل داده‌ای شامل ۲۵۶ کاراکتر است که با روش عادی هر یک با ۸ بیت نمایش داده می‌شوند. نشان دهید اگر تعداد بیشترین تکرار از کمترین تکرار، کمتر از ۲ برابر باشد، کد هافمن مزیتی به کد عادی ثابت ۸ بیتی ندارد.

### ۳- زمان‌بندی بر مبنای کمینه کردن زمان کل

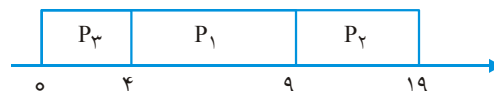
فرض کنید ۳ پردازش که زمان اجرای آنها به ترتیب  $p_1 = 5$  و  $p_2 = 10$  و  $p_3 = 4$  است را می‌خواهیم توسط یک پردازنده اجرا کنیم و می‌خواهیم مجموع زمانهای برگشت، مینیمم شود. منظور از زمان برگشت، فاصله زمانی بین ورود تا خروج یک پردازش است. اگر فرض کنیم ترتیب سرویس‌دهی به این برنامه‌ها به ترتیب از چپ به راست  $[p_1, p_2, p_3]$  باشد آنگاه نمودار زمانی زیر را خواهیم داشت:



یعنی پردازش  $p_1$  از زمان ۰ تا ۵، پردازش  $p_2$  از زمان ۵ تا ۱۵ و پردازش  $p_3$  از زمان ۱۵ تا ۱۹ اجرا شده‌اند. مجموع زمانهای برگشت این ۳ پردازش برابر است با:

$$5 + 15 + 19 = 39$$

ولی اگر ترتیب  $[p_3, p_1, p_2]$  باشد آنگاه مجموع زمانهای برگشت برابر است با:



$$4 + 9 + 19 = 32$$

به راحتی مشاهده می‌شود که اگر پردازشها به ترتیب صعودی زمانشان اجرا شوند، بهینه است. پس الگوریتم این کار درواقع الگوریتم مرتب‌سازی است که مرتبه آن  $O(n \lg n)$  است.

### ۴- انتخاب (زمان‌بندی) فعالیتها

فرض کنید مجموعه‌ای از  $n$  فعالیت  $S = \{1, 2, \dots, n\}$  پیشنهاد شده است که می‌خواهند از یک منبع مشترک استفاده کنند. هر فعالیت یک زمان شروع و یک زمان پایان دارد. زمان شروع فعالیت  $i$  برابر  $s_i$  و زمان پایان آن  $e_i$  است و  $s_i \leq e_i$  یعنی فعالیت  $i$  در فاصله زمانی  $[s_i, e_i]$  انجام می‌شود. وقتی یک فعالیت در حال انجام است، سایر فعالیت‌ها نمی‌توانند انجام شوند. بنابراین دو

فعالیت  $i$  و  $j$  به شرطی هر دو انجام می‌شوند که یا  $s_i \geq e_j$  یا  $s_j \geq e_i$  در این صورت  $i$  و  $j$  را سازگار گویند. هدف، انتخاب بیشترین تعداد از فعالیت‌های سازگار است. یک الگوریتم حریصانه برای این مسأله به صورت زیر است. فرض شده است که فعالیت‌ها طبق ترتیب صعودی زمان خاتمه‌شان مرتب شده‌اند:  $e_1 \leq e_2 \leq e_3 \leq \dots \leq e_n$ .

Algorithm Activity \_ Selection (S , e)

```

A = {1} , j = 1
for i = 2 to n do
    if S[i] ≥ e[j] then
        {
            A = A ∪ {i}
            j = i
        }
return A

```

$S$  و  $e$  آرایه‌هایی هستند که زمانهای شروع و خاتمه فعالیت‌ها را ذخیره کرده‌اند. فعالیت‌های انتخاب شده در آرایه  $A$  نگهداری می‌شوند. متغیر  $j$  معرف آخرین فعالیت انتخاب شده (اضافه شده به  $A$ ) است. اگر  $e_i$  ها از قبل صعودی مرتب باشند، مرتبه الگوریتم فوق  $\theta(n)$  است در غیر این صورت، باید مرتب‌سازی انجام شود با مرتبه  $O(n \lg n)$ .

**مثال ۵:** با توجه به جدول فعالیت‌های زیر، چه فعالیت‌هایی قابل انجام هستند؟

| i     | ۱ | ۲ | ۳ | ۴ | ۵ | ۶ | ۷  | ۸  | ۹  | ۱۰ | ۱۱ |
|-------|---|---|---|---|---|---|----|----|----|----|----|
| $s_i$ | ۱ | ۳ | ۰ | ۵ | ۳ | ۵ | ۶  | ۸  | ۸  | ۲  | ۱۱ |
| $e_i$ | ۴ | ۵ | ۶ | ۷ | ۸ | ۹ | ۱۰ | ۱۱ | ۱۲ | ۱۳ | ۱۴ |

**پاسخ:** با توجه به اینکه فعالیت‌ها از اول به صورت صعودی زمان خاتمه‌شان مرتب شده‌اند، لذا نیازی به مرتب‌سازی نیست. فعالیت ۱ انجام می‌شود در زمان (۱, ۴) فعالیت‌های ۲ و ۳ قابل انجام نیستند چون  $s$  آنها از ۴ کوچکتر است. فعالیت ۴ در زمان (۵, ۷) اجرا می‌شود. پس فعالیت ۸ در زمان (۸, ۱۱) و در نهایت فعالیت ۱۱، یعنی فعالیت‌های ۱ و ۴ و ۸ و ۱۱ انجام می‌شوند.

**سوال:** فرض کنید بجای انتخاب فعالیتی که زمان خاتمه‌اش از بقیه کوچکتر است، فعالیتی انتخاب شود که زمان شروعش از بقیه بیشتر است، آیا جواب صحیح را می‌دهد؟

## ۵- زمان‌بندی فعالیت‌ها با مهلت معین (Scheduling with Dead Lines)

مجموعه‌ای از  $n$  فعالیت (job) مفروض است. این فعالیت‌ها همگی توسط یک پردازنده باید انجام شوند و هر کدام یک واحد زمانی طول می‌کشند. این فعالیت‌ها هر کدام دارای یک مهلت  $d_i$  و



یک ارزش یا سود  $p_i$  هستند و سود  $p_i$  در صورتی حاصل می‌شود که فعالیت مورد نظر در مهلت  $d_i$  انجام شود. مثلاً اگر فعالیتی دارای مهلت ۲ و سود ۴۰ باشد، سود ۴۰ فقط در صورتی حاصل می‌شود که این فعالیت در زمان ۱ یا ۲ انجام شود و اگر در زمان ۳ یا بیشتر انجام شود هیچ سودی حاصل نمی‌شود.

**مثال ۶:** فرض کنید  $n = 4$  فعالیت به شرح  $(p_1, p_2, p_3, p_4) = (100, 10, 15, 27)$  و  $(d_1, d_2, d_3, d_4) = (2, 1, 2, 1)$  انواع زمان‌بندی‌های ممکن و سود حاصل عبارت است از:

|   | زمان‌بندی            | بهره کل          |
|---|----------------------|------------------|
| ۱ | $[1, 3]$ یا $[3, 1]$ | $100 + 15 = 115$ |
| ۲ | $[2, 1]$             | $100 + 10 = 110$ |
| ۳ | $[2, 3]$             | $10 + 15 = 25$   |
| ۴ | $[4, 1]$             | $100 + 27 = 127$ |
| ۵ | $[4, 3]$             | $27 + 15 = 42$   |
| ۶ | $[1]$                | ۱۰۰              |
| ۷ | $[2]$                | ۱۰               |
| ۸ | $[3]$                | ۱۵               |
| ۹ | $[4]$                | ۲۷               |

ملاحظه می‌شود که جواب ۴ بهینه است و بیشترین سود حاصل ۱۲۷ است. یعنی اول فعالیت ۴ و بعد فعالیت ۱ انجام شوند. دقت کنید فعالیت‌های  $[4]$  و  $[2]$  قابل اجرا نیستند زیرا هر دو باید در زمان ۱ انجام شوند که امکان‌پذیر نیست. ولی فعالیت‌های ۱ و ۳ امکان‌پذیر است زیرا هر دو مهلت‌شان زمان ۲ است یعنی می‌توان یکی را در زمان ۱ و دیگری را در زمان ۲ انجام داد.

### الگوریتم

کارها را برحسب ترتیب نزولی سودشان مرتب می‌کنیم و به ترتیب آنها را انتخاب می‌کنیم و هر کار که انتخاب می‌شود را به‌طور موقت به مجموعه فعالیت‌های انتخاب شده تاکنون ( $j$ ) اضافه می‌کنیم و بعد بررسی می‌کنیم مجموعه جدید، امکان‌پذیر است یا خیر. برای بررسی امکان‌پذیر بودن کافیه کارهای موجود در  $J$  را به ترتیب صعودی مهلت‌هایشان مرتب کنیم و فقط امکان‌پذیر بودن این حالت را بررسی کنیم یعنی لازم نیست همه جایگشت‌ها را بررسی کنیم.

**مثال ۷:** بهترین زمان‌بندی برای مقادیر زیر را جهت به دست آوردن حداکثر سود بیابید.

$$(p_1, p_2, p_3, p_4, p_5, p_6, p_7) = (40, 35, 30, 25, 20, 15, 10)$$

$$(d_1, d_2, d_3, d_4, d_5, d_6, d_7) = (3, 1, 1, 3, 1, 3, 2)$$

**پاسخ:** سودها به صورت نزولی مرتب هستند.

۱- ابتدا مجموعه  $J$  را برابر  $\varnothing$  قرار می‌دهیم  $J = \{\}$

۲- کار ۱ را انتخاب می‌کنیم  $J = \{1\}$ ، امکان‌پذیر هست.

۳- کار ۲ را انتخاب می‌کنیم  $J = \{1, 2\}$  قبول می‌شود زیرا ترتیب  $[2, 1]$  امکان‌پذیر است.

۴- کار ۳ انتخاب نمی‌شود چون ترتیب  $[3, 2, 1]$  با مهلت‌های  $[1, 1, 3]$  امکان‌پذیر نیست.

۵- کار ۴ انتخاب می‌شود و  $J = \{1, 2, 4\}$ . این مجموعه با توجه به ترتیب  $[2, 1, 4]$  و مهلت‌های  $[1, 3, 3]$  قابل قبول است.

۶- کار ۵ انتخاب نمی‌شود زیرا ترتیب  $[2, 5, 1, 4]$  با توجه به مهلت‌های  $[1, 1, 3, 3]$  امکان‌پذیر نیست.

۷- کار ۶ انتخاب نمی‌شود زیرا ترتیب  $[2, 1, 4, 6]$  با توجه به مهلت‌های  $[1, 3, 3, 3]$  امکان‌پذیر نیست.

۸- کار ۷ انتخاب نمی‌شود زیرا ترتیب  $[2, 7, 1, 4]$  با توجه به مهلت‌های  $[1, 2, 3, 3]$  امکان‌پذیر نیست.

پس جواب مسأله، مجموعه  $J = \{1, 2, 4\}$  و ترتیب ممکن  $[2, 1, 4]$  یا  $[2, 4, 1]$  است و سود کل  $100 = 40 + 35 + 25$  است.

بنابراین الگوریتم زمان‌بندی فعالیتها با مهلت معین، ابتدا باید  $n$  کار را به ترتیب نزولی سودشان مرتب کند که زمان  $\theta(n \lg n)$  نیاز دارد و سپس در هر مرحله بررسی کند که آیا انتخاب انجام شده، امکان‌پذیر است یا خیر که مرتبه این عمل  $\theta(n^2)$  است. پس مرتبه کل  $\theta(n^2 + n \lg n)$  یا  $\theta(n^2)$  است.

#### نکته

با استفاده از ساختمان داده مجموعه‌های مجزا می‌توان الگوریتمی با مرتبه  $\theta(n \lg n + n \cdot \alpha)$  در حدود  $lg^*$  (است) برای این مسئله ارائه داد. الگوریتم به این صورت است که ابتدا فعالیتها براساس سود به صورت نزولی مرتب شوند و سپس با حفره‌ها (مهلت‌ها) مجموعه‌های تک عضوی ساخته شود و اگر مهلت فعالیت  $x$  است از حفره  $x$  به سمت حفره ۱ بررسی شود و فعالیت در اولین حفره خالی اجرا شود و سپس آن حفره با حفره خالی قبل از خودش اجتماع گرفته شود.

## خلاصه فصل

- ۱- روش‌های حریصانه برای حل برخی مسائل بهینه‌سازی کاربرد دارند. این روش‌ها بسیار ساده هستند و قدرت زیادی ندارند.
- ۲- الگوریتم‌های کراسکال، پریم، دایجسترا، کوله‌پشتی کسری، هافمن همگی حریصانه هستند.
- ۳- کوله‌پشتی کسری با  $n$  شی با روش حریصانه با مرتبه  $O(n \lg n)$  قابل حل است چون باید مرتب‌سازی انجام شود. ولی روش تقسیم و غلبه‌ای وجود دارد که این مسئله را با مرتبه  $O(n)$  حل می‌کند.
- ۴- کد هافمن یک کد بهینه پیشوندی است و با مرتبه  $O(n \lg n)$  می‌توان برای  $n$  نوع کاراکتر، این کد را تولید کرد.
- ۵- مسئله انتخاب فعالیت‌ها با مهلت معین با مرتبه  $O(n \lg n)$  قابل حل است.

## [سؤال‌های چهارگزینه‌ای فصل هشتم]

۱- متنی شامل ۷۰۰۰ حرف از حروف a و b و c و d و e و f با دفعات تکرار  $a=1000$  و  $b=1200$  و  $c=800$  و  $d=1500$  و  $e=1800$  و  $f=700$  موجود است. چنانچه کدی بهینه برای حروف بالا انتخاب نماییم، تعداد کل بیت‌های لازم برای تبدیل متن مذکور به مجموعه ای از بیت‌ها چقدر است؟

(دولتی ۸۱)

- (۱) ۱۴۶۰۰ (۲) ۱۷۷۰۰ (۳) ۲۴۳۰۰ (۴) ۳۵۲۰۰

۲- حداکثر طول یک کد برای n عنصر که به روش هافمن کدگذاری می‌شوند چقدر است؟  
(تخصصی نرم افزار دولتی ۸۲ و ۸۵)

- (۱)  $n-2$  (۲)  $\lceil \lg n \rceil$  (۳)  $n-1$  (۴)  $\left\lceil \frac{n}{2} \right\rceil$

۳- پنج فایل مرتب شده به اندازه‌های ۵ و ۱۰ و ۲۰ و ۲۵ و ۳۰ مفروضند. می‌خواهیم از ادغام دو به دوی آنها یک فایل مرتب شده واحد شامل همه رکوردها به دست آوریم. در هر ادغام رکوردهای فایل‌های ورودی ممکن است چند بار از یک فایل خوانده و در یک فایل دیگر نوشته شوند. به هر کدام از این نوشتن و خواندن یک جابجایی می‌گوییم. حداقل تعداد کل این جابجایی‌ها برای ادغام همه فایل‌ها چقدر است؟  
(تخصصی هوش مصنوعی دولتی ۸۲)

- (۱) ۱۹۵ (۲) ۲۰۰ (۳) ۱۸۵ (۴) ۲۱۵

۴- در متنی n کاراکتر وجود دارد که فراوانی آنها از تصاعد هندسی با جمله اول a و قدر نسبت  $q=2$  تبعیت می‌کند. در مورد هزینه کد پیشوندی هافمن کدام گزینه صحیح است؟  
(تخصصی هوش مصنوعی دولتی ۸۶)

- (۱)  $a(2^{n+1} - n - 3)$  (۲)  $a(2^{n+1} + n - 3)$

- (۳)  $a(2^{n+1} - 2n - 2)$  (۴)  $a(2^{n+1} - n + 3)$

۵- اگر رشته  $a b c a b b a c c a a b d f f e$  را با روش کدینگ هافمن کد کنیم، طول کد چند بیت خواهد شد؟  
(IT ۸۵)

- (۱) ۳۸ (۲) ۳۶ (۳) ۳۴ (۴) هیچکدام

۶- مجموعه الفبای  $\{a, b, c, d, e, f\}$  را در نظر بگیرید و فرض کنید فرکانس رخ دادهای این حروف در یک فایل داده‌ای به ترتیب برابر است با  $\{0.16, 0.3, 0.2, 0.35, 0.25, 0.1\}$  باشد. اگر در این حالت درخت کد گذاری بهینه هافمن تشکیل شود، کدام یک از زوج حروف ذیل در یک سطح از درخت قرار ندارند؟  
(IT ۸۶)

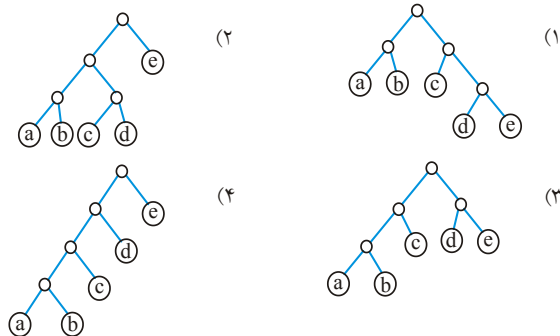
- (۱) d و f (۲) b و d (۳) a و e (۴) b و c

## ۷- حروف e و d و c و b و a با جدول فراوانی

| حرف     | a    | b   | c    | d    | e    |
|---------|------|-----|------|------|------|
| فراوانی | ۰/۰۵ | ۰/۱ | ۰/۲۵ | ۰/۲۸ | ۰/۳۲ |

داده شده است. درخت هافمن وابسته به این حروف به قرار ..... است.

(علوم کامپیوتر ۷۹)



## ۸- در فشرده سازی با استفاده از کدها هافمن اگر x و y دو کاراکتری باشد که کمترین تکرار را

داشته باشند در آن صورت کد بهینه برای دو نویسه (Character) ..... (علوم کامپیوتر ۸۰)

(۱) دارای پیشوند یکسان می باشند فقط در آخرین لیست متفاوت است.

(۲) دارای پسوند یکسان می باشند فقط در اولین لیست متفاوت است.

(۳) دارای پیشوند یکسان نمی باشند و فقط کد آخر آنها برابر است.

(۴) دارای پسوند یکسان نمی باشند و فقط کد اول آنها با هم برابر است.

## ۹- رشته متنی (a b a a b a c a d c a d e) را در نظر بگیرید، کدها هافمن حاصل برای هر یک

از نویسه ها برابر است با ..... (علوم کامپیوتر ۸۰)

|         |             |           |           |           |
|---------|-------------|-----------|-----------|-----------|
| a = ۰۰  | (۴) a = ۰۰۰ | (۳) a = ۰ | (۲) a = ۱ | (۱) a = ۱ |
| b = ۰۱  | b = ۰۰۱     | b = ۱۰۰   | b = ۰۱    | b = ۰۱    |
| c = ۱۰  | d = ۰۰۰۱    | c = ۱۰۱   | c = ۰۰۱   | c = ۰۰۱   |
| d = ۱۱  | d = ۰۱۱     | d = ۱۱۰   | d = ۰۰۰۱  | d = ۰۰۰۱  |
| e = ۱۰۰ | e = ۱۰۰     | e = ۱۱۱   | e = ۰۰۰۰  | e = ۰۰۰۰  |

## ۱۰- جمله زیر را اگر به روش هافمن فشرده سازی کنیم. اندازه فشرده شده چند بیت خواهد

بود؟ (U فاصله خالی است) (علوم کامپیوتر ۸۲)

this<sub>U</sub> is<sub>U</sub> the<sub>U</sub> set<sub>U</sub> that<sub>U</sub> has<sub>U</sub> set

۹۲ (۴)

۷۶ (۳)

۶۸ (۲)

۶۰ (۱)

۱۱- ۶ کار (job) به شرح ذیل داریم.  $g_i$  نشان‌دهنده سود حاصل از اجرای کار  $i$ ام است اگر و فقط اگر بعد از زمان  $d_i$  انجام نشود، فرض کنید هر کار در واحد زمان انجام می‌شود:

|       |    |    |    |   |   |   |
|-------|----|----|----|---|---|---|
| $i$   | ۱  | ۲  | ۳  | ۴ | ۵ | ۶ |
| $G_i$ | ۲۰ | ۱۵ | ۱۰ | ۷ | ۵ | ۳ |
| $D_i$ | ۳  | ۱  | ۱  | ۳ | ۱ | ۳ |

حداکثر سود حاصل از اجرا چقدر است؟ (آزاد ۸۲)

۴۲ (۱)      ۴۵ (۲)      ۳۷ (۳)      ۳۲ (۴)

۱۲-  $n$  بازه  $[L_i, U_i]$  برای  $i = 1, \dots, n$  داده شده‌اند  $L_i$  می‌خواهیم  $S$  مجموعه بازه‌های دو به دو ناهمپوشان با بیشترین طول (مجموع طول بازه‌های انتخاب شده) را پیدا کنیم. برای این کار الگوریتم زیر پیشنهاد شده است:

بازه‌ها را به یک ترتیب انتخاب کن، برای هر بازه انتخابی، بازه‌هایی دیگر که با آن هم پوشانی دارند را حذف کن و این کار را تکرار کن. کدام گزینه ترتیب درست برای این الگوریتم حریصانه است؟ (دولتی ۸۱)

(۱) ترتیب برحسب  $L_i$  بازه‌ها      (۲) ترتیب برحسب  $U_i$  بازه‌ها  
(۳) ترتیب برحسب طول بازه‌ها      (۴) هیچکدام از ترتیب‌های فوق درست نیست.

۱۳- اگر قرار باشد ۶ سخنرانی زیر با شروع و پایان مشخص را طوری برنامه ریزی کنیم که بیشترین تعداد سخنرانی در یک سالن قابل ارائه باشد. بیشترین تعداد سخنرانی‌های ممکن کدام است؟ (IT ۸۵)

|               |       |   |       |   |       |       |
|---------------|-------|---|-------|---|-------|-------|
| شماره سخنرانی | ۱     | ۲ | ۳     | ۴ | ۵     | ۶     |
| شروع          | ۲     | ۱ | ۳     | ۳ | ۴     | ۶     |
| پایان         | ۹     | ۲ | ۴     | ۵ | ۷     | ۸     |
|               | ۳ (۴) |   | ۴ (۳) |   | ۵ (۲) | ۲ (۱) |

۱۴- کد هافمن عبارت AABAABAACAABAACACABA چند بیت دارد؟ (IT ۸۸)

۲۳ (۱)      ۲۴ (۲)      ۲۷ (۳)      ۳۰ (۴)

۱۵- در الگوریتم فشردن هافمن، اگر برای یافتن دو نویسه با کمترین فراوانی از جست و جوی خطی به جای هرم (heap) استفاده شود، زمان اجرای آن چه خواهد بود؟ ( $n$  تعداد نویسه‌هایی است که قرار است کدگذاری شوند) (نرم‌افزار ۹۰)

(۱)  $\Theta(n)$       (۲)  $\Theta(n^2)$       (۳)  $\Theta(n^2 \lg n)$       (۴)  $\Theta(n \lg n)$

۱۶-  $n$  بازه‌ی  $I_i = [x_i, y_i]$  که  $1 \leq i \leq n$  داده شده‌اند. هر بازه نشان‌دهنده‌ی زمان یک کلاس درس است و هر کلاس به صورت مستقل احتیاج به یک اتاق دارد. می‌خواهیم کم‌ترین تعداد اتاق‌های لازم را پیدا کنیم تا بتوان بدون تداخل زمانی، به همه‌ی کلاس‌ها اتاق تخصیص داد. سریع‌ترین الگوریتم برای یافتن این تعداد اتاق کمینه، از چه مرتبه زمانی است؟ (هوش مصنوعی ۹۰)

$$(۱) O(n^3) \quad (۲) O(n^2)$$

$$(۳) O(n \lg n) \quad (۴) \text{راه حل چند جمله‌ای ندارد.}$$

۱۷- فرض کنید  $n$  پردازش در اختیار داریم که برای هر یک زمان شروع و پایان اجرای آن مشخص است. می‌خواهیم با کمترین تعداد پردازنده همه پردازش‌ها را اجرا کنیم. الگوریتم زیر را در نظر بگیرید: در مرحله  $i$ ام از بین پردازش‌های باقیمانده، بیشینه تعداد پردازش‌هایی که با یکدیگر از نظر زمانی همپوشانی ندارند انتخاب کرده و آنها را به پردازنده  $i$ ام اختصاص می‌دهیم. این الگوریتم زمانی پایان می‌پذیرد که پردازش‌های بدون پردازنده باقی نماند. بزرگترین  $i$  (شماره آخرین مرحله الگوریتم) به عنوان خروجی الگوریتم گزارش می‌شود. کوچکترین  $n$  که الگوریتم فوق جواب بهینه تولید نمی‌کند کدام است؟ (نرم‌افزار - ۹۳)

$$(۱) ۱ \quad (۲) ۲ \quad (۳) ۳ \quad (۴) ۵$$

۱۸- می‌خواهیم  $n$  پردازش را بر روی یک پردازنده اجرا کنیم. اجرای پردازش  $i$ ام  $p_i$  ثانیه طول می‌کشد و اجرای آن باید حداکثر تا زمان  $d_i$  به پایان برسد، در غیر این صورت باید به میزان  $t_i - d_i$  جریمه‌ی دیر کرد پرداخت شود که  $t_i$  زمان اتمام اجرای پردازش  $i$ ام است. هدف پیدا کردن الگوریتمی برای زمان‌بندی پردازش‌ها است که مجموع جریمه‌ی دیرکردها کمینه شود. اجرای پردازش‌ها براساس کدام یک از ترکیب‌های زیر مجموع جریمه‌ی دیرکردها را کمینه می‌کند؟ (هوش مصنوعی - ۹۳)

$$(۱) \text{به ترتیب غیرنزولی } d_i \text{ ها} \quad (۲) \text{به ترتیب غیرنزولی } p_i \text{ ها}$$

$$(۳) \text{به ترتیب غیرنزولی } d_i - p_i \text{ ها} \quad (۴) \text{هیچ کدام}$$

۱۹- فرض کنید  $n$  کاراکتر  $a_1, a_2, \dots, a_n$  هر یک با فراوانی  $f_1, f_2, \dots, f_n$  در یک فایل متنی ظاهر شده‌اند. مقادیر  $f_i$  به صورت زیر تعریف می‌شوند:

$$f_1 = f_2 = 1$$

$$f_i = f_{i-1} + f_{i-2}, \quad \forall i > 2$$

کدام یک از روابط زیر مجموع طول کد هافمن  $n$  کاراکتر را برحسب بیت به درستی بیان می‌کند؟ (علوم کامپیوتر - ۹۳)

$$(۱) \frac{n(n+1)}{2} \quad (۲) \frac{n(n+1)}{2} - 1 \quad (۳) \frac{n(n-1)}{2} \quad (۴) \frac{n(n-1)}{2} - 1$$

۲۰- کدام یک از دنباله‌های زیر (به ازای  $n$  های بزرگ) بیش‌ترین ارتفاع ممکن را برای درخت هافمن ایجاد می‌کند؟ اعضای دنباله‌ها نشان‌دهنده‌ی تعداد تکرار کاراکترها در متن ورودی است نه خود کاراکترها.

(نرم‌افزار - ۹۴)

- (۱) دنباله‌ای از  $n$  عدد برابر (۲) دنباله‌ای از  $n$  عدد فیبوناچی پشت سر هم  
 (۳) دنباله‌ی  $\{1, 2, 3, 4, 5, \dots, n\}$  (۴) دنباله‌ی  $\{1^2, 2^2, 3^2, 4^2, 5^2, \dots, n^2\}$

۲۱- الگوریتم خرد کردن پول با روش حریصانه‌ی «استفاده از پرازش‌ترین سکه، تا حد امکان» روی کدام یک از مجموعه سکه‌های زیر لزوماً جواب بهینه (با کم‌ترین تعداد سکه) را تولید نمی‌کند؟ فرض کنید از سکه‌های هر مجموعه به تعداد نامتناهی داریم.

(نرم‌افزار - ۹۴)

- (۱)  $\{1, 2, 5\}$  (۲)  $\{1, 4, 7\}$  (۳)  $\{1, 5, 10\}$  (۴)  $\{1, 7, 10\}$

۲۲- ارتفاع درخت هافمن اگر ورودی ۱۰ نشانه با بسامدهای ۱ تا ۱۰ باشد، چقدر است؟

(هوش مصنوعی - ۹۴)

- (۱) ۳ (۲) ۴ (۳) ۵ (۴) ۶

۲۳- متنی با چهار حرف  $a_1, a_2, a_3, a_4$  و با احتمالات  $P_1 \geq P_2 \geq P_3 \geq P_4$  را در نظر بگیرید. فرض کنید  $P_1 > P_2 = P_3 = P_4$  باشد. کوچک‌ترین عددی مانند  $Q$  که  $P_1 > Q$  نتیجه دهد  $n_1 = 1$ ، کدام است؟

(علوم کامپیوتر - ۹۴)

- (۱)  $0/2$  (۲)  $0/3$  (۳)  $0/4$  (۴)  $0/5$

۲۴- فرض کنید یک متن فقط از حروف  $a, b, c, d, e$  تشکیل شده است و تعداد تکرارهای این حروف به ترتیب  $3, 16, 8, 4, 10$  است. طول کد هافمن (برحسب بیت) این متن کدام است؟

(دکتری کامپیوتر - ۹۵)

- (۱) ۸۸ (۲) ۸۹ (۳) ۹۷ (۴) ۱۲۳

۲۵- فرض کنید می‌خواهیم برای تعدادی کلاس درس که هر کدام ساعت شروع و خاتمه‌شان مشخص است، اتاق رزرو کنیم، هدف کمینه کردن تعداد اتاق‌هاست. به این منظور از الگوریتم حریصانه زیر استفاده می‌کنیم:

- درس‌ها را بر اساس زمان خاتمه‌شان صعودی مرتب می‌کنیم.

- به ترتیب لیست صعودی به درس‌ها بدین شکل اتاق اختصاص می‌دهیم: اگر در میان اتاق‌هایی که تا الان از آنها استفاده شده اتاقی باشد که بتوان این درس را در آنجا برگزار کرد (یعنی با درس‌هایی که قبلاً به این اتاق تخصیص داده شده‌اند همپوشانی ندارد)، این



کار را انجام می‌دهیم در غیر این صورت اتاق جدیدی به این درس اختصاص می‌دهیم و این اتاق نیز به مجموعهٔ اتاق‌های ما اضافه می‌شود.

اگر  $n$  تعداد درس‌ها باشد، کوچکترین  $n$  ی که الگوریتم فوق جواب بهینه تولید نمی‌کند کدام است؟

(۱) ۲

(۲) ۳

(۳) ۴

(۴) این الگوریتم به ازای هر  $n$  همیشه جواب بهینه تولید می‌کند.

۲۶- حداقل و حداکثر طول کد یک عنصر در فشرده‌سازی  $n$  عنصر با روش هافمن چقدر می‌توانند باشند؟

(دکتری علوم کامپیوتر - ۹۵)

$$(1) \log_2 n, \log_2 n \quad (2) n-1, \log_2 n \quad (3) \left\lceil \frac{n}{2} \right\rceil, n-1 \quad (4) \left\lceil \frac{n}{2} \right\rceil, n-2$$

۲۷- یک متن شامل ۲۵۶ نویسه‌ی ۸ بیتی است که در آن تعداد نویسه‌ای که بیش‌ترین تکرار را دارد از دو برابر تعداد نویسه‌ی با کم‌ترین تکرار کم‌تر است. در این حالت اندازه‌ی متن فشرده شده با الگوریتم هافمن چقدر است؟

(نرم‌افزار - ۹۵)

(۱) برابر اندازه‌ی متن اصلی است.

(۲) نصف اندازه‌ی متن اصلی است.

(۳) کم‌تر از نصف اندازه‌ی متن اصلی است.

(۴) کم‌تر از اندازه‌ی متن اصلی ولی بیش‌تر از نصف اندازه‌ی متن اصلی است.

۲۸- دو پردازنده‌ی مشابه داریم و  $n$  عدد کار  $t_1$  تا  $t_n$  که زمان انجام کار  $i$ ام بر روی هر کدام از این پردازنده‌ها برابر  $d_i$  است، می‌خواهیم این کارها را طوری زمان‌بندی کنیم که:

حالت (۱) متوسط زمان پاسخ کارها کمینه شود.

حالت (۲) آخرین زمانی که همه‌ی پردازنده‌ها بی‌کار می‌شوند کمینه شود.

زمان پاسخ یک کار زمانی است که آن کار از یکی از پردازنده‌ها خارج شود. وضعیت گزاره‌های زیر کدام است؟

الف) برای حالت ۱ یک الگوریتم چندجمله‌ای حریصانه وجود دارد.

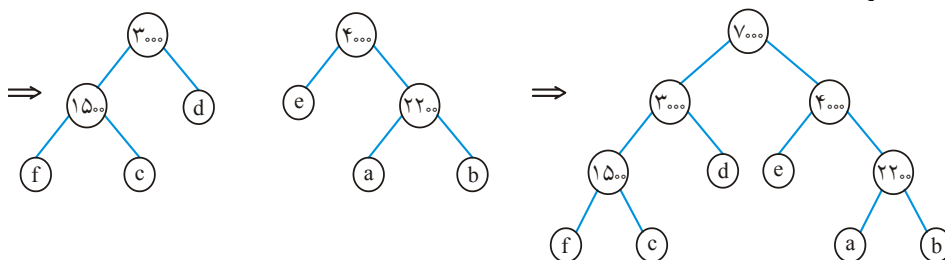
ب) برای حالت ۲ یک الگوریتم چندجمله‌ای حریصانه وجود دارد.

(۱) الف: درست، ب: درست. (۲) الف: نادرست، ب: درست.

(۳) الف: درست، ب: نادرست. (۴) الف: نادرست، ب: نادرست.

## پاسخنامه سؤال‌های چهارگزینه‌ای فصل هشتم

۱- گزینه (۲).

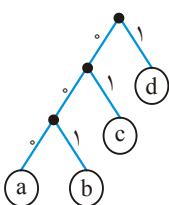


در نتیجه f و c و a و b، ۳ بیتی و d و e، ۲ بیتی هستند:

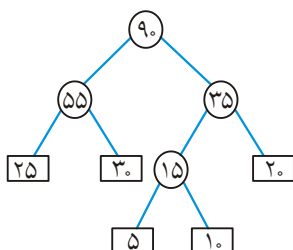
$$= (700 + 800 + 1000 + 1200) \times 3 + (1500 + 1800) \times 2 = 17700$$

تعداد کل بیت‌های لازم

۲- گزینه (۳). مثلاً برای  $n = 4$  عنصر درخت هافمن ممکن است به شکل مقابل شود که طول a برابر ۳ بیت است.



۳- گزینه (۱). الگوریتم مورد نظر یک الگوریتم حریصانه شبیه هافمن است که ابتدا فایل‌های کوچک با یکدیگر ادغام می‌شوند:



$$= 15 + 35 + 55 + 90 = 195$$

تعداد کل جابجایی‌ها

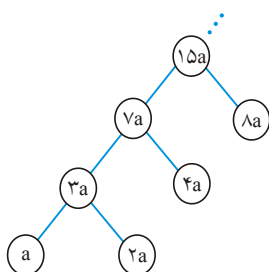
۴- گزینه (۱).

$$a, 2a, 4a, 8a, \dots, 2^{n-1}a$$

فراوانی‌ها

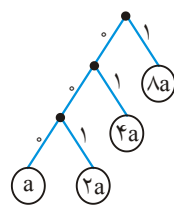
درخت حاصل به شکل روبرو می‌شود:

بنابراین ۲ کاراکتر با فراوانی a و ۲a هر کدام  $n-1$  بیتی هستند کاراکتر 4a دارای  $n-2$  بیت و ... پس:



$$= (a + 2a)(n-1) + 4a(n-2) + 8a(n-3) + \dots + 2^{n-1}a(n - (n-1)) = a(2^{n+1} - n - 3)$$

هزینه کد پیشوندی



مثلاً فرض کنید  $n = 4$  در این صورت فراوانی‌ها  $8a$  و  $4a$  و  $2a$  و  $a$  می‌باشد و درخت هافمن به شکل مقابل است:  
و جمع هزینه‌های این کد برابر است:

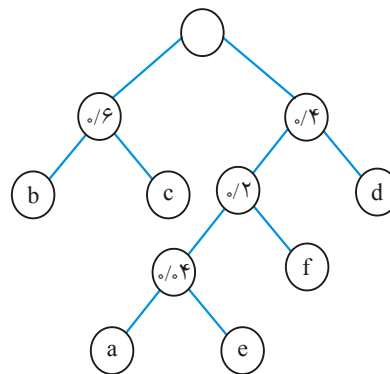
$$(a + 2a) \times 3 + 4a \times 2 + 8a \times 1 = 25a$$

۵- گزینه (۱).

| کاراکتر | d | e | f | c | b | A |
|---------|---|---|---|---|---|---|
| فراوانی | ۱ | ۱ | ۲ | ۳ | ۴ | ۵ |

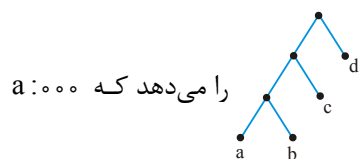
اگر درخت حاصل را بکشید،  $a$  و  $b$  و  $c$  دوییتی،  $f$  سه بیتی و  $d$  و  $e$  چهار بیتی هستند:  
تعداد کل بیت‌های لازم  $= (5 + 4 + 3) \times 2 + 2 \times 3 + (1 + 1) \times 4 = 38$

۶- گزینه (۱).



۷- گزینه (۳).

۸- گزینه (۱).



را می‌دهد که  $a:000$

درخت

| کاراکتر | a | b | c  | d  |
|---------|---|---|----|----|
| فراوانی | ۱ | ۱ | ۱۰ | ۳۰ |

مثلاً

و  $b:001$  است یعنی  $a$  و  $b$  پیشوند یکسان دارند.

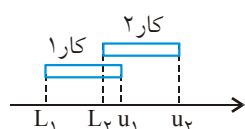
۹- گزینه (۲). به راحتی می‌توان درخت هافمن را رسم کرد.  $a$  یک بیتی و بقیه ۳ بیتی می‌شوند.

۱۰- گزینه (۳). درخت بکشید.

۱۱- گزینه (۱). کارهای ۱ و ۲ و ۴ با توجه به مهلت داده شده، قابل انجام هستند و سود حاصل

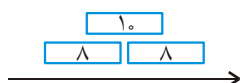
$$20 + 15 + 7 = 42$$

۱۲- گزینه (۴). این مسأله همان مسأله استاندارد انتخاب فعالیت‌ها (Activity Selection) است



با این تفاوت که در اینجا مجموع زمان فعالیت‌های انتخاب شده (مجموع طول بازه‌های انتخاب شده) خواسته شده است و نه تعداد آنها. با مثال نقض نشان می‌دهیم که گزینه‌های ۱ و ۲ و ۳ نادرست هستند.

در شکل فوق، فرض کنید ۲ کار داریم که طول کار ۲ بیشتر از کار ۱ است. اگر به ترتیب  $L_1$  ها انتخاب کنیم، کار اول انتخاب می‌شود و کار ۲ کنار گذاشته می‌شود در صورتی که طول کار ۲ بیشتر است، پس گزینه ۱ غلط است. به همین دلیل گزینه ۲ نیز با شکل فوق نقض می‌شود. برای نقض گزینه ۳ شکل زیر را در نظر بگیرید که شامل ۳ کار است:



اگر کار با طول ۱۰ انتخاب شود آنگاه دو کار با طول ۸ انتخاب نمی‌شوند. در حالی که باید آن دو کار را با مجموع ۱۶ انتخاب می‌کردیم.

۱۳- گزینه (۴). ابتدا کارها را براساس زمان پایان آنها ( $E[i]$ ) مرتب می‌کنیم:

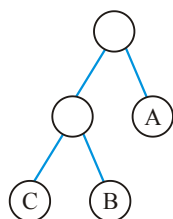
| i      | ۱ | ۲ | ۳ | ۴ | ۵ | ۶ |
|--------|---|---|---|---|---|---|
| $S[i]$ | ۱ | ۳ | ۳ | ۴ | ۶ | ۲ |
| $E[i]$ | ۲ | ۴ | ۵ | ۷ | ۸ | ۹ |

سپس به ترتیب کارهایی را انتخاب می‌کنیم که شروع آنها از پایان کار قبلی انتخاب شده کمتر نباشد به این ترتیب کارهای ۱ و ۲ و ۴ انجام می‌شوند.

۱۴- گزینه (۳). جدول فراوانی را تشکیل می‌دهیم:

|         | A  | B | C |
|---------|----|---|---|
| کاراکتر |    |   |   |
| فراوانی | ۱۳ | ۴ | ۳ |

درخت هافمن به شکل روبرو است:



بنابراین A یک بیتی و B و C دو بیتی هستند پس:

$$27 \text{ bit} = 13 \times 1 + (4 + 3) \times 2$$

۱۵- گزینه (۲). نیاز به  $n-1$  بار جستجوی خطی است که از مرتبه  $\theta(n^2)$  می‌شود.

۱۶- گزینه (۳). کافی است بازه‌ها را براساس  $x_i$  به صورت صعودی مرتب کنید (زمان  $n \cdot \lg n$ ) و

سپس هر کار را به اولین کلاس خالی تخصیص دهید (زمان  $n$ ).

- ۱۷- گزینه (۴). به ازای یک پردازش الگوریتم یک پردازنده تخصیص می‌دهد و درست کار می‌کند به ازای دو پردازش، دو حالت وجود دارد: یا دو پردازش با هم همپوشانی دارند یا خیر. اگر همپوشانی دارند الگوریتم ما، دو پردازنده استفاده می‌کند و اگر ندارند، یک پردازنده استفاده می‌شود که صحیح است. به ازای سه پردازش سه حالت وجود دارد: ۱- هر سه پردازش همپوشان هستند. ۲- دو تا از پردازش‌ها همپوشان هستند. ۳- هیچ کدام همپوشان نیستند. که در هر سه حالت الگوریتم ما درست کار می‌کند.
- ۱۸- گزینه (۴). برای هم حالات گفته شده، مثال نقض می‌آوریم:

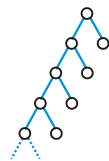
گزینه ۱ :  $d_i = 1, 3, 4, 5$  نقض گزینه ۱

$$P_i = 1, 1, 1, 1$$

$= 9 + 8 + 8 + 8 = 33$  : مجموع جریمه دیر کرد

حال آنکه اگر کارها را برعکس اجرا می‌کردیم مجموع جریمه دیر کرد ۹ می‌شد. به همین ترتیب برای سایر گزینه‌ها مثال نقض وجود دارد.

- ۱۹- گزینه (۲). درخت حاصل به شکل مقابل است که دارای ۲ کاراکتر  $n-1$  بیتی، ۱ کاراکتر  $n-2$  بیتی، ۱ کاراکتر  $n-3$  بیتی و به همین ترتیب ۱ کاراکتر ۱ بیتی است پس مجموع طول کد هافمن برابر است با:

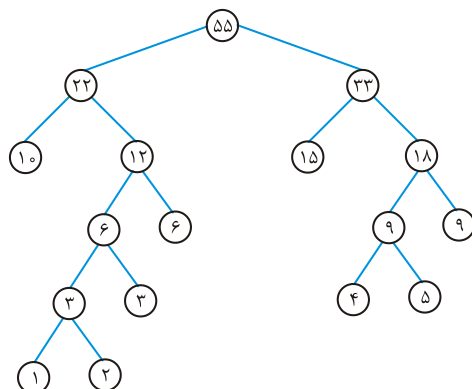


$$2(n-1) + n-2 + n-3 + \dots + 1 = (n + n-1 + n-2 + n-3 + \dots + 1) - 1 = \frac{n(n+1)}{2} - 1$$

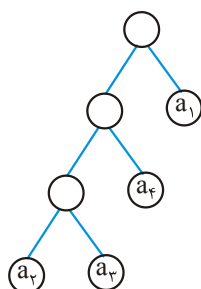
- ۲۰- گزینه (۲). بیشترین ارتفاع وقتی است که مجموع هر دو فراوانی مینیمم، خود مینیمم باشد. در دنباله فیبوناچی همیشه  $f_{i-1} + f_i = f_{i+1}$  یعنی جمع هر دو جمله مینیمم با مینیمم سوم، برابر است و درخت هافمن درختی به ارتفاع  $n-1$  می‌شود که هر نود داخلی دارای ۲ فرزند است و این بیشترین ارتفاع ممکن است.

- ۲۱- گزینه (۴). فرض کنید می‌خواهیم با مجموعه سکه‌های  $\{1, 7, 10\}$  مبلغ ۱۴ واحد را خرد کنیم. با روش حریصانه تعداد ۵ سکه  $(10 + 1 + 1 + 1 + 1)$  انتخاب می‌شود، حال آنکه جواب بهینه، تعداد ۲ سکه  $(7 + 7)$  می‌باشد.

۲۲- گزینه (۳). درخت هافمن برای این مسئله به شکل مقابل است که ارتفاع ۵ دارد:

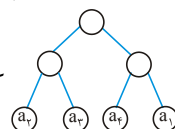


۲۳- گزینه (۳). شکل درخت هافمن باید به شکل مقابل باشد که  $a_1$  یک بیتی شود:



$$\text{هزینه} = 3P_1 + 3P_2 + 2P_3 + P_4 = 8P_1 + P_2$$

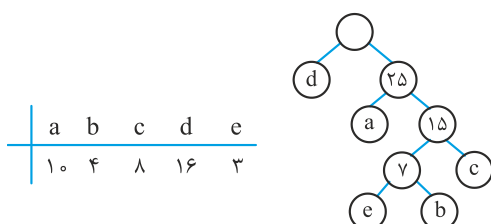
از طرفی  $1 = P_1 + P_2 + P_3 + P_4 + P_5 + P_6 = 1 + 3P_1$  (جمع همه احتمالات برابر ۱ است). پس هزینه درخت برابر  $5P_1 + 1$  است. برای آنکه  $a_1$  یک بیتی باشد باید هزینه درخت فوق از درخت



کمتر باشد یعنی باید  $5P_1 + 1 < 2(P_1 + P_2 + P_3 + P_4) = 2$  از

پس  $2 < 5P_1 + 1$  در نتیجه  $P_1 < \frac{1}{5}$  یعنی  $P_1$  حداکثر  $\frac{1}{5}$  است. بنابراین  $P_1$  حداکثر می‌تواند  $\frac{1}{4}$  باشد.

۲۴- گزینه (۱).



| a  | b | c | d  | e |
|----|---|---|----|---|
| ۱۰ | ۴ | ۸ | ۱۶ | ۳ |

$$\rightarrow 16 \times 1 + 10 \times 2 + 8 \times 3 + (3 + 4) \times 4 = 88$$

۲۵- گزینه (۳). به ازای  $n = 2$  الگوریتم درست جواب می‌دهد. زیرا ۲ درس یا همپوشانی ندارند که آنگاه نیاز به یک کلاس دارند و یا همپوشانی دارند که نیاز به ۲ کلاس دارند و در هر ۲ حالت الگوریتم ما درست جواب می‌دهد.

به ازای  $n = 3$  الگوریتم درست جواب می‌دهد. زیرا ۳ درس حالت‌های زیر را دارند که در هر صورت الگوریتم درست جواب می‌دهد.



به ازای  $n = 4$  الگوریتم لزوماً درست جواب نمی‌دهد. مثلاً ۴ درس با بازه‌های زمانی  $D_1 = [1, 4)$  و  $D_2 = [2, 6)$  و  $D_3 = [6, 8)$  و  $D_4 = [5, 10)$  را در نظر بگیرید. تخصیص بهینه به این شکل است که  $D_1$  و  $D_4$  به کلاس ۱ و  $D_2$  و  $D_3$  به کلاس ۲ تخصیص می‌یابند. حال آنکه الگوریتم ما، ابتدا  $D_1$  را به کلاس ۱ سپس  $D_2$  را به کلاس ۲، سپس  $D_3$  را به کلاس ۱ و در نهایت  $D_4$  را به کلاس ۳ تخصیص می‌دهد.

۲۶- گزینه (۱). ولی سوال غلط است. حداقل طول کد برابر ۱ و حداکثر  $n-1$  است.

۲۷- گزینه (۱). اگر  $2^n$  کاراکتر داشته باشیم با این خاصیت که جمع فراوانی هر دو مینیمم از ماکزیمم فراوانی بیشتر (یا مساوی) شود آنگاه درخت هافمن یک درخت پر می‌شود و هزینه آن هیچ تفاوتی با روش عادی کدگذاری ندارد.

۲۸- گزینه (۳). گزاره الف درست و گزاره ب نادرست است. گزاره «ب» با روش پویا قابل حل است.

## فصل ۹

## برنامه‌نویسی پویا

## مقدمه

روش تقسیم و غلبه یک روش کل به جز یا بالا به پایین (top – down) بود. ولی روش برنامه نویسی پویا یک روش جزء به کل یا پایین به بالا (bottom- up) می‌باشد. در تکنیک تقسیم و غلبه از مسئله اصلی شروع می‌کردیم و سپس آن را به اجزای کوچکتر تقسیم می‌کردیم و تقسیم را تا زمانی ادامه می‌دادیم که به مسائل کوچک قابل حل برسیم و سپس با حل آنها به تدریج مسئله اصلی حل می‌شد.

در برنامه نویسی پویا، ابتدا مسائل کوچک حل می‌شوند و در یک مکان ذخیره می‌شوند و سپس به تدریج به حل مسئله اصلی می‌رسیم. در این الگوریتم‌ها یک مسئله کوچک فقط یک بار محاسبه می‌شود.

شباهت تقسیم و غلبه و برنامه نویسی پویا این است که هر دو نیاز به یک رابطه بازگشتی دارند و تفاوت آنها این است که تقسیم و غلبه مسئله را از بالا به پایین حل می‌کند، در صورتی که برنامه نویسی پویا آن را از پایین به بالا حل می‌کند. (البته پویا یک مدل از بالا به پایین نیز دارد که با شیوه caching یا memo نوشته می‌شود)

**مثال ۱:** رابطه بازگشتی  $T(n) = T(n-1) + 2$  را با فرض  $T(1) = 1$  حل کنید.

**پاسخ:** روش اول (بالا به پایین):

$$\begin{aligned} T(n) &= T(n-1) + 2 = (T(n-2) + 2) + 2 \\ &= T(n-3) + 2 + 2 + 2 = T(n-3) + 3 \times 2 = \dots \\ &= T(n - (n-1)) + (n-1) \times 2 = T(1) + (n-1) \times 2 = 2n - 1 \end{aligned}$$



$$T(1) = 1$$

روش دوم (پایین به بالا):

$$T(2) = T(1) + 2 = 1 + 2 = 3$$

$$T(3) = T(2) + 2 = 3 + 2 = 5$$

$$T(4) = T(3) + 2 = 5 + 2 = 7$$

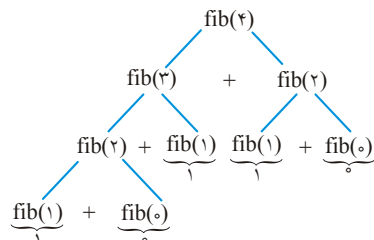
⋮

$$T(n) = 2n - 1$$

**مثال ۲:** اگر بخواهیم جمله  $n$ ام سری فیبوناچی را با روش تقسیم و غلبه بیابیم در این صورت الگوریتم آن به صورت زیر است:

```
int fib (int n)
{
    if (n <= 1)
        return n;
    else
        return fib (n-1) + fib (n-2);
}
```

با توجه به الگوریتم فوق اگر بخواهیم  $\text{fib}(4)$  را بیابیم به شکل زیر عمل می‌کنیم:



همانطور که ملاحظه می‌شود، محاسبات تکراری در درخت وجود دارد و مثلاً  $\text{fib}(2)$ ، ۲ بار

محاسبه می‌شود. مرتبه اجرایی این الگوریتم بیش از  $2^{\frac{n}{2}}$  می‌باشد. (درواقع  $\theta\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right)$  است)

اگر بخواهیم جمله  $n$ ام سری فیبوناچی را با روش برنامه نویسی پویا بیابیم، الگوریتم زیر را ارائه

می‌دهیم:

```
int fib (int n)
{
    int i, f[0...n]; f[0] = 0;
    if (n > 0)
    {
        f[1] = 1;
        for (i = 2; i <= n; i++) f[i] = f[i-1] + f[i-2];
    }
    return f[n];
}
```

البته می‌توان جمله  $n$ ام سری فیبوناچی را به صورت پویا و با مصرف حافظه کمتر با توجه به الگوریتم زیر بدست آورد:

```
int fib (int n)
{
    int f1, f2, f;  f1 = 0; f2 = 1;
    if (n == 0) return f1;
    if (n == 1) return f2;
    for (i = 2; i <= n; i++)
    {
        f = f1 + f2;  f1 = f2;  f2 = f;
    }
    return f;
}
```

با توجه به دو الگوریتم نوشته شده به شیوه پویا مشاهده می‌شود که مرتبه اجرایی  $\theta(n)$  می‌باشد.

دلیل اصلی اینکه فیبوناچی بازگشتی دارای مرتبه نمایی و فیبوناچی غیر بازگشتی دارای مرتبه خطی است این است که در فیبوناچی بازگشتی زیر مسائل به هم ارتباط دارند. مثلاً  $\text{fib}(3)$  و  $\text{fib}(4)$  هر دو دارای زیر مسائل مشترک هستند.

**تمرین:** روشی وجود دارد که می‌تواند جمله  $n$ ام فیبوناچی را با مرتبه  $O(\lg n)$  بیابد. آن روش را ارائه دهید!

اصولاً روش تقسیم و غلبه برای الگوریتم‌هایی مناسب است که زیر مسائل آنها با هم اشتراک نداشته باشند مثل مرتب‌سازی ادغامی که نمونه‌های تقسیم شده ارتباطی به هم ندارند.

به طور کلی می‌توان گفت مسائلی که به روش برنامه‌سازی پویا حل می‌شوند دارای ویژگی‌های زیر هستند:

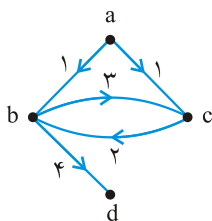
۱) بهینه‌سازی، در اکثر الگوریتم‌های برنامه‌سازی پویا، فقط بدست آوردن جواب کافی نیست، بلکه جواب باید بهینه نیز باشد.

۲) مسائل را از پایین‌ترین سطح به طرف بالاترین سطح حل می‌کنیم یعنی پایین به بالا. البته شیوه از بالا به پایین نیز مطرح است که با ایده memoized (خاطر سپاری) از فراخوانیهای تکراری جلوگیری می‌کند.

۳) برخلاف روش تقسیم و حل که برای هر مساله در سطح  $L$  تنها از مسائل سطح  $L-1$  استفاده می‌کنیم، در روش برنامه‌سازی پویا، برای حل هر مساله در سطح  $L$  می‌توانیم از کلیه مسائل سطوح پایین‌تر که لازم باشد استفاده کنیم.

۴) در هر سطح کلیه مسائل موجود آن سطح حل می‌گردند و نتایج نگهداری می‌شود.

**توجه:** در کلیه مسائل بهینه سازی که با روش برنامه سازی پویا قابل حل هستند باید اصل بهینگی (Principle of optimality) برقرار باشد. اصل بهینگی در صورتی برقرار است که زیر مسائل یک مسائل بهینه خود بهینه باشند. به عنوان مثال اگر کوتاهترین مسیر از  $a$  به  $b$  حتماً از  $c$  می‌گذرد آنگاه باید مسیر از  $a$  به  $c$  و از  $c$  به  $b$  خود کوتاهترین باشند.



ولی در مسئله یافتن طولانی‌ترین مسیر بین دو راس اصل بهینگی صادق نیست و نمی‌توان آن را از طریق برنامه‌ریزی پویا حل کرد. به عنوان مثال در گراف زیر، طولانی‌ترین مسیر ساده (مسیر بهینه) از  $a$  به  $d$  مسیر  $\langle a, c, b, d \rangle$  می‌باشد ولی زیر مسیر  $\langle a, c \rangle$  یک مسیر بهینه (طولانی‌ترین) از  $a$  به  $c$  نیست و مسیر طولانی‌تر به صورت  $\langle a, b, c \rangle$  می‌باشد.

### ۱- ضرب زنجیره‌ای ماتریس‌ها

همانطور که می‌دانیم ضرب ماتریس‌ها دارای خاصیت شرکت‌پذیری می‌باشد ولی جابجایی ندارد. علاوه بر این دو ماتریس  $A$  و  $B$  فقط به شرطی به صورت  $A \times B$  قابل ضرب هستند که بُعد وسط آنها یکسان باشد یعنی تعداد ستون‌های  $A$  با تعداد سطرهای  $B$  مساوی باشد. علاوه بر این می‌دانیم که ضرب  $A_{m \times n} \times B_{n \times p}$  نیاز به  $m \times n \times p$  عمل ضرب دارد.

**مثال ۳:** ضرب سه ماتریس  $A_{7 \times 5}$  و  $B_{5 \times 3}$  و  $C_{3 \times 2}$  به صورت  $ABC$  حداقل چند عمل ضرب نیاز دارد؟

**پاسخ:** سه ماتریس را می‌توان به ۲ صورت  $(AB)C$  و  $A(BC)$  در هم ضرب کرد. بنابراین هر دو حالت را بررسی می‌کنیم:

$$(A_{7 \times 5} \times B_{5 \times 3}) \times C_{3 \times 2} \Rightarrow \text{تعداد ضرب} = 7 \times 5 \times 3 + 7 \times 3 \times 2 = 147$$

$$A_{7 \times 5} \times (B_{5 \times 3} \times C_{3 \times 2}) \Rightarrow \text{تعداد ضرب} = 5 \times 3 \times 2 + 7 \times 5 \times 2 = 100$$

بنابراین جواب ۱۰۰ می‌باشد.

**مثال ۴:** چه رابطه‌ای بین  $w$  و  $x$  و  $y$  و  $z$  برقرار باشد تا ضرب ۳ ماتریس  $A_{w \times x}$  و  $B_{x \times y}$  و  $C_{y \times z}$  به صورت  $(AB)C$  از  $A(BC)$  بهتر باشد یعنی تعداد عملیات ضرب کمتری داشته باشد؟

**پاسخ:**

$$A_{w \times x} \times B_{x \times y} \Rightarrow \text{تعداد ضرب} = wxy$$

$$(AB)_{w \times y} \times C_{y \times z} \Rightarrow \text{تعداد ضرب} = wyz$$

$$(AB)C \Rightarrow \text{تعداد ضرب} = wxy + wyz$$

به همین ترتیب تعداد ضرب  $A(BC)$  برابر  $xyz + wxz$  می‌باشد بنابراین

$$wxy + wyz < xyz + wxz$$

طرفین رابطه را به  $wxyz$  تقسیم می‌کنیم:

$$\frac{1}{z} + \frac{1}{x} < \frac{1}{w} + \frac{1}{y}$$

حال می‌خواهیم ترتیب ضرب بهینه  $n$  ماتریس  $M = M_1 \times M_2 \times \dots \times M_n$  را پیدا کنیم، یعنی بررسی کنیم چگونه پرانتزگذاری کنیم که تعداد عمل ضرب عددی، مینیمم شود. می‌دانیم تعداد حالاتی که می‌توان  $n+1$  ماتریس را درهم ضرب کرد برابر عدد کاتالان یعنی  $\frac{1}{n+1} \binom{2n}{n}$  می‌باشد. اگر بخواهیم تمام حالات ضرب را آزمایش کنیم تا به ضرب بهینه برسیم بسیار زمانگیر است و می‌توان ثابت کرد تابع رشد کاتالان به صورت  $\Omega\left(\frac{4^n}{n^{\frac{3}{2}}}\right)$  می‌باشد که بیشتر از نمایی  $2^n$  است.

می‌خواهیم با مرتبه چند جمله‌ای و با استفاده از برنامه‌ریزی پویا، ضرب بهینه را بیابیم. روشن است که اصل بهینگی برای ضرب ماتریس‌ها برقرار است زیرا اگر مثلاً ضرب  $(M_1 M_2)(M_3 M_4)$  بهینه باشد باید  $M_1 M_2$  و  $M_3 M_4$  بهینه باشند. در ادامه بحث فرض می‌کنیم ماتریس  $M_i$  دارای ابعاد  $r_{i-1} \times r_i$  ( $i = 1, 2, \dots, n$ ) می‌باشد مثلاً ماتریس  $M_1$  یک ماتریس  $r_0 \times r_1$  می‌باشد. همچنین فرض می‌کنیم  $m_{ij}$  حداقل تعداد عملیات ضرب عددی برای محاسبه حاصل  $M_i \times M_{i+1} \times \dots \times M_j$  می‌باشد. مثلاً  $m_{13}$  یعنی حداقل تعداد ضرب ماتریس‌های  $M_1 \times M_2 \times M_3$  واضح است که  $m_{ii} = 0$  می‌باشد.

برای محاسبه  $m_{ij}$  فرض کنید ضرب  $M_i \times \dots \times M_j$  را به صورت زیر نوشته‌ایم: (هر طور پرانتزگذاری کنیم یک نقطه شکست مثل  $k$  وجود دارد)

$$(M_i \times \dots \times M_k) \cdot (M_{k+1} \times \dots \times M_j) \quad (1)$$

بنابراین  $m_{ij}$  از رابطه زیر بدست می‌آید:

$$m_{ij} = \min_{i \leq k < j} (m_{ik} + m_{k+1,j} + r_{i-1} \times r_k \times r_j) \quad (2)$$

که در این رابطه  $m_{ik}$  تعداد حداقل اعمال ضرب لازم برای محاسبه

$$M_i \times M_{i+1} \times \dots \times M_k \quad (3)$$

می‌باشد و  $m_{k+1,j}$  حداقل ضرب لازم برای محاسبه

$$M_{k+1} \times M_{k+2} \times \dots \times M_j \quad (۴)$$

می‌باشد و جمله  $r_{i-1} \times r_k \times r_j$  تعداد ضرب لازم برای ضرب دو ماتریس حاصل از هر قسمت (۳) و (۴) می‌باشد.

با توجه به فرمول (۲) مراحل زیر را انجام می‌دهیم تا به  $m_{1n}$  یعنی جواب برسیم:

**مرحله ۰:** هیچ عمل ضرب ماتریسی. این مرحله معادل یادداشت  $m_{ii} = 0$ ،  $i = 1, 2, \dots, n$  می‌باشد.

**مرحله ۱:** یک عمل ضرب ماتریسی. در این مرحله کلیه مقادیر  $m_{i,i+1}$  و  $i = 1, 2, \dots, n-1$  را محاسبه و در جدول می‌گذاریم.

**مرحله ۲:** دو عمل ضرب ماتریسی. در این مرحله کلیه مقادیر  $m_{i,i+2}$  و  $i = 1, 2, \dots, n-2$  را محاسبه می‌کنیم.

⋮

**مرحله n-۱:** n-۱ عمل ضرب ماتریسی. در این مرحله  $m_{1n}$  را محاسبه می‌کنیم.

**مثال ۵:** فرض کنید ماتریس‌های  $M_1$  و  $M_2$  و  $M_3$  و  $M_4$  به ترتیب  $10 \times 20$  و  $20 \times 50$  و  $50 \times 10$  باشند. کمیت‌های گفته شده را محاسبه می‌کنیم:

$$m_{11} = m_{22} = m_{33} = m_{44} = 0$$

$$\begin{cases} m_{12} = \min(m_{11} + m_{22} + 10 \times 20 \times 50) = 10000 \\ m_{23} = \min(m_{22} + m_{33} + 20 \times 50 \times 10) = 1000 \\ m_{34} = \min(m_{33} + m_{44} + 50 \times 10 \times 10) = 5000 \end{cases}$$

$$\begin{cases} m_{13} = \min(m_{11} + m_{33} + 10 \times 20 \times 10, m_{12} + m_{33} + 10 \times 50 \times 10) = 1200 \\ m_{24} = \min(m_{22} + m_{44} + 20 \times 50 \times 10, m_{23} + m_{44} + 20 \times 10 \times 10) = 3000 \end{cases}$$

$$m_{14} = \min(m_{11} + m_{44} + 10 \times 20 \times 10, m_{12} + m_{44} + 10 \times 50 \times 10, m_{13} + m_{44} + 10 \times 10 \times 10) = 2200$$

خلاصه عملیات فوق در جدول زیر آمده است:

|         |                  |                 |                 |              |
|---------|------------------|-----------------|-----------------|--------------|
| مرحله ۰ | $m_{11} = 0$     | $m_{22} = 0$    | $m_{33} = 0$    | $m_{44} = 0$ |
| مرحله ۱ | $m_{12} = 10000$ | $m_{23} = 1000$ | $m_{34} = 5000$ |              |
| مرحله ۲ | $m_{13} = 1200$  | $m_{24} = 3000$ |                 |              |
| مرحله ۳ | $m_{14} = 2200$  |                 |                 |              |

الگوریتم optmm تعداد حداقل ضرب را برای  $n$  ماتریس بدست می‌آورد:

```
int optmm(int m[1..n][1..n], int n)
{
    int i, j, L;
    for(i = 1; i <= n; i++) m[i][i] = 0;
    for(L = 1; L <= n; L++) // بهتر است  $L \leq n-1$  باشد
        for(i = 1; i <= n - L; i++)
        {
            j = i + L;
            m[i][j] = min_{i \leq k < j} {m[i][k] + m[k+1][j] + r_{i-1} * r_k * r_j}
        }
    return m[1][n];
}
```

الگوریتم فوق از مرتبه  $O(n^3)$  می‌باشد. زیرا تعداد اجرای جمله اصلی که همان مینیمم‌گیری است برابر است با:

$$\sum_{L=1}^n \sum_{i=1}^{n-L} \sum_{k=i}^{i+L-1} 1 = \frac{n^2(n+1)}{2} - \frac{n(n+1)(2n+1)}{6} = \frac{n^3 - n}{6}$$

آرایه  $m$  برای مثال قبل عبارت است از:

$$m = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 10000 & 1200 & 2200 \\ & 0 & 1000 & 3000 \\ & & 0 & 5000 \\ & & & 0 \end{bmatrix} \end{matrix}$$

که  $m[1][4] = 2200$  تعداد حداقل ضرب ۴ ماتریس گفته شده را نشان می‌دهد.

برای تعیین ترتیب بهینه ضرب ماتریس‌ها آرایه دیگری به نام  $p$  که ابعاد آن برابر ابعاد  $m$  است تعریف می‌کنیم و هر بار مقدار  $k$  را که به ازای آن  $m_{ij}$  مینیمم شد در محل  $P_{ij}$  ذخیره می‌کنیم. در مثال ضرب  $M_1 \times M_2 \times M_3 \times M_4$  دیدیم که  $m_{14}$  از رابطه زیر بدست آمد:

$$m_{14} = \min \left( \underbrace{23000}_{k=1}, \underbrace{65000}_{k=2}, \underbrace{2200}_{k=3} \right) = 2200$$

یعنی مینیمم ضرب  $m_{14}$  به ازای  $k=3$  بدست آمد یعنی:  $(M_1 \times M_2 \times M_3) \cdot (M_4)$ .

حال برای آنکه ببینیم نقطه شکست  $M_1 \times M_2 \times M_3$  کجاست، به  $m_{13}$  نگاه می‌کنیم:

$$m_{13} = \min \left( \underbrace{1200}_{k=1}, \underbrace{10500}_{k=2} \right) = 1200$$

پس نقطه شکست  $M_1 M_2 M_3$ ،  $k=1$ ،  $(M_2 M_3)$  می‌باشد یعنی ضرب بهینه به صورت  $M_4.(M_1.(M_2.M_3))$  می‌باشد.  
ماتریس  $p$  برای این مثال به شکل زیر است:

$$p = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} & & & \\ & 1 & 1 & 3 \\ & & 2 & 3 \\ & & & 3 \end{bmatrix} \end{matrix}$$

**مثال ۶:** می‌خواهیم ۶ ماتریس  $M_1 M_2 M_3 M_4 M_5 M_6$  را در هم ضرب کنیم. ماتریس  $p$  به صورت زیر بدست آمده است. ترتیب بهینه ضرب را مشخص کنید.

$$p = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{matrix} & \begin{bmatrix} & & & & & \\ & 1 & 1 & 1 & 1 & 1 \\ & & 2 & 3 & 4 & 5 \\ & & & 3 & 4 & 5 \\ & & & & 4 & 5 \\ & & & & & 5 \end{bmatrix} \end{matrix}$$

**پاسخ:** عنصر  $p[1,6]=1$  یعنی نقطه شکست برای ضرب  $M_1..M_6$ ، نقطه ۱ است پس:  
 $(M_1)(M_2 M_3 M_4 M_5 M_6)$   
حال باید  $p[2,6]$  را بررسی کنیم که مقدار آن ۵ می‌باشد پس نقطه شکست  $M_2..M_6$  نقطه ۵ است یعنی:

$$(M_1)((M_2 M_3 M_4 M_5)(M_6))$$

حال  $p[2,5]=4$  پس:

$$(M_1)((M_2 M_3 M_4)(M_5)(M_6))$$

و با توجه به اینکه  $p[2,4]=3$  در نتیجه:

$$M_1(((M_2 M_3)M_4)M_5)M_6$$

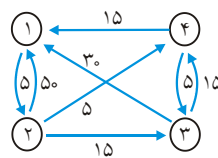
ترتیب بهینه ضرب است.

## ۲- الگوریتم فلویید برای یافتن تمام کوتاهترین مسیرها در گراف جهت‌دار

فرض کنید  $G = (V, E)$  یک گراف جهت‌دار با  $n$  راس است که  $V = \{1, 2, \dots, n\}$  مجموعه رئوس گراف و  $E$  مجموعه یال‌ها است. همچنین فرض کنید  $C$  ماتریس هزینه‌های گراف است و به صورت زیر تعریف می‌شود:

$$C[i, j] = \begin{cases} 0 & \text{اگر } i=j \text{ باشد} \\ \infty & \text{اگر از راس } i \text{ به } j \text{ یال نباشد.} \\ \text{وزن یال} & \text{اگر یالی از راس } i \text{ به } j \text{ باشد} \end{cases}$$

به عنوان مثال برای گراف مقابل ماتریس  $C$  عبارت است از:



$$\Rightarrow C = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 5 & 5 & 15 \\ 5 & 0 & 15 & 5 \\ 3 & 0 & 0 & 15 \\ 15 & 0 & 5 & 0 \end{bmatrix} \end{matrix}$$

می‌خواهیم تمامی کوتاهترین مسیرها بین رئوس گراف را بیابیم. به عبارتی می‌خواهیم یک ماتریس  $A_{n \times n}$  بیابیم که درایه  $A[i, j]$  حاوی طول کوتاهترین مسیر از راس  $i$  به راس  $j$  می‌باشد. به عنوان مثال برای گراف شکل فوق ماتریس  $A$  به صورت زیر بدست خواهد آمد:

$$A = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 5 & 15 & 10 \\ 20 & 0 & 10 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{bmatrix} \end{matrix}$$

مثلاً درایه  $A[1, 3] = 15$  به این معنی است که کوتاهترین مسیر بین راس ۱ و راس ۳ که مسیر  $\langle 1, 2, 4, 3 \rangle$  است دارای طول ۱۵ می‌باشد و به همین ترتیب.

اگر بخواهیم این مساله را با بررسی همه حالات حل کنیم مرتبه زمانی، بدتر از نمایی ( $O(n!)$ ) خواهد بود ولی با برنامه‌ریزی پویا الگوریتمی ارائه می‌دهیم از مرتبه  $O(n^3)$ .

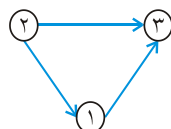
لازم به ذکر است که در این مساله، اصل بهینگی برقرار است. زیرا اگر مسیر  $i$  به  $j$  که از  $k$  می‌گذرد کوتاهترین باشد آنگاه مسیر  $i$  به  $k$  و  $k$  به  $j$  نیز کوتاهترین است.

برای حل مساله، ماتریس‌های  $A_0$  تا  $A_n$  را بدست می‌آوریم.  $A_0$  همان ماتریس هزینه یعنی  $C$  می‌باشد و  $A_k$  از روی  $A_{k-1}$  بدست می‌آید. و  $A_n$  همان  $A$  یعنی جواب می‌باشد.



درایه‌های  $A_k$  طول کوتاهترین مسیرهایی را می‌دهند که فقط رئوس  $\{1, 2, \dots, k\}$  به عنوان رئوس میانی می‌توانند باشند.

مثلاً  $A_1[2, 3]$  یعنی طول کوتاهترین مسیر از راس ۲ به راس ۳ به شرطی که این مسیر فقط بتواند از راس ۱ عبور کند. می‌توان گفت



$$A_1[2, 3] = \min(A_0[2, 3], A_0[2, 1] + A_0[1, 3])$$

زیرا کوتاهترین مسیر از راس ۲ به راس ۳، به شرطی که فقط راس ۱ بتواند در مسیر حضور داشته باشد یا مسیر مستقیم  $\langle 2, 3 \rangle$  می‌باشد که هزینه آن  $A_0[2, 3]$  است یا مسیر  $\langle 2, 1, 3 \rangle$  می‌باشد که هزینه آن مجموع  $A_0[2, 1] + A_0[1, 3]$  است.

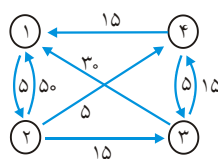
پس از اینکه همه درایه‌های ماتریس  $A_1$  محاسبه شد، ماتریس  $A_2$  را محاسبه می‌کنیم یعنی تاثیر راس ۲ را روی تمام مسیرها محاسبه می‌کنیم. بنابراین  $A_2[i, j]$  یعنی کوتاهترین مسیر از راس  $i$  به راس  $j$  به شرطی که رئوس  $\{1, 2\}$  بتوانند رئوس واسطه باشند. واضح است که

$$A_2[i, j] = \min \left( \underbrace{A_1[i, j]}_{\text{کوتاهترین مسیر از } i \text{ به } j \text{ با واسطه راس ۱}}, \underbrace{A_1[i, 2]}_{\text{کوتاهترین مسیر از } i \text{ به ۲ به واسطه راس ۱}} + \underbrace{A_1[2, j]}_{\text{کوتاهترین مسیر از ۲ به } j \text{ با واسطه راس ۱}} \right)$$

به طور کلی می‌توان فرمول بهینه‌سازی زیر را ارائه داد:

$$A_k[i, j] = \min(A_{k-1}[i, j], A_{k-1}[i, k] + A_{k-1}[k, j])$$

**مثال ۷:** مراحل الگوریتم فلوید را روی گراف زیر نشان دهید.



پاسخ:

$$A_0 = C = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 50 & \infty & 15 \\ 50 & 0 & 15 & 30 \\ 30 & \infty & 0 & 15 \\ 15 & \infty & 15 & 0 \end{bmatrix} \end{matrix} \Rightarrow A_1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 50 & \infty & 15 \\ 50 & 0 & 15 & 15 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{bmatrix} \end{matrix} \Rightarrow A_2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 50 & 20 & 10 \\ 50 & 0 & 15 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{bmatrix} \end{matrix}$$

$$\Rightarrow A_3 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 5 & 20 & 10 \\ 45 & 0 & 15 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{bmatrix} \end{matrix} \Rightarrow A_4 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 5 & 15 & 10 \\ 20 & 0 & 10 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{bmatrix} \end{matrix}$$

مثلاً برای محاسبه  $A_1[3,2]$  این گونه عمل شده است:

$$A_1[3,2] = \min(A_0[3,2], A_0[3,1] + A_0[1,2]) \\ = \min(\infty, 30 + 5) = 35$$

به همین ترتیب با توجه به فرمول بهینه‌سازی می‌توان سایر درایه‌ها را محاسبه کرد.

**توجه:** سطر  $k$ ، ستون  $k$  از ماتریس  $A_k$  با  $A_{k-1}$  مساوی است.

الگوریتم را می‌توان به صورت زیر نوشت:

```
for(k=1; k <= n; k++)
  for(i=1; i <= n; i++)
    for(j=1; j <= n; j++)
      A[i,j] = min(A[i,j], A[i,k] + A[k,j])
```

توجه کنید از آنجا که پس از محاسبه ماتریس  $A_k$  نیاز به  $A_{k-1}$  نداریم، در هر مرحله جواب  $A_k$  را در ماتریس قدیمی  $A_{k-1}$  ذخیره کرده ایم و در نهایت  $(k=n)$ ، بدست آمده، جواب است.

**توجه:** مرتبه الگوریتم نوشته شده چون سه حلقه تو در تو دارد  $O(n^3)$  است.

تاکنون ماتریس  $A$  را پیدا کردیم که حداقل طول مسیر از هر گره  $i$  به هر گره  $j$  را نشان می‌دهد. حال می‌خواهیم خود مسیر را مشخص کنیم. برای این منظور ماتریس  $p$  را بدست می‌آوریم.  $P$  یک ماتریس  $n \times n$  با مقدار اولیه صفر است:

```
for(k=1; k <= n; k++)
  for(i=1; i <= n; i++)
    for(j=1; j <= n; j++)
      if (A[i,k] + A[k,j] < A[i,j])
      {
        p[i,j] = k;
        A[i,j] = A[i,k] + A[k,j];
      }
```

برای گراف مثال قبل داریم:

$$p_0 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

پس از تاثیر گره ۱ ( $k=1$ ):

$$\Rightarrow p_1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \end{matrix}$$

پس از تاثیر گره ۲ ( $k=2$ ):

$$\Rightarrow p_2 = \begin{bmatrix} 0 & 0 & 2 & 2 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

پس از تاثیر گره ۳ ( $k=3$ ):

$$\Rightarrow p_3 = \begin{bmatrix} 0 & 0 & 2 & 2 \\ 3 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

پس از تاثیر گره ۴ ( $k=4$ ):

$$\Rightarrow p = p_4 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 0 & 4 & 2 \\ 4 & 0 & 4 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \end{matrix}$$

درایه  $p[1,3]=4$  یعنی کوتاهترین مسیر از راس ۱ به ۳ شامل راس ۴ است. لذا باید  $p[1,4]$  و  $p[4,3]$  را بررسی کنیم.  $p[1,4]=2$  یعنی در مسیر راس ۱ به راس ۴ وجود دارد.  $p[4,3]=0$  یعنی در مسیر ۴ به ۳ راس واسطی وجود ندارد. پس کوتاهترین مسیر از راس ۱ به راس ۳ مسیر:  $1 \rightarrow 2 \rightarrow 4 \rightarrow 3$  می‌باشد.

تابع زیر رئوس مسیر  $i$  به  $j$  را چاپ می‌کند:

```
void path (int i, int j)
{
    if (p[i][j] != 0)
    {
        path(i, p[i][j]);
        printf(p[i][j]);
        path(p[i][j], j);
    }
}
```

#### نکته

- ۱- فلویید برای گراف جهت‌دار با یال منفی نیز درست جواب می‌دهد و می‌توان با فلویید سیکل منفی را نیز در گراف تشخیص داد (به عنوان تمرین بررسی کنید فلویید چگونه سیکل منفی را می‌فهمد)
- ۲- می‌توان با روش ساده دیگری (ولی بد) که شبیه ضرب ماتریس‌هاست، نیز طول کوتاه‌ترین مسیر بین هر دو رأس را یافت. اگر  $C$  ماتریس هزینه باشد،  $L_1$  و  $L_2$  و ...  $L_{n-1}$  را محاسبه می‌کنیم:

$$L_1 = C, L_2 = L_1 \cdot C = C^2, L_3 = L_2 \cdot C = C^3, \dots, L_{n-1} = L_{n-2} \cdot C = C^{n-1}$$

این روابط همان ضرب ماتریس‌هاست<sup>۱</sup> با این تفاوت که به جای عمل ضرب ( $\bullet$ ) عمل  $+$  و به جای عمل جمع ( $\Sigma$ )، عمل مینیمم‌گیری قرار دهید.

روی گراف مثال قبل نشان می‌دهیم:

$$L_1 = C = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & \infty & 0 & 15 \\ 15 & \infty & 5 & 0 \end{bmatrix} \end{matrix} \Rightarrow L_2 = C \cdot C = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 5 & 20 & 10 \\ 20 & 0 & 10 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{bmatrix} \end{matrix}$$

$$\Rightarrow L_3 = L_2 \cdot C = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 5 & 15 & 10 \\ 20 & 0 & 10 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{bmatrix} \end{matrix}$$

۱- در ضرب ماتریس‌ها  $A_{n \times n} \times B_{n \times n} = C_{n \times n}$ ، درایه  $c_{ij}$  از ماتریس  $C$  به صورت  $c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}$  محاسبه می‌شود.

که  $L_3$  را جواب است. دقت کنید مثلاً درایه سطر ۱ ستون ۴ از  $L_3$  از عمل روی سطر ۱ در ستون ۴ از  $L_1$  به دست می‌آید:

$$\min(0 + \infty, 5 + 5, \infty + 15, \infty + 0) = 10$$

مرتبه این الگوریتم  $\theta(n^4)$  است ولی می‌توان با کمک روابط زیر با مرتبه  $\theta(n^3 \lg n)$  الگوریتم را پیاده‌سازی کرد:

$$L_1 = C, L_2 = C^2 = C \cdot C, L_4 = C^2 \cdot C^2, L_8 = C^4 \cdot C^4, \dots$$

همانطور که ملاحظه می‌شود به صورت لگاریتمی به  $L_{n-1}$  می‌رسیم.

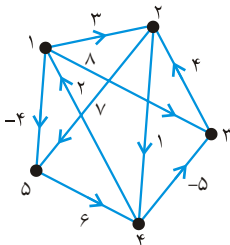
**توجه:** اگر  $A$  ماتریس مجاورت باشد و  $L_{ij}^{(k)}$  برابر کوتاهترین مسیر از  $i$  به  $j$  با حداکثر  $k$  یال

باشد آنگاه فرمول بازگشتی روش شبه ضرب ماتریسی  $(L_{ij}^{(k)} = \min_{1 \leq t \leq n} (L_{it}^{(k-1)} + w(t, j)))$  است و

فرمول بازگشتی روش تسریع شده  $(L_{ij}^{(k)} = \min_{1 \leq t \leq n} (L_{it}^{\lfloor \frac{k}{2} \rfloor} + L_{tj}^{\lceil \frac{k}{2} \rceil}))$  می‌باشد که  $L_1 = A$  و  $L_{n-1}$

جواب است.

**مثال ۸:** برای گراف مقابل با روش فوق،  $L_4$  (جواب) محاسبه شده است:



$$L_1 = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 1 & 0 & 3 & 8 & \infty & -4 \\ 2 & \infty & 0 & \infty & 1 & 7 \\ 3 & \infty & 4 & 0 & \infty & \infty \\ 4 & 2 & \infty & -5 & 0 & \infty \\ 5 & \infty & \infty & \infty & 6 & 0 \end{bmatrix} \Rightarrow L_2 = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 1 & 0 & 3 & 8 & 2 & -4 \\ 2 & 3 & 0 & -4 & 1 & 7 \\ 3 & \infty & 4 & 0 & 5 & 11 \\ 4 & 2 & -1 & -5 & 0 & -2 \\ 5 & 8 & \infty & 1 & 6 & 0 \end{bmatrix} \Rightarrow L_3 = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 1 & 0 & 3 & -3 & 2 & -4 \\ 2 & 3 & 0 & -4 & 1 & -1 \\ 3 & 7 & 4 & 0 & 5 & 11 \\ 4 & 2 & -1 & -5 & 0 & -2 \\ 5 & 8 & 5 & 1 & 6 & 0 \end{bmatrix}$$

$$\Rightarrow L_4 = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 1 & 0 & 1 & -3 & 2 & -4 \\ 2 & 3 & 0 & -4 & 1 & -1 \\ 3 & 7 & 4 & 0 & 5 & 3 \\ 4 & 2 & -1 & -5 & 0 & -2 \\ 5 & 8 & 5 & 1 & 6 & 0 \end{bmatrix}$$

البته می‌توان  $L_3$  را محاسبه نکرد ( $L_4 = L_2 \cdot L_2$ )

### ۳- کوله پشتی ۱-۰ با ارزش ماکزیمم

مساله کوله پشتی را در بخش قبل مطرح کردیم و یک راه حل بهینه به روش حریصانه ارائه نمودیم. در آنجا مجاز بودیم که یک شی را کامل انتخاب کنیم و یا کسری از شی را انتخاب کنیم. چنانچه مجاز نباشیم بخشی از شی را انتخاب کنیم یعنی شی یا باید انتخاب شود و یا اصلاً انتخاب نشود، در این صورت مسئله را کوله پشتی ۱-۰ می‌گویند. با روش حریصانه نمی‌توان جواب بهینه برای کوله پشتی ۱-۰ بدست آورد.

راه حل‌های ارائه شده برای کوله پشتی ۱-۰ دارای پیچیدگی محاسباتی بالایی هستند و در واقع این مساله یک مساله رام نشدنی (intractable) است. برای تعداد اشیاء محدود و ظرفیت کم با برنامه‌نویسی پویا می‌توان یک جواب بهینه برای مساله پیدا کرد.

فرضیات:

$M =$  حداکثر ظرفیت کوله

$(w_1, w_2, \dots, w_n)$  = جرم اشیاء

$(p_1, p_2, \dots, p_n)$  = ارزش اشیاء

$x_1, x_2, \dots, x_n$  = میزان انتخاب از هر شی ،  $x_i = 0$  یا ۱

هدف ماکزیمم‌سازی  $\sum_{i=1}^n p_i x_i$  به شرطی که  $\sum_{i=1}^n w_i x_i \leq M$

**مثال ۹:** اگر  $(w_1, w_2, w_3) = (5, 10, 20)$  و  $(p_1, p_2, p_3) = (50, 60, 140)$  و  $M = 30$  باشد

در این صورت  $\left( \frac{p_1}{w_1}, \frac{p_2}{w_2}, \frac{p_3}{w_3} \right) = (10, 6, 7)$  و حال اگر طبق روش حریصانه اشیاء را

انتخاب کنیم (یا انتخاب نکنیم ۱ و  $x_i = 0$ ) آنگاه اشیاء ۱ و ۳ انتخاب می‌شوند و سود حاصل ۱۹۰ می‌باشد. حال آنکه بهینه آن است که شی ۲ و ۳ انتخاب شوند تا سود حاصل ۲۰۰ باشد. ملاحظه می‌شود که روش حریصانه برای کوله پشتی ۱-۰ حل بهینه تولید نمی‌کند.

برای مساله کوله پشتی ۱-۰، اصل بهینگی برقرار است که نشان می‌دهیم.

اگر  $A$  یک زیر مجموعه بهینه از  $n$  قطعه باشد، دو حالت امکان پذیر است: یا  $A$  حاوی قطعه  $n$ ام هست یا نیست. اگر  $A$  حاوی قطعه  $n$ ام نباشد،  $A$  با زیر مجموعه‌ای بهینه از  $(n-1)$  قطعه اول برابر است. اگر  $A$  حاوی قطعه  $n$ ام باشد، منفعت کل برابر  $p_n$  به علاوه منفعت بهینه است هنگامی که قطعات را از  $n-1$  قطعه اول انتخاب می‌کنیم (البته با رعایت این اصل که وزن کل از  $M - w_n$  بیشتر نشود) لذا اصل بهینگی برقرار است.

اگر  $c[i][m]$  سود بهینه حاصل از انتخاب  $i$  قطعه اول باشد به گونه‌ای که وزن کل آن از  $m$

تجاوز نکند داریم:

$$c[i][m] = \begin{cases} 0 & \text{if } i = 0 \text{ or } m = 0 \\ \max(c[i-1][m], p_i + c[i-1][m-w_i]) & \text{if } w_i \leq m \\ c[i-1][m] & \text{if } w_i > m \end{cases}$$

و جواب مسئله  $c[n][M]$  می‌باشد.

الگوریتم زیر  $c[0..n][0..M]$  را به ترتیب سطری محاسبه می‌کند و  $c[n][M]$  جواب است.

$p[1..n]$  values ,  $w[1..n]$  weights

for  $m \leftarrow 0$  to  $M$  do  $c[0][m] \leftarrow 0$ .

for  $i \leftarrow 1$  to  $n$  do

{

$c[i][0] \leftarrow 0$ .

for  $m \leftarrow 1$  to  $M$  do

if  $w[i] \leq m$

then if  $p[i] + c[i-1][m-w[i]] > c[i-1][m]$

then  $c[i][m] \leftarrow p[i] + c[i-1][m-w[i]]$

else  $c[i][m] \leftarrow c[i-1][m]$

else  $c[i][m] \leftarrow c[i-1][m]$

}

مرتبه این الگوریتم  $\theta(nM)$  است و اگر  $M$  از  $2^n$  بیشتر باشد، مرتبه الگوریتم از روش عادی

با مرتبه  $\theta(2^n)$  نیز بدتر می‌شود ولی می‌توان طوری آن را بهبود بخشید که مرتبه کوله پشتی

صفر و یک  $O(\min(2^n, nM))$  باشد (با روش backtrack).

**تمرین:** در مسئله کوله پشتی صفر و یک، فرض کنید عناصر را به ترتیب صعودی وزن که مرتب

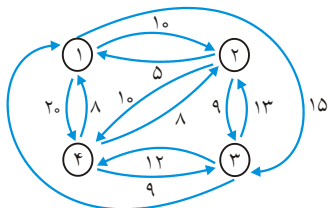
کردیم، به ترتیب نزولی ارزش مرتب شده‌اند، یک الگوریتم کارا برای یافتن جواب بهینه بیابید.

#### ۴- فروشنده دوره‌گرد

در یک گراف وزن‌دار، می‌خواهیم یک سیکل بهینه هامیلتونی (تور) بیابیم. یعنی می‌خواهیم از

یک راس شروع کنیم، همه رئوس را یک بار و فقط یک بار ملاقات کنیم و به راس شروع باز گردیم

به شرطی که جمع وزن یال‌ها مینیمم باشد.



**مثال ۱۰:** در گراف مقابل سیکل [۱ و ۲ و ۳ و ۴ و ۱] در گراف مقابل سیکل [۱ و ۲ و ۳ و ۴ و ۱]

سیکل بهینه هامیلتونی است که هزینه آن

$35 = 10 + 10 + 9 + 6$  می‌باشد.

در یک گراف  $n$  راسی اگر بخواهیم با روش معمولی کل سیکل‌ها را بررسی کنیم و سیکل بهینه را بیابیم به  $(n-1)!$  بررسی نیاز است. ولی با روش برنامه‌سازی پویا می‌توان الگوریتمی بیان کرد از مرتبه  $\theta(n^2 2^n)$  که بسیار بهتر از  $(n-1)!$  است. مثلاً اگر هر عمل اصلی یک میکروثانیه به طول انجامد برای گراف  $n = 20$  راسی داریم:

سال  $3857 \mu\text{sec} = (20-1)! \mu\text{sec}$  = روش معمولی یافتن کلیه تورها

دقیقه  $7 \mu\text{sec} \simeq 20 \times 20 \mu\text{sec}$  = روش برنامه‌نویسی پویا

### نکته

- ۱- حافظه مصرفی الگوریتم دوره‌گرد با روش برنامه‌سازی پویا از مرتبه  $\theta(n^2)$  است.
- ۲- تاکنون کسی نتوانسته است برای مساله فروشنده دوره گرد الگوریتمی بهتر از حالت نمایی پیدا کند، و همچنین کسی هم عدم امکان یافتن چنین الگوریتمی را اثبات نکرده است. (همانند مساله کوله پشتی  $(0-1)$ )

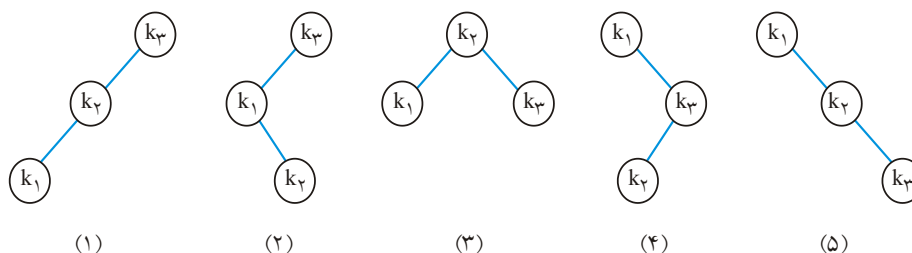
## ۵- درخت جستجوی دودویی بهینه (Optimal BST)

فرض کنید  $n$  تا کلید متمایز  $k_1 < k_2 < \dots < k_n$  وجود دارد. و احتمال آنکه کلید  $k_i$  را جستجو کنیم برابر  $p_i$  است. می‌خواهیم با این کلیدها یک BST بسازیم به طوریکه عبارت

$$\sum_{i=1}^n c_i p_i \quad \text{مینیمم شود. } c_i \text{ برابر تعداد مقایسه‌ها برای یافتن } k_i \text{ در BST است. عبارت}$$

$$\sum_{i=1}^n c_i p_i \quad \text{زمان جستجوی میانگین برای جستجوی کلیدهاست که می‌خواهیم مینیمم شود.}$$

**مثال ۱۱:** با ۳ کلید  $k_1 < k_2 < k_3$ ، درخت جستجوی دودویی وجود دارد. با فرض اینکه  $p_1 = 0/7$  و  $p_2 = 0/2$  و  $p_3 = 0/1$ ، میانگین زمان جستجو را برای هر درخت محاسبه کنید.



(دقت کنید در درخت (۱) برای یافتن  $k_3$ ، یک مقایسه برای یافتن  $k_2$ ، دو مقایسه و برای  $k_1$ ، سه مقایسه انجام می‌شود)



$$۱) ۳(۰/۷) + ۲(۰/۲) + ۱(۰/۱) = ۲/۶$$

$$۲) ۲(۰/۷) + ۳(۰/۲) + ۱(۰/۱) = ۲/۱$$

$$۳) ۲(۰/۷) + ۱(۰/۲) + ۲(۰/۱) = ۱/۸$$

$$۴) ۱(۰/۷) + ۳(۰/۲) + ۲(۰/۱) = ۱/۵$$

$$۵) ۱(۰/۷) + ۲(۰/۲) + ۳(۰/۱) = ۱/۴$$

در این مثال درخت ۵، بهینه است. درواقع کلیدهایی که خیلی به ندرت جستجو می‌شوند (احتمال‌شان کم است) بهتر است در سطوح آخر و کلیدهایی که احتمال جستجوی آنها زیاد است، در سطوح بالایی باشند.

برای یافتن BST بهینه، شاید یک راه آن باشد که همه حالات را بررسی کنیم. ولی مرتبه این عمل از نمایی بدتر است. می‌خواهیم با روش برنامه‌نویسی پویا، BST بهینه را بیابیم. زیر درختان یک درخت BST بهینه، خود BST بهینه هستند. یعنی اصل بهینگی برقرار است.

در مثال قبل درخت ۵ بهینه بود، می‌توان بررسی کرد که  $k_2$  نیز بهینه است.

برای رسیدن به الگوریتم، ماتریس A را تعریف می‌کنیم که مفهوم هر درایه آن به صورت زیر است:

$A[i][j] = k_i$  تا  $k_j$  با BST زمان جستجو برای

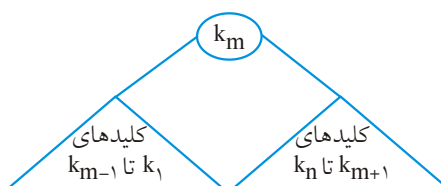
به طوریکه  $k_i \leq k_j$  و  $1 \leq i \leq j \leq n$

بدیهی است که  $A[i][i] = p_i$  چرا که در این حالت BST فقط شامل  $k_i$  است و یک مقایسه

$$A[i][i] = 1 \times p_i = 1 \text{ است پس } p_i \text{ احتمال آن نیز } p_i \text{ است}$$

جواب ما  $A[1][n]$  است. فرض کنید BST بهینه را یافته‌ایم و ریشه آن  $k_m$  است. بنابر اصل

بهینگی زیر درختان چپ و راست نیز خودشان BST بهینه هستند:



$$A[1][n] = \underbrace{A[1][m-1]}_{(۱)} + \underbrace{p_1 + \dots + p_{m-1}}_{(۲)} + \underbrace{p_m}_{(۳)} + \underbrace{A[m+1][n]}_{(۴)} + \underbrace{p_{m+1} + \dots + p_n}_{(۵)}$$

$$= A[1][m-1] + A[m+1][n] + \sum_{t=1}^n p_t$$

در رابطه فوق،

عبارت ۱: مینیمم زمان جستجو در زیر درخت چپ.

عبارت ۲: هر یک از نودهای زیر درخت چپ را که بخواهیم جستجو کنیم، بخاطر مقایسه با ریشه، یک مقایسه اضافی دارد.

عبارت ۳: خود ریشه یک مقایسه دارد و احتمال آن نیز  $p_m$  است.

عبارت ۴: مینیمم زمان جستجو در زیر درخت راست.

عبارت ۵: مشابه عبارت ۲، هر یک از نودهای زیر درخت راست، یک مقایسه اضافی با ریشه دارند.

رابطه فوق با این فرض بود که  $k_m$  ریشه باشد ولی با توجه به اینکه هر یک از کلیدهای ۱ تا  $n$  می‌توانند ریشه باشند:

$$A[1][n] = \min_{1 \leq m \leq n} (A[1][m-1] + A[m+1][n]) + \sum_{t=1}^n p_t$$

بنابراین فرمول کلی برای  $A[i][j]$  عبارت است از:

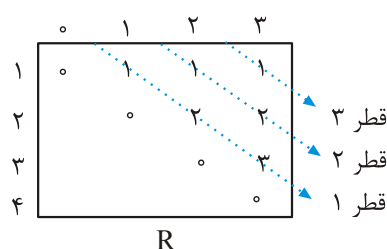
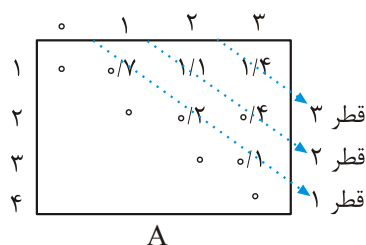
$$A[i][j] = \min_{i \leq m \leq j} (A[i][m-1] + A[m+1][j]) + \sum_{t=i}^j p_t, \quad (i < j)$$

$$A[i][i] = p_i$$

از این فرمول برای ساخت ماتریس  $A$  استفاده می‌شود. جهت سهولت ماتریس را یک ماتریس  $(n+1) \times (n+1)$  به شکل  $A[1..n+1, 0..n]$  در نظر می‌گیریم که قطر اصلی آن صفر است. ماتریس دیگری به نام  $R$  نیز تعریف می‌کنیم که  $R[i][j]$  بیانگر اندیس کلیدی است که به عنوان ریشه انتخاب شده (درخت شامل  $k_i$  تا  $k_j$ ). از ماتریس  $R$  برای ساخت خود درخت کمک خواهیم گرفت. ابعاد ماتریس  $R$  نیز مشابه  $A$  است و این ماتریسها، قطر به قطر پر می‌شوند.

**مثال ۱۲:** درخت BST بهینه و هزینه آن را برای ۳ کلید  $k_1 < k_2 < k_3$  با احتمال  $p_1 = 0.1$  و  $p_2 = 0.2$  و  $p_3 = 0.7$  بیابید.

پاسخ:



**قطر ۱:** با توجه به اینکه  $A[i][i] = p_i$  پس در قطر ۱  $(A[۳][۳], A[۲][۲], A[۱][۱])$  احتمالات  $(p_۳, p_۲, p_۱)$  را قرار می‌دهیم. در ماتریس  $R$  نیز قرار می‌دهیم  $R[i][i] = i$  چرا که با کلید  $k_i$  ریشه خود  $k_i$  است.

$$\begin{aligned} A[۱][۲] &= \min_{۱ \leq m \leq ۲} (A[۱][m-۱] + A[m+۱][۲]) + \sum_{t=۱}^۲ p_t \\ &= \min (\underbrace{A[۱][۰]}_{\circ} + \underbrace{A[۲][۲]}_{\circ/۲}, \underbrace{A[۱][۱]}_{\circ/۱} + \underbrace{A[۳][۲]}_{\circ}) + (p_۱ + p_۲) \\ &= \min (\underbrace{\circ/۲}_{m=۱}, \underbrace{\circ/۱}_{m=۲}) + (\circ/۱ + \circ/۲) = ۱/۱ \end{aligned}$$

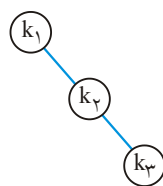
پس در  $R[۱][۲]$  عدد  $m=۱$  قرار می‌گیرد.

$$\begin{aligned} A[۲][۳] &= \min_{۲ \leq m \leq ۳} (A[۲][m-۱] + A[m+۱][۳]) + \sum_{t=۲}^۳ p_t \\ &= \min (\underbrace{A[۲][۱]}_{\circ} + \underbrace{A[۳][۳]}_{\circ/۱}, \underbrace{A[۲][۲]}_{\circ/۲} + \underbrace{A[۴][۳]}_{\circ}) + (p_۲ + p_۳) \\ &= \circ/۱ + (\circ/۲ + \circ/۱) = \circ/۴ \Rightarrow R[۲][۳] = ۲ \end{aligned}$$

$$\begin{aligned} A[۱][۳] &= \min_{۱ \leq m \leq ۳} (A[۱][m-۱] + A[m+۱][۳]) + \sum_{t=۱}^۳ p_t \\ &= \min (A[۱][۰] + A[۲][۳], A[۱][۱] + A[۳][۳], A[۱][۲] + A[۴][۳]) + (p_۱ + p_۲ + p_۳) \\ &= \min (\underbrace{\circ + \circ/۴}_{k=۱}, \underbrace{\circ/۱ + \circ/۱}_{k=۲}, \underbrace{۱/۱ + \circ}_{k=۳}) + (\circ/۱ + \circ/۲ + \circ/۱) = ۱/۴ \Rightarrow R[۱][۳] = ۱ \end{aligned}$$

بنابراین متوسط زمان جستجو  $۱/۴$  است. برای رسم درخت، با توجه به اینکه  $R[۱][۳] = ۱$  پس  $k_۱$  ریشه درخت شامل کلیدهای  $k_۱$  تا  $k_۳$  است.

و چون  $R[۲][۳] = ۲$  پس  $k_۲$  ریشه درخت شامل کلیدهای  $k_۲$  و  $k_۳$  است و چون  $k_۲ > k_۱$  پس  $k_۲$  را سمت راست  $k_۱$  می‌کشیم و در نهایت چون  $k_۳ > k_۲$ ،  $k_۳$  را نیز سمت راست  $k_۲$  می‌کشیم:



برای پیاده‌سازی الگوریتم، نیاز به ۳ حلقه تو در توست و مرتبه آن  $\theta(n^۳)$

است.

## ۶- بزرگترین زیردنباله مشترک (LCS)<sup>۱</sup>

فرض کنید دنباله  $X = \langle A, B, C, B, D, A, B \rangle$  داده شده است زیر دنباله  $Z = \langle B, C, A \rangle$  از  $X$  زیردنباله‌ای است که متناظر با دنباله اندیسه‌های  $\langle ۲, ۳, ۶ \rangle$  است. یعنی زیردنباله  $Z$ ، به ترتیب اندیسه‌های ۲ و ۳ و ۶ دنباله  $X$  را شامل شده است. فرض کنید دنباله  $Y = \langle B, D, C, A, B, A \rangle$  در این صورت  $Z$  زیردنباله  $Y$  نیز هست که متناظر با اندیسه‌های  $\langle ۱, ۳, ۴ \rangle$  است. یعنی  $Z$  یک زیردنباله مشترک برای  $X$  و  $Y$  است که طولش ۳ است. زیردنباله  $M = \langle B, C, B, A \rangle$  نیز زیردنباله مشترک برای  $X$  و  $Y$  است. که طولش ۴ است. می‌توان بررسی کرد که زیردنباله مشترک با طول بیشتر از ۴ برای  $X$  و  $Y$  وجود ندارد، پس  $M$  بزرگترین زیردنباله مشترک (LCS) برای  $X$  و  $Y$  است. البته  $N = \langle B, D, A, B \rangle$  نیز یک LCS برای  $X$  و  $Y$  است. ما می‌خواهیم طول  $LCS(X, Y)$  و همچنین خود  $LCS(X, Y)$  را بیابیم. مثلاً طول  $LCS(X, Y)$  برابر ۴ است و خود  $LCS(X, Y)$  برابر  $\langle B, D, A, B \rangle$  یا  $\langle B, C, B, A \rangle$  یا حتی  $\langle B, C, A, B \rangle$  است.

فرض کنید  $X = \langle x_1, x_2, \dots, x_m \rangle$  و  $Y = \langle y_1, y_2, \dots, y_n \rangle$ . یک روش برای یافتن  $LCS(X, Y)$  این است که تمام زیردنباله‌های  $X$  که  $2^m$  است را بیابیم و بررسی کنیم کدام، زیردنباله  $Y$  نیز هست و بزرگترین آن را انتخاب کنیم که این از مرتبه نمایی است. ولی می‌توان با روش برنامه‌ریزی پویا،  $LCS$  را یافت. فرض کنید  $Z = \langle z_1, z_2, \dots, z_k \rangle$ ،  $LCS$  دنباله‌های  $X$  و  $Y$  باشد؛ و همچنین فرض کنید  $X_i$  پیشوند  $i$ ام  $X$  باشد. مثلاً اگر  $X = \langle A, B, C, B, D, A, B \rangle$  در این صورت  $X_3 = \langle A, B, C \rangle$  یا  $X_0 = \langle \rangle$  و  $X_Y = X$ . جملات بازگشتی زیر قابل بیان است.

$$۱) \text{ if } x_m = y_n \Rightarrow (z_k = x_m = y_n) \text{ and } Z_{k-1} = LCS(X_{m-1}, Y_{n-1})$$

$$۲) \text{ if } x_m \neq y_n \Rightarrow \begin{cases} \text{if } z_k \neq x_m \Rightarrow Z = LCS(X_{m-1}, Y) \\ \text{if } z_k \neq y_n \Rightarrow Z = LCS(X, Y_{n-1}) \end{cases}$$

دقت کنید که  $Z$ ،  $LCS$  دنباله‌های  $X$  و  $Y$  است. پس جمله ۱ فوق صحیح است زیرا اگر آخرین کاراکتر دنباله‌های  $X$  و  $Y$  مساوی باشد ( $x_m = y_n$ ) آنگاه حتماً آخرین کاراکتر دنباله  $Z$  یعنی  $z_k$  با  $x_m$  یا  $y_n$  مساوی است و  $Z_{k-1} = \langle z_1, z_2, \dots, z_{k-1} \rangle$  یک  $LCS$  برای  $X_{m-1}$  و  $Y_{n-1}$  است. بررسی کنید که جمله ۲ فوق نیز صحیح است.

فرض کنید  $C[i, j]$  برابر طول LCS دنباله‌های  $X_i$  و  $Y_j$  باشد واضح است که اگر  $i = 0$  یا  $j = 0$  در این صورت  $C[i, j] = 0$  است. با توجه به بحث‌های انجام شد. فرمول بازگشتی زیر بدست می‌آید:

$$C[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ C[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j \\ \max(C[i, j-1], C[i-1, j]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j \end{cases}$$

الگوریتم زیر با روش پویا و مرتبه  $\theta(mn)$ ، سطر به سطر  $C$  را می‌سازد تا به  $C[m, n]$  برسیم که طول LCS را می‌دهد. آرایه  $C$  به صورت  $C[0..m, 0..n]$  تعریف می‌شود. همچنین در این الگوریتم آرایه  $b[1..m, 1..n]$  مقداردهی می‌شود که با کمک آن می‌توان خود LCS را یافت:

#### LCS - LENGTH (X, Y, m, n)

```
for i ← 1 to m do C[i, 0] ← 0
for j ← 0 to n do C[0, j] ← 0
for i ← 1 to m
  do for j ← 1 to n
    do if  $x_i = y_j$ 
      then  $C[i, j] \leftarrow C[i-1, j-1] + 1$ ,  $b[i, j] \leftarrow "\diagdown"$ 
    else if  $C[i-1, j] \geq C[i, j-1]$ 
      then  $C[i, j] \leftarrow C[i-1, j]$ ,  $b[i, j] \leftarrow "\uparrow"$ 
    else  $C[i, j] \leftarrow C[i, j-1]$ ,  $b[i, j] \leftarrow "\leftarrow"$ 
return b and C
```

برای چاپ خود LCS می‌توان الگوریتم زیر را با مرتبه  $O(m+n)$  ارائه داد:

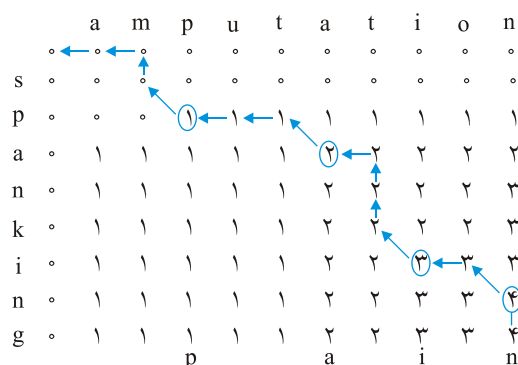
#### PRINT(b, X, i, j)

```
if  $i = 0$  or  $j = 0$  then return
if  $b[i, j] = "\diagdown"$  then PRINT(b, X, i-1, j-1), write  $x_i$ 
else if  $b[i, j] = "\uparrow"$  then PRINT(b, X, i-1, j)
else PRINT(b, X, i, j-1)
```

فراخوانی اولیه این تابع  $\text{PRINT}(b, X, m, n)$  است.

در تابع فوق هرگاه  $b[i, j] = "\diagdown"$  یعنی یک کاراکتر مشترک در  $X$  و  $Y$  دیده شده است پس LCS در واقع مکان‌هایی هستند که علامت  $\diagdown$  دارند.

**مثال ۱۳:** با توجه به شکل زیر دو رشته  $X=\text{spanking}$  و  $Y=\text{amputation}$  داده شده‌اند که  $\text{Lcs}(X,Y)$  برابر رشته Pain می‌باشد.



## ۷- ضریب دو جمله‌ای

ضریب دو جمله‌ای  $\binom{n}{k}$  دارای فرمول بازگشتی مقابل است:

$$\binom{n}{k} = \begin{cases} \binom{n-1}{k-1} + \binom{n-1}{k} & 0 < k < n \\ 1 & k = 0 \text{ or } k = n \end{cases}$$

اگر بخواهیم با روش تقسیم و غلبه  $\binom{n}{k}$  را محاسبه کنیم، تعداد فراخوانیها  $\binom{n}{k} - 1$  است که بسیار زیاد است. با برنامه‌نویسی پویا می‌توان از پایین به بالا نتایج را محاسبه کرده و در یک آرایه ذخیره کرد و به  $\binom{n}{k}$  رسید.

مطابق الگوریتم زیر:

```
int bin(int n, int k)
{
    int i, j;
    int B[0...n][0...k];
    for (i=0; i<=n; i++)
        for (j=0; j<=min(i,k); j++)
            if (j==0 || j==i)
                B[i][j]=1;
            else
                B[i][j]=B[i-1][j-1]+B[i-1][j];
    Return B[n][k];
}
```

تعداد اجرای if برابر است با:

$$1 + 2 + 3 + \dots + k + \underbrace{(k+1) + (k+1) + \dots + (k+1)}_{\text{بار } n-k+1} = \frac{k(k+1)}{2} + (n-k+1)(k+1)$$

$$= \frac{(2n-k+2)(k+1)}{2} \in \theta(nk)$$

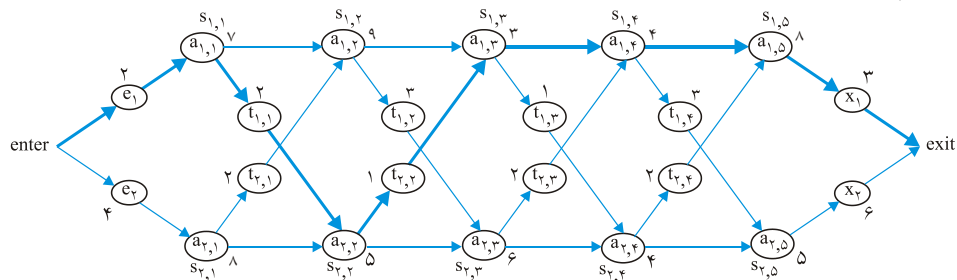
که خیلی از روش تقسیم و غلبه سریعتر است.

لازم به ذکر است که الگوریتم فوق را می‌توان با کمک یک آرایه یک بعدی  $B[0 \dots k]$  نوشت که به عهده شما.

### ۸- زمان‌بندی خط تولید (assembly-line scheduling)

در یک کارخانه اتومبیل‌سازی، دو خط تولید وجود دارد، هر خط دارای  $n$  ایستگاه است  $(S_{1,1}, S_{1,2}, \dots, S_{1,n})$  برای خط ۱ و  $(S_{2,1}, S_{2,2}, \dots, S_{2,n})$  ایستگاه‌های خط ۲. ایستگاه  $i$  ام خط ۱ و ۲ یک کار یکسان انجام می‌دهند  $(S_{1,i})$  با  $(S_{2,i})$  و ... یکسان هستند ولی هزینه آنها لزوماً یکسان نیست. هزینه ایستگاه  $S_{i,j}$  ( $i=1,2$ ) برابر  $a_{i,j}$  فرض می‌شود. هزینه ورود بدنه اتومبیل به خط ۱ برابر  $e_1$  و به خط ۲ برابر  $e_2$  و هزینه خروج از خطوط ۱ و ۲ به ترتیب  $x_1$  و  $x_2$  است. بدنه می‌تواند از یک خط به خط دیگر نیز منتقل شود. هزینه انتقال از  $S_{1,j}$  به  $S_{2,j+1}$  برابر  $t_{1,j}$  است و هزینه انتقال از ایستگاه  $S_{2,j}$  به  $S_{1,j+1}$  برابر  $t_{2,j}$  است.

می‌خواهیم ایستگاه‌هایی از خطوط ۱ و ۲ را انتخاب کنیم که کل هزینه‌ها مینیمم شود. در شکل زیر مسیر پررنگ نشان‌دهنده مسیر بهینه است که هزینه آن  $35 = 2 + 7 + 2 + 5 + 1 + 3 + 4 + 8 + 3$  است.



برای حل مسئله،  $f_i[j]$  را برابر کمترین هزینه رسیدن به ایستگاه  $S_{i,j}$  ( $j=1, \dots, n, i=1,2$ ) فرض می‌کنیم (هزینه خود  $S_{i,j}$  نیز در نظر گرفته می‌شود) مثلاً در شکل فوق  $f_1[1] = 2 + 7 = 9$  و یا  $f_2[1] = 4 + 8 = 12$ .  $f^* = 35$  در شکل فوق  $f^*$  را برابر کمترین هزینه کل فرض می‌کنیم. واضح است که  $f^* = \min(f_1[n] + x_1, f_2[n] + x_2)$  به راحتی می‌توان فرمولهای بازگشتی زیر را

نوشت:

$$f_1[1] = e_1 + a_{1,1}, \quad f_2[1] = e_2 + a_{2,1}$$

$$f_1[j] = \min(f_1[j-1] + a_{1,j}, f_2[j-1] + t_{2,j-1} + a_{1,j}) \quad j = 2, \dots, n$$

$$f_2[j] = \min(f_2[j-1] + a_{2,j}, f_1[j-1] + t_{1,j-1} + a_{2,j}) \quad j = 2, \dots, n$$

با تعاریف فوق می‌توان هزینه بهینه را یافت ( $f^*$ ) ولی برای آنکه مشخص شود، چه ایستگاه‌هایی باید انتخاب شوند،  $L_i[j]$  را برابر شماره خطی می‌گیریم که ایستگاه  $j-1$  آن قبل از  $S_{i,j}$  استفاده شده است یعنی ایستگاه  $S_{L_i[j],j-1}$  قبل از ایستگاه  $S_{i,j}$  است.  $(i=1,2, j=2,\dots,n)$  و  $L^*$  را شماره خطی می‌گیریم که ایستگاه  $n$  از آن انتخاب شده است. در شکل قبل،  $L^* = 1$  و  $L_1[2]=1$  زیرا برای رسیدن به  $S_{1,2}$  بهترین مسیر، عبور از  $S_{1,1}$  است. و یا  $L_2[2]=1$  زیرا بهترین مسیر برای رسیدن به  $S_{2,2}$  عبور از  $S_{1,1}$  است:

| $f_1[1]$ | $f_1[2]$ | $f_1[3]$ | $f_1[4]$ | $f_1[5]$ | $f_2[1]$ | $f_2[2]$ | $f_2[3]$ | $f_2[4]$ | $f_2[5]$ |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| ۹        | ۱۸       | ۲۰       | ۲۴       | ۳۲       | ۱۲       | ۱۶       | ۲۲       | ۲۵       | ۳۰       |

$$f^* = 35$$

| $L_1[2]$ | $L_1[3]$ | $L_1[4]$ | $L_1[5]$ | $L_2[2]$ | $L_2[3]$ | $L_2[4]$ | $L_2[5]$ |
|----------|----------|----------|----------|----------|----------|----------|----------|
| ۱        | ۲        | ۱        | ۱        | ۱        | ۲        | ۱        | ۲        |

$$L^* = 1$$

الگوریتم یافتن بهترین مسیر، با روش برنامه‌ریزی پویا به قرار زیر است:

FASTEST-WAY( $a, t, e, x, n$ )

$$f_1[1] \leftarrow e_1 + a_{1,1}$$

$$f_2[1] \leftarrow e_2 + a_{2,1}$$

for  $j \leftarrow 2$  to  $n$

do {if  $f_1[j-1] + a_{1,j} \leq f_2[j-1] + t_{2,j-1} + a_{1,j}$

then  $f_1[j] \leftarrow f_1[j-1] + a_{1,j}$ ,  $L_1[j] \leftarrow 1$

else  $f_1[j] \leftarrow f_2[j-1] + t_{2,j-1} + a_{1,j}$ ,  $L_1[j] \leftarrow 2$

if  $f_2[j-1] + a_{2,j} \leq f_1[j-1] + t_{1,j-1} + a_{2,j}$

then  $f_2[j] \leftarrow f_2[j-1] + a_{2,j}$ ,  $L_2[j] \leftarrow 2$

else  $f_2[j] \leftarrow f_1[j-1] + t_{1,j-1} + a_{2,j}$ ,  $L_2[j] \leftarrow 1$

}



```

if  $f_l[n] + x_l \leq f_r[n] + x_r$ 
  then  $f^* = f_l[n] + x_l, L^* = l$ 
  else  $f^* = f_r[n] + x_r, L^* = r$ 

```

و برای چاپ مسیر بهینه می‌توان تابع زیر را نوشت:

```

PRINT-STATIONS(L,n)
   $i \leftarrow L^*$ 
  Print "line" i "station" n
  for  $j \leftarrow n$  downto 2
  do  $i \leftarrow L_i[j]$ , print "line" i "station" j-1

```

خروجی این الگوریتم برای شکل قبل عبارت است از:

```

Line 1 station 5
Line 1 station 4
Line 1 station 3
Line 2 station 2
Line 1 station 1

```

مرتبه هر دو تابع فوق  $\theta(n)$  است.

**تمرین:** جداول  $L_i[j]$  و  $f_i[j]$  جمعاً شامل  $4n-2$  خانه هستند. نشان دهید که چگونه می‌توان فضا را به  $2n+2$  خانه کاهش داد به شرطی که بتوان  $f^*$  را محاسبه کرد و همچنین مسیر را چاپ کرد.

## ۹- خرد کردن سکه

دیدیم که این مسئله با روش حریصانه لزوماً جواب درستی نمی‌دهد، ولی می‌توان با برنامه‌نویسی پویا، این مسئله را حل کرد. مثلاً فرض کنید سکه‌های موجود ۱ و ۴ و ۶ واحدی هستند و باقیمانده پولی که باید پرداخت شود ۸ واحد است. با روش حریصانه، یک سکه ۶ واحدی و دو سکه ۱ واحدی یعنی جمعاً ۳ سکه پرداخت می‌شود حال آنکه جواب درست ۲ سکه ۴ واحدی است که با روش پویا می‌توان به جواب درست دست یافت.

فرض کنید  $n$  نوع سکه مختلف با ارزش‌های  $d_1, d_2, \dots, d_n$  موجود است (و از هر نوع بی‌شمار سکه) و باقیمانده پولی که باید پرداخت شود  $M$  است. برای حل مسئله جدول  $c[1..n, 0..M]$  در نظر می‌گیریم که درایه  $c[i, j]$  برابر تعداد سکه‌های پرداختی است به طوری که جمع مبلغ آنها برابر  $j$  باشد و فقط از سکه‌های ۱ تا  $i$  (با ارزش‌های  $d_1, \dots, d_i$ ) استفاده شود. پس جواب  $c[n, M]$  است که برابر تعداد سکه‌های پرداختی به مشتری است.

به راحتی می‌توان تساوی زیر را نشان داد:

$$c[i, j] = \min(c[i-1, j], 1 + c[i, j-d_i])$$

زیرا دو حالت وجود دارد، اگر از سکه‌های  $i$  استفاده نشود آنگاه  $c[i, j] = c[i-1, j]$  و اگر یک سکه  $i$  پرداخت شود آنگاه  $c[i, j] = 1 + c[i, j-d_i]$  که باید بین این دو مقدار، می‌نیمم انتخاب شود.

اگر  $i=1$  آنگاه  $c[0, j]$  تعریف نشده است که برابر  $+\infty$  تعریف می‌کنیم به همین ترتیب اگر  $j < d_i$  آنگاه  $c[i, j-d_i]$  که تعریف نشده است برابر  $+\infty$  تعریف می‌شود.

فرض کنید سکه‌های موجود  $d_1=1$  و  $d_2=4$  و  $d_3=6$  واحدی هستند و باقیمانده پولی که باید پرداخت شود  $M=8$  است. ماتریس  $c[1..3, 0..8]$  مطابق شکل زیر مقدار می‌گیرد:

$$\begin{array}{c} \begin{array}{cccccccc} & 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \end{array} \\ \begin{array}{l} d_1=1 \\ d_2=4 \\ d_3=6 \end{array} \begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ 0 & 1 & 2 & 3 & 1 & 2 & 3 & 4 & 2 \\ 0 & 1 & 2 & 3 & 1 & 2 & 1 & 2 & 2 \end{bmatrix} \end{array}$$

ستون اول ماتریس  $c$  همیشه صفر است و سایر درایه‌ها طبق فرمول گفته شده محاسبه می‌شوند مثلاً

$$c[3, 8] = \min(c[2, 8], 1 + c[3, 8-d_3]) = \min(2, 1+2) = 2$$

طبق ماتریس  $c$ ، جواب ۲ سکه است  $(c[3, 8])$ . الگوریتم به شکل زیر است:

```

d[1..n]
c[1..n, 0..M]
for i ← 1 to n do c[i, 0] ← 0
for i ← 1 to n do
  for j ← 1 to M do
    if i = 1 and j < d[i] then c[i, j] ← +∞
    else if i = 1 then c[i, j] ← 1 + c[1, j-d[1]]
    else if j < d[i] then c[i, j] ← c[i-1, j]
    else c[i, j] ← min(c[i-1, j], 1 + c[i, j-d[i]])
return c[n, M]
```

مرتبه این الگوریتم  $\theta(nM)$  است.

الگوریتم فوق فقط تعداد سکه پرداختی را مشخص می‌کند. برای تشخیص خود سکه‌ها می‌توان از ماتریس  $c$  کمک گرفت برای این منظور از  $c[n, M]$  (در این مثال از  $c[3, 8]$ ) شروع می‌کنیم. اگر

یعنی از سکه  $n$  ام هیچ پرداخت نشده و به  $c[n-1, M]$  حرکت می‌کنیم. و اگر  $c[n, M] = 1 + c[n, M - d_n]$  یعنی یک سکه از نوع  $n$  ام پرداخت شده که ارزش آن  $d_n$  است و به چپ و به  $c[n, M - d_n]$  حرکت می‌کنیم برای بررسی بعدی تا در نهایت به  $c[0, 0]$  برسیم. الگوریتمی که این عملیات را انجام دهد از مرتبه  $\theta(n + c[n, M])$  است.

در این مثال  $c[3, 8] = 2$  که با  $c[2, 8]$  مساوی است پس از سکه  $d_3 = 6$  پرداخت نشده، حال  $c[2, 8] = 1 + c[2, 8 - 4] = 1 + 1 = 2$  پس یک سکه  $d_2 = 4$  پرداخت شده و به  $c[2, 4]$  منتقل می‌شویم و چون  $c[2, 4] = 1 + c[2, 4 - 4] = 1 + 0 = 1$  پس یک سکه دیگر  $d_2 = 4$  پرداخت شده است.

### ۱۰- برش میله

یک میله به طول  $n$  داریم و می‌خواهیم از سر چپ آن را به قطعات با اندازه صحیح تقسیم کنیم. هر قطعه دارای ارزشی است، هدف این است که بیشترین سود از فروش قطعات بدست آوریم. مثلاً فرض کنید میله‌ای به طول ۴ اینچ داریم و در جدول زیر ارزش قطعات آمده است. میله ۴ اینچی را به ۸ طریق می‌توان برش زد که در شکل زیر آمده است.

| اندازه $i$ | ۱ | ۲ | ۳ | ۴ |
|------------|---|---|---|---|
| $P_i$ قیمت | ۱ | ۵ | ۸ | ۹ |



(a)



(b)



(c)



(e)



(f)



(g)



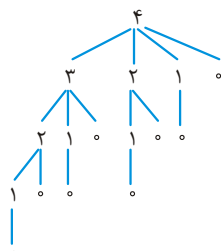
(h)

همانطور که مشاهده می‌شود، حالت (c) سود ۱۰ دارد و بهترین است (یعنی میله به طول ۴ به صورت ۲+۲ برش زده می‌شود و سود ۵+۵ دارد). به طور کلی میله به طول  $n$  را به  $2^{n-1}$  حالت می‌توان به قطعات کوچکتر تقسیم کرد. فرض کنید  $r_n$  سود حاصل از برش میله به طول  $n$  باشد، و  $P_i$  سود میله به طول  $i$  باشد، می‌توان رابطه بازگشتی  $r_n = \max_{1 \leq i \leq n} (P_i + r_{n-i})$  که  $r_0 = 0$  را برای سود بهینه نوشت. الگوریتم زیر به صورت بازگشتی، سود بهینه را می‌یابد.

```

array of prices CUT_ROD(P, n) // P[1..n]
if (n==0) return 0
q = -∞
for (i=1 to n)
    q = max(q, p[i] + CUT_ROD(P, n-i))
return q

```



فرض کنید  $T(n)$  تعداد فراخوانیهای الگوریتم فوق باشد. به ازای  $n = 4$ ، درخت بازگشت زیر فراخوانیها را نشان می‌دهد که تعدادش ۱۶ تاست.

به طور کلی می‌توان نوشت  $T(n) = 1 + \sum_{j=0}^{n-1} T(j)$  و  $T(0) = 1$  که

حل این رابطه  $T(n) = 2^n$  است که نمایی است.

حال می‌خواهیم با شیوه پویا بنویسیم. معمولاً دو روش برای پیاده‌سازی برنامه‌نویسی پویا وجود دارد. یک روش بالا به پایین به صورت خاطر سپاری (memoization) است که الگوریتم به صورت بازگشتی معمولی نوشته می‌شود ولی نتیجه هر زیر برنامه در یک جدول یا آرایه ذخیره می‌شود. رویه هر بار بررسی می‌کند که آیا زیر برنامه حل شده یا خیر، اگر قبلاً حل شده که حل مجدد انجام نمی‌شود و همان نتیجه‌ای که ذخیره شده استفاده می‌شود. روش دوم همان روش معمول پایین به بالاست که تاکنون عمل می‌کردیم. برش میله به صورت پویا و با هر دو روش عبارت است از:

روش بالا به پایین memoized

MEMOIZED-CUT-ROD(P,n)

for ( $i = 0$  to  $n$ )  $r[i] = -\infty$

return MEMO-AUX(p,n,r)

MEMO-AUX(p,n,r)

// if ( $r[n] \geq 0$ ) return  $r[n]$  بررسی اینکه آیا قبلاً حل شده

if ( $n == 0$ )  $q = 0$

else {  $q = -\infty$

for ( $i = 1$  to  $n$ )

$q = \max(q, p[i] + \text{MEMO-AUX}(p, n-i, r))$  }

$r[n] = q$

Return  $q$

روش پایین به بالا

BOTTOM-UP-CUT-ROD(p,n)

$r[0] = 0$

for ( $j = 1$  to  $n$ )

{  $q = -\infty$

for ( $i = 1$  to  $j$ )

$q = \max(q, p[i] + r[j-i])$

$r[j] = q$  }

return  $r[n]$

هر دو الگوریتم فوق از مرتبه  $\theta(n^2)$  است. ولی روش پایین به بالا بهتر است و سر بار کمتری دارد. تاکنون ما فقط سود بهینه را محاسبه کردیم ولی مشخص نکردیم چگونه برش بزیم یعنی قطعات چه اندازه‌ای باشند. تابع قبل را تغییر می‌دهیم که اندازه‌ها را نیز در یک آرایه به اسم S ذخیره کند:

#### extend-bottom-UP-CUT-ROD(p,n)

```

r[۰] = ۰
for(j = ۱ to n)
    { q = -∞
      for(i = ۱ to j)
          if (q < p[i] + r[j-i])
              q = p[i] + r[j-i], S[j] = i }
    r[j] = q
return r and S

```

### ۱۱- مسابقات جهانی

تیم‌های A و B، حداکثر  $2n-1$  مسابقه می‌دهند و هر تیمی n بار برنده شد، برنده نهایی است. احتمال برد A در هر بازی P و احتمال برد B برابر  $q = 1-p$  است. فرض کنید  $P(i,j)$  احتمال برد نهایی A به شرطی که نیاز به i برد دیگر داشته باشد و در عین حال B نیاز به j برد دیگر داشته باشد. بنابراین قبل از شروع بازیها، احتمال برنده شده A در تورنمنت  $P(n,n)$  است. در ضمن می‌توان گفت  $P(0,i) = 1$  و  $1 \leq i \leq n$ ، یعنی A برنده نهایی شده است. همچنین  $P(i,0) = 0$  و  $1 \leq i \leq n$ . همچنین  $P(0,0)$  بی‌معنی است. می‌توان نوشت:

$$P(i,j) = p \cdot P(i-1,j) + q \cdot P(i,j-1), \quad i \geq 1, j \geq 1$$

این رابطه بازگشتی اگر به صورت تقسیم و غلبه حل شود از مرتبه نمایی است. ولی تابع زیر آن را به صورت پویا از پایین به بالا با مرتبه  $\theta(n^2)$  حل می‌کند.

#### function series (n,p)

```

array P[۰..n, ۰..n]
q = ۱-p
for(s = ۱ to n)
    { P[۰,s] = ۱; P[s,۰] = ۰
      for(k = ۱ to s-۱)
          p[k,s-k] = p.P[k-۱,s-k] + q.P[k,s-k-۱] }
for(s = ۱ to n)
    for(k = ۰ to n-s)
        p[s+k,n-k] = p.P[s+k-۱,n-k] + q.P[s+k,n-k-۱]
return P[n,n]

```

## خلاصه فصل

- ۱- برنامه‌نویسی پویا یک شیوه حل مسئله است که برای حل مسائل بهینه‌سازی کاربرد دارد. در این شیوه برای مسئله، یک رابطه بازگشتی یافت می‌شود و معمولاً از پایین به بالا این رابطه حل می‌شود. البته شیوه از بالا به پایین با تکنیک memo نیز در روش پویا قابل استفاده است.
- ۲- جمله  $n$ ام فیبوناچی با روش پویا با مرتبه  $\theta(n)$  قابل محاسبه است. البته روش‌هایی با مرتبه  $\theta(\ln g)$  نیز برای این منظور وجود دارد.
- ۳- یافتن ترتیب بهینه ضرب  $n$  ماتریس با روش پویا و با مرتبه  $\theta(n^3)$  انجام می‌شود.
- ۴- الگوریتم فلویید با روش پویا و با مرتبه  $\theta(n^3)$  کوتاهترین مسیرها را بین هر جفت رأس می‌یابد. در گراف جهت‌دار، این الگوریتم، سیکل منفی را تشخیص می‌دهد. در گراف غیرجهت‌دار اگر یال با وزن منفی وجود داشته، فلویید لزوماً درست کار نمی‌کند.
- ۵- مسائل کوله‌پشتی صفر و یک، فروشنده دوره‌گرد و خرد کردن سکه با روش پویا قابل حل هستند.
- ۶- یافتن درخت جستجوی دودویی بهینه با روش پویا و با مرتبه  $\theta(n^3)$  امکان‌پذیر است.
- ۷- یافتن بزرگترین زیردنباله مشترک دو دنباله به طول‌های  $m$  و  $n$ ، با روش پویا و با مرتبه  $\theta(mn)$  انجام می‌شود.
- ۸- مسائل زمان‌بندی خط تولید، برش میله و مسابقات جهانی با روش پویا قابل حل هستند.
- ۹- می‌توان  $\binom{n}{k}$  را با روش پویا و با مرتبه  $\theta(n.k)$  محاسبه کرد.

## سؤال‌های چهار گزینه‌ای فصل نهم

۱- فرض کنید A و B و C و D به ترتیب ماتریس‌های  $۱۳ \times ۵$  و  $۵ \times ۸۹$  و  $۸۹ \times ۳$  و  $۳ \times ۳۴$  باشند، حداقل تعداد ضرب مورد نیاز برای محاسبه  $M = ABCD$  چیست؟

(مبانی نرم افزار آزاد ۷۷ و ۷۹ و تخصصی نرم افزار آزاد ۸۱ و ۸۳ IT)

(۱) ۴۰۵۵ (۲) ۵۴۲۰۱ (۳) ۳۴۲۵ (۴) ۲۸۵۶

۲- کدامیک از عبارات زیر صحیح است؟

(مبانی نرم افزار آزاد ۷۹)

(۱) برای حل هر مساله می‌توان یک الگوریتم با استفاده از روش برنامه‌ریزی پویا (Dynamic programming) طراحی کرد.

(۲) روش حریصانه (Greedy) همیشه یک راه حل بهینه را بدست می‌دهد.

(۳) روش تقسیم و غلبه یک روش بالا به پایین می‌باشد در صورتیکه روش برنامه‌ریزی پویا یک روش پایین به بالا می‌باشد.

(۴) برای اینکه روش تقسیم و غلبه برای یک مساله مورد استفاده قرار گیرد اصل optimality بایستی برقرار باشد.

۳- می‌خواهیم برای ماتریس‌های  $M_1(۱ \times ۲)$  و  $M_2(۲ \times ۵)$  و  $M_3(۵ \times ۱)$  و  $M_4(۱ \times ۱)$  ترکیب بهینه پراتنزیندی پیدا نماییم تا تعداد ضرب‌های کل جهت محاسبه عبارت ذیل

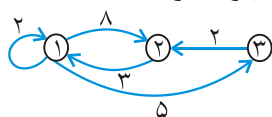
حداقل گردد این ترکیب بهینه عبارت است از: (آزاد ۸۴)

$$M = M_1 \times M_2 \times M_3 \times M_4$$

$$(1) (M_1 \times (M_2 \times M_3)) \times M_4 \quad (2) ((M_1 \times M_2) \times M_3) \times M_4$$

$$(3) M_1 \times ((M_2 \times M_3) \times M_4) \quad (4) \text{هیچکدام}$$

۴- با استفاده از الگوریتم فلوید کوتاهترین مسیر بین گره‌های ذیل عبارتند از: (آزاد ۸۴)



|   | ۱ | ۲ | ۳ | (۴) |
|---|---|---|---|-----|
| ۱ | ۰ | ۸ | ۵ |     |
| ۲ | ۳ | ۰ | ۸ |     |
| ۳ | ۵ | ۲ | ۰ |     |

|   | ۱ | ۲ | ۳ | (۳) |
|---|---|---|---|-----|
| ۱ | ۰ | ۷ | ۵ |     |
| ۲ | ۲ | ۰ | ۸ |     |
| ۳ | ۵ | ۲ | ۰ |     |

|   | ۱ | ۲ | ۳ | (۲) |
|---|---|---|---|-----|
| ۱ | ۰ | ۷ | ۵ |     |
| ۲ | ۳ | ۰ | ۸ |     |
| ۳ | ۵ | ۲ | ۰ |     |

|   | ۱ | ۲  | ۳ | (۱) |
|---|---|----|---|-----|
| ۱ | ۰ | ۵  | ۷ |     |
| ۲ | ۳ | ۰  | ۸ |     |
| ۳ | ۵ | ۱۰ | ۰ |     |

۵- اگر در مساله فروشنده دوره گرد ماتریس همجواری شهرها به صورت زیر باشد، حداقل مسیر کدام است؟ (با شروع از  $V_1$ ) (آزاد ۸۳)

|       | $V_1$ | $V_2$ | $V_3$ | $V_4$ | $V_5$ |        |
|-------|-------|-------|-------|-------|-------|--------|
| $V_1$ | ۰     | ۱۴    | ۴     | ۱۰    | ۲۰    | ۳۰ (۱) |
| $V_2$ | ۱۴    | ۰     | ۷     | ۸     | ۷     | ۳۱ (۲) |
| $V_3$ | ۴     | ۵     | ۰     | ۷     | ۱۶    | ۴۳ (۳) |
| $V_4$ | ۱۱    | ۷     | ۹     | ۰     | ۲     | ۳۴ (۴) |
| $V_5$ | ۱۸    | ۷     | ۱۷    | ۴     | ۰     |        |

۶- در الگوریتم فلوید برای پیدا کردن طول کوتاهترین مسیرها بین هر جفت گره از یک گراف جهت دار  $n$  گره‌ای از روش پویا استفاده می‌شود و در هر خانه  $[i, j]$  از ماتریسی به نام  $D$  طول کوتاهترین مسیرها بین گره‌های  $i$  تا  $j$  ثبت می‌شود. همچنین ماتریسی به نام  $p$  تشکیل می‌شود. در این ماتریس مقدار  $p[i, j]$  برابر گره‌ای است که در مسیر بهینه از  $i$  به  $j$  باید از آن عبور کرد. فرض کنیم که برای گرافی با ۴ گره ماتریس  $p$  به شکل زیر باشد. کدام یک از گزینه‌های زیر مسیر بهینه برای رفتن از گره ۱ به ۳ را مشخص می‌کند؟

(تخصصی هوش مصنوعی دولتی ۸۳)

|                                                                                                      |             |
|------------------------------------------------------------------------------------------------------|-------------|
| $p = \begin{bmatrix} 0 & 0 & 4 & 2 \\ 4 & 0 & 4 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$ | ۱ ۴ ۳ (۱)   |
|                                                                                                      | ۱ ۲ ۳ (۲)   |
|                                                                                                      | ۱ ۴ ۲ ۳ (۳) |
|                                                                                                      | ۱ ۲ ۴ ۳ (۴) |

۷- الگوریتم فلوید کوتاهترین مسیر بین همه زوج نقطه‌ها را در گراف جهت دار و وزن دار  $G$  محاسبه می‌کند. اگر یال‌های با وزن منفی هم داشته باشیم، کدام یک از گزینه‌های زیر بهترین توصیف این الگوریتم است؟ (تخصصی نرم افزار دولتی ۸۴)

- (۱) با این الگوریتم می‌توان وجود یا عدم وجود دور منفی را تشخیص داد.
- (۲) الگوریتم برای یال‌های منفی، ولی بدون دور منفی، ممکن است در دور بیافتد.
- (۳) الگوریتم برای یال‌های منفی، ولی بدون دور منفی، متوقف می‌شود، ولی درست کار نمی‌کند.
- (۴) الگوریتم با یال‌های منفی ولی بدون دور منفی، متوقف می‌شود. ولی جواب آن همیشه غلط است.



۸- الگوریتم زیر هزینه ضرب بهینه  $n$  تا ماتریس  $M_1 \times M_2 \times \dots \times M_n$  را که ابعاد  $M_i$  برابر  $d_{i-1} \times d_i$  است را مشخص می‌کند. مجموع تعداد خواندن درایه‌های  $C$  در این الگوریتم چند تاست؟

(تخصصی نرم افزار دولتی ۸۵)

```
for i:=1 to n do
  c[i,i] := 0
for L:=2 to n do
  for i:=1 to n-L+1 do
    {
      j:=i+L-1
      c[i,j] := min (c[i,k]+c[k+1,j]+d_{i-1}*d_k*d_j)
    }
    {
      sum_{L=2}^n (2L-1)(n-L) (۲)
      sum_{L=1}^n (2L-1)(n-L) (۱)
      sum_{L=2}^n 2(L-1)(n-L) (۴)
      sum_{L=2}^n 2(L-1)(n-L+1) (۳)
    }
```

۹- ۴ ماتریس زیر با ابعاد داده شده را در نظر بگیرید:  $D_{3 \times 4}$  و  $C_{25 \times 3}$  و  $B_{2 \times 25}$  و  $A_{10 \times 2}$  کدام یک از گزینه‌های زیر مربوط به پرانتزبندی بهینه برای محاسبه حاصلضرب آنها است؟

(IT ۸۴)

$$\begin{aligned} (1) & (A \times B) \times (C \times D) \\ (2) & (A \times (B \times C)) \times D \\ (3) & A \times ((B \times C) \times D) \\ (4) & ((A \times B) \times C) \times D \end{aligned}$$

۱۰- یک مسیر ساده در گراف مسیری است که از هیچ راسی بیش از یک بار عبور نکند. مسئله پیدا کردن طولانی‌ترین مسیر ساده برای گراف بدون وزن و جهت دار  $G$  را در نظر می‌گیریم. برای حل این مسئله:

(IT ۸۴)

- (۱) می‌توان از روش برنامه‌ریزی پویا استفاده کرد چون بهینگی برقرار هست.
- (۲) با وجودی که اصل بهینگی برقرار است نمی‌توان از روش برنامه‌ریزی پویا استفاده کرد.
- (۳) نمی‌توان از روش برنامه‌ریزی پویا استفاده کرد چون اصل بهینگی برقرار نیست.
- (۴) با وجودی که اصل بهینگی برقرار نیست می‌توان از روش برنامه‌ریزی پویا استفاده کرد.

۱۱- در مساله knapsack ۱-۵ با ۵ شی به شرح زیر  $(p_i)$  سود و  $w_i$  وزن شی  $(\lambda m)$  ظرفیت کیسه برابر  $13\text{kg}$  می‌باشد. مقدار سود ماکزیمم چیست؟

(IT ۸۵)

|                |          |      |          |      |          |
|----------------|----------|------|----------|------|----------|
| i              | ۱        | ۲    | ۳        | ۴    | ۵        |
| p <sub>i</sub> | \$۳۵     | \$۳۰ | \$۲۰     | \$۱۲ | \$۳      |
| w <sub>i</sub> | ۷        | ۵    | ۲        | ۳    | ۱        |
|                | \$۸۰ (۴) |      | \$۷۰ (۳) |      | \$۶۸ (۲) |
|                | \$۶۵ (۱) |      |          |      |          |

۱۲- فرض کنید که  $X = x_1x_2...x_m$  و  $Y = y_1y_2...y_n$  دو رشته با الفبای  $\{A, C, G, T\}$  باشد. طولانی‌ترین زیر دنباله مشترک (LCS) رشته  $X$  و  $Y$  با برنامه‌ریزی پویا بدست می‌آید. برای محاسبه  $c[i, j]$  که  $1 \leq i \leq m$  و  $1 \leq j \leq n$  است. در بدترین حالت چند خانه جدول بررسی می‌شود؟ (IT ۸۶)

(۱)  $n$  (۲)  $2$  (۳)  $3$  (۴)  $n+m$

۱۳- فرض کنید  $G$  یک گراف وزن‌دار همبند غیرجهت‌دار باشد که وزن یال‌های آن اعداد صحیح است. در مورد اعمال کردن الگوریتم Floyd-warshall روی گراف  $G$  چه می‌توان گفت؟ (IT ۸۶)

- (۱) چون گراف  $G$  جهت‌دار نیست نمی‌توان این الگوریتم را استفاده کرد.
- (۲) چون گراف  $G$  غیر جهت‌دار است این الگوریتم قادر نیست که حلقه‌های منفی را تشخیص دهد.
- (۳) چون گراف  $G$  غیر جهت‌دار است تابع زمان اجرای این الگوریتم روی  $G$  رشد کمتری نسبت به گراف‌های جهت‌دار دارد.
- (۴) هیچکدام

۱۴- الگوریتم زیر برای محاسبه سری فیبوناچی است:

```
function fibo (n)
  if  $n \leq 2$  return (n)
  else return (fibo(n-1) + fibo(n-2))
end;
```

(علوم کامپیوتر ۷۹)

گزینه صحیح را انتخاب کنید.

- (۱) این الگوریتم از رده تقسیم و غلبه است و مرتبه آن نمایی می‌باشد.
  - (۲) این الگوریتم از رده برنامه‌ریزی پویا است و مرتبه آن خطی می‌باشد.
  - (۳) این الگوریتم از رده برنامه‌ریزی پویا است و مرتبه آن نمایی می‌باشد.
  - (۴) این الگوریتم از رده تقسیم و غلبه است و مرتبه آن خطی می‌باشد.
- ۱۵- برنامه زیر برای محاسبه سری فیبوناچی است. گزینه صحیح را انتخاب کنید.

(علوم کامپیوتر ۸۰)

```
function fibo (n:integer): integer;
var
  f, f1, f2, i : integer;
begin
  f1 := 1;
  f2 := 1;
  for i := 1 to n do
    begin
      f := f1 + f2;
      f1 := f2;
      f2 := f;
    end;
  fibo:=f;
end;
```

(۱) الگوریتم این برنامه از رده برنامه‌ریزی پویا و مرتبه آن خطی است.

(۲) الگوریتم این برنامه از رده تقسیم و غلبه و مرتبه آن خطی است.

(۳) الگوریتم این برنامه از رده برنامه‌ریزی پویا است و مرتبه آن بیش از خطی است.

(۴) الگوریتم این برنامه از رده تقسیم و غلبه است و مرتبه آن بیش از خطی است.

۱۶- در ضرب سه ماتریس  $N_1$  و  $N_2$  و  $N_3$  به ابعاد  $w \times x$  و  $x \times y$  و  $y \times z$ ، تحت چه شرایطی ضرب  $(N_1 N_2) N_3$  سریع تر از ضرب  $N_1 (N_2 N_3)$  است؟ (علوم کامپیوتر ۸۱)

$$(1) \quad x > y \quad (2) \quad \frac{1}{x} + \frac{1}{z} < \frac{1}{w} + \frac{1}{y}$$

$$(3) \quad \frac{1}{w} + \frac{1}{x} < \frac{1}{y} + \frac{1}{z} \quad (4) \quad w + x > y + z$$

۱۷- یک درخت دودویی جستجوی  $T$  با ۵ گره با برچسب‌های  $a_1 < a_2 < a_3 < a_4 < a_5$  را در نظر بگیرید. به گره  $a_i$  عدد  $x_i$  را نسبت می‌دهیم. فرض کنید  $x_1 = 1$  و  $x_i = 2$  برای  $i = 2, \dots, 5$ . می‌خواهیم  $T$  را طوری بسازیم که عبارت  $C_T = \sum_{i=1}^5 x_i (\text{depth}(a_i) + 1)$  مینیمم شود.  $\text{depth}(a_i)$  عمق گره  $a_i$  در  $T$  است. کمترین مقدار  $C_T$  چند است؟ (دولتی ۷۹)

$$(1) \quad 13 \quad (2) \quad 14 \quad (3) \quad 15 \quad (4) \quad 16$$

۱۸- می‌خواهیم یک درخت دودویی جستجو با عنصر  $a_1 < a_2 < \dots < a_6$  بسازیم تا متوسط عمق عناصر در آن کمینه شود. اگر  $p_i$  احتمال  $a_i$  باشد، متوسط عمق برابر  $\sum_{i=1}^6 p_i \text{depth}(q_i)$  تعریف می‌شود. اگر  $p_1 = \frac{2}{7}$  و  $p_i = \frac{1}{7}$  برای  $2 \leq i \leq 6$  باشد، متوسط عمق درخت بهینه چند است؟ (عمق ریشه صفر فرض می‌شود) (علوم کامپیوتر ۸۲)

$$(1) \quad \frac{8}{7} \quad (2) \quad \frac{9}{7} \quad (3) \quad \frac{10}{7} \quad (4) \quad \frac{11}{7}$$

۱۹- کدام عبارت در مورد روش برنامه‌ریزی پویا نادرست است؟ (۸۷ IT)

(۱) این روش جزو روش‌های بالا به پایین است.

(۲) مسایل قابل حل با این روش باید تعریف بازگشتی داشته باشند.

(۳) در این روش از یک حافظه کمکی برای ثبت نتایج میانی استفاده می‌شود.

(۴) این روش برای حل مسایلی مناسب است که در آنها زیر مسایل روی هم افتادگی داشته باشند.

۲۰- یک تیر چوبی به طول  $L$  متر را می‌خواهیم از فاصله‌های  $I_1, I_2, \dots, I_n$  (نسبت به سر سمت چپ آن تیر) ببریم. یعنی  $0 < I_1 < I_2 < \dots < I_n < L$  می‌دانیم که هزینه‌ی برش یک تیر به طول  $r$  از هر نقطه‌ای بر روی آن تیر برابر  $r$  است. می‌خواهیم تیر را از این  $n$  نقطه به ترتیبی ببریم که کل هزینه‌ی صرف شده برای این کار کمینه شود. (تخصصی نرم‌افزار ۸۷)

(۱) این مسئله راه‌حل چند جمله‌ای ندارد.

(۲) این مسئله راه‌حل پویا از مرتبه‌ی چند جمله‌ای دارد.

(۳) این مسئله راه‌حل حریصانه از مرتبه‌ی چند جمله‌ای دارد.

(۴) این مسئله راه‌حل تقسیم و حل از مرتبه‌ی چند جمله‌ای دارد.

۲۱- محاسبه کدام تابع با روش برنامه‌نویسی پویا می‌تواند میزان محاسبات مورد نیاز را نسبت به روش بازگشتی بیش از بقیه کاهش دهد؟ (۸۸ IT)

$$F(n) = aF(n-1) + b \quad (۲) \quad F(n) = \sum_{i=1}^{n-1} F(i) \quad (۱)$$

$$F(n) = aF\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + bF\left(\left\lceil \frac{n}{2} \right\rceil\right) \quad (۴) \quad F(n) = aF(n-1) + bF(n-2) \quad (۳)$$

۲۲- جواب حاصل از رابطه بازگشتی زیر فضای حالات کدام یک از مسائل را می‌تواند بشمارد؟ (علوم کامپیوتر ۸۹)

$$\begin{cases} T(1) = 1 \\ T(n) = \sum_{i=1}^{n-1} T(i)T(n-i) \end{cases}$$

(۱) تعداد ضرب زنجیره‌ای  $n$  ماتریس (۲) تعداد پرانتزگذاری یک عبارت با  $n$  عملگر  
(۳) تعداد درخت‌های باینری با  $n-1$  گره (۴) تعداد درختان باینری با  $n$  گره

۲۳- یک دنباله‌ی  $X = (x_1, x_2, \dots, x_n)$  داده شده است.  $Y = (y_1, \dots, y_k)$  یک «زیر دنباله» از  $X$  است اگر برای هر  $y_i = x_{j_i}; 1 \leq i \leq k$  نیز  $j_1 < j_2 < \dots < j_k$  مثلاً  $(2, 3, 7, 9)$  و  $(2, 3, 9)$  زیر دنباله‌های  $(2, 3, 1, 7, 4, 9)$  هستند. با داشتن یک دنباله‌ی  $L$  با  $n$  عنصر، می‌خواهیم بزرگ‌ترین (از نظر تعداد عناصر) زیر دنباله‌ی افزایشی  $L$  را به دست آوریم. زیر دنباله‌ای افزایشی است که هر عنصر آن از عنصر بعدی‌اش اکیداً کوچک‌تر باشد این مسئله: (تخصصی هوش مصنوعی ۸۹)

(۱) با استفاده از الگوریتم یافتن بزرگ‌ترین زیر دنباله‌ی مشترک (Longest common substring) می‌توان این مسئله را هم حل کرد.

(۲) راه حل حریصانه و از مرتبه  $O(n^2)$  دارد.

(۳) راه حلی جز backtracking ندارد.

(۴) راه حل حریصانه و از مرتبه  $O(n \log n)$  دارد.

۲۴- یک گراف جهت‌دار  $G = (V, E)$  نشان دهنده‌ی یک شبکه‌ی کامپیوتری با  $n$  رأس است که وزن هر یال  $(u, v)$  برابر احتمال خرابی (قطع کامل) آن یال است که با  $p(u, v)$  نشان می‌دهیم و داریم  $0 \leq p(u, v) \leq 1$ . می‌خواهیم در این گراف احتمال خرابی قابل اعتمادترین مسیر از هر رأس  $i$  به یک رأس  $j$  را پیدا کنیم. این مسیری است که احتمال خرابی آن

کمینه است. فرض کنید که احتمال خرابی یال‌ها مستقل از هم هستند. می‌خواهیم از الگوریتم Floyd برای حل این مسئله به صورت زیر استفاده کنیم. (تخصصی نرم‌افزار ۸۹)

```
for k = ۱ to n do
  for i = ۱ to n do
    for j = ۱ to n do
      (a) .....
```

اگر  $P_{ij}$  احتمال خرابی یک مسیر بین دو رأس  $i$  و  $j$  با کم‌ترین احتمال خرابی باشد، چه عبارتی در سطر (a) قرار دهیم تا الگوریتم کار کند؟

$$\begin{aligned} P_{ij} &= \min \{P_{ij}, \max \{P_{ik}, P_{kj}\}\} & (۱) \\ P_{ij} &= \max \{P_{ij}, \min \{P_{ik}, P_{kj}\}\} & (۲) \\ P_{ij} &= \min \{P_{ij}, (P_{ik} + P_{kj})\} & (۳) \\ P_{ij} &= \min \{P_{ij}, P_{ik} * P_{kj}\} & (۴) \end{aligned}$$

۲۵- در یک راهرو  $n$  تابلو پشت سرهم برای نصب پوستر آماده شده است (تابلوهای  $b_1$  تا  $b_n$ ).

طبق مقررات، یک پوستر نباید در دو تابلوی پشت سرهم و در یک تابلو نباید بیش از یک عدد از یک پوستر نصب شود. برای هر تابلوی  $b_i$  یک «ضریب دید»  $W_i$  تعیین شده که نشان دهنده‌ی میزان دید آن تابلو است (هر چه این عدد بزرگ‌تر باشد به این معنی است که پوستر این تابلو بیش‌تر از بقیه دیده می‌شود) (بدیهی است که پوسترها نباید هم دیگر را بپوشانند. اما همیشه جا برای نصب پوستر در هر تابلو هست)

با داشتن  $W$ ها برای همه‌ی تابلوها، می‌خواهیم یک پوستر را در تعدادی از این تابلوها نصب کنیم که مجموع ضریب دید آن بیشینه شود. می‌توان نشان داد که مسئله راه‌حل پویا دارد. برای این کار فرض کنید  $f(i)$  مجموع ضریب دید (وزن) تابلوهای با شماره ۱ تا  $i$  است که بر روی آنها پوستر نصب می‌شود. در آن صورت کدام یک از رابطه‌های بازگشتی زیر درست است؟ (بدیهی است که  $f(0) = 0$  و  $f(1) = W_1$ ) (نرم‌افزار ۹۰)

$$\begin{aligned} f(i) &= \max \{f(i-1), w_1 + f(i-2)\} & (۱) \\ f(i) &= \max \{f(i-1), w_i + f(i-2)\} & (۲) \\ f(i) &= \max \{f(i-1) + w_1, f(i-2)\} & (۳) \\ f(i) &= \max \{f(i-2), w_1 + f(i-1)\} & (۴) \end{aligned}$$

۲۶- برنامه زیر برای یافتن عدد فیبوناچی  $n$ ام تدوین شده است. مرتبه زمانی آن چیست؟ فرض

کنید که  $ftab$  آرایه یک بعدی است که کلیه مقادیر آن در ابتدا ۱- است. (هوش مصنوعی ۹۰)

```
int fib(int n)
{
  if (ftab[n] != 0) return ftab[n];
  if (n == 0 || n == 1) {ftab[n] = n; return n;}
  ftab[n] = fib(n-1) + fib(n-2);
  return ftab[n];
}
```

$$(1)^n$$

$$n$$

$$(1/75)^n$$

$$\left(\frac{1+\sqrt{5}}{2}\right)^n$$

۲۷- کدام یک از الگوریتم‌های زیر حریصانه (greedy) نیست؟

- (۱) kruskal (۲) Floyd (۳) Huffman (۴) Dijkstra

۲۸- کدام الگوریتم از روش حریصانه (greedy) استفاده نمی‌کند؟

- (۱) پیدا کردن کوتاهترین مسیر در یک گراف با روش دایکسترا  
(۲) پیدا کردن مینیمم درخت فراگیر، kruskal  
(۳) پیدا کردن مینیمم درخت فراگیر، Prim  
(۴) پیدا کردن بزرگترین زیر رشته مشترک بین دو رشته

۲۹- بهترین الگوریتم برای یافتن  $n$  آمین عنصر دنباله فیبوناچی به کمک ماتریس تبدیل

(علوم کامپیوتر ۸۹)  $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$  دارای چه هزینه زمانی می‌باشد؟

- (۱)  $O(\varphi^n)$  و  $\varphi = \frac{1+\sqrt{5}}{2}$  (۲)  $O(\log_2^n)$   
(۳)  $O(n^2)$  (۴)  $O(n)$

۳۰- دانشگاهی  $n$  کلاس درس دارد که در یک ردیف کنار هم هستند، یعنی کلاس  $i$  (برای

$1 < i < n$ ) مجاور کلاس‌های  $i-1$  و  $i+1$  است. می‌دانیم که ظرفیت کلاس  $i$  برابر  $u_i$  نفر است. می‌خواهیم تعدادی کلاس با بیش‌ترین مجموع ظرفیت را تخصیص دهیم که هیچ دو کلاس تخصیص داده شده مجاور نباشند. اگر  $C(k)$  برای  $k = 1..n$  بیش‌ترین ظرفیت قابل تخصیص (به صورت گفته شده) از کلاس‌های  $1$  تا  $k$  باشد، کدام یک از رابطه‌های بازگشتی

زیر درست است؟ بدیهی است که  $C(1) = u_1$  (نرم‌افراز - ۹۳)

$$(1) C(k) = \max\{C(k-1), C(k-2) + u_k\}$$

$$(2) C(k) = C(k-1) + u_k$$

$$(3) C(k) = C(k-2) + u_k$$

$$(4) C(k) = \max\{C(k-1), C(k-2)\} + u_k$$

۳۱- قرار است درخت دودویی جست‌وجویی از عناصر  $a_1 < a_2 < \dots < a_n$  بسازیم. فرض کنید که

احتمال جست‌وجو برای  $a_i$  برابر  $p_i$  است و می‌دانیم که  $\sum_{i=1}^n p_i = 1$ . می‌خواهیم درختی

بسازیم که میانگین زمان جست‌وجو برای عناصر آن، یعنی  $\sum_{i=1}^n p_i \times [\text{depth}(a_i) + 1]$ ،

کمینه شود. اگر  $C_{ij}$  هزینه‌ی بهینه‌ی ساخت درخت دودویی جست‌وجو از عناصر

$a_i < a_{i+1} < \dots < a_j$  باشد، کدام یک از رابطه‌های زیر درست است؟ (هوش مصنوعی - ۹۳)

$$C_{ij} = \min_{i \leq k \leq j} \{C_{i,k-1}, C_{k+1,n}\} + \sum_{r=i}^j p_r \quad (۱)$$

$$C_{ij} = \min_{i \leq k \leq j} \{C_{i,k-1}, C_{k+1,n}\} \quad (۲)$$

$$C_{ij} = \min_{i < k < j} \{C_{i,k-1}, C_{k+1,n}\} + \sum_{r=i}^j p_r \quad (۳)$$

$$C_{ij} = \min_{i < k < j} \{C_{i,k-1}, C_{k+1,n}\} \quad (۴)$$

۳۲- فرض کنید  $A = \langle a_1, \dots, a_n \rangle$  دنباله‌ای از اعداد حقیقی مثبت باشد. در مسئله‌ی «پیدا

کردن زیردنباله‌ی متوالی با حاصل ضرب بیشینه» هدف پیدا کردن زیردنباله‌ی

$A_{ij} = \langle a_i, \dots, a_j \rangle$  است که حاصل ضرب اعضای آن در بین تمام زیردنباله‌های متوالی  $A$

بیشینه شود. چند تا از گزاره‌های زیر صحیح است؟ (۹۳-IT)

- بزرگ‌ترین عضو دنباله‌ی  $A$  حتماً عضوی از زیردنباله‌ی جواب  $A_{ij}$  است.
- با فرض آن که عملیات‌های جبری رایج در  $O(۱)$  قابل انجام است، این مسئله در زمان  $O(n)$  قابل تبدیل به مسئله‌ی «پیدا کردن زیردنباله‌ی متوالی با جمع بیشینه» است.
- اولین و آخرین عنصر زیردنباله‌ی جواب  $A_{ij}$  حتماً ناکوچک‌تر از یک هستند.
- اگر همه‌ی عناصر دنباله‌ی  $A$  در عدد مثبت  $c$  ضرب شوند، زیردنباله‌ی جواب این دنباله جدید همان زیردنباله‌ی جواب  $A$  خواهد بود که عناصر آن در  $c$  ضرب شده‌اند.

۱ (۲)      ۲ (۲)      ۳ (۳)      ۴ (۴)

۳۳- با داده‌های زیر می‌خواهیم یک درخت BST بهینه (درخت جستجوی دودویی با کمترین

زمان متوسط جستجو) بسازیم. ساختار درخت حاصل کدام است؟ (علوم کامپیوتر - ۹۳)

داده‌ها : A      B      C      D      E  
 احتمال جستجو : ۰/۲۵      ۰/۳۵      ۰/۲۵      ۰/۰۵      ۰/۱



۳۴- گراف جهت‌دار  $G = (V, E)$  با مجموعه‌ی رأس‌های  $V = \{1, 2, \dots, n\}$  داده شده است.

وزن هر یال  $(i, j)$  را با  $w(i, j)$  نشان می‌دهیم. اگر یال  $(i, j)$  وجود نداشته قرار می‌دهیم

$w(i, j) = +\infty$ . همچنین به ازای هر رأس  $i$  قرار می‌دهیم  $w(i, i) = 0$ ، می‌خواهیم با

استفاده از روش برنامه‌ریزی پویا کوتاه‌ترین مسیر بین هر زوج رأس این گراف را به دست

آوریم. به ازای کدام یک از رابطه‌های بازگشتی زیر مقدار  $d[i, j, n]$  برابر کوتاه‌ترین مسیر

بین رأس‌های  $i$  و  $j$  خواهد بود؟ (نرم‌افزار - ۹۴)

$$d[i, j, k] = \begin{cases} w(i, j) & k = 1 \\ \min_{1 \leq r \leq n} \{d[i, r, k-1] + w(r, j)\} & k > 1 \end{cases} \quad (۱)$$

$$d[i, j, k] = \begin{cases} w(i, j) & k = 0 \\ \min \{d[i, j, k-1], d[i, k, k-1] + d[k, j, k-1]\} & k > 0 \end{cases} \quad (۲)$$

$$d[i, j, k] = \begin{cases} w(i, j) & k = 1 \\ \min_{1 \leq r \leq n} \left\{ d\left[i, r, \left\lfloor \frac{k}{r} \right\rfloor\right] + d\left[r, j, \left\lceil \frac{k}{r} \right\rceil\right] \right\} & k > 1 \end{cases} \quad (۳)$$

(۴) هر سه مورد بالا

**۳۵-** می‌خواهیم یک چوب به طول  $r$  (عدد صحیح) را به اندازه‌های صحیح برش دهیم و بفروشیم تا بیش‌ترین سود را به دست آوریم. می‌دانیم که  $p_i$  قیمت فروش یک قطعه چوب به اندازه‌ی  $i \leq r$  است. کدام یک از رابطه‌های بازگشتی زیر برای  $C_r$  (سود بیشینه حاصل از تکه‌تکه کردن و فروش قطعات یک تکه چوب به اندازه‌ی  $r$ ) درست است؟ فرض کنید  $C_0 = 0$ .

(هوش مصنوعی - ۹۴)

$$\begin{aligned} C_r &= \max_{1 \leq i < r} \{C_i + C_{r-i}\} & (۲) & \quad C_r = \max_{1 \leq i \leq r} \{p_i + C_{r-i}\} & (۱) \\ C_r &= \max_{1 \leq i \leq r} \{C_i + C_{r-i}\} & (۴) & \quad C_r = \max_{1 \leq i < r} \{p_i + C_{r-i}\} & (۳) \end{aligned}$$

**۳۶-** طول بزرگ‌ترین زیر دنباله مشترک (LCS) دو دنباله به طول‌های  $m$  و  $n$  را با چه مرتبه حافظه‌ای می‌توان محاسبه کرد؟ بهترین گزینه را انتخاب کنید.

(دکتری کامپیوتر - ۹۵)

$$\begin{aligned} O(n+m) & \quad (۲) & O(nm) & \quad (۱) \\ O(\max\{n, m\}) & \quad (۴) & O(\min\{n, m\}) & \quad (۳) \end{aligned}$$

**۳۷-** فرض کنید  $f(n)$  برابر  $n$  امین عدد فیبوناچی باشد. بهترین الگوریتم برای محاسبه  $f(n)$

به پیمانه ۱۰۰۰ دارای چه مرتبه زمانی است؟ در گزینه‌های زیر  $\phi = \frac{1+\sqrt{5}}{2}$  است.

(دکتری کامپیوتر - ۹۵)

$$O(\phi^n) \quad (۴) \quad O(\log n) \quad (۳) \quad O(n) \quad (۲) \quad O(1) \quad (۱)$$

**۳۸-** مسئله حاصل ضرب زنجیره‌ای ماتریس‌ها را در نظر بگیرید:

$$M = M_1 \times M_2 \times \dots \times M_n$$

آیا الگوریتم حریصانه زیر منجر به یافتن ترتیب بهینه (از نظر تعداد ضرب‌ها) برای مسئله می‌شود؟

(دکتری علوم کامپیوتر - ۹۵)



الگوریتم - در هر مرحله ماتریس‌هایی که مجاورند و بُعد مشترک آنها از بُعد مشترک هر دو ماتریس مجاور دیگر بیشتر باشد، در هم ضرب می‌شوند و این کار تا رسیدن به ماتریس  $M$  ادامه می‌یابد.

(۱) بلی - همیشه منجر به جواب بهینه می‌شود.

(۲) خیر - گاهی اوقات منجر به جواب بهینه نمی‌شود.

(۳) خیر - الگوریتم داده شده هیچ‌وقت جواب بهینه را تولید نمی‌کند.

(۴) بلی - برای  $n$ های کوچک منجر به جواب بهینه می‌شود.

۳۹- فرض کنید  $D$  ماتریس فاصله‌ها در گراف وزن‌دار  $G$  با  $n$  رأس است. یعنی  $D[i, j]$  نشان‌دهنده‌ی اندازه‌ی کوتاه‌ترین مسیر بین رأس‌های  $i$  و  $j$  در گراف  $G$  است. ماتریس  $D$  و گراف  $G$  داده شده‌اند. فرض کنید وزن یک یال  $e$  از  $w_e$  به  $w'_e$  کاهش یافته است. در چه زمانی می‌توان ماتریس  $D$  را با توجه به کاهش وزن یال  $e$  به روز رسانی کرد؟ (نرم‌افزار - ۹۵)

(۱)  $O(n^2)$  (۲)  $O(n^2 \log n)$  (۳)  $O(n \log n)$  (۴)  $O(n)$

۴۰- دو دنباله‌ی  $A$  و  $B$  از اعداد طبیعی داده شده است. می‌خواهیم بزرگ‌ترین زیردنباله‌ی مشترک صعودی این دو دنباله را محاسبه کنیم. کدام یک از الگوریتم‌های زیر درست کار می‌کند؟ الف) بزرگ‌ترین زیردنباله‌ی مشترک  $A$  و  $B$  را محاسبه می‌کنیم و  $C$  می‌نامیم. سپس بزرگ‌ترین زیردنباله‌ی صعودی  $C$  را محاسبه و گزارش می‌کنیم.

ب) دنباله‌ی  $A$  را مرتب می‌کنیم و آن را  $A'$  می‌نامیم. بزرگ‌ترین زیردنباله‌ی مشترک  $A$ ،  $A'$  و  $B$  را محاسبه گزارش می‌کنیم. (نرم‌افزار - ۹۵)

(۱) الف: درست، ب: درست. (۲) الف: نادرست، ب: درست.

(۳) الف: درست، ب: نادرست. (۴) الف: نادرست، ب: نادرست.

۴۱- در مسئله‌ی یافتن همه‌ی کوتاه‌ترین مسیرها بین هر دو رأس در یک گراف جهت‌دار و وزن‌دار  $G$  که وزن یال‌ها می‌تواند منفی باشد و گراف دور منفی ندارد، فرض کنید ماتریس مجاورت گراف  $W = (w_{ij})$   $(n \times n)$  باشد که:

$$W[i, j] = \begin{cases} w_{ij} & \text{if } (i, j) \in E \\ \infty & \text{if } (i, j) \notin E \\ 0 & \text{if } i = j \end{cases}$$

اگر  $d_{ij}^{(m)}$  وزن کوتاه‌ترین مسیر  $i \rightarrow j$  که حداکثر  $m$  یال داشته باشد،

$$d_{ij}^{(0)} = \begin{cases} 0 & \text{if } i = j \\ \infty & \text{if } i \neq j \end{cases}$$

در آن صورت کدام یک از گزینه‌های زیر درست است؟ (هوش مصنوعی - ۹۵)

$$d_{ij}^{(m)} = \min_{1 \leq k \leq n} \{d_{ik}^{(m-1)} + w_{kj}\} \quad (۲) \quad d_{ij}^{(m)} = \min_{i \leq k < n} \{d_{ik}^{(m-1)} + w_{kj}\} \quad (۱)$$

$$d_{ij}^{(m)} = \min_{i \leq k < j} \{d_{ik}^{(m-1)} + w_{kj}\} \quad (۴) \quad d_{ij}^{(m)} = \min_{i \leq k \leq n} \{d_{ik}^{(m-1)} + w_{kj}\} \quad (۳)$$

۴۲- یک دنباله را «آینه‌ای» می‌گوییم اگر با معکوس خودش برابر باشد. یک دنباله‌ی  $A$  به طول

$n$  داده شده است. می‌خواهیم طول بزرگ‌ترین زیردنباله‌ی آینه‌ای (نه لزوماً پیوسته‌ی)  $A$

را پیدا کنیم. کدام یک از الگوریتم‌های زیر درست کار می‌کند؟ (هوش مصنوعی - ۹۵)

الف) معکوس  $A$  را محاسبه کرده و آن را  $A'$  می‌نامیم. طول بزرگ‌ترین زیردنباله‌ی مشترک  $A$  و  $A'$  را محاسبه و گزارش می‌کنیم.

ب) دو حالت را براساس آن که  $A[1]$  در جواب شاد یا خیر بررسی می‌کنیم و بین این دو، طول زیردنباله‌ی بزرگ‌تر را گزارش می‌کنیم. با فرض بودن  $A[1]$  بزرگ‌ترین  $i$  که  $A[i] = A[1]$  است را پیدا می‌کنیم. به صورت بازگشتی بزرگ‌ترین زیردنباله‌ی آینه‌ای  $A[1 \dots i-1]$  را محاسبه می‌کنیم. با فرض نبودن  $A[1]$  بزرگ‌ترین زیردنباله‌ی آینه‌ای  $A[2 \dots n]$  را محاسبه می‌کنیم.

(۱) الف: درست، ب: درست. (۲) الف: نادرست، ب: درست.

(۳) الف: درست، ب: نادرست. (۴) الف: نادرست، ب: نادرست.

۴۳- دنباله‌ی  $A[1 \dots n]$  از اعداد صحیح داده شده است. یک زیردنباله‌ی پیوسته‌ی  $A[i \dots j]$  از

$A$  را یک «بازه‌ی مثبت» می‌کنیم اگر جمع اعضای  $A[i]$  تا  $A[j]$  بزرگ‌تر از صفر شود. می‌خواهیم کم‌ترین تعداد بازه‌های مثبت که همه‌ی اعداد مثبت  $A$  را پوشش دهد پیدا کنیم. به ازای دنباله‌ی زیر این عدد چند است؟ (IT - ۹۵)

$$A = (۳, -۵, ۷, -۴, ۱, -۸, ۳, -۷, ۵, -۹, ۵, -۲, ۴)$$

$$(۱) ۵ \quad (۲) ۷ \quad (۳) ۲ \quad (۴) ۳$$

۴۴- فرض کنید  $A$  یک دنباله‌ی صعودی از اعداد طبیعی متمایز باشد. کدام یک از الگوریتم‌های

زیر طول بزرگ‌ترین زیردنباله‌ی صعودی (نه لزوماً پیوسته‌ی)  $A$  را به درستی محاسبه

می‌کند؟ (IT - ۹۵)

الف) مرتب‌شده‌ی  $A$  را در  $B$  نگه می‌داریم. سپس بزرگ‌ترین زیردنباله‌ی مشترک  $A$  و  $B$  را محاسبه و گزارش می‌کنیم.

ب) طول بزرگ‌ترین دنباله‌ی صعودی منتهی به  $A[i]$  را در  $B[i]$  نگه می‌داریم. برای محاسبه‌ی  $B[i]$  بین تمام  $j$ هایی که  $j < i$  و  $A[j] < A[i]$  هستند فرض کنید  $j'$  دارای

بیشترین  $B[j']$  باشد.  $B[i]$  را برابر  $B[j'] + 1$  قرار می‌دهیم. در انتها بین همه‌ی  $B[i]$  ها بیشترین را پیدا کرده و گزارش می‌کنیم.

(۱) الف: درست، ب: درست. (۲) الف: نادرست، ب: درست.

(۳) الف: درست، ب: نادرست. (۴) الف: نادرست، ب: نادرست.

۴۵- زمان و حافظه مصرفی الگوریتم فلوید - مارشال برای یافتن تمام کوتاه‌ترین مسیرها

(All shortest path) در یک گراف با  $n$  گره کدام است؟ (علوم کامپیوتر - ۹۵)

(۱) زمان  $O(n^3)$  و حافظه  $O(n^2)$  (۲) زمان  $O(n^2)$  و حافظه  $O(n^2)$

(۳) زمان  $O(n^3)$  و حافظه  $O(n^3)$  (۴) زمان  $O(n^2)$  و حافظه  $O(n^3)$

۴۶- عدد مثبت و صحیح  $n$  را در نظر بگیرید. می‌خواهیم بزرگترین عدد مثبت و صحیح  $x$  را پیدا

کنیم به طوری که  $x^2 \leq n$  باشد. بهترین الگوریتم برای یافتن  $x$  دارای چه هزینه‌ی زمانی

خواهد بود؟ (علوم کامپیوتر - ۹۵)

(۱)  $O(n)$  (۲)  $O(\sqrt{n})$  (۳)  $O(\log n)$  (۴)  $O(n \log n)$

۴۷- فرض کنید می‌خواهیم چهار ماتریس زیر را در یکدیگر ضرب کنیم، به طوری که تعداد ضرب‌ها حداقل شود.

$$A_1_{5 \times 2}, A_2_{2 \times 3}, A_3_{3 \times 4}, A_4_{4 \times 6}$$

برای انجام این کار از برنامه‌نویسی پویا (Dynamic Programmig) استفاده می‌کنیم.

ماتریس  $M$  در زیر نشان‌دهنده نتایج میانی ضرب این ماتریس‌ها می‌باشد. مقدار درایه

$M[1][4]$  چه خواهد بود؟ (علوم کامپیوتر - ۹۵)

|   | ۱ | ۲  | ۳  | ۴  |
|---|---|----|----|----|
| ۱ | ۰ | ۳۰ | ۶۴ | □  |
| ۲ |   | ۰  | ۲۴ | ۷۲ |
| ۳ |   |    | ۰  | ۷۲ |
| ۴ |   |    |    | ۰  |

$$M[1][4] = 72 \quad (1)$$

$$M[1][4] = 24 \quad (2)$$

$$M[1][4] = 132 \quad (3)$$

$$M[1][4] = 64 \quad (4)$$

## پاسخنامه سؤال‌های چهارگزینه‌ای فصل نهم

۱- گزینه (۴). بهتر است طوری ضرب کنیم که بزرگترین اعداد یعنی ۸۹ و ۳۴ فقط یک بار در عمل ضرب شرکت کنند: (البته این قانون قطعی نیست ولی اگر اعداد اختلاف زیادی داشته باشند قانون خوبی است!)

$$A_{13 \times 5} B_{5 \times 89} C_{89 \times 3} D_{3 \times 34} \Rightarrow (A(BC))D$$

تعداد عملیات ضرب برابر است با:

$$5 \times 89 \times 3 + 13 \times 5 \times 3 + 13 \times 3 \times 34 = 2856$$

۲- گزینه (۳). البته پویا، مدل بالا به پایین هم دارد.

۳- گزینه (۱). بزرگترین عدد ۱۰۰ و بعد از آن ۵۰ است. این اعداد باید یک بار در عمل ضرب شرکت کنند:

$$(M_1 (M_2 M_3)) M_4$$

۴- گزینه (۲).

$$\begin{matrix} & 1 & 2 & 3 \\ \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 7 & 5 \\ 3 & 0 & 8 \\ 5 & 2 & 0 \end{bmatrix} \end{matrix}$$

۵- گزینه (۲). تور بهینه عبارت است از:

$$V_1 \xrightarrow{4} V_3 \xrightarrow{5} V_2 \xrightarrow{7} V_5 \xrightarrow{4} V_4 \xrightarrow{11} V_1$$

$$\text{جمع} = 4 + 5 + 7 + 4 + 11 = 31$$

۶- گزینه (۴).

$$p[1,3] = 4 \Rightarrow 1 \dots 4 \dots 3$$

$$p[1,4] = 2 \Rightarrow 1 \dots 2 \dots 4$$

$$p[4,3] = 0 \Rightarrow 4 \rightarrow 3$$

$$p[1,2] = 0 \Rightarrow 1 \rightarrow 2$$

$$p[2,4] = 0 \Rightarrow 2 \rightarrow 4$$

$$1 \rightarrow 2 \rightarrow 4 \rightarrow 3$$

بنابراین مسیر بهینه از ۱ به ۳ عبارت است از:

۷- گزینه (۱). اگر پس از پایان اجرا، عدد منفی در قطر اصلی ماتریس ایجاد شود، گراف سیکل منفی دارد و برعکس.

۸- گزینه (۳). در جمله اصلی الگوریتم، ۲ بار  $C$  خوانده می‌شود یکی  $C[i, k]$  و یکی  $C[k+1, j]$ . پس تعداد کل خواندنهای  $C$  برابر است با:

$$\sum_{L=2}^n \sum_{i=1}^{n-L+1} \sum_{k=i+1}^j 2 = \sum_{L=2}^n \sum_{i=1}^{n-L+1} 2(j-i+1) = \sum_{L=2}^n \sum_{i=1}^{n-L+1} 2(j-i)$$

با توجه به اینکه  $j = i + L - 1$  داریم:

$$\begin{aligned} \sum_{L=2}^n \sum_{i=1}^{n-L+1} 2(i+L-1-i) &= \sum_{L=2}^n \sum_{i=1}^{n-L+1} 2(L-1) \\ &= \sum_{L=2}^n (2(L-1)(n-L+1-1+1)) = \sum_{L=2}^n 2(L-1)(n-L+1) \end{aligned}$$

۹- گزینه (۳).

۱۰- گزینه (۳). در مسئله طولانی‌ترین مسیر، اصل بهینگی برقرار نیست و نمی‌توان از روش پویا استفاده کرد.

۱۱- گزینه (۳). قطعات ۱ و ۳ و ۴ و ۵ را انتخاب می‌کنیم:

$$W = 7 + 2 + 3 + 1 = 13 \text{ Kg}$$

$$P_1 = 35 + 20 + 12 + 3 = 70 \$$$

۱۲- گزینه (۲). برای محاسبه  $C[i, j]$  به  $C[i-1, j-1]$  و  $C[i-1, j]$  و  $C[i, j-1]$  نیاز هستند ولی یا  $c[i-1, j-1]$  استفاده می‌شود و یا  $c[i-1, j]$  و  $c[i, j-1]$  بررسی می‌شوند یعنی حداکثر ۲ خانه جدول بررسی می‌شوند. البته کلید اولیه طراح گزینه ۳ است.

۱۳- گزینه (۲). اگر گراف غیرجهت‌دار یال منفی داشته باشد الگوریتم فلوید لزوماً جواب درست نمی‌دهد و اگر سیکل منفی داشته باشد تشخیص نمی‌دهد.

۱۴- گزینه (۱). بازگشتی (تقسیم و غلبه) است و مرتبه آن از  $2^{\frac{n}{2}}$  بیشتر است.

۱۵- گزینه (۱).

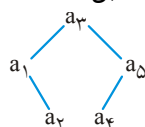
۱۶- گزینه (۲).

$$(N_1 N_2) N_3 \Rightarrow \text{تعداد ضرب} = wxy + wyz$$

$$N_1 (N_2 N_3) \Rightarrow \text{تعداد ضرب} = xyz + wxz$$

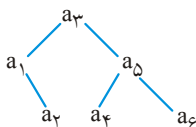
$$\Rightarrow wxy + wyz < xyz + wxz \Rightarrow \frac{1}{z} + \frac{1}{x} < \frac{1}{w} + \frac{1}{y}$$

۱۷- گزینه (۱). عمق ریشه صفر فرض می‌شود، یکی از جواب‌های این سؤال درخت مقابل است:



$$C_T = 1(0+1) + 2(1+1) + 1(1+1) + 1(2+1) + 1(2+1) = 13$$

۱۸- گزینه (۲).



$$\frac{1}{7} \times 0 + \frac{2}{7} \times 1 + \frac{1}{7} \times 1 + \frac{1}{7} \times 2 \times 3 = \frac{9}{7}$$

۱۹- گزینه (۱).

۲۰- گزینه (۲). فرض کنید تیر چوبی به دو قطعه تقسیم شده است. باید برای هر یک از دو قطعه برش بهینه را بیابیم یعنی اصل بهینگی برقرار است و راه حل پویا دارد.

۲۱- گزینه (۱). برنامه‌نویسی پویا برای روابط بازگشتی مناسب است که اشتراک وجود داشته باشد مثل محاسبه جمله  $n$ ام فیبوناچی که گزینه ۳ و ۱ چنین است ولی گزینه ۱ با روش پویا مناسب‌تر است. چون اشتراک بیشتری دارد.

۲۲- گزینه (۳). جواب این رابطه  $\frac{1}{n} \binom{2n-2}{n-1}$  است که جمله  $n-1$  کاتالان است. می‌توان آزمایش کرد. با توجه به رابطه داده شده  $T(2)=1$  و  $T(3)=2$  و  $T(4)=5$  می‌شود. البته گزینه ۱ نیز صحیح است!

۲۳- گزینه (۱). ابتدا عناصر دنباله  $L$  را مرتب صعودی کرده و  $L'$  می‌نامیم. حال LCS دو دنباله  $L$  و  $L'$  جواب سوال است: مثلاً:

$$L = (2, 3, 1, 7, 4, 9) \quad \text{یا} \quad \text{LCS}(L, L') = (2, 3, 7, 9) \quad \text{یا} \quad (2, 3, 4, 9) \\ L' = (1, 2, 3, 4, 7, 9)$$

۲۴- گزینه (۴). مسیر  $i$  به  $j$  یا از راس  $k$  می‌گذرد یا خیر. اگر از راس  $k$  نگذرد که همان  $P_{ij}$  (احتمال خرابی مسیر  $i$  به  $j$ ) که تاکنون محاسبه شده جواب است. ولی اگر از  $k$  بگذرد آنگاه احتمال خرابی  $i$  به  $k$  ( $P_{ik}$ ) را باید در احتمال خرابی  $k$  به  $j$  ( $P_{kj}$ ) ضرب کنیم. و سپس بین  $P_{ij}$  و  $P_{ik} * P_{kj}$  مینیمم بگیریم.

۲۵- گزینه (۱). دو حالت وجود دارد یا به تابلوی  $i$  پوستر چسبانده می‌شود یا خیر. در حالت اول، ضریب دید  $w_i$  را دریافت می‌کنیم ولی مجاز به چسباندن پوستر به تابلوی  $i-1$  نیستیم و باید به تابلوهای ۱ تا  $i-2$  فکر کنیم که ضریب دید آنها  $f(i-2)$  می‌شود. در حالت دوم باید به تابلوهای ۱ تا  $i-1$  فکر کنیم که ضریب دید آنها  $f(i-1)$  می‌شود. از این دو حالت آنکه ضریب دید بیشتری می‌دهد باید انتخاب شود.

۲۶- گزینه (۲). تابع به صورت پویا ولی با شیوه بالا به پایین و تکنیک memoized نوشته شده است که تعداد فراخوانی‌ها  $\theta(n)$  است.

۲۷- گزینه (۲).

۲۸- گزینه (۴).

۳۹- گزینه (۲). فرض کنید  $F_n$  جمله  $n$ ام فیبوناچی است، به روابط زیر توجه کنید:

$$F_1 = F_1, F_2 = F_0 + F_1 \Rightarrow \begin{bmatrix} F_1 \\ F_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} F_0 \\ F_1 \end{bmatrix} \text{ و } \begin{bmatrix} F_2 \\ F_3 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} F_1 \\ F_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}^2 \begin{bmatrix} F_0 \\ F_1 \end{bmatrix}$$

به همین ترتیب:

$$\begin{bmatrix} F_n \\ F_{n+1} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}^n \begin{bmatrix} F_0 \\ F_1 \end{bmatrix}$$

بنابراین برای محاسبه  $F_n$  کفایت، ماتریس  $2 \times 2$  را به توان  $n$  برسانیم و این عمل را می‌توان با مرتبه  $O(\lg n)$  انجام داد زیرا اگر  $x$  ماتریس باشد، ابتدا  $x^2 = xx$  سپس  $x^4 = x^2 \cdot x^2$  سپس  $x^8 = x^4 \cdot x^4$  و ... محاسبه می‌شوند که تعداد مراحل لگاریتمی است. (توان را با مرتبه لگاریتمی می‌توان محاسبه کرد).

۳۰- گزینه (۱). دو حالت وجود دارد یا کلاس  $k$ ام را تخصیص می‌دهیم یا خیر. اگر تخصیص دهیم آنگاه ظرفیت  $v_k$  را داریم و دیگر نمی‌توان کلاس  $k-1$ ام را تخصیص داد و باید کلاسهای ۱ تا  $k-2$  بازگشت کنیم که  $C(k-2)$  است. و اگر کلاس  $k$ ام را تخصیص ندهیم باید کلاسهای ۱ تا  $k-1$  بازگشت کنیم که  $C(k-1)$  است و جواب ماکزیمم این دو است:

$$C(k) = \max \{C(k-1), C(k-2) + v_k\}$$

۳۱- گزینه (۱). اگر ریشه  $a_k$  باشد، هزینه زیر درخت چپ  $(a_i \dots a_{k-1})$  برابر  $C_{i,k-1}$  و هزینه زیر درخت راست  $(a_{k+1} \dots a_j)$  برابر  $C_{k+1,j}$  می‌باشد و هزینه ریشه نیز  $p_k$  است. ولی تمام عناصر موجود در زیر درخت چپ و راست ابتدا با ریشه مقایسه می‌شوند پس به اندازه احتمالشان باید به هزینه اضافه شود یعنی کل هزینه برابر است با:

$$C_{i,k-1} + C_{k+1,j} + P_k + P_i + \dots + P_{k-1} + P_{k+1} + \dots + P_j$$

حال  $a_k$  می‌تواند هر یک از  $a_i$  تا  $a_j$  باشد که باید مینیمم را انتخاب کرد پس:

$$C_{ij} = \min(C_{i,k-1} + C_{k+1,j}) + P_i + \dots + P_j$$

$$i \leq k \leq j$$

در گزینه‌ها اشتباهاً بجای  $j$ ،  $n$  تایپ شده است !

۳۲- گزینه (۲). گزاره ۱ نادرست است. مثلاً دنباله  $\langle \frac{1}{4}, \frac{1}{8}, 2, 2, 2 \rangle$  جوابش  $\langle 2, 2, 2 \rangle$  است که شامل ۴ نیست گزاره ۲ درست است. اگر از همه اعداد دنباله لگاریتم گرفته شود آنگاه این مسئله تبدیل به مسئله‌ای می‌شود که باید زیردنباله متوالی با جمع بیشینه پیدا شود. گزاره ۳ درست است. زیرا در غیر این صورت می‌توان اولین یا آخرین عدد را حذف کرد و

دنباله بهتری یافت البته به شرطی که همه اعداد دنباله کوچکتر از ۱ نباشند. گزاره ۴ نادرست است. مثلاً جواب دنباله  $\langle \frac{1}{4}, 4, \frac{1}{8}, 2, 2, 2 \rangle$ ، دنباله  $\langle 2, 2, 2 \rangle$  است ولی اگر همه عناصر در  $\frac{1}{4}$  ضرب شوند دنباله  $\langle \frac{1}{4}, 2, \frac{1}{8}, 1, 1, 1 \rangle$  حاصل می‌شود که جوابش  $\langle 2 \rangle$  است. گزینه (۳).

$(A + b + C + D + E)$  هزینه درخت

$$1 \text{ هزینه درخت } = 0/25 \times 3 + 0/35 \times 2 + 0/25 \times 1 + 0/05 \times 3 + 0/1 \times 2 = 2/05$$

$$2 \text{ هزینه درخت } = 0/25 \times 3 + 0/35 \times 2 + 0/25 \times 1 + 0/05 \times 2 + 0/1 \times 3 = 2/1$$

$$3 \text{ هزینه درخت } = 0/25 \times 2 + 0/35 \times 1 + 0/25 \times 2 + 0/05 \times 4 + 0/1 \times 3 = 1/85$$

$$4 \text{ هزینه درخت } = 0/25 \times 2 + 0/35 \times 1 + 0/25 \times 3 + 0/05 \times 2 + 0/1 \times 3 = 2$$

گزینه (۴). هر سه رابطه بازگشتی داده شده، صحیح هستند. گزینه ۱ شبه ضرب ماتریسی، گزینه ۲ فلویید و گزینه ۳ تسریع شده شبه ضرب ماتریسی است.

گزینه (۱). حالات مختلف وجود دارد.

حالت ۱: از انتهای چوب یک قطعه به اندازه ۱ با قیمت  $P_1$  جدا کنیم و روی قطعه چوب به اندازه  $r-1$  بازگشت کنیم که  $C_{r-1}$  سود خواهد داشت.  
حالت ۲: از انتهای چوب یک قطعه به اندازه ۲ با قیمت  $P_2$  جدا کنیم و روی قطعه چوب به اندازه  $r-2$  بازگشت کنیم که  $C_{r-2}$  سود خواهد داشت.  
حالت  $r$ : چوب به اندازه  $r$  با قیمت  $P_r$  را بفروشیم و روی  $C_r$  بازگشت کنیم، مقدار  $C_r$ ، ماکزیمم این مقادیر است:

$$C_r = \max(P_1 + C_{r-1}, P_2 + C_{r-2}, \dots, P_r + C_0)$$

گزینه (۳). طبق تمرین «۴-۱۵.۴» کتاب CLRS، برای محاسبه طول LCS دو رشته به طول  $m$  و  $n$  می‌توان با  $\min(m, n)$  فضای حافظه، این عمل را انجام داد.

گزینه (۳). جمله  $n$ ام فیبوناچی را می‌توان با کمک ماتریس  $\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$  و با مرتبه  $O(\lg n)$

محاسبه کرد. اگر  $f_0$  و  $f_1$  دو جمله اول فیبوناچی باشند آنگاه  $\begin{bmatrix} f_n \\ f_{n+1} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}^n \begin{bmatrix} f_0 \\ f_1 \end{bmatrix}$  که

محاسبه  $\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}^n$  از مرتبه  $O(\lg n)$  است.

گزینه (۲). روش گفته شده گاهی جواب نادرست می‌دهد. مثلاً  $A_{10 \times 12} \times B_{12 \times 8} \times C_{8 \times 5}$  را با روش گفته شده بررسی کنید.



۳۹- گزینه (۱). ماتریس  $n \times n$  دارای  $n^2$  درایه است که این درایه‌ها باید بررسی و در صورت لزوم اصلاح شوند. هر درایه با زمان  $\theta(1)$  اصلاح می‌شود. فرض کنید  $e = (x, y)$  و وزن یال  $e$  از  $w(x, y)$  به  $w'(x, y)$  تغییر یافته آنگاه هر درایه با فرمول زیر اصلاح می‌شود:

$$D[i, j] = \min(D[i, j], D[i, x] + w'(x, y) + D[y, j])$$

۴۰- گزینه (۲). الف: نادرست و ب: درست است.

۴۱- گزینه (۲). برای یافتن کوتاهترین مسیر از رأس  $i$  به رأس  $j$  که حاوی حداکثر  $m$  یال است، باید کوتاهترین مسیر از رأس  $i$  به رأس  $k$  با حداکثر  $m-1$  یال پیدا شود و اندازه این مسیر با وزن یال از  $k$  به  $j$  جمع شود که  $k$  می‌تواند هر یک از رئوس گراف باشد یعنی  $1 \leq k \leq n$ .

۴۲- گزینه (۱). هر دو گزاره صحیح هستند. در مورد الف مثلاً  $A = xzywx$  و  $A' = xwyzx$  و  $LCS(A, A') = xyx$  که درست است. گزاره ب نیز با روش پویا، مسئله را حل می‌کند.

۴۳- گزینه (۴).

$$\{3, -5, 7, -4, 1\} \quad \{3, -7, 5\} \quad \{5, -2, 4\}$$

۴۴- گزینه (۱). هر دو گزاره صحیح هستند. الف تکراری است و ب نیز با روش پویا مسئله را حل می‌کند.

۴۵- گزینه (۱). در سؤال اشتباهاً به جای وارشان گفته شده مارشال ☹. در این الگوریتم یک ماتریس  $n \times n$  تعریف می‌شود پس مرتبه حافظه آن  $\theta(n^2)$  است و نمی‌توان کاهش داد. مرتبه زمانی فلویید نیز  $\theta(n^3)$  است.

۴۶- گزینه (۳). درواقع  $x \leq \sqrt{n}$ . ظاهراً از نظر طراح امکان گرفتن جذر وجود ندارد چون اگر این امکان باشد مرتبه این کار  $O(1)$  می‌باشد، در غیر این صورت مقادیر  $x$  را برابر ۱، ۲، ۴، ۸ و ... قرار می‌دهیم و هر بار نامساوی  $x^2 \leq n$  را بررسی می‌کنیم یعنی با مرتبه  $O(\lg n)$  می‌توان به جواب رسید.

۴۷- گزینه (۳).

$$M_{ij} = \min_{i \leq k < j} (M_{ik} + M_{k+1j} + r_{i-1} r_k r_j)$$

$$\begin{aligned} M_{14} &= \min \left( \underbrace{M_{11} + M_{24} + r_0 r_1 r_4}_{k=1}, \underbrace{M_{12} + M_{34} + r_0 r_2 r_4}_{k=2}, \underbrace{M_{13} + M_{44} + r_0 r_3 r_4}_{k=3} \right) \\ &= \min \left( \underbrace{0 + 72 + 5 \times 2 \times 6}_{132}, \underbrace{30 + 72 + 5 \times 3 \times 6}_{192}, \underbrace{64 + 0 + 5 \times 4 \times 6}_{184} \right) \\ &= 132 \end{aligned}$$

## بازگشت به عقب و انشعاب و تحدید

### فصل ۱۰

#### مقدمه

روش بازگشت به عقب (پسگرد، عقبگرد، backtrack) در مسائل تصمیم گیری استفاده می شود. این روش در کاهش مرتبه زمانی تاثیر مثبت نمی گذارد، اما با کاهش حالت های قابل بررسی سعی می کند زمان اجرای الگوریتم را برای  $n$  کوچک، قابل قبول کند. برخی از ویژگی های مسائلی که به روش پسگرد حل می شوند و ویژگی های خود روش عبارتند از:

(۱) مرتبه زمانی نامعقول. اکثر مسائلی که به روش بازگشت به عقب حل می شوند ذاتاً مسائل سختی هستند و مرتبه زمانی نامعقولی ( $2^n$ ،  $n!$  و  $3^n$  و ...) دارند. روش بازگشت به عقب نیز از کاهش مرتبه زمانی آنها و معقول (چند جمله ای) کردن آنها عاجز است، اما با کاهش حالت ها تنها زمان اجرا را برای  $n$  کوچک، کاهش می دهد.

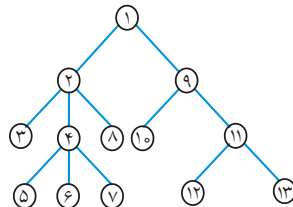
(۲) روش پسگرد از اصول و فنون درخت ها استفاده می کند. بخصوص از روش پیمایش preorder درخت.

(۳) مسائلی که به روش پسگرد حل می شوند، اغلب مسائل تصمیم گیری هستند. تصمیمات، مرحله به مرحله اتخاذ می گردد. مجموعه تصمیم ها به صورت یک درخت قابل نمایش است که به آن، درخت تصمیم (Decision tree) می گویند.

(۴) چنانچه در مرحله ای از الگوریتم، کلیه انتخاب های ممکن (تصمیم ها) بررسی گردد و هیچ کدام قابل قبول نباشند، باید تصمیم مرحله قبل را تغییر داد. این بدان معنی است که باید از یک گره درخت، به گره پدر آن، بازگشت کنیم.

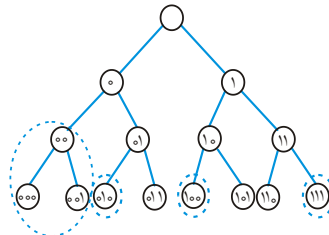
(۵) چنانچه مساله بیش از یک جواب داشته باشد، همه جواب ها را باید پیدا کنیم.

**یادآوری:** گره‌های درخت مقابل به ترتیب preorder شماره گذاری شده‌اند:



به عنوان مثال یک قفل رمزی دیجیتالی را در نظر بگیرید. فرض کنید رمز قفل یک عدد  $n$  بیتی است. اگر هیچ اطلاعاتی راجع به رمز نداشته باشیم باید  $2^n$  حالت را امتحان کنیم. در حالیکه قفل فقط با یک حالت باز می‌شود.

اگر  $n = 3$  باشد باید ۸ حالت را آزمایش کنیم ولی اگر بدانیم که رمز شامل دو رقم یک بوده، می‌توان سریع‌تر به جواب رسید. درخت زیر که به درخت فضای حالت (State spacetree) معروف است تمام حالت‌های مختلف را نشان می‌دهد:



در درخت فوق، قسمت‌هایی که نقطه چین مشخص شده را بررسی نمی‌کنیم. یعنی هرگاه به گره‌ای رسیدیم که مطمئن بودیم ادامه آن به جواب نمی‌رسد به گره پدر باز می‌گردیم مثلاً در درخت فوق وقتی به گره ۰۰ می‌رسیم، ادامه آن به جواب نخواهد رسید و به گره ۰ باز می‌گردیم و مسیری دیگر را ادامه می‌دهیم. یک گره را ناامید بخش گوئیم اگر ادامه آن منجر به جواب نشود. مثلاً گره ۰۰ در درخت فوق، ناامیدبخش است. در غیر این صورت گره را امیدبخش گویند.

بنابراین روش پسگرد، عبارت از جستجوی در عمق در درخت فضای حالت، بررسی امیدبخش بودن آن، و در صورت امیدبخش نبودن گره، بازگشت به گره پدر، است. این فرآیند را هرس کردن درخت فضای حالت گویند. الگوریتم عمومی روش پسگرد به شکل زیر است:

```
void bt (node v)
{
    if (promising (v))
        if (there is a solution at v)
            write the solution;
        else
            for (each child u of v)
                bt (u);
}
```

```
void bt'(node v)
{
    for (each child u of v)
        if (promising (u))
            if (there is a solution at u)
                write the solution
            else bt'(u) }
```

ریشه درخت فضای حالت به الگوریتم  $bt$  داده می‌شود. (البته  $bt'$  بهتر است ولی فهم  $bt$  راحت‌تر است) تابع  $promising$  بررسی می‌کند که آیا گره  $v$  امیدبخش است یا خیر. اگر امیدبخش بود، بررسی می‌شود که آیا در آن گره جواب هست یا خیر اگر بود جواب چاپ می‌شود و در غیر این صورت فرزندان آن گره بررسی می‌شوند. چون الگوریتم  $bt$  بازگشتی است، اگر یک گره امیدبخش نبود، خود به خود عقب گرد کرده و کار را با فرزند دیگر پدر آن گره ادامه می‌دهد.

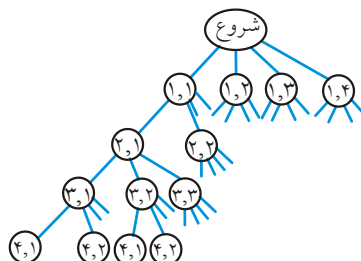
### مساله $n$ وزیر

می‌خواهیم  $n$  وزیر را در صفحه شطرنجی  $n \times n$  طوری قرار دهیم که هیچ دو وزیری یکدیگر را تهدید نکنند (سطر، ستون و قطر آنها یکسان نباشد).

به عنوان مثال به ازای  $n = 8$  یک جواب در شکل زیر آمده است:

|   | ۱ | ۲ | ۳ | ۴ | ۵ | ۶ | ۷ | ۸ |
|---|---|---|---|---|---|---|---|---|
| ۱ |   |   |   | * |   |   |   |   |
| ۲ |   |   |   |   |   | * |   |   |
| ۳ |   |   |   |   |   |   |   | * |
| ۴ |   | * |   |   |   |   |   |   |
| ۵ |   |   |   |   |   |   | * |   |
| ۶ | * |   |   |   |   |   |   |   |
| ۷ |   |   | * |   |   |   |   |   |
| ۸ |   |   |   |   | * |   |   |   |

فرض کنید به ازای  $n = 4$  می‌خواهیم مسئله را حل کنیم. با توجه به اینکه هیچ دو وزیری نمی‌توانند در یک سطر قرار بگیرند، برای هر وزیر ۴ حالت وجود دارد بنابراین تعداد کل حالات ممکن  $4 \times 4 \times 4 \times 4 = 256$  می‌باشد. این ۲۵۶ حالت را می‌توان با درخت فضای حالت نشان داد. در درخت، گره‌ها با زوج  $(i, j)$  نشان داده می‌شوند که  $i$  وزیر نام است که در سطر نام قرار دارد و  $j$  شماره ستونی است که وزیر در آن قرار دارد. مثلاً زوج  $(1, 2)$  یعنی وزیر دوم در ستون یک قرار دارد:



فرض کنید  $col[i]$  شماره ستونی است که وزیر سطر  $i$ ام در آن قرار دارد. مثلاً اگر  $col[2] = 3$  یعنی وزیر سطر دوم در ستون سوم است.

بنابراین شماره ستون وزیر  $i$ ام  $col[i]$  و شماره ستون وزیر  $k$ ام  $col[k]$  می‌باشد. شرط هم ستون بودن این دو وزیر  $col[i] = col[k]$  و شرط هم قطر بودن آنها  $|col[i] - col[k]| = |i - k|$  می‌باشد (بررسی کنید).

الگوریتم را می‌توان به صورت زیر نوشت:

```
void queens (i)
{
    if (promising (i))
        if (i == n)
            printf (col[1] through col[n])
        else
            // بررسی اینکه آیا وزیر سطر i + 1 را می‌توان در یکی از ستون‌ها قرار داد
            for (j = 1 to n)
            {
                col[i + 1] = j;
                queens (i + 1);
            }
}
```

تابع promising در الگوریتم queens بررسی می‌کند که آیا دو وزیر هم ستون یا هم قطر هستند یا خیر. اگر بودند یعنی امیدبخش نیست.

```

bool promising (i)
{
    k = ۱;
    switch = true;
    while (k < i) and switch do
    {
        if (col[i] == col[k]) or (abs (col[i] - col[k]) = abs(i - k))
            Switch = false;
        k = k + ۱;
    }
    return switch;
}

```

همانطور که قبلاً اشاره شد، الگوریتم پسگرد، تمام جواب‌ها را چاپ می‌کند البته می‌توان آن را طوری تغییر داد که فقط تعدادی جواب را چاپ کند. برای اجرا باید تابع queens را به صورت queens (0) صدا بزنیم. به ازای  $n = 4$  خروجی به صورت زیر است:

۳ و ۱ و ۴ و ۲  
۲ و ۴ و ۱ و ۳

یعنی اینکه برای  $n = 4$  دو جواب وجود دارد که شکل دو جواب به صورت زیر است:

|   |   |   |   |
|---|---|---|---|
|   | * |   |   |
|   |   |   | * |
| * |   |   |   |
|   |   | * |   |

جواب ۱

|   |   |   |   |
|---|---|---|---|
|   |   | * |   |
| * |   |   |   |
|   |   |   | * |
|   | * |   |   |

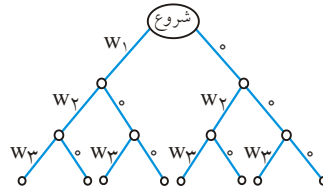
جواب ۲

برای  $n = 5$ ، ۱۰ جواب و برای  $n = 3$  هیچ جوابی وجود ندارد.

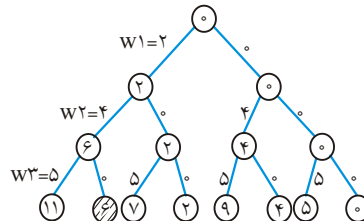
### مجموع زیر مجموعه‌ها

فرض کنید  $w = \{w_1, w_2, \dots, w_n\}$  یک مجموعه  $n$  عضوی از اعداد صحیح و مثبت است و می‌خواهیم همه زیر مجموعه‌های  $W$  را بیابیم که مجموع اعضای هر یک  $m$  باشد. مثلاً اگر  $w = \{w_1 = 5, w_2 = 13, w_3 = 7, w_4 = 8\}$  و  $m = 20$  آنگاه دو زیر مجموعه  $\{w_1 = 5, w_3 = 7, w_4 = 8\}$  و  $\{w_2 = 13, w_3 = 7\}$  جمع اعضایشان ۲۰ می‌باشد.

فرض کنید  $W = \{w_1, w_2, w_3\}$  باشد درخت فضای حالت را برای  $W$  می‌توان به شکل زیر کشید:



یال چپ یعنی انتخاب و یال راست یعنی عدم انتخاب. مثلاً اگر از ریشه به چپ برویم یعنی  $w_1$  را انتخاب کرده‌ایم و سطوح بعدی، انتخاب یا عدم انتخاب  $w_2$  و  $w_3$  را نشان می‌دهند اگر داخل گره‌ها جمع وزن‌های انتخاب شده تا آن گره را بنویسیم در این صورت با فرض  $m = 6$  و  $W = \{w_1 = 2, w_2 = 4, w_3 = 5\}$  درخت فضای حالت به صورت زیر است:

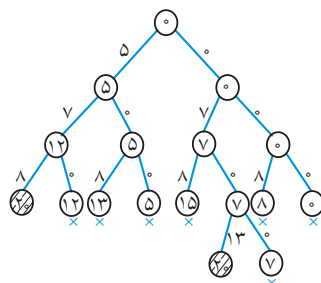


این مساله فقط یک جواب دارد  $\{w_1, w_2\}$ .

اگر فرض کنیم  $w_i$  ها صعودی هستند و اگر weight حاصل جمع  $w_i$  ها تا سطح  $i$ ام باشد در این صورت اگر انتخاب  $w_{i+1}$  باعث شود weight (یعنی جمع اعضای انتخاب شده) از  $m$  بیشتر شود، بدیهی است که آن گره امیدبخش نیست و باید پسگرد کنیم پس یکی از شرایط غیر امیدبخش بودن:  $\text{weight} + w_{i+1} > m$  می‌باشد.

علاوه بر این اگر فرض کنیم total برابر جمع همه اعداد باقی مانده است در این صورت اگر  $\text{weight} + \text{total} < m$  به این معنی است که آن گره نیز امیدبخش نیست زیرا اگر همه اعداد باقی مانده نیز انتخاب شوند، مجموع برابر  $m$  نمی‌شود.

با این دو شرط ناامیدبخش بودن می‌توان درخت فضای حالت را هرس کرد و همه حالت را بررسی نکرد.



**مثال ۱:** فرض کنید  $W = \{w_1 = 5, w_2 = 7, w_3 = 8, w_4 = 13\}$

و  $m = 20$ . درخت فضای حالت هرس شده به شکل مقابل است:

توجه کنید که برخی الگوریتم‌های عقبگرد، مثل  $n$  وزیر، در برگ‌ها به جواب می‌رسند. ولی برخی مثل جمع زیرمجموعه‌ها،

ممکن است قبل از رسیدن به برگ نیز به جواب برسند.

الگوریتم به صورت زیر است:

```
void sum-of-subset (index i, int weight, int total)
{
    if (promising (i))
        if (weight == m)
            printf (include [۱] through include [i]);
    else {
        //انتخاب [i+1] w
        include[i+۱]='yes';
        sum-of-subsets(i+۱, weight+w[i+۱], total-w[i+۱]);
        //عدم انتخاب [i+1] w
        include[i+۱]="no";
        sum-of-subsets(i+۱, weight, total-w[i+۱]);
    }
}
```

در تابع فوق  $n$  و  $w$  و  $m$  و  $include$  سراسری هستند. خروجی این الگوریتم تمامی زیر مجموعه‌هایی است که جمعشان  $m$  می‌شود. تابع  $promising$  به شکل زیر است:

```
bool promising (index i)
{
    return (weight + total >= m) and ((weight == m) or (weight + w[i+۱] <= m));
}
```

تابع اصلی باید به صورت  $sum-of-subsets(0,0,total)$  صدا زده شود که در ابتدا  $total = w[۱] + w[۲] + \dots + w[n]$ .

در بدترین حالت تعداد گره‌هایی که در درخت فضای حالت الگوریتم، بررسی می‌شوند برابر است با:

$$1 + 2 + 2^2 + \dots + 2^n = 2^{n+1} - 1$$

هر چند که در موارد زیادی تعداد واقعی گره‌های مورد بررسی از حداکثر فوق کمتر است ولی به هر حال الگوریتم فوق از مرتبه نمایی است و ممکن است در شرایطی لازم باشد همه گره‌ها بررسی شوند مثلاً در حالتی که داشته باشیم:

$$w_1 + w_2 + \dots + w_{n-1} < m, \quad w_n = m$$

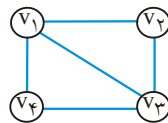
در این صورت تنها جواب مسئله  $\{w_n\}$  است که برای یافتن آن باید همه گره‌ها بررسی شوند.



## رنگ آمیزی گراف

می‌خواهیم تمام راه‌های ممکن برای رنگ آمیزی رئوس یک گراف با استفاده از  $m$  رنگ مختلف را بیابیم به طوری که هیچ دو راس مجاور هم‌رنگ نباشند.

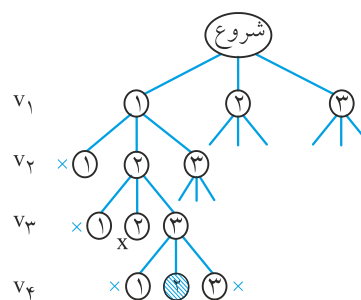
**مثال ۲:** در گراف شکل زیر:



فرض کنید با ۳ رنگ ۱ و ۲ و ۳ می‌خواهیم رئوس را رنگ کنیم برای این مساله ۶ راه وجود دارد که ۲ تای آن عبارتند از:

| $V_1$ | $V_2$ | $V_3$ | $V_4$ |
|-------|-------|-------|-------|
| ۱     | ۲     | ۳     | ۲     |
| ۲     | ۱     | ۳     | ۱     |

در درخت فضای حالت این مساله، رنگ‌های ممکن برای راس  $V_1$  در سطح ۱ امتحان می‌شوند، سپس رنگ‌های ممکن برای راس  $V_2$  در سطح ۲ آزمایش می‌گردد و به همین ترتیب تا اینکه رنگ‌های ممکن برای  $V_n$  در سطح  $n$  امتحان می‌شود. هر مسیر از ریشه تا برگ یک حل کاندید است. یک گره در صورتی ناامیدبخش است که گره مجاورش قبلاً با همان رنگ، رنگ آمیزی شده باشد. شکل زیر بخشی از درخت فضای حالت را برای گراف مثال قبل نشان می‌دهد.



اولین حل در گره سایه‌دار است. این حل نشان می‌دهد که راس  $V_1$  با رنگ ۱، راس  $V_2$  با رنگ ۲، راس  $V_3$  با رنگ ۳ و راس  $V_4$  با رنگ ۲، رنگ شده‌اند.

الگوریتم به شکل زیر است:

```
void m_coloring (index i)
{
    int color;
    if (promising (i))
        if (i == n)
            printf(vcolor[۱] through vcolor[n]);
    else
```

هر رنگ بعدی را برای گره بعدی آزمایش می‌کند//

```
    for (color = ۱ to m)
    {
        vcolor[i + ۱] = color ;
        m_coloring (i + ۱);
    }
}
```

در تابع فوق،  $n$  تعداد رئوس،  $m$  تعداد رنگ‌ها، و آرایه  $vcolor[1 \dots n]$  خروجی الگوریتم است که  $vcolor[i]$  رنگ نسبت داده شده به راس  $i$  را نشان می‌دهد. تابع  $promising$  به شکل زیر است:

```
bool promising (index i)
{
    index j = ۱;
    bool switch = true ;
    while ((j < i) and switch) // check if an adjacent
    {
        // vertex is already this color
        if (w[i][j] and (vcolor[i] == vcolor[j]))
            switch = false ;
        j = j + ۱;
    }
    return switch
}
```

در تابع فوق،  $w$  ماتریس همجواری گراف است. اگر راس  $i$  و  $j$  مجاور باشند آنگاه  $w[i][j]=true$  در غیر این صورت  $false$  خواهد بود.

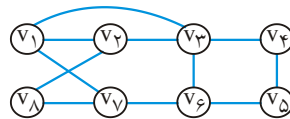
تابع را به صورت  $m\_coloring(0)$  فراخوانی می‌کنیم. تعداد گره‌های درخت فضای حالت برابر است با:

$$1 + m + m^2 + \dots + m^n = \frac{m^{n+1} - 1}{m - 1}$$

می‌توان برای حالت خاص  $m = 2$  الگوریتمی نوشت که پیچیدگی آن در بدترین حالت نمایی نباشد. ولی برای  $m \geq 3$  تاکنون کسی الگوریتمی که نمایی نباشد، ابداع نکرده است.

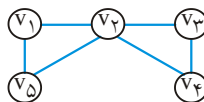
### سیکل همیلتونی

سیکل همیلتونی، دوری است که از همه رئوس فقط و فقط یکبار عبور کند. مثلاً در گراف زیر



دنباله  $[V_1 V_2 V_8 V_7 V_6 V_5 V_4 V_3 V_1]$  یک سیکل همیلتونی است.

ولی در گراف شکل زیر سیکل همیلتونی وجود ندارد.



می‌خواهیم تمام سیکل‌های همیلتونی یک گراف غیر جهت دار را بیابیم.

فرض کنید مجموعه رئوس  $V = \{0, 1, 2, \dots, n-1\}$  باشد.

درخت فضای حالت برای این مساله به این شرح است: راس شروع را در سطح ۰ درخت قرار

می‌دهیم.

در سطح ۱ به بعد همه رئوس بجز راس شروع می‌توانند باشند.

با در نظر گرفتن ملاحظات زیر می‌توان در این درخت، عقبگرد کرد:

(۱) راس  $i$ ام از مسیر باید مجاور با راس  $(i-1)$ ام از آن باشد.

(۲) راس  $(n-1)$ ام باید مجاور راس صفر (راس شروع) باشد.

(۳) راس  $i$ ام نمی‌تواند هیچکدام از  $i-1$  راس قبلی باشد.

الگوریتم به صورت زیر است. راس  $V_1$  به عنوان راس شروع در نظر گرفته شده است. گراف با

ماتریس همجواری  $w[1..n, 1..n]$  نمایش داده شده است.

خروجی الگوریتم  $vindex[0..n-1]$  است که در آن  $vindex[i]$ ، راس  $i$ ام مسیر است. راس

شروع،  $vindex[0]$  است:

```

void Hamilton (i)
{
    if promising (i) then
        if (i = n-1)
            printf (vindex [0] through vindex [n-1])
        else
            for (j=2 to n-1) // try all vertices as next one
            {
                Vindex [i+1]=j;
                Hamilton (i+1);
            }
}

```

که در آن تابع promising به صورت زیر پیاده سازی می‌شود:

```

bool promising (i)
{
    if ((i = n-1) and (w [vindex [n-1], vindex [0]] = 0))
        switch = false; // اولین راس باید با آخری مجاور باشد
    else
        if (i > 0 and (w[vindex [i-1], vindex [i]] = 0))
            switch = false; // راس iام باید با (i-۱)ام مجاور باشد
        else
        {
            Switch=true;
            j=1;
            While ((j<i) and switch) // بررسی اینکه راس قبلاً انتخاب شده
            {
                if (vindex [i] = vindex [j])
                    switch = false;
                j=j+1;
            }
        }
    return switch;
}

```

فراخوانی تابع Hamilton باید به صورت زیر باشد:

```

Vindex [0]=1;
Hamilton (0);

```

تعداد گره‌های درخت فضای حالت برابر است با:

$$1 + (n-1) + (n-1)^2 + \dots + (n-1)^{n-1} = \frac{(n-1)^n - 1}{n-2}$$

که بسیار بدتر از نمایی است. اگرچه تمام گره‌های فضای حالت بررسی نمی‌شوند، با این وجود، تعداد گره‌های بررسی شده، بدتر از نمایی است. مثلاً فرض کنید راس  $V_1$  تنها به  $V_2$  وصل است ولی بقیه رئوس غیر از  $V_1$  همگی به هم وصل هستند، این گراف سیکل همیلتونی ندارد ولی الگوریتم، زمانی متوجه این مطلب می‌شود که تقریباً همه گره‌ها را بررسی کرده باشد.

### حل مسأله کوله‌پشتی صفر و یک با تکنیک عقبگرد

در کتاب مرجع مسأله کوله‌پشتی صفر و یک به شیوه عقبگرد نیز حل شده است که مطالعه آن را بر عهده دانشجویان قرار می‌دهیم. در اینجا فقط این نکته را یادآوری می‌کنیم که حل این مسأله با روش برنامه‌نویسی پویا از مرتبه  $O(\text{Min}(2^n, nW))$  بود و در روش عقبگرد در بدترین حالت  $\theta(2^n)$  گره را چک می‌کند. ممکن است به خاطر  $nW$  تصور شود که تکنیک پویا به تکنیک عقبگرد در این مسأله ارجحیت دارد ولی با اجرای واقعی روی کامپیوتر هورویتز و شانی در سال ۱۹۷۸ نشان دادند که در مسأله کوله‌پشتی صفر و یک روش عقبگرد معمولاً بازدهی بیشتری نسبت به روش پویا دارد.

همچنین هورویتز و شانی در سال ۱۹۷۴ با ترکیب روش تقسیم و حل و روش برنامه‌نویسی پویا الگوریتمی را طراحی کردند که در بدترین حالت به  $O(2^{n/2})$  تعلق داشت و نشان دادند این الگوریتم اغلب بازدهی بیشتری نسبت به الگوریتم عقبگرد دارد.

**توجه:** تکنیک پیاده‌سازی دیگری برای الگوریتم‌ها شبیه تکنیک عقبگرد موسوم به Branch & Bound وجود دارد. در این تکنیک معمولاً از پیمایش سطحی BFS استفاده می‌شود.

## [سؤال‌های چهارگزینه‌ای فصل دهم]

- ۱- فرض کنید  $(i, j)$  و  $(k, L)$  دو عنصر در یک صفحه  $8 \times 8$  باشند (مانند صفحه شطرنج) کدامیک از گزینه‌های زیر هم قطر بودن آنها را تعیین می‌کند؟ (تهدید دو مهره فیل)  
(علوم کامپیوتر ۷۹ و ۸۴)

(۱) if  $(i = k)$  or  $(j = L)$  then

(۲) if  $(i - k = j - L)$  or  $(i - k = L - j)$  then

(۳) if  $(i - k = j - L)$  then

(۴) if  $(i - k = j - L)$  or  $(i - k = L + 8 - j)$  then

- ۲- قرار است الگوریتم مقابل کلیه زیرمجموعه‌هایی از اعداد صحیح که در آرایه  $W[1..n]$  قرار دارند و مجموع مؤلفه‌های آنها برابر  $m$  است را پیدا و چاپ کند. آرایه  $x$  سراسری است و  $x[i] = 1$  یعنی  $W[i]$  انتخاب شده و  $x[i] = 0$  یعنی  $W[i]$  انتخاب نشده. متغیر  $m$

نیز سراسری است. الگوریتم به صورت  $\text{sumofsub}(\sum_{i=1}^m W[i], 0, 1)$  از بیرون فراخوانی خواهد شد؟  
(ساختمان داده‌ها مهندسی کامپیوتر دولتی ۸۵)

```
void sumofsub (s , r , k)
{
    x[k]=1 ;
    if (s+w[k]== m) print (x[1] , x[2] , ... , x[k]) ;
    else if (s+w[k]+w[k+1]<= m)
        sumofsub (s+w[k] , r-w[k] , k+1);
    if ((s+r-w[k]>=m) && (s+w[k+1]<=m))
    {
        x[k]=0 ;
        sumofsub (s , r-w[k] , k+1);
    }
}
```

(۱) الگوریتم فقط یک جواب را پیدا می‌کند.

(۲) الگوریتم همه جواب‌ها را به درستی پیدا کرده چاپ می‌کند.

(۳) الگوریتم شرط توقف ندارد و در حلقه نامتناهی فراخوانی بازگشتی خواهد افتاد.

(۴) الگوریتم یک جواب درست و چندین جواب نادرست پیدا خواهد کرد چرا که آرایه  $x$  برای جواب‌های بعدی مقداردهی اولیه نمی‌شود.

۳- در مسأله کوله پشتی صفر و یک با داشتن  $n$  شی و ظرفیت  $w$  برای کوله پشتی، کدام گزینه نادرست است؟

(۸۴ IT)

- ۱) کاراترین الگوریتم برای این مسأله الگوریتم حریصانه با  $\theta(n)$  زمان مورد نیاز است.
- ۲) الگوریتم برنامه‌ریزی پویا در بدترین حالت برای این مسأله  $\theta(nw)$  زمان نیاز دارد.
- ۳) الگوریتم شاخه و قید (branch & bound) در بدترین حالت برای این مسئله  $\theta(2^n)$  زمان نیاز دارد.
- ۴) الگوریتم برگشت به عقب (back tracking) در بدترین حالت برای این مسئله  $\theta(2^n)$  زمان نیاز دارد.

۴- الگوریتم زیر مسئله‌ی  $n$  وزیر را حل می‌کند که در آن همه‌ی جواب‌ها برای نحوه‌ی قرار گرفتن  $n$  وزیر در صفحه‌ی شطرنج  $n \times n$  که هیچ‌کدام دیگری را تهدید نکنند را می‌نویسد. برای حل این مسئله  $N$ -Queens ( $A, 1, n$ ) با مقدار اولیه  $A[i] = [1, 2, 3, \dots, n]$  فراخوانده می‌شود. خروجی هر حالت آرایه‌ی  $A$  است که  $A[i]$  نشان می‌دهد که وزیر  $i$  (که در ستون  $i$  ام قرار دارد) در چه سطر از آن ستون قرار می‌گیرد. سطر ۴ بررسی می‌کند که آیا وزیر  $i$  وزیر  $j$  را تهدید می‌کند یا خیر.

(تخصصی هوش مصنوعی ۸۷)

```

N- Queens (A, i, n)
1  if (i > n)
2    for i=1 to n do
3      for j = i+1 to n do
4        if (|A[i]-A[j]| = |i-j|)
5          then exit
6        endif
7      endfor
8    endfor
9    Print A[1 .. n]
10  else
11    for j=i to n
12      Swap (A[i], A[j])
13      N- Queen (A, i+1, n)
14      ?????
15    endfor
16  endif

```

دستورالعملی که باید در سطر ۱۴ قرار بگیرد تا الگوریتم کار کند کدام است؟

- ۱) Swap (A[i], A[j])
- ۲) Swap (A[j], A[j+1]) if  $j < n$
- ۳) Swap (A[i], A[i+1]) if  $i < n$
- ۴) هیچ دستورالعملی لازم نیست.

## پاسخنامه سؤال‌های چهارگزینه‌ای فصل دهم

- ۱- گزینه (۲). شرط را به صورت  $(|i - k| = |j - L|)$  if نیز می‌توان نوشت.
- ۲- گزینه (۲). البته در صورت تست اشکال وجود دارد و فراخوانی باید به صورت  $\text{sumofsub}(0, \sum_{i=1}^m W[i], 1)$  انجام شود. در این صورت طبق متن کتاب تمام جواب‌ها را چاپ می‌کند.
- ۳- گزینه (۱).
- ۴- گزینه (۱). ابتدا وزیر ستون ۱ را در اولین مکان سطر ۱ قرار داده سپس تمام حالت‌های مجاز برای سایر وزیرها را در ستون‌های ۲ تا  $n$  یافته و اگر وزیر ۱ با آن مشکلی نداشت، یک جواب به دست آمده است. سپس جای وزیر ۲ را با ۱ عوض می‌کنیم یعنی فرض می‌کنیم وزیر ستون ۲ در سطر ۱ قرار گرفته و سایر وزیرها را به صورت بازگشتی قرار می‌دهیم و در صورتی که با وزیر ستون ۲ مشکل نداشته باشد، جواب چاپ می‌شود. سپس وزیر ۱ با ۳ عوض می‌شود، اما قبل از آن باید در هر مرحله (خط ۱۲)، وقتی عنصر  $i$  با  $j$  عوض شد، پس از آنکه سایر جواب‌ها بدست آمد (خط ۱۳)، دوباره جای  $i$  و  $j$  به شکل اول برمی‌گردد.





## آشنایی با نظریه NP فصل ۱۱

**توجه:** برای این فصل حتماً فیلم آموزشی که در آپارات قرار داده‌ام را ببینید!

### مسائل بغرنج (intractable)

**تعریف:** الگوریتم زمانی چندجمله‌ای، الگوریتمی است که در بدترین حالت پیچیدگی زمانی آن تابع چندجمله‌ای از اندازه‌ی ورودی باشد. مثلاً الگوریتم‌های با پیچیدگی زمانی زیر (در بدترین حالت) از نوع چندجمله‌ای هستند:

$$23n \quad 15n^4 + 14n^2 \quad 28n^2 + n^9 \quad n^2 \lg n$$

توجه کنید از آنجا که  $n^2 \lg n < n^3$  است پس  $n^2 \lg n$  محدود به چند جمله‌ای است.

ولی الگوریتم‌هایی با پیچیدگی‌های زمانی بدترین حالت زیر از نوع چندجمله‌ای نیستند:

$$3^n \quad 3^{0.1n} \quad 3^{\sqrt{n}} \quad n!$$

مسئله‌ای بغرنج است که حل آن توسط الگوریتمی با زمان چندجمله‌ای غیرممکن باشد.

توجه کنید بحث ما بر سر خود مسئله است و نه الگوریتم حل مسئله. ممکن است برای حل مسئله الگوریتم‌های متعددی با زمان‌های مختلف وجود داشته باشد. مثلاً حل مسئله‌ی سری فیبوناچی با روش تقسیم و حل از مرتبه‌ی نمایی و با روش پویا از مرتبه خطی است، پس مسئله‌ی فیبوناچی از نوع بغرنج نیست.

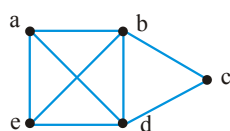
برای آنکه مسئله‌ای بغرنج باشد، باید اثبات کرد که هیچ الگوریتمی با مرتبه‌ی چندجمله‌ای برای آن وجود ندارد.

از نظر بغرنج بودن یا نبودن، مسائل علوم کامپیوتر سه دسته هستند:

- ۱- مسائلی که الگوریتم‌هایی با پیچیدگی زمانی چندجمله‌ای برای آنها پیدا شده است.
  - ۲- مسائلی که اثبات شده است که بغرنج هستند، یعنی الگوریتم چند جمله‌ای برای آن یافت نمی‌شود.
  - ۳- مسائلی که بغرنج بودن آنها ثابت نشده است ولی از طرف دیگر هیچ الگوریتم چندجمله‌ای هم برای آنها پیدا نشده است.
- مثلاً جستجوی دودویی در آرایه مرتب  $\theta(\lg n)$ ، مسأله مرتب‌سازی آرایه‌ها  $\theta(n \lg n)$ ، ضرب زنجیره‌ای ماتریس‌ها  $\theta(n^3)$ ، از دسته اول هستند. همچنین مسائل کوتاه‌ترین مسیر فلوید  $\theta(n^3)$ ، درخت جستجوی دودویی بهینه  $\theta(n^3)$  و مسأله درخت پوشای کمینه پریم  $\theta(n^2)$ ، با اینکه الگوریتم‌های نمایی نیز دارند ولی چون برای آنها الگوریتم‌هایی از نوع چندجمله‌ای یافته شده است از این دسته‌اند.
- تعداد مسائلی که جزو دسته دوم بوده و بغرنج بودن آنها اثبات شده است، نسبتاً کم است. نمونه‌ای از این مسائل دسته دوم، حساب پرزبرگر است.
- مسأله کوله‌پشتی صفر و یک و مسأله فروشنده دوره‌گرد از دسته سوم هستند.
- یک رابطه مشخص و جالبی بین بسیاری از مسائل دسته سوم وجود دارد که در ادامه فصل به دنبال آن هستیم.

### رابطه مسائل بهینه‌سازی با مسائل تصمیم‌گیری

خروجی مسائل بهینه‌سازی یک حل بهینه است و خروجی مسائل تصمیم‌گیری، جواب بله یا خیر است. نکته مهم آن است که هر مسأله بهینه‌سازی را می‌توان متناظر با یک مسأله تصمیم‌گیری در نظر گرفت. مثال‌های زیر این موضوع را نشان می‌دهند.



**مثال ۱: مسأله کلیک (clique):** کلیک در یک گراف بدون جهت

$G = (V, E)$  عبارت از زیرمجموعه  $W \subseteq V$  است به قسمی که هر رأس در  $W$  مجاور همه رئوس دیگر  $W$  باشد. مثلاً در گراف زیر،  $W = \{a, b, d\}$  یک کلیک است ولی  $\{a, b, c\}$  کلیک نیست چون  $a$  و  $c$  مجاور نیستند. کلیک بیشینه یک کلیک با بیشترین تعداد رأس است. درگراف فوق  $\{a, b, d, e\}$  تنها کلیک بیشینه است. پس **مسئله بهینه‌سازی کلیک**، تعیین اندازه یک کلیک بیشینه برای یک گراف مفروض است.

**مسئله تصمیم‌گیری کلیک**، این است که به ازای یک عدد صحیح و مثبت  $k$ ، تعیین کنیم که آیا کلیک حاوی حداقل  $k$  رأس وجود دارد یا خیر. این مسئله همان پارامترهای مسئله بهینه‌سازی کلیک را دارد به علاوه پارامتر  $k$ .

**مثال ۲: مسأله بهینه‌سازی فروشنده دوره‌گرد** این بود که یک تور با حداقل کل وزن یال‌ها را به دست آوریم. منظور از تور مسیری است که از یک رأس شروع و به همان رأس ختم شود و از بقیه رئوس دیگر دقیقاً یک بار عبور کند. مسأله تصمیم‌گیری فروشنده دوره‌گرد آن است که مشخص سازیم آیا به ازای عدد مثبت  $d$  داده شده، توری با وزن کل کمتر یا مساوی  $d$  وجود دارد یا نه. تابع حل این مسأله همان پارامترهای مسأله بهینه‌سازی را دارد به علاوه پارامتر اضافی  $d$ .

**مثال ۳: مسأله بهینه‌سازی کوله‌پشتی صفر** و یک انتخاب قطعاتی از  $n$  قطعه جواهر با وزن‌ها و ارزش‌های معین بود به گونه‌ای که ارزش کل حداکثر بوده و علاوه بر آن وزن کل از  $W$  تجاوز نکند. مسأله تصمیم‌گیری کوله‌پشتی صفر و یک آن است که برای یک ارزش داده شده  $P$ ، معین کنیم که آیا می‌توان کوله‌پشتی را به نحوی با قطعات بار کرد که ارزش کل حداقل  $P$  و وزن کل حداکثر  $W$  باشد یا خیر. بدیهی است این مسأله نیز همان پارامترهای مسأله بهینه‌سازی کوله‌پشتی را دارد همراه با پارامتر اضافی  $P$ .

**مثال ۴: مسأله بهینه‌سازی رنگ‌آمیزی گراف** تعیین حداقل تعداد رنگ‌های مورد نیاز برای رنگ‌آمیزی گرافی معین است به گونه‌ای که هیچ دو رأس مجاوری هم رنگ نباشند و این عدد مورد نظر را عدد کروماتیک گراف می‌گوئیم. مسأله تصمیم‌گیری رنگ‌آمیزی گراف آن است که برای یک عدد صحیح  $m$  داده شده تعیین کنیم که آیا رنگ‌آمیزی وجود دارد که حداکثر از این  $m$  رنگ استفاده کرده و هیچ دو رأس مجاوری هم رنگ نباشند. این مسأله نیز همان پارامترهای بهینه‌سازی رنگ‌آمیزی گراف را دارد به علاوه یک پارامتر اضافی  $m$ .

تا امروز برای حالت‌های تصمیم‌گیری و بهینه‌سازی‌های فوق الگوریتمی از مرتبه چندجمله‌ای پیدا نشده است. اما اگر بتوان یک الگوریتم از مرتبه چندجمله‌ای برای بهینه‌سازی پیدا کرد برای مسأله تصمیم‌گیری متناظر با آن نیز یک الگوریتم از مرتبه چندجمله‌ای خواهیم داشت. مثلاً اگر در مدت زمان چندجمله‌ای دریابیم که وزن یک تور بهینه برای گرافی معین  $350$  است پاسخ مسأله تصمیم‌گیری برای مثلاً  $370$ ، بلی می‌باشد.

پس در ادامه به جای مسائل بهینه‌سازی روی مسائل تصمیم‌گیری تمرکز می‌کنیم.

## ۱- کلاس P

به طور کلی مسائل تصمیم‌گیری را به چهار کلاس تقسیم می‌کنیم. مسائلی که برای حل آنها الگوریتم یا الگوریتم‌هایی با مرتبه زمانی چند جمله‌ای یافت شده است کلاس الگوریتم‌های معین<sup>۱</sup> یا الگوریتم‌های قطعی را تشکیل می‌دهند. این کلاس را با علامت ویژه  $p$  که کوتاه شده polynomial می‌باشد نمایش می‌دهیم.

<sup>1</sup> -Deterministic algorithms

الگوریتم‌های تصمیم‌گیری که در فصل‌های قبلی بررسی نمودیم و مرتبه زمانی آنها چند جمله‌ای است به این کلاس تعلق دارند. الگوریتم‌هایی که برای کامپیوترهای رایج طراحی می‌گردند الگوریتم‌های معین نامیده می‌شوند. در این الگوریتم‌ها نتیجه هر عمل کاملاً معین، مشخص و قطعی است. یک نمونه ساده از الگوریتم‌های معمول را بیان می‌کنیم که ضمن مرور این نوع الگوریتم‌ها بتوان آن را با الگوریتم متناظرش به روش نامعین نیز مقایسه کرد. قرار است داده  $x$  در آرایه عددی  $A$  جستجو گردد و چنانچه پیدا شد  $true$  (بله) وگرنه  $false$  (خیر) برگردد. به الگوریتم جستجوی ترتیبی  $dsearch$  توجه کنید.

```
boolean dsearch (float A[], int n, float x)
{
    int i;
    for(i = 0; i < n; i++)
        if(A[i] == x){
            print f(i);
            return true;
        }
    print f(-1);
    return false;
}
```

مرتبه زمانی این الگوریتم  $O(n)$  است.

**تعریف:**  $p$  کلاس کلیه مسائل تصمیم‌گیری است که برای حل آنها الگوریتم معین با مرتبه زمانی چند جمله‌ای وجود دارد.

## ۲- کلاس NP (Nondeterministic polynomial)

برای معرفی مسائل کلاس NP باید ماشینی (کامپیوتری) با مشخصات جدید تعریف کنیم. ماشینی را تصور کنید که افزون بر توانایی اجرای دستورهای معین و قطعی قادر است برخی دستورهای نامعین را نیز اجرا کند. یک دستور نامعین دستوری است که نتیجه اجرای آن از قبل قابل پیش‌بینی نیست. برای مثال فرض کنید دستوری داشته باشیم که از بین اعداد یک تا ده یکی را انتخاب نماید، قبل از اجرای چنین دستوری نمی‌توان پیش‌بینی کرد که دقیقاً کدامیک از اعداد ۱ تا ۱۰ انتخاب خواهد شد. چنین ماشینی را یک ماشین نامعین<sup>۱</sup> یا یک کامپیوتر نامعین<sup>۲</sup> می‌نامیم. الگوریتم‌هایی که برای یک کامپیوتر نامعین طراحی می‌شوند الگوریتم‌های نامعین<sup>۳</sup> نامیده

---

1- Nondeterministic machine  
2- Nondeterministic computer  
3- Nondeterministic algorithm

می‌شوند. برای بیان اینگونه الگوریتم‌ها افزون بر دستورهایی که برای بیان الگوریتم‌های معین وجود دارد از سه تابع زیر استفاده خواهیم کرد.

**الف) تابع choice (S):** این تابع یکی از عناصر مجموعه S را به دلخواه انتخاب می‌کند. عمل انتخاب برای یک کامپیوتر نامعین عمل ساده‌ای (غیرمرکب) است و مرتبه زمانی آن  $O(1)$  است.

**ب) تابع failure():** این تابع پایان ناموفق عملیات الگوریتم را اعلام می‌کند. این اعلام به مفهوم جواب false برای مسأله تصمیم‌گیری مورد نظر می‌باشد. مرتبه زمانی این تابع هم  $O(1)$  است.

**پ) تابع success ():** این تابع پایان موفق عملیات الگوریتم را اعلام می‌کند. این اعلام به مفهوم جواب true برای مسأله تصمیم‌گیری مورد نظر می‌باشد. مرتبه زمانی این تابع نیز برای کامپیوترهای نامعین  $O(1)$  است.

حال به حل مسأله جستجوی x در آرایه A از اعداد برای یک کامپیوتر نامعین می‌پردازیم.

```
void ndsearch (float A[], int n, float x)
{
    int i;
    i = choice (1 .. n);
    if (A[i] == x) {
        printf(i);
        success();
    }
    printf(-1);
    failure();
}
```

مرتبه زمانی این الگوریتم  $O(1)$  است.

تشخیص اول بودن یا نبودن یک عدد طبیعی، مرتب کردن اطلاعات و صدق‌پذیری<sup>۱</sup> نمونه‌هایی از مسائلی هستند که می‌توان الگوریتم‌های نامعین ساده‌ای برایشان طراحی کرد. همان‌گونه که در

بیان یک الگوریتم معین لزوماً نباید کلیه دستورهایی زبان بیان الگوریتم را به

کار گرفت و مثلاً در یک الگوریتم ممکن است از ساختار case (یا switch) استفاده نشود، در بیان یک الگوریتم نامعین نیز ممکن است از همه دستورها

و بویژه از دستورهایی choice، failure و success استفاده نشود. بنابراین هر الگوریتم معین توسط یک کامپیوتر نامعین قابل اجراست. پس  $P \subseteq NP$ . محققین زیادی سعی کرده‌اند ثابت کنند  $P = NP$  اگر این مسأله ثابت شود مفهوم آن این است که هر مسأله‌ای که برای آن الگوریتم

1- Satisfiability

نامعین با مرتبه زمانی چند جمله‌ای وجود دارد می‌توان برای آن یک الگوریتم معین با مرتبه زمانی چند جمله‌ای پیدا کرد. در وضعیت فعلی شکل روبرو رابطه کلاسه‌های P و NP را نشان می‌دهد.

### مسئله صدق‌پذیری

یکی از مسائل عضو کلاس NP مسئله صدق‌پذیری است. فرض کنید  $x_1, x_2, \dots, x_n$  متغیرهای بولین<sup>۱</sup> باشند، که مقدار هریک از آنها می‌تواند true یا false باشد؛ فرض کنید  $\bar{x}_i$  نشان دهنده نقیض  $x_i$  باشد،  $i = 1, 2, \dots, n$ . یک لفظ<sup>۲</sup> می‌تواند یک متغیر یا نقیض آن باشد. یک عبارت منطقی ترکیب درستی از لفظها و عملگرهای "and"، "or" و "()" است. برای مثال چنانچه عملگر and را با  $\wedge$  و عملگر or را با  $\vee$  نشان دهیم، عبارت‌های ذیل عبارت‌های منطقی می‌باشند.

$$(x_1 \wedge \bar{x}_2) \vee (x_3 \wedge x_4)$$

$$(x_3 \wedge \bar{x}_1) \vee (x_4 \wedge x_3)$$

مسئله صدق‌پذیری بررسی این مطلب است که آیا یک عبارت منطقی داده شده به ازاء مقادیری از لفظ‌های تشکیل‌دهنده آن درست خواهد بود. طراحی یک الگوریتم نامعین که تشخیص بدهد یک عبارت داده شده صدق‌پذیر است یا خیر مشکل نیست. الگوریتم ndsatisfy پایان (successful) خواهد داشت اگر و فقط اگر عبارت E صدق‌پذیر باشد وگرنه پایان ناموفق (failure) خواهد داشت.

```
void ndsatisfy(E, n)
{
    boolean x[];
    // متغیرهای عبارت E،  $x_1$  و  $x_2$  و .....  $x_n$  هستند.
    for(i = 1; i <= n; i++)
        x[i] = choice (false, true);
    if (E(x)) // اگر حاصل ارزیابی عبارت درست است ....
        success();
    else
        failure();
}
```

1- Boolean  
2- Literal

مرتبه زمانی الگوریتم ndsatisfy برابر  $O(n)$  است، حال اگر الگوریتم معینی برای این مسأله بنویسیم مرتبه زمانی آن  $O(2^n)$  خواهد بود. دلیل آن این است که هریک از متغیرها می‌تواند true یا false باشد، برای ترکیب  $n$  متغیر باید  $2^n$  حالت را بررسی کرد.

عملکرد یک ماشین نامعین را چگونه می‌توانیم تفسیر کنیم؟ به الگوریتم ndsearch توجه کنید. در دستور  $i = \text{choice}(1..n)$  مقدار  $i$  می‌تواند یکی از مقادیر ۱، ۲، ... و  $n$  باشد، یک کامپیوتر چند پردازنده<sup>۱</sup> را در نظر بگیرید که یکی از پردازنده‌ها عملیات ndsearch را آغاز می‌نماید و در لحظه‌ای که دستور  $i = \text{choice}(1..n)$  اجرا شود تعداد  $n$  پردازنده هر یک یکی از مقادیر مجاز  $i$  را انتخاب می‌کنند و یک کپی از بقیه روال ndsearch را برمی‌دارند و عملیات آن را دنبال می‌کنند. اگر در یکی از این پردازنده‌ها  $A[i] = x$  باشد، این پردازنده مقدار  $i$  را چاپ کرده و اعلام خواهد کرد که success اتفاق افتاده و کار پایان می‌یابد. اگر هیچ‌یک از پردازنده‌ها success اعلام نکنند آنگاه مقدار ۱- چاپ خواهد شد و اعلام خواهد شد که failure اتفاق افتاده است. در این تفسیر تعداد پردازنده‌های مورد نیاز به تعداد داده‌ها ( $n$ ) بستگی دارد و پرواضح است که تعداد پردازنده‌های کامپیوترهای موجود محدود می‌باشد و برای هر مقدار دلخواه  $n$  کفایت نمی‌کند. چنانچه تعداد پردازنده‌ها نامتناهی باشد، امکان اجرای الگوریتم‌های نامعین فراهم می‌گردد.

در سال ۱۹۷۱، اس.ای. کوک<sup>۲</sup> ثابت کرد که چنانچه ثابت شود که صدق‌پذیری به کلاس  $P$  تعلق دارد آنگاه  $P = NP$  است و بالعکس با بیان این مطلب مسأله صدق‌پذیری از اهمیت بالایی برخوردار می‌شود، به گونه‌ای که کلاسهای NP-hard و NP-complete بر محور این قضیه شکل می‌گیرند. افزون بر آن تلاش دانشمندانی را که در مسیر اثبات  $P = NP$  فعالیت می‌کنند جهت‌دار می‌نماید و فعالیت آنها را حول محور تلاش برای یافتن راه حل معین چندجمله‌ای برای مسأله صدق‌پذیری متمرکز می‌کند.

### ۳- کلاس NP-hard

ابتدا مفهوم کاهش‌پذیری<sup>۳</sup> بین مسائل را تعریف می‌کنیم.

**تعریف:** فرض کنید  $M_1$  و  $M_2$  دو مسأله باشند. مسأله  $M_1$  به مسأله  $M_2$  کاهش پیدا می‌کند (این گونه می‌نویسیم  $M_1 \Rightarrow M_2$ ) اگر و فقط اگر چنانچه یک الگوریتم معین با مرتبه زمانی چند جمله‌ای برای  $M_2$  وجود داشته باشد، آنگاه بتوان با به کارگیری الگوریتم معین با مرتبه زمانی چند جمله‌ای که  $M_2$  را حل می‌کند،  $M_1$  را با الگوریتمی معین و زمان چند جمله‌ای حل کرد.

1- Multiprocessor.

2- S.A. Cook

3- Reducibility



مفهوم تعریف بالا آن است که، چنانچه برای مسئله  $M_2$  الگوریتمی معین با مرتبه زمانی چند جمله‌ای یافت شود، آنگاه برای  $M_1$  نیز الگوریتمی معین با مرتبه زمانی چند جمله‌ای یافت خواهد شد. همچنین از این تعریف نتیجه می‌شود که اگر  $M_1 \Rightarrow M_2$  آنگاه  $M_1$  قابل تبدیل به  $M_2$  است و برای این تبدیل تلاشی «معین» با مرتبه زمانی چندجمله‌ای کفایت می‌کند. رابطه کاهش‌پذیری یک رابطه با خاصیت تعدی است. یعنی اینکه اگر  $M_1 \Rightarrow M_2$  و  $M_2 \Rightarrow M_3$  آنگاه  $M_1 \Rightarrow M_3$ .

**تعریف:** مسئله  $M$  یک مسئله NP-hard است اگر و فقط اگر صدق‌پذیری به آن کاهش پیدا کند (satisfiability  $\Rightarrow M$ ).

کلاس NP-hard تنها به مسائل تصمیم‌گیری محدود نمی‌شود. نمونه‌ای از مسائل NP-hard که در عین حال NP-complete نیست «مسئله توقف‌پذیری»<sup>۱</sup> است. در این مسئله هدف ساختن الگوریتمی است که مشخص کند آیا یک برنامه کامپیوتری داده شده هیچ‌وقت پایان خواهد یافت (یعنی اجرای آن تکمیل خواهد شد و توقف خواهد کرد و یا وارد یک حلقه نامتناهی خواهد گردید). ورودی چنین الگوریتمی می‌تواند هر برنامه کامپیوتری و طیف داده‌های آن باشد و خروجی چنین الگوریتمی بله یا خیر است. بله یعنی اینکه برنامه ورودی توقف خواهد کرد و خیر یعنی برنامه ورودی در حلقه‌ای نامتناهی گیر خواهد افتاد. الگوریتمی که برای هر برنامه ورودی و طیف داده‌های آن جواب درستی بدهد محاسبه‌ناپذیر<sup>۲</sup> است، یعنی هیچ راه‌حل الگوریتمی برای آن وجود ندارد. نتیجه اینکه «مسئله توقف‌پذیری» به کلاس NP تعلق ندارد.

#### ۴- کلاس NP-complete

کلاس NP-complete زیرکلاس، کلاسهای NP و NP-hard است. در واقع کلاس NP-complete فصل مشترک کلاسهای NP و NP-hard را تشکیل می‌دهد. به تعریف ذیل دقت کنید.

**تعریف:** مسئله  $M$  یک مسئله NP-complete است اگر و فقط اگر  $M$  یک مسئله NP-hard باشد و در عین حال  $M \in NP$ ، [Gar79].

برخی از عبارتهای منطقی به صورت ترکیبی عطفی<sup>۳</sup> نوشته می‌شوند. این گونه عبارتهای منطقی یا یک بند<sup>۴</sup> هستند و یا ترکیب عطفی (and) چند بند. می‌گوییم این عبارتها به شکل نرمال عطفی<sup>۵</sup> هستند و با علامت اختصاری CNF نشان داده می‌شوند. عبارت

1- Halting problem

2- Noncomputable

3- Conjunctive

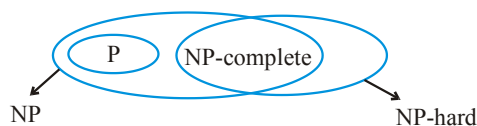
4- Clause

5-Conjunctive normal form

به شکل  $(p + \bar{q} + r)(\bar{p} + q + r)(q)(\bar{r})$  CNF است. شکل نرمال عطفی در واقع همان شکل حاصل ضرب مجموع‌هاست.<sup>۱</sup> ثابت می‌شود که صدق‌پذیری عبارتهای CNF یک مسأله NP-complete است. متعاقباً می‌توان ثابت کرد که صدق‌پذیری خود به کلاس NP-complete تعلق دارد، چرا که  $\text{CNF} - \text{satisfiability} \Rightarrow \text{satisfiability}$  و با توجه به اصل تعدی و از آنجا که  $\text{satisfiability} \Rightarrow \text{CNF} - \text{satisfiability}$  می‌توان نتیجه مطلوب را استخراج کرد.

نمونه دیگری از مسائل NP-complete مسأله سه رنگ‌پذیری<sup>۲</sup> است. اگر  $G$  یک گراف بدون جهت باشد مسأله سه رنگ‌پذیری بررسی می‌نماید که آیا این گراف سه رنگ‌پذیر می‌باشد یا خیر. آیا می‌شود گره‌های گراف را با استفاده از تنها سه رنگ، رنگ‌آمیزی کرد که هیچ دو گره مجاور رنگ یکسانی نداشته باشند؟ چهار رنگ‌پذیری گرافها نیز یک مسأله NP-complete است. اما دو رنگ‌پذیری یک مسأله P است.

به طور کلی وضعیت فعلی چهار کلاس مسائل در شکل زیر آمده است.




---

1- Product of sums  
2- Three- colorability

## سؤال‌های چهار گزینه‌ای فصل یازدهم

۱- گزینه صحیح را انتخاب کنید. (علوم کامپیوتر - ۸۱)

(۱) مسائل NP-Complete زیرمجموعه مسائل NP-hard می‌باشند.

(۲) مسائل NP-hard زیرمجموعه مسائل NP-Complete می‌باشند.

(۳) مسائل NP زیرمجموعه مسائل P می‌باشند.

(۴)  $P = NP$

۲- اگر یک مسئله NP-Complete مانند L وجود داشته باشد که  $L \in P$  باشد در آن صورت:

(علوم کامپیوتر - ۸۲)

(۲)  $P \neq NP$

(۱)  $P = NP$

(۴)  $L \notin NP\text{-hard}$

(۳)  $L \in NP\text{-hard}$

۳- کدام گزینه نادرست است؟ (علوم کامپیوتر - ۸۳)

(۱) تمام مسائل P به وسیله یک الگوریتم غیرقطعی در زمان چندجمله‌ای حل می‌شوند.

(۲) تمام مسائل NP به وسیله یک الگوریتم غیرقطعی در زمان چندجمله‌ای حل می‌شوند.

(۳) تمام مسائل NP-hard به وسیله یک الگوریتم غیرقطعی در زمان چندجمله‌ای حل می‌شوند.

(۴) تمام مسائل NP-Complete به وسیله یک الگوریتم غیرقطعی در زمان چندجمله‌ای حل می‌شوند.

۴- کدام گزاره نادرست است؟ (علوم کامپیوتر - ۸۷)

(۲)  $P \subseteq NP$

(۱)  $P = NP \cap CO\text{-}NP$

(۴)  $\text{if } NP \neq CO\text{-}NP \Rightarrow P \neq NP$

(۳)  $P \subseteq CO\text{-}NP$

۵- پیچیدگی زمان الگوریتم محاسبه عبارات true برای یک عبارت منطقی (satisfiability) با

(علوم کامپیوتر - ۸۷)

n متغیر برابر است با:

(۱)  $O(2^n)$  (۲)  $O(n^{2^n})$  (۳)  $O(n^k)$  (۴)  $O(2^{2^n})$

۶- کدام گزینه نادرست است؟ (علوم کامپیوتر - ۸۸)

(۱)  $P \subset NP\text{-Complete}$

(۲)  $A \leq_p B \Rightarrow A \in NP$

(۳)  $NP\text{-Complete} \subset NP\text{-hard}$

(۴) برای  $A \leq_p B$  داریم: اگر B در زمان چند جمله‌ای حل شود A نیز حل می‌شود.

۷- مسئله زیر را در نظر بگیرید:

(علوم کامپیوتر ۸۸)

$$\begin{aligned} \sum_{1 \leq i \leq n} x_i w_i &\leq M \\ \max \sum_{1 \leq i \leq M} x_i p_i \\ \text{s.t. } x_i &= \begin{cases} 0 \\ 1 \end{cases} \end{aligned}$$

کدام گزینه نادرست است؟

- (۱) این مسئله یک مسئله NP نمی‌باشد.
- (۲) این مسئله یک مسئله NP-hard است.
- (۳) این مسئله راه حل چند جمله‌ای برحسب هیچ پارامتری ندارد.
- (۴) این مسئله یک راه حل چند جمله‌ای برحسب  $n$  و  $M$  دارد.

۸- پیچیدگی الگوریتم غیرقطعی برای مسئله  $k$ -clique در یک گراف  $G = (V, E)$  برابر است

(علوم کامپیوتر ۸۹):

$$(۱) O(k^2) \quad (۲) O(|V|^2 k) \quad (۳) O(k + |V|^2) \quad (۴) O(|V| + |E|)$$

۹- کدام یک از مسائل زیر NP می‌باشد؟

(علوم کامپیوتر ۸۹)

$$\begin{aligned} (۱) \text{ Minimum node cover} \quad (۲) \text{ Max clique} \\ (۳) 3\text{-sat} \quad (۴) \text{ همه موارد} \end{aligned}$$

۱۰- در مسئله کوله‌پشتی، اگر وزن کوله‌پشتی با یک تابع چندجمله‌ای از تعداد اشیاء محدود

شود، آنگاه این مسئله متعلق به کدام رده از مسائل است؟ (علوم کامپیوتر ۹۳)

$$(۱) \text{ NP\_Complete} \quad (۲) \text{ NP\_Hard} \quad (۳) \text{ CO\_NP} \quad (۴) \text{ P}$$

۱۱- فرض کنید  $A \leq_p B$  بوده و  $A$  یک مسئله NP\_کامل باشد. در مورد مسئله  $B$  کدام

(علوم کامپیوتر ۹۳)

گزینه صحیح است؟

- (۱) مسئله  $B$  حتماً یک مسئله NP\_کامل است.
- (۲) مسئله  $B$  نمی‌تواند جزء مسائل NP باشد.
- (۳) در رابطه با مسئله  $B$  نمی‌توان نظر داد.
- (۴) اگر  $B \in \text{NP}$  باشد، آنگاه  $B$  یک مسئله NP\_کامل است.

۱۲- فرض کنید  $\{x_1, x_2, \dots, x_n\}$  مجموعه‌ای از اعداد صحیح و  $S$  حاصل جمع آنها باشد.

$$(S = \sum_{i=1}^n x_i) \text{ . بهترین الگوریتم برای افراز این مجموعه به دو زیرمجموعه (در صورت}$$

وجود) به طوری که جمع عناصر دو زیرمجموعه مساوی باشند، دارای چه مرتبه زمانی است؟ (علوم کامپیوتر - ۹۴)

(۱)  $O(nS)$  (۲)  $O(n + S)$  (۳)  $O(n^2)$  (۴)  $O(n \log n)$

۱۳- برای اینکه نشان دهیم مسئله  $A$  متعلق به خانواده مسائل NP-Complete است، یافتن یک مسئله NP-Complete مانند  $B$  و کاهش  $B$  به  $A$  ( $B \leq_p A$ ) ... (علوم کامپیوتر - ۹۴)

(۱) شرط لازم و کافی است. (۲) شرط کافی است ولی لازم نیست.  
(۳) شرط لازم است ولی کافی نیست. (۴) نه شرط لازم و نه شرط کافی است.

۱۴- فرض کنید  $Q(x_1, \dots, x_n)$  یک عبارت بولی باشد. به یک مقداردهی متغیرهای  $x_1, \dots, x_n \in \{0, 1\}$  یک جواب  $Q$  می‌گوییم اگر عبارت  $Q$  به ازای آن مقداردهی برابر ۱ شود، کدام یک از دو مسئله زیران پی - سخت است؟ (دکتری کامپیوتر - ۹۵)

(الف) پیدا کردن تعداد جواب‌های  $Q$  اگر  $Q$  در قالب DNF داده شده باشد.  
(ب) پیدا کردن تعداد جواب‌های  $Q$  اگر  $Q$  در قالب CNF داده شده باشد.  
(۱) فقط الف  
(۲) فقط ب  
(۳) هر دو الف و ب  
(۴) با این اطلاعات در مورد ان پی - سخت بودن این مسائل نمی‌توان اظهار نظر کرد.

۱۵- همه موارد زیر صحیح‌اند، به غیر از: (دکتری علوم کامپیوتر - ۹۵)

(۱) اگر  $P = NP$  باشد آنگاه  $NP = NP - complete$  خواهد بود.  
(۲) هر مسئله از کلاس NP می‌تواند در زمان نمایی حل شود.  
(۳) مسئله «برای هر  $n = p \times q$  که در آن  $p$  و  $q$  اعداد اول  $k$  بیتی هستند،  $p$  و  $q$  را پیدا کن» یک مسئله از کلاس NP است.  
(۴) اگر یک مسئله  $x$  بتواند به یک مسئله شناخته شده از کلاس NP - Hard کاهش یابد، آنگاه  $x$  باید از کلاس NP - Hard باشد.

۱۶- چند تا از گزاره‌های زیر درست است؟ (هوش مصنوعی - ۹۵)

(الف) در گراف جهت‌دار شبکه‌ی شار اگر هر یال را با یال بدون جهت و با همان ظرفیت تعویض کنیم، مقدار شار بیشینه تغییر نمی‌کند.  
(ب) اگر ظرفیت یال‌ها متمایز باشد، شار بیشینه (نه مقدار آن) یکتا است.  
(ج) مسئله‌ی یافتن کوتاه‌ترین دور فروشنده‌ی دورگرد ان پی است.  
(۱) ۰ (۲) ۱ (۳) ۲ (۴) ۳

۱۷- در مورد مسئله‌ی کوله‌پشتی که قرار است یک کوله‌پشتی به اندازه‌ی  $M$  (عدد صحیح) را با  $N$  نوع بار با وزن‌های متفاوت ولی با ارزش یکسان کاملاً پر کنیم، اگر از هر نوع بار به تعداد خیلی زیاد موجود باشد، کدام گزاره درست است؟ بهترین گزینه را انتخاب کنید. (IT – ۹۵)

(۱) این مسئله ان پی – سخت است.

(۲) برای این مسئله راه‌حل  $O(N \log M)$  وجود دارد.

(۳) برای این مسئله راه‌حل  $O(M \log N)$  وجود دارد.

(۴) اگر از هر نوع بار فقط یک عدد داشته باشیم، مسئله در زمان چندجمله‌ای قابل حل است.

۱۸- چند تا از گزاره‌های زیر درست است؟ (IT – ۹۵)

الف) با یک الگوریتم خطی  $O(V + E)$  می‌توان تشخیص داد که یک DAG مسیر همیلتنی دارد یا خیر.

ب)  $MST \leq_p 3SAT$

ج)  $3SAT \leq_p MST$

(۱) ۰ (۲) ۱ (۳) ۲ (۴) ۳

## پاسخنامه سؤال‌های چهارگزینه‌ای فصل یازدهم

- ۱- گزینه (۱). مسائل  $P$  زیرمجموعه مسائل  $NP$  می‌باشند لذا گزینه ۳ غلط است. هنوز اثبات نشده است که آیا  $NP = P$  است یا خیر. پس گزینه ۴ هم غلط است.
- ۲- گزینه (۱). اگر هر یک از مسائل  $NP$  کامل با مرتبه چندجمله‌ای حل شوند آنگاه همه مسائل  $NP$  با مرتبه چندجمله‌ای حل می‌شوند، یعنی  $P = NP$  می‌شوند.
- ۳- گزینه (۳). از آنجا که گزینه ۲ درست بوده و می‌دانیم  $P \subseteq NP$  است پس گزینه ۱ هم درست می‌باشد. از آنجا که  $NP$  کامل زیرمجموعه  $NP$  می‌باشد پس گزینه ۴ هم درست است.
- ۴- گزینه (۱). هنوز درستی گزینه (۱) را نمی‌دانیم.
- ۵- گزینه (۲). اگر مقدار متغیرها از قبل معلوم باشد در زمان  $O(n)$  می‌توان نتیجه عبارت را بدست آورد. ولی اگر مقدار متغیرها معلوم نباشد، هر متغیر ۲ حالت دارد پس  $2^n$  حالت برای متغیرها وجود دارد و  $n-1$  عملگر باید بررسی شوند که مرتبه  $O(2^n)$  می‌شود.
- ۶- گزینه (۱ و ۲).  $P$  زیرمجموعه  $NP$  است و نه  $NPC$ . در گزینه (۲) مشخص نشده  $B$  چه نوع مسئله‌ای است.
- ۷- گزینه (۱). مسئله بهینه‌سازی کوله‌پشتی صفر و یک در کلاس  $NP$  نیست، بلکه  $NP$ -hard است. ولی مسئله تصمیم‌گیری کوله‌پشتی صفر و یک در کلاس  $NP$ -complete است. در ضمن این مسئله از مرتبه  $\theta(nM)$  است، که برحسب  $n$  و  $M$  چندجمله‌ای است، ولی برحسب  $n$  چندجمله‌ای نیست.
- ۸- گزینه (۱).
- ۹- گزینه (۳). گزینه‌های ۱ و ۲ بهینه‌سازی هستند و در کلاس  $NP$ -hard هستند و در کلاس  $NP$  نیستند. کلاً کلاس  $NP$  فقط شامل مسائل تصمیم‌گیری است.
- ۱۰- گزینه (۴).
- ۱۱- گزینه (۴). چون  $B$  از  $A$  آسان‌تر نیست و سخت‌ترین مسائل  $NP$  همان  $NPC$  ها هستند.
- ۱۲- گزینه (۱). مثل مسئله کوله‌پشتی حل می‌شود.
- ۱۳- گزینه (۳). اگر  $A$  یک مسئله  $NP$ -complete باشد آنگاه وجود دارد مسئله‌ای  $NP$ -complete مثل  $B$  که  $B$  را می‌توان با زمان چند جمله‌ای به  $A$  تبدیل کرد ولی عکس این مطلب صحیح نیست.
- ۱۴- گزینه (۳).

- ۱۵- گزینه (۴). گزینه‌های ۱ و ۲ و ۳ جملات درستی هستند.
- ۱۶- گزینه (۱). الف) مثلاً در گراف  $(t) \leftarrow (u) \leftarrow (s)$  هیچ شاری نمی‌توان از  $s$  به  $t$  فرستاد ولی اگر جهت‌ها را نادیده بگیریم می‌توان ۴ واحد شار از  $s$  به  $t$  ارسال کرد. ب) حتی در صورت متفاوت بودن ظرفیت‌ها ممکن است یال‌ها شارهای متفاوتی را منتقل کنند و ممکن است شار بیشینه با حالات مختلف حاصل شود. ج) یافتن کوتاه‌ترین دور فروشنده دوره‌گرد، یک مسئله ان پی سخت است و ان پی کامل نیست در نتیجه ان پی نیست. متأسفانه در کتاب CLRS گفته شده این مسئله ان پی کامل است که البته در آنجا منظور مسئله تصمیم‌گیری فروشنده دوره‌گرد می‌باشد و نه یافتن دور کمینه همیلتنی.
- ۱۷- گزینه (۱). این مسئله ان پی سخت است.
- ۱۸- گزینه (۳). الف و ب صحیح هستند. با ترتیب توپولوژیکی می‌توان الف را حل کرد. مسئله MST با زمان چندجمله‌ای به مسئله ۳SAT که ان پی کامل است، قابل تبدیل است. به عبارتی MST از ۳SAT مسئله آسان‌تری است.





## کنکورهای سراسری ارشد و دکتری ۹۹-۹۶

### [سؤالات چهارگزینه‌ای مهندسی کامپیوتر ۱۳۹۶]

- ۱- برای دنباله‌های متشکل از اعداد تعاریف زیر را در نظر بگیرید:
- $LCS(X_1, \dots, X_k)$ : بزرگ‌ترین زیردنباله‌ی مشترک دنباله‌های  $X_1, \dots, X_k$
  - $LIS(X)$ : بزرگ‌ترین زیردنباله‌ی صعودی دنباله‌ی  $X$
  - $SRT(X)$ : دنباله‌ی مرتب شده (به صورت صعودی) دنباله‌ی  $X$
- حال فرض کنید  $A$  و  $B$  دو دنباله از اعداد متمایز باشند. کدام یک از عبارات زیر بزرگ‌ترین زیردنباله‌ی صعودی مشترک  $A$  و  $B$  را محاسبه می‌کند؟
- (۱)  $LCS(LIS(A), B)$       (۲)  $LIS(LCS(A, B))$
- (۳)  $LCS(A, B, SRT(A))$       (۴)  $LCS(LCS(A, SRT(A)), B)$
- ۲- فرض کنید یک گراف کامل وزن دار همبند  $n$  رأسی با وزن‌های مثبت و متمایز داده شده است. رئوس گراف را به دو دسته‌ی  $A$  و  $B$  با اندازه‌های مساوی (با حداکثر اختلاف ۱) افزایش می‌کنیم. درخت پوشای کمینه بر روی مجموعه‌های  $A$  و  $B$  را به طور مستقل به وسیله‌ی یکی از الگوریتم‌های پریم یا کروسکال محاسبه می‌کنیم و در نهایت سبک‌ترین یالی که یک سر آن در  $A$  و دیگری در  $B$  هست برای اتصال دو درخت برمی‌گزینیم تا یک درخت پوشا ایجاد شود. در مورد وزن این درخت چه می‌توان گفت؟ فرض کنید  $n$  حداقل ۴ است.
- (۱) وزن آن برابر وزن درخت پوشای کمینه‌ی گراف است.
- (۲) وزن آن حداکثر دو برابر وزن درخت پوشای کمینه‌ی گراف است.
- (۳) مثالی وجود دارد که وزن آن  $n$  برابر وزن درخت پوشای کمینه‌ی گراف است.
- (۴) یال‌های این درخت با یال‌های درخت پوشای کمینه می‌تواند اشتراکی نداشته باشد.

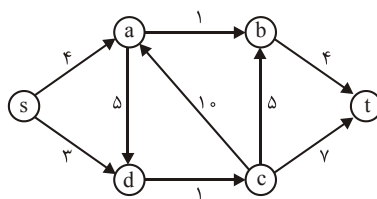
۳- فرض کنید یک گراف ۵ رأسی همبند بدون جهت داریم که رأس‌های آن با شماره‌های ۱ تا ۵ شماره‌گذاری شده‌اند. فرض کنید اگر از رأس ۱ روی درخت DFS را اجرا کنیم، تمام حالت‌هایی که رأس‌ها می‌توانند ملاقات شوند عبارتند از  $(1, 2, 4, 3, 5)$ ،  $(1, 3, 4, 2, 5)$  و  $(1, 3, 5, 4, 2)$ . حال اگر از رأس ۵ DFS را اجرا کنیم، ترتیب ملاقات رأس‌ها کدام یک از گزینه‌های زیر می‌تواند باشد؟

- (۱)  $5, 3, 2, 1, 4$  (۲)  $5, 3, 1, 4, 2$   
(۳)  $5, 4, 2, 1, 3$  (۴)  $5, 3, 4, 2, 1$

۴- فرض کنید در یک گراف وزن‌دار (با وزن‌های مثبت و منفی) که وزن همه‌ی دورها در آن مثبت است می‌خواهیم کوتاه‌ترین فاصله از رأس  $s$  به بقیه‌ی رئوس را محاسبه کنیم. برای این کار ابتدا قرار می‌دهیم  $d(s) = 0$  و  $\forall u \neq s: d(u) = +\infty$ . سپس هر بار به دل‌خواه یک یال  $(u, v)$  را که  $d(u) + w(u, v) < d(v)$  انتخاب کرده و مقدار  $d(v)$  را با مقدار  $d(u) + w(u, v)$  به روزرسانی می‌کنیم که  $w(u, v)$  وزن یال  $(u, v)$  است. کدام یک از گزاره‌های زیر درست است؟

- (۱) ترتیب یال‌ها را می‌توان به گونه‌ای انتخاب کرد که به روزرسانی فوق هیچ‌گاه متوقف نشود.  
(۲) به هر ترتیبی یال‌ها انتخاب شوند با  $O(n^2)$  بار به روزرسانی به ازای  $c$  ثابت الگوریتم متوقف خواهد شد.  
(۳) حتی اگر الگوریتم به روزرسانی متوقف شود، لزوماً به ازای هر  $u$ ،  $d(u)$  برابر طول کوتاه‌ترین مسیر از  $s$  به  $u$  نیست.  
(۴) الگوریتم حتماً بعد از تعداد متناهی مرحله متوقف می‌شود، و در پایان به ازای هر گره  $u$ ،  $d(u)$  طول کوتاه‌ترین مسیر از  $s$  به  $u$  است.

۵- در شبکه‌ی شار زیر حداکثر چند واحد شار می‌توان از رأس  $s$  به رأس  $t$  عبور داد؟



- (۱) ۲  
(۲) ۷  
(۳) ۱۱  
(۴) ۱۲

## [سؤالات چهارگزینه‌ای مهندسی فناوری اطلاعات ۱۳۹۶]

۶- الگوریتم دایکسترای استاندارد را بدون هیچ تغییری بر روی گرافی که حداقل یک یال آن دارای وزن منفی است، اما دور منفی ندارد اجرا می‌کنیم. کدام گزینه صحیح است؟

- (۱) الگوریتم ممکن است هیچگاه متوقف نشود.
- (۲) الگوریتم ممکن است جواب صحیح برگرداند.
- (۳) الگوریتم همیشه جواب صحیح برمی‌گرداند.
- (۴) الگوریتم همیشه جواب غلط برمی‌گرداند.

۷- فرض کنید  $LCS(X_1, \dots, X_k)$  بزرگ‌ترین زیر دنباله‌ی مشترک دنباله‌های  $X_1, \dots, X_k$  باشد. در ضمن فرض کنید  $REV(X)$  معکوس آینه‌ی دنباله‌ی  $X$  باشد. چند تا از عبارت‌های زیر صحیح‌اند؟

$$LCS(A, B, C) = LCS(LCS(A, B), C)$$

$$LCS(LCS(A, B), C) = LCS(A, LCS(B, C))$$

$$LCS(A, REV(A)) = 0$$

- (۱) ۰      (۲) ۱      (۳) ۲      (۴) ۳

۸- در یک گراف همبند بدون وزن و بدون جهت، از یک راس مشخص BFS را اجرا می‌کنیم. کدام گزینه در مورد درخت BFS ایجاد شده نادرست است؟

- (۱) بین دو گره‌ای که اختلاف سطح آن‌ها بیش از یک است، یالی وجود ندارد.
- (۲) هر گره تنها با گره‌هایی که با آن اختلاف سطح حداکثر یک دارند می‌تواند همسایه باشد.
- (۳) سطح هر گره برابر طول کوتاه‌ترین مسیر از راس شروع به آن راس است.
- (۴) بیشترین سطح برابر با قطر گراف است.

۹- رابطه‌ی بازگشتی زیر در الگوریتم «فلوید - وارشال» برقرار است:

$$d_{uv}^k = \min \{d_{uv}^{(k-1)}, d_{uk}^{(k-1)} + d_{kv}^{(k-1)}\}$$

کدام یک از گزینه‌های زیر تعریف درست برای  $d_{uv}^k$  است؟

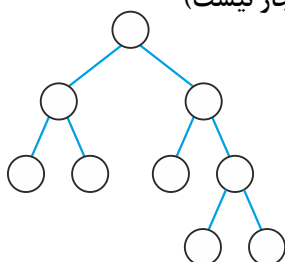
- (۱) طول کوتاه‌ترین مسیر از  $u$  به  $v$  در  $G$  که حداکثر  $k$  عدد یال در آن باشد.
- (۲) طول کوتاه‌ترین مسیر از  $u$  به  $v$  در  $G$  که راس‌های  $\{1, 2, \dots, k\}$  در آن باشد.
- (۳) طول کوتاه‌ترین مسیر از  $u$  به  $v$  در  $G$  که حداکثر  $k$  عدد راس در آن باشد.
- (۴) طول کوتاه‌ترین مسیر از  $u$  به  $v$  در  $G$  که بیشترین وزن یال در آن  $k$  باشد.

- ۱۰- شرط لازم و کافی برای یکتا بودن درخت پوشای کمینه در یک گراف همبند چیست؟
- (۱) وزن همه‌ی یال‌ها متمایز باشد.
  - (۲) در هر دور، سنگین‌ترین یال یکتا باشد.
  - (۳) به ازای هر افراز رئوس به دو مجموعه، سبک‌ترین یالی که هر سر آن در یکی از دو مجموعه فوق است یکتا باشد.
  - (۴) هیچ یک از گزینه‌های ۱ تا ۳.
- ۱۱- فرض کنید  $n$  کاراکتر داخل متن داده شده را با روش کدگذاری هافمن کد کرده‌ایم. بیشترین طول کدها چند می‌تواند باشد؟ بهترین گزینه را انتخاب کنید.

$$(۱) \lceil \log n \rceil + 1 \quad (۲) n \quad (۳) \left\lceil \frac{n}{2} \right\rceil + 1 \quad (۴) n - 1$$

### سؤال‌های چهارگزینه‌ای علوم کامپیوتر ۱۳۹۶

- ۱۲- به چند حالت می‌توان اعداد ۱ تا ۹ را در گره‌های درخت دودویی زیر قرار داد، به طوری که عدد هر گره از اعداد فرزندان آن بزرگتر باشد؟ (تکرار اعداد مجاز نیست)



- (۱) ۹۸۲
- (۲) ۸۹۶
- (۳) ۶۴۸
- (۴) ۵۱۲

- ۱۳- پیمایش پس ترتیب یک درخت جستجوی دودویی DEBFCA بوده است. پیمایش پیش ترتیب آن کدام است؟

$$(۱) ABFCDE \quad (۲) ABDCEF \quad (۳) ABDECF \quad (۴) ADBFEC$$

- ۱۴- در الگوریتم mergesort در هر مرحله لیست را به نسبت ۹ به ۱ تقسیم می‌کنیم، یعنی یک قسمت ۹ برابر قسمت دیگر است و سپس در مرحله ترکیب، این دو لیست با هم ادغام می‌شوند. در این حالت پیچیدگی الگوریتم از چه مرتبه‌ای است؟

$$(۱) \theta(n^2) \quad (۲) \theta(n^{\frac{2}{3}}) \quad (۳) \theta(n \log n) \quad (۴) \theta(n^2 \log n)$$

۱۵- یک ساختار داده‌ای، عمل insert را حمایت می‌کند به طوری که یک دنباله از  $n$  عمل insert به اندازه  $\theta(n \log n)$  در بدترین حالت زمان می‌برد. زمان Amortized و زمان worst-case عمل insert به ترتیب (از راست به چپ) کدام است؟

(۱)  $\theta(n)$  و  $\theta(n \log n)$  (۲)  $\theta(n)$  و  $\theta(\log n)$

(۳)  $\theta(\log n)$  و  $\theta(\log n)$  (۴)  $\theta(\log n)$  و  $\theta(n \log n)$

۱۶- فرض کنید دنباله  $S = [x_1, \dots, x_n]$  از اعداد مثبت صحیح داده شده است. می‌خواهیم یک زیردنباله  $A = [x_{i_1}, x_{i_2}, \dots, x_{i_j}]$  که  $i_1 < i_2 < \dots < i_j$  است را پیدا کنیم که مقدار  $\text{sum} = x_{i_1} - x_{i_2} + x_{i_3} - \dots \pm x_{i_j}$  ماکزیمم باشد. برای یافتن مقدار sum چه روشی از نظر زمانی مناسب‌ترین است؟

(۱) Greedy (۲) Dynamic Programming

(۳) Brute force (۴) Divide and conquer

۱۷- می‌خواهیم در یک آرایه  $n$  تایی نامرتب، عنصری که حداقل  $\left\lfloor \frac{n}{3} \right\rfloor + 1$  بار تکرار شده است را بیابیم، کدام گزینه صحیح است؟

(۱) الگوریتمی با هزینه حداکثر  $O(n)$  وجود دارد.

(۲) الگوریتمی با هزینه حداکثر  $O(\sqrt{n})$  وجود دارد.

(۳) بهترین الگوریتم با زمان اجرای میانگین  $O(n)$  و مبتنی بر درهم‌سازی است.

(۴) بهترین الگوریتم با زمان اجرای میانگین  $O(n \log n)$  و مبتنی بر مرتب‌سازی است.

۱۸- الگوریتم زیر آدرس ریشه یک درخت دودویی را به عنوان ورودی می‌گیرد و با استفاده از یک پشته آن را پیمایش می‌کند، پیمایش انجام شده کدام است؟

```
void Traverse(p) {
    push(p)
    while(stack is not empty){
        q = pop()
        if(q != null){
            push(q → left)
            print(q → data)
            push(q → right)
        }
    }
}
```

(۱) پیمایش inorder  
(۲) پیمایش preorder  
(۳) برعکس پیمایش inorder  
(۴) برعکس پیمایش preorder

۱۹- اگر  $s$  آدرس شروع یک لیست متصل غیرتهی باشد، الگوریتم زیر با فراخوانی  $s = w(\text{null}, s, s)$  چه عملی روی این لیست انجام می‌دهد؟

```
w(m, t, s){
    if(t → link != null){
        s = w(t, t → link, s)
    }
    else s = t
    t → link = m
    return(s)
}
```

(۱) لیست متصل را دایره‌ای می‌کند.

(۲) لیست متصل را برعکس می‌کند.

(۳) لیست متصل را به دو لیست تبدیل می‌کند.

(۴) لیست متصل را به گره‌های بدون اتصال تبدیل می‌کند.

۲۰- فرض کنید گراف کامل وزن دار  $G = (V, E)$  که در آن وزن یال  $i$ ام برابر  $i$ امین عدد در دنباله فیبوناچی (با شروع از ۱) داده شده باشد. تعداد درختان پوشای مینیمم برای  $G$  چند است؟

(۱)  $|V|$  (۲)  $|V| \times |E|$  (۳) ۲ (۴) ۱

۲۱- همهٔ جملات زیر صحیح‌اند، به غیر از:

(۱) اگر  $T$  یک درخت پوشای می‌نیمم گراف  $G$  باشد، آنگاه برای هر جفت از رئوس  $s$  و  $t$ ،

کوتاه‌ترین مسیر از  $s$  به  $t$  در گراف  $G$  لزوماً یک مسیر از  $s$  به  $t$  در درخت  $T$  نیست.

(۲) اگر  $T$  یک درخت پوشای می‌نیمم گراف  $G$  باشد، آنگاه مسیر بین هر جفت رئوس  $s$  و  $t$

در درخت  $T$  لزوماً یک کوتاه‌ترین مسیر در گراف  $G$  نیست.

(۳) هر درخت جستجوی دودویی با  $n$  گره دارای عمق  $O(\log n)$  است.

(۴) زمان جستجوی یک کلید در درخت جستجوی دودویی لزوماً  $O(\log n)$  نیست.

۲۲- اگر اعداد زیر از چپ به راست در یک هرم مینیمم درج شوند ارتفاع این هرم کدام است؟

۱۹، ۱۱، ۴، ۹، ۸، ۱۵، ۱۲، ۶، ۱۷، ۱، ۲، ۳، ۱۴، ۷، ۵، ۱۳، ۲۰

(۱) ۵ (۲) ۶ (۳) ۷ (۴) ۸

۲۳- یک لیست نامرتب دارای  $n$  عنصر است. با این فرض که  $n$  زوج است، کدام گزینه نشان‌دهنده تعداد مقایسه‌های لازم برای تعیین بزرگترین و کوچکترین عنصر در لیست می‌باشد؟

$$\begin{array}{ll} T(n) = 2T\left(\frac{n}{2}\right) + 1 & (۲) \\ T(n) = 2T\left(\frac{n}{2}\right) + 2 & (۱) \\ T(n) = T\left(\frac{n}{2}\right) + \log_2 n - 1 & (۴) \\ T(n) = 2\lceil \log_2 n \rceil + 1 & (۳) \end{array}$$

۲۴- فرض کنید گراف  $G = (V, E)$  داده شده باشد، بهترین زمان برای یافتن یک درخت پوشا برای این گراف کدام است؟

$$\begin{array}{ll} O(|V|) & (۲) \\ O(|E|) & (۱) \\ O(|V| \times |E|) & (۴) \\ O(|E| \times \log |E|) & (۳) \end{array}$$

۲۵- فرض کنید فایلی شامل کاراکترهای  $a$  تا  $f$  با تعداد تکرار داده شده به صورت جدول زیر باشد. اگر از روش هافمن برای فشرده‌سازی این فایل استفاده شود، اندازه فایل فشرده شده چند بیت خواهد بود؟

| کاراکتر     | a  | b | c  | d | e | f |
|-------------|----|---|----|---|---|---|
| تعداد تکرار | ۱۰ | ۷ | ۱۵ | ۶ | ۳ | ۲ |

$$\begin{array}{llll} ۳۴۴ & (۴) & ۱۸۲ & (۳) \\ ۱۰۲ & (۲) & ۹۶ & (۱) \end{array}$$

۲۶- کدام گزینه صحیح است؟

(TSP مخفف مسئله فروشنده دوره گرد و HC مخفف مسئله دور همیلتونی است)

$$\begin{array}{ll} SAT \leq_p HC \leq_p Clique \leq_p 3SAT & (۱) \\ SAT \leq_p 3SAT \leq_p TSP \leq_p Clique & (۲) \\ SAT \leq_p 3SAT \leq_p Clique \leq_p Vertex Cover & (۳) \\ 3SAT \leq_p Clique \leq_p HC \leq_p Vertex Cover & (۴) \end{array}$$



## [پاسخنامه مهندسی کامپیوتر ۱۳۹۶]

۱- گزینه (۳). برای درک بهتر، فرض کنید  $A = \text{pnmghiab}$  و  $B = \text{pnmdeab}$  در این صورت بزرگترین زیر دنباله صعودی مشترک  $A$  و  $B$  برابر  $ab$  است و حال گزینه‌ها را بررسی می‌کنیم:

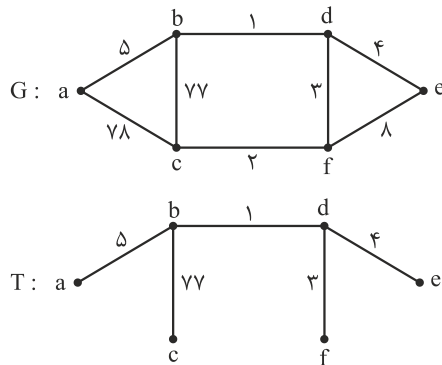
$$\text{LCS}(\text{LIS}(A), B) = \text{LCS}(\text{ghi}, \text{pnmdeab}) = \text{NULL} \neq ab$$

$$\text{LIS}(\text{LCS}(A, B)) = \text{LIS}(\text{pnm}) \neq ab$$

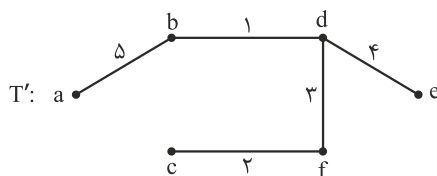
$$\text{LCS}(A, B, \text{SRT}(A)) = \text{LCS}(\text{pnmghiab}, \text{pnmdeab}, \text{abghimnp}) = ab$$

$$\text{LCS}(\text{LCS}(A, \text{SRT}(A)), B) = \text{LCS}(\text{ghi}, \text{pnmdeab}) = \text{NULL} \neq ab$$

۲- گزینه (۳). برای درک بهتر سوال، گراف  $G$  را در نظر بگیرید.  $A = \{a, b, c\}$  و  $B = \{d, e, f\}$  در نظر می‌گیریم. پوشای  $A$  شامل یالهای  $ab$  و  $bc$  است. درخت پوشای  $B$  شامل یالهای  $de$  و  $df$  است؛ در نهایت یال  $bd$  انتخاب می‌شود که دو درخت به هم متصل شوند و نتیجه درخت  $T$  است:



حال آنکه درخت پوشای مینیمم  $G$  درخت  $T'$  است:



پس گزینه ۱ لزوماً درست نیست. وزن یالهای  $T$  برابر ۹۰ و وزن یالهای  $T'$  برابر ۱۵ است که وزن  $T$  شش برابر  $T'$  است پس گزینه ۳ تأیید می‌شود و گزینه ۲ نقض می‌شود. در ضمن حتماً درخت حاصل از این روش یعنی  $T$  با درخت پوشای کمینه یعنی  $T'$  حداقل یک یال مشترک دارد، سبک‌ترین یالی که  $A$  را به  $B$  متصل می‌کند؛ پس گزینه ۴ نادرست است.

۳- گزینه (۴). گراف این سوال به شکل مقابل است:



این گراف با شروع از ۱ فقط پیمایش‌های عمقی گفته شده را دارد. حال با شروع از ۵ پیمایش‌های عمقی این گراف  $\langle 5, 3, 4, 2, 1 \rangle$  و  $\langle 5, 3, 1, 2, 4 \rangle$  می‌باشد.

- ۴- گزینه (۴). الگوریتم گفته شده پس از تعداد متناهی مرحله متوقف می‌شود و در پایان به ازای هر گره  $u$ ،  $d(u)$  طول کوتاه‌ترین مسیر از  $s$  به  $u$  است. گزینه ۲ هم غلط نیست ولی گزینه ۴ کامل است.
- ۵- گزینه (۱). حداکثر شار برابر ظرفیت برش کمینه است. برش کمینه در این گراف،  $\{s, a, d\}, \{b, c, t\}$  است که یالهای این برش  $de$  و  $ab$  هستند که جمع ظرفیت این یالها ۲ است و این برابر حداکثر شار است.

### پاسخنامه مهندسی فناوری اطلاعات ۱۳۹۶

- ۶- گزینه (۲). توجه کنید دایکسترا برای گرافهای با وزن مثبت جواب درست می‌دهد و برای گرافهای دارای یال منفی و بدون سیکل منفی ممکن است جواب درست بدهد یا ندهد. مثلاً برای گراف فقط با یک یال، وزن یال هر چه باشد، دایکسترا جواب درست می‌دهد.
- ۷- گزینه (۱). هر سه گزاره نادرست هستند. برای نقض گزاره اول، فرض کنید  $A = abt$  و  $B = btda$  و  $C = mna$  در این صورت  $LCS(A, B) = bt$  و  $LCS(A, B, C) = a$  و  $LCS(LCS(A, B), C) = LCS(bt, mna) = NULL$  برای نقض گزاره دوم همین سه رشته را در نظر بگیرید:

$$LCS(LCS(A, B), C) = LCS(bt, mna) = NULL$$

$$LCS(A, LCS(B, C)) = LCS(abt, a) = a$$

گزاره سوم نیز به وضوح نادرست است. مثلاً  $LCS(abt, tba) \neq \emptyset$ .

- ۸- گزینه (۴). برای نقض گزینه ۴، گراف  $e \text{---} d \text{---} a \text{---} b \text{---} c$  را در نظر بگیرید که قطر آن ۴ است و حال اگر با شروع از  $a$  پیمایش BFS انجام دهیم، آنگاه درخت حاصل، سه سطحی (ارتفاع ۲) خواهد شد. سایر گزینه‌ها خاصیت BFS هستند و به راحتی قابل بررسی هستند.

- ۹- گزینه (۲).  $d_{uv}^k$  برابر طول کوتاهترین مسیر از  $u$  به  $v$  است که رئوس  $\{1, 2, \dots, k\}$  در آن باشد. در واقع برای یافتن کوتاهترین مسیر از  $u$  به  $v$  که رئوس  $\{1, 2, \dots, k\}$  در آن باشد، باید بین کوتاهترین مسیر از  $u$  به  $v$  که رئوس  $\{1, 2, \dots, k-1\}$  در آن باشد و کوتاهترین مسیر از  $u$  به  $k$  بعلاوه  $k$  به  $v$  مینیمم گرفته شود.

- ۱۰- گزینه (۴). هیچ شرط لازم و کافی برای یکتا بودن MST وجود ندارد. توجه کنید اگر وزن‌ها متمایز باشند آنگاه MST منحصر به فرد است ولی عکس این مطلب درست نیست.
- ۱۱- گزینه (۴). طول بیش‌ترین کد برای  $n$  کاراکتر  $n-1$  بیت می‌باشد که در درس آموخته‌اید.

## [پاسخنامه علوم کامپیوتر ۱۳۹۶]

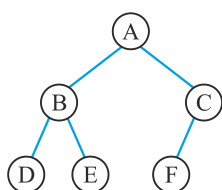
۱۲- گزینه (۲). روش ۱: عدد ۹ باید ریشه باشد از ۸ عدد باقی مانده باید ۳ عدد برای زیردرخت

چپ (یا ۵ عدد برای زیردرخت راست) انتخاب شود که  $\binom{8}{3}$  (یا  $\binom{8}{5}$  حالت دارد) سپس در زیردرخت چپ و راست همین عمل انجام شود:

$$\binom{8}{3} \times \binom{2}{1} \times \binom{4}{1} \times \binom{2}{1} = ۸۹۶$$

روش ۲: برای هر نود اگر تعداد کل نودهای زیردرختان آن  $x$  باشد که از این تعداد  $y$  تا نود در زیردرخت چپ و  $z$  تا نود در زیردرخت راست باشند عامل  $\frac{x!}{y!z!}$  را در جواب ضرب می کنیم:

$$\frac{8!}{3!5!} \times \frac{2!}{1!1!} \times \frac{4!}{1!3!} \times \frac{2!}{1!1!} = ۸۹۶$$



Pre : ABDECF

۱۳- گزینه (۲). روش معمول این است که پیمایش صورت سؤال را با پیمایشی که در گزینه ها داده شده با هم در نظر بگیریم و سعی کنیم درخت بکشیم. ولی می توان با آزمایش و خطا به جواب رسید. مثلاً فرض کنیم درخت کامل است و پیمایش پس ترتیب را در آن قرار دهیم و سپس پیش ترتیب پیمایش کنیم:

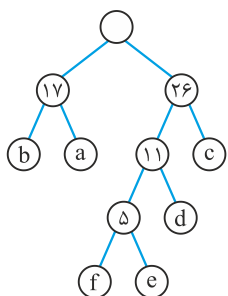
۱۴- گزینه (۳). رابطه بازگشتی زمان اجرا  $T(n) = T(\frac{n}{10}) + T(\frac{9n}{10}) + \theta(n)$  است که از مرتبه  $\theta(n \lg n)$  است.

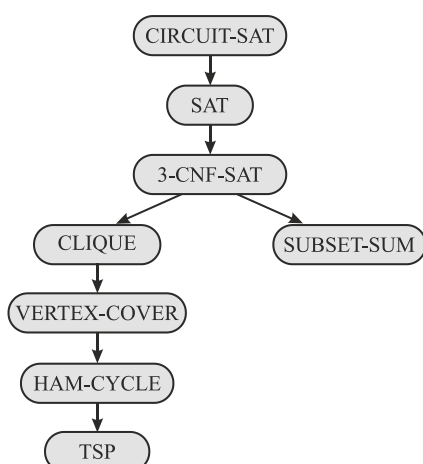
۱۵- گزینه (۴). زمان سرشکنی برابر میانگین هزینه  $n$  عمل در بدترین حالت است پس برابر  $\frac{\theta(n \lg n)}{n}$  یعنی  $\theta(\lg n)$  است. بدترین حالت در این تست مشخص نیست و اگر جمع  $n$  عمل از مرتبه  $\theta(n \lg n)$  است. ممکن است همگی  $\theta(\lg n)$  بوده اند یا ممکن است مثلاً یکی از عملیات  $\theta(n \lg n)$  و بقیه  $\theta(1)$  بوده اند. پس گزینه های ۳ و ۴ صحیح هستند. کلید گزینه ۴ است.

۱۶- گزینه (۲). با برنامه نویسی پویا و با مرتبه  $\theta(n)$  این مسئله قابل حل است. در کتاب نرنجی پوران پژوهش این سؤال کامل بررسی شده است.

۱۷- گزینه (۱). می توان میانه را با مرتبه  $O(n)$  یافته و سپس با مقایسه عناصر با میانه یا عنصر مورد نظر پیدا می شود یا متوجه می شویم چنین عنصری وجود ندارد.

- ۱۸- پاسخ صحیح وجود ندارد. اگر الگوریتم را روی درخت  انجام دهید خروجی  $acb <$  خواهد بود که برعکس پس‌ترتیب است. کلید گزینه (۴) است.
- ۱۹- گزینه (۲). این کد به صورت بازگشتی لیست را معکوس می‌کند.
- ۲۰- گزینه (۴). اعداد فیبوناچی متمایز هستند به جز دو تا عدد کمینه. از آنجایی که دو تا عدد کمینه حتماً در MST وجود دارند پس MST منحصر به فرد است.
- ۲۱- گزینه (۳). هزینه جستجو در BST برابر  $O(n)$  است. ارتفاع BST برابر  $O(n)$  است.
- ۲۲- گزینه (۱). تعداد کلیدهای داده شده ۱۷ تاست و هرم درخت کامل است. با ۱۷ کلید تعداد سطوح درخت کامل برابر ۵ است پس ارتفاع برابر ۴ است که در گزینه‌ها نیست. در این تست طراح، ارتفاع را برابر تعداد سطوح تعریف کرده است.
- ۲۳- گزینه (۱). در صورت سؤال باید گفته شود که  $n$  توانی از ۲ است و نه زوج! در این صورت گزینه (۱) صحیح است که در درس دیده‌ایم.
- ۲۴- گزینه (۱). با الگوریتم‌های DFS و BFS می‌توان از گراف همبند غیرجهت‌دار، یک درخت پوشا استخراج کرد که مرتبه آن  $O(n+e)$  است ولی چون گراف همبند است، مرتبه  $O(e)$  است.

- ۲۵- گزینه (۲).  $a$  و  $b$  دو بیتی،  $d$  سه بیتی،  $f$  و  $e$  چهار بیتی می‌شوند:
- 
- $$102 = (10 + 7 + 15) \times 2 + 6 \times 3 + (3 + 2) \times 4 = \text{هزینه}$$



- ۲۶- همه گزینه‌ها صحیح هستند. البته با فرض اینکه نسخه تصمیم‌گیری مسائل TSP و Vertex Cover و Clique مدنظر باشد. این تست به طرز ناشیانه‌ای از یکی از مباحث کتاب CLRS استنباط شده است. در آن کتاب درختی کشیده شده که نشان می‌دهد چطور مسائل NPC اثبات شده‌اند. در این کتاب ترتیب تبدیل مسائل به هم مطابق این درخت است که گزینه ۳ کلید طراح است. ولی با توجه به مفهوم  $A \leq_p B$  که یعنی  $A$  از  $B$  سخت‌تر نیست همه گزینه‌ها صحیح هستند.

## [سؤال‌های چهار گزینه‌ای مهندسی کامپیوتر ۱۳۹۷]

- ۱- یک درخت دودویی جست‌وجو شامل  $n$  عدد و ارتفاع  $O(\log n)$  در اختیار داریم. به ازای هر گره در درخت فوق تعداد نوادگان آن گره به عنوان اطلاعات اضافه، ذخیره شده است. کدام مورد را در زمان  $O(\log n)$  نمی‌توان پاسخ داد؟
- (۱) میانگین اعداد ذخیره شده در درخت که در بازه داده شده  $[a, b]$  قرار دارند.
  - (۲) تعداد اعداد کوچک‌تر از عدد داده شده  $a$
  - (۳) تعداد اعداد ذخیره شده در درخت که در بازه داده شده  $[a, b]$  قرار دارند.
  - (۴) میانه اعداد ذخیره شده در درخت که در بازه داده شده  $[a, b]$  قرار دارند.
- ۲- آرایه یک بعدی  $A$ ، شامل  $n$  عدد صفر و یک است. اگر به ازای هر صفر، اولین یک سمت چپ (با اندیس کمتر) و به ازای هر یک، اولین صفر سمت چپ آن را پیدا کنیم، هزینه سرشکن این محاسبه برای هر عدد، کدام است؟ (بهترین پاسخ را انتخاب کنید).
- (۱)  $O(\log n)$
  - (۲)  $O(\log \log n)$
  - (۳)  $O(1)$
  - (۴)  $O(n)$
- ۳- جواب رابطه بازگشتی  $T(n) = T(\sqrt{n}) + O(\log \log n)$ ، کدام است؟
- (۱)  $O(\log \log n)$
  - (۲)  $O(\log^2 \log n)$
  - (۳)  $O(\log n)$
  - (۴)  $O(\log^2 n)$
- ۴- آرایه  $A$  از  $n$  عدد دلخواه متمایز تشکیل شده و  $k$  یک عدد از پیش مشخص است. فرض کنید عملیات  $\text{sort}(i)$  به ازای  $1 \leq i \leq n - k + 1$ ، زیرآرایه  $A[i..i+k-1]$  را مرتب می‌کند. در بدترین حالت چند عملیات  $\text{sort}$  برای مرتب کردن آرایه  $A$  لازم است؟ (بهترین پاسخ را انتخاب کنید).
- (۱)  $O(n \log_k n)$
  - (۲)  $O(n^2 / k)$
  - (۳)  $O(n^2 / k^2)$
  - (۴)  $O(n \log n)$
- ۵- یک جدول در هم‌ساز داریم. فرض کنید برای رفع مشکل تصادم از روش واریسی خطی استفاده شده است. با در نظر گرفتن فرض یکنواختی تابع در هم‌ساز، کلید بعدی با چه احتمالی در خانه دوم قرار می‌گیرد؟ (خانه‌های جدول از چپ به راست از ۱ تا ۱۸ شماره‌گذاری شده‌اند).

|   |  |   |  |  |  |    |  |   |   |  |    |  |   |  |   |   |   |
|---|--|---|--|--|--|----|--|---|---|--|----|--|---|--|---|---|---|
| ۵ |  | ۷ |  |  |  | ۱۱ |  | ۲ | ۹ |  | ۱۴ |  | ۳ |  | ۱ | ۴ | ۶ |
|---|--|---|--|--|--|----|--|---|---|--|----|--|---|--|---|---|---|

$$\frac{1}{18} \quad (۴)$$

$$\frac{5}{18} \quad (۳)$$

$$\frac{1}{18} \quad (۲)$$

$$\frac{1}{18} \quad (۱)$$

۶- آرایه  $A$  شامل  $n$  عدد داده شده است. همچنین یک جعبه سیاه داریم که به عنوان ورودی یک زیرمجموعه  $S \subseteq \{1, 2, \dots, n\}$  با اندازه حداکثر  $k$  و یک عدد  $x$  را به عنوان ورودی می‌گیرد و اگر عدد  $i \in S$  وجود داشت طوری که  $A[i] = x$ ، مقدار یک را برمی‌گرداند و در غیر این صورت صفر برمی‌گرداند. با چند مرتبه استفاده از این جعبه سیاه می‌توانیم به ازای یک عدد دلخواه  $x$  در صورت وجود، اندیس  $y$  را که  $A[y] = x$  است پیدا کنیم؟ (بهترین پاسخ را انتخاب کنید).

- (۱)  $O(n/k + \log k)$  (۲)  $O(n)$   
(۳)  $O(n/k)$  (۴)  $O(\log n)$

۷- گراف وزن دار و همبند  $G$  را در نظر بگیرید (وزن‌ها مثبت هستند). وزن یک مسیر ساده (بدون رأس تکراری) در گراف را برابر وزن یالی که در مسیر کمترین وزن را دارد، تعریف می‌کنیم. در الگوریتم‌های بلمن - فورد و دایکسترا،  $d[u]$  برابر سبک‌ترین مسیر ساده به دست آمده تاکنون از مبدأ در نظر گرفته می‌شود. اگر در این الگوریتم‌ها به ازای یال  $(u, v)$  با وزن  $w(u, v)$  به روز رسانی را به این شکل تغییر دهیم که  $d[v] = \min(w(u, v), d(u))$ ، کدام یک از دو الگوریتم فوق با تغییر انجام شده، همیشه درست کار می‌کند؟ (مقدار اولیه  $d(u)$  در هر دو الگوریتم برابر مثبت بی‌نهایت قرار داده می‌شود).

- (۱) فقط الگوریتم بلمن - فورد (۲) هر دو الگوریتم  
(۳) فقط الگوریتم دایکسترا (۴) هیچ یک از این دو الگوریتم

۸- گراف وزن دار، همبند و بدون جهت  $G = (V, E)$  را در نظر بگیرید. الگوریتم زیر را روی  $G$  اجرا می‌کنیم.

در ابتدا  $M = (V, \{\})$  قرار می‌دهیم. سپس یال‌های  $G$  را به ترتیب دلخواه در  $M$  درج می‌کنیم. بعد از درج هر یال، اگر  $M$  دارای دور بود، به ازای هر دور در  $M$  سنگین‌ترین یال آن دور را حذف می‌کنیم. کدام گزاره‌ها درست هستند؟

(a)  $M$  همیشه برابر درخت پوشای کمینه  $G$  است.

(b) اگر یال‌ها به ترتیب وزن (از کوچک به بزرگ) درج شوند،  $M$  حتماً درخت پوشای کمینه خواهد بود.

- (۱)  $a$  نادرست،  $b$  نادرست (۲)  $a$  درست،  $b$  درست  
(۳)  $a$  نادرست،  $b$  درست (۴)  $a$  نادرست،  $b$  درست

۹- برای دنباله  $X = \langle x_1, \dots, x_n \rangle$  متشکل از اعداد متمایز، فرض کنید  $LIS(X)$  بزرگ‌ترین زیردنباله صعودی  $X$  و  $LIS(X, a)$  بزرگ‌ترین زیردنباله صعودی  $X$  که عنصر آخر آن حداکثر  $a$  می‌باشد. چه تعداد از گزاره‌های زیر درست هستند؟

(در زیر  $X_i = \langle x_1, \dots, x_i \rangle$  و عملگر  $\max$  دنباله با طول بزرگ‌تر را برمی‌گرداند.)

- $LIS(X_n) = \max_{i=1}^n (< LIS(X_{i-1}, x_i), x_i >)$
- $LIS(X_n) = < LIS(X_{n-1}, x_n), x_n >$
- $LIS(X_n) = \max(< LIS(X_{n-1}), < LIS(X_{n-1}, x_n), x_n >)$

۳ (۱)      ۰ (۲)      ۱ (۳)      ۲ (۴)

۱۰- فرض کنید برای ساخت درخت کد هافمن از الگوریتم زیر استفاده کنیم. حروف الفبا را به دو دسته  $A$  و  $B$  به گونه‌ای افراز می‌کنیم که اختلاف تعداد تکرارهای حروف الفبا در  $A$  و  $B$  کمینه شود. به طور بازگشتی درخت کد هافمن را برای هر یک از این دو دسته می‌سازیم. سپس دو درخت به دست آمده برای  $A$  و  $B$  را به عنوان زیردرخت‌های ریشه قرار می‌دهیم. (اگر تعداد حروف الفبا ۱ باشد، درخت کد هافمن تک رأسی است.) اگر  $n$  تعداد حروف الفبا باشد، کوچک‌ترین مقدار  $n$  که برای آن الگوریتم فوق درخت بهینه را تولید نمی‌کند، کدام است؟

۵ (۱)      ۲ (۲)      ۳ (۳)      ۴ (۴)

### سؤال‌های چهارگزینه‌ای مهندسی فناوری اطلاعات ۱۳۹۷

۱۱- جواب رابطه بازگشتی  $T(n) = 2T\left(\frac{n}{4}\right) + \log n$ ، کدام است؟

(۱)  $O(\sqrt{n})$       (۲)  $O(\log n)$       (۳)  $O(\log^2 n)$       (۴)  $O(\sqrt{n} \log n)$

۱۲- فرض کنید  $n$  عدد داخل آرایه  $A$ ، به صورتی قرار گرفته است که به ازای هر اندیس  $i \leq n - k$  داریم  $A[i] > A[i + k]$  ( $k$  مقداری ثابت است). این آرایه را در چه زمانی می‌توان مرتب کرد؟ (بهترین پاسخ را انتخاب کنید.)

(۱)  $O(n)$       (۲)  $O(n \log k)$       (۳)  $O(n \log n)$       (۴)  $O(n \log_k n)$

۱۳- یک داده ساختار در اختیار داریم که از یک هرم کمینه و یک پشته تشکیل شده است. در ابتدا هر دو خالی هستند. در هر مرحله یکی از کارهای زیر را می‌توان انجام داد.

- یک عدد از ورودی خواند و آن را داخل هرم کیمنه ریخت.
- کوچک‌ترین عدد را از هرم کمینه استخراج کرد و داخل پشته push کرد.
- عمل pop را روی پشته اجرا و به عنوان خروجی گزارش کرد.

اگر ورودی از چپ به راست ۱، ۶، ۴، ۵، ۲، ۳ باشد، کدام خروجی (به ترتیب از چپ به راست) را نمی‌توان تولید کرد؟

(۱) هر جایگشت از اعداد ۱ تا ۶ را می‌توان تولید کرد.

(۲) ۱، ۲، ۳، ۴، ۵، ۶

(۳) ۴، ۵، ۳، ۲، ۱، ۶

(۴) ۱، ۲، ۳، ۴، ۵، ۶

۱۴- کدام تابع درهم‌ساز داده شده  $h(x)$ ، یکنواخت است؟ (برد توابع اعداد طبیعی است).

(۱)  $2^x \bmod 7$  (۲)  $2^x \bmod 11$  (۳)  $x^2 \bmod 7$  (۴)  $x^2 \bmod 11$

۱۵- یک آرایه با اندازه  $n$  داریم که هر خانه آن به یک گره دلخواه از یک لیست پیوندی دوسویه اشاره می‌کند. هیچ دو خانه‌ای از آرایه به یک گره اشاره نمی‌کند. لیست پیوندی دوسویه دقیقاً شامل  $n$  گره است و هر گره یک عدد متمایز در خود نگه داشته است. لیست پیوندی براساس این اعداد به صورت صعودی مرتب شده می‌باشد. (یعنی اگر لیست را از ابتدا تا انتها پیمایش کنیم اعداد به صورت صعودی مشاهده می‌شوند). آیا عدد داده شده  $x$  در لیست پیوندی دوسویه موجود است و در بدترین حالت با چه مرتبه زمانی می‌توان این موضوع را تشخیص داد؟

(۱)  $O(1)$  (۲)  $O(n)$  (۳)  $O(\sqrt{n})$  (۴)  $O(\log n)$

۱۶- طول متن uselessness، با کدگذاری هافمن چند بیت می‌شود؟

(۱) ۲۰ (۲) ۲۲ (۳) ۲۵ (۴) ۲۸

۱۷- در یک گراف جهت‌دار و وزن‌دار (با وزن‌های مثبت) برای محاسبه کوتاه‌ترین مسیر از مبدأ داده شده به بقیه رأس‌ها از الگوریتم دایکسترا استفاده شده است. کدام مورد، همیشه درست است؟

منظور از relax یال  $(u, v)$  در الگوریتم دایکسترا تابع زیر است، که در آن  $w(u, v)$  وزن یال  $(u, v)$  است.

$\text{relax}(u, v)$ :

if  $d[v] > d[u] + w(u, v)$  then  $d[v] = d[u] + w(u, v)$

(۱) هر یال دقیقاً یک بار relax می‌شود.

(۲) هر یال حداکثر یک بار relax می‌شود.

(۳) ممکن است بعضی از یال‌ها بیش از یک بار relax شوند.

(۴) مثالی وجود دارد که بعضی از یال‌ها بیش از یک بار relax می‌شوند، اما به طور متوسط تعداد relax ها  $O(1)$  است.



۱۸- در آرایه مرتب شده A تمام عناصر به جز یک عنصر که تنها یک بار ظاهر شده است، دقیقاً دو بار ظاهر شده‌اند. عنصری که تنها یک بار ظاهر شده را در چه زمانی می‌توان به دست آورد؟

$$O(1) \quad (1) \quad O(n) \quad (2) \quad O(\log n) \quad (3) \quad O\left(\frac{n}{\log n}\right) \quad (4)$$

۱۹- برای پیدا کردن کوتاه‌ترین مسیر بین تمام رأس‌های یک گراف وزن دار تنک (که تعداد یال‌های آن از مرتبه تعداد رأس‌ها است)، استفاده از کدام الگوریتم از نظر مرتبه زمانی بهتر است؟ (وزن یال‌ها می‌تواند منفی هم باشد).

(۱) الگوریتم جانسون

(۲) الگوریتم فلویید - وارشال

(۳) n بار اجرای الگوریتم دایکسترا برای هر رأس

(۴) n بار اجرای الگوریتم بلمن - فورد برای هر رأس

۲۰- یک برنامه‌نویس تازه‌کار، تابع زیر را برای محاسبه  $\binom{n}{k}$  نوشته است. مدیر شرکت به عنوان

تنبیه دستور داده است که این برنامه‌نویس تعداد بارهایی که این الگوریتم خودش را برای

محاسبه  $\binom{20}{3}$  صدا می‌زند را به صورت دستی محاسبه کند. این تعداد چند مرتبه است؟

```
int comb (int n, int k) {
    if ((k = 0) or (n = k))
        return 1
    else
        return comb (n - 1, k - 1) + comb (n - 1, k)
}
```

$$1939 \quad (1) \quad 1957 \quad (2) \quad 2279 \quad (3) \quad 2291 \quad (4)$$

۲۱- برای محاسبه شار بیشینه در شبکه داده شده با n رأس و m یال و ظرفیت‌های صحیح، دو

الگوریتم با زمان‌های (i)  $O(mnC)$  و (ii)  $O(m^2 \log C)$  وجود دارد که C بیشترین

ظرفیت یال‌ها است. (زمان اجرای کدام الگوریتم بر حسب اندازه ورودی، چندجمله‌ای است؟

$$(1) \text{ و } (ii) \quad (2) \text{ فقط } (i) \quad (3) \text{ فقط } (ii) \quad (4) \text{ هیچ کدام}$$

۲۲- فرض کنید یک گراف جهت‌دار G داده شده است. الگوریتم جستجوی عمق اول (DFS) را

بر روی گراف G اجرا می‌کنیم تا همه رئوس گراف را ملاقات کنیم. برای هر گره v،  $S_v$  و

$f_v$  را به ترتیب زمان قرار گرفتن  $v$  در پشته و زمان خارج شدن از پشته تعریف می‌کنیم. گراف  $G'$  را گرافی در نظر بگیرید که هر گره آن معادل یک مؤلفهٔ ماکزیمال همبند قوی از  $G$  است. از  $u'$  به  $v'$  در  $G'$  یال جهت‌دار وجود دارد. اگر از یک رأس از مؤلفه همبند معادل  $u'$  به یک رأس مؤلفهٔ همبند معادل  $v'$  یال وجود داشته باشد، برای هر گره  $u$  در  $G$ ، فرض کنید  $u'$  گره‌ای در  $G'$  باشد که معادل مؤلفهٔ همبندی است که  $u$  متعلق به آن است. کدام مورد درست است؟

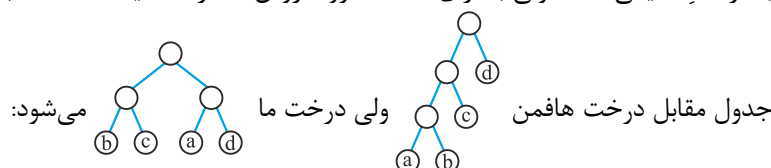
- (۱) اگر  $u$  رأسی با بیشترین  $s_u$  باشد،  $u'$  ورودی ندارد.
- (۲) اگر  $u$  رأسی با بیشترین  $s_u$  باشد،  $u'$  ورودی ندارد.
- (۳) اگر  $u$  رأسی با بیشترین  $f_u$  باشد،  $u'$  خروجی ندارد.
- (۴) اگر  $u$  رأسی با بیشترین  $f_u$  باشد،  $u'$  خروجی ندارد.

## پاسخنامه مهندسی کامپیوتر ۱۳۹۷

- ۱- گزینه (۱). درخت توصیف شده درخت مرتبه آماری است که می‌توان مرتبه  $a$  و  $b$  را با زمان  $O(\lg n)$  یافت پس می‌توان میانه  $a$  و  $b$  و همچنین تعداد عناصر بین  $a$  و  $b$  را نیز با زمان  $O(\lg n)$  یافت. ولی برای میانگین باید اعداد  $[a, b]$  با هم جمع شوند که این عمل  $O(n)$  است چون ممکن است مثلاً مجبور باشیم همه  $n$  عنصر درخت را جمع کنیم (اگر  $a = \min$  و  $b = \max$  باشد)
- ۲- گزینه (۳). از سمت چپ یک بررسی خطی انجام دهیم و موقعیت اولین ۱ و اولین ۰ را بیابیم سپس به ازای هر بیت، فقط باید یکی از دو موقعیتی را که یافته‌ایم گزارش کنیم. بررسی خطی  $O(n)$  است پس به ازای  $n$  بیت هزینه سرشکنی  $O(1)$  می‌شود.
- ۳- گزینه (۲).  $n = 2^m \rightarrow T(2^m) = T(2^{\frac{m}{2}}) + \log m$
- $$\rightarrow F(m) = F\left(\frac{M}{2}\right) + \log m = O(\log^2 m) \Rightarrow T(n) = O(\log^2 \lg n)$$
- ۴- گزینه (۳). با هر فراخوانی تابع داده شده  $k$  تا از اعداد آرایه سورت می‌شوند یعنی  $\frac{k(k-1)}{2}$  از وارونگی‌ها اصلاح می‌شود و چون باید حداکثر  $\frac{n(n-1)}{2}$  وارونگی اصلاح شود پس تعداد فراخوانی تابع داده شده برابر است با:
- $$\frac{\frac{n(n-1)}{2}}{\frac{k(k-1)}{2}} = O\left(\frac{n^2}{k^2}\right)$$
- ۵- گزینه (۳). برای آنکه عنصر جدید به خانه ۲ وارد شود یا باید با ۱ و ۴ و ۶ و ۵ برخورد کند یا مستقیماً به خانه ۲ وارد شود که ۵ حالت دارد. پس احتمال مورد نظر  $\frac{5}{18}$  است.
- ۶- گزینه (۴). با فرض اینکه مقدار  $k$  را خودمان مشخص می‌کنیم و به ما تحمیل نمی‌شود، ابتدا  $k$  را برابر  $n$  قرار می‌دهیم و جعبه سیاه را اجرا می‌کنیم که بهفهمیم آیا عنصر  $x$  اصلاً در آرایه هست یا نه. اگر بود،  $k$  را برابر  $\frac{n}{2}$  قرار می‌دهیم و نیمه اول  $A$  را بررسی می‌کنیم که آیا  $x$  در آنجا هست اگر بود مجدد  $k$  را نصف می‌کنیم تا به اندیس عنصر  $x$  برسیم. مرتبه این عملیات  $O(\lg n)$  است.

- ۷- گزینه (۱ یا ۴). سؤال ابهام دارد اگر سؤال این است که می‌خواهیم کوتاهترین مسیر را بیابیم هیچ یک از دو الگوریتم گفته شده با این تغییر جواب نمی‌دهند. ولی اگر هدف این است که در مسیر بین دو رأس کم وزن‌ترین یال را بیابیم آنگاه فقط بلمن فورد جواب می‌دهد.
- ۸- گزینه (۲). هر دو گزاره صحیح هستند، زیرا اولاً خروجی این داستان حتماً همبند است، چون در هر دور یک یال حذف می‌شود. ثانیاً کمینه است چون در هر دور یال بیشینه حذف می‌شود.
- ۹- گزینه (۴). با روش پویا گزاره‌های اول و سوم درست هستند. مثلاً گزاره سوم می‌گوید برای یافتن  $LIS(X_n)$  دو حالت وجود دارد یا کاراکتر آخر یعنی  $x_n$  در  $LIS$  هست یا نیست. اگر نبود که جواب  $LIS(X_{n-1})$  است و اگر بود کافی است  $x_n$  را حذف کنیم و  $LIS(X_{n-1})$  که کاراکترهای آن از  $x_n$  بزرگتر نیستند را بیابیم و سپس  $x_n$  را به انتهای آنها الحاق کنیم.

- ۱۰- گزینه (۴). به ازای  $n=2$  به وضوح درست است چون هافمن و روش گفته شده هر دو درخت  را تولید می‌کنند. به ازای  $n=3$  فرض کنید  $a < b < c$  آنگاه حتماً  $A = \{a, b\}$  و  $B = \{c\}$  زیرا  $a + b$  با  $c$  کمترین اختلاف را دارد و در نتیجه درخت هافمن با درخت ما یکی است. ولی به ازای  $n=4$  لزوماً روش ما درست نیست. مثلاً با توجه به



| نویسه   | a | b  | c  | d  |
|---------|---|----|----|----|
| فراوانی | ۹ | ۲۵ | ۲۵ | ۴۱ |

$A = \{b, c\}, B = \{a, d\}$

## پاسخنامه مهندسی فناوری اطلاعات ۱۳۹۷

- ۱۱- گزینه (۱). با توجه به قضیهٔ مستر،  $n^{\log_2 \sqrt{n}} = \sqrt{n}$  که رشد آن به صورت غیرلگاریتمی از  $\lg n$  بیشتر است پس  $T(n) = O(\sqrt{n})$ .
- ۱۲- گزینه (۲). به چنین آرایه‌ای  $k$  مرتب گویند (البته در آرایهٔ  $k$  مرتب به ازای هر  $i$  رابطهٔ  $A[i] \leq A[i+k]$  درست است که نوع مسئله تفاوتی نمی‌کند) و می‌دانیم مرتب کردن یک آرایهٔ  $k$  مرتب یعنی ادغام  $k$  تا لیست مرتب  $\frac{n}{k}$  عنصری که از مرتبهٔ  $O(n \lg k)$  است.
- ۱۳- گزینه (۳). برای تولید گزینه ۲ ابتدا تمام اعداد ورودی را در هرم درج می‌کنیم و سپس از کوچک به بزرگ در پشته پوش می‌کنیم و سپس از بزرگ به کوچک پاپ می‌کنیم. گزینه ۴

نیز به این شکل تولید می‌شود که ابتدا تمام عناصر ورودی را در هرم درج کنیم سپس به ترتیب هر عنصر را از هرم حذف و در پشته پوش و از پشته پاپ کنیم. یعنی ابتدا ۱ از هرم حذف و در پشته پوش از پشته پاپ شود و بعد ۲ و به همین ترتیب. اما گزینه ۳ قابل تولید نیست زیرا برای آنکه عدد ۶ را در خروجی ظاهر کنیم باید قبل از آن ۲ و ۳ و ۴ و ۵ در پشته قرار گرفته باشند که امکان ندارد این چهار عنصر به ترتیب در پشته باشند، از چپ به راست این چهار عنصر می‌توانند به شکل‌های زیر در پشته باشند (عدد اول عدد پایین پشته است)

۳۲۵۴ و ۲۳۴۵ و ۲۳۵۴ و ...

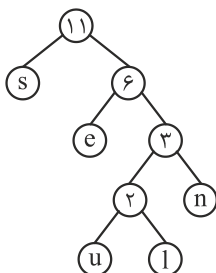
و حالت ۴۵۳۲ ممکن نیست.

۱۴- گزینه (۲). گزینه‌های ۳ و ۴ قطعاً یکنوا نیستند به طور کلی  $x^2 \bmod k$  یکنوا نیست. مثلاً در گزینه ۳، می‌گوییم هر عددی به یکی از شکل‌های  $7t$ ،  $7t \pm 1$  و  $7t \pm 2$  و  $7t \pm 3$  است. کلیدهای به شکل  $7t$  به حفره صفر، کلیدهای به شکل  $7t \pm 1$  به حفره ۱، کلیدهای به شکل  $7t \pm 2$  به حفره ۲ و کلیدهای به شکل  $7t \pm 3$  به حفره ۳ وارد می‌شوند و مثلاً آدرس حفره‌های ۳ و ۵ و ۶ تولید نمی‌شود. همچنین در گزینه ۴ آدرس حفره‌های ۲ و ۶ و ۷ و ۸ تولید نمی‌شود.

برای بررسی گزینه ۱ کلیدهای ۰ و ۱ و ۲ و ۳ و ۴ و ۵ و ۶ را در نظر بگیرید. آدرس تولید شده برای اینها به ترتیب ۱ و ۲ و ۴ و ۱ و ۲ و ۴ و ۱ و ۴ و ۱ می‌باشد یعنی اصلاً آدرس حفره‌های ۰ و ۳ و ۵ و ۶ تولید نمی‌شود. در گزینه ۲ اگر کلیدهای متوالی ۰ تا ۱۰ را به تابع بدهیم آدرس‌های ۱ و ۲ و ۴ و ۸ و ۵ و ۱۰ و ۹ و ۷ و ۳ و ۶ و ۱ تولید می‌شود که فقط آدرس صفر تولید نمی‌شود. که البته با توجه به صورت سؤال که گفته بُرد طبیعی است و اگر اعداد طبیعی را  $\{1, 2, 3, \dots\}$  فرض کنیم این گزینه درست است.

۱۵- گزینه (۲). جستجوی کلید  $x$  در لیست پیوندی از هر نوعی از مرتبه  $O(n)$  است و وجود آرایه گفته شده حل مسئله را سریعتر نمی‌کند.

۱۶- گزینه (۲). در این سؤال درخت هافمن منحصر به فرد نیست. ولی هزینه همگی یکسان و کمینه است. یکی از درخت‌ها به شکل زیر است:



که مجموع هزینه آن برابر است با:

$$(1+1) \times 4 + 1 \times 3 + 3 \times 2 + 5 \times 1 = 22$$

۱۷- گزینه (۱ و ۲). در دایجسترا اگر صف اولویت خالی شود، هر یال دقیقاً یکبار relax می‌شود ولی اگر یک رأس در صف بماند، برخی یالها relax نمی‌شوند. در کلید نهایی گزینه‌های ۱ و ۲ درست اعلام شدند.

۱۸- گزینه (۳). کافی است عنصر وسط را با دو عنصر چپ و راست آن مقایسه کنیم اگر عنصر وسط یکتا باشد کار تمام است وگرنه در سمتی که تعداد عناصر فرد است (عنصر وسط با جفتش را نادیده می‌گیریم) فراخوانی بازگشتی می‌کنیم. پس مرتبه زمانی  $O(\lg n)$  است.

۱۹- گزینه (۱). برای گراف خلوت از بین الگوریتم‌های فلویدوارشال، شبه ضرب ماتریسی و جانسون بهترین روش جانسون است. یادآوری می‌کنیم جانسون ابتدا وزن یالها را با روش هوشمندانه‌ای نامنفی می‌کند و سپس  $n$  بار دایجسترا می‌زند که مرتبه آن  $O(n(e + n \lg n))$  است (برای دایجسترا از صف اولویت با هیپ فیبوناچی استفاده شده) می‌باشد که برای گراف خلوت  $O(n^2 \lg n)$  می‌باشد.

۲۰- گزینه (۳). در محاسبه  $\binom{n}{k}$  با تقسیم و غلبه تعداد عمل جمع  $\binom{n}{k} - 1$  و تعداد فراخوانی

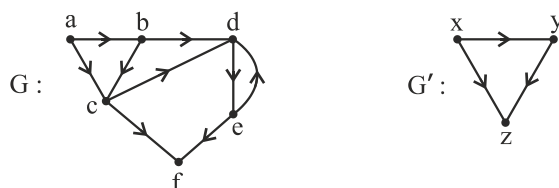
$$2^{\binom{n}{k}} - 1 \text{ می‌باشد که برابر است با:}$$

$$2^{\binom{20}{3}} - 1 = 2280.$$

ولی چون سؤال گفته الگوریتم چند بار خودش را صدا می‌کند یعنی فراخوانی اولیه مدنظر نیست پس جواب ۲۲۷۹ است.

۲۱- کلید سؤال گزینه ۳ است. ولی اگر  $C$  برابر  $2^{2^n}$  باشد آنگاه قسمت (ii) نیز نمایی می‌شود.

۲۲- گزینه (۳).



گراف  $G'$  قطعاً dag می‌باشد. در گراف  $G$  شما از هر رأسی شروع کنید و پیمایش عمقی انجام دهید حتماً زمان پایان حداقل یکی از رئوس  $a$  و  $b$  و  $c$  از بقیه بیشتر است، (چرا؟) پس  $x$  که معادل  $a$  و  $b$  و  $c$  است ورودی ندارد.

## سؤال‌های چهار گزینه‌ای مهندسی کامپیوتر ۱۳۹۸

۱- پاسخ رابطه بازگشتی زیر کدام است؟

$$T(n) = T\left(\frac{n}{\log n}\right) + O(1)$$

- (۱)  $O(\log n)$  (۲)  $O(\log \log n)$   
(۳)  $O(\log n / \log \log n)$  (۴)  $O(\log n / (\log \log n)^2)$

۲- فرض کنید یک درخت دودویی جستجو با  $n$  گره داریم. به ازای گره  $u$  از این درخت، وزن آن را تعداد گره‌ها در زیر درخت به ریشه  $u$  (شامل  $u$ ) در نظر بگیرید. می‌دانیم در درخت فوق به ازای هر گره داخلی  $u$  نسبت وزن فرزند چپ و فرزند راست حداقل  $0.5$  و حداکثر  $2$  است. بهترین کران بالا برای زمان جستجو در این درخت در بدترین حالت، در بین گزینه‌ها کدام است؟ (مبنای لگاریتم‌ها ۲ است).

- (۱)  $1/5 \log n$  (۲)  $2/5 \log n$  (۳)  $2 \log n$  (۴)  $3 \log n$

۳- در الگوریتم مرتب‌سازی سریع در هر مرحله یک عنصر دلخواه به عنوان محور (pivot) انتخاب و بقیه اعداد با آن مقایسه می‌شود. اگر در هر مرحله میانه اعداد به عنوان محور انتخاب شود و از الگوریتم با تعداد خطی مقایسه برای محاسبه میانه استفاده شود، در بدترین حالت تعداد مقایسه‌ها در این نسخه تغییر یافته الگوریتم مرتب‌سازی سریع کدام است؟

- (۱)  $O(n \log^2 n)$  (۲)  $O(n \log n)$  (۳)  $O(n^2)$  (۴)  $O(n)$

۴- فرض کنید  $k$  پشته  $S_1, \dots, S_k$  را در اختیار داریم. تنها اعمال مجاز گرفتن یک ورودی و پوش کردن آن داخل پشته  $S_1$ ، پاپ کردن یک عنصر از پشته  $S_k$  و قرار دادن در خروجی، پاپ کردن یک عنصر از پشته  $S_i$  (برای  $i < k$ ) و پوش کردن آن داخل پشته  $S_{i+1}$  است. فرض کنید اعداد  $1, \dots, n$  به ترتیب از کوچک به بزرگ به عنوان ورودی داده می‌شود. کوچکترین  $k$  که می‌توان همه جایگشت‌های  $1, \dots, n$  را با اعمال مجاز گفته شده تولید کرد، در بین گزینه‌ها کدام است؟

- (۱) ۱ (۲) ۲ (۳)  $n$  (۴)  $n-1$

۵- فرض کنید از آدرس‌دهی باز و وارسی خطی برای درهم‌سازی استفاده شده و تابع درهم‌سازی  $i^2$  به پیمانه ۷ است. بعد از دریافت همه اعداد ۰, ..., ۶، می‌دانیم نحوه قرارگیری اعداد در جدول درهم‌ساز به صورت زیر است:

$$A[0, \dots, 6] = 0, 6, 4, 3, 1, 5, 2$$

به ازای چند جایگشت ورودی وضعیت جدول درهم‌ساز به شکل بالا خواهد بود؟

$$18 \quad (1) \quad 21 \quad (2) \quad 36 \quad (3) \quad 120 \quad (4)$$

۶- در گراف همبند بدون جهت  $G = (V, E)$  شامل  $n$  رأس، اگر از هر رأس BFS را اجرا کنیم، ارتفاع درخت BFS حداکثر ۲ می‌شود. کدام گزینه در خصوص تعداد یال‌های این گراف درست است؟

$$|E| = \Theta(n) \quad (1)$$

$$|E| = \Theta(n^2) \quad (2)$$

$$|E| = \Omega(n \log n) \quad (3)$$

(۴) برای هر  $n(n-1)/2 \leq i \leq n-1$  می‌توان گرافی با  $i$  یال مثال زد که این ویژگی را داشته باشد.

۷- رشته  $A$  شامل  $n$  کاراکتر را در نظر بگیرید. می‌خواهیم این رشته را به یک رشته آینه‌ای تبدیل کنیم. اعمال مجاز، حذف یک کاراکتر یا درج یک کاراکتر در هر جای رشته است. حداقل چند عمل نیاز است تا این کار انجام شود؟

( $\hat{A}$  رشته آینه‌ای  $A$  است و منظور از LCS و ED به ترتیب طول بزرگترین زیردنباله مشترک و فاصله ویرایشی (با فرض عمل‌های حذف، درج و جایگزینی) است.)

$$n - \text{LCS}(A, \hat{A}) \quad (1) \quad n - \text{ED}(A, \hat{A}) \quad (2)$$

$$\text{LCS}(A, \hat{A}) \quad (3) \quad \text{ED}(A, \hat{A}) \quad (4)$$

۸- فرض کنید در یک شبکه شار برای محاسبه شار از رأس  $t$  از الگوریتم زیر استفاده کنیم. شار عبوری از همه یال‌ها را صفر قرار می‌دهیم. در هر مرحله یک مسیر از  $s$  به  $t$  انتخاب می‌کنیم که یال‌های مسیر همچنان ظرفیت خالی داشته باشند. شار همه یال‌های این مسیر را به اندازه یکسان افزایش می‌دهیم طوری که حداقل یک یال مسیر ظرفیتش تکمیل شود. این کار را آنقدر ادامه می‌دهیم تا چنین مسیری یافت نشود. در مورد اندازه شار حاصل از این الگوریتم کدام مورد درست است؟

(۱) با شار بیشینه برابر است.

(۲) حداقل به اندازه نصف شار بیشینه است.

(۳) حداقل به اندازه یک چهارم شار بیشینه است.

(۴) می‌توان شبکه‌ای مثال زد که برای آن خروجی الگوریتم فوق کمتر از  $1/1000$  شار بیشینه باشد.



- ۹- فرض کنید  $n$  بازه روی محور اعداد حقیقی داریم. می‌خواهیم بیشترین تعداد بازه را پیدا کنیم که با یکدیگر هم‌پوشانی نداشته باشند. برای بازه  $I$  فرض کنید  $e(I)$  برابر تعداد بازه‌هایی است که با  $I$  هم‌پوشانی دارند. برای حل این مسئله از الگوریتم حریصانه زیر استفاده می‌کنیم. در هر مرحله از بین بازه‌های باقی مانده بازه  $I$  را که  $e(I)$  آن از همه کمتر است انتخاب و تمام بازه‌هایی که با آن هم‌پوشانی دارند حذف می‌کنیم. اگر چند بازه وجود داشت که تعداد هم‌پوشانی‌شان کمینه بود یکی را به دلخواه انتخاب می‌کنیم. کمترین  $n$  که الگوریتم حریصانه فوق لزوماً جواب بهینه را تولید نمی‌کند در بین گزینه‌ها کدام است؟
- ۱) ۹      ۲) ۵      ۳) ۳      ۴) ۲

- ۱۰- گراف وزن دار، همبند و بدون جهت  $G$  با  $n$  رأس را در نظر بگیرید. می‌خواهیم درخت پوشای کمینه را با استفاده از الگوریتم کروسکال محاسبه کنیم. اگر یال‌ها را به ترتیب وزن داشته باشیم و دیگر در الگوریتم کروسکال نیاز به مرتب کردن یال‌ها نباشد، الگوریتم کروسکال در چه زمانی قابل پیاده‌سازی است؟
- ۱)  $O(n^2 \log n)$       ۲)  $O(n^2)$       ۳)  $O(n \log n)$       ۴)  $O(n)$

### سؤال‌های چهارگزینه‌ای مهندسی فناوری اطلاعات ۱۳۹۸

- ۱۱- رابطه بازگشتی  $T(n) = \lambda T(n/2) + f(n)$  را در نظر بگیرید. فرض کنید تابع  $g(n)$  یکی از توابع  $n^4 \log n, n^3, n \log n, n$  باشد. به ازای چند تا از این ۴ تابع، تابع  $f(n)$  وجود دارد، به طوری که  $T(n) = O(g(n))$ ؟
- ۱) ۱      ۲) ۲      ۳) ۳      ۴) ۴
- ۱۲- در کد زیر دستور  $A$  چند بار اجرا می‌شود؟

```
for i = ۱ to n do {
    j = i;
    when j > ۱ do {
        A;
        j = j / ۳;
    }
}
```

- ۱)  $O(n)$       ۲)  $O(n^2)$       ۳)  $O(n \log n)$       ۴)  $O(n^{1+1/3})$

۱۳- فرض کنید دو لیست غیر تهی مرتب شده با تعداد عناصر  $a$  و  $b$  داده شده است. برای مرتب

کردن این دو لیست، حداقل و حداکثر چند مقایسه بین عناصر نیاز است؟

$$(۱) \quad ۱, a+b \quad (۲) \quad \min(a, b), a+b-۱$$

$$(۳) \quad \max(a, b), a+b-۱ \quad (۴) \quad \min(a, b), \max(a, b)$$

۱۴- یک درخت دودویی جست و جو متوازن با  $n$  گره داریم که به علت نویز، اعداد ذخیره شده در

برخی از گره‌های آن تغییر کرده است. تنها عملی که می‌توان برای اصلاح این درخت انجام داد،

جابه‌جا کردن مقادیر ذخیره شده در یک گره و یکی از فرزندان آن است. در بدترین حالت با

چند عمل فوق می‌توان درخت را به درخت دودویی جست و جوی معتبر تبدیل کرد؟

$$(۱) \quad O(n) \quad (۲) \quad O(n^2) \quad (۳) \quad O(n \log n) \quad (۴) \quad O(n \log \log n)$$

۱۵- از یک لیست یک سویه اشاره‌گر به ابتدای آن را در اختیار داریم. متأسفانه اشاره‌گر گره

آخر به جای آنکه  $Nil$  باشد به یکی از گره‌های موجود اشاره می‌کند. اگر  $O(۱)$  حافظه در

اختیار داشته باشیم، بهترین مرتبه زمانی برای محاسبه تعداد اعضای لیست از بین گزینه‌ها

کدام است؟ (فرض کنید  $n$  تعداد گره‌های لیست می‌باشد که قرار است محاسبه شود).

$$(۱) \quad O(\log n) \quad (۲) \quad O(n^2) \quad (۳) \quad O(3^n) \quad (۴) \quad \text{قابل محاسبه نیست.}$$

۱۶- زوج‌های مرتب داده شده را در نظر بگیرید:  $(۳, A), (۷, B), (۵, C), (۲, D), (۱, E), (۴, F), (۶, G)$ .

درختی ریشه‌دار با ۷ گره را تصور کنید که در هر گره آن یکی از این زوج‌ها قرار گرفته

است و این درخت براساس درایه اول این زوج‌ها یک هرم بیشینه و براساس درایه دوم یک

درخت جست و جوی دودویی است. با این زوج‌ها چند درخت متمایز با خاصیت گفته شده

می‌توان ساخت؟

$$(۱) \quad ۰ \quad (۲) \quad ۱ \quad (۳) \quad ۲ \quad (۴) \quad ۴$$

۱۷- فرض کنید رئوس گراف همبند و بدون جهت  $G$  با اعداد  $۱, ۲, ۳, \dots, n$  شماره‌گذاری شده‌اند.

از رأس شماره ۱ الگوریتم BFS را اجرا کرده‌ایم و ترتیب ملاقات رئوس از چپ به راست به

ترتیب  $۱, ۲, ۳, \dots, n$  شده است. کدام مورد درست است؟

(۱) از رأس  $n$  می‌توان به گونه‌ای BFS را اجرا کرد که ترتیب ملاقات رئوس از چپ به راست

$۱, ۲, \dots, n$  شود.

(۲) به ازای هر  $i > ۱$  حتماً حداقل یک  $j < i$  وجود دارد که بین  $i$  و  $j$  یک یال وجود دارد.

(۳) بین رأس  $i$  و  $i+۱$  به ازای هر  $i$  یال وجود دارد.

(۴) هیچ کدام از موارد

۱۸- فرض کنید متنی شامل  $n$  کاراکتر متمایز است و کاراکتر  $i$  ام ( $i=1, \dots, n$ ) در متن  $i^{\text{ام}}$  بار تکرار شده است. طول کد هافمن این متن برای  $n=7$  کدام است؟

(۱) ۲۵۶ (۲) ۴۹۲ (۳) ۵۱۱ (۴) ۵۱۲

۱۹- فرض کنید  $G$  یک گراف همبند، جهت دار و وزن دار (با وزن‌های مثبت) و  $\ell$  و یک عدد صحیح مثبت است. می‌خواهیم از یک رأس به رأس دیگر مسیری به وزن حداکثر  $\ell$  (مجموع وزن یال‌های مسیر حداکثر  $\ell$  باشد) پیدا کنیم که تعداد رئوس میانی مسیر کمینه باشد. در این مورد کدام گزینه صحیح است؟

- (۱) یک مسئله ان پی - کامل است.
- (۲) یک مسئله ان پی - سخت است.
- (۳) در خروجی الگوریتم دایکسترا، تعداد رئوس میانی کمینه است.
- (۴) در زمان چندجمله‌ای برحسب اندازه ورودی می‌توان مسئله را حل کرد.

۲۰- فرض کنید  $P(x) = a_n x^n + \dots + a_1 x + a_0$  یک چندجمله‌ای از درجه  $n$  باشد. به ازای  $b$  داده شده، کدام یک از گزاره‌های زیر در مورد  $P(b)$  درست است؟ (فرض کنید  $b$  و تمام ضرایب چندجمله‌ای، اعداد ۱۶ بیتی هستند.)

- (الف) می‌توان  $P(b)$  را با  $n$  عمل جمع و  $n$  عمل ضرب محاسبه کرد.
- (ب)  $P(b)$  در بدترین حالت می‌تواند  $\Theta(n)$  بیتی باشد.
- (۱) هر دو مورد درست است.
- (۲) هر دو مورد نادرست است.
- (۳) (الف) درست و (ب) نادرست است.
- (۴) (الف) نادرست و (ب) درست است.

۲۱- دنباله  $x_1, \dots, x_k$  را دنباله خوب می‌نامیم، اگر به ازای هر  $i$  داشته باشیم  $x_i < x_{i-1} + x_{i+1}$ . فرض کنید می‌خواهیم طول بزرگ‌ترین زیردنباله  $a_1, \dots, a_n$  را طوری به دست آوریم که خوب باشد. به این منظور آرایه دو بعدی  $A$  را بدین شکل تعریف می‌کنیم:  $A[i, j]$  برابر طول بزرگ‌ترین زیردنباله خوب  $a_i, \dots, a_j$ ، فرض کنید  $L(i, j)$  و  $\bar{L}(i, j)$  به ترتیب برابر طول بزرگ‌ترین زیردنباله صعودی و طول بزرگ‌ترین زیردنباله نزولی دنباله  $a_i, \dots, a_j$  باشند. کدام رابطه زیر درست است؟

- (۱)  $A(1, n) = L(1, n)$
- (۲)  $A(1, n) = \max_{i=1}^n (\bar{L}(1, i) + L(i, n) - 1)$
- (۳)  $A(1, n) = \max_{i=1}^n (\bar{L}(1, i) + L(i, n))$
- (۴) هیچ یک از موارد

۲۲- فرض کنید یک گراف وزن دار داریم که دور منفی ندارد. برای محاسبه کوتاه‌ترین مسیر از رأس  $s$  به رأس  $t$  از الگوریتم زیر استفاده می‌کنیم.

همه وزن‌ها را با یک عدد مثبت مناسب  $x$  جمع می‌زنیم تا همگی مثبت شوند. گراف حاصل را  $G_x$  می‌نامیم. در  $G_x$  با استفاده از الگوریتم دایکسترا کوتاه‌ترین مسیر از  $s$  به رأس  $t$  را محاسبه می‌کنیم. طول این مسیر (جمع وزن یال‌های مسیر) را منهای تعداد یال‌ها ضربدر  $x$  می‌کنیم و به عنوان خروجی گزارش می‌دهیم. در مورد خروجی الگوریتم کدام مورد درست است؟

- (۱) برابر طول کوتاه‌ترین مسیر از  $s$  به  $t$  در  $G$  است.
- (۲) حداکثر ۲ برابر طول کوتاه‌ترین مسیر از  $s$  به  $t$  در  $G$  است.
- (۳) حداکثر ۴ برابر طول کوتاه‌ترین مسیر از  $s$  به  $t$  در  $G$  است.
- (۴) می‌توان گرافی مثال زد که خروجی الگوریتم حداقل ۱۳۹۸ برابر طول کوتاه‌ترین مسیر از  $s$  به  $t$  در  $G$  است.

## پاسخنامه مهندسی کامپیوتر ۱۳۹۸

۱- گزینه (۳). با جایگذاری از بالا:

$$\begin{aligned} T(n) &= T\left(\frac{n}{\lg n}\right) + 1 = T\left(\frac{\frac{n}{\lg n}}{\lg \frac{n}{\lg n}}\right) + 2 = T\left(\frac{n}{\lg n (\lg n - \lg \lg n)}\right) + 2 \\ &\approx T\left(\frac{n}{\lg^2 n}\right) + 2 = T\left(\frac{\frac{n}{\lg^2 n}}{\lg \frac{n}{\lg^2 n}}\right) + 3 = T\left(\frac{n}{\lg^2 n (\lg n - \lg \lg^2 n)}\right) + 3 \\ &\approx T\left(\frac{n}{\lg^3 n}\right) + 3 = \dots \approx T\left(\frac{n}{\lg^k n}\right) + k \end{aligned}$$

حال فرض می‌کنیم شرط اولیه  $T(1) = 1$  پس  $\frac{n}{\lg^k n} = 1$  در نتیجه  $n = \lg^k n$  پس

$k = \frac{\lg n}{\lg \lg n}$  در نتیجه  $T(n) = 1 + k$  پس  $T(n) = 1 + \frac{\lg n}{\lg \lg n}$  که برابر  $\theta\left(\frac{\lg n}{\lg \lg n}\right)$  می‌شود.

۲- گزینه (۳). به ازای هر گره اگر سمت چپ آن  $x$  گره باشد سمت راست آن حداکثر  $2x$  گره

است پس به ازای ریشه می‌توان گفت سمت چپ آن  $\frac{n}{3}$  و راست آن حداکثر  $\frac{2n}{3}$  گره است

پس بدترین زمان جستجوی یک کلید  $\log_{\frac{2}{3}} n$  است که برابر است با  $\frac{\lg n}{\lg \frac{2}{3}}$  یعنی

$\log_{\frac{2}{3}} \lg n$  که  $\frac{1}{\lg 3} \lg n$  می‌شد و گزینه ۱ و ۳ می‌تواند درست باشد. کلید گزینه ۳ است.

۳- گزینه (۲). زمان اجرا برابر با  $T(n) = 2T\left(\frac{n}{3}\right) + \theta(n)$  می‌شود که از مرتبه  $\theta(n \lg n)$

است. توجه کنید وقتی میانه را به عنوان محور انتخاب می‌کنید بعد از افراز دو طرف محور تعداد عناصر مساوی است.

۴- گزینه (۴). سوال در سطح کارشناسی و ارشد نیست! در مقاله‌ای گفته شده که کمترین تعداد پشته برای این منظور حدود  $\lg n$  تاست و مجبوریم گزینه ۴ را انتخاب کنیم.

۵- گزینه (۲).

$$h(i) = i^2 \bmod 7$$

| ۰ | ۱ | ۲ | ۳ | ۴ | ۵ | ۶ |
|---|---|---|---|---|---|---|
| ۰ | ۶ | ۴ | ۳ | ۱ | ۵ | ۲ |

| کلید | ۰ | ۱ | ۲ | ۳ | ۴ | ۵ | ۶ |
|------|---|---|---|---|---|---|---|
| آدرس | ۰ | ۱ | ۴ | ۲ | ۲ | ۴ | ۱ |

برای راحتی نام کلیدها را a و b و c و d و e و f و g می‌نامیم:

|   | a | b | c | d | e | f | g |
|---|---|---|---|---|---|---|---|
| ۰ | ۱ | ۴ | ۲ | ۲ | ۴ | ۱ |   |

 $\Rightarrow$ 

| ۰ | ۱ | ۲ | ۳ | ۴ | ۵ | ۶ |
|---|---|---|---|---|---|---|
| a | g | e | d | b | f | c |

g و e در مکان طبیعی خود هستند ولی سایر کلیدها نه. کلید a هر زمان درج شود در حفرة صفر قرار می‌گیرد پس فعلاً a را نادیده بگیرید، سایر کلیدها به صورت egdbfc و gedbfc و edgbfc اگر درج شوند آنگاه جدول فوق تولید می‌شود حال a هر زمان می‌تواند درج شود که در هر یک از ۳ حالت گفته شده ۷ حالت می‌توان a را درج کرد پس جمعاً ۲۱ حالت وجود دارد.

۶- گزینه (۴). در هر گراف کامل از هر رأس BFS بزنیم ارتفاع درخت حاصل برابر ۱ می‌شود

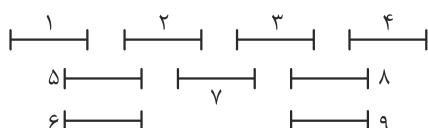
پس برای گراف کامل درست است که تعداد یال این گراف  $\frac{n(n-1)}{2} = \theta(n^2)$  است ولی برای برخی گراف‌های خلوت نیز درست است. مثلاً برای درخت ستاره‌ای از هر رأس BFS بزنید ارتفاع درخت حاصل حداکثر ۲ می‌شود پس گزینه ۴ درست است.

۷- گزینه (۱). فاصله ویرایشی ED یعنی حداقل تعداد تغییراتی است که لازم است یک رشته تبدیل به یک رشته دیگر شود مثلاً  $ED(ab, ac) = 1$  چون برای آنکه ab تبدیل به ac شود یک تغییر (تبدیل b به c) لازم است. رشته آینه‌ای یعنی رشته‌ای که از دو طرف به یک شکل خوانده شود مثلاً abccba آینه‌ای است. فرض کن  $A = abcdefg$  را بخواهیم آینه‌ای کنیم ۳ تغییر لازم است که  $\hat{A} = abcdcba$  حاصل شود.

حال  $ED(A, \hat{A}) = 3$  و  $LCS(A, \hat{A}) = 4$  و  $n = 7$  که گزینه‌های ۲ و ۳ رد می‌شوند و کلید گزینه ۱ است ولی گزینه ۴ هم معنی صورت سوال است و غلط نیست.

۸- گزینه (۴). گراف پسماند بدست نیامده و الگوریتم گفته شده لزوماً شار بیشینه را نمی‌دهد و مقدار شاری که توسط این الگوریتم بدست می‌آید کمتر یا مساوی شار بیشینه است.

۹- گزینه (۱). مثال نقض حداقل با ۹ فعالیت بدست می‌آید:



در ۹ فعالیت مقابل حداکثر می‌توان ۴ فعالیت انتخاب کرد ولی روش این سوال ممکن است ابتدا فعالیت ۷ را انتخاب کند و لذا ۲ و ۳ حذف شوند و حال حداکثر ۲ فعالیت دیگر قابل انتخاب است یعنی جمعاً ۳ فعالیت که غلط است.

- ۱۰- گزینه (۲). مرتبه کراسکال در این صورت  $O(e \cdot \alpha)$  است که حدوداً  $O(e)$  می‌شود و  $e$  حداکثر  $n^2$  است.

### [پاسخنامه مهندسی کامپیوتر ۱۳۹۸]

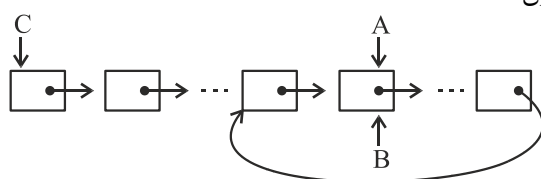
- ۱۱- گزینه (۲). با توجه به وجود  $T(\frac{n}{4}) \wedge T(n) = \Omega(n^3)$  داریم یعنی رشد  $(n)$  از  $n^3$  بیشتر یا مساوی است پس  $n^3$  و  $n^4 \lg n$  می‌توانند رشد این تابع باشند.

- ۱۲- گزینه (۳). حلقه while لگاریتمی است و حلقه for خطی است البته در حلقه while مقدار  $i$  وابسته است ولی در این مثال خاص تأثیری در مرتبه ندارد و جواب  $n \times \log_3 n$  می‌شود.

- ۱۳- گزینه (۲).

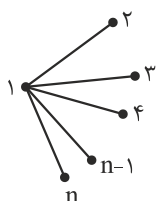
- ۱۴- گزینه (۳). در بدترین حالت هر گره باید ارتفاع درخت را طی کند تا در جای درست خود قرار گیرد و چون درخت متوازن است زمان برای  $n$  گره  $n \lg n$  می‌شود.

- ۱۵- گزینه (۲). سه اشاره گر  $A$  و  $B$  و  $C$  تعریف می‌کنیم.  $A$  و  $B$  را در یک حلقه while جلو می‌بریم،  $A$  با سرعت یک نود و  $B$  با سرعت ۲ نود جلو می‌رود و هر جا که  $A$  و  $B$  به هم رسیدند توقف می‌کنیم که مرتبه تاکنون  $n$  است.



سپس اشاره گر  $C$  را روی نود اول قرار می‌دهیم و  $A$  را حرکت می‌دهیم اگر به  $C$  رسید یعنی نقطه‌ای که نود آخر به آن اشاره کرده پیدا شد، اگر به  $B$  رسید،  $C$  را یک نود جلو می‌بریم و مجدد  $A$  را حرکت می‌دهیم، این عملیات  $O(n^2)$  است و می‌توان نقطه‌ای که نود آخر به آن اشاره می‌کند پیدا کرد که با یافتن آن نقطه می‌توان تعداد نودها را با مرتبه  $n$  شمرد.

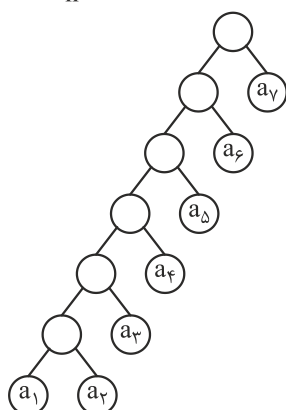
- ۱۶- گزینه (۲). درخت توصیف شده treap است که در ساختمان داده دیده‌اید (البته با کمی تغییرات) اگر عناصر متمایز باشند فقط و فقط یک treap می‌توان ساخت.



۱۷- گزینه (۲). گزینه ۱ و ۳ غلط هستند و نقض آنها گراف ستاره‌ای مقابل است:

گزینه ۲ صحیح است. مثلاً اگر  $i = 5$  حتماً رأسی مثل ۱ یا ۲ یا ۳ یا ۴ وجود دارد که به رأس ۵ وصل است چون اگر هیچ از رؤس  $j$  که  $j < i$  به  $i$  وصل نباشند آنگاه امکان ملاقات رأس  $i$  نخواهد بود.

۱۸- گزینه (۲).



| کاراکتر | $a_1$ | $a_2$ | $a_3$ | $a_4$ | $a_5$ | $a_6$ | $a_7$ |
|---------|-------|-------|-------|-------|-------|-------|-------|
| فراوانی | ۲     | ۴     | ۸     | ۱۶    | ۳۲    | ۶۴    | ۱۲۸   |

$$\begin{aligned} \text{هزینه درخت} &= (2+4)(6) + (8)(5) + (16)(4) + (32)(3) \\ &\quad + (64)(2) + (128)(1) = 492 \end{aligned}$$

۱۹- گزینه (۴). در دایجسترا تعداد رؤس مسیر لزوماً کمینه نیست بلکه اندازه مسیر یعنی جمع وزن یالها کمینه است. در این مسئله اصل بهینگی برقرار است و می‌توان با یک روش پویا و زمان چندجمله‌ای مسئله را حل کرد.

۲۰- گزینه (۱). (الف) را می‌توان با روش هورنر انجام داد که تعداد ضرب و جمع آن  $\theta(n)$  است.

در مورد (ب) می‌دانیم عدد مثلاً  $K$  دارای  $\lg(K)$  بیت است پس  $a_n \times b^n$  دارای  $\lg a_n + n \times \lg b$  بیت است و چون  $a_n$  و  $b$ ، ۱۶ بیتی هستند پس لگاریتم آنها عدد ثابت است پس تعداد بیت‌ها  $\theta(n)$  می‌شود ( $n \times \lg b = \theta(n)$ ).

۲۱- گزینه (۴). دنباله  $\langle 1, 2, 3, 4, 5 \rangle$  را در نظر بگیرید، این دنباله شامل دنباله خوب نیست.

یعنی زیردنباله خوب این دنباله تهی است یعنی اسم این دنباله اگر  $A$  باشد آنگاه  $A(1, 5) = 0$  ولی  $L(1, 5) = 5$  و  $\bar{L}(1, 5) = 1$  (هر کاراکتر یک جواب است) پس گزینه ۱

غلط است، گزینه ۲ هم غلط هست چون طبق این گزینه

$$A(1, 5) = \max(\underbrace{\bar{L}(1, 1)}_1 + \underbrace{L(1, 5)}_5 - 1, \dots) \neq 0$$

گزینه ۳ هم با همین مثال غلط است.

۲۲- گزینه (۴). الگوریتم گفته شده غلط است و نمی‌تواند کوتاهترین مسیر را حساب کند و خروجی آن می‌تواند هر چه باشد یعنی خروجی الگوریتم می‌تواند  $x$  برابر خروجی دایجسترا باشد.



## [سؤال‌های چهار گزینه‌ای مهندسی کامپیوتر ۱۳۹۹]

۱- به ازای چند زوج  $(a, b)$  از اعداد طبیعی کوچکتر از ۵، جواب رابطه بازگشتی

$$T(n) = aT(n/b) + n^2 \quad \theta(n^2) \text{ می‌شود؟}$$

- (۱) ۷ (۲) ۸ (۳) ۱۱ (۴) ۱۲

۲- جدول درهم‌سازی ۱۰ خانه‌ای و تابع درهم‌ساز  $h(x) = 3x + 5 \bmod 10$  و روش زنجیره‌ای

به عنوان روش رفع تصادم را در نظر بگیرید. در این خصوص کدام گزینه درست است؟

- (۱) احتمال آن که ورودی  $x = 4$  به خانه ۷ نگاشت شود برابر ۱ است.  
 (۲) احتمال آن که ورودی  $x = 4$  به خانه ۷ نگاشت شود برابر ۰ است.  
 (۳) احتمال آن که ورودی  $x = 4$  به خانه ۷ نگاشت شود برابر  $\frac{1}{10}$  است.  
 (۴) احتمال آن که دو ورودی مختلف به یک نگاشت شوند برابر  $\frac{1}{10}$  است.

۳- فرض کنید یک آرایه مرتب از  $n$  عدد صحیح در اختیار داریم. با چه تعداد مقایسه می‌توانیم بفهمیم عددی بیش از  $n/5$  بار در آرایه تکرار شده است یا خیر؟ (بهترین گزینه را انتخاب کنید.)

- (۱)  $O(n)$  (۲)  $O(\log n)$  (۳)  $O(\log^* n)$  (۴)  $O(\log \log n)$

۴- اعداد صحیح  $x_1, \dots, x_n$  را در یک درخت دودویی جستجو با ارتفاع  $h$  ذخیره کرده‌ایم.

هزینه جستجوی  $x_i$  (تعداد مقایسه‌های لازم در درخت برای پیدا کردن  $x_i$  برابر  $c_i$

باشد. می‌دانیم  $\sum_{i=1}^n c_i = O(n \log n)$  است. کدام گزینه زیر درست است؟

- (۱)  $h = \Omega(\sqrt{n})$  (۲)  $h = O(\log n)$   
 (۳)  $h = O(\sqrt{n \log n})$  (۴) می‌توان مثالی زد که  $h = \Omega(n)$  باش.

۵- فرض کنید  $T(n)$  متوسط زمان اجرای الگوریتم مرتب‌سازی سریع به ازای همه

جایگشت‌های ممکن ورودی از  $n$  عدد متمایز باشد. کدام رابطه بازگشتی زیر درست است؟

- (۱)  $T(n) = O(n) + \sum_{i=0}^{n-1} (T(i) + T(n-i))$   
 (۲)  $T(n) = O(n) + (1/n) \sum_{i=0}^{n-1} (T(i) + T(n-i))$   
 (۳)  $T(n) = (1/n) \left( O(n) + \sum_{i=0}^{n-1} (T(i) + T(n-i)) \right)$   
 (۴)  $T(n) = O(n) + (1/n) \sum_{i=0}^{n-1} \min(T(i), T(n-i))$

۶- فرض کنید  $T$  درخت جستجوی عمق اول گراف همبند و بدون جهت  $G$  است. دو رأس  $u$  و  $v$  در این درخت برگ و در  $G$  دارای درجه حداقل ۲ هستند. کدام یک از گزاره‌های زیر صحیح است؟

- الف) باید یک رأس  $w$  وجود داشته باشد که با  $u$  و  $v$  در  $G$  همسایه باشد.  
 ب) باید یک رأس  $w$  وجود داشته باشد که حذف آن  $u$  را از  $v$  در  $G$  جدا می‌کند.  
 (۱) (الف) نادرست، (ب) نادرست (۲) (الف) نادرست، (ب) درست  
 (۳) (الف) درست، (ب) نادرست (۴) (الف) درست، (ب) درست

۷- فرض کنید آرایه  $A[1..n]$  شامل  $n$  عدد صحیح متمایز است که به صورت صعودی مرتب شده‌اند. چند تا از مسائل زیر را می‌توان با مرتبه زمانی  $O(\log n)$  حل کرد؟

- پیدا کردن یک اندیس  $i$  طوری که  $A[i] = i$  شود.
  - پیدا کردن یک اندیس  $i$  طوری که  $A[i] = 3i + 2$  شود.
  - پیدا کردن یک اندیس  $i$  طوری که  $A[i] = 4i^2 + 3i + 5$  شود.
- (۱) ۰ (۲) ۱ (۳) ۲ (۴) ۳

۸- چند مورد از مسائل ان پی - سخت زیر هنگامی که ورودی یک درخت بدون وزن است، در زمان چندجمله‌ای قابل حل هستند؟

- پیدا کردن طولانی‌ترین مسیر
- محاسبه کمینه رنگ مورد نیاز برای رنگ‌آمیزی رأسی به طوری که رئوس مجاور هم‌رنگ نباشند.
- پیدا کردن بزرگترین مجموعه مستقل (مجموعه رئوسی که بین هر دو رأس یالی وجود نداشته باشد).

- (۱) ۰ (۲) ۱ (۳) ۲ (۴) ۳

۹- گراف همبند و وزن دار  $G$  را در نظر بگیرید. درخت پوشای کمینه  $G$  لزوماً یکتا نیست و  $G$  می‌تواند چندین درخت پوشای کمینه داشته باشد. فرض کنید  $T$  یک درخت پوشای کمینه دلخواه از  $G$  باشد. کدام یک از گزاره‌های زیر درست هستند؟

- (الف) حتماً یک اجرا از الگوریتم پریم وجود دارد که  $T$  را تولید کند.  
 (ب) حتماً یک اجرا از الگوریتم کروسکال وجود دارد که  $T$  را تولید کند.  
 توجه کنید زمانی که وزن یال‌ها متمایز نیست، می‌توان اجراهای متفاوتی برای هر دو الگوریتم پریم و کروسکال در نظر گرفت. در واقع وقتی چندین یال دارای وزن یکسان باشند، می‌توان هر ترتیبی از آن‌ها را برای پردازش مورد نظر الگوریتم‌های فوق در نظر گرفت.

- (۱) (الف) درست، (ب) درست (۲) (الف) درست، (ب) نادرست  
 (۳) (الف) نادرست، (ب) درست (۴) (الف) نادرست، (ب) نادرست

۱۰- فرض کنید در کدگذاری هافمن، طول کد همه کاراکترها یکسان شده است. با فرض آن که تعداد کاراکترها ۳۲ می‌باشد، چند تا از گزاره‌های زیر همیشه درست است؟

- تعداد تکرار همه کاراکترها یکسان است.
- اختلاف تکرار هر دو کاراکتر حداکثر یک است.
- اختلاف تکرار هر دو کاراکتر حداکثر دو است.
- به ازای هر عدد ثابت  $c$  می‌توان مثالی زد که دو کاراکتر وجود داشته باشند که اختلاف تکرارشان حداقل  $c$  باشد.

(۱) ۰ (۲) ۱ (۳) ۲ (۴) ۳

۱۱- الگوریتم خرد کردن پول با روش حریصانه «استفاده از پر ارزش‌ترین سکه، تا حد امکان» روی کدام مجموعه سکه‌ها، لزوماً جواب بهینه (با کم‌ترین تعداد سکه) را تولید نمی‌کند؟ (فرض کنید از سکه‌های هر مجموعه به تعداد نامتناهی داریم.)

(۱)  $\{1, 2, 5\}$  (۲)  $\{1, 4, 7\}$  (۳)  $\{1, 5, 10\}$  (۴)  $\{1, 7, 10\}$

۱۲- فرض کنید در الگوریتم مرتب‌سازی سریع پس از عمل بخش‌بندی (Partition) آرایه  $\langle 9, 6, 7, 8, 5, 4, 2, 1, 3 \rangle$  به دست آمده است. چند عدد از بین ۹ عدد در آرایه ممکن است محور این بخش‌بندی قرار گرفته باشند؟

(۱) ۲ (۲) ۳ (۳) ۴ (۴) ۵

۱۳- فرض کنید برای درهم‌سازی از روش زنجیره‌ای با یک جدول به اندازه  $m$  استفاده شده است. تابع درهم‌ساز رکورد با کلید  $k$  را به خانه  $k \bmod m$  نگاشت می‌کند. اگر بدانیم کلید رکوردها، زیرمجموعه  $\{i \mid 1 \leq i \leq 100\}$  است، به ازای کدام  $m$  هزینه جست‌وجو در بدترین حالت کمتر است؟

(۱) ۷ (۲) ۹ (۳) ۱۱ (۴) ۱۲

۱۴- کدام یک از دنباله‌های زیر (به ازای  $n$ های بزرگ) بیشترین ارتفاع ممکن را برای درخت هافمن ایجاد می‌کند؟

(اعضای دنباله‌ها نشان‌دهنده تعداد تکرار کاراکترها در متن ورودی است نه خود کاراکترها.)

(۱) دنباله از  $n$  عدد برابر (۲) دنباله از  $n$  عدد فیبوناچی پشت سرهم

(۳) دنباله  $(1, 2, 3, 4, 5, \dots, n)$  (۴) دنباله  $(1^2, 2^2, 3^2, 4^2, 5^2, \dots, n^2)$

## سؤالهای چهارگزینه‌ای فناوری اطلاعات ۱۳۹۹

۱۵- کدام گزینه درست است؟ (دقت کنید که در زیر از حرف O کوچک استفاده شده است.)

(۱)  $n! = o(n^n)$ ,  $\log n! = \theta(\log n^n)$

(۲)  $n! = o(n^n)$ ,  $\log n! = o(\log n^n)$

(۳)  $n! = \theta(n^n)$ ,  $\log n! = \theta(\log n^n)$

(۴)  $n! = o(n^n)$ ,  $\log n! = \omega(\log n^n)$

۱۶- وضعیت جدول درهم‌سازی  $H[0..9]$  بعد از درج هفت عدد  $S_1, \dots, S_7$  به صورت زیر است:

$$H[0..9] = [S_7, S_1, \square, S_4, S_2, S_5, \square, S_6, S_3]$$

که  $\square$  نشان‌دهنده خانه خالی است. برای درهم‌سازی از روش درهم‌سازی باز با واریسی خطی استفاده شده است. برای جستجوی عنصری که در جدول نیست حداکثر چند مقایسه باید انجام شود؟ (دقت کنید چک کردن آن که یک خانه خالی است خود به یک مقایسه نیاز دارد.)

(۱) ۲ (۲) ۳ (۳) ۴ (۴) ۵

۱۷- چند درخت دودویی جست‌وجوی متفاوت با  $n$  گره و برچسب‌های ۱ تا  $n$  وجود دارد، به طوری که پیمایش پیش‌ترتیب و میان‌ترتیب آنها یکسان باشد؟

(۱) ۰ (۲) ۱ (۳)  $n!$  (۴) عدد  $n$  ام کاتالان

۱۸- اعداد ۱ تا ۵۰۰ را در یک درخت دودویی جست‌وجو ذخیره کرده‌ایم. می‌خواهیم عدد ۱۹۳ را در این درخت جست‌وجو کنیم. کدام دنباله نمی‌تواند مسیر جست‌وجو برای عدد ۱۹۳ باشد؟

(۱) ۴, ۲۰, ۳۰, ۵۵, ۱۰۱, ۱۰۲, ۱۰۵, ۱۷۷, ۱۹۳

(۲) ۵, ۴۵۴, ۳۰۰, ۱۰۰, ۲۵۰, ۱۵۰, ۲۰۰, ۱۹۳

(۳) ۴۳۷, ۱۵۷, ۲۳۷, ۲۳۱, ۲۰۱, ۱۴۳, ۱۹۰, ۱۹۳

(۴) ۵۰۰, ۴۰۰, ۳۰۰, ۲۰۰, ۵۰, ۱۰۰, ۱۵۰, ۱۹۳

۱۹- آرایه  $A[1...13]$  شامل ۱۳ عدد صحیح را در نظر بگیرید. می‌توانیم هر بار دو خانه دلخواه از این آرایه را با هم جابه‌جا کنیم، کدام گزینه را نمی‌توان با حداکثر یک بار جابه‌جایی به هرم پیشینه تبدیل کرد؟

$$A[1...13] = 89, 19, 70, 17, 12, 40, 2, 5, 7, 11, 6, 9, 10 \quad (1)$$

$$A[1...13] = 89, 19, 40, 17, 12, 70, 2, 5, 7, 11, 6, 9, 10 \quad (2)$$

$$A[1...13] = 89, 19, 70, 17, 2, 40, 12, 5, 7, 11, 6, 9, 10 \quad (3)$$

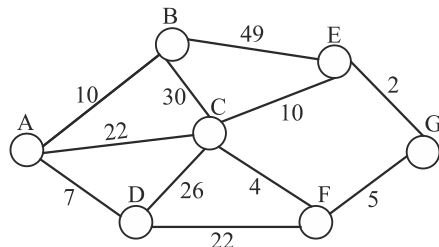
$$A[1...13] = 89, 19, 40, 17, 12, 10, 2, 5, 7, 11, 6, 9, 70 \quad (4)$$

۲۰- به ازای چند تا از الگوریتم‌های مرتب‌سازی زیر، پیچیدگی زمانی حالت متوسط، بدترین حالت و بهترین حالت یکسان است؟

- مرتب‌سازی سریع
- مرتب‌سازی ادغامی
- مرتب‌سازی شمارشی
- مرتب‌سازی درجی

۰ (۱)      ۱ (۲)      ۲ (۳)      ۳ (۴)

۲۱- در گراف زیر، الگوریتم پریم را با شروع از رأس A اجرا کرده‌ایم. کدام ترتیب زیر (از چپ به راست) می‌تواند ترتیب اضافه شدن یال‌ها به درخت پوشای کمینه باشد؟



$$(A, D), (A, B), (D, F), (F, C), (F, G), (G, E) \quad (1)$$

$$(A, B), (A, D), (D, F), (F, G), (G, E), (F, C) \quad (2)$$

$$(A, D), (A, B), (A, C), (C, F), (G, E), (F, G) \quad (3)$$

$$(E, G), (C, F), (F, G), (A, D), (A, B), (A, C) \quad (4)$$

۲۲- در گراف همبند، بدون جهت و بدون وزن G، الگوریتم دایکسترا را با شروع از رأس S اجرا می‌کنیم. در هر گام از الگوریتم دایکسترا یک رأس مختومه می‌شود. به این معنی که طول

کوتاهترین مسیر به آن رأس محاسبه می‌شود. حال ترتیبی که رؤس در الگوریتم دایکسترا مختومه شده‌اند را در نظر بگیرید. در خصوص گزاره‌های زیر کدام گزینه صحیح است؟  
(الف) همیشه یک ترتیب BFS از رؤس وجود دارد که با ترتیب مختومه شدن رؤس در الگوریتم دایکسترا یکسان است.

(ب) همیشه یک ترتیب DFS از رؤس وجود دارد که با ترتیب مختومه شدن رؤس در الگوریتم دایکسترا یکسان است.

(۱) (الف) درست، (ب) درست

(۳) (الف) نادرست، (ب) درست

(۲) (الف) درست، (ب) نادرست

(۴) (الف) نادرست، (ب) نادرست

۲۳- فرض کنید در یک گراف همبند و بدون جهت  $G$ ، الگوریتم جست‌وجوی سطح اول را با شروع از رأس  $r$  اجرا کنیم. فرض کنید  $u$  و  $v$  دو رأس دلخواه و متمایز  $G$  به غیر از  $r$  باشند. همچنین فرض کنید  $d(r, u)$  و  $d(r, v)$  طول کوتاهترین مسیر از  $r$  به  $u$  و  $v$  باشند. اگر  $u$  قبل از  $v$  در جست‌وجوی سطح اول ملاقات شده باشد، کدام گزینه صحیح است؟

$$(۱) d(r, u) \leq d(r, v)$$

$$(۲) d(r, u) < d(r, v)$$

$$(۳) d(r, u) > d(r, v)$$

(۴) هیچ یک از موارد صحیح نیست.

۲۴- فرض کنید یک کوله‌پشتی با ظرفیت ۱۰۰ داریم. تعدادی شیء با حجم و ارزش صحیح داده شده است. می‌خواهیم تعدادی از این اشیاء را در کوله بگذاریم، طوری که اولاً در کوله جا شوند و ثانیاً مجموع ارزش‌شان بیشینه شود. الگوریتم حریصانه زیر را در نظر بگیرید. اشیاء را به صورت صعودی براساس ارزش روی حجم مرتب می‌کنیم. اشیاء را به ترتیب لیست فوق مورد بررسی قرار داده و اگر فضای خالی کوله‌پشتی حداقل به اندازه شیء مورد بررسی بود آن شیء را در کوله قرار می‌دهیم.

به ازای کدام ورودی زیر الگوریتم حریصانه جواب بهینه برنمی‌گرداند؟ (در زیر حجم و ارزش اشیاء به ترتیب در آرایه‌های  $V$  و  $W$  آمده است. یعنی شیء  $i$ ام دارای حجم  $V[i]$  و ارزش  $W[i]$  است.)

$$(۱) W[1..5] = [3, 1, 1, 1, 1], V[1..5] = [76, 25, 25, 25, 25]$$

$$(۲) W[1..5] = [2, 5, 9, 1, 1], V[1..5] = [7, 3, 45, 25, 40]$$

$$(۳) W[1..5] = [8, 2, 2, 2, 2], V[1..5] = [8, 25, 25, 25, 25]$$

(۴) همه موارد فوق

۲۵- یک شبکه اجتماعی را در نظر بگیرید که در آن دوستی‌ها لزوماً دو طرفه نیست. بنابراین اگر شخص  $u$ ، شخص  $v$  را بشناسد (دوست باشد) در گراف شبکه اجتماعی یک یال جهت‌دار از  $u$  به  $v$  درج می‌شود. در این شبکه اجتماعی اگر کسی از خبری مطلع شود آن را به اطلاع همه دوستان خود خواهد رساند. می‌خواهیم یک خبر را به اطلاع همه در این شبکه برسانیم. حداقل چند نفر را باید از این خبر مطلع کنیم تا همه (با نشر خبر) از آن مطلع شوند؟

(۱) یک نفر

(۲) به تعداد مؤلفه‌های قویاً همبند گراف شبکه اجتماعی

(۳) به تعداد مؤلفه‌های قویاً همبند گراف شبکه اجتماعی که ورودی از هیچ مؤلفه همبند قوی دیگر ندارند.

(۴) هیچ یک از گزینه‌ها

۲۶- آرایه  $A[0..n-1]$  از اعداد حقیقی داده شده است. می‌خواهیم از ماتریس  $B[0..n-1, 0..n-1]$  را طوری بسازیم، که به ازای هر  $i \leq j$  داشته باشیم

$$B[i, j] = \sum_{k=i}^j A[k]$$

الگوریتم کارایی که این عملیات را انجام دهد از چه مرتبه‌ای است؟

(۱)  $O(n)$       (۲)  $O(n^2)$       (۳)  $O(n^3)$       (۴)  $O(n \log n)$

## [پاسخنامه مهندسی کامپیوتر ۱۳۹۹]

۱- گزینه (۳).

$$a=1 \rightarrow b=2, 3, 4$$

$$a=2 \rightarrow b=2, 3, 4$$

$$a=3 \rightarrow b=2, 3, 4$$

$$a=4 \rightarrow b=2, 3$$

۲- گزینه (۱). ورودی  $x=4$  در خانه  $h(4) = (3(4) + 5) \bmod 10 = 7$  نگاشت می‌شود پس احتمال این داستان صددرصد است.

۳- گزینه (۲). با توجه به اینکه آرایه مرتب است می‌توان شبیه جستجوی دودویی با مرتبه لگاریتمی به جواب رسید.

۴- گزینه (۳). ارتفاع چنین درختی حداکثر  $\sqrt{n \lg n}$  است. زیرا اگر ارتفاع از این بیشتر شود آنگاه  $\sum_{i=1}^n C_i$  از  $n \lg n$  بیشتر می‌شود.

۵- گزینه (۲). زمان اجرای مرتب‌سازی سریع  $T(n) = T(k) + T(n-k-1) + O(n)$  است که به ازای  $0 \leq k \leq n-1$  حالات مختلف بوجود می‌آید پس متوسط می‌شود:

$$T(n) = \frac{1}{n} \sum_{k=0}^{n-1} (T(k) + T(n-k-1)) + O(n)$$

۶- گزینه (۱). در گراف  $C_4$  اگر از  $u$  شروع کرده و به ترتیب  $x$  و  $y$  و  $v$  را ملاقات کنید درخت حاصل می‌شود که  $u$  و  $v$  درجه یک هستند و در خود گراف این دو درجه ۲ هستند و این مثال هر دو گزاره الف و ب را نقض می‌کند یعنی رأس مثل  $w$  نیست که هم با  $u$  و هم با  $v$  مجاور باشد. و رأسی مثل  $w$  نیست که حذف آن گراف را ناهمبند کند.

۷- گزینه (۲). فقط گزاره اول را می‌توان با مرتبه  $O(\lg n)$  حل کرد که شبیه جستجوی دودویی است ولی دو گزاره دیگر مرتبه  $O(n)$  لازم دارند.

۸- گزینه (۴). مسائل آن پی سخت که روی گراف اجرا می‌شوند، روی درخت از مرتبه  $O(n)$  هستند. مثل یافتن طولانی‌ترین مسیر، رنگ‌آمیزی درخت (درخت با ۲ رنگ رنگ‌آمیزی می‌شود و این مسئله چندان جمله‌ای است)، یافتن بزرگترین مجموعه مستقل



- ۹- گزینه (۱). وقتی وزن‌های مساوی وجود دارد چندین MST ممکن است وجود داشته باشد. و کراسکال و پریم می‌توانند هر MST که وجود دارد را پیدا کنند. مثلاً کراسکال موقع سورت یالها دقیقاً به همان ترتیبی که قرار است MST مورد نظر حاصل شود سورت انجام دهد. یا پریم موقع انتخاب یال دقیقاً یالی را انتخاب کند که باید در MST حاصل باشد.
- ۱۰- گزینه (۲). فقط گزاره آخر درست است.
- ۱۱- گزینه (۴). فرض کنید مبلغ ۱۴ را می‌خواهیم خرد کنیم جواب بهینه دو سکه ۷ و ۷ است حال آنکه روش حریصانه ۵ سکه ۱۰ و ۱ و ۱ و ۱ را انتخاب می‌کند.
- ۱۲- گزینه (۲). هر عددی مثل  $x$  که اعداد قبل از آن از  $x$  کوچکتر و اعداد بعد از آن از  $x$  بزرگتر باشند می‌تواند محور بوده باشد یعنی اعداد ۴ و ۵ و ۹.
- ۱۳- گزینه (۳). در واقع باید طول بزرگترین زنجیر مینیمم باشد که برای  $m = ۱۱$  این رخ می‌دهد که البته تست تکراری است و بررسی آن بسیار وقت‌گیر است.
- ۱۴- گزینه (۲). بیشترین ارتفاع یعنی  $n - ۱$  وقتی حاصل می‌شود که جمع هر دو فراوانی کمینه خود کمینه یا دومین کمینه شود که اعداد فیبوناچی و تصاعد هندسی در مثال از این هستند.

## پاسخنامه فناوری اطلاعات ۱۳۹۹

- ۱۵- گزینه (۱).  $\lg(n!)$  با  $n \lg n$  هم رشد است و  $n!$  از  $n^n$  رشد کمتری دارد.
- ۱۶- گزینه (۴). حداکثر تعداد مقایسه وقتی است که تابع، آدرس  $۷$  را تولید کند که در این صورت، حفره‌های  $۷$  و  $۸$  و  $۰$  و  $۱$  و  $۲$  بررسی می‌شوند.
- ۱۷- گزینه (۲). چنین درختی فقط می‌تواند مورب راست باشد که چون BST است فقط یک حالت دارد و کلیدها اجازه ندارند جایگشت بزنند.
- ۱۸- گزینه (۳). به ازای هر عدد  $x$  در دنباله همه اعداد بعد از  $x$  باید از  $x$  کوچکتر باشند یا همگی از  $x$  بزرگتر باشند. در گزینه ۳، اگر  $x = ۱۵۷$  آنگاه برخی کلیدهای بعد از  $x$  از  $x$  بزرگتر و برخی (۱۴۳) کوچکتر هستند.
- ۱۹- گزینه (۴). آرایه گزینه یک خود هرم بیشینه است. گزینه ۲ فقط ۴۰ با ۷۰ تعویض شود هرم بیشینه می‌شود. در گزینه ۳ فقط ۲ با ۱۱ تعویض شود هرم بیشینه می‌شود. گزینه ۴ دو تعویض نیاز دارد، ۷۰ با ۱۰ و سپس ۷۰ با ۴۰.

۲۰- گزینه (۳). مرتب‌سازی ادغامی برای مرتب‌سازی  $n$  عدد همواره از مرتبه  $O(n \lg n)$  است. مرتب‌سازی شمارشی برای مرتب‌سازی  $n$  عدد صحیح در بازه  $۱$  تا  $k$  از مرتبه  $O(n+k)$  است. مرتب‌سازی سریع در حالت متوسط و بهترین از مرتبه  $n \lg n$  است و در بدترین حالت از مرتبه  $n^2$  است. مرتب‌سازی درجی در بهترین حالت از مرتبه  $n$  است و در متوسط و بدترین حالت از مرتبه  $n^2$  است.

۲۱- گزینه (۱). به راحتی قابل بررسی است.

۲۲- گزینه (۲). الگوریتم BFS و دایجسترا هر دو یک وظیفه دارند و آن یافتن کوتاهترین مسیر نسبت به یک رأس است. BFS برای گراف غیروزن‌دار یا وزن‌دار با وزن‌های مساوی و دایجسترا برای گراف وزن‌دار استفاده می‌شوند. دایجسترا هر درختی تولید کند حتماً BFS هم می‌تواند آن را تولید کند ولی عکس این موضوع صحت ندارد. ولی DFS این کاره نیست!

۲۳- گزینه (۲). اگر  $u$  قبل از  $v$  ملاقات شود یعنی فاصله  $r$  تا  $u$  کمتر از  $r$  تا  $v$  یعنی  $d(r, u) < d(r, v)$  است. توجه کنید  $d(r, u)$  با  $d(r, v)$  نمی‌تواند مساوی باشد چون  $u$  قبل از  $v$  ملاقات شده و نه همزمان.

۲۴- گزینه (۰). این تست حذف شود زیرا الگوریتم به اشتباه طرح شده و به جای نزولی گفته شده صعودی.

۲۵- گزینه (۳). اگر یک رأس مثل  $x$  مطلع شود همه رئوسی که از رأس  $x$  قابل دسترسی هستند مطلع می‌شوند توجه کنید

نمی‌توان گفت به تعداد مؤلفه‌های متصل قوی. مثلاً گراف مقابل دو مؤلفه متصل قوی دارد.  $\{a, b\}, \{c\}$  ولی اگر فقط  $a$  از خبر مطلع شود کافی است. ولی جواب به تعداد مؤلفه‌های همبند قوی است که ورودی از هیچ مؤلفه همبند قوی دیگر ندارد که در مورد گراف فوق، جواب یک رأس می‌شود.

۲۶- گزینه (۲). ماتریس  $B$  دارای  $n^2$  درایه است و می‌خواهیم درایه‌های قطر اصلی و بالای قطر را محاسبه کنیم یعنی  $\frac{n(n+1)}{2}$  درایه. پس حداقل  $\Omega(n^2)$  زمان نیاز است. حال هر درایه

با فرمول  $B[i, j] = \sum_{k=i}^j A[k]$  محاسبه می‌شود که این فرمول ظاهراً مرتبه  $n$  است ولی

می‌توان این فرمول را با مرتبه یک حساب کرد:

$$B[i, j] = A[i] + A[i+1] + \dots + A[j-1] + A[j] = B[i, j-1] + A[j]$$

پس کل محاسبات از مرتبه  $\theta(n^2)$  می‌شود.

## سؤال‌های چهار گزینه‌ای ساختمان داده و الگوریتم دکتری ۱۳۹۶

- ۱- کدام مورد، جواب رابطه بازگشتی  $T(n) = T(\sqrt{n}) + O(\log n)$  است؟  
 (۱)  $O(\log n)$  (۲)  $O(\log^2 n)$  (۳)  $O(\sqrt{n})$  (۴)  $O(n)$
- ۲- یک هرم کمینه با  $n$  عنصر متمایز داده شده است. می‌خواهیم به ازای عدد صحیح داده شده  $k$  ( $k \leq \sqrt{n}$ )،  $k$  آمین کوچک‌ترین عنصر را در این هرم پیدا کنیم (یعنی عددی که دقیقاً  $k-1$  عنصر از آن کوچکتر هستند)، با چه مرتبه زمانی این کار امکان‌پذیر است؟ (بهترین گزینه را انتخاب کنید).  
 (۱)  $O(n)$  (۲)  $O(\sqrt{n})$  (۳)  $O(k \log n)$  (۴)  $O(k \log k)$
- ۳- یک درخت دودویی جستجو شامل  $n$  عنصر داده شده است. با فرض دانستن محل عنصر  $x$  در این درخت، کوچکترین عنصر بزرگتر از  $x$  را در چه زمانی می‌توان در درخت به دست آورد؟  
 (فرض کنید تمام عناصر درخت متمایزند و درخت به صورت استاندارد و بدون هیچ گونه اطلاعات کمکی ذخیره شده است).  
 (۱)  $O(\log n)$  (۲)  $O(\log^2 n)$  (۳)  $O(n)$  (۴)  $O(1)$
- ۴- آرایه‌ای شامل  $n$  عدد صحیح داده شده است. به ازای  $1 \leq i \leq j \leq n$ ، مقدار  $c_{ij}$  را برابر مجموع مقادیر قرار گرفته در بازه  $i$  تا  $j$  از این آرایه تعریف می‌کنیم. می‌خواهیم میانگین تمام  $c_{ij}$ ‌های ممکن در این آرایه را پیدا کنیم. با چه مرتبه زمانی این کار امکان‌پذیر است؟ (فرض کنید چهار عمل اصلی در  $O(1)$  قابل انجام‌اند).  
 (۱)  $O(n \log n)$  (۲)  $O(n \log^2 n)$  (۳)  $O(n^2)$  (۴)  $O(n)$
- ۵- فرض کنید یک کاهش چندجمله‌ای از مسئله ۱ به مسئله ۲ داریم. کدام مورد، درست است؟  
 (۱) اگر مسئله ۲ ان پی - سخت باشد، آنگاه مسئله ۱ ان پی - تمام است.  
 (۲) اگر مسئله ۱ ان پی - تمام باشد، آنگاه مسئله ۲ نیز ان پی - تمام است.  
 (۳) اگر مسئله ۱ ان پی - تمام باشد، آنگاه مسئله ۲ نیز ان پی - سخت است.  
 (۴) اگر مسئله ۲ ان پی - سخت باشد، آنگاه مسئله ۱ نیز ان پی - سخت است.

- ۶- کدام مورد در خصوص الگوریتم دایکسترا درست است؟  
 (۱) هزینه سرشکن به روزرسانی هر رأس  $O(1)$  است.  
 (۲) هزینه سرشکن به روزرسانی هر رأس  $O(n)$  است.  
 (۳) هزینه سرشکن به روزرسانی هر رأس  $O(m/n)$  است.  
 (۴) فاصله هر رأس تا مبدأ در طول الگوریتم دقیقاً یک بار به روز می‌شود.
- ۷- کدام یک از توابع درهم‌سازی زیر یکنوا (uniform) است؟ (فرض کنید اندازه جدول درهم‌سازی  $k$  است.)  
 (۱)  $h(x) = kx \bmod (k-1)$   
 (۲)  $h(x) = (k-1)x \bmod k$   
 (۳)  $h(x) = x \bmod (k-1)$   
 (۴)  $h(x) = x^2 \bmod k$
- ۸- چه تعداد از گزاره‌های زیر، درست است؟  
 - اگر وزن تمام یال‌های یک گراف با مقدار مثبت  $C$  جمع شود، کوتاه‌ترین مسیرها تغییر نمی‌کند.  
 - اگر وزن تمام یال‌های یک گراف در مقدار مثبت  $C$  ضرب شود، کوتاه‌ترین مسیرها تغییر نمی‌کند.  
 - اگر وزن تمام یال‌های یک گراف با مقدار منفی  $C$  جمع شود، کوتاه‌ترین مسیرها تغییر نمی‌کند.  
 - اگر وزن تمام یال‌های یک گراف در مقدار منفی  $C$  ضرب شود، کوتاه‌ترین مسیرها تغییر نمی‌کند.
- ۹- میانگین ارتفاع درخت DFS بر روی یک گراف کامل با فرض آنکه رأس شروع تصادفی انتخاب شده است از چه مرتبه‌ای است؟  
 (۱)  $O(1)$  (۲)  $O(n)$  (۳)  $O(\sqrt{n})$  (۴)  $O(\log n)$
- ۱۰- شبکه‌ای متشکل از  $n$  رأس، دو رأس معین  $s$  و  $t$  داده شده است. فرض کنید ظرفیت تمام یال‌های شبکه نامتناهی است. به ازای یک شار  $f$  از  $s$  به  $t$ ، یالی که بیشترین شار از آن عبور می‌کند را یال تنگنا و مقدار شار عبوری از آن یال را «تنگنای» شار  $f$  می‌نامیم. می‌خواهیم به ازای یک مقدار صحیح  $C$  داده شده، شاری با مقدار  $C$  را با کمترین تنگنا از  $s$  به  $t$  منتقل کنیم. با چند بار استفاده از الگوریتم فورد - فالکرسن می‌توان این شار را به دست آورد؟ (بهترین گزینه را انتخاب کنید.)  
 (۱)  $O(n \log C)$  (۲)  $O(\log C)$  (۳)  $O(n)$  (۴)  $O(1)$

۱۱- در مرتب‌سازی آرایه‌ای به طول  $N$  با الگوریتم‌های Randomized Insertion sort، MergeSort، quicksort میزان استفاده از پشته فراخوانی (Callstack) به ترتیب از چه مرتبه‌ای است؟

(۱)  $O(1)$ ,  $O(\log_2 N)$ ,  $O(1)$

(۲)  $O(\log_2 N)$ ,  $O(\log_2 N)$ ,  $O(1)$

(۳)  $O(N \log_2 N)$ ,  $O(N \log_2 N)$ ,  $O(N^2)$

(۴)  $O(N \log_2 N)$ ,  $O(N \log_2 N)$ ,  $O(1)$

۱۲- وزارت ارشاد قصد دارد یک کتاب داستان آموزنده را از زبان انگلیسی به زبان‌های رایج در ایران ترجمه و منتشر نماید. هزینه ترجمه یک صفحه بین هر دو زبان به هزار تومان در جدول زیر داده شده است. اگر این کتاب صد صفحه داشته باشد، کمترین هزینه ترجمه آن به همه زبان‌ها چند تومان است؟

|         | لری | عربی | کردی | ترکی | فارسی |
|---------|-----|------|------|------|-------|
| انگلیسی | ۸   | ۵    | ۸    | ۷    | ۵     |
| فارسی   | ۱   | ۲    | ۱    | ۱    | ۰     |
| ترکی    | ۳   | ۲    | ۲    | ۰    |       |
| کردی    | ۲   | ۵    | ۰    |      |       |
| عربی    | ۱۰  | ۰    |      |      |       |

(۱) یک میلیون (۲) سه میلیون و سیصد هزار

(۳) پنج میلیون (۴) هشت میلیون

۱۳- آرایه‌ای به طول  $n$  داده شده که  $n$  توان درست ۲ است. الگوریتم زیر را در نظر بگیرید:  
۱- لیست را به  $n/k$  زیر لیست  $k$  تائی تقسیم کنید. هر زیر لیست را با Insertion sort مرتب کنید.

۲- متغیر  $i$  را برابر ۲ قرار دهید.

۳- تا زمانی که  $k \times i$  کوچکتر یا مساوی  $n$  است، عملیات زیر را تکرار کنید:

۳-۱- آرایه را به صورت قسمت‌های  $k \times i$  در نظر بگیرید.

۳-۲- هر قسمت را از وسط به دو زیر لیست تقسیم کرده و آن‌ها را با هم ادغام merge کنید.

۳-۳- متغیر  $i$  را دو برابر کنید.

هزینه الگوریتم در بدترین حالت، کدام است؟

$$\theta(n \cdot \log n) \quad (۱) \quad \theta\left(\left(\frac{n}{k}\right) \cdot \log\left(\frac{n}{k}\right)\right) \quad (۲)$$

$$\theta\left(nk + n \cdot \log\left(\frac{n}{k}\right)\right) \quad (۳) \quad \theta\left(nk + \left(\frac{n}{k}\right) \cdot \log\left(\frac{n}{k}\right)\right) \quad (۴)$$

۱۴- پیمایش Preorder و Postorder یک درخت دودویی داده شده است. پیمایش inorder آن، کدام است؟

Preoder : fgbceda

Postorder : gedcabf

(۱) gfecdba (۲) gfecabd (۳) gfebcbda (۴) نمی توان به دست آورد.

۱۵- استفاده از کدام داده ساختار، در مرتب سازی ادغامی (mergesort) به پیچیدگی  $O(n \log n)$  منجر می شود؟

(i) لیست پیوندی یک طرفه (ii) لیست پیوندی دو طرفه (iii) آرایه

(۱) فقط ii (۲) فقط iii (۳) i و ii (۴) هر سه مورد

۱۶- در صورتی که یک آرایه مرتب شده (صعودی) داشته باشیم، کدام الگوریتم مرتب سازی بهترین عملکرد را دارد؟

(۱) ادغامی (۲) درجی (۳) سریع (۴) هیپ

۱۷- فرض کنید که  $2n+1$  عدد داریم و می دانیم که هر کدام از این اعداد دقیقاً دو بار آمده است به جز یک عدد. پیچیدگی زمانی الگوریتمی که عدد یکتا را تعیین کند چقدر است؟ فرض کنید اعمال رایج روی دو عدد در  $O(۱)$  انجام می شود.

$$\theta(n \log n) \quad (۱) \quad O(\log n) \quad (۲) \quad O(n^2) \quad (۳) \quad O(n) \quad (۴)$$

۱۸- مرتبه زمانی قطعه کد زیر، کدام است؟

for k = n Downto n-۱۰۰۰

```
{ j = ۱;
  while (j <= n)
  { j = j * ۲;
    i = ۰;
    b = ۱;
    while (b == ۱ and i < j)
    { if (i + j) % ۲ == ۰
      { b = ۰;
        i ++;
      }
    }
  }
}
```

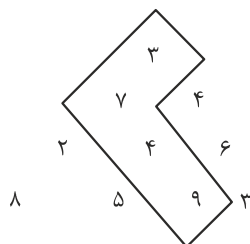
$$O(n^2) \quad (۱) \quad O(\log n) \quad (۲) \quad O(n \log n) \quad (۳) \quad O(n(\log n)^2) \quad (۴)$$

۱۹- کدام مورد، خروجی تابع زیر برای ورودی  $f(2,5)$  است؟

```
int f(int n, int m)
{
    if (m < n)
        return ۲ × m;
    else
        return (f(min(m, n), max(m, n) - ۱) + (۳ × n));
}
```

(۱) ۲۰ (۲) ۲۶ (۳) ۴۴ (۴) خاتمه نمی‌یابد

۲۰- مثلثی از اعداد صحیح و مثبت در  $n$  ردیف به صورت زیر داده شده است. از رأس مثلث شروع کرده و در هر قدم به عدد مجاور در سطر پایین حرکت می‌کنیم. هدف پیدا کردن مسیری حداکثری از مجموع اعداد هم‌جوار است. (برای مثال در شکل زیر مسیر حداکثری به طول ۲۳ نشان داده شده است.) هزینه زمانی بهترین الگوریتمی که می‌توان برای یافتن این مسیر حداکثری نوشت، کدام است؟



(۱)  $O(n^2 \log n)$

(۲)  $O(n \log n)$

(۳)  $O(n^2)$

(۴)  $O(2^n)$

## پاسخنامه سؤال‌های چهارگزینه‌ای ساختمان داده و الگوریتم دکتری ۱۳۹۶

۱- گزینه (۱).

$$n = 2^k \rightarrow T(2^k) = T(2^{\frac{k}{2}}) + k \xrightarrow{T(2^k)=F(k)} F(k) = F\left(\frac{k}{2}\right) + k = \theta(k)$$

$$\Rightarrow T(n) = \theta(\lg n)$$

۲- گزینه (۴). یک هرم جدید تهی بسازید و سپس ریشه هرم اصلی به همراه اندیش را در هرم جدید درج کنید و حال  $k$  بار این عمل را تکرار کنید: از هرم جدید ریشه‌اش را حذف کنید و فرزندان او را از هرم اصلی به همراه اندیششان در هرم جدید درج کنید.  $k$  آمین عمل حذف از هرم جدید  $k$  آمین مینیمم است. مرتبه این عملیات  $O(k \lg k)$  است چون ارتفاع هرم جدید حداکثر  $\lg k$  است و  $k$  بار عملیات درج و حذف در آن انجام می‌شود.

۳- گزینه (۳). عمل یافتن  $\text{succ}(x)$  از مرتبه  $O(h)$  است که چون BST ارتفاعش  $O(n)$  است پس مرتبه عمل  $O(n)$  است. توجه کن BST متوازن نیست.

۴- گزینه (۴). در واقع سوال  $\frac{\sum c_{ij}}{p}$  را خواسته که  $p$  تعداد همه  $c_{ij}$ ‌های ممکن است که یک

$$\text{عدد مشخص و قابل محاسبه است (با ترکیب با تکرار ثابت کن } (p = \binom{n+1}{2})$$

هر عنصر  $A[i]$  به تعداد بار مشخصی در  $\sum$  ظاهر می‌شود که قابل محاسبه است فرض کن

$$\frac{\sum_{i=1}^n x_i \times A[i]}{p}$$

تعداد باری که  $A[i]$  در مجموع ظاهر می‌شود  $x_i$  است پس سوال از ما را

خواسته که این محاسبه  $n$  عمل جمع و ضرب دارد و از مرتبه  $\theta(n)$  است.

۵- گزینه (۳). مسئله ۲ از مسئله ۱ سخت‌تر است یا در اندازه مسئله ۱ است. اگر مسئله ۲ از پی سخت باشد. مسئله ۱ می‌تواند هرچه باشد  $(Npc, p)$  پس گزینه ۱ و ۴ غلط هستند. اگر مسئله ۱ از پی تمام است آنگاه مسئله ۲ حتماً از پی سخت است (از پی تمام نیز بخشی از آن پی سخت است) پس گزینه ۳ درست است.

۶- گزینه (۳). فرض کن  $n$  راس و  $m$  یال داریم دایجسترا  $m$  بار عمل به روزرسانی انجام می‌دهد

$$\text{که سهم هر رأس می‌شود } \frac{m}{n}.$$

۷- گزینه (۲). تابع  $h(x) = x \bmod k$  یکنواست چون همه آدرس‌های  $\{0, 1, 2, \dots, k-1\}$  را با شانس مساوی تولید می‌کند. ولی تابعی که مُد  $k-1$  می‌گیرد نمی‌تواند آدرس  $k-1$  را



تولید کند پس گزینه ۱ و ۳ غلط هستند. تابع  $h(x) = x^2 \bmod k$  نیز بسیاری از آدرس‌ها را تولید نمی‌کند که با همنهشتی می‌شود اثبات کرد یا به ازای مثلاً  $k = 5$  و کلیدهای ۰ و ۱ و ۲ و ۳ و ۴ بررسی کنید که این تابع آدرس‌های ۰ و ۱ و ۴ و ۹ را تولید می‌کند و آدرس‌های ۲ و ۳ را تولید نمی‌کند. گزینه ۲ با  $h(x) = -x \bmod k$  مساوی است که مثل  $x \bmod k$  یکنواست.

۸- گزینه (۱). اگر وزن یالها با عدد  $c$  جمع یا تفریق شود، کوتاهترین مسیرها ممکن است تغییر کند ولی اگر در عدد مثبت  $c$  ضرب شود کوتاهترین مسیرها (نه اندازه آنها) تغییر نمی‌کند. فقط جمله دوم درست است.

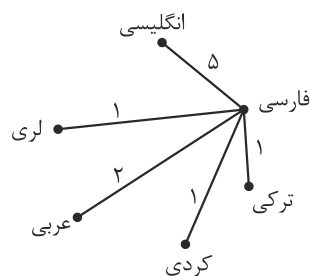
۹- گزینه (۲). چون گراف کامل است از هر رأس شروع کنید همه رئوس بدون بازگشت ملاقات می‌شوند و یک درخت به ارتفاع  $\theta(n)$  حاصل می‌شود.

۱۰- گزینه (۴). ابتدا ظرفیت همه یالها را یک قرار می‌دهیم و شار بیشینه را می‌یابیم (با اجرای فورד فالکرسون) فرض کن شار بیشینه  $m$  شود که این شار  $m$  دارای تنگنای یک است.

حال ظرفیت یالها را در  $\frac{c}{m}$  ضرب می‌کنیم و با این کار مقدار شار بیشینه  $m \times \frac{c}{m} = c$  می‌شود و تنگنای آن  $\frac{c}{m}$  است. پس با  $O(1)$  بار اجرای فورد فالکرسون به جواب رسیدیم.

۱۱- گزینه (۲). مرتب‌سازی درجی که اصلاً نیازی به پشته ندارد. مرتب‌سازی سریع تصادفی و ادغامی به  $O(\lg n)$  اندازه پشته نیاز دارند.

۱۲- گزینه (۱). جواب سوال MST است:



$$\Rightarrow 10 \Rightarrow 10 \times 100 = 1000$$

پس هزینه ۱۰۰۰ هزار تومان یعنی یک میلیون تومان است.

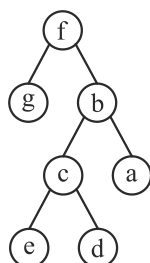
۱۳- گزینه (۳). الگوریتم گفته شده همان مرتب‌سازی ادغامی

است که لیست‌ها به اندازه  $k$  تقسیم شده‌اند و سپس این

لیست‌ها با درجی مرتب می‌شوند و ادغام می‌شوند فقط این الگوریتم از پایین به بالاست.

مرتبه مرتب‌سازی  $\frac{n}{k}$  زیرلیست  $k$  تایی با درجی  $k^2 \times \frac{n}{k}$  است و سپس ادغام کردن‌ها نیاز به

زمان  $n \lg \frac{n}{k}$  دارد.



۱۴- گزینه (۱). گره تک فرزندی وجود ندارد (چرا؟) و تعداد ۴ برگ و ۳ گره دو فرزندی وجود دارد (چرا؟) و گره‌های geda برگ هستند (چرا؟) و می‌توان درخت را با الگوریتم‌هایی که می‌شناسیم بکشیم:

$\Rightarrow \text{in: g f e c d b a}$

۱۵- گزینه (۴). مرتب‌سازی ادغامی روی هر ساختاری مثل آرایه و لیست پیوندی هر نوعی از مرتبه  $O(n \lg n)$  است چون ادغام لیست‌های مرتب چه آرایه و چه لیست پیوندی باشند زمان یکسان می‌برد.

۱۶- گزینه (۲). درجی برای آرایه مرتب شده صعودی بهترین عملکرد را دارد.

۱۷- گزینه (۴). یک روش جالب این است که همه اعداد آرایه را با هم XOR کنید، عنصر تک خودش را لو می‌دهد!

۱۸- گزینه (۲). حلقه for،  $\theta(1)$  است چون ۱۰۰۰ بار اجرا می‌شود. حلقه while اول مرتبه‌اش  $\theta(\lg n)$  است و حلقه while دوم  $\theta(1)$  است (چرا؟) پس کلاً  $\theta(\lg n)$  است.

۱۹- گزینه (۲).

$$f(2, 5) = f(2, 4) + 3 \times 2$$

$$f(2, 4) = f(2, 3) + 3 \times 2$$

$$f(2, 3) = f(2, 2) + 3 \times 2$$

$$f(2, 2) = f(2, 1) + 3 \times 2$$

$$f(2, 1) = 2 \times 1 = 2 \Rightarrow f(2, 2) = 8 \Rightarrow f(2, 3) = 14 \Rightarrow f(2, 4) = 20 \Rightarrow f(2, 5) = 26$$

۲۰- گزینه (۳).  $n$  ردیف که جمعاً شامل  $\frac{n(n+1)}{2} = \theta(n^2)$  عدد هستند و برای یافتن جواب

می‌توان با یک روش پویا از پایین‌ترین ردیف شروع کنیم به ازای هر عدد با زمان  $\theta(1)$  جواب را در آن عدد ذخیره کنیم که کل زمان  $\theta(n^2)$  می‌شود.

## سؤال‌های چهارگزینه‌ای ساختمان داده و الگوریتم دکتری ۱۳۹۷

- ۱- میزان رشد توابع زیر به ترتیب صعودی (از چپ به راست) کدام است؟  
 $n \log^*(n), \log(n)^{\log(n)}, \log(n!), \log(\log(n^n))$   
 (۱)  $n \log^*(n), \log(n!), \log(n)^{\log(n)}, \log(\log(n^n))$   
 (۲)  $\log(\log(n^n)) \log(n!), n \log^*(n), \log(n)^{\log(n)}$   
 (۳)  $\log(\log(n^n)), \log(n!), \log(n)^{\log(n)}, n \log^*(n)$   
 (۴)  $\log(\log(n^n)), n \log^*(n), \log(n!), \log(n)^{\log(n)}$
- ۲- جواب دو رابطه بازگشتی زیر کدام است؟  
 $T(n) = T(3/7n) + T(4/7n) + n, T(1) = 1$   
 $T'(n) = T'(2/7n) + T'(4/7n) + n, T'(1) = 1$   
 (۱)  $T(n) = \Theta(n), T'(n) = \Theta(n)$   
 (۲)  $T(n) = \Theta(n), T'(n) = \Theta(n \log n)$   
 (۳)  $T(n) = \Theta(n \log n), T'(n) = \Theta(n)$   
 (۴)  $T(n) = \Theta(n \log n), T'(n) = \Theta(n \log n)$
- ۳- فرض کنید یک زبان از حروف الفبای  $\{a, b, c, d, e, f, g, h, i\}$  تشکیل شده است و احتمال وقوع  $a$  برابر ۱۸ درصد،  $b$  برابر ۴ درصد،  $c$  برابر ۸ درصد،  $d$  برابر ۱۰ درصد،  $e$  برابر ۲۰ درصد،  $f$  برابر ۵ درصد،  $g$  برابر ۵ درصد،  $h$  برابر ۱۵ درصد و  $i$  برابر ۱۵ درصد است. درخت هافمن این زبان چند گره دارد؟  
 (۱) ۱۸ (۲) ۱۷ (۳) ۱۶ (۴) ۱۵
- ۴- فرض کنید  $H: \{1, \dots, n\} \rightarrow \{1, \dots, n\}$  یک تابع درهم‌ساز یکنواخت باشد. برای ورودی  $x$  عدد  $z$  را برابر تعداد صفرهای سمت راست  $H(x)$  قرار می‌دهیم. برای عدد  $0 \leq c \leq 1$ ، احتمال  $z \geq c \log n$  از چه مرتبه‌ای است؟ (فرض کنید  $c$  ثابت است).  
 (۱)  $O(1/n)$  (۲)  $O(1/n^c)$  (۳)  $O(1/\log n)$  (۴)  $O(1/\log^c n)$
- ۵- چه تعداد از تبدیل‌های زیر در زمان  $O(n)$  قابل انجام است؟
- تبدیل پیمایش پیش ترتیب عناصر یک درخت دودویی کامل به پیمایش پس‌ترتیب آن
  - تبدیل پیمایش پس‌ترتیب یک درخت دودویی کامل به پیمایش پیش‌ترتیب آن

- تبدیل پیمایش میان ترتیب عناصر یک درخت دودویی کامل به یک درخت دودویی

جست‌وجو

۳ (۱) ۲ (۲) ۱ (۳) ۰ (۴)

- ۶- در یک داده ساختار هرم با  $n$  عنصر، عدد بعدی یک رأس (عددی که در دنباله‌ی مرتب شده بعد از عدد این رأس می‌آید) را در چه زمانی می‌توان به دست آورد؟

۱ (۱)  $O(1)$  ۲ (۲)  $O(n)$  ۳ (۳)  $O(\sqrt{n})$  ۴ (۴)  $O(\log n)$

- ۷- اعداد ۱ تا ۱۵ درون آرایه  $A$  به گونه‌ای ذخیره شده‌اند که تشکیل یک هرم کمینه متوازن می‌دهند. حداکثر تعداد نابه‌جایی‌های  $A$  چه تعداد است؟

(دو درایه  $A[i]$  و  $A[j]$  تشکیل یک نابه‌جایی می‌دهند اگر  $i < j$  و  $A[i] > A[j]$ )

۱ (۱) ۱۱۰ ۲ (۲) ۹۴ ۳ (۳) ۷۱ ۴ (۴) ۵۹

- ۸- آرایه  $A$  از  $n$  عدد دلخواه داده شده است. فرض کنید عملیات  $\text{reverse}(i, j)$  برای  $1 \leq i < j \leq n$ ، زیر آرایه  $A[i..j]$  را معکوس می‌کند، یعنی به ازای هر  $0 \leq k \leq j - i$ ،  $A[j - k]$  را با  $A[i + k]$  تعویض می‌کند. با حداقل چند بار استفاده از این عملیات می‌توان آرایه  $A$  را مرتب کرد؟

۱ (۱)  $O(n \log n)$  ۲ (۲)  $O(n\sqrt{n})$  ۳ (۳)  $O(n^2)$  ۴ (۴)  $O(n)$

- ۹- آرایه  $A$  شامل  $n$  عدد مختلف است. حال می‌خواهیم آرایه  $B$  را به این صورت پر کنیم که به ازای هر  $i$ ،  $B[i]$  برابر با میانه اعداد  $A[1]$  تا  $A[i]$  باشد. بهترین الگوریتم برای این کار از چه مرتبه‌ای است؟

۱ (۱)  $O(n^2)$  ۲ (۲)  $O(n\sqrt{n})$  ۳ (۳)  $O(n \log n)$  ۴ (۴)  $O(n^2 \log n)$

- ۱۰- فرض کنید گراف  $G$  یک گراف جهت‌دار و وزن‌دار است که دور منفی ندارد. رئوس این گراف را با اعداد ۱ تا  $n$  برچسب‌گذاری می‌کنیم و وزن یال از  $i$  به  $j$  را با  $w(i, j)$  نشان می‌دهیم. اگر گراف  $G'$  همان گراف  $G$  باشد، که فقط وزن یال‌های آن که با  $w'$  نشان می‌دهیم، طبق قاعده‌های زیر تغییر کرده است، به ازای چند تا از این قاعده‌ها، کوتاه‌ترین مسیر (خود مسیر نه طول مسیر) بین هر دو رأس داده شده در دو گراف  $G$  و  $G'$  یکسان است؟

$$w'(i, j) = w(i, j) + i - j$$

$$w'(i, j) = w(i, j) + j - i$$

$$w'(i, j) = w(i, j) + i + j$$

۰ (۱) ۱ (۲) ۲ (۳) ۳ (۴)

۱۱- الگوریتمی را در نظر بگیرید که ورودی  $a_1, \dots, a_n$  عدد مجزا را به ترتیب داده شده می‌خواند و هنگام خواندن  $a_i$  مقدار متغیر  $x$  را به احتمال  $1/i$  برابر  $a_i$  قرار می‌دهد. الگوریتم در پایان مقدار  $x$  را به عنوان خروجی گزارش می‌کند. با چه احتمالی خروجی الگوریتم برابر  $a_i$  است؟

- (۱) می‌تواند هر مقداری کوچکتر یا مساوی  $1/i$  باشد.
- (۲) می‌تواند هر مقداری در بازه  $[1/n, 1/i]$  باشد.
- (۳)  $1/i$
- (۴)  $1/n$

۱۲- در گراف جهت‌دار  $G$  وزن یال‌ها را یک قرار می‌دهیم و شار بیشینه از رأس  $s$  به  $t$  را محاسبه می‌کنیم. مقدار جریان بیشینه برابر کدام مورد است؟

- (۱) تعداد مسیرهای بین  $s$  و  $t$
- (۲) تعداد کوتاه‌ترین مسیرهای بین  $s$  و  $t$
- (۳) تعداد مسیرهای مجزای یالی بین  $s$  و  $t$
- (۴) تعداد مسیرهای مجزای رأسی بین  $s$  و  $t$

۱۳- چند تا از گزاره‌های زیر درست است؟

- اگر مسئله تصمیم‌گیری  $X$  در ان پی باشد، مسئله تصمیم‌گیری  $\text{not } X$  نیز در ان پی است.
- هر مسئله ان پی - سخت به یک مسئله ان پی - کامل قابل کاهش است.
- تمام مسائل ان پی - کامل به تمام مسائل ان پی - سخت قابل کاهش‌اند.

- (۱) ۰ (۲) ۱ (۳) ۲ (۴) ۳

۱۴- فرض کنید گراف جهت‌دار  $G$  شامل  $n$  رأس و  $m$  یال، روابط دوستی بین  $n$  فرد را مدل می‌کند. به عبارت دقیق‌تر از  $u$  به  $v$  یال وجود دارد، اگر شخص  $u$  شخص  $v$  را بشناسد. در این شبکه هرگاه شخص از یک خبر مطلع شود آن را به اطلاع همه دوستان خود می‌رساند. می‌خواهیم یک خبر مشخص را به اطلاع همه برسانیم. می‌خواهیم کمترین تعداد افرادی را پیدا کنیم که با مطلع شدن آن‌ها، همه از خبر فوق مطلع شوند. بهترین الگوریتم برای محاسبه این مجموعه افراد از چه مرتبه‌ای است؟

- (۱)  $O(m+n)$
- (۲)  $O(\min(n, m))$
- (۳)  $O(n \log n + m)$
- (۴) این مسئله ان پی - سخت است.

۱۵- در جست‌وجوی سطح اول (BFS)، به هر رأس یک بازه زمانی نسبت می‌دهیم، طوری که زمان قرار دادن رأس در صف شروع بازه و زمان برداشتن آن از صف انتهایی بازه باشد. کدام مورد درست است؟

- (۱) ترتیب طول بازه‌های بین رئوس، معادل با ترتیب فاصله‌ی آنان از رأس شروع است.
- (۲) اگر یک رأس از نوادگان رأس دیگر در درخت BFS باشد، آنگاه بازه‌ی آن‌ها اشتراک ندارد.
- (۳) اگر بازه‌ی دو رأس اشتراک نداشته باشند، آنگاه یکی از آنها نوه‌ی دیگری در درخت BFS است.
- (۴) اگر شروع بازه‌ی  $b$  بعد از شروع بازه‌ی  $a$  باشد، فاصله‌ی رأس شروع تا  $b$  بیشتر از فاصله‌ی رأس شروع تا  $a$  است.

۱۶- فرض کنید  $T = (V, E)$  یک درخت شامل  $n$  رأس باشد. می‌خواهیم  $C \subseteq V$  با حداقل تعداد رأس پیدا کنیم، طوری که برای هر یال در  $E$ ، حداقل یکی از دو سر آن یال در  $C$  باشد. الگوریتم حریصانه زیر را در نظر بگیرید.  $C$  را در ابتدا تهی قرار می‌دهیم. در هر مرحله رأس با درجه بیشینه در  $T$  را در  $C$  قرار می‌دهیم (در صورتی که بیش از یک رأس با درجه بیشینه بود، یکی از آنها را به دلخواه انتخاب می‌کنیم) و تمام یال‌های مجاور آن را از  $T$  حذف می‌کنیم. این کار را تا زمانی که  $T$  تهی از یال شود ادامه می‌دهیم. کمترین مقدار  $n$  که برای آن، الگوریتم حریصانه فوق درست کار نمی‌کند، کدام است؟

(۱) ۳

(۲) ۴

(۳) ۵

(۴) برای هر  $n$  الگوریتم حریصانه جواب بهینه را برمی‌گرداند.

۱۷- گراف  $G = (V, E)$  با وزن‌های مثبت را در نظر بگیرید.  $d(u, v, k)$  را برابر طول کوتاه‌ترین مسیر از  $u$  به  $v$  در نظر بگیرید که حداکثر  $k$  یال داشته باشد. چندتا از رابطه‌های بازگشتی زیر برای  $k > 1$  درست می‌باشند؟  
 $w = (u, v)$  وزن یال  $(u, v)$  را نشان می‌دهد و وزن یالی که وجود نداشته باشد بی‌نهایت در نظر گرفته می‌شود.

$$d(u, v, k) = \min_{x \in V} (d(u, v, k-1), d(u, x, k-1) + w(x, v))$$

$$d(u, v, k) = \min_{x \in V} (d(u, x, k-1) + w(x, v))$$

$$d(u, v, k) = \min_{x \in V} (d(u, v, k-1), d(u, x, \lceil k/2 \rceil) + d(x, v, \lfloor k/2 \rfloor))$$

○ (۴)

۱ (۳)

۲ (۲)

۳ (۱)

۱۸- در یک گراف همبند با  $n$  رأس و  $m$  یال، وزن یال‌ها ۱ یا ۲ هستند. بهترین الگوریتم برای یافتن درخت پوشای کمینه این گراف دارای چه زمان اجرایی است؟

- (۱)  $O(n \log n + m)$  (۲)  $O(n \log n)$   
(۳)  $O(n)$  (۴)  $O(m)$

۱۹- سعید و وحید بازی زیر را انجام می‌دهند. ابتدا دو بازیکن روی یک عدد  $n$  توافق می‌کنند. بازی با  $x = 2$  آغاز می‌شود. بازیکن اول با انداختن سکه انتخاب شده و بازی را آغاز می‌کند. از این به بعد هر کدام از بازیکنان به دلخواه  $x$  را به توان ۲ یا ۳ می‌رساند. مثلاً اگر سعید بازی را آغاز کرده و ۳ را انتخاب کند،  $x$  برابر با ۸ خواهد شد. بعد از حرکت هر کدام از بازیکنان که  $x > n$  شود، بازی متوقف شده و بازیکن آخر به عنوان برنده بازی انتخاب می‌شود. بازی‌ها به طور متوسط در چند مرحله به پایان می‌رسند؟

- (۱)  $\Theta(\log \log n)$  (۲)  $\Theta(\log^2 n)$  (۳)  $\Theta(\log n)$  (۴) قابل محاسبه نیست.

۲۰- می‌خواهیم ماتریس‌های  $M_1(5 \times 10)$  و  $M_2(10 \times 3)$  و  $M_3(3 \times 12)$  و  $M_4(12 \times 5)$  را با همین ترتیب درهم ضرب کنیم. این کار با حداقل چند ضرب عددی قابل انجام است؟

- (۱) ۳۶۰ (۲) ۴۰۵ (۳) ۵۸۰ (۴) ۶۳۰

## پاسخنامه سؤال‌های چهارگزینه‌ای ساختمان داده و الگوریتم دکتری ۱۳۹۷

۱- گزینه (۴). فقط مشکل سؤال این است که منظور از  $\log(n)^{\log(n)}$  به شکل  $(\log n)^{\log n}$  است:

$$\lg(\lg n^n) = \lg(n \lg n) < n \lg^* n < \lg(n!) \sim n \lg n < (\lg n)^{\lg n} \quad \text{گزینه (۳).}$$

$$T(n) = T\left(\frac{3n}{4}\right) + T\left(\frac{n}{4}\right) + n = \theta(n \lg n) \quad \left(\frac{3}{4} + \frac{1}{4} = 1\right)$$

$$T'(n) = T'\left(\frac{3n}{4}\right) + T'\left(\frac{n}{4}\right) + n = \theta(n) \quad \left(\frac{3}{4} + \frac{1}{4} = \frac{6}{4} < 1\right)$$

۳- گزینه (۲). اصلاً نیاز به رسم درخت نیست. ۹ تا کاراکتر پس ۹ تا برگ پس ۸ تا گره داخلی در نتیجه ۱۷ تا نود وجود دارد.

۴- گزینه (۲). برای درک یک آزمایش می‌کنم. فرض کن  $n = ۸$  پس  $H(x) \in \{1, 2, \dots, ۸\}$  حال  $H(x)$  های ممکن را باینری می‌نویسیم و در هر حالت  $z$  را مشخص می‌کنیم:

$$00001 \quad z = 0$$

$$00010 \quad z = 1$$

$$00011 \quad z = 0$$

$$00100 \quad z = 2$$

$$00101 \quad z = 0$$

$$00110 \quad z = 1$$

$$00111 \quad z = 0$$

$$01000 \quad z = 3$$

حال اگر  $c = 0$  آنگاه باید  $z \geq 0$  که احتمال آن ۱۰۰٪ است (پس گزینه ۱ و ۳ غلط)

حال اگر  $c = 1$  آنگاه  $z \geq \lg ۸ = ۳$  که در این مثال فقط یک عدد از ۸ عدد هست که

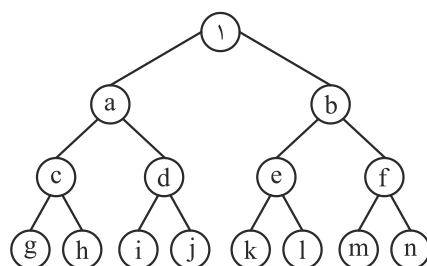
$$z \geq ۳ \quad \text{پس احتمال آن } \frac{1}{۸} \text{ است که گزینه ۲ حاصل می‌شود.}$$

۵- گزینه (۲). گزاره‌های ۱ و ۲ درست هستند چون درخت کامل است و شکل مشخصی دارد. ولی گزاره ۳ غلط است با این که درخت کامل است ولی BST است و مرتبه  $n \lg n$  می‌خواهد. این تست در کلید اولیه گزینه ۱ ولی در کلید نهایی کلید درست شد و گزینه ۲ شد.

۶- گزینه (۲). در هرم عملیاتی مثل succ و pred و جستجو از مرتبه  $O(n)$  هستند.



۷- گزینه (۳). من نظر طراح را می‌گویم ولی تست کلاً مشکل داره و جواب درست نداره! حداکثر وارونگی وقتی است که ۱ در  $A[1]$  باشد (که باید باشد) ولی سایر عناصر طوری باشند که:



$a > b \text{ e f k l m n}$  ۷  
 $b > c \text{ d g h i j}$  ۶  
 $c > d \text{ e f i j k l m n}$  ۹  
 $d > e \text{ f g h k l m n}$  ۸  
 $e > f \text{ g h i j m n}$  ۷  
 $f > g \text{ h i j k l}$  ۶  
 $g > h \text{ i j k l m n}$  ۷  
 $h > i \text{ j k l m n}$  ۶  
 $i > j \text{ k l m n}$  ۵  
 $j > k \text{ l m n}$  ۴  
 $k > l \text{ m n}$  ۳  
 $l > m \text{ n}$  ۲  
 $m > n$  ۱

که جلوی هر یک تعداد وارونگی‌ها ذکر شده که جمعاً ۷۱ می‌شود. ولی این شرایط همه با هم نمی‌توانند برقرار باشند که طراح توجه نکرد.

۸- گزینه (۴). کافی است مینیمم را پیدا کنیم (که هزینه‌ای برای آن در سوال ذکر نشده) و اگر مینیمم در خانه  $k$  باشد سپس  $\text{reverse}(1, k)$  را اجرا کنیم که مینیمم به خانه اول بیاید. حال مینیمم دوم را پیدا کنیم و اگر در خانه  $t$  باشد  $\text{reverse}(2, t)$  را اجرا کنیم که ایشون به خانه دوم بیاید و به همین ترتیب  $n$  بار عملیات  $\text{reverse}$  انجام می‌شود.

۹- گزینه (۳). به ازای  $i$  از ۱ تا  $n$  این عمل را انجام می‌دهیم: عنصر  $A[i]$  را در یک درخت مرتبه آماری درج می‌کنیم سپس میانه درخت را یافته و در  $B[i]$  ذخیره می‌کنیم. درج و یافتن میانه در درخت مرتبه آماری از مرتبه  $\lg n$  است و  $n$  بار انجام عمل از مرتبه  $n \lg n$  می‌شود.

۱۰- گزینه (۳). با انجام عمل تغییر وزن  $w'(i, j) = w(i, j) + f(i) - f(j)$  یا  $w'(i, j) = w(i, j) - f(i) + f(j)$ ، کوتاهترین مسیرها عوض نمی‌شود. از این فرمول در الگوریتم جانسون استفاده می‌شود. پس موارد اول و دوم درست هستند.

۱۱- گزینه (۴). احتمال آنکه خروجی در نهایت  $a_i$  شود آن است که هنگام خواندن  $a_i$  مقدار  $x$  برابر  $a_i$  شود با احتمال  $\frac{1}{i}$  (قبل از  $a_i$  مهم نیست  $x$  چه عددی شده) و حال عناصر بعد از

$a_i$  که خوانده می‌شوند  $x$  عوض نشود و همان  $a_i$  بماند. احتمال آنکه  $a_{i+1}$  خوانده شده ولی  $x$  برابر  $a_{i+1}$  نشود برابر  $1 - \frac{1}{i+1}$  است و به همین ترتیب پس احتمال مورد نظر برابر است با:

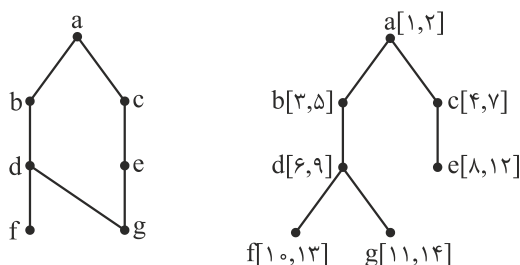
$$\frac{1}{i} \times \left(1 - \frac{1}{i+1}\right) \times \left(1 - \frac{1}{i+2}\right) \times \dots \times \left(1 - \frac{1}{n}\right) = \frac{1}{i} \times \frac{i}{i+1} \times \frac{i+1}{i+2} \times \dots \times \frac{n-1}{n} = \frac{1}{n}$$

۱۲- گزینه (۳). یکی از کاربردهای مسئله شار همین است. اگر ظرفیت‌ها یک باشند آنگاه عدد شار بیشینه برابر است با تعداد مسیرهای یال مجزا از  $s$  به  $t$

۱۳- گزینه (۲). گزاره اول غلط است. مسئله‌ای که در  $Np$  هست لزوماً مکمل آن در  $Np$  نیست. گزاره دوم غلط است زیرا مسائل ان‌پی کامل هستند که به ان‌پی سخت قابل کاهش هستند گزاره سوم درست است.

۱۴- گزینه (۱). هدف یافتن پایگاه رأس است (node base) که می‌توان با الگوریتم DFS و یافتن مولفه‌های متصل قوی و سپس تبدیل گراف به یک DAG و ترتیب توپولوژیکی به جواب رسید.

۱۵- گزینه (۲). فرض کنید راس شروع  $a$  در گراف مقابل در زمان ۱ وارد صف شود.



حال  $a$  در زمان ۲ از صف خارج شده و فرزندانش  $b$  و  $c$  در زمان‌های ۳ و ۴ وارد صف می‌شوند (می‌توان فرض کرد  $b$  و  $c$  هر دو در زمان ۳ وارد صف می‌شوند) حال  $b$  در زمان ۵ از صف خارج شده و  $d$  در زمان ۶ وارد صف می‌شود. حال  $c$  در زمان ۷ از صف خارج شده و  $e$  در زمان ۸ وارد می‌شود حال  $d$  در زمان ۹ خارج شده و  $f$  و  $g$  در زمان‌های ۱۰ و ۱۱ وارد می‌شوند. حال  $e$  و  $f$  و  $g$  در زمان‌های ۱۲ و ۱۳ و ۱۴ خارج می‌شوند.

واضح است که طول بازه ارتباطی به فاصله تا رأس شروع ندارد (گزینه ۱ غلط). گزینه ۲ حتماً درست است چون نواذگان یک نود وقتی وارد صف می‌شوند که جدّ آنها از صف خارج شده پس بازه آنها اشتراک ندارد. گزینه ۳ غلط است در همین مثال بازه  $b$  و  $e$  اشتراک ندارد

ولی نواده هم نیستند. شروع بازه ربطی به فاصله تا راس شروع ندارد مثلاً  $d$  و  $e$  شروع‌های متفاوت دارند ولی فاصله آنها تا  $a$  یکسان است پس گزینه ۴ غلط است.

۱۶- گزینه (۳). اولین مثال نقض در درخت ۵ رأسی  $a-b-c-d-e$  است که جواب

بهینه  $\{b, d\}$  است. ولی این الگوریتم ممکن است ابتدا  $c$  را انتخاب کند که در این صورت به اشتباه یک مجموعه ۳ عضوی را به عنوان جواب اعلام خواهد کرد.

۱۷- گزینه (۱). هر سه جمله گفته شده روش شبه ضرب ماتریس برای یافتن کوتاهترین مسیر بین هر دو رأس هستند.

۱۸- گزینه (۴). در چنین گرافی می‌توان با مرتبه  $O(n+m)$  که چون  $n \leq m$  (همبند) پس با مرتبه  $O(m)$  به جواب رسید.

۱۹- گزینه (۱). حداقل تعداد مراحل وقتی است که هر دو بازیکن عدد دریافتی را به توان ۳ برسانند که در این صورت تعداد مراحل  $\log_3 \log_3 n$  است و حداکثر تعداد مراحل وقتی است که هر دو بازیکن به توان ۲ برسانند که در این صورت تعداد مراحل  $\log_2 \log_2 n$  است پس تعداد مراحل همیشه  $\theta(\lg \lg n)$  است.

۲۰- گزینه (۲). بهتر است سعی و خطا کنیم چون الگوریتم اصلی که پویاست وقت‌گیر است.

$$(5 \times 10 \quad 10 \times 3) \times (3 \times 12 \quad 12 \times 5)$$

بهترین حالت ضرب به شکل فوق است که تعداد عمل ضرب عددی آن

$$3 \times 12 \times 5 + 5 \times 10 \times 3 + 5 \times 3 \times 5 = 405$$

می‌باشد.

## [سؤالهای چهارگزینه‌ای ساختمان داده و الگوریتم دکتری ۱۳۹۸]

۱- یک ماتریس دوبعدی  $n \times n$  از اعداد داده شده، که اعداد هر سطر و هر ستون آن مرتب شده است. به ازای عدد داده شده  $x$ ، جست‌وجوی  $x$  در این ماتریس در چه زمانی امکان‌پذیر است؟

- (۱)  $O(n)$       (۲)  $O(\log n)$       (۳)  $O(\log^2 n)$       (۴)  $O(n \log n)$

۲- می‌خواهیم بزرگترین زیر دنباله مشترک دو دنباله  $a_1, \dots, a_n$  و  $b_1, \dots, b_m$  را محاسبه کنیم. فرض کنید  $L(i, j)$  برابر طول بزرگترین زیر دنباله مشترک  $a_1, \dots, a_i$  و  $b_1, \dots, b_j$  باشد. کدام یک از تعاریف بازگشتی زیر درست است؟

- الف)  $L(n, m) = \max(L(n-1, m), L(n, m-1), L(n-1, m-1) + 1 \text{ if } a_n = b_m)$   
 ب)  $L(n, m) = \max(L(n-1, m), L(n-1, k-1) + 1)$  که  $k$  برابر بزرگترین عددی است که  $a_n = b_k$  (در صورت عدم وجود  $k = 0$  خواهد بود)  
 فرض کنید  $L(i, 0) = L(0, i) = 0$  و  $L(i, -1) = -1$  برای هر  $i \geq 0$ .  
 (۱) فقط الف      (۲) فقط ب      (۳) الف و ب      (۴) هیچ یک از الف و ب

۳- فرض کنید یک آرایه دوبعدی  $m \times n$  در اختیار داریم که هر ردیف آن مرتب شده است. فرض کنید همه اعداد متمایز هستند. می‌خواهیم  $k$  آمین عدد در آرایه را پیدا کنیم. در چه زمانی این کار امکان‌پذیر است؟

- (۱)  $O(m \cdot n)$       (۲)  $O(\log n \log m)$   
 (۳)  $O(\log n + \log m)$       (۴)  $O(m(\log n + \log m))$

۴- اگر ظرفیت همه یال‌ها در یک شبکه برابر  $C$  باشد، زمان اجرای الگوریتم فورد – فالکرسون برای محاسبه شار بیشینه از مبدا  $s$  به مقصد  $t$  در بدترین حالت کدام مورد خواهد بود؟  
 (فرض کنید تعداد رئوس و یال‌های گراف به ترتیب  $n$  و  $m$  هستند و درجه خروجی  $s$  برابر  $k$  باشد. همچنین فرض کنید در هر مرحله الگوریتم بیشترین شار ممکن را از مسیر انتخاب شده، عبور می‌دهد.)

- (۱)  $O(kC + m + n)$       (۲)  $O(kC(m + n))$   
 (۳)  $O(C(m + n))$       (۴)  $O(k(m + n))$

۵- فرض کنید ۱۳۹۷ نقطه متمایز روی محور اعداد حقیقی داده شده است. می‌خواهیم این ۱۳۹۷ نقطه را طوری رنگ‌آمیزی کنیم که به ازای هر بازه  $[a, b]$  روی محور اعداد حقیقی،

از بین نقاطی که در این بازه قرار گرفته‌اند حداقل یک نقطه وجود داشته باشد که رنگ آن با بقیه نقاط داخل بازه متفاوت باشد. حداقل چند رنگ برای این کار نیاز است؟

(۱) ۶ (۲) ۱۱ (۳) ۳۸ (۴) ۱۳۹۷

۶- فرض کنید یک B-tree داریم با  $n$  برگ که درجه هر گره حداقل  $\log n$  و حداکثر  $1 - \log n$  است. هزینه جست‌وجوی یک عدد در این درخت کدام است؟ (فرض کنید کلیدها داخل هر گره میانی در یک لیست پیوندی یک سویه ذخیره شده‌اند.)

(۱)  $O(\log n)$  (۲)  $O(\log n \log \log n)$   
 (۳)  $O(\log n \log^2 \log n)$  (۴)  $O(\log^2 n / \log \log n)$

۷- فرض کنید یک گراف وزن‌دار همبند داده شده است که وزن یال‌ها متمایز است. یک یال را امن گوییم اگر در هیچ دوری حضور نداشته باشد و یک یال را خطرناک گوییم اگر سنگین‌ترین یال در یک دور باشد. کدام یک از دو گزاره زیر درست است؟

الف) هر یال امن عضو درخت پوشای کمینه است.  
 ب) هر یال خطرناک عضو درخت پوشای کمینه نیست.

(۱) الف (۲) ب  
 (۳) الف و ب (۴) هیچ یک از الف و ب

۸- گراف جهت‌دار  $G$  با  $n$  رأس و  $m$  یال داده شده است. هر رأس  $i$  از گراف ارزشی به اندازه  $V_i$  دارد. به ازای هر رأس  $i$  از گراف، با ارزش‌ترین رأسی که از رأس  $i$  قابل دسترسی است  $W_i$  می‌نامیم. می‌خواهیم تمام  $W_i$ ها را به ازای  $i$  از ۱ تا  $n$  محاسبه کنیم. این کار در چه زمانی قابل انجام است؟ (بهترین گزینه را انتخاب کنید.)

(۱)  $O(m + n)$  (۲)  $O(m + n^2)$   
 (۳)  $O(n(m + n))$  (۴)  $O(m + n \log n)$

۹- یک درخت جست‌وجوی دودویی با  $n$  گره داریم که به علت نویز، اعداد ذخیره شده در برخی از گره‌های آن تغییر کرده است. تنها عملی که می‌توان برای اصلاح این درخت انجام داد جابجا کردن مقادیر ذخیره شده در یک گره و یکی از فرزندان آن است. کمینه تعداد اعمال مورد نیاز برای تبدیل درخت به یک درخت دودویی جست‌وجو در بدترین حالت کدام است؟ (دقت کنید که درخت اولیه لزوماً متوازن نیست.)

(۱)  $O(n)$  (۲)  $O(n^2)$  (۳)  $O(n \log n)$  (۴)  $O(n \log \log n)$

۱۰- زوج‌های مرتب زیر را در نظر بگیرید:

$$(1, A), (2, B), (5, C), (7, D), (8, E), (1, F), (4, G)$$

فرض کنید درختی داریم که براساس مؤلفه‌های اول این زوج‌ها یک هرم کمینه، و براساس مؤلفه‌های دوم یک درخت جست‌وجوی دودویی است. ارتفاع این درخت کدام است؟

$$(1) \quad 2 \quad (2) \quad 3 \quad (3) \quad 4 \quad (4) \quad 5$$

۱۱- در گراف همبند و بدون جهت  $G$  با  $n$  رأس، از یک رأس مشخص BFS و DFS را اجرا می‌کنیم، ترتیب ملاقات رئوس در هر دو اجرا یکسان شده است. در این خصوص کدام مورد درست است؟

(۱) گراف  $G$  فقط ستاره‌ای است.

(۲) گراف  $G$  فقط یک مسیر است.

(۳) تعداد یال‌های  $G$  از  $O(n)$  است.

(۴) تعداد یال‌های  $G$  می‌تواند  $\Omega(n \log n)$  باشد.

۱۲- فرض کنید گراف  $G$  همبند، بدون جهت و وزن دار است به طوری که می‌تواند دور منفی هم داشته باشد. در مورد مسئله پیدا کردن کوتاه‌ترین مسیر از یک رأس به رأس دیگر طوری که از هر رأسی حداکثر یک بار عبور کند، چه می‌توان گفت؟

(۱) یک مسئله آن‌پی - تمام است.

(۲) یک مسئله آن‌پی - سخت است.

(۳) به علت وجود دور منفی در گراف لزوماً چنین مسیری وجود ندارد.

(۴) در زمان چندجمله‌ای برحسب اندازه ورودی می‌توان مسئله را حل کرد.

۱۳- تعدادی فایل با اندازه‌های مشخص را می‌خواهیم روی نوار ذخیره کنیم. فرض کنید

$f_1, \dots, f_n$  به ترتیب (از راست به چپ) روی نوار ذخیره شده باشند، هزینه خواندن فایل

$f_i$  برابر  $\sum_{k=1}^i |f_k|$  خواهد بود که  $|f_k|$  برابر طول فایل  $f_k$  می‌باشد. فرض کنید قرار

است هر فایل تنها یک بار خوانده شود. می‌خواهیم مجموع هزینه را کمینه کنیم. بدین

منظور از الگوریتم حریصانه زیر استفاده می‌کنیم. فایل‌ها را به ترتیب اندازه از کوچک به

بزرگ روی نوار ذخیره می‌کنیم. اگر  $n$  تعداد فایل‌ها باشد، کمترین  $n$  که به ازای آن

الگوریتم فوق لزوماً درست کار نمی‌کند، کدام است؟

$$(1) \quad 2 \quad (2) \quad 3$$

(۳) ۴ (۴) به ازای هر  $n$ ، الگوریتم فوق بهینه عمل می‌کند.

۱۴- آرایه A شامل n عنصر داده شده است. می‌دانیم که تمام عناصر به جز  $\sqrt[4]{n}$  عنصر، در محل مرتب شده خود هستند ولی مکان عناصر نامرتب را نمی‌دانیم. این آرایه را در چه زمانی می‌توان مرتب کرد؟

- (۱)  $O(n)$  (۲)  $O(n\sqrt[4]{n})$  (۳)  $O(n \log n)$  (۴)  $O(\sqrt{n} \log n)$

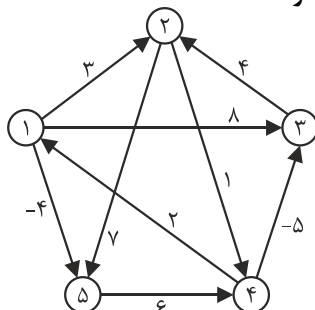
۱۵- پیمایش‌های پیش‌ترتیب و پس‌ترتیب یک درخت دودویی به صورت زیر است:  
 preorder : abcdefg , postorder : cbfgeda  
 با فرض ذخیره‌سازی درخت در آرایه (ریشه در خانه‌ی ۱ و فرزندان گره اندیس i در اندیس‌های  $2i$  و  $2i+1$ )، حداکثر تعداد خانه‌های بلااستفاده قبل از محل آخرین گره در آرایه کدام است؟

- (۱) ۵ (۲) ۶ (۳) ۷ (۴) ۸

۱۶- یک ساختمان داده را در نظر بگیرید که از دو پشته  $S_1$  و  $S_2$  تشکیل شده است. این ساختمان داده دو عمل درج و استخراج را پشتیبانی می‌کند. به هنگام درج عنصر x در این ساختمان داده،  $push(S_1, x)$  را اجرا می‌کنیم، به هنگام استخراج اگر  $S_2$  خالی نبود،  $Pop(S_2)$  را اجرا می‌کنیم، در غیر این صورت همه عناصر داخل  $S_1$  را پاپ و داخل  $S_2$  پوش می‌کنیم و بعد دستور  $Pop(S_2)$  را اجرا و به عنوان خروجی دستور استخراج در نظر می‌گیریم. اگر دو پشته در ابتدا خالی باشد و n عمل درج و استخراج به ترتیب دلخواه انجام شود، هزینه سرشکن این عمل‌ها کدام است و ساختمان داده فوق چه ساختمان داده‌ای را پیاده‌سازی می‌کند؟

- (۱)  $O(1)$  و صف (۲)  $O(n)$  و صف (۳)  $O(1)$  و پشته (۴)  $O(n)$  و پشته

۱۷- اگر الگوریتم جانسون برای یافتن کوتاه‌ترین مسیر بین تمام رأس‌های گراف را روی گراف وزن‌دار زیر اجرا کنیم، پس از اجرای مرحله تغییر وزن یال‌ها در الگوریتم، وزن جدید یال بین رأس‌های ۱ و ۵ که وزن اولیه آن ۴- است، کدام مقدار خواهد شد؟



- (۱) ۴-  
 (۲) ۴  
 (۳) ۳  
 (۴) ۰

۱۸- اگر رشته  $ababbcbabdadad$  را به وسیله الگوریتم هافمن کدگذاری کنیم، طول رشته حاصل چند بیت خواهد بود؟

(۱) ۲۰ (۲) ۲۴ (۳) ۲۸ (۴) ۳۰

۱۹- فرض کنید  $n$  عدد صحیح  $k$  بیتی داریم. فرض کنید هزینه جمع، تفريق و مقایسه دو عدد  $k$  بیتی  $O(k)$  است. اگر  $k = O(\log n)$  باشد، کدام گزینه در مورد الگوریتم‌های مرتب‌سازی درست است؟

- (۱) زمان اجرای الگوریتم مرتب‌سازی شمارشی  $O(n)$  است.
- (۲) زمان اجرای الگوریتم مرتب‌سازی سریع  $O(n \log n)$  است.
- (۳) زمان اجرای الگوریتم مرتب‌سازی ادغامی  $O(n \log^2 n)$  است.
- (۴) زمان اجرای الگوریتم مرتب‌سازی درجی  $O(n^2 \log^2 n)$  است.

۲۰- آرایه‌ای شامل  $n$  عدد داریم. اگر در اجرای الگوریتم مرتب‌سازی ادغامی روی این آرایه هرگاه تعداد اعداد کمتر از  $\sqrt{n}$  شد، روال بازگشتی را متوقف و از الگوریتم مرتب‌سازی درجی استفاده کنیم، زمان اجرای الگوریتم کدام مورد خواهد بود؟ (فرض کنید زمان اجرای الگوریتم مرتب‌سازی درجی از مرتبه  $O(m^2)$  است که  $m$  تعداد اعداد می‌باشد.)

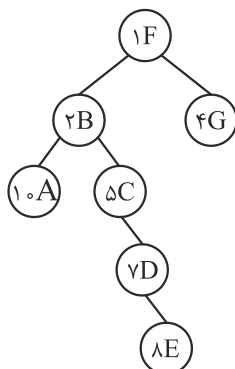
(۱)  $O(n^2)$  (۲)  $O(n\sqrt{n})$  (۳)  $O(n \log n)$  (۴)  $O(n \log n \sqrt{n})$



## پاسخنامه سؤال‌های چهارگزینه‌ای ساختمان داده و الگوریتم دکتری ۱۳۹۸

- ۱- گزینه (۱). ماتریس یانگ است که حداکثر  $2n - 1$  مقایسه نیاز است به این صورت که  $x$  را ابتدا با عنصر گوشه پایین چپ مقایسه کنیم. اگر  $x$  بزرگتر بود، به راست حرکت کنیم و اگر کوچکتر بود به بالا حرکت کنیم.
- ۲- گزینه (۳). مسئله یافتن  $|LCS|$  است که «الف» در درس گفته شده «ب» نیز یک فرمول درست است.
- ۳- گزینه (۴). گزینه‌های ۲ و ۳ غلط هستند چون در هر ردیف حتی اگر یک عنصر باشد یعنی  $n = 1$  باید  $m$  تا عدد را بررسی کنیم و حداقل  $\theta(m)$  زمان نیاز است. گزینه ۱ نیز روش brute force است که در بین  $mn$  عدد می‌توان با مرتبه  $O(mn)$  عنصر  $k$  ام را یافت ولی روش‌های سریعتری وجود دارد مثل استفاده از هرم یا روش‌هایی که به گزینه ۴ منجر می‌شود.
- ۴- گزینه (۴). شار بیشینه حداکثر  $kc$  است و در هر مرحله  $c$  واحد به شار نزدیک می‌شویم پس  $k$  مرحله مسیریابی با مرتبه  $m + n$  باید انجام شود.
- ۵- گزینه (۲). نقطه وسط را رنگ ۱ بدهید و دیگر از رنگ ۱ استفاده نکنید حال چپ و راست نقطه وسط را مشابه یکدیگر و با همین الگوریتم رنگ‌آمیزی کنید. تعداد رنگ‌ها  $\lfloor \lg n \rfloor + 1$  می‌شود که در این سوال  $\lfloor \lg 1397 \rfloor + 1 = 11$  می‌شود.
- ۶- گزینه (۴). در هر نود باید جستجوی خطی شود چون نودها لیست پیوندی هستند پس مرتبه جستجو  $O(t \cdot h)$  است که  $t = \log n$  و  $h = \log_t n$  است پس زمان جستجو برابر است با:
 
$$O(\log n \times \log_t n) = O\left(\log n \times \frac{\log n}{\log t}\right) = O\left(\log n \times \frac{\log n}{\log \log n}\right) = O\left(\frac{\log^2 n}{\log \log n}\right)$$
- ۷- گزینه (۳). سبکترین یال در هر دور حتماً در MST هست و سنگین‌ترین یال در هر دور حتماً در MST نیست (وزن‌ها متمایز است)
- ۸- گزینه (۱). با پیمایش‌های معمول می‌توان به جواب رسید.
- ۹- گزینه (۲). در بدترین حالت هر کلید باید ارتفاع درخت را طی کند تا در جای درست قرار گیرد و چون درخت متوازن نیست برای هر کلید  $O(n)$  زمان لازم است پس برای  $n$  کلید زمان  $O(n^2)$  است.

۱۰- گزینه (۳). درخت treap و به شکل مقابل است.



۱۱- گزینه (۴). اینکه ترتیب ملاقات یکی شده در خیلی از گراف‌ها ممکنه رخ بده (توجه کن اگر گفته شود درخت حاصل از دو پیمایش یکی شده فقط و فقط در صورتی امکان داره که گراف اصلی درخت باشد). در گراف کامل ترتیب رئوس DFS با BFS می‌تواند یکی شود و این گراف  $\theta(n^2)$  یال دارد که  $\Omega(n \lg n)$  است. کلید اولیه گزینه ۳ بود ولی اصلاح شد.

۱۲- گزینه (۲). یافتن کوتاهترین مسیر در گراف غیرجهت‌دار که یال منفی دارد یک مسئله ان‌پی سخت است. (از هر رأس حداکثر یک بار باید عبور شود)

۱۳- گزینه (۴). برای خواندن فایل  $f_i$  باید فایل‌های  $f_1$  تا  $f_i$  همگی خوانده شوند پس هزینه خواندن  $n$  فایل برابر است با:

$$|f_1| + (|f_1| + |f_2|) + (|f_1| + |f_2| + |f_3|) + \dots + (|f_1| + \dots + |f_n|)$$

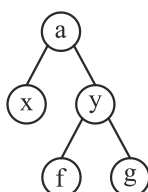
و این وقتی کمینه است که  $|f_1| \leq |f_2| \leq \dots \leq |f_n|$

۱۴- گزینه (۱). با مرتبه  $\theta(n)$  می‌توان مکان عناصر مرتب و نامرتب را فهمید حال  $n - \sqrt[n]{n}$  عنصر مرتب و  $\sqrt[n]{n}$  عنصر نامرتب داریم که با مرتبه  $O(\sqrt[n]{n} \lg \sqrt[n]{n})$  عناصر نامرتب را مرتب می‌کنیم و دو لیست مرتب را ادغام می‌کنیم که  $O(n)$  زمان می‌خواهد و زمان کل نیز  $O(n)$  می‌شود (جمع همه زمان‌ها)

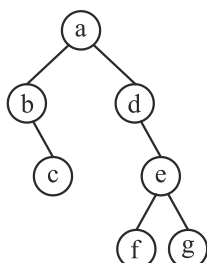
۱۵- گزینه (۴).  $b$  و  $d$  تک فرزندی هستند و فرزند این دو به ترتیب  $c$  و  $e$  هستند.  $bc$  را  $x$  و  $de$  را  $y$  می‌نامیم:

pre: a x y f g

post: x f g y a



حال درخت را می‌کشیم (با روش‌هایی که بلدم)  
برای آنکه بیشترین خانه بلااستفاده رخ دهد باید تک فرزندها  
راست پدر قرار گیرند.



این درخت ۴ سطح دارد و اگر کامل شود ۱۵ گره خواهد داشت که ۸ تایی آن بلااستفاده است.

۱۶- گزینه (۱). ساخت صف با دو پشته را نشان می‌دهد که هزینه استهلاکی عملیات  $O(1)$

است. در واقع اگر به ازای هر عمل درج مبلغ ۳ تومان و به ازای هر عمل استخراج مبلغ ۱ تومان هزینه شود. همیشه می‌توان هر ترتیبی از درج و استخراج را انجام داد.

۱۷- گزینه (۴). رأس  $s$  را اضافه کرده و یال‌هایی با وزن صفر از  $s$  به رئوس می‌کشیم و سپس کوتاهترین مسیر از  $s$  به رأس  $i$  یعنی  $\delta(s, i)$  را یافته و  $f(i)$  می‌نامیم:

$$f(1) = \delta(s, 1) = 0, \quad f(5) = \delta(s, 5) = -4$$

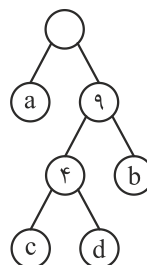
حال طبق فرمول تغییر وزن:

$$w'(1, 5) = w(1, 5) + f(1) - f(5) = -4 + 0 - (-4) = 0.$$

۱۸- گزینه (۳).

| کاراکتر | a | b | c | d |
|---------|---|---|---|---|
| فراوانی | ۶ | ۵ | ۱ | ۳ |

$$6 \times 1 + 5 \times 2 + 1 \times 3 + 3 \times 3 = 28$$



۱۹- گزینه (۳). مرتب‌سازی ادغامی  $n \lg n$  تا عمل مقایسه بین اعداد دارد که هر مقایسه  $\lg n$

زمان لازم دارد (طبق سوال) پس زمان کل  $n \lg^2 n$  می‌شود.

۲۰- گزینه (۲). مرتب‌سازی ادغامی در صورتی که پس از شکستن، لیست‌ها  $k$  عنصری شوند و

مرتب‌سازی درجی و سپس ادغام انجام دهد از مرتبه  $nk + n \lg \frac{n}{k}$  است که در این سؤال

$k = \sqrt{n}$  پس زمان کل  $n\sqrt{n} + n \lg \sqrt{n}$  یعنی  $n\sqrt{n}$  است.

## سؤال‌های چهارگزینه‌ای ساختمان داده و الگوریتم دکتری ۱۳۹۹

- ۱- فرض کنید متنی به طول  $n$  در اختیار داریم. در خصوص گزاره‌های زیر کدام گزینه صحیح است؟
- الف) کد هافمن یک کاراکتر یک بیتی است، اگر و فقط اگر تعداد تکرار آن کاراکتر کمتر از جمع تعداد تکرار بقیه کاراکترها نباشد.
- ب) اگر کاراکتری بیشترین تکرار را داشته باشد و تعداد تکرارهای آن بیش از  $\frac{n}{3}$  باشد، آنگاه کد هافمن آن کاراکتر تک بیتی است.
- (۱) الف) درست و (ب) درست (۲) الف) نادرست و (ب) درست  
(۳) الف) درست و (ب) نادرست (۴) الف) نادرست و (ب) نادرست
- ۲- یک گراف کامل  $10$  رأسی را در نظر بگیرید، که رأس‌های آن از  $1$  تا  $10$  شماره‌گذاری شده‌اند. فرض کنید وزن یال بین  $i$  و  $j$  برابر  $i + j$  است. آخرین یال درخت پوشای کمینه که توسط الگوریتم پریم با شروع از رأس  $10$  اضافه می‌شود، چه وزنی دارد؟
- (۱) ۹ (۲) ۱۰ (۳) ۱۱ (۴) ۱۷
- ۳- کدام الگوریتم مرتب‌سازی در بهترین حالت، زمان اجرای کمتری دارد؟
- (۱) Insertion Sort (۲) Selection Sort (۳) Merge Sort (۴) Quick Sort
- ۴- برای پیاده‌سازی یک لیست پیوندی حلقوی، کدام ساختمان داده قابل استفاده است؟
- (۱) پشته (۲) صف  
(۳) صف و پشته (۴) هیچ یک از صف و پشته
- ۵- در پیاده‌سازی متعارف جستجوی عمق اول و جستجوی سطح اول، به ترتیب از کدام داده ساختارها استفاده می‌شود؟
- (۱) پشته و صف (۲) صف و پشته (۳) پشته و لیست (۴) لیست و پشته
- ۶- مسئله جمع زیرمجموعه بدین شکل تعریف می‌شود: یک مجموعه از اعداد مثبت  $S = \{a_1 \dots a_n\}$  به همراه عدد  $W$  داده شده است. آیا زیرمجموعه‌ای از  $S$  پیدا می‌شود که جمع اعضای آن  $W$  شود؟
- برای حل این مسئله به روش برنامه‌ریزی پویا یک آرایه دو بُعدی  $X[1..n, 0..W]$  تعریف می‌کنیم که  $X[i, j]$  برابر True است. اگر زیرمجموعه‌ای از  $S = \{a_1 \dots a_i\}$  وجود داشته باشد که جمع اعضای آن  $j$  شود، در این خصوص کدام رابطه درست است؟

$$X[i, j] = X[i-1, j] \vee X[i, j-a_i] \quad (۱)$$

$$X[i, j] = X[i-1, j] \wedge X[i, j-a_i] \quad (۲)$$

$$X[i, j] = X[i-1, j] \vee X[i-1, j-a_i] \quad (۳)$$

$$X[i, j] = X[i-1, j] \wedge X[i-1, j-a_i] \quad (۴)$$

۷- برای گراف بدون جهت  $G$  با  $n$  رأس دو مسئله زیر را در نظر بگیرید:

- مسئله A: آیا  $G$  یک زیرمجموعه مستقل ۴ رأسی دارد؟
- مسئله B: آیا  $G$  یک زیرمجموعه مستقل  $n-4$  رأسی دارد؟

در خصوص این دو مسئله کدام مورد درست است؟

- (۱) مسئله A عضو کلاس P و مسئله B عضو کلاس NP-Complete است.
- (۲) مسئله A عضو کلاس NP-Complete و مسئله B عضو کلاس P است.
- (۳) هر دو مسئله عضو کلاس NP-Complete هستند.
- (۴) هر دو مسئله عضو کلاس P هستند.

۸- فرض کنید  $G = (V, E)$  یک گراف بدون جهت و گراف  $G' = (V', E')$  یک زیرگراف  $G$

است. یال‌های  $G$  را بدین شکل وزن‌دار می‌کنیم: اگر  $c \in E'$  باشد، وزن آن را صفر و در غیر این صورت ۱ می‌گذاریم. از رأس دلخواه  $v \in V'$  الگوریتم دایکسترا را برای محاسبه کوتاهترین مسیر به بقیه رئوس اجرا می‌کنیم. کدام مسئله را می‌توان با استفاده از طول کوتاهترین مسیرهای محاسبه شده، حل کرد؟

- (۱) آیا  $G'$  درخت است؟
- (۲) آیا  $G'$  همبند است؟
- (۳) آیا  $G'$  تشکیل خوشه می‌دهد؟
- (۴) تعداد یال‌ها در کوتاهترین مسیر از  $v$  به بقیه رئوس چند است؟

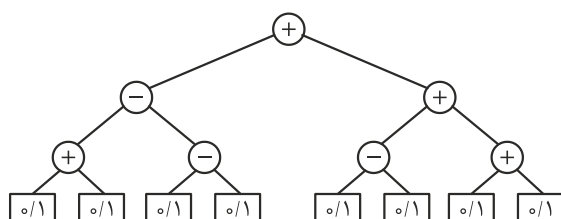
۹- فرض کنید در داخل یک درخت دودویی جستجو، اعداد ۱ تا ۱۰۰۰ ذخیره شده‌اند و ما می‌خواهیم دنبال عدد ۳۶۵ بگردیم. کدام دنباله (از چپ به راست) نمی‌تواند مسیر جستجو باشد؟

- (۱) ۳۶۵ و ۲۸۰ و ۳۸۳ و ۳۸۴ و ۲۶۸ و ۲۲۱ و ۳۸۹ و ۴۰۱ و ۴
- (۲) ۳۶۵ و ۳۶۴ و ۲۶۰ و ۹۰۰ و ۲۴۶ و ۹۱۳ و ۲۲۲ و ۹۲۶
- (۳) ۳۶۵ و ۳۹۹ و ۳۴۶ و ۳۳۲ و ۴۰۰ و ۴۰۳ و ۲۵۴ و ۴
- (۴) ۳۶۵ و ۲۴۷ و ۹۱۴ و ۲۴۲ و ۹۱۳ و ۲۰۴ و ۹۲۷

۱۰- فرض کنید یک آرایه مرتب از  $n$  عدد در اختیار داریم. به ازای یک  $k$  داده شده، می‌خواهیم دو عدد  $a$  و  $b$  از آرایه را پیدا کنیم که  $|a - b| = k$  شود. سریع‌ترین الگوریتم برای حل این مسئله دارای چه مرتبه زمانی است؟

- (۱)  $O(n)$  (۲)  $O(n^2)$  (۳)  $O(\log n)$  (۴)  $O(n \log n)$

۱۱- در درخت میانوندی داده شده، مقدار هر برگ می‌تواند صفر یا یک باشد. ماکزیمم خروجی این عبارت کدام است؟



- (۱) ۲  
(۲) ۳  
(۳) ۴  
(۴) ۶

۱۲- می‌دانیم ترتیب شروع و پایان فعالیت‌های  $A$  و  $B$  و  $C$  و  $D$  و  $E$  و  $F$  و  $G$  و  $H$  به صورت  $a_s, b_s, c_s, a_e, d_s, c_e, e_s, f_s, b_e, d_e, g_s, e_e, f_e, h_s, g_e, h_e$  است. در اینجا  $x_s$  زمان شروع و  $x_e$  زمان پایان فعالیت  $X$  می‌باشد. می‌خواهیم این فعالیت‌ها را در تعدادی اتاق که در اختیار داریم انجام دهیم. یک فعالیت در یک اتاق قابل انجام است، اگر در تمام مدت زمان آن فعالیت اتاق به طور کامل در اختیارش باشد. حداقل تعداد اتاق‌های مورد نیاز برای انجام همه فعالیت‌ها کدام است؟

- (۱) ۳ (۲) ۴ (۳) ۵ (۴) ۶

۱۳- اعداد صحیح بین ۱ تا ۱۳۹۸ به عنوان ورودی داده شده است. کدام تابع درهم‌ساز، اعداد داده شده را به طور یکنواخت بین ۱۰ خانه جدول درهم‌سازی توزیع می‌کند؟ (یک توزیع یکنواخت است، اگر تفاضل تعداد اعداد نگاشت شده به هر دو خانه از جدول حداکثر ۱ باشد.)

$$h(i) = i^3 \bmod 10 \quad (۲) \quad h(i) = i^2 \bmod 10 \quad (۱)$$

$$h(i) = 4i^2 + 6 \bmod 10 \quad (۴) \quad h(i) = 12i \bmod 10 \quad (۳)$$

۱۴- آرایه  $A[1..13] = ۸۹, ۱۹, ۴۰, ۱۷, ۱۲, ۱۰, ۲, ۵, ۷, ۱۱, ۶, ۹, ۷۰$  داده شده است. می‌توانیم هر بار دو خانه دلخواه از این آرایه را با هم جابجا کنیم. با حداقل چند جابجایی می‌توان این آرایه را به یک هرم بیشینه تبدیل کرد؟

- (۱) ۰ (۲) ۱ (۳) ۲ (۴) ۳

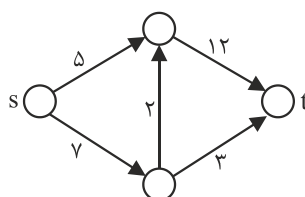
۱۵- فرض کنید در گراف وزن دار و جهت‌دار  $G$  با  $n$  رأس، تنها وزن یال‌های خارج شده از رأس  $s$  ممکن است منفی باشند. (البته می‌دانیم گراف دور منفی ندارد.) بزرگترین  $n$  که به ازای آن

الگوریتم دایکسترا روی هر گراف  $n$  رأسی با فرض‌های گفته شده کوتاه‌ترین مسیر از  $s$  به بقیه رئوس را درست محاسبه می‌کند، کدام است؟

(۱) ۲ (۲) ۳

(۳) ۴ (۴) به ازای هر  $n$  همیشه درست کار می‌کند.

۱۶- در شبکه داده شده فقط مجاز هستیم ظرفیت یک یال را به هر میزان که بخواهیم افزایش دهیم. با این کار شار بیشینه از  $s$  به  $t$  را حداکثر چه میزان می‌توان افزایش داد؟



(۱)  $\infty$

(۲) ۵

(۳) ۲

(۴) ۱

۱۷- اگر مسئله  $X$  عضو کلاس NP-Complete به مسئله  $Y$  عضو کلاس  $P$  در زمان چندجمله‌ای تبدیل شود، کدام گزینه نادرست است؟

(۱)  $NP = P$

(۲)  $NP - Complete = P$

(۳)  $NP - Hard = NP$

(۴) مسئله 3-SAT در زمان چندجمله‌ای حل می‌شود.

۱۸- زمان اجرای الگوریتمی به صورت  $T(n) = T(n/11) + T(6n/11) + n$  است. مرتبه زمانی اجرای این الگوریتم کدام است؟

(۱)  $O(n)$  (۲)  $O(n^2)$  (۳)  $O(\log n)$  (۴)  $O(n \log n)$

۱۹- فرض کنید  $G$  یک گراف وزن دار و جهت دار با  $n$  رأس و  $m$  یال است. با فرض اینکه فاصله کوتاه‌ترین مسیر برای هر دو رأس را در یک ماتریس  $n \times n$  در اختیار داریم، مطلع شده‌ایم که وزن تنها یک یال  $(u, v)$  تغییر پیدا کرده است. می‌خواهیم به ازای دو رأس مشخص  $s$  و  $t$  طول کوتاه‌ترین مسیر بین این دو رأس را به روز کنیم. این کار را در چه زمانی می‌توان انجام داد؟

(۱)  $O(n)$  (۲)  $O(n + m)$  (۳)  $O(1)$  (۴)  $O(n \log n + m)$

۲۰- دو هرم کمینه در اختیار داریم که هر یک شامل  $n$  عدد است. می‌خواهیم یک هرم کمینه برای همه این  $2n$  عدد بسازیم. با چه مرتبه زمانی می‌توان این کار را انجام داد؟ (فرض کنید هرم‌های کمینه با آرایه پیاده‌سازی شده‌اند.)

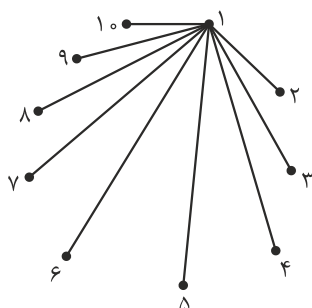
(۱)  $O(n)$  (۲)  $O(n \log n)$  (۳)  $O(n \log^* n)$  (۴)  $O(n \log \log n)$

## پاسخنامه سؤال‌های چهارگزینه‌ای ساختمان داده و الگوریتم دکتری ۱۳۹۹

۱- گزینه (۴). الف غلط است مثلاً با فراوانی‌های ۱ و ۲ و ۴ و ۸ و ۱۴ فراوانی ۱۴ یک بیتی می‌شود ولی ۱۴ از جمع  $۱+۲+۴+۸$  کمتر است.

ب غلط است مثلاً فراوانی‌های ۵ و ۷ و ۷ و ۱۱ را در نظر بگیرید ۱۱ از  $\frac{۱۱+۷+۷+۵}{۳}$  بیشتر است ولی بررسی کنید که یک بیتی نمی‌شود.

۲- گزینه (۲). ترتیب یال‌ها از چپ به راست:



$(۱,۱), (۱,۲), (۱,۳), (۱,۴), (۱,۵), (۱,۶), (۱,۷), (۱,۸), (۱,۹)$

آخرین یال  $(۱,۹)$  و وزن آن ۱۰ می‌باشد.

۳- گزینه (۱). درجی در بهترین  $n$  است. انتخابی  $n^2$  است. ادغامی و سریع در بهترین حالت  $n \lg n$  هستند.

۴- گزینه (۴). این تست اساساً مشکل دارد و در کلید نهایی به طرز عجیبی گزینه‌های ۲ و ۳ اعلام شد.

۵- گزینه (۲). جستجوی عمقی پشته می‌خواهد و سطحی صف

۶- گزینه (۳). عضو  $a_i$  دو حالت دارد انتخاب نشود یا شود. اگر  $a_i$  انتخاب نشود آنگاه  $X[i-1, j]$  جواب است یا  $a_i$  انتخاب شود که جواب  $X[i-1, j-a_i]$  است که عمل OR بین اینها جواب می‌شود.

۷- گزینه (۴). در مسئله A باید حداکثر  $\binom{n}{۴}$  حالت بررسی شود و در مسئله B نیز حداکثر

$\binom{n}{n-۴}$  حالت باید بررسی شوند که هر دو چندجمله‌ای هستند.

۸- گزینه (۲). اگر نتیجه دایجسترا فقط شامل وزن صفر باشد آنگاه  $G'$  همبند است ولی اگر در نتیجه دایجسترا یالی با وزن یک باشد آنگاه  $G'$  ناهمبند است.



- ۹- گزینه (۴). به ازای هر عدد  $x$  همه اعداد بعد از آن باید از  $x$  کوچکتر باشند یا همگی از  $x$  بزرگتر باشند که در گزینه ۴ بعد از عدد ۹۱۳ اعدادی که آمده‌اند مشکل دارند.
- ۱۰- گزینه (۱). دو متغیر  $i$  و  $j$  تعریف کن،  $i$  اول آرایه و  $j$  آخر آرایه حال  $A[j] - A[i]$  سه حالت دارد اگر با  $k$  مساوی بود که تمام اگر کمتر از  $k$  بود  $i++$  ولی اگر بیشتر از  $k$  بود  $j--$  شود که این عملیات  $O(n)$  است.
- ۱۱- گزینه (۴). اگر برگ‌ها را به ترتیب  $a$  و  $b$  ... بنامیم، خروجی این درخت برابر است با:  

$$((a+b)-(c-d)) + ((e-f)+(g+h))$$
که این عبارت وقتی بیش‌تر است که  $a=b=d=e=g=h=1$  و  $c=f=0$   

$$((1+1)-(0-1)) + ((1-0)+(1+1)) = 6$$
- ۱۲- گزینه (۲). شمارنده  $C=0$  را تعریف کن و از چپ زمان‌ها را دنبال کن به ازای هر شروع  $C++$  و به ازای هر پایان  $C--$  انجام بده. مقدار بیشینه  $C$  جواب است:
- |       |       |       |       |       |       |       |       |       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| $a_s$ | $b_s$ | $c_s$ | $a_e$ | $d_s$ | $c_e$ | $e_s$ | $f_s$ | $b_e$ | $d_e$ | $g_s$ | $e_e$ | $f_e$ | $h_s$ | $g_e$ | $h_e$ |
| ۱     | ۲     | ۳     | ۲     | ۳     | ۲     | ۳     | ۴     | ۳     | ۲     | ۳     | ۲     | ۱     | ۲     | ۱     | ۰     |
| $C$   | ۱     | ۲     | ۳     | ۲     | ۳     | ۲     | ۳     | ۴     | ۳     | ۲     | ۳     | ۲     | ۱     | ۲     | ۱     |
- ۱۳- گزینه (۲). تابع  $i \bmod 10$  و  $i^3 \bmod 10$  یکنوا هستند. مثلاً اگر کلیدهای ۱ تا ۱۰ را به تابع  $i^3 \bmod 10$  بدهید. این ۱۰ کلید به ۱۰ حفره مختلف می‌روند.
- ۱۴- گزینه (۳). دو جایجایی، ابتدا  $(70, 10)$  و سپس  $(70, 40)$ .
- ۱۵- گزینه (۴). با شرط سؤال دایجسترا همواره جواب درست می‌دهد.
- ۱۶- گزینه (۲). اگر ۵ را به  $\infty$  افزایش دهیم آنگاه شار بیشینه ۱۵ می‌شود که نسبت به ۱۰ که شار بیشینه اولیه است ۵ واحد افزایش می‌یابد.
- ۱۷- گزینه (۳). در این صورت  $P = NP = Npc$  ولی راجع به Nphard نمی‌توان نظر داد.
- ۱۸- گزینه (۱). چون  $\frac{4}{11} + \frac{6}{11} < 1$  پس جواب  $\theta(n)$  است.
- ۱۹- گزینه (۳). کافی است با فرمول  $\delta(s, t) = \min(\delta(s, t), \delta(s, u) + w'(u, v) + \delta(v, t))$  وزن جدید مسیر بین  $s$  و  $t$  را محاسبه کنیم.
- ۲۰- گزینه (۱). ادغام دو هرم  $m$  و  $n$  تایی که آرایه‌ای هستند نیاز به ساخت یک هرم  $m+n$  تایی دارد و از مرتبه  $O(m+n)$  است.